Learn Being Forward

COURSES · CAMPUS PARTNERSHIPS · PLACEMENTS

COURSE NOTES · BASIC TO ADVANCED

C# Programming

Complete notes for the Learn Being Forward C# Programming syllabus: every module, topic and subtopic explained step by step, with diagrams, verified examples and a practice checkpoint for each module.

24	3	71	464
modules	levels	minimum days	C# programs

How to use these notes

These notes follow the **Learn Being Forward C# Programming syllabus** exactly: the same 24 modules, in the same order, with every topic and subtopic explained step by step. Each subtopic number (for example **3.7.2**) matches the syllabus, so you can always find where you are.

Level	Modules	Minimum	Outcome
1. Beginner	1 to 9	17 days	Write C# console programs with variables, operators, control flow, arrays, strings and methods on .NET
2. Intermediate	10 to 17	25 days	Design classes with OOP, use structs, records, exceptions, generics, collections, delegates, events, LINQ and files
3. Advanced	18 to 24	29 days	Async and multithreading, advanced language features, CLR internals, EF Core, ASP.NET Core Web APIs, testing, design patterns and interview readiness
Total	1 to 24	71 days	About 12 weeks at 6 study days per week

Read
Read the concept for each subtopic and note the key points in your own words.

Type & run
Type out and run every example. Do not copy and paste code. Compare with the OUTPUT box.

Checkpoint
Finish the practice checkpoint at the end of each module before moving on.

Revise
At the end of each level, revise its modules and build one small project.

The examples target **.NET 10** (LTS) with **C# 14** (features added in recent versions are marked with their version where they appear) and use top-level statements unless a full class structure is the point of the example. The build environment used to typeset these notes had no .NET SDK, so the OUTPUT boxes were worked out by tracing each program by hand against the documented .NET behaviour and checked a second time; when you run the programs, values that depend on the machine or the moment (timings, hash codes, thread ids, dates, the current culture) will differ, and number formatting assumes an English (en-US/en-IN style) culture.

HOW TO READ THE BOXES

Dark boxes with a gold top line are **C# code**. Dashed cream boxes labelled **OUTPUT** show what the program prints. Gold-bordered boxes are notes and tips, red boxes warn about common mistakes, and navy boxes hold the key idea to remember.

Course plan at a glance

Module	Topic	Level	Min days
01	Introduction to C# and .NET	Beginner	1
02	Setup and your first program	Beginner	1
03	Variables and data types	Beginner	2
04	Operators and expressions	Beginner	1
05	Console input, output and formatting	Beginner	1
06	Control flow	Beginner	3

Module	Topic	Level	Min days
07	Arrays	Beginner	3
08	Strings	Beginner	2
09	Methods and recursion	Beginner	3
10	Object-oriented programming: classes and objects	Intermediate	4
11	Inheritance, polymorphism, abstraction and interfaces	Intermediate	4
12	Structs, enums, records and nullable reference types	Intermediate	2
13	Exception handling	Intermediate	2
14	Generics and collections	Intermediate	4
15	Delegates, events and lambda expressions	Intermediate	3
16	LINQ (Language Integrated Query)	Intermediate	4
17	File handling and serialization	Intermediate	2
18	Asynchronous programming and multithreading	Advanced	5
19	Advanced C# language features	Advanced	4
20	.NET runtime internals and memory management	Advanced	2
21	Data access with ADO.NET and Entity Framework Core	Advanced	4
22	Web development with ASP.NET Core	Advanced	6
23	Testing, debugging, design patterns and best practices	Advanced	3
24	Platforms, data structures, projects and interviews	Advanced	5

Table of contents

LEVEL 1 · BEGINNER LEVEL · MODULES 1 TO 9

Module 01 · Introduction to C# and .NET 8

1.1 What is C#? 8
1.2 History of C# 9
1.3 What is .NET? 11
1.4 How a C# program runs 12
1.5 Why do we need C#? 13
1.6 Where is C# used? 14
1.7 C# compared with other languages 15

Module 02 · Setup and your first program 18

2.1 Setting up the environment 18
2.2 The dotnet CLI 19
2.3 Project structure 21
2.4 Hello World, line by line 22
2.5 Basic syntax rules 24
2.6 Comments 26
2.7 Errors 27

Module 03 · Variables and data types 30

3.1 Variables 30
3.2 Types of variables 32
3.3 Value types vs reference types 34
3.4 Built-in value types 36
3.5 Built-in reference types 40
3.6 Literals 42
3.7 Constants 44
3.8 Type conversion 44
3.9 Nullable value types 49
3.10 Default values 50

Module 04 · Operators and expressions 53

4.1 Arithmetic operators 53
4.2 Relational and equality operators 55
4.3 Logical operators 56
4.4 Bitwise and shift operators 57
4.5 Assignment operators 58
4.6 Null operators 59
4.7 Other operators 61
4.8 Precedence and associativity 63
4.9 The Math class 65

Module 05 · Console input, output and formatting 68

5.1 Output 68
5.2 Input 72
5.3 Console features 74
5.4 Command-line arguments 75

Module 06 · Control flow 78

6.1 Decision-making statements 78
6.2 switch 80

6.3 Loops 84
6.4 Nested loops and patterns 89
6.5 Jump statements 92
6.6 Classic loop problems 94

Module 07 · Arrays 101

7.1 Single-dimensional arrays 101
7.2 Multi-dimensional arrays 104
7.3 Jagged arrays 106
7.4 Arrays as reference types 107
7.5 The Array class 108
7.6 Array algorithms 109

Module 08 · Strings 116

8.1 String basics 116
8.2 Comparing strings 120
8.3 String methods 122
8.4 StringBuilder 127
8.5 Characters 129
8.6 Regular expressions (introduction) 130
8.7 Classic string problems 132

Module 09 · Methods and recursion 138

9.1 Method basics 138
9.2 Parameters 142
9.3 Returning multiple values 149
9.4 Method overloading 151
9.5 Recursion 154

LEVEL 2 · INTERMEDIATE LEVEL · MODULES 10 TO 17

Module 10 · Object-oriented programming: classes and objects 162

10.1 OOP concepts 162
10.2 Classes and objects 164
10.3 Fields 166
10.4 Properties 167
10.5 Constructors 171
10.6 Access modifiers and encapsulation 176
10.7 Other class members 178
10.8 Partial classes and methods 184
10.9 Nested classes 185
10.10 Extension methods and extension members 186
10.11 System.Object members 188

Module 11 · Inheritance, polymorphism, abstraction and interfaces 193

11.1 Inheritance 193
11.2 Polymorphism 197
11.3 Abstraction 201
11.4 Interfaces 203
11.5 Relationships between classes 210

Module 12 · Structs, enums, records and nullable reference types 214

12.1 Structs 214
12.2 Enums 218
12.3 Records (C# 9) 220
12.4 Tuples 224

12.5 Nullable reference types (C# 8) 225
12.6 Anonymous types 228

Module 13 · Exception handling 231

13.1 Basics 231
13.2 Handling exceptions 234
13.3 Throwing exceptions 238
13.4 Custom exceptions 241
13.5 Resource cleanup 243
13.6 Best practices 245

Module 14 · Generics and collections 251

14.1 Generics 251
14.2 Non-generic collections (legacy) 258
14.3 Generic collections 259
14.4 Collection interfaces 265
14.5 Specialised collections 267
14.6 Sorting and comparing 270
14.7 Iterators 272
14.8 Collection expressions and spread `..` (C# 12) 274
14.9 Choosing the right collection and Big-O summary 275

Module 15 · Delegates, events and lambda expressions 278

15.1 Delegates 278
15.2 Built-in delegates 281
15.3 Anonymous methods and lambdas 283
15.4 Events 286
15.5 Expression trees (overview) 291

Module 16 · LINQ (Language Integrated Query) 294

16.1 Introduction 295
16.2 Filtering and projection 297
16.3 Sorting 300
16.4 Element operators 301
16.5 Aggregation 302
16.6 Quantifiers 304
16.7 Grouping and joining 304
16.8 Set operations 308
16.9 Partitioning and generation 308
16.10 Conversion 310
16.11 LINQ providers 311
16.12 Performance considerations and multiple enumeration 313

Module 17 · File handling and serialization 316

17.1 The System.IO namespace 316
17.2 Reading and writing text files 319
17.3 Streams 321
17.4 Async file I/O (overview, detailed in Module 18) 323
17.5 Serialization 324
17.6 Watching the file system 329

LEVEL 3 · ADVANCED LEVEL · MODULES 18 TO 24

Module 18 · Asynchronous programming and multithreading 333

18.1 Concepts 333
18.2 Threads 335

18.3 Synchronisation 336
18.4 Task Parallel Library (TPL) 341
18.5 async and await 345
18.6 Concurrent collections 352
18.7 Timers 354

Module 19 · Advanced C# language features 358

19.1 Pattern matching in depth 358
19.2 High-performance types 363
19.3 Reflection and attributes 369
19.4 Compile-time code generation 373
19.5 Other advanced features 375
19.6 Recent language versions 377

Module 20 · .NET runtime internals and memory management 382

20.1 The CLR 382
20.2 Memory 384
20.3 Garbage collection 386
20.4 Deterministic cleanup 389
20.5 Memory leaks in .NET 391
20.6 Performance tools 392

Module 21 · Data access with ADO.NET and Entity Framework Core 397

21.1 Database refresher 397
21.2 ADO.NET 399
21.3 Entity Framework Core 404
21.4 Architecture 413

Module 22 · Web development with ASP.NET Core 417

22.1 Web fundamentals 417
22.2 ASP.NET Core basics 419
22.3 Dependency injection 423
22.4 Building APIs 425
22.5 Security 433
22.6 Web UI options (overview) 437
22.7 Going further 438

Module 23 · Testing, debugging, design patterns and best practices 446

23.1 Debugging 446
23.2 Unit testing 448
23.3 Code quality 454
23.4 Design principles 455
23.5 Design patterns in C# 457
23.6 Tooling 462

Module 24 · Platforms, data structures, projects and interviews 466

24.1 Other C# platforms (overview) 466
24.2 Data structures and algorithms in C# 468
24.3 Capstone projects 481
24.4 Interview preparation 482
24.5 Career paths with C# 486

LEVEL 1

Beginner level

Outcome: write C# console programs with variables, operators, control flow, arrays, strings and methods on .NET.

9 MODULES · 17 DAYS MINIMUM

Module 01 · Introduction to C# and .NET	1 day
Module 02 · Setup and your first program	1 day
Module 03 · Variables and data types	2 days
Module 04 · Operators and expressions	1 day
Module 05 · Console input, output and formatting	1 day
Module 06 · Control flow	3 days
Module 07 · Arrays	3 days
Module 08 · Strings	2 days
Module 09 · Methods and recursion	3 days

MODULE 01 · BEGINNER

Introduction to C# and .NET

Minimum duration **1 day** | Level **Beginner**

Before writing any code, this module explains what C# is, where it came from, what the .NET platform provides, and how a C# program travels from source code to running machine code. It ends by comparing C# with the other languages a learner is likely to meet.

IN THIS MODULE

1.1 What is C#?
1.2 History of C#
1.3 What is .NET?
1.4 How a C# program runs
1.5 Why do we need C#?
1.6 Where is C# used?
1.7 C# compared with other languages

1.1 What is C#?

1.1.1 Definition: a modern, object-oriented, type-safe, general-purpose programming language

C# is a programming language created by Microsoft for building software on the **.NET** platform. Each word of the definition matters:

Term	Meaning for C#
Modern	designed in 2000 and updated every year; features such as async/await, LINQ, records and pattern matching are built into the language
Object-oriented	programs are organised into classes and objects that combine data with the code that works on it
Type-safe	every value has a type that the compiler checks; you cannot treat a string as a number by mistake, read past the end of an array, or use uninitialised memory in safe code
General-purpose	not limited to one kind of program: web servers, desktop apps, mobile apps, games, cloud services, tools and scripts

A FIRST TASTE OF C# — Program.cs

```
Console.WriteLine("Hello from C#!");
int year = 2002;
string language = "C#";
Console.WriteLine($"{language} was released in {year}.");
```

OUTPUT

```
Hello from C#!
C# was released in 2002.
```

1.1.2 Multi-paradigm: object-oriented, component-oriented and functional features

C# is mainly object-oriented, but it supports several styles of programming, and good C# code mixes them:

Object-oriented

Classes, inheritance, interfaces and polymorphism model the things in your program (Module 10 and 11).

Component-oriented

Properties, events and attributes are built into the language, so self-describing reusable components (controls, services) are natural.

Functional

Lambdas, LINQ, immutable records, pattern matching and expression-bodied members let you write code as transformations of data.

TWO STYLES IN ONE LANGUAGE

Program.cs

```
using System.Linq;

int[] marks = [72, 91, 58, 84, 66];

// Imperative style: say how, step by step
int countAbove70 = 0;
foreach (int m in marks)
    if (m > 70) countAbove70++;

// Functional style with LINQ: say what you want
int sameCount = marks.Count(m => m > 70);
double average = marks.Average();

Console.WriteLine($"{countAbove70} {sameCount} {average}");
```

OUTPUT

```
3 3 74.2
```

1.1.3 Pronunciation ("C sharp") and the C-family syntax

C# is pronounced "see sharp", like the musical note C♯ (a semitone higher than C). The name hints that it is a step up from C and C++, and the ♯ symbol even looks like four plus signs (++ stacked on ++). Because the musical sharp sign is not on keyboards, the language is written with the hash character #. C# belongs to the **C family**: statements end with semicolons, blocks use braces, and `if`, `for`, `while` and `switch` look the same as in C, C++, Java and JavaScript. Learners who know any of these read C# almost immediately.

1.2 History of C#

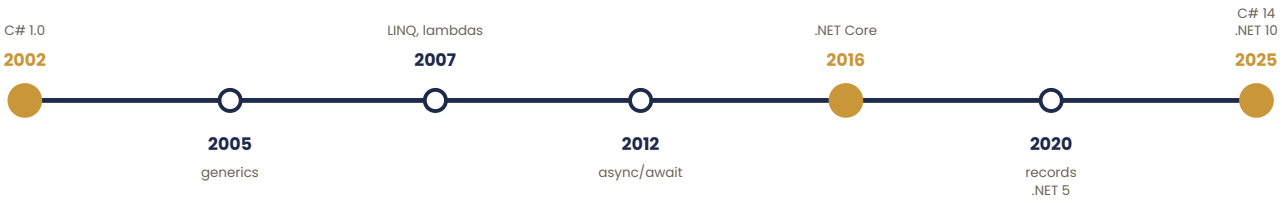
1.2.1 Created by Anders Hejlsberg at Microsoft, released in 2002 with .NET Framework 1.0

C# was designed by a team led by **Anders Hejlsberg**, who had earlier created Turbo Pascal and Delphi and later also designed TypeScript. Work began around 1999 under the code name "Cool" ("C-like Object Oriented Language"). C# 1.0 was released in January 2002 together with **.NET Framework 1.0** and Visual Studio .NET. It was standardised by ECMA (ECMA-334) and ISO, which allowed independent implementations such as Mono.

1.2.2 Version milestones

Version	Year	Headline features
C# 1.0	2002	classes, structs, interfaces, properties, events, delegates, attributes
C# 2.0	2005	**generics**, nullable value types, iterators (yield), anonymous methods, partial classes

Version	Year	Headline features
C# 3.0	2007	**LINQ**, **lambda expressions**, extension methods, var, anonymous types, object initializers
C# 4.0	2010	dynamic, optional and named parameters, generic variance
C# 5.0	2012	**async/await**, caller info attributes
C# 6.0	2015	string interpolation, null-conditional ?., nameof, expression-bodied members
C# 7.x	2017	**tuples**, **pattern matching**, out variables, local functions, ref returns, Span support
C# 8.0	2019	**nullable reference types**, switch expressions, ranges and indices, default interface methods, async streams
C# 9.0	2020	**records**, init-only setters, top-level statements, target-typed new
C# 10	2021	global usings, file-scoped namespaces, record structs
C# 11	2022	raw string literals, required members, list patterns, generic math
C# 12	2023	primary constructors, collection expressions, default lambda parameters
C# 13	2024	params collections, the new Lock type, partial properties
C# 14	2025	extension members, the field keyword, null-conditional assignment, implicit span conversions



KEY MOMENTS IN THE HISTORY OF C# AND .NET

1.2.3 .NET Framework → .NET Core → unified .NET (5 and later)

For its first 14 years C# ran on the **.NET Framework**, which worked only on Windows. In 2016 Microsoft released **.NET Core**, a rewritten, cross-platform and much faster implementation. The two lines merged in 2020 into a single product simply called **.NET** (starting at .NET 5, skipping 4 to avoid confusion with .NET Framework 4.x). .NET Framework 4.8 is still supported on Windows for old applications, but all new development uses modern .NET.



FROM WINDOWS-ONLY TO ONE CROSS-PLATFORM .NET

1.2.4 Open source and cross-platform since .NET Core (2016)

With .NET Core, the runtime, the libraries and the C# compiler (Roslyn) became **open source** on GitHub under the MIT licence, developed in public with the **.NET Foundation**. The same C# program now runs on Windows, Linux and macOS, on x64 and ARM processors, in containers and in the browser (through WebAssembly).

1.2.5 Yearly release cycle: a new .NET version every November; even-numbered versions are LTS

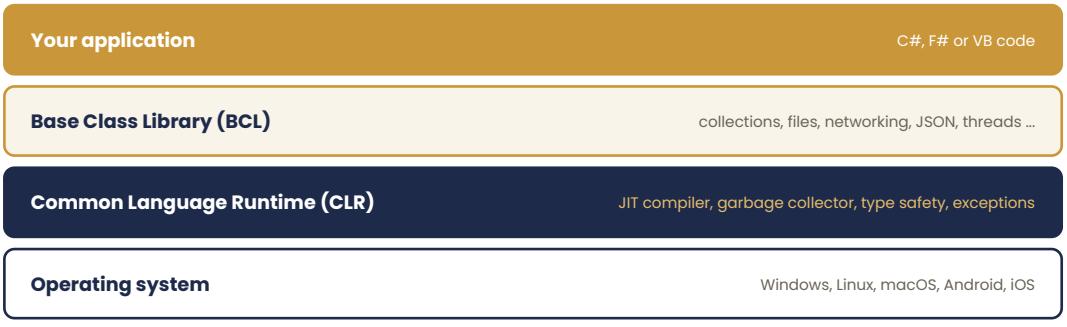
A new major version of .NET ships every November, together with a new C# version. **Even-numbered** releases (.NET 6, 8, 10) are **Long-Term Support** (LTS): supported for three years, the usual choice for production systems. **Odd-numbered** releases (.NET 7, 9) are **Standard-Term Support** (STS): supported for two years (18 months before .NET 9). Each .NET version comes with the C# version that matches it: .NET 8 with C# 12, .NET 9 with C# 13, .NET 10 with C# 14.

.NET	Released	C#	Support
.NET 6	Nov 2021	C# 10	LTS (ended Nov 2024)
.NET 7	Nov 2022	C# 11	STS (ended May 2024)
.NET 8	Nov 2023	C# 12	LTS (until Nov 2026)
.NET 9	Nov 2024	C# 13	STS (until Nov 2026)
.NET 10	Nov 2025	C# 14	LTS (until Nov 2028)

1.3 What is .NET?

1.3.1 .NET as a platform: runtime + base class library + SDK + tools

.NET is not a language but a **platform**: everything needed to build and run applications.



THE LAYERS OF THE .NET PLATFORM

Part	What it is
Runtime (CLR)	executes compiled .NET code: JIT compilation, memory management, security, exceptions
Base Class Library (BCL)	thousands of ready-made types: `string`, `List<T>`, `File`, `HttpClient`, `Task` ...
SDK	the compiler, the `dotnet` command-line tool, project templates and build system (MSBuild)
Frameworks	ASP.NET Core (web), EF Core (databases), MAUI (mobile/desktop), WPF/WinForms (Windows desktop)

1.3.2 Common Language Runtime (CLR)

The **Common Language Runtime** is the virtual machine that runs .NET programs. It loads compiled code, translates it to machine code just before it runs (JIT), allocates objects and frees them automatically with the **garbage collector**, enforces type safety, handles exceptions and manages threads. Code that runs under the CLR's control is called **managed code**.

1.3.3 Base Class Library (BCL)

The **Base Class Library** is the standard library shared by all .NET languages, organised into **namespaces**: `System` (basic types, Console, Math), `System.Collections.Generic` (List, Dictionary), `System.IO` (files), `System.Linq`, `System.Net.Http`, `System.Text.Json`, `System.Threading.Tasks` and many more. Because the library is so large, most everyday tasks need no third-party packages.

1.3.4 Common Type System (CTS) and Common Language Specification (CLS)

The **Common Type System** defines how types are declared and used in the runtime, so that every .NET language shares the same types: C#'s `int` and VB's `Integer` are both `System.Int32`. The CTS divides types into **value types** and **reference types** (Module 3). The **Common Language Specification** is a subset of rules (for example, no public members that differ only by letter case, no unsigned types in public APIs) that a library follows so that it can be used from *any* .NET language.

1.3.5 Other .NET languages: F#, Visual Basic

Because all .NET languages compile to the same intermediate language and share the CTS, they can use each other's libraries freely. **F#** is a functional-first language popular in finance and data work. **Visual Basic .NET** is mainly maintained for existing applications. C# is by far the most widely used .NET language.

1.4 How a C# program runs

1.4.1 Source code (.cs) → compiler (Roslyn) → Intermediate Language (IL) in an assembly (.dll / .exe)

C# is compiled in two stages. First, the C# compiler, called **Roslyn**, checks your `.cs` files and translates them into **Intermediate Language** (IL, also called CIL or MSIL): a CPU-independent instruction set for a stack-based virtual machine. The IL, together with **metadata** describing every type and member, is stored in an **assembly**, a `.dll` file (with a small native launcher `.exe` for applications).



FROM SOURCE CODE TO RUNNING MACHINE CODE

A METHOD AND ITS IL — C#

```
// C# source
static int Add(int a, int b) => a + b;

// The IL that Roslyn produces for Add (as shown by ILSpy or SharpLab)
.method private hidebysig static int32 Add(int32 a, int32 b) cil managed
{
    .maxstack 8
    IL_0000: ldarg.0      // push a onto the evaluation stack
    IL_0001: ldarg.1      // push b
    IL_0002: add          // pop two values, push their sum
    IL_0003: ret          // return the value on top of the stack
}
```

1.4.2 Just-In-Time (JIT) compilation to machine code by the CLR

When the program runs, the CLR does not interpret IL. The first time a method is called, the **JIT compiler** (RyuJIT) translates its IL into native machine code for the actual processor, and later calls run that native

code directly. Modern .NET uses **tiered compilation**: a method is first compiled quickly with few optimisations, and if it turns out to be called often ("hot"), it is recompiled in the background with full optimisations, even using information gathered while running (dynamic PGO). That is why .NET code is fast even though it starts as IL.

1.4.3 Ahead-of-Time compilation (Native AOT, overview)

Native AOT (ahead-of-time) compiles the whole application to native machine code at publish time (`dotnet publish -p:PublishAot=true`). The result starts almost instantly, uses less memory and needs no .NET runtime installed, which suits command-line tools, microservices and serverless functions. The trade-off: some dynamic features (unrestricted reflection, loading code at run time) are limited.

1.4.4 Managed code and garbage collection

In C# you create objects with `new` but never free them: the **garbage collector** (GC) periodically finds objects that the program can no longer reach and reclaims their memory. This removes whole classes of bugs common in C and C++: memory leaks from forgotten frees, double frees and dangling pointers. Resources other than memory (files, network connections) are still released explicitly with `Dispose` and `using` (Module 13). Module 20 explains how the GC works.

	C / C++	C# (.NET)
Compiled to	native machine code, ahead of time	IL, then JIT (or AOT) to machine code
Memory freed by	the programmer (free / delete, or RAII)	the garbage collector
Array bounds checked	no	yes: IndexOutOfRangeException
Runs on	the platform it was compiled for	any platform with a .NET runtime

1.5 Why do we need C#?

1.5.1 Productive, readable and type-safe

C# lets developers express ideas with little ceremony while the compiler catches mistakes early: type errors, possible null references (with nullable reference types), unreachable code, missing cases in switch expressions. Features such as properties, LINQ, string interpolation, records and pattern matching make common tasks short and readable.

1.5.2 Automatic memory management

The garbage collector handles memory, so programmers concentrate on the problem rather than on ownership and lifetimes, and the most dangerous class of security bugs (buffer overflows, use-after-free) is absent from safe C# code.

1.5.3 Huge standard library and NuGet ecosystem

The Base Class Library covers collections, text, files, networking, JSON, cryptography, threading and more. Beyond it, **NuGet** (nuget.org), the .NET package manager, hosts over 400,000 packages: database drivers, testing frameworks, logging, PDF and Excel libraries, cloud SDKs. Adding one is a single command: `dotnet add package Serilog`.

1.5.4 One language for web, desktop, mobile, cloud and games

With one language and one set of skills a developer can build a web API (ASP.NET Core), a browser front end (Blazor), a mobile and desktop app (.NET MAUI), Windows desktop software (WPF, WinForms), cloud functions

(Azure Functions), and games (Unity).

1.5.5 Excellent tooling (Visual Studio, Rider, VS Code)

C# has some of the best developer tools of any language: **Visual Studio** (Windows), **JetBrains Rider** (all platforms) and **VS Code** with the C# Dev Kit provide IntelliSense code completion, refactoring, a powerful debugger, profilers, test runners and live error checking as you type.

1.5.6 Strong enterprise demand

C# and .NET are used by banks, insurers, governments, healthcare, e-commerce and IT service companies worldwide, and are consistently among the most popular languages in developer surveys and job listings. In India, service companies and product firms hire large numbers of .NET developers for web and cloud projects.

1.6 Where is C# used?

C# appears in almost every area of software development:

Area	Technology	Examples
Web applications and APIs	ASP.NET Core	Stack Overflow, Microsoft's own online services, e-commerce and banking portals
Enterprise and banking software	.NET services, WPF, ASP.NET	core banking systems, insurance, ERP and HR software
Game development	**Unity** (C# scripting)	a large share of mobile and indie games, including Pokémon GO and Hollow Knight
Desktop apps	WPF, Windows Forms, WinUI	internal business tools, point-of-sale software, parts of Visual Studio
Cross-platform mobile and desktop	.NET MAUI	one code base for Android, iOS, Windows and macOS
Cloud services and microservices	Azure, Azure Functions, .NET Aspire	serverless functions, containerised microservices
Web front ends	Blazor	interactive web UIs written in C# instead of JavaScript (WebAssembly)
IoT and machine learning	.NET IoT, ML.NET, Microsoft.Extensions.AI	Raspberry Pi devices, predictions inside .NET apps, LLM integration

1.6.1 Web applications and APIs (ASP.NET Core)

ASP.NET Core is one of the fastest mainstream web frameworks (consistently near the top of the TechEmpower benchmarks). It builds REST APIs, server-rendered sites and real-time apps (SignalR), and is where most C# developers work. Modules 21 and 22 build a complete API.

1.6.2 Enterprise and banking software

Large organisations value C#'s strong typing, long-term support releases, security updates and the integration with Microsoft products (SQL Server, Azure, Active Directory, Office), so a lot of business software is written in it.

1.6.3 Game development (Unity)

Unity, one of the two most popular game engines, uses C# for all game logic. Learning C# is the first step for most aspiring game developers, and the same skills transfer to AR/VR applications built with Unity.

1.6.4 Desktop apps (WPF, Windows Forms, WinUI)

Windows Forms (2002) offers quick drag-and-drop desktop UIs; **WPF** (2006) uses XAML markup with data binding for rich interfaces; **WinUI 3** is the modern Windows UI framework. They remain common for internal business tools.

1.6.5 Cross-platform mobile and desktop apps (.NET MAUI)

.NET MAUI (Multi-platform App UI), the successor of Xamarin, builds native apps for Android, iOS, macOS and Windows from a single C# and XAML code base.

1.6.6 Cloud services and microservices (Azure)

.NET is a first-class citizen on Microsoft Azure (and runs equally well on AWS and Google Cloud): web apps, Azure Functions, containers on Kubernetes, and **.NET Aspire** for orchestrating cloud-native applications.

1.6.7 Web front ends with Blazor

Blazor lets developers write interactive web UIs in C# and Razor syntax. The code runs either on the server (with UI updates sent over a real-time connection) or directly in the browser via **WebAssembly**, so the same C# skills cover the front end too.

1.6.8 IoT and machine learning (ML.NET)

.NET IoT libraries control sensors and devices on boards such as the Raspberry Pi. **ML.NET** trains and runs machine-learning models (classification, regression, recommendation) inside .NET applications, and **Microsoft.Extensions.AI** provides a common interface for calling large language models.

1.7 C# compared with other languages

1.7.1 C# vs Java

C# and Java are close relatives: both are statically typed, object-oriented, garbage-collected languages compiled to an intermediate form run by a virtual machine. C# has evolved faster and includes features Java lacks or added later.

Aspect	C#	Java
Runtime	CLR (.NET)	JVM
Properties	built in (`public int Age { get; set; }`)	getter and setter methods (or records)
Value types	user-defined **structs**	only primitives (value classes in progress)
Generics	reified: type information kept at run time, no boxing for `List<int>`	type erasure: `List<Integer>` boxes
Async	`async` / `await` since 2012	virtual threads (Java 21)
Query language	LINQ in the language	Streams API

Aspect	C#	Java
Other	events, delegates, operator overloading, nullable reference types, unsafe code	checked exceptions

C#: A CLASS WITH PROPERTIES Program.cs

```
// C#
public class Student
{
    public string Name { get; set; } = "";
    public int Age { get; set; }
}

var s = new Student { Name = "Asha", Age = 20 };
Console.WriteLine($"{s.Name} is {s.Age}");
```

OUTPUT

```
Asha is 20
```

JAVA: THE SAME CLASS Java

```
// Java: the same class
public class Student {
    private String name = "";
    private int age;
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }
}

Student s = new Student();
s.setName("Asha");
s.setAge(20);
System.out.println(s.getName() + " is " + s.getAge());
```

1.7.2 C# vs C++

C++ compiles directly to machine code and gives full control over memory and hardware, which suits game engines, operating systems and embedded systems. C# trades some of that control for safety and productivity: garbage collection, bounds checking, a huge standard library, and much faster development. For most business, web and application software C# is the more productive choice; C# can still reach C++-like performance in hot paths with `Span<T>`, structs and unsafe code (Module 19).

Aspect	C#	C++
Memory	garbage collected	manual / RAII, smart pointers
Safety	bounds checks, no dangling pointers in safe code	undefined behaviour possible
Compilation	IL + JIT (or AOT)	native, ahead of time
Multiple inheritance	only of interfaces	of classes
Typical use	web, enterprise, Unity games, desktop	engines, systems, embedded, HPC

1.7.3 C# vs Python

Python is dynamically typed and interpreted, with very concise syntax, and dominates data science and scripting. C# is statically typed and compiled: errors are caught before the program runs, IDE support is stronger, and execution is typically 10 to 100 times faster for CPU-heavy code. Python is quicker for small scripts and experiments; C# scales better to large code bases maintained by teams.

Aspect	C#	Python
Typing	static (checked at compile time)	dynamic (checked at run time)
Speed	fast (JIT-compiled)	slower (interpreted)
Syntax	braces and semicolons	indentation
Strengths	large applications, web back ends, games, tooling	data science, ML, scripting, automation

STATIC TYPING CATCHES MISTAKES EARLY

Program.cs

```
// C#: the type of every variable is known at compile time
int total = 0;
foreach (int n in new[] { 4, 8, 15 })
    total += n;
Console.WriteLine(total);
// total = "sixty";   // error CS0029: Cannot implicitly convert type 'string' to 'int'
```

OUTPUT

```
27
```

C# is a modern, type-safe, object-oriented language that runs on the cross-platform, open-source .NET platform. Roslyn compiles C# into IL stored in assemblies; at run time the CLR JIT-compiles the IL to machine code and the garbage collector manages memory. With one language you can build web APIs, desktop and mobile apps, cloud services and Unity games.

PRACTICE CHECKPOINT

Explain in your own words the path from a **.cs file to running machine code** (compiler, IL, CLR, JIT), and list **five products or platforms** that use C#.

Sample answer. When I build my project, the C# compiler (Roslyn) checks `Program.cs` and translates it into Intermediate Language, which is saved with metadata in an assembly (`MyApp.dll`). When I run the program, the Common Language Runtime loads the assembly; the first time each method is called, the JIT compiler translates its IL into machine code for my processor, and that native code runs. Methods that are called often are recompiled with more optimisations (tiered compilation). Meanwhile the garbage collector frees objects that are no longer used.

Five products or platforms using C#: ASP.NET Core web applications (for example Stack Overflow), Unity games (for example Pokémon GO), Visual Studio and other Windows desktop applications built with WPF, mobile apps built with .NET MAUI, and serverless Azure Functions.

MODULE 02 · BEGINNER

Setup and your first program

Minimum duration **1 day** | Level **Beginner**

By the end of this module the learner has the .NET SDK and an IDE installed, can create, build and run projects from the command line, and understands every line of a console application, in both the modern top-level style and the classic Main-method style.

IN THIS MODULE

2.1 Setting up the environment
2.2 The dotnet CLI
2.3 Project structure
2.4 Hello World, line by line
2.5 Basic syntax rules
2.6 Comments
2.7 Errors

2.1 Setting up the environment

2.1.1 Installing the latest .NET SDK (LTS)

The **.NET SDK** (Software Development Kit) contains everything needed to build and run C# programs: the compiler, the runtime, the base class library and the `dotnet` command-line tool. Download it from **dot.net** (dotnet.microsoft.com/download). Choose the latest **LTS** version for long-term projects, or the latest version to get the newest language features. Installing the SDK also installs the matching runtime.

Operating system	How to install
Windows	run the installer from dot.net, or `winget install Microsoft.DotNet.SDK.9`; Visual Studio also installs it
macOS	the .pkg installer from dot.net, or `brew install --cask dotnet-sdk`
Ubuntu / Debian	`sudo apt install dotnet-sdk-8.0` (or the version you need)
Any (scripted)	the `dotnet-install.sh` / `dotnet-install.ps1` script, e.g. in CI pipelines

Several SDK versions can be installed side by side; a project chooses its target framework in its project file, and a `global.json` file can pin the SDK version used in a folder.

2.1.2 IDEs: Visual Studio (Community), VS Code with C# Dev Kit, JetBrains Rider

IDE	Platforms	Cost	Notes
Visual Studio 2022 (Community edition)	Windows	free for students, individuals and small teams	the most complete .NET IDE: designers, profiler, debugger; choose the ".NET desktop development" and "ASP.NET and web development" workloads
VS Code + C# Dev Kit	Windows, macOS, Linux	free (Dev Kit free for individuals)	lightweight editor; the extension adds a solution explorer, IntelliSense, debugging and test running

IDE	Platforms	Cost	Notes
JetBrains Rider	Windows, macOS, Linux	free for non-commercial use	fast, powerful refactorings and code analysis

2.1.3 Verifying: dotnet --version

Open a terminal (Command Prompt, PowerShell or a shell) and check the installation:

Checking the installed SDKs

```
$ dotnet --version
9.0.100
$ dotnet --list-sdks
8.0.404 [/usr/share/dotnet/sdk]
9.0.100 [/usr/share/dotnet/sdk]
$ dotnet --list-runtimes
Microsoft.AspNetCore.App 9.0.0 [/usr/share/dotnet/shared/Microsoft.AspNetCore.App]
Microsoft.NETCore.App 9.0.0 [/usr/share/dotnet/shared/Microsoft.NETCore.App]
```

If the command is not found, the SDK is not on the PATH: restart the terminal (or the computer) after installing.

2.2 The dotnet CLI

The `dotnet` command-line interface (CLI) creates, builds, runs, tests and publishes projects. IDEs call the same commands behind the scenes, so knowing the CLI helps you understand what the IDE does and lets you work on any platform and in build servers.

2.2.1 dotnet new console -n MyApp

`dotnet new` creates a project from a **template**. `console` is the template for a console application, and `-n MyApp` (or `--name`) names the project and its folder.

Creating a console project

```
$ dotnet new console -n MyApp
The template "Console App" was created successfully.

Processing post-creation actions...
Restoring /home/asha/MyApp/MyApp.csproj:
Restore succeeded.

$ cd MyApp
$ ls
MyApp.csproj  Program.cs  obj
```

2.2.2 dotnet build and dotnet run

`dotnet build` restores NuGet packages and compiles the project into `bin/Debug/net9.0/MyApp.dll`. `dotnet run` builds (if needed) and runs the program. Arguments after `--` are passed to the program itself.

Building and running

```
$ dotnet build
  Determining projects to restore...
  All projects are up-to-date for restore.
  MyApp -> /home/asha/MyApp/bin/Debug/net9.0/MyApp.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:01.84
$ dotnet run
Hello, World!
$ dotnet run -c Release -- first second      # Release build, two program arguments
```

Command	Purpose
`dotnet new <template>`	create a project, solution or file from a template
`dotnet restore`	download the NuGet packages the project needs (run automatically by build)
`dotnet build`	compile; `-c Release` for an optimised build
`dotnet run`	build and run
`dotnet watch`	run, and rebuild/restart automatically when files change
`dotnet test`	run unit tests
`dotnet add package <Name>`	add a NuGet package reference
`dotnet publish`	produce the files to deploy
`dotnet clean`	delete build outputs

2.2.3 Running a single .cs file directly (dotnet run app.cs, .NET 10)

Starting with .NET 10, a single C# file can be run without a project file: `dotnet run hello.cs`. The SDK creates a hidden project behind the scenes. Such "file-based apps" can reference packages with directives at the top of the file (`#:package Humanizer@2.14.1`), and on Linux and macOS a shebang line makes them executable scripts. When the script grows, `dotnet project convert hello.cs` turns it into a normal project.

A FILE-BASED APP (.NET 10) C#

```
#!/usr/bin/env dotnet
// hello.cs: run with "dotnet run hello.cs" (.NET 10 or later)
Console.WriteLine($"Hello from a single file at {DateTime.Now:HH:mm}!");
```

2.2.4 Project templates

Some of the built-in templates (abridged)

```
$ dotnet new list
Template Name                 Short Name      Language    Tags
----------------------------  --------------  ----------  ----------------------
ASP.NET Core Empty            web             [C#],F#     Web/Empty
ASP.NET Core Web API          webapi          [C#],F#     Web/WebAPI
Blazor Web App                blazor          [C#]        Web/Blazor
Class Library                 classlib        [C#],F#,VB  Common/Library
Console App                   console         [C#],F#,VB  Common/Console
MSTest Test Project           mstest          [C#],F#,VB  Test/MSTest
NUnit 3 Test Project          nunit           [C#],F#,VB  Test/NUnit
Solution File                 sln                         Solution
Worker Service                worker          [C#],F#     Common/Worker/Web
xUnit Test Project            xunit           [C#],F#,VB  Test/xUnit
...
```

2.3 Project structure

2.3.1 Solution (.sln / .slnx) and project (.csproj) files

A **project** (`.csproj`) produces one assembly: an application or a library. A **solution** (`.sln`, or the newer XML-based `.slnx`) groups related projects, for example an API, a class library and a test project, so they can be built and opened together. The project file is a short XML file in the "SDK style":

MYAPP.CSPROJ

text

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>                  <!-- an executable, not a library -->
    <TargetFramework>net9.0</TargetFramework>     <!-- which .NET version to build for -->
    <ImplicitUsings>enable</ImplicitUsings>       <!-- common using directives added automatically -->
    <Nullable>enable</Nullable>                   <!-- nullable reference type warnings on -->
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Humanizer" Version="2.14.1" />   <!-- a NuGet package -->
  </ItemGroup>

</Project>
```

A solution with a library and an app

```
$ dotnet new sln -n School
$ dotnet new classlib -n School.Core
$ dotnet new console -n School.App
$ dotnet sln add School.Core School.App
$ dotnet add School.App reference School.Core     # the app uses the library
$ dotnet build School.sln
```

2.3.2 Program.cs

`Program.cs` holds the program's entry point. In the console template it contains a single statement, using **top-level statements**: the compiler generates the surrounding class and `Main` method for you (2.4). All `.cs` files in the project folder (and subfolders) are compiled automatically; there is no list of source files to maintain.

2.3.3 bin and obj folders

`obj/` holds intermediate files (restored package information, generated code, incremental-build caches). `bin/` holds the build output: `bin/Debug/net9.0/` contains `MyApp.dll` (the IL), `MyApp.exe` or `MyApp` (a small native launcher), `MyApp.pdb` (debug symbols) and `MyApp.runtimeconfig.json`. Both folders can be deleted at any time and are excluded from source control (the `dotnet new gitignore` template lists them).

2.3.4 Assemblies and namespaces

An **assembly** is the unit of deployment: a `.dll` file containing IL, metadata and resources, with a name and version. A **namespace** is a logical grouping of type names that prevents clashes: `System.IO.File` and `MyCompany.Storage.File` can coexist. The two are independent: one assembly may contain many namespaces, and one namespace may be spread over several assemblies.

	Assembly	Namespace
What it is	a physical file (.dll)	a logical name prefix
Purpose	deployment, versioning, security, loading	organising and naming types
Created by	a project	the `namespace` keyword
Example	`System.Text.Json.dll`	`System.Text.Json`

2.4 Hello World, line by line

2.4.1 Top-level statements: Console.WriteLine("Hello, World!");

PROGRAM.CS CREATED BY THE CONSOLE TEMPLATE

Program.cs

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Hello, World!");
```

OUTPUT

```
Hello, World!
```

That is the whole program. `Console` is a class in the `System` namespace that represents the terminal; `WriteLine` is a method of it that prints its argument followed by a newline; `"Hello, World!"` is a string literal; the semicolon ends the statement. Since C# 9 a program may start directly with statements (**top-level statements**) in exactly one file of the project. The compiler wraps them in a generated `Program` class with a `Main` method; `args` is available as a variable, and `return 3;` sets the exit code.

MORE STATEMENTS

Program.cs

```
Console.WriteLine("Hello, World!");
Console.WriteLine("Welcome to C#");
Console.Write("No newline here, ");
Console.WriteLine("then a newline.");
Console.WriteLine();                          // an empty line
Console.WriteLine($"You passed {args.Length} argument(s).");
```

OUTPUT

```
Hello, World!
Welcome to C#
No newline here, then a newline.

You passed 0 argument(s).
```

2.4.2 The classic structure: namespace, class Program, static void Main(string[] args)

Before C# 9 every program had to be written out in full, and you will see this structure in older code, books and many companies' code bases. Both forms compile to the same IL.

THE SAME PROGRAM IN THE CLASSIC FORM Program.cs

```
using System;                          // 1. import the System namespace

namespace HelloApp                     // 2. a namespace for our types
{
    class Program                      // 3. a class: all code lives inside types
    {
        static void Main(string[] args)   // 4. the entry point
        {
            Console.WriteLine("Hello, World!");   // 5. a statement
        }
    }
}
```

OUTPUT

```
Hello, World!
```

Line	Meaning
`using System;`	lets us write `Console` instead of `System.Console`
`namespace HelloApp`	puts the class in a namespace to avoid name clashes
`class Program`	C# code must be inside a type; the name Program is just a convention
`static`	Main belongs to the class itself; no Program object is created to call it
`void`	Main returns nothing (it may also return `int` as the exit code, or `Task` for async code)
`Main`	the method where execution starts: there must be exactly one
`string[] args`	the command-line arguments (optional parameter)

2.4.3 using directives, global usings and implicit usings

A `using` directive imports a namespace so its types can be used without the full name. `global using System.Text;` (C# 10) applies to every file in the project. With `<ImplicitUsings>enable</ImplicitUsings>` in the project file, the SDK generates global usings for the most common namespaces, which is why the template's one-line program needs no `using` at all. Other forms: `using static System.Math;` imports the static members of a class, and `using Json = System.Text.Json.JsonSerializer;` creates an alias.

THE IMPLICIT GLOBAL USINGS OF A CONSOLE PROJECT C#

```
// <auto-generated/> obj/Debug/net9.0/MyApp.GlobalUsings.g.cs for a console project
global using global::System;
global using global::System.Collections.Generic;
global using global::System.IO;
global using global::System.Linq;
global using global::System.Net.Http;
global using global::System.Threading;
global using global::System.Threading.Tasks;
```

USING STATIC AND USING ALIASES Program.cs

```
using static System.Math;                // static members of Math without the class name
using Dict = System.Collections.Generic.Dictionary<string, int>;   // an alias

double hyp = Sqrt(Pow(3, 2) + Pow(4, 2));
Console.WriteLine($"hypotenuse = {hyp}, PI = {Round(PI, 4)}");

var stock = new Dict { ["pen"] = 12, ["ink"] = 3 };
Console.WriteLine($"{stock.Count} items, pens = {stock["pen"]}");
```

OUTPUT

```
hypotenuse = 5, PI = 3.1416
2 items, pens = 12
```

2.4.4 File-scoped namespaces

A file usually contains types from one namespace. Since C# 10, `namespace School.Models;` at the top of the file applies to the whole file, saving one level of braces and indentation. It is the recommended style for new code.

MODELS.CS WITH A FILE-SCOPED NAMESPACE C#

```
namespace School.Models;                // file-scoped: applies to everything below

public class Student
{
    public string Name { get; set; } = "";
}

public class Course
{
    public string Title { get; set; } = "";
}
```

2.5 Basic syntax rules

2.5.1 Case sensitivity and semicolons

C# is **case-sensitive**: `name`, `Name` and `NAME` are three different identifiers, and `console.writeline` is an error. Every statement ends with a semicolon; a statement can span several lines and several statements can share a line (not recommended).

WRONG LETTER CASE Program.cs

```
int count = 5;
Console.WriteLine(Count);
console.WriteLine(count);
```

COMPILER OUTPUT

```
Program.cs(2,19): error CS0103: The name 'Count' does not exist in the current context
Program.cs(3,1): error CS0103: The name 'console' does not exist in the current context
```

2.5.2 Blocks { } and whitespace

Braces group statements into a **block**: the body of a class, a method, a loop or an if. A block also defines a scope for variables declared inside it. Whitespace (spaces, tabs, newlines) is ignored by the compiler except

inside strings, so indentation is for humans; the convention in C# (Allman style) puts each brace on its own line and indents by four spaces.

A BLOCK Program.cs

```
int marks = 82;
if (marks >= 40)
{                                   // the block starts
    string grade = marks >= 75 ? "distinction" : "pass";
    Console.WriteLine($"Result: {grade}");
}                                   // grade no longer exists after this brace
Console.WriteLine("Done");
```

OUTPUT

```
Result: distinction
Done
```

2.5.3 Keywords and contextual keywords

Keywords are reserved words with a fixed meaning that cannot be used as identifiers: `int`, `class`, `if`, `return`, `new`, `static` and about 75 more. (Prefix one with `@` to use it as a name anyway: `@class`.) **Contextual keywords** have special meaning only in certain places and remain valid identifiers elsewhere, which lets the language add features without breaking old code: `var`, `async`, `await`, `yield`, `get`, `set`, `init`, `value`, `record`, `required`, `when`, `where`, `nameof`, `field`.

Reserved keywords (examples)	Contextual keywords (examples)
`abstract as base bool break byte case catch char class const continue decimal default delegate do double else enum event false finally float for foreach if int interface internal is long namespace new null object out override private protected public readonly ref return sealed short static string struct switch this throw true try typeof uint using virtual void while`	`add and async await by dynamic equals field file get global init into join let nameof not on or orderby partial record remove required scoped select set unmanaged value var when where with yield`

2.5.4 Identifiers and naming conventions (PascalCase, camelCase, _camelCase for private fields)

Identifiers (names) consist of letters, digits and underscores, must not start with a digit, and must not be keywords. The .NET naming conventions are followed almost universally:

Element	Convention	Example
Namespaces, classes, structs, records, enums	PascalCase	`StudentRecord`, `System.IO`
Methods, properties, events, public fields	PascalCase	`CalculateTotal()`, `FirstName`
Interfaces	I + PascalCase	`IComparable`, `IStudentRepository`
Local variables and parameters	camelCase	`totalMarks`, `studentId`
Private fields	_camelCase	`_balance`, `_logger`
Constants	PascalCase	`MaxStudents`
Type parameters	T or T + PascalCase	`T`, `TKey`, `TResult`
Async methods	end in Async	`LoadStudentsAsync()`

2.6 Comments

2.6.1 Single-line // and multi-line /* */

Comments are ignored by the compiler. `//` comments to the end of the line; `/* ... */` spans several lines (and cannot be nested). Good comments explain *why* something is done; the code itself should show *what*. In Visual Studio, Ctrl+K, Ctrl+C comments out the selected lines.

SINGLE-LINE AND MULTI-LINE COMMENTS Program.cs

```
// Calculate the simple interest for a fixed deposit
decimal principal = 50000m;
decimal rate = 7.5m;           // percent per year
int years = 3;
/* Simple interest = P x R x T / 100.
   decimal is used because this is money. */
decimal interest = principal * rate * years / 100;
Console.WriteLine($"Interest: {interest:N2}");      // N2: thousands separators, 2 decimals
```

OUTPUT

```
Interest: 11,250.00
```

2.6.2 XML documentation comments ///

Comments starting with `///` placed before a type or member are **XML documentation comments**. IDEs show them as tooltips in IntelliSense, and with `<GenerateDocumentationFile>true</GenerateDocumentationFile>` the compiler collects them into an XML file used by documentation generators. Common tags: `<summary>`, `<param>`, `<returns>`, `<exception>`, `<example>`, `<see cref=""/>`.

DOCUMENTING A METHOD Program.cs

```
Console.WriteLine(Finance.Emi(500000, 9.5, 60));

static class Finance
{
    /// <summary>Calculates the monthly instalment (EMI) of a loan.</summary>
    /// <param name="principal">The amount borrowed.</param>
    /// <param name="annualRatePercent">The yearly interest rate, for example 9.5.</param>
    /// <param name="months">The number of monthly instalments.</param>
    /// <returns>The EMI rounded to two decimal places.</returns>
    public static double Emi(double principal, double annualRatePercent, int months)
    {
        double r = annualRatePercent / 12 / 100;
        double factor = Math.Pow(1 + r, months);
        return Math.Round(principal * r * factor / (factor - 1), 2);
    }
}
```

OUTPUT

```
10500.93
```

NOTE

With top-level statements, other types (here the class `Finance`) must come after the top-level code in the file.

2.7 Errors

2.7.1 Compile-time errors and warnings

Compile-time errors are found by the compiler before the program runs: syntax errors, type mismatches, undefined names. The program cannot run until they are fixed. Each has a code such as `CS1002` that you can search for. **Warnings** point out suspicious code that still compiles: an unused variable, a possible null dereference, unreachable code. Treat warnings seriously (many teams enable `<TreatWarningsAsErrors>`).

TYPICAL COMPILE-TIME ERRORS — Program.cs

```
int total = 10
string name = "Asha";
int age = "twenty";
Console.WriteLine(totl);
```

COMPILER OUTPUT

```
Program.cs(1,15): error CS1002: ; expected
Program.cs(3,11): error CS0029: Cannot implicitly convert type 'string' to 'int'
Program.cs(4,19): error CS0103: The name 'totl' does not exist in the current context
```

TYPICAL WARNINGS (THE PROGRAM STILL BUILDS) — Program.cs

```
int unused = 5;
string? city = null;
Console.WriteLine(city.Length);
return;
Console.WriteLine("never runs");
```

COMPILER OUTPUT

```
Program.cs(1,5): warning CS0219: The variable 'unused' is assigned but its value is never used
Program.cs(3,19): warning CS8602: Dereference of a possibly null reference.
Program.cs(5,1): warning CS0162: Unreachable code detected
```

2.7.2 Runtime exceptions

A program that compiles can still fail while running: dividing an integer by zero, reading past the end of an array, converting "abc" to a number, using a null reference. The runtime then throws an **exception**. If nothing catches it (Module 13), the program stops and prints the exception type, its message and a stack trace.

AN UNHANDLED EXCEPTION — Program.cs

```
int[] marks = { 70, 85, 90 };
Console.WriteLine("Reading marks...");
Console.WriteLine(marks[3]);          // there is no element 3
Console.WriteLine("This line is never reached");
```

OUTPUT

```
Reading marks...
Unhandled exception. System.IndexOutOfRangeException: Index was outside the bounds of the array.
   at Program.<Main>$(String[] args) in /home/asha/MyApp/Program.cs:line 3
```

2.7.3 Logical errors

A **logical error** (a bug) produces wrong results without any error message: the compiler and the runtime are satisfied, but the program does not do what was intended. These are found by testing, by checking results by hand, and with the debugger.

A LOGICAL ERROR

Program.cs

```
int a = 7, b = 8;
double average = (a + b) / 2;          // bug: integer division happens first
Console.WriteLine($"Average: {average}");
double correct = (a + b) / 2.0;        // fix: divide by a double
Console.WriteLine($"Average: {correct}");
```

OUTPUT

```
Average: 7
Average: 7.5
```

2.7.4 Reading the Error List and stack traces

In Visual Studio the **Error List** window (View > Error List) shows every error and warning with its code, description, file and line; double-clicking jumps to the line. In the terminal, `dotnet build` prints the same information as `File(line,column): error CODE: message`. Read the **first** error first: later ones are often caused by it.

A **stack trace** lists the chain of method calls that were active when an exception occurred, innermost first. Find the first line that belongs to your code: it shows the method, file and line that failed.

READING A STACK TRACE

Program.cs

```
Console.WriteLine(Report.AverageOf(new int[0]));

static class Report
{
    public static int AverageOf(int[] values) => Sum(values) / values.Length;
    static int Sum(int[] values)
    {
        int s = 0;
        foreach (int v in values) s += v;
        return s;
    }
}
```

OUTPUT

```
Unhandled exception. System.DivideByZeroException: Attempted to divide by zero.
   at Report.AverageOf(Int32[] values) in /home/asha/MyApp/Program.cs:line 5
   at Program.<Main>$(String[] args) in /home/asha/MyApp/Program.cs:line 1
```

Reading from the top: the exception is a `DivideByZeroException`, thrown in `Report.AverageOf` at line 5 (the division by `values.Length`, which is 0), called from the top-level code at line 1. The fix is to handle the empty array (return 0, or throw a clearer `ArgumentException`).

Install the .NET SDK and an IDE, then use the dotnet CLI: new, build, run, test, publish. A project (.csproj) builds one assembly; a solution groups projects. Programs can use top-level statements or the classic namespace/class/Main structure. C# is case-sensitive, statements end with semicolons, blocks use braces, and names follow PascalCase/camelCase conventions. Fix compile-time errors from the first one down, read stack traces from the top, and test to find logical errors.

PRACTICE CHECKPOINT

Create, build and run a console app **from the CLI and from an IDE**, once with **top-level statements** and once with the **classic Main method**.

Step 1: both styles from the command line

```
$ dotnet new console -n Greeter
$ cd Greeter
$ dotnet run
Hello, World!
$ dotnet new console -n GreeterClassic --use-program-main     # template option: classic Main style
$ cd ../GreeterClassic
$ dotnet run
Hello, World!
```

STEP 2: TOP-LEVEL VERSION Program.cs

```
// Greeter/Program.cs: top-level statements
Console.Write("What is your name? ");
string name = Console.ReadLine() ?? "friend";
Console.WriteLine($"Hello, {name}! Today is {DateTime.Today:dddd}.");
```

CONSOLE (user input in gold)

```
What is your name? Asha
Hello, Asha! Today is Tuesday.
```

STEP 3: CLASSIC VERSION Program.cs

```
// GreeterClassic/Program.cs: classic structure
namespace GreeterClassic;

internal class Program
{
    private static void Main(string[] args)
    {
        Console.Write("What is your name? ");
        string name = Console.ReadLine() ?? "friend";
        Console.WriteLine($"Hello, {name}! Today is {DateTime.Today:dddd}.");
    }
}
```

CONSOLE (user input in gold)

```
What is your name? Asha
Hello, Asha! Today is Tuesday.
```

Step 4: open each folder in Visual Studio, VS Code or Rider and press F5 (or the Run button): the IDE runs the same `dotnet build` and starts the program in its terminal. The day of the week shown depends on when you run it.

MODULE 03 · BEGINNER

Variables and data types

Minimum duration **2 days** | Level **Beginner**

C# is statically typed: every variable has a type fixed at compile time, and every type belongs to the .NET Common Type System. This module covers declaring variables, the kinds of variables, the crucial difference between value types and reference types, all the built-in types, literals, constants, conversions, nullable value types and default values.

IN THIS MODULE

3.1	Variables	3.6	Literals
3.2	Types of variables	3.7	Constants
3.3	Value types vs reference types	3.8	Type conversion
3.4	Built-in value types	3.9	Nullable value types
3.5	Built-in reference types	3.10	Default values

3.1 Variables

3.1.1 Declaration and initialization

A **variable** is a named storage location that holds a value of a particular type. A **declaration** states the type and the name; an **initialization** gives the first value. They can be combined, and several variables of the same type can be declared in one statement. The value of a variable can change later (that is why it is called a variable), but its type cannot.



A VARIABLE: A TYPED, NAMED BOX

DECLARING AND INITIALIZING VARIABLES Program.cs

```
int age;                        // declaration
age = 21;                       // assignment (here: initialization)
string name = "Asha";           // declaration + initialization
double height = 1.62, weight = 54.5;   // two variables of one type
bool isEnrolled = true;
char grade = 'A';

age = age + 1;                  // the value can change ...
// age = "twenty-two";          // ... the type cannot: error CS0029

Console.WriteLine($"{name}, {age}, {height} m, {weight} kg, grade {grade}, enrolled: {isEnrolled}");
```

OUTPUT

```
Asha, 22, 1.62 m, 54.5 kg, grade A, enrolled: True
```

NOTE

`bool` values print as `True` and `False` with a capital letter, because they are converted with `bool.ToString()`; the C# literals themselves are written `true` and `false`.

3.1.2 Definite assignment rule

C# never lets you read a local variable that might not have been assigned. The compiler follows every path through the code; if some path reaches a use of the variable without assigning it, compilation fails. This removes the "garbage value" bugs of C and C++. (Fields and array elements are different: they are automatically set to default values, 3.10.)

USE OF AN UNASSIGNED LOCAL VARIABLE Program.cs

```
int total;
int marks = 75;
if (marks >= 40)
    total = marks + 5;          // assigned only on this path
Console.WriteLine(total);       // error: maybe unassigned
```

COMPILER OUTPUT

```
Program.cs(5,19): error CS0165: Use of unassigned local variable 'total'
```

FIXED: EVERY PATH ASSIGNS THE VARIABLE Program.cs

```
int total;
int marks = 75;
if (marks >= 40)
    total = marks + 5;
else
    total = 0;                  // now every path assigns total
Console.WriteLine(total);
```

OUTPUT

```
80
```

3.1.3 Implicitly typed variables with var

With `var` the compiler infers the type from the initializer. The variable is still statically typed: `var x = 5;` makes `x` an `int` forever. `var` requires an initializer (and not `null` alone) and is allowed only for local variables. Use it when the type is obvious from the right-hand side or very long; write the type when it helps the reader.

VAR INFERS THE TYPE AT COMPILE TIME

Program.cs

```
var count = 10;                      // int
var price = 99.5;                    // double
var amount = 99.5m;                  // decimal
var city = "Mysuru";                 // string
var scores = new List<int> { 90, 85 };   // List<int>: no need to write the type twice
var lookup = new Dictionary<string, List<int>>();

Console.WriteLine(count.GetType());
Console.WriteLine(price.GetType());
Console.WriteLine(amount.GetType());
Console.WriteLine(city.GetType());
Console.WriteLine(scores.GetType().Name);
// count = "ten";                    // error CS0029: count is an int
```

OUTPUT

```
System.Int32
System.Double
System.Decimal
System.String
List`1
```

NOTE

`List`1` is the runtime name of the generic type `List<T>`: the backtick and 1 mean "one type parameter".

3.1.4 Naming rules and conventions

Rules (enforced by the compiler): a name contains letters, digits and underscores; it cannot start with a digit; it cannot be a keyword (unless prefixed with `@`); it is case-sensitive. Conventions (followed by everyone): local variables and parameters in **camelCase** (`totalMarks`), private fields in **_camelCase**, everything public in **PascalCase**. Choose names that say what the value means: `daysUntilExam`, not `d` or `temp2`.

Valid	Invalid	Why invalid
`studentName`	`2ndPlace`	starts with a digit
`_count`	`total-marks`	hyphen is the minus operator
`marks2025`	`class`	keyword (use `@class` if you must)
`तापमान` (Unicode letters allowed)	`my name`	contains a space

3.2 Types of variables

3.2.1 Local variables

A **local variable** is declared inside a method, block, loop or top-level code. It exists only while that block runs and is visible only inside it. Locals must be definitely assigned before use.

3.2.2 Fields (instance and static)

A **field** is a variable declared directly in a class or struct. An **instance field** belongs to each object: every `Student` has its own name. A **static field** belongs to the type itself and is shared by all objects: one counter for all students. Fields get default values automatically if not initialised.

3.2.3 Parameters

A **parameter** is a variable that receives a value passed to a method. Parameters behave like local variables initialised by the caller. Module 9 covers `ref`, `out`, `in` and `params` parameters.

LOCALS, FIELDS AND PARAMETERS

Program.cs

```
var a = new Student("Asha");
var b = new Student("Ravi");
Console.WriteLine($"{a.Name} is student {a.Id}, {b.Name} is student {b.Id}");
Console.WriteLine($"Students created: {Student.Count}");
Console.WriteLine(Student.Describe(a, 20));

class Student
{
    public static int Count;           // static field: one copy shared by all students
    public string Name;                // instance field: each student has its own
    public int Id;

    public Student(string name)        // name is a parameter
    {
        Count++;
        Id = Count;
        Name = name;
    }

    public static string Describe(Student s, int age)   // two parameters
    {
        string label = $"{s.Name} ({age})";              // a local variable
        return label;
    }
}
```

OUTPUT

```
Asha is student 1, Ravi is student 2
Students created: 2
Asha (20)
```

3.2.4 Scope and lifetime

Scope is the region of source code where a name can be used; **lifetime** is the period during execution when the storage exists. A local's scope is its enclosing block, from the declaration onwards. C# forbids declaring a local with the same name as another local in an enclosing scope, which avoids confusing shadowing. Fields are in scope throughout their class and live as long as their object (instance fields) or the whole program (static fields).

SCOPE OF LOCAL VARIABLES Program.cs

```
int x = 10;
for (int i = 0; i < 3; i++)
{
    int square = i * i;                // square and i exist only inside the loop
    Console.Write($"{square} ");
}
Console.WriteLine();
// Console.WriteLine(i);               // error CS0103: The name 'i' does not exist in the current
context
{
    int y = x * 2;                     // a nested block: x is visible here
    Console.WriteLine(y);
    // int x = 5;                      // error CS0136: a local named 'x' cannot be declared in this
scope
}
```

OUTPUT

```
0 1 4
20
```



NESTED SCOPES: INNER BLOCKS SEE OUTER NAMES, NOT THE REVERSE

3.3 Value types vs reference types

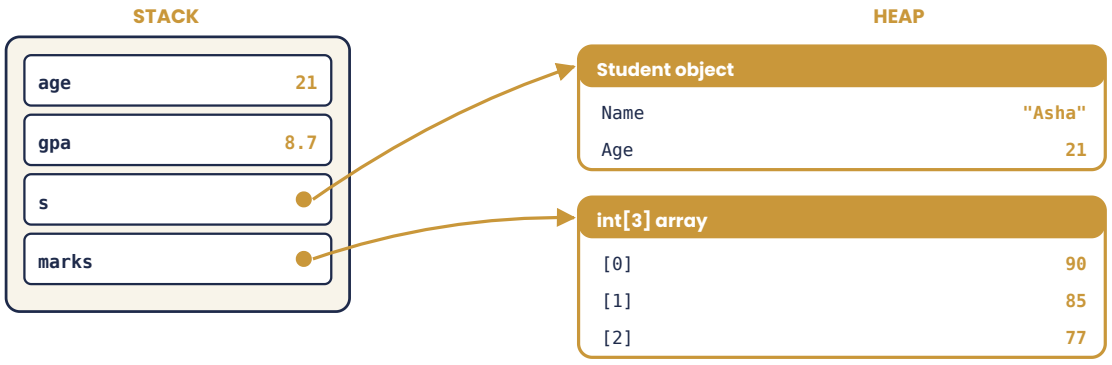
Every type in C# is either a **value type** or a **reference type**, and the difference affects how assignment, method calls and comparison behave. This is one of the most important ideas in the language and a favourite interview topic.

	Value types	Reference types
Examples	int, double, bool, char, decimal, DateTime, enums, structs, tuples	string, arrays, classes, records, interfaces, delegates, object
Variable holds	the value itself	a reference (address) to an object on the heap
Assignment copies	the whole value	only the reference: both variables point to one object
Can be null	no (unless nullable: `int?`)	yes
Defined with	`struct`, `enum`	`class`, `record`, `interface`, `delegate`

3.3.1 Stack vs heap (simplified model)

In the simplified model used by most teachers: local variables live on the **stack**, a fast region that grows and shrinks as methods are called and return; objects created with `new` live on the **managed heap**, cleaned up

by the garbage collector. A local of a value type holds its value directly on the stack; a local of a reference type holds, on the stack, a reference to the object on the heap. (The precise rule is about lifetime, not type: value types that are fields of a class live inside that object on the heap, Module 20.)



VALUE TYPES HOLD THEIR DATA; REFERENCE TYPES POINT TO OBJECTS ON THE HEAP

3.3.2 Copy behaviour of value types and reference types

COPYING VALUES VS COPYING REFERENCES

Program.cs

```
// Value type: assignment copies the value
int a = 10;
int b = a;          // b gets its own copy of 10
b = 99;
Console.WriteLine($"a = {a}, b = {b}");

// Reference type: assignment copies the reference
int[] first = { 1, 2, 3 };
int[] second = first;   // both variables refer to the SAME array
second[0] = 99;
Console.WriteLine($"first[0] = {first[0]}, second[0] = {second[0]}");

// A struct (value type) vs a class (reference type)
PointStruct p1 = new PointStruct { X = 1 };
PointStruct p2 = p1;    // copy
p2.X = 50;
PointClass c1 = new PointClass { X = 1 };
PointClass c2 = c1;     // same object
c2.X = 50;
Console.WriteLine($"struct: p1.X = {p1.X}   class: c1.X = {c1.X}");

struct PointStruct { public int X; }
class PointClass { public int X; }
```

OUTPUT

```
a = 10, b = 99
first[0] = 99, second[0] = 99
struct: p1.X = 1   class: c1.X = 50
```



COPYING AN INT DUPLICATES THE VALUE; COPYING AN ARRAY REFERENCE SHARES THE OBJECT

3.3.3 System.Object as the root of all types

Every type in .NET, value or reference, built-in or your own, derives directly or indirectly from `System.Object` (keyword `object`). So every value has the methods `ToString()`, `Equals()`, `GetHashCode()` and `GetType()`, and any value can be stored in a variable of type `object`. Value types derive from `System.ValueType`, which itself derives from `object`.

EVERYTHING IS AN OBJECT Program.cs

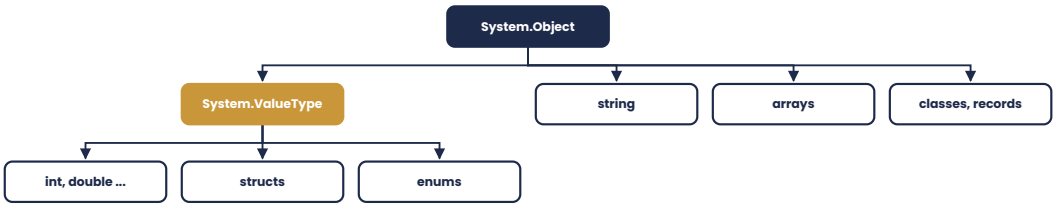
```
object[] things = { 42, 3.14, "text", 'c', true, new int[] { 1, 2 }, DateTime.MinValue };
foreach (object o in things)
    Console.WriteLine($"{o.GetType().Name,-10} value type? {o.GetType().IsValueType,-5}  ToString: {o}");
```

OUTPUT

```
Int32      value type? True   ToString: 42
Double     value type? True   ToString: 3.14
String     value type? False  ToString: text
Char       value type? True   ToString: c
Boolean    value type? True   ToString: True
Int32[]    value type? False  ToString: System.Int32[]
DateTime   value type? True   ToString: 01-01-0001 00:00:00
```

NOTE

The format of `DateTime` depends on the computer's culture: the line above is an Indian English (en-IN) machine; en-US prints `1/1/0001 12:00:00 AM`. Arrays do not override `ToString()`, so they print their type name.



EVERY TYPE DERIVES FROM SYSTEM.OBJECT

3.4 Built-in value types

3.4.1 Integer types: sbyte, byte, short, ushort, int, uint, long, ulong, nint, nuint

Keyword	.NET type	Size	Range	Typical use
`sbyte`	System.SByte	1 byte	-128 to 127	rare
`byte`	System.Byte	1 byte	0 to 255	binary data, file contents, pixels

Keyword	.NET type	Size	Range	Typical use
`short`	System.Int16	2 bytes	-32,768 to 32,767	compact storage
`ushort`	System.UInt16	2 bytes	0 to 65,535	ports, protocols
`int`	System.Int32	4 bytes	about ±2.1 billion	**the default integer**
`uint`	System.UInt32	4 bytes	0 to about 4.29 billion	bit flags, interop
`long`	System.Int64	8 bytes	about $\pm 9.2 \times 10^{18}$	large counts, file sizes, ticks
`ulong`	System.UInt64	8 bytes	0 to about 1.8×10^{19}	hashes, IDs
`nint` / `nuint`	System.IntPtr / UIntPtr	4 or 8 bytes	depends on the process (32/64-bit)	native interop, pointer-sized values

INTEGER TYPES Program.cs

```
int population = 1_428_000_000;          // India's population fits in an int
long worldPopulation = 8_100_000_000;    // too big for an int
byte brightness = 255;
short temperature = -40;
uint flags = 0xFF00_FF00;
ulong bigId = 18_000_000_000_000_000_000;
Console.WriteLine($"{population} {worldPopulation} {brightness} {temperature} {flags} {bigId}");
Console.WriteLine($"sizeof(int) = {sizeof(int)}, sizeof(long) = {sizeof(long)}, nint size =
{nint.Size}");
```

OUTPUT

```
1428000000 8100000000 255 -40 4278255360 18000000000000000000
sizeof(int) = 4, sizeof(long) = 8, nint size = 8
```

3.4.2 Floating-point types: float, double, decimal (for money)

Keyword	Size	Precision	Approximate range	Use for
`float`	4 bytes	6-9 digits	$\pm 1.5 \times 10^{-45}$ to $\pm 3.4 \times 10^{38}$	graphics, games (Unity), large arrays of measurements
`double`	8 bytes	15-17 digits	$\pm 5.0 \times 10^{-324}$ to $\pm 1.7 \times 10^{308}$	**the default** for scientific and general calculations
`decimal`	16 bytes	28-29 digits	$\pm 1.0 \times 10^{-28}$ to $\pm 7.9 \times 10^{28}$	**money** and any value that must be exact in decimal

`float` and `double` store numbers in **binary** (IEEE 754), so most decimal fractions such as 0.1 cannot be represented exactly and tiny rounding errors appear. `decimal` stores a 96-bit integer and a power of ten, so 0.1 is exact; it is slower and has a smaller range, but it is the right type for prices, balances and interest.

DOUBLE VS DECIMAL Program.cs

```
double d = 0.1 + 0.2;
decimal m = 0.1m + 0.2m;
float f = 0.1f + 0.2f;
Console.WriteLine($"double : {d}");
Console.WriteLine($"decimal: {m}");
Console.WriteLine($"float  : {f}");
Console.WriteLine($"0.1 + 0.2 == 0.3 ? double: {d == 0.3}, decimal: {m == 0.3m}");

double total = 0;
for (int i = 0; i < 10; i++) total += 0.1;     // add 10 paise ten times
Console.WriteLine($"ten times 0.1 as double = {total}");
decimal exact = 0;
for (int i = 0; i < 10; i++) exact += 0.1m;
Console.WriteLine($"ten times 0.1 as decimal = {exact}");
Console.WriteLine($"1.0 / 3 = {1.0 / 3}, 1.0m / 3 = {1.0m / 3}");
```

OUTPUT

```
double : 0.30000000000000004
decimal: 0.3
float  : 0.3
0.1 + 0.2 == 0.3 ? double: False, decimal: True
ten times 0.1 as double = 0.9999999999999999
ten times 0.1 as decimal = 1.0
1.0 / 3 = 0.3333333333333333, 1.0m / 3 = 0.3333333333333333333333333333
```

NOTE

The float sum prints `0.3` only because float has fewer digits: its result 0.3 (rounded to float) happens to be the float closest to 0.3. A decimal also keeps trailing zeros from its operands: ten additions of 0.1m give `1.0`.

WATCH OUT

Never use `float` or `double` for money. Use `decimal` (with the `m` suffix on literals), and compare doubles with a tolerance (`Math.Abs(a - b) < 1e-9`) instead of `==`.

3.4.3 char (UTF-16)

`char` holds one **UTF-16 code unit** (2 bytes), written in single quotes: `'A'`, `'₹'`, `'\n'`. Characters are numbers underneath: `'A'` is 65. Characters outside the Basic Multilingual Plane, such as many emoji, need two chars (a surrogate pair), which is why `"😀".Length` is 2. The `char` type has helpful static methods: `char.IsDigit`, `char.IsLetter`, `char.ToUpper` (Module 8).

WORKING WITH CHAR Program.cs

```
char letter = 'A';
char rupee = '\u20B9';                  // Unicode escape for the rupee sign
int code = letter;                       // char converts implicitly to int
char next = (char)(letter + 1);          // int to char needs a cast
Console.WriteLine($"{letter} has code {code}; next is {next}");
Console.WriteLine($"rupee sign code: {(int)rupee}, is letter? {char.IsLetter(rupee)}, is symbol?
{char.IsSymbol(rupee)}");
Console.WriteLine($"'7' is digit: {char.IsDigit('7')}, numeric value: {char.GetNumericValue('7')}, '7' -
'0' = {'7' - '0'}");
```

OUTPUT

```
A has code 65; next is B
rupee sign code: 8377, is letter? False, is symbol? True
'7' is digit: True, numeric value: 7, '7' - '0' = 7
```

3.4.4 bool

`bool` has exactly two values, `true` and `false`. Unlike C and C++, C# does not convert between `bool` and numbers: `if (count)` is an error, you must write `if (count != 0)`. This prevents the classic `if (x = 5)` mistake.

NO CONVERSION BETWEEN INT AND BOOL Program.cs

```
int count = 3;
if (count) Console.WriteLine("non-zero");
if (count = 5) Console.WriteLine("five");
```

COMPILER OUTPUT

```
Program.cs(2,5): error CS0029: Cannot implicitly convert type 'int' to 'bool'
Program.cs(3,5): error CS0029: Cannot implicitly convert type 'int' to 'bool'
```

3.4.5 Ranges with MinValue and MaxValue

THE RANGE OF EVERY NUMERIC TYPE Program.cs

```
Console.WriteLine($"sbyte  : {sbyte.MinValue} to {sbyte.MaxValue}");
Console.WriteLine($"byte   : {byte.MinValue} to {byte.MaxValue}");
Console.WriteLine($"short  : {short.MinValue} to {short.MaxValue}");
Console.WriteLine($"ushort : {ushort.MinValue} to {ushort.MaxValue}");
Console.WriteLine($"int    : {int.MinValue} to {int.MaxValue}");
Console.WriteLine($"uint   : {uint.MinValue} to {uint.MaxValue}");
Console.WriteLine($"long   : {long.MinValue} to {long.MaxValue}");
Console.WriteLine($"ulong  : {ulong.MinValue} to {ulong.MaxValue}");
Console.WriteLine($"float  : {float.MinValue} to {float.MaxValue}");
Console.WriteLine($"double : {double.MinValue} to {double.MaxValue}");
Console.WriteLine($"decimal: {decimal.MinValue} to {decimal.MaxValue}");
Console.WriteLine($"char   : {(int)char.MinValue} to {(int)char.MaxValue}");
```

OUTPUT

```
sbyte  : -128 to 127
byte   : 0 to 255
short  : -32768 to 32767
ushort : 0 to 65535
int    : -2147483648 to 2147483647
uint   : 0 to 4294967295
long   : -9223372036854775808 to 9223372036854775807
ulong  : 0 to 18446744073709551615
float  : -3.4028235E+38 to 3.4028235E+38
double : -1.7976931348623157E+308 to 1.7976931348623157E+308
decimal: -79228162514264337593543950335 to 79228162514264337593543950335
char   : 0 to 65535
```

Floating-point types also have special values: `double.PositiveInfinity`, `double.NegativeInfinity` and `double.NaN` ("not a number"). Dividing a non-zero double by zero gives infinity rather than an exception; `0.0 / 0.0` gives NaN; integer division by zero throws `DivideByZeroException`.

INFINITY AND NAN

Program.cs

```
Console.WriteLine(1.0 / 0);              // double division by zero
Console.WriteLine(-1.0 / 0);
Console.WriteLine(0.0 / 0);
double nan = double.NaN;
Console.WriteLine($"NaN == NaN? {nan == nan}, double.IsNaN: {double.IsNaN(nan)}");
Console.WriteLine($"smallest positive double: {double.Epsilon}");
```

OUTPUT

```
∞
-∞
NaN
NaN == NaN? False, double.IsNaN: True
smallest positive double: 5E-324
```

NOTE

Since .NET Core 3.0, infinity prints as the symbol ∞ (the exact text comes from the culture; some consoles show `Infinity`).

3.4.6 C# keywords vs .NET type names (int = System.Int32)

Each built-in keyword is just an alias for a .NET type in the `System` namespace: `int` is exactly `System.Int32`, `string` is `System.String`, and so on. The two spellings are interchangeable. Convention: use the keyword for declarations (`int count`) and either form for static members (`int.Parse` or `Int32.Parse`).

Keyword	.NET type	Keyword	.NET type
`bool`	System.Boolean	`float`	System.Single
`byte`	System.Byte	`double`	System.Double
`char`	System.Char	`decimal`	System.Decimal
`short`	System.Int16	`string`	System.String
`int`	System.Int32	`object`	System.Object
`long`	System.Int64	`nint`	System.IntPtr

ALIASES ARE THE SAME TYPE

Program.cs

```
int a = 5;
System.Int32 b = 5;
Console.WriteLine($"{typeof(int) == typeof(Int32)} {a.GetType().FullName} {b.GetType().FullName}");
```

OUTPUT

```
True System.Int32 System.Int32
```

3.5 Built-in reference types

3.5.1 string

`string` (System.String) is a sequence of UTF-16 characters. It is a reference type, but it behaves like a value in two ways: strings are **immutable** (every "change" creates a new string), and `==` compares their **contents**, not their references. Module 8 covers strings in depth.

STRING COMPARES BY CONTENT AND IS IMMUTABLE Program.cs

```
string s1 = "hello";
string s2 = "HELLO".ToLower();       // a different object with the same characters
Console.WriteLine($"s1 == s2: {s1 == s2}");
Console.WriteLine($"same object: {object.ReferenceEquals(s1, s2)}");
string s3 = s1;
s1 += " world";                      // creates a NEW string; s3 still refers to the old one
Console.WriteLine($"s1 = {s1}, s3 = {s3}");
```

OUTPUT

```
s1 == s2: True
same object: False
s1 = hello world, s3 = hello
```

3.5.2 object

A variable of type `object` can hold anything. To use the value as its real type again you must cast it back, which is checked at run time. Before generics, collections stored `object`; today `object` is used mainly in general-purpose APIs (such as `Console.WriteLine(object)`) and with pattern matching.

OBJECT HOLDS ANYTHING Program.cs

```
object o = 42;
// int bad = o + 1;                  // error CS0019: Operator '+' cannot be applied to 'object' and
'int'
int n = (int)o + 1;                  // cast back to int
Console.WriteLine(n);
o = "now a string";
if (o is string text)                // safe check-and-cast with a pattern
    Console.WriteLine($"{text.Length} characters");
```

OUTPUT

```
43
12 characters
```

3.5.3 dynamic (overview)

`dynamic` turns off compile-time type checking for a variable: member access and operations are resolved at **run time** by the Dynamic Language Runtime. Mistakes therefore show up as run-time exceptions instead of compile errors. It is useful for COM interop (Office automation), dynamic languages and some JSON scenarios, but should be rare in ordinary code (Module 19).

DYNAMIC DEFERS CHECKING TO RUN TIME Program.cs

```
dynamic d = 10;
Console.WriteLine(d + 5);            // resolved at run time: int + int
d = "ten";
Console.WriteLine(d.ToUpper());      // now string.ToUpper
d = 3.5;
Console.WriteLine(d.ToUpper());      // compiles, but fails at run time
```

OUTPUT

```
15
TEN
Unhandled exception. Microsoft.CSharp.RuntimeBinder.RuntimeBinderException: 'double' does not contain a
definition for 'ToUpper'
   at CallSite.Target(Closure, CallSite, Object)
   at Program.<Main>$(String[] args) in /home/asha/MyApp/Program.cs:line 6
```

3.6 Literals

A **literal** is a value written directly in the code. Its form decides its type.

3.6.1 Integer literals: decimal, hexadecimal 0x, binary 0b

THE SAME NUMBER IN THREE NOTATIONS

Program.cs

```
int dec = 255;
int hex = 0xFF;            // hexadecimal: 0x prefix
int bin = 0b1111_1111;     // binary: 0b prefix (C# 7)
Console.WriteLine($"{dec} {hex} {bin}");
Console.WriteLine($"{255:X} {255:X4} {Convert.ToString(255, 2)} {Convert.ToString(255, 8)}");
```

OUTPUT

```
255 255 255
FF 00FF 11111111 377
```

3.6.2 Digit separators 1_000_000

Underscores can be placed anywhere between digits to make long literals readable (C# 7). They are ignored by the compiler. Indian-style grouping works too: `1_00_00_000` is one crore.

DIGIT SEPARATORS

Program.cs

```
long crore = 1_00_00_000;
int million = 1_000_000;
int mask = 0b1010_0101;
double avogadro = 6.022_140_76e23;
Console.WriteLine($"{crore} {million} {mask} {avogadro}");
```

OUTPUT

```
10000000 1000000 165 6.02214076E+23
```

3.6.3 Suffixes: L, U, F, D, M

An integer literal is `int` if it fits, otherwise `uint`, `long` or `ulong`. A literal with a decimal point or exponent is `double`. Suffixes force another type (upper or lower case; prefer upper-case `L`, because `l` looks like `1`).

Literal	Type	Literal	Type
`42`	int	`42L`	long
`42U`	uint	`42UL`	ulong
`4.2`	double	`4.2D`	double
`4.2F`	float	`4.2M`	decimal
`3_000_000_000`	uint (too big for int)	`1e3`	double (1000)

A DOUBLE LITERAL DOES NOT CONVERT IMPLICITLY TO FLOAT OR DECIMAL

Program.cs

```
float f = 3.14;
decimal price = 99.99;
```

COMPILER OUTPUT

```
Program.cs(1,11): error CS0664: Literal of type double cannot be implicitly converted to type 'float';
use an 'F' suffix to create a literal of this type
Program.cs(2,17): error CS0664: Literal of type double cannot be implicitly converted to type 'decimal';
use an 'M' suffix to create a literal of this type
```

3.6.4 Character, string, verbatim @"..." and raw string literals """..."""

Form	Example	Notes
char	`'A'`, `'\n'`, `'\u0041'`	exactly one character
regular string	`"Line1\nLine2"`	escape sequences: `\n \t \\ \" \0 \u20B9`
verbatim string	`@"C:\temp\new"`	backslashes are literal; may span lines; `""` for a quote
interpolated string	`$"Total: {sum}"`	expressions in braces (5.1.2)
raw string (C# 11)	`"""He said "hi" """`	no escaping at all; multi-line; indentation of the closing quotes is removed
UTF-8 string (C# 11)	`"abc"u8`	a `ReadOnlySpan<byte>` of UTF-8 bytes

STRING LITERAL FORMS

Program.cs

```
string regular = "C:\\temp\\new.txt\t<- tab";
string verbatim = @"C:\temp\new.txt";
string quote = @"She said ""Namaste""";
string json = """
    {
      "name": "Asha",
      "path": "C:\data"
    }
    """;
Console.WriteLine(regular);
Console.WriteLine(verbatim);
Console.WriteLine(quote);
Console.WriteLine(json);
```

OUTPUT

```
C:\temp\new.txt <- tab
C:\temp\new.txt
She said "Namaste"
{
  "name": "Asha",
  "path": "C:\data"
}
```

NOTE

In the raw string literal, the four spaces before the closing `"""` are removed from every line, and the backslash in `C:\data` needs no escaping.

3.7 Constants

3.7.1 const

A `const` is a value fixed at **compile time**; the compiler substitutes the value wherever the constant is used. Constants must be initialised with a constant expression and can only be of built-in types (numbers, `bool`, `char`, `string`), enums, or `null`. They are implicitly static when declared in a class. Use them for values that truly never change: mathematical constants, fixed limits.

CONST VALUES Program.cs

```
const double Pi = 3.14159;
const int DaysInWeek = 7;
const string AppName = "LBF Portal";
const int HoursInWeek = DaysInWeek * 24;   // constant expressions are allowed

double radius = 2.5;
Console.WriteLine($"{AppName}: area = {Pi * radius * radius}, hours per week = {HoursInWeek}");
// DaysInWeek = 8;                         // error CS0131: The left-hand side of an assignment must be a variable
```

OUTPUT

```
LBF Portal: area = 19.6349375, hours per week = 168
```

3.7.2 readonly (introduced; detailed in Module 10)

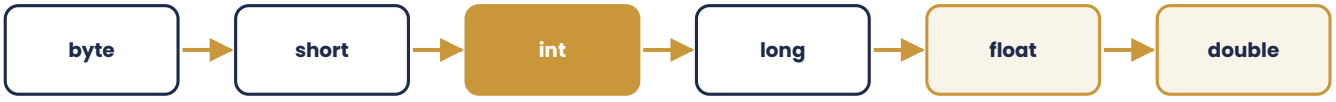
A `readonly` field can be assigned only in its declaration or in a constructor, and is fixed afterwards. Unlike `const`, its value is computed at **run time**, so it can be any type and can differ per object or per run (a creation timestamp, a configuration value).

	`const`	`readonly`
Value fixed	at compile time	at run time (declaration or constructor)
Allowed types	built-in types, enums, string, null	any type
Applies to	locals and fields	fields only
Per object?	no (implicitly static)	yes, unless declared static
Example	`const int MaxSeats = 60;`	`readonly DateTime _created = DateTime.Now;`

3.8 Type conversion

3.8.1 Implicit conversion (widening)

A conversion that can never lose information or fail happens automatically: smaller integer types to larger ones, integers to floating point, `char` to `int`, `float` to `double`. (Converting a large `long` to `double` may lose low-order digits but never the magnitude, so it is still implicit.)



IMPLICIT (WIDENING) CONVERSIONS GO LEFT TO RIGHT; DECIMAL ACCEPTS ALL INTEGER TYPES

WIDENING CONVERSIONS NEED NO CAST

Program.cs

```
byte b = 200;
int i = b;                // byte -> int
long l = i;               // int -> long
double d = l;             // long -> double
char c = 'Z';
int code = c;             // char -> int
decimal money = i;        // int -> decimal
Console.WriteLine($"{i} {l} {d} {code} {money}");
```

OUTPUT

```
200 200 200 90 200
```

3.8.2 Explicit casting (narrowing)

A conversion that might lose data needs an explicit **cast** `(type)value`: you tell the compiler you accept the risk. Casting a floating-point number to an integer **truncates** towards zero (it does not round). Casting an integer to a smaller type keeps only the low-order bits.

NARROWING CASTS

Program.cs

```
double price = 99.99;
int whole = (int)price;               // fractional part discarded
int negative = (int)-7.9;             // truncates towards zero
long big = 3_000_000_000;
int wrapped = (int)big;               // too big: only the low 32 bits remain
int n = 300;
byte small = (byte)n;                 // 300 - 256 = 44
Console.WriteLine($"{whole} {negative} {wrapped} {small}");
Console.WriteLine($"rounded instead: {(int)Math.Round(price)}");
```

OUTPUT

```
99 -7 -1294967296 44
rounded instead: 100
```

3.8.3 checked and unchecked overflow behaviour

By default integer arithmetic and casts are **unchecked**: on overflow the value silently wraps around. In a `checked` block or expression, overflow throws `System.OverflowException` instead. You can turn checking on for a whole project with `<CheckForOverflowUnderflow>true</CheckForOverflowUnderflow>`. Constant expressions that overflow are compile errors even without `checked`.

OVERFLOW: SILENT BY DEFAULT, AN EXCEPTION WHEN CHECKED

Program.cs

```
int max = int.MaxValue;
int wrapped = max + 1;                          // unchecked by default: wraps around
Console.WriteLine($"int.MaxValue + 1 = {wrapped}");

try
{
    int safe = checked(max + 1);                // checked expression
    Console.WriteLine(safe);
}
catch (OverflowException e)
{
    Console.WriteLine($"checked: {e.Message}");
}

int n = 300;
checked
{
    try { byte b = (byte)n; }                   // a checked cast
    catch (OverflowException) { Console.WriteLine("checked cast to byte failed"); }
}
Console.WriteLine($"unchecked cast: {unchecked((byte)n)}");
```

OUTPUT

```
int.MaxValue + 1 = -2147483648
checked: Arithmetic operation resulted in an overflow.
checked cast to byte failed
unchecked cast: 44
```

NOTE

The cast converts the variable `n`: casting the constant `(byte)300` directly inside `checked` would be rejected at compile time (error CS0221), because the compiler evaluates constant expressions itself.

3.8.4 Convert class

`System.Convert` converts between base types: `Convert.ToInt32`, `ToDouble`, `ToDecimal`, `ToBoolean`, `ToString`, `ToChar`. Unlike a cast, converting a floating-point number **rounds** to the nearest integer, using **banker's rounding** (a value exactly halfway goes to the even neighbour: 2.5 becomes 2, 3.5 becomes 4). `Convert.ToInt32(null)` returns 0, and out-of-range values throw `OverflowException`.

CONVERT ROUNDS AND PARSES

Program.cs

```
Console.WriteLine(Convert.ToInt32(3.7));        // rounds
Console.WriteLine(Convert.ToInt32(2.5));        // halfway: to the even number
Console.WriteLine(Convert.ToInt32(3.5));
Console.WriteLine((int)3.7);                     // a cast truncates
Console.WriteLine(Convert.ToInt32("123") + 1);   // parse a string
Console.WriteLine(Convert.ToBoolean("true"));
Console.WriteLine(Convert.ToString(42, 2));      // to binary text
Console.WriteLine(Convert.ToInt32("ff", 16));    // from hexadecimal text
Console.WriteLine(Convert.ToInt32(null));
```

OUTPUT

```
4
2
4
3
124
True
101010
255
0
```

3.8.5 Parse() and TryParse()

Every numeric type (and `bool`, `DateTime`, `Guid` …) has a static `Parse` method that converts a string, throwing `FormatException` if the text is not valid and `OverflowException` if it is out of range. `TryParse` returns `true` or `false` instead of throwing, and delivers the value through an `out` parameter. Use `TryParse` for anything a user typed.

PARSE THROWS, TRYPARSE REPORTS

Program.cs

```
int a = int.Parse("250");
double b = double.Parse("3.75");
bool c = bool.Parse("False");
Console.WriteLine($"{a + 1} {b * 2} {c}");

string[] inputs = { "42", "4x2", "", "99999999999", " 7 " };
foreach (string input in inputs)
{
    if (int.TryParse(input, out int value))
        Console.WriteLine($"\"{input}\" -> {value}");
    else
        Console.WriteLine($"\"{input}\" is not a valid int");
}

try
{
    int bad = int.Parse("abc");
}
catch (FormatException e)
{
    Console.WriteLine(e.Message);
}
```

OUTPUT

```
251 7.5 False
"42" -> 42
"4x2" is not a valid int
"" is not a valid int
"99999999999" is not a valid int
" 7 " -> 7
The input string 'abc' was not in a correct format.
```

NOTE

Leading and trailing spaces are allowed by default, so `" 7 "` parses. Parsing `"3.75"` uses the current culture; in cultures with a decimal comma (such as German) pass `CultureInfo.InvariantCulture` (Module 5).

3.8.6 Boxing and unboxing

Boxing converts a value type to `object` (or to an interface it implements): the runtime allocates an object on the heap and copies the value into it. **Unboxing** extracts the value back with an explicit cast, which must name exactly the original type. Boxing is implicit and therefore easy to miss; it costs an allocation each time, which is why generic collections such as `List<int>` (no boxing) replaced `ArrayList` (Module 14).



int m = (int)o; unboxing copies 42 back out

BOXING COPIES A VALUE INTO A HEAP OBJECT

BOXING AND UNBOXING Program.cs

```
int n = 42;
object boxed = n;                 // boxing: a copy of 42 is put in a heap object
n = 100;                          // changing n does not affect the box
int unboxed = (int)boxed;         // unboxing: explicit cast to the exact type
Console.WriteLine($"n = {n}, boxed = {boxed}, unboxed = {unboxed}");

try
{
    long wrong = (long)boxed;     // the box holds an int, not a long
}
catch (InvalidCastException e)
{
    Console.WriteLine(e.Message);
}
long right = (long)(int)boxed;    // unbox to int first, then convert
Console.WriteLine(right);
```

OUTPUT

```
n = 100, boxed = 42, unboxed = 42
Unable to cast object of type 'System.Int32' to type 'System.Int64'.
42
```

3.9 Nullable value types

3.9.1 int? and Nullable<T>

Value types always hold a value, but sometimes "no value" is meaningful: a database column that allows NULL, an optional rating, a test not yet taken. `int?` (shorthand for `Nullable<int>`) is a value type that can hold any `int` or `null`. Arithmetic with a null operand gives null ("lifted" operators), and comparisons with null are false.

NULLABLE VALUE TYPES Program.cs

```
int? rating = null;               // no rating yet
Nullable<double> discount = 12.5; // the long form
int? doubled = rating * 2;        // null propagates
Console.WriteLine($"rating: '{rating}', doubled: '{doubled}', discount: {discount}");
rating = 4;
doubled = rating * 2;
Console.WriteLine($"rating: {rating}, doubled: {doubled}");
Console.WriteLine($"rating > 3: {rating > 3}, null > 3: {(int?)null > 3}");
// int plain = rating;            // error CS0266: Cannot implicitly convert type 'int?' to 'int'
int plain = rating ?? 0;          // ?? supplies a value for null (Module 4)
Console.WriteLine(plain);
```

OUTPUT

```
rating: '', doubled: '', discount: 12.5
rating: 4, doubled: 8
rating > 3: True, null > 3: False
4
```

3.9.2 HasValue, Value, GetValueOrDefault()

WORKING WITH A NULLABLE VALUE

Program.cs

```
int? marks = null;
Console.WriteLine($"HasValue: {marks.HasValue}, GetValueOrDefault(): {marks.GetValueOrDefault()},
GetValueOrDefault(35): {marks.GetValueOrDefault(35)}");
try
{
    Console.WriteLine(marks.Value);     // no value: throws
}
catch (InvalidOperationException e)
{
    Console.WriteLine(e.Message);
}
marks = 88;
if (marks.HasValue)
    Console.WriteLine($"Value: {marks.Value}");
if (marks is int m)                     // pattern: test and extract in one step
    Console.WriteLine($"pattern: {m}");
```

OUTPUT

```
HasValue: False, GetValueOrDefault(): 0, GetValueOrDefault(35): 35
Nullable object must have a value.
Value: 88
pattern: 88
```

3.10 Default values

3.10.1 default keyword and default values by type

Fields and array elements that are not explicitly initialised get their type's **default value**: zero for numbers, `false` for `bool`, `'\0'` for `char`, `null` for reference types and nullable types, and for a struct, every field set to its default. The `default` expression produces this value: `default(int)`, or simply `default` when the type is known from context.

DEFAULT VALUES BY TYPE

Program.cs

```
Console.WriteLine($"int: {default(int)}");
Console.WriteLine($"double: {default(double)}");
Console.WriteLine($"decimal: {default(decimal)}");
Console.WriteLine($"bool: {default(bool)}");
Console.WriteLine($"char code: {(int)default(char)}");
Console.WriteLine($"string is null: {default(string) is null}");
Console.WriteLine($"int? is null: {default(int?) is null}");
Console.WriteLine($"DateTime: {default(DateTime):yyyy-MM-dd}");

int[] counts = new int[3];        // elements get default values
bool[] flags = new bool[2];
string[] names = new string[2];
Console.WriteLine($"{counts[0]} {flags[1]} {names[0] ?? "null"}");
DateTime when = default;          // target-typed default
Console.WriteLine(when.Year);
```

OUTPUT

```
int: 0
double: 0
decimal: 0
bool: False
char code: 0
string is null: True
int? is null: True
DateTime: 0001-01-01
0 False null
1
```

Variables have a fixed type; var lets the compiler infer it. Locals must be definitely assigned; fields and array elements start at default values. Value types (numbers, bool, char, structs, enums) are copied on assignment; reference types (string, arrays, classes) share one object. Use int, long, double for general numbers and decimal for money. Widening conversions are implicit; narrowing needs a cast (which truncates), Convert rounds, and TryParse safely parses user input. Boxing copies a value into a heap object. T? adds null to a value type.

PRACTICE CHECKPOINT

Print the **min and max of every numeric type**, show the difference between **double and decimal for 0.1 + 0.2**, and demonstrate **boxing and unboxing**.

SAMPLE SOLUTION

Program.cs

```
Console.WriteLine("--- Ranges ---");
Console.WriteLine($"{"byte",-8}{byte.MinValue,30}{byte.MaxValue,32}");
Console.WriteLine($"{"short",-8}{short.MinValue,30}{short.MaxValue,32}");
Console.WriteLine($"{"int",-8}{int.MinValue,30}{int.MaxValue,32}");
Console.WriteLine($"{"long",-8}{long.MinValue,30}{long.MaxValue,32}");
Console.WriteLine($"{"float",-8}{float.MinValue,30}{float.MaxValue,32}");
Console.WriteLine($"{"double",-8}{double.MinValue,30}{double.MaxValue,32}");
Console.WriteLine($"{"decimal",-8}{decimal.MinValue,30}{decimal.MaxValue,32}");

Console.WriteLine("--- 0.1 + 0.2 ---");
double d = 0.1 + 0.2;
decimal m = 0.1m + 0.2m;
Console.WriteLine($"double : {d}  equals 0.3? {d == 0.3}");
Console.WriteLine($"decimal: {m}  equals 0.3? {m == 0.3m}");

Console.WriteLine("--- Boxing ---");
int value = 25;
object box = value;             // boxing
value++;                        // the box keeps its own copy
int back = (int)box;            // unboxing
Console.WriteLine($"value = {value}, box = {box}, back = {back}, box type = {box.GetType().Name}");
```

OUTPUT

```
--- Ranges ---
byte                                   0                             255
short                              -32768                           32767
int                           -2147483648                      2147483647
long                 -9223372036854775808              9223372036854775807
float                      -3.4028235E+38                    3.4028235E+38
double          -1.7976931348623157E+308          1.7976931348623157E+308
decimal -79228162514264337593543950335   79228162514264337593543950335
--- 0.1 + 0.2 ---
double : 0.30000000000000004  equals 0.3? False
decimal: 0.3  equals 0.3? True
--- Boxing ---
value = 26, box = 25, back = 25, box type = Int32
```

MODULE 04 · BEGINNER

Operators and expressions

Minimum duration **1 day** | Level **Beginner**

Operators combine values into expressions: arithmetic, comparisons, logic, bit manipulation and assignment. Beyond the usual C-family set, C# adds operators that make null handling safe and concise (??, ??=, ?. and !), plus is, as, nameof, ranges and indices. The module ends with precedence rules and the Math and Random classes.

IN THIS MODULE

4.1 Arithmetic operators
4.2 Relational and equality operators
4.3 Logical operators
4.4 Bitwise and shift operators
4.5 Assignment operators
4.6 Null operators
4.7 Other operators
4.8 Precedence and associativity
4.9 The Math class

4.1 Arithmetic operators

4.1.1 + - * / %

The arithmetic operators work on all numeric types. `%` is the remainder after division; its result takes the sign of the left operand. `+` also joins strings: if either operand is a string, the other is converted to a string.

Operator	Meaning	Example	Result
`+`	addition / string concatenation	`7 + 3`, `"Rs " + 50`	10, "Rs 50"
`-`	subtraction, unary minus	`7 - 3`, `-x`	4
`*`	multiplication	`7 * 3`	21
`/`	division	`7 / 2`, `7.0 / 2`	3, 3.5
`%`	remainder (modulus)	`7 % 3`, `-7 % 3`	1, -1

ARITHMETIC IN ACTION Program.cs

```
int a = 17, b = 5;
Console.WriteLine($"{a} + {b} = {a + b}");
Console.WriteLine($"{a} - {b} = {a - b}");
Console.WriteLine($"{a} * {b} = {a * b}");
Console.WriteLine($"{a} / {b} = {a / b}");
Console.WriteLine($"{a} % {b} = {a % b}");
Console.WriteLine($"-{a} % {b} = {-a % b}");
Console.WriteLine($"7.5 % 2 = {7.5 % 2}");
Console.WriteLine("Total: " + a + b);        // left to right: string + 17, then + 5
Console.WriteLine("Total: " + (a + b));      // parentheses first
```

OUTPUT

```
17 + 5 = 22
17 - 5 = 12
17 * 5 = 85
17 / 5 = 3
17 % 5 = 2
-17 % 5 = -2
7.5 % 2 = 1.5
Total: 175
Total: 22
```

4.1.2 Integer vs floating-point division

When both operands of `/` are integers, the result is an integer and the fractional part is **discarded** (truncated towards zero). If at least one operand is floating point, the division is floating point. Integer division by zero throws `DivideByZeroException`; floating-point division by zero gives infinity or NaN (3.4.5).

INTEGER DIVISION AND ITS USES Program.cs

```
int total = 7, count = 2;
Console.WriteLine(total / count);              // 3: integer division
Console.WriteLine((double)total / count);      // 3.5: cast one operand first
Console.WriteLine((double)(total / count));    // 3: too late, the division already happened
Console.WriteLine(total / 2.0);                // 3.5
Console.WriteLine(-7 / 2);                     // -3: truncates towards zero

int minutes = 135;
Console.WriteLine($"{minutes} minutes = {minutes / 60} h {minutes % 60} min");
```

OUTPUT

```
3
3.5
3
3.5
-3
135 minutes = 2 h 15 min
```

TIP

Integer division and remainder together split a quantity into units: hours and minutes, rupees and paise (`paise / 100`, `paise % 100`), the digits of a number (`n % 10`, `n / 10`).

4.1.3 Increment and decrement (prefix and postfix)

`++` adds 1 and `--` subtracts 1. In **prefix** form (`++x`) the variable is changed first and the new value is used; in **postfix** form (`x++`) the old value is used and the variable is changed afterwards. As a separate statement both forms do the same thing; the difference only matters inside a larger expression.

PREFIX VS POSTFIX Program.cs

```
int x = 5;
int a = x++;        // a gets 5, then x becomes 6
int b = ++x;        // x becomes 7, then b gets 7
Console.WriteLine($"a = {a}, b = {b}, x = {x}");

int y = 10;
Console.WriteLine(y--);   // prints 10, then y becomes 9
Console.WriteLine(y);     // 9
Console.WriteLine(--y);   // y becomes 8, prints 8

int i = 1;
int result = i++ + ++i;   // 1 + 3 (C# evaluates strictly left to right)
Console.WriteLine($"result = {result}, i = {i}");
```

OUTPUT

```
a = 5, b = 7, x = 7
10
9
8
result = 4, i = 3
```

NOTE

Unlike C and C++, C# defines the order of evaluation: operands are always evaluated left to right, so `i++ + ++i` has a well-defined result. It is still hard to read; write such code as separate statements.

4.2 Relational and equality operators

4.2.1 == != > < >= <=

Relational operators compare two values and produce a `bool`. `==` is equality (not the assignment `=`), `!=` is "not equal".

COMPARISONS PRODUCE BOOL VALUES Program.cs

```
int marks = 72;
Console.WriteLine($"marks == 72 : {marks == 72}");
Console.WriteLine($"marks != 72 : {marks != 72}");
Console.WriteLine($"marks > 75  : {marks > 75}");
Console.WriteLine($"marks >= 40 : {marks >= 40}");
Console.WriteLine($"'a' < 'b'   : {'a' < 'b'}");
bool passed = marks >= 40;                    // a comparison result stored in a bool
Console.WriteLine($"passed = {passed}");
```

OUTPUT

```
marks == 72 : True
marks != 72 : False
marks > 75  : False
marks >= 40 : True
'a' < 'b'   : True
passed = True
```

4.2.2 Value equality vs reference equality

For value types, `==` compares the values. For reference types, `==` by default compares **references**: two variables are equal only if they point to the **same object**. Some types redefine `==` to compare contents: `string` and **records** do, ordinary classes do not. `Equals()` follows the same pattern, and `ReferenceEquals` always compares references.

REFERENCE EQUALITY FOR CLASSES, VALUE EQUALITY FOR RECORDS AND STRINGS

Program.cs

```
var p1 = new PointClass(1, 2);
var p2 = new PointClass(1, 2);
var p3 = p1;
Console.WriteLine($"class : p1 == p2 {p1 == p2}, p1 == p3 {p1 == p3}");

var r1 = new PointRecord(1, 2);
var r2 = new PointRecord(1, 2);
Console.WriteLine($"record: r1 == r2 {r1 == r2}");

string s1 = "hi", s2 = new string(new[] { 'h', 'i' });
Console.WriteLine($"string: s1 == s2 {s1 == s2}, same object {ReferenceEquals(s1, s2)}");

class PointClass { public int X, Y; public PointClass(int x, int y) { X = x; Y = y; } }
record PointRecord(int X, int Y);
```

OUTPUT

```
class : p1 == p2 False, p1 == p3 True
record: r1 == r2 True
string: s1 == s2 True, same object False
```

4.3 Logical operators

4.3.1 &&, ||, ! and short-circuiting

`&&` (AND) is true when both operands are true; `||` (OR) when at least one is; `!` (NOT) inverts a bool. `&&` and `||` are **short-circuiting**: the right operand is evaluated only if it can still change the result. This allows guards such as `s != null && s.Length > 0`: if `s` is null, `s.Length` is never touched.

a	b	a && b	a || b	!a
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

LOGICAL OPERATORS AND SHORT-CIRCUITING

Program.cs

```
int age = 22;
bool hasLicence = true;
Console.WriteLine($"can drive: {age >= 18 && hasLicence}");
Console.WriteLine($"discount : {age < 12 || age >= 60}");
Console.WriteLine($"not adult: {!(age >= 18)}");

string? name = null;
if (name != null && name.Length > 3)          // safe: Length is not evaluated when name is null
    Console.WriteLine("long name");
else
    Console.WriteLine("no name or short name");

static bool Check(string label, bool value) { Console.Write($"[{label}] "); return value; }
Console.WriteLine(Check("A", false) && Check("B", true));   // B is never called
Console.WriteLine(Check("C", true) || Check("D", true));    // D is never called
```

OUTPUT

```
can drive: True
discount : False
not adult: False
no name or short name
[A] False
[C] True
```

4.3.2 Non-short-circuit &, |, ^ on bools

On `bool` operands, `&` and `|` compute AND and OR **without** short-circuiting: both sides are always evaluated (useful only when the right side has a side effect that must happen). `^` is exclusive OR: true when exactly one operand is true.

NON-SHORT-CIRCUIT LOGIC AND XOR

Program.cs

```
static bool Log(string s, bool v) { Console.Write($"{s} "); return v; }
Console.WriteLine(Log("left", false) & Log("right", true));     // both evaluated
Console.WriteLine(Log("left", true) | Log("right", false));
Console.WriteLine($"true ^ false = {true ^ false}, true ^ true = {true ^ true}");
```

OUTPUT

```
left right False
left right True
true ^ false = True, true ^ true = False
```

4.4 Bitwise and shift operators

4.4.1 & | ^ ~ << >> >>>

On integers, the bitwise operators work on each bit separately. Shifts move all bits left or right: `x << n` multiplies by 2^n, `x >> n` divides by 2^n keeping the sign (arithmetic shift), and `x >>> n` (C# 11) shifts in zeros (logical shift). They are used for flags, permissions, colours, hashing and fast arithmetic.

Operator	Name	12 op 10 (1100 op 1010)	Result
`&`	AND	1100 & 1010	1000 = 8
`|`	OR	1100 | 1010	1110 = 14
`^`	XOR	1100 ^ 1010	0110 = 6

Operator	Name	12 op 10 (1100 op 1010)	Result
`~`	NOT (complement)	~12	-13
`<<`	left shift	12 << 2	48
`>>`	right shift	12 >> 2	3

BITWISE OPERATORS Program.cs

```
int a = 12, b = 10;
Console.WriteLine($"a & b = {a & b}, a | b = {a | b}, a ^ b = {a ^ b}, ~a = {~a}");
Console.WriteLine($"a << 2 = {a << 2}, a >> 2 = {a >> 2}");
Console.WriteLine($"-16 >> 2 = {-16 >> 2}, -16 >>> 28 = {-16 >>> 28}");
Console.WriteLine($"binary of 12: {Convert.ToString(12, 2).PadLeft(8, '0')}");

for (int n = 5; n <= 8; n++)
    Console.WriteLine($"{n} is {((n & 1) == 0 ? "even" : "odd")}");     // the lowest bit decides

int x = 7, y = 3;
x ^= y; y ^= x; x ^= y;                     // swap without a temporary (a classic trick)
Console.WriteLine($"x = {x}, y = {y}");
```

OUTPUT

```
a & b = 8, a | b = 14, a ^ b = 6, ~a = -13
a << 2 = 48, a >> 2 = 3
-16 >> 2 = -4, -16 >>> 28 = 15
binary of 12: 00001100
5 is odd
6 is even
7 is odd
8 is even
x = 3, y = 7
```

NOTE

`-16` is `11111111 11111111 11111111 11110000` in 32-bit two's complement. `>> 2` copies the sign bit (giving -4); `>>> 28` fills with zeros and leaves the top four 1-bits: `1111` = 15.

4.5 Assignment operators

4.5.1 = and compound assignment

`=` stores the value of the right side in the variable on the left, and is itself an expression whose value is the assigned value (so `a = b = 0` sets both). Compound assignment operators combine an operation with assignment: `x += 5` means `x = x + 5`. They include an implicit cast back to the variable's type, which is why `byte b = 10; b += 5;` compiles even though `b = b + 5` (an int) would not.

Operator	Same as	Operator	Same as
`x += y`	x = x + y	`x &= y`	x = x & y
`x -= y`	x = x - y	`x |= y`	x = x | y
`x *= y`	x = x * y	`x ^= y`	x = x ^ y
`x /= y`	x = x / y	`x <<= y`	x = x << y
`x %= y`	x = x % y	`x ??= y`	assign only if x is null (4.6.2)

COMPOUND ASSIGNMENT

Program.cs

```
int balance = 1000;
balance += 500;    Console.WriteLine($"deposit 500   -> {balance}");
balance -= 200;    Console.WriteLine($"withdraw 200  -> {balance}");
balance *= 2;      Console.WriteLine($"doubled       -> {balance}");
balance /= 3;      Console.WriteLine($"split by 3    -> {balance}");
balance %= 100;    Console.WriteLine($"remainder 100 -> {balance}");

string msg = "Hello";
msg += ", World";
Console.WriteLine(msg);

byte b = 250;
b += 10;           // implicit cast back to byte: wraps to 4
Console.WriteLine(b);
int p, q;
p = q = 7;         // assignment is right-associative
Console.WriteLine($"{p} {q}");
```

OUTPUT

```
deposit 500   -> 1500
withdraw 200  -> 1300
doubled       -> 2600
split by 3    -> 866
remainder 100 -> 66
Hello, World
4
7 7
```

4.6 Null operators

Null references are a major source of bugs (`NullReferenceException`). C# provides four operators that make null handling short and safe.

4.6.1 Null-coalescing ??

`a ?? b` returns `a` if it is not null, otherwise `b`. It provides a default value in one expression and works with nullable value types and reference types. It can be chained: `a ?? b ?? c`.

DEFAULT VALUES WITH ??

Program.cs

```
string? nickname = null;
string? fullName = "Asha Rao";
Console.WriteLine($"Hello, {nickname ?? "guest"}!");
Console.WriteLine($"Display: {nickname ?? fullName ?? "unknown"}");
int? score = null;
int shown = score ?? 0;
Console.WriteLine($"score: {shown}");
```

OUTPUT

```
Hello, guest!
Display: Asha Rao
score: 0
```

4.6.2 Null-coalescing assignment ??=

`x ??= value` assigns `value` to `x` only if `x` is currently null. It is ideal for lazy initialisation.

LAZY INITIALISATION WITH ??= Program.cs

```
List<string>? log = null;
log ??= new List<string>();          // created because log was null
log.Add("first");
log ??= new List<string>();          // not replaced: log already exists
log.Add("second");
Console.WriteLine($"{log.Count} entries");

string? city = "Pune";
city ??= "Mysuru";
Console.WriteLine(city);
```

OUTPUT

```
2 entries
Pune
```

4.6.3 Null-conditional ?. and ?[]

`a?.Member` accesses a member only if `a` is not null; otherwise the whole expression is `null` (and the rest of the chain is skipped). `a?[i]` does the same for indexing. Combined with `??` it replaces nested if statements. Since the result can be null, a value-type result becomes nullable (`s?.Length` is an `int?`).

SAFE NAVIGATION WITH ?. Program.cs

```
Student? s1 = new Student { Name = "Ravi", Address = new Address { City = "Hubballi" } };
Student? s2 = new Student { Name = "Meera" };          // no address
Student? s3 = null;

Console.WriteLine(s1?.Address?.City ?? "no city");
Console.WriteLine(s2?.Address?.City ?? "no city");
Console.WriteLine(s3?.Address?.City ?? "no city");
int? len = s3?.Name.Length;                            // null: s3 is null
Console.WriteLine($"length: {len?.ToString() ?? "n/a"}");

int[]? marks = null;
Console.WriteLine(marks?[0] ?? -1);
Action? onSave = null;
onSave?.Invoke();                                      // safe: nothing happens

class Student { public string Name = ""; public Address? Address; }
class Address { public string City = ""; }
```

OUTPUT

```
Hubballi
no city
no city
length: n/a
-1
```

4.6.4 Null-forgiving !

With nullable reference types enabled (Module 12), the compiler warns when you dereference something that might be null. The postfix `!` operator tells the compiler "I know this is not null here", suppressing the warning. It has **no effect at run time**: if you are wrong, you still get a `NullReferenceException`. Use it rarely, when you know more than the compiler can prove.

THE NULL-FORGIVING OPERATOR

Program.cs

```
string? input = Console.ReadLine();       // string? : may be null at end of input
// int bad = input.Length;                 // warning CS8602: Dereference of a possibly null reference
int len = input!.Length;                   // "trust me": no warning, no run-time check
Console.WriteLine($"You typed {len} characters");
```

CONSOLE (user input in gold)

```
Learn Being Forward
You typed 19 characters
```

4.7 Other operators

4.7.1 Ternary ?:

`condition ? a : b` evaluates to `a` when the condition is true and to `b` otherwise. It is an expression, so it can be used inside other expressions and assignments. Both branches must have compatible types. Nested ternaries are possible but quickly become unreadable; prefer a switch expression for more than two cases.

THE CONDITIONAL OPERATOR

Program.cs

```
int marks = 67;
string result = marks >= 40 ? "Pass" : "Fail";
string grade = marks >= 75 ? "A" : marks >= 60 ? "B" : marks >= 40 ? "C" : "F";
Console.WriteLine($"{result}, grade {grade}");
int a = 8, b = 13;
Console.WriteLine($"max = {(a > b ? a : b)}, {a} is {(a % 2 == 0 ? "even" : "odd")}");
```

OUTPUT

```
Pass, grade B
max = 13, 8 is even
```

4.7.2 is and as

`x is T` tests whether a value is of type `T` (or derives from it); with a pattern, `x is T t` also gives you the converted value in a new variable. `x as T` converts a reference to type `T`, returning `null` instead of throwing if the conversion is not possible; it works only with reference and nullable types. `is null` and `is not null` are the clearest null checks.

TYPE TESTS WITH IS AND AS Program.cs

```
object[] items = { "hello", 42, 3.5, null!, new List<int> { 1, 2 } };
foreach (object o in items)
{
    if (o is string s) Console.WriteLine($"string of length {s.Length}");
    else if (o is int n && n > 40) Console.WriteLine($"big int {n}");
    else if (o is null) Console.WriteLine("null");
    else Console.WriteLine($"something else: {o.GetType().Name}");
}

object obj = "text";
string? asString = obj as string;          // succeeds
List<int>? asList = obj as List<int>;      // fails quietly: null
Console.WriteLine($"{asString}, {asList is null}");
```

OUTPUT

```
string of length 5
big int 42
something else: Double
null
something else: List`1
text, True
```

4.7.3 typeof, sizeof, nameof

`typeof(T)` returns the `System.Type` object describing a type at compile time (use `x.GetType()` for the run-time type of a value). `sizeof(T)` gives the size in bytes of an unmanaged value type. `nameof(x)` returns the name of a variable, type or member as a string, checked by the compiler, so it stays correct when you rename things; it is ideal in error messages and logging.

TYPEOF, SIZEOF AND NAMEOF Program.cs

```
Console.WriteLine(typeof(decimal));
Console.WriteLine(typeof(List<string>).Name);
Console.WriteLine($"sizeof: int {sizeof(int)}, double {sizeof(double)}, decimal {sizeof(decimal)}, char
{sizeof(char)}, bool {sizeof(bool)}");

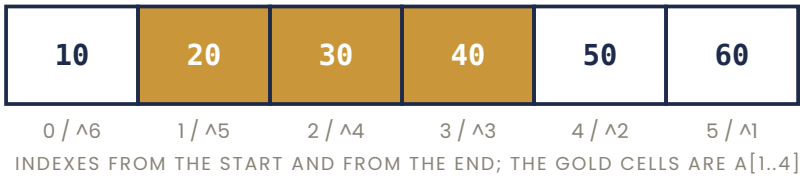
int studentAge = 17;
Console.WriteLine(nameof(studentAge));
Console.WriteLine(nameof(Console.WriteLine));
static void SetAge(int age)
{
    if (age < 0) throw new ArgumentOutOfRangeException(nameof(age), "age cannot be negative");
}
try { SetAge(-1); }
catch (ArgumentOutOfRangeException e) { Console.WriteLine(e.Message); }
```

OUTPUT

```
System.Decimal
List`1
sizeof: int 4, double 8, decimal 16, char 2, bool 1
studentAge
WriteLine
age cannot be negative (Parameter 'age')
```

4.7.4 Range .. and index from end ^

C# 8 added **indices** and **ranges** for arrays, strings, spans and lists. `^1` is the last element, `^2` the second last. `a..b` is the range from index `a` (inclusive) to `b` (exclusive); either end may be omitted (`..3`, `2..`, `..`) and either may use `^`. A range on an array or string returns a **new** copy.



INDEXES FROM THE START AND FROM THE END; THE GOLD CELLS ARE A[1..4]

INDICES AND RANGES Program.cs

```
int[] a = { 10, 20, 30, 40, 50, 60 };
Console.WriteLine($"a[^1] = {a[^1]}, a[^2] = {a[^2]}");
Console.WriteLine($"a[1..4]  = [{string.Join(", ", a[1..4])}]");
Console.WriteLine($"a[..2]   = [{string.Join(", ", a[..2])}]");
Console.WriteLine($"a[3..]   = [{string.Join(", ", a[3..])}]");
Console.WriteLine($"a[^3..]  = [{string.Join(", ", a[^3..])}]");
Console.WriteLine($"a[1..^1] = [{string.Join(", ", a[1..^1])}]");

string word = "Programming";
Console.WriteLine($"{word[..3]} {word[^4..]} {word[3..7]}");
Range middle = 2..5;                        // ranges and indices are values (System.Range,
System.Index)
Index last = ^1;
Console.WriteLine($"{word[middle]} {word[last]}");
```

OUTPUT

```
a[^1] = 60, a[^2] = 50
a[1..4]  = [20, 30, 40]
a[..2]   = [10, 20]
a[3..]   = [40, 50, 60]
a[^3..]  = [40, 50, 60]
a[1..^1] = [20, 30, 40, 50]
Pro ming gram
ogr g
```

4.7.5 checked and unchecked

As shown in 3.8.3, `checked(expr)` makes integer overflow in the expression throw `OverflowException`, and `unchecked(expr)` lets it wrap silently even in a project compiled with overflow checking. Both also exist as statement blocks.

CHECKED AND UNCHECKED EXPRESSIONS Program.cs

```
int big = int.MaxValue;
Console.WriteLine(unchecked(big * 2));
try { Console.WriteLine(checked(big * 2)); }
catch (OverflowException) { Console.WriteLine("checked: overflow detected"); }
```

OUTPUT

```
-2
checked: overflow detected
```

4.8 Precedence and associativity

When an expression has several operators, **precedence** decides which are applied first (`*` before `+`), and **associativity** decides the order among operators of the same precedence: most binary operators are **left-associative** (`a - b - c` is `(a - b) - c`); assignment, `??` and the conditional operator are **right-associative**. When in doubt, add parentheses: they cost nothing and help the reader.

Precedence (high to low)	Operators
Primary	`x.y`, `x?.y`, `f(x)`, `a[i]`, `x++`, `x--`, `x!`, `new`, `typeof`, `checked`, `nameof`
Unary	`+x`, `-x`, `!x`, `~x`, `++x`, `--x`, `^x`, `(T)x`, `await`
Range	`x..y`
switch / with	`switch` expression, `with`
Multiplicative	`*` `/` `%`
Additive	`+` `-`
Shift	`<<` `>>` `>>>`
Relational, type testing	`<` `>` `<=` `>=` `is` `as`
Equality	`==` `!=`
Bitwise / logical AND, XOR, OR	`&`, then `^`, then `|`
Conditional AND, OR	`&&`, then `||`
Null-coalescing	`??`
Conditional	`?:`
Assignment and lambda	`=` `+=` … `??=` `=>`

PRECEDENCE IN PRACTICE

Program.cs

```
Console.WriteLine(2 + 3 * 4);            // * first
Console.WriteLine((2 + 3) * 4);          // parentheses first
Console.WriteLine(20 - 6 - 4);           // left to right: (20 - 6) - 4
Console.WriteLine(100 / 10 / 5);         // (100 / 10) / 5
Console.WriteLine(2 + 3 > 4 && 10 % 3 == 1);   // arithmetic, then comparison, then &&
int x = 5;
Console.WriteLine(x > 3 ? "big" : "small" + "!");   // + binds tighter than ?:
string? s = null;
Console.WriteLine(s ?? "a" + "b");       // + before ??
```

OUTPUT

```
14
20
10
2
True
big
ab
```

WATCH OUT

A common surprise: `Console.WriteLine("Sum: " + 2 + 3)` prints `Sum: 23`, because `+` is evaluated left to right and the first operation is string concatenation. Write `"Sum: " + (2 + 3)` or use interpolation.

4.9 The Math class

4.9.1 Math.Abs, Pow, Sqrt, Round, Ceiling, Floor, Max, Min, Clamp

`System.Math` is a static class with mathematical constants and functions. Most methods work on `double` (and some have overloads for `decimal`, `int` and others). `Math.Round` uses **banker's rounding** by default (2.5 rounds to 2, 3.5 to 4); pass `MidpointRounding.AwayFromZero` for school-style rounding.

Method	Meaning	Example	Result
`Math.Abs(x)`	absolute value	`Math.Abs(-7.5)`	7.5
`Math.Pow(x, y)`	x to the power y	`Math.Pow(2, 10)`	1024
`Math.Sqrt(x)`	square root	`Math.Sqrt(144)`	12
`Math.Round(x, d)`	round to d decimals	`Math.Round(3.14159, 2)`	3.14
`Math.Ceiling(x)`	round up	`Math.Ceiling(4.1)`	5
`Math.Floor(x)`	round down	`Math.Floor(4.9)`	4
`Math.Truncate(x)`	drop the fraction	`Math.Truncate(-4.9)`	-4
`Math.Max` / `Min`	larger / smaller of two	`Math.Max(3, 8)`	8
`Math.Clamp(x, lo, hi)`	limit to a range	`Math.Clamp(120, 0, 100)`	100
`Math.PI`, `Math.E`	constants		3.14159..., 2.71828...

USING MATH

Program.cs

```
double r = 7;
Console.WriteLine($"area = {Math.PI * Math.Pow(r, 2):F2}");
Console.WriteLine($"hypotenuse = {Math.Sqrt(3 * 3 + 4 * 4)}");
Console.WriteLine($"Abs(-15) = {Math.Abs(-15)}, Max = {Math.Max(12, 7)}, Min = {Math.Min(12, 7)}");
Console.WriteLine($"Ceiling(4.1) = {Math.Ceiling(4.1)}, Floor(4.9) = {Math.Floor(4.9)}, Floor(-4.1) =
{Math.Floor(-4.1)}");
Console.WriteLine($"Round(2.5) = {Math.Round(2.5)}, Round(3.5) = {Math.Round(3.5)}");
Console.WriteLine($"Round(2.5, AwayFromZero) = {Math.Round(2.5, MidpointRounding.AwayFromZero)}");
Console.WriteLine($"Round(3.14159, 3) = {Math.Round(3.14159, 3)}");
Console.WriteLine($"Clamp(150, 0, 100) = {Math.Clamp(150, 0, 100)}, Clamp(-5, 0, 100) = {Math.Clamp(-5,
0, 100)}");
int pages = (int)Math.Ceiling(53 / 10.0);          // 53 items, 10 per page
Console.WriteLine($"pages needed: {pages}");
```

OUTPUT

```
area = 153.94
hypotenuse = 5
Abs(-15) = 15, Max = 12, Min = 7
Ceiling(4.1) = 5, Floor(4.9) = 4, Floor(-4.1) = -5
Round(2.5) = 2, Round(3.5) = 4
Round(2.5, AwayFromZero) = 3
Round(3.14159, 3) = 3.142
Clamp(150, 0, 100) = 100, Clamp(-5, 0, 100) = 0
pages needed: 6
```

4.9.2 Random and Random.Shared

`System.Random` generates pseudo-random numbers. `Next(max)` returns an int in [0, max), `Next(min, max)` in [min, max) (the upper bound is **exclusive**), `NextDouble()` a double in [0, 1). `Random.Shared` (.NET 6) is a

ready-made, thread-safe instance: use it instead of creating new `Random` objects repeatedly. Passing a seed (`new Random(42)`) makes the sequence repeatable, which is useful in tests and games with replays. For passwords, tokens and anything security-related use `System.Security.Cryptography.RandomNumberGenerator` instead.

RANDOM NUMBERS

Program.cs

```
int dice = Random.Shared.Next(1, 7);            // 1 to 6
Console.WriteLine($"dice in 1..6: {dice >= 1 && dice <= 6}");
double chance = Random.Shared.NextDouble();      // 0.0 <= chance < 1.0
Console.WriteLine($"chance in [0, 1): {chance >= 0 && chance < 1}");

var r1 = new Random(2026);                       // same seed ...
var r2 = new Random(2026);
bool same = true;
for (int i = 0; i < 1000; i++)
    if (r1.Next(100) != r2.Next(100)) same = false;
Console.WriteLine($"same seed, same sequence: {same}");   // ... same numbers: repeatable tests

string[] names = { "Asha", "Ravi", "Meera", "Kiran" };
Random.Shared.Shuffle(names);                    // .NET 8: shuffle in place (order varies)
Console.WriteLine($"still 4 names after shuffle: {names.Length == 4 && names.Contains("Asha")}");
string winner = names[Random.Shared.Next(names.Length)];
Console.WriteLine($"winner is one of them: {names.Contains(winner)}");
```

OUTPUT

```
dice in 1..6: True
chance in [0, 1): True
same seed, same sequence: True
still 4 names after shuffle: True
winner is one of them: True
```

The program prints properties of the random values rather than the values themselves, because those change on every run. Print `dice` or `string.Join(", ", names)` to see this run's actual values.

Integer / integer truncates; cast first to get a fractional result. ++ and -- have prefix and postfix forms. && and || short-circuit, which makes guards safe. Bitwise operators manipulate bits and flags. ?? supplies defaults, ??= initialises lazily, ?. navigates safely through possible nulls, and ! only silences warnings. Use is and as for type tests, nameof for names, ^ and .. for indices and ranges, parentheses for clarity, and Math and Random.Shared for calculations.

PRACTICE CHECKPOINT

A **calculator**, a program that uses **?? and ?.** to safely print a possibly-null name, and an **even/odd check with bitwise AND**.

SAMPLE SOLUTION

Program.cs

```
// 1. Calculator
Console.Write("First number: ");
double a = double.Parse(Console.ReadLine()!);
Console.Write("Operator (+ - * / %): ");
string op = Console.ReadLine()!;
Console.Write("Second number: ");
double b = double.Parse(Console.ReadLine()!);
string result = op == "+" ? $"{a + b}"
              : op == "-" ? $"{a - b}"
              : op == "*" ? $"{a * b}"
              : op == "/" ? (b == 0 ? "cannot divide by zero" : $"{a / b:F2}")
              : op == "%" ? $"{a % b}"
              : "unknown operator";
Console.WriteLine($"Result: {result}");

// 2. Safe name printing
Person? nobody = null;
Person noName = new Person();
Person someone = new Person { Name = "Kiran" };
foreach (Person? p in new[] { nobody, noName, someone })
    Console.WriteLine($"Name: {p?.Name?.ToUpper() ?? "(not available)"}, length: {p?.Name?.Length ??
0}");

// 3. Even / odd with bitwise AND
foreach (int n in new[] { 14, 7, 0, -3 })
    Console.WriteLine($"{n,3} is {((n & 1) == 0 ? "even" : "odd")}");

class Person { public string? Name { get; set; } }
```

CONSOLE (user input in gold)

```
First number: 22
Operator (+ - * / %): /
Second number: 7
Result: 3.14
Name: (not available), length: 0
Name: (not available), length: 0
Name: KIRAN, length: 5
 14 is even
  7 is odd
  0 is even
 -3 is odd
```

NOTE

`-3 & 1` is 1, because negative integers are stored in two's complement and their lowest bit is still set for odd numbers; so the bitwise test also works for negatives (unlike `n % 2 == 1`, which is false for -3 because `-3 % 2` is -1).

MODULE 05 · BEGINNER

Console input, output and formatting

Minimum duration **1 day** | Level **Beginner**

This module makes programs interactive. It covers writing to the console with string interpolation and composite formatting, format specifiers and alignment for tidy reports, culture-specific formatting, reading and validating input, single key presses, console colours, and command-line arguments.

IN THIS MODULE

5.1 Output
5.2 Input
5.3 Console features
5.4 Command-line arguments

NOTE

Number and date formatting depends on the **current culture** of the computer. To make the outputs in this module predictable, the examples that format numbers set `CultureInfo.CurrentCulture` to **en-US** first (5.1.6 explains cultures).

5.1 Output

5.1.1 Console.Write() and Console.WriteLine()

`Console.WriteLine(x)` writes a value followed by a newline; `Console.Write(x)` writes it without a newline, so the next output continues on the same line. Both accept any type: the value is converted with its `ToString()` method. `Console.WriteLine()` with no argument writes an empty line.

WRITE VS WRITELINE

Program.cs

```
Console.Write("Loading");
Console.Write(".");
Console.Write(".");
Console.WriteLine(".");          // ends the line
Console.WriteLine(42);
Console.WriteLine(3.75);
Console.WriteLine(true);
Console.WriteLine('Z');
Console.WriteLine();             // blank line
Console.WriteLine("Done");
```

OUTPUT

```
Loading...
42
3.75
True
Z

Done
```

5.1.2 String interpolation $"..."

An **interpolated string** starts with `$` and contains expressions in braces: `$"Total: {price * qty}"`. Each expression is evaluated, formatted and inserted. A format specifier can follow a colon (`{price:F2}`) and an alignment a comma (`{name,-10}`). Double the braces to print a literal brace (`{{ }}`). Interpolation is the modern, preferred way to build output.

STRING INTERPOLATION Program.cs

```
using System.Globalization;
CultureInfo.CurrentCulture = new CultureInfo("en-US");

string name = "Asha";
int qty = 3;
decimal price = 249.5m;
Console.WriteLine($"{name} bought {qty} books for {qty * price} rupees.");
Console.WriteLine($"Formatted: {qty * price:F2}, with separators: {1234567.891:N2}");
Console.WriteLine($"Expressions work: {name.ToUpper()}, {(qty > 2 ? "bulk order" : "single")}");
Console.WriteLine($"Braces: {{ {qty} }}");
Console.WriteLine($"Today is {new DateTime(2026, 10, 7):dddd, dd MMMM yyyy}");
string multi = $"""
    Student: {name}
    Books:   {qty}
    """;                                        // interpolated raw string literal (C# 11)
Console.WriteLine(multi);
```

OUTPUT

```
Asha bought 3 books for 748.5 rupees.
Formatted: 748.50, with separators: 1,234,567.89
Expressions work: ASHA, bulk order
Braces: { 3 }
Today is Wednesday, 07 October 2026
Student: Asha
Books:   3
```

5.1.3 Composite formatting {0}

Before interpolation, C# used **composite formatting**: numbered placeholders `{0}`, `{1}` in a format string, followed by the values. It is still used by `string.Format`, logging libraries (where the template is stored separately) and resource strings for translation. The same specifiers and alignment work: `{0,8:F2}`.

COMPOSITE FORMATTING Program.cs

```
using System.Globalization;
CultureInfo.CurrentCulture = new CultureInfo("en-US");

string item = "Pen";
int count = 12;
double cost = 7.5;
Console.WriteLine("Item: {0}, count: {1}, total: {2:F2}", item, count, count * cost);
Console.WriteLine("{1} before {0}, and {0} again", "first", "second");   // any order, reuse allowed
string s = string.Format("{0} x {1} = {2}", 6, 7, 6 * 7);
Console.WriteLine(s);
```

OUTPUT

```
Item: Pen, count: 12, total: 90.00
second before first, and first again
6 x 7 = 42
```

5.1.4 Format specifiers: N2, C, P, D5, X, F2, E

A **standard format specifier** is a letter, optionally followed by a precision number. They work in interpolation, composite formatting and `ToString("...")`.

Specifier	Name	Example (en-US)	Result
`F` / `F2`	fixed-point	`3.14159.ToString("F2")`	3.14
`N` / `N2`	number with group separators	`1234567.891.ToString("N2")`	1,234,567.89
`C` / `C2`	currency (culture symbol)	`1234.5m.ToString("C")`	$1,234.50
`P` / `P1`	percent (multiplies by 100)	`0.256.ToString("P1")`	25.6%
`D` / `D5`	decimal integer, zero-padded	`42.ToString("D5")`	00042
`X` / `X4`	hexadecimal (x for lower case)	`255.ToString("X4")`	00FF
`E` / `E2`	scientific	`1234.5678.ToString("E2")`	1.23E+003
`G`	general (the default)	`0.5.ToString("G")`	0.5
`R`	round-trip	`(0.1 + 0.2).ToString("R")`	0.30000000000000004

Custom format strings give full control: `0` is a digit that is always shown, `#` a digit shown only if significant, `.` the decimal point, `,` the group separator. Dates have their own custom codes: `dd` day, `MM` month, `yyyy` year, `HH` hour (24 h), `mm` minutes, `ddd` / `dddd` day name, `MMM` / `MMMM` month name.

STANDARD AND CUSTOM FORMAT STRINGS

Program.cs

```
using System.Globalization;
CultureInfo.CurrentCulture = new CultureInfo("en-US");

double value = 1234.5678;
Console.WriteLine($"F2: {value:F2}   N1: {value:N1}   E3: {value:E3}   G: {value:G}");
Console.WriteLine($"C : {1499.9m:C}   P : {0.075:P}   P0: {0.875:P0}");
Console.WriteLine($"D6: {427:D6}   X : {3054:X}   x8: {3054:x8}");
Console.WriteLine($"custom 0000.00: {value:0000.00}   #,##0.0: {value:#,##0.0}   #.##: {0.5:#.##}");
Console.WriteLine($"phone: {9876543210:(###) ###-####}");
DateTime exam = new DateTime(2026, 12, 15, 9, 30, 0);
Console.WriteLine($"{exam:dd/MM/yyyy}  {exam:yyyy-MM-dd HH:mm}  {exam:ddd, MMM d}  {exam:hh:mm tt}");
```

OUTPUT

```
F2: 1234.57   N1: 1,234.6   E3: 1.235E+003   G: 1234.5678
C : $1,499.90   P : 7.50%   P0: 88%
D6: 000427   X : BEE   x8: 00000bee
custom 0000.00: 1234.57   #,##0.0: 1,234.6   #.##: .5
phone: (987) 654-3210
15/12/2026  2026-12-15 09:30  Tue, Dec 15  09:30 AM
```

NOTE

`0.875:P0` gives 88% because 87.5 is rounded away from zero in formatting (formatting does not use banker's rounding). `#.##` drops the leading zero, so 0.5 shows as `.5` ; use `0.##` to keep it.

5.1.5 Alignment {value,10} and {value,-10}

An alignment component after a comma pads the value to a minimum width: a **positive** width right-aligns (good for numbers), a **negative** width left-aligns (good for text). Combined with format specifiers it produces neat tables. Values longer than the width are not cut.

A FORMATTED BILL WITH ALIGNMENT

Program.cs

```
using System.Globalization;
CultureInfo.CurrentCulture = new CultureInfo("en-US");

var items = new (string Name, int Qty, decimal Price)[]
{
    ("Notebook", 3, 45m), ("Pen", 10, 12.5m), ("Geometry box", 1, 185m)
};
Console.WriteLine($"{"Item",-14}{"Qty",5}{"Price",10}{"Amount",11}");
Console.WriteLine(new string('-', 40));
decimal total = 0;
foreach (var (name, qty, price) in items)
{
    decimal amount = qty * price;
    total += amount;
    Console.WriteLine($"{name,-14}{qty,5}{price,10:F2}{amount,11:N2}");
}
Console.WriteLine(new string('-', 40));
Console.WriteLine($"{"TOTAL",-29}{total,11:N2}");
Console.WriteLine($"[{"left",-8}] [{"right",8}] [{"too long for six",6}]");
```

OUTPUT

```
Item            Qty     Price     Amount
----------------------------------------
Notebook          3     45.00     135.00
Pen              10     12.50     125.00
Geometry box      1    185.00     185.00
----------------------------------------
TOTAL                             445.00
[left    ] [   right] [too long for six]
```

5.1.6 Culture-specific formatting (CultureInfo, overview)

Countries write numbers, currency and dates differently. A **culture** (`System.Globalization.CultureInfo`, named like `en-US`, `en-IN`, `hi-IN`, `de-DE`) bundles these rules. Formatting and parsing use `CultureInfo.CurrentCulture` unless you pass a culture explicitly. For data files, logs and anything read by programs, use `CultureInfo.InvariantCulture`, so that "3.5" always means three and a half.

ONE VALUE, FOUR CULTURES

Program.cs

```
using System.Globalization;

decimal amount = 1234567.891m;
DateTime d = new DateTime(2026, 10, 7);
foreach (string name in new[] { "en-US", "en-IN", "de-DE", "fr-FR" })
{
    var c = new CultureInfo(name);
    Console.WriteLine($"{name}: {amount.ToString("N2", c),-16} {amount.ToString("C", c),-18}
{d.ToString("d", c)}");
}
Console.WriteLine($"invariant: {amount.ToString(CultureInfo.InvariantCulture)}");
double parsed = double.Parse("3,5", new CultureInfo("de-DE"));   // German decimal comma
Console.WriteLine(parsed.ToString(CultureInfo.InvariantCulture));
```

OUTPUT

```
en-US: 1,234,567.89     $1,234,567.89      10/7/2026
en-IN: 12,34,567.89     ₹12,34,567.89      07-10-2026
de-DE: 1.234.567,89     1.234.567,89 €     07.10.2026
fr-FR: 1 234 567,89     1 234 567,89 €     07/10/2026
invariant: 1234567.891
3.5
```

NOTE

The exact symbols come from the operating system's culture data (ICU on Linux and macOS, NLS or ICU on Windows) and can differ slightly between versions; for example, French uses a narrow no-break space as the group separator.

5.2 Input

5.2.1 Console.ReadLine() returns string?

`Console.ReadLine()` waits until the user presses Enter and returns the line typed, **without** the newline, as a `string`. Its return type is `string?` because it returns `null` when the input stream has ended (the user pressed Ctrl+Z on Windows or Ctrl+D on Linux/macOS, or the input was redirected from a file that ran out). The input is always text: numbers must be converted.

READING LINES OF TEXT Program.cs

```
Console.Write("Enter your name: ");
string? name = Console.ReadLine();
Console.Write("Enter your city: ");
string city = Console.ReadLine() ?? "unknown";        // handle null with ??
Console.WriteLine($"Hello {name}, from {city}!");
Console.WriteLine($"Your name has {name?.Length ?? 0} characters.");
```

CONSOLE (user input in gold)

```
Enter your name: Ravi Kumar
Enter your city: Mangaluru
Hello Ravi Kumar, from Mangaluru!
Your name has 10 characters.
```

5.2.2 Converting input with int.Parse and int.TryParse

`int.Parse(text)` converts or throws `FormatException`; `int.TryParse(text, out int n)` returns `false` instead. With user input, always prefer `TryParse`: users mistype, leave fields empty and paste odd characters. The same methods exist for `double`, `decimal`, `DateTime`, `bool` and more.

PARSE VS TRYPARSE ON USER INPUT Program.cs

```
Console.Write("Age: ");
int age = int.Parse(Console.ReadLine()!);             // fine only if the user types a valid number
Console.Write("Height in metres: ");
double height = double.Parse(Console.ReadLine()!);
Console.WriteLine($"In 5 years you will be {age + 5}; height {height * 100} cm");

Console.Write("Marks: ");
if (int.TryParse(Console.ReadLine(), out int marks))
    Console.WriteLine($"Marks + grace = {marks + 5}");
else
    Console.WriteLine("That was not a whole number.");
```

CONSOLE (user input in gold)

```
Age: 19
Height in metres: 1.68
In 5 years you will be 24; height 168 cm
Marks: eighty
That was not a whole number.
```

5.2.3 Validating input in a loop

Robust programs ask again until the input is valid. The pattern is a loop that reads, tries to convert, checks the range, and only exits when everything is fine. Putting it in a small helper method keeps the main code clean.

ASKING UNTIL THE INPUT IS VALID Program.cs

```
int marks = ReadInt("Enter marks (0-100): ", 0, 100);
Console.WriteLine($"Accepted: {marks}");

static int ReadInt(string prompt, int min, int max)
{
    while (true)
    {
        Console.Write(prompt);
        string? text = Console.ReadLine();
        if (!int.TryParse(text, out int value))
            Console.WriteLine("  Please type a whole number.");
        else if (value < min || value > max)
            Console.WriteLine($"  The number must be between {min} and {max}.");
        else
            return value;                // valid: leave the loop
    }
}
```

CONSOLE (user input in gold)

```
Enter marks (0-100): abc
  Please type a whole number.
Enter marks (0-100): 150
  The number must be between 0 and 100.
Enter marks (0-100): 87
Accepted: 87
```

5.2.4 Console.ReadKey()

`Console.ReadKey()` reads a **single key press** without waiting for Enter and returns a `ConsoleKeyInfo` with the character (`KeyChar`), the key (`Key`, such as `ConsoleKey.Escape` or `ConsoleKey.UpArrow`) and modifier keys. `ReadKey(true)` does not display the key, which suits menus, games and password entry.

SINGLE KEYS AND HIDDEN INPUT Program.cs

```
Console.Write("Continue? (y/n) ");
ConsoleKeyInfo key = Console.ReadKey();
Console.WriteLine();
Console.WriteLine(key.KeyChar == 'y' || key.KeyChar == 'Y' ? "Continuing..." : "Stopped.");

Console.Write("PIN: ");
string pin = "";
while (true)
{
    ConsoleKeyInfo k = Console.ReadKey(intercept: true);   // not echoed
    if (k.Key == ConsoleKey.Enter) break;
    pin += k.KeyChar;
    Console.Write("*");
}
Console.WriteLine($"\nPIN has {pin.Length} digits");
```

CONSOLE (user input in gold)

```
Continue? (y/n) y
Continuing...
PIN: ****
PIN has 4 digits
```

5.3 Console features

5.3.1 Colours, Clear(), Title

The `Console` class can change the text colour (`ForegroundColor`) and background (`BackgroundColor`) using the `ConsoleColor` enum, restore them with `ResetColor()`, clear the screen with `Clear()`, set the window `Title`, move the cursor with `SetCursorPosition(x, y)`, and play a `Beep()`. Colours make output easier to scan: errors in red, success in green.

COLOURED OUTPUT Program.cs

```
Console.Title = "Result Checker";
Console.ForegroundColor = ConsoleColor.Cyan;
Console.WriteLine("=== RESULTS ===");
Console.ResetColor();

foreach (var (name, marks) in new[] { ("Asha", 82), ("Ravi", 35), ("Meera", 58) })
{
    Console.Write($"{name,-6}");
    Console.ForegroundColor = marks >= 40 ? ConsoleColor.Green : ConsoleColor.Red;
    Console.WriteLine(marks >= 40 ? "PASS" : "FAIL");
    Console.ResetColor();
}
Console.BackgroundColor = ConsoleColor.DarkBlue;
Console.ForegroundColor = ConsoleColor.Yellow;
Console.WriteLine(" 2 of 3 passed ");
Console.ResetColor();
```

OUTPUT

```
=== RESULTS ===
Asha  PASS
Ravi  FAIL
Meera PASS
 2 of 3 passed
```

In a terminal "=== RESULTS ===" appears in cyan, PASS in green, FAIL in red and the summary in yellow on dark blue.

Member	Purpose
`ForegroundColor`, `BackgroundColor`	text and background colour (ConsoleColor enum)
`ResetColor()`	back to the default colours
`Clear()`	clear the screen
`Title`	window title text
`SetCursorPosition(x, y)`, `CursorVisible`	move / hide the cursor (menus, games)
`WindowWidth`, `WindowHeight`	size of the console window
`Beep()`	a short sound (Windows)
`Console.Error.WriteLine`	write to the error stream (kept separate when output is redirected)

5.4 Command-line arguments

5.4.1 args array

Values typed after the program name on the command line arrive in the `string[] args` array: with top-level statements `args` is available automatically, in the classic form it is the parameter of `Main`. Arguments are separated by spaces; quote a value that contains spaces. With `dotnet run`, put the program's arguments after `--`. Arguments let scripts and other programs control yours without interactive input.

USING COMMAND-LINE ARGUMENTS — Program.cs

```
Console.WriteLine($"{args.Length} argument(s)");
for (int i = 0; i < args.Length; i++)
    Console.WriteLine($"  args[{i}] = {args[i]}");

if (args.Length == 3 && double.TryParse(args[0], out double a) && double.TryParse(args[2], out double b))
{
    double result = args[1] switch { "+" => a + b, "-" => a - b, "x" => a * b, "/" => a / b, _ =>
double.NaN };
    Console.WriteLine($"{a} {args[1]} {b} = {result}");
}
else
{
    Console.WriteLine("usage: calc <number> <+|-|x|/> <number>");
}
```

OUTPUT

```
4 argument(s)
  args[0] = 12
  args[1] = x
  args[2] = 3.5
  args[3] = extra words
usage: calc <number> <+|-|x|/> <number>
```

Running the program with different arguments

```
$ dotnet run -- 12 x 3.5 "extra words"
4 argument(s)
  args[0] = 12
  args[1] = x
  args[2] = 3.5
  args[3] = extra words
usage: calc <number> <+|-|x|/> <number>
$ dotnet run -- 12 x 3.5
3 argument(s)
  args[0] = 12
  args[1] = x
  args[2] = 3.5
12 x 3.5 = 42
```

NOTE

`*` is avoided as the multiplication symbol because many shells expand it into a list of file names; quoting it (`"*"`) also works. In Visual Studio, arguments are set in the project's Debug properties (launch profile).

Write and WriteLine print values; interpolation $"..." with {value,alignment:format} is the modern way to build output, and composite formatting {0} remains for templates. Learn the common specifiers (F2, N2, C, P, D5, X, E) and custom patterns, and remember that cultures change separators and symbols. ReadLine returns string? text: convert it with TryParse inside a validation loop. ReadKey reads single keys, Console offers colours and cursor control, and args carries command-line arguments.

PRACTICE CHECKPOINT

Read a student's **name and three marks with input validation**, then print a **formatted report card** with the average to two decimal places.

SAMPLE SOLUTION

Program.cs

```
using System.Globalization;
CultureInfo.CurrentCulture = new CultureInfo("en-US");

Console.Write("Student name: ");
string name = Console.ReadLine()?.Trim() ?? "";
while (name.Length == 0)
{
    Console.Write("  Name cannot be empty. Student name: ");
    name = Console.ReadLine()?.Trim() ?? "";
}

string[] subjects = { "Mathematics", "Science", "English" };
int[] marks = new int[subjects.Length];
for (int i = 0; i < subjects.Length; i++)
    marks[i] = ReadMark($"{subjects[i]} marks (0-100): ");

int total = marks[0] + marks[1] + marks[2];
double average = total / 3.0;
string grade = average >= 75 ? "Distinction" : average >= 60 ? "First class" : average >= 40 ? "Pass" :
"Fail";

Console.WriteLine();
Console.WriteLine(new string('=', 34));
Console.WriteLine($"{"REPORT CARD",22}");
Console.WriteLine(new string('=', 34));
Console.WriteLine($"Name: {name}");
Console.WriteLine(new string('-', 34));
```

```
for (int i = 0; i < subjects.Length; i++)
    Console.WriteLine($"{subjects[i],-20}{marks[i],10}");
Console.WriteLine(new string('-', 34));
Console.WriteLine($"{"Total",-20}{total,10}");
Console.WriteLine($"{"Average",-20}{average,10:F2}");
Console.WriteLine($"{"Result",-20}{grade,10}");
Console.WriteLine(new string('=', 34));

static int ReadMark(string prompt)
{
    while (true)
    {
        Console.Write(prompt);
        if (int.TryParse(Console.ReadLine(), out int m) && m >= 0 && m <= 100) return m;
        Console.WriteLine("  Invalid: enter a whole number from 0 to 100.");
    }
}
```

CONSOLE (user input in gold)

```
Student name:
  Name cannot be empty. Student name: Meera Iyer
Mathematics marks (0-100): 92
Science marks (0-100): 105
  Invalid: enter a whole number from 0 to 100.
Science marks (0-100): 88
English marks (0-100): seventy
  Invalid: enter a whole number from 0 to 100.
English marks (0-100): 79

==================================
           REPORT CARD
==================================
Name: Meera Iyer
----------------------------------
Mathematics                  92
Science                      88
English                      79
----------------------------------
Total                       259
Average                   86.33
Result              Distinction
==================================
```

MODULE 06 · BEGINNER

Control flow

Minimum duration **3 days** | Level **Beginner**

Control flow decides which statements run and how many times. This module covers if and else ladders, the switch statement and the modern switch expression with patterns, the four loops (while, do-while, for, foreach), nested loops for star and number patterns, jump statements, and a set of classic loop problems that every programmer should be able to solve.

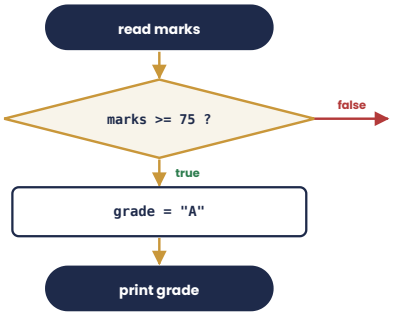
IN THIS MODULE

6.1 Decision-making statements
6.2 switch
6.3 Loops
6.4 Nested loops and patterns
6.5 Jump statements
6.6 Classic loop problems

6.1 Decision-making statements

6.1.1 if, if-else, else-if ladder, nested if

An `if` statement runs a block only when its condition (a `bool` expression) is true. An `else` block runs when it is false. An **else-if ladder** tests several conditions in order and runs the **first** block whose condition is true. Ifs can be **nested** inside other ifs. Always use braces, even for one statement: it prevents bugs when a second line is added later.



AN IF STATEMENT: THE CONDITION CHOOSES THE PATH

ALL FORMS OF IF

Program.cs

```
int temperature = 34;

if (temperature > 30)                          // simple if
{
    Console.WriteLine("It is hot today.");
}

if (temperature % 2 == 0)                      // if-else
    Console.WriteLine("Even temperature");
else
    Console.WriteLine("Odd temperature");

int marks = 68;                                // else-if ladder: the first true condition wins
if (marks >= 90) Console.WriteLine("Grade: O (outstanding)");
else if (marks >= 75) Console.WriteLine("Grade: A");
else if (marks >= 60) Console.WriteLine("Grade: B");
else if (marks >= 40) Console.WriteLine("Grade: C");
else Console.WriteLine("Grade: F");

int age = 20;                                  // nested if
bool hasId = false;
if (age >= 18)
{
    if (hasId) Console.WriteLine("Entry allowed");
    else Console.WriteLine("Adult, but please show an ID");
}
else
{
    Console.WriteLine("Entry not allowed");
}
```

OUTPUT

```
It is hot today.
Even temperature
Grade: B
Adult, but please show an ID
```

WATCH OUT

Order matters in a ladder. If the first test were `marks >= 40`, every passing student would get grade C, because the later tests are never reached. Test the most specific (or largest) condition first.

LEAP YEARS: A CLASSIC DECISION PROBLEM

Program.cs

```
int year = 2024;
bool isLeap = (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
Console.WriteLine($"{year} leap year? {isLeap}");

foreach (int y in new[] { 1900, 2000, 2025, 2028 })
{
    if (y % 400 == 0) Console.WriteLine($"{y}: leap (divisible by 400)");
    else if (y % 100 == 0) Console.WriteLine($"{y}: not leap (century)");
    else if (y % 4 == 0) Console.WriteLine($"{y}: leap");
    else Console.WriteLine($"{y}: not leap");
}
```

OUTPUT

```
2024 leap year? True
1900: not leap (century)
2000: leap (divisible by 400)
2025: not leap
2028: leap
```

6.2 switch

6.2.1 switch statement with case, break, default

A `switch` statement compares one value against several constant **case labels** and runs the matching section. `default` runs when nothing matches. Each section must end with `break`, `return`, `throw` or `goto`. Several labels can share a section. It is clearer than a long else-if ladder when you compare one variable against many fixed values (ints, chars, strings, enums).

THE SWITCH STATEMENT

Program.cs

```
int day = 3;
switch (day)
{
    case 1:
        Console.WriteLine("Monday");
        break;
    case 2:
        Console.WriteLine("Tuesday");
        break;
    case 3:
        Console.WriteLine("Wednesday");
        break;
    case 6:
    case 7:                                  // two labels, one section
        Console.WriteLine("Weekend");
        break;
    default:
        Console.WriteLine("Another weekday");
        break;
}

string command = "STOP".ToLower();
switch (command)                             // strings work too
{
    case "start": Console.WriteLine("Starting engine"); break;
    case "stop":  Console.WriteLine("Stopping engine"); break;
    default:      Console.WriteLine("Unknown command"); break;
}
```

OUTPUT

```
Wednesday
Stopping engine
```

6.2.2 No implicit fall-through rule

In C and C++, forgetting `break` silently "falls through" into the next case, a frequent bug. C# forbids it: a non-empty case section that reaches the next label is a compile error. Empty labels stacked together (like `case 6: case 7:`) are allowed.

FALLING THROUGH IS A COMPILE ERROR

Program.cs

```
int n = 1;
switch (n)
{
    case 1:
        Console.WriteLine("one");      // no break: not allowed
    case 2:
        Console.WriteLine("two");
        break;
}
```

COMPILER OUTPUT

```
Program.cs(4,5): error CS0163: Control cannot fall through from one case label ('case 1:') to another
```

6.2.3 goto case

When you really want to continue into another case, say so explicitly with `goto case value;` or `goto default;`. It is rarely needed.

EXPLICIT FALL-THROUGH WITH GOTO CASE

Program.cs

```
string size = "large";
decimal price = 0;
switch (size)
{
    case "large":
        price += 40;          // extra for large
        goto case "medium";   // then also add the medium price
    case "medium":
        price += 60;
        goto case "small";
    case "small":
        price += 100;
        break;
}
Console.WriteLine($"{size} coffee costs Rs {price}");
```

OUTPUT

```
large coffee costs Rs 200
```

6.2.4 Pattern-based cases with when guards

Since C# 7, case labels can be **patterns**: type patterns (`case int n:`), relational patterns (`case >= 90:` , C# 9) and combinations with `and` , `or` , `not` . A `when` clause adds an extra condition (a **guard**).

PATTERNS AND GUARDS IN CASE LABELS

Program.cs

```
object[] values = { 42, -7, "hello", 3.5, null!, 'x' };
foreach (object v in values)
{
    switch (v)
    {
        case int n when n > 0:
            Console.WriteLine($"{n} is a positive int");
            break;
        case int n:
            Console.WriteLine($"{n} is a non-positive int");
            break;
        case string s:
            Console.WriteLine($"string \"{s}\" of length {s.Length}");
            break;
        case double or float:
            Console.WriteLine($"floating point {v}");
            break;
        case null:
            Console.WriteLine("null");
            break;
        default:
            Console.WriteLine($"other: {v.GetType().Name}");
            break;
    }
}

int marks = 83;
switch (marks)
{
    case >= 90: Console.WriteLine("O"); break;
    case >= 75 and < 90: Console.WriteLine("A"); break;
    case >= 40: Console.WriteLine("Pass"); break;
    default: Console.WriteLine("Fail"); break;
}
```

OUTPUT

```
42 is a positive int
-7 is a non-positive int
string "hello" of length 5
floating point 3.5
null
other: Char
A
```

6.2.5 switch expressions (C# 8) x switch { ... }

A **switch expression** produces a value instead of running statements. It is written `value switch { pattern => result, ... }`, arms are separated by commas, `_` is the discard pattern that matches anything (the default). It is compact, it is an expression (so it can be assigned, returned or used inside interpolation), and the compiler warns if the arms do not cover every possible value. If nothing matches at run time, it throws `SwitchExpressionException`.

SWITCH EXPRESSIONS Program.cs

```
int day = 6;
string name = day switch
{
    1 => "Monday", 2 => "Tuesday", 3 => "Wednesday", 4 => "Thursday", 5 => "Friday",
    6 or 7 => "Weekend",
    _ => "Invalid day"
};
Console.WriteLine(name);

static string Grade(int marks) => marks switch
{
    >= 90 => "O",
    >= 75 => "A",
    >= 60 => "B",
    >= 40 => "C",
    < 0 or > 100 => throw new ArgumentOutOfRangeException(nameof(marks)),
    _ => "F"
};
foreach (int m in new[] { 95, 76, 61, 40, 12 })
    Console.Write($"{m}:{Grade(m)} ");
Console.WriteLine();

static decimal Shipping(string zone, decimal weight) => (zone, weight) switch     // tuple pattern
{
    ("local", <= 1) => 40,
    ("local", _) => 60,
    ("national", <= 1) => 90,
    ("national", _) => 150,
    _ => 500
};
Console.WriteLine($"local 0.5 kg: {Shipping("local", 0.5m)}, national 3 kg: {Shipping("national", 3m)},
abroad: {Shipping("abroad", 1m)}");
```

OUTPUT

```
Weekend
95:O 76:A 61:B 40:C 12:F
local 0.5 kg: 40, national 3 kg: 150, abroad: 500
```

	switch statement	switch expression
Produces	runs statements	a value
Syntax	`case x: ... break;`	`x => result,`
Default	`default:`	`_ =>`
Unmatched value	nothing happens (if no default)	SwitchExpressionException
Best for	several statements per case	mapping a value to a result

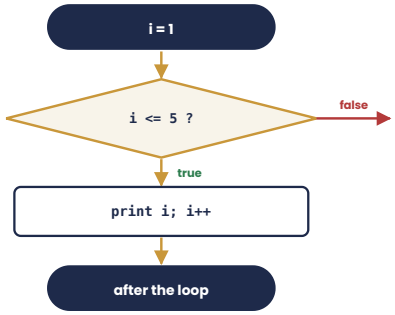
6.3 Loops

Loops repeat a block of code. C# has four: `while` and `do-while` repeat while a condition holds, `for` is designed for counting, and `foreach` visits every element of a collection.

6.3.1 while

A `while` loop checks its condition **before** each iteration, so the body may run zero times. Use it when you do not know in advance how many iterations are needed. Something in the body must eventually make the

condition false, or the loop never ends.



A WHILE LOOP: TEST, RUN THE BODY, GO BACK TO THE TEST

WHILE LOOPS Program.cs

```
int i = 1;
while (i <= 5)
{
    Console.Write($"{i} ");
    i++;                                    // without this, the loop never ends
}
Console.WriteLine();

int amount = 10000, years = 0;              // how many years until the deposit doubles at 8%?
double balance = amount;
while (balance < 2 * amount)
{
    balance *= 1.08;
    years++;
}
Console.WriteLine($"Doubles after {years} years: {balance:F2}");

int n = 0;
while (n > 0) Console.WriteLine("never printed");   // condition false at the start: zero iterations
```

OUTPUT

```
1 2 3 4 5
Doubles after 10 years: 21589.25
```

6.3.2 do-while

A `do-while` loop checks its condition **after** each iteration, so the body always runs **at least once**. Note the semicolon after the condition. It is the natural loop for menus and for "read input until it is valid".

A MENU WITH DO-WHILE

Program.cs

```
int choice;
do
{
    Console.WriteLine("1. Balance  2. Deposit  0. Exit");
    Console.Write("Choose: ");
    choice = int.TryParse(Console.ReadLine(), out int c) ? c : -1;
    Console.WriteLine(choice switch
    {
        1 => "Balance: Rs 5,000",
        2 => "Deposit accepted",
        0 => "Goodbye",
        _ => "Invalid choice"
    });
} while (choice != 0);
```

CONSOLE (user input in gold)

```
1. Balance  2. Deposit  0. Exit
Choose: 1
Balance: Rs 5,000
1. Balance  2. Deposit  0. Exit
Choose: 7
Invalid choice
1. Balance  2. Deposit  0. Exit
Choose: 0
Goodbye
```

6.3.3 for

A `for` loop puts the three parts of a counting loop on one line: `for (initialiser; condition; iterator)`. The initialiser runs once, the condition is tested before each iteration, and the iterator runs after each iteration. The loop variable exists only inside the loop. Any part may be omitted, and several variables can be used, separated by commas.

```
for (int i = 0; i < 5; i++)
```

① once, at the start ② before each pass ③ after each pass

THE THREE PARTS OF A FOR LOOP

FOR LOOPS

Program.cs

```
for (int i = 0; i < 5; i++)
    Console.Write($"{i} ");
Console.WriteLine();

for (int i = 10; i >= 0; i -= 2)            // counting down in steps of 2
    Console.Write($"{i} ");
Console.WriteLine();

for (int i = 1; i <= 10; i++)               // multiplication table of 7
    Console.Write($"{7 * i} ");
Console.WriteLine();

for (int lo = 0, hi = 9; lo < hi; lo++, hi--)   // two variables
    Console.Write($"({lo},{hi}) ");
Console.WriteLine();

long factorial = 1;
for (int k = 2; k <= 20; k++) factorial *= k;
Console.WriteLine($"20! = {factorial:N0}");
```

OUTPUT

```
0 1 2 3 4
10 8 6 4 2 0
7 14 21 28 35 42 49 56 63 70
(0,9) (1,8) (2,7) (3,6) (4,5)
20! = 2,432,902,008,176,640,000
```

6.3.4 foreach

`foreach (var item in collection)` visits every element of an array, list, string, dictionary or any `IEnumerable`, in order, without an index. The loop variable is read-only: you cannot assign to it or add and remove elements of the collection while iterating (that throws `InvalidOperationException`). Use `for` when you need the index or want to modify elements of an array.

FOREACH OVER ARRAYS, STRINGS AND DICTIONARIES

Program.cs

```
string[] cities = { "Mysuru", "Hubballi", "Udupi" };
foreach (string city in cities)
    Console.Write($"{city} ");
Console.WriteLine();

foreach (char c in "C#!")
    Console.Write($"[{c}]");
Console.WriteLine();

var stock = new Dictionary<string, int> { ["pen"] = 40, ["ink"] = 5 };
foreach (var (item, qty) in stock)              // deconstruct each KeyValuePair
    Console.WriteLine($"{item}: {qty}");

int[] marks = { 70, 85, 90 };
int sum = 0;
foreach (int m in marks) sum += m;
Console.WriteLine($"average = {sum / (double)marks.Length:F1}");
// foreach (int m in marks) m += 5;            // error CS1656: Cannot assign to 'm' because it is a
'foreach iteration variable'
for (int i = 0; i < marks.Length; i++) marks[i] += 5;   // use for to modify
Console.WriteLine(string.Join(", ", marks));
```

OUTPUT

```
Mysuru Hubballi Udupi
[C][#][!]
pen: 40
ink: 5
average = 81.7
75, 90, 95
```

6.3.5 Infinite loops and choosing the right loop

A loop whose condition never becomes false runs forever. Sometimes that is intended (a server, a game loop, a menu): write `while (true)` or `for (;;)` and leave with `break` or `return`. Accidental infinite loops come from forgetting to update the loop variable or from a condition that can never be reached (`for (int i = 0; i != 10; i += 3)`). In a console app, Ctrl+C stops a runaway program.

Situation	Best loop
Known number of iterations, need the index	`for`
Visit every element of a collection	`foreach`
Repeat while a condition holds, possibly zero times	`while`
Body must run at least once (menus, input validation)	`do-while`
Run until something inside decides to stop	`while (true)` + `break`

WHILE (TRUE) WITH BREAK

Program.cs

```
int attempts = 0;
while (true)                        // intentional infinite loop
{
    attempts++;
    int roll = attempts * 7 % 6 + 1;  // a pretend dice roll: 2, 3, 4, 5, 6, 1 ...
    Console.Write($"{roll} ");
    if (roll == 6) break;             // the way out
}
Console.WriteLine($"\nGot a 6 after {attempts} attempts");
```

OUTPUT

```
2 3 4 5 6
Got a 6 after 5 attempts
```

6.4 Nested loops and patterns

A loop inside another loop is a **nested loop**: for each iteration of the outer loop, the inner loop runs completely. Printing patterns is the classic way to practise: the outer loop usually controls the **rows**, the inner loop(s) the **columns** (how many spaces, how many stars). Work out, for row i, how many of each character are needed, then write the loops.

6.4.1 Star patterns (square, triangles, pyramid, diamond)

SQUARE AND TRIANGLES

Program.cs

```
int n = 4;
Console.WriteLine("Square:");
for (int i = 1; i <= n; i++)
{
    for (int j = 1; j <= n; j++) Console.Write("* ");
    Console.WriteLine();
}
Console.WriteLine("Right triangle:");
for (int i = 1; i <= n; i++)
{
    for (int j = 1; j <= i; j++) Console.Write("* ");      // row i has i stars
    Console.WriteLine();
}
Console.WriteLine("Inverted triangle:");
for (int i = n; i >= 1; i--)
{
    for (int j = 1; j <= i; j++) Console.Write("* ");
    Console.WriteLine();
}
```

OUTPUT

```
Square:
* * * *
* * * *
* * * *
* * * *
Right triangle:
*
* *
* * *
* * * *
Inverted triangle:
* * * *
* * *
* *
*
```

Row i (n = 4)	Spaces	Stars	Pyramid row
1	n − i = 3	2i − 1 = 1	`   *`
2	2	3	`  ***`
3	1	5	`*****`
4	0	7	`*******`

PYRAMID AND DIAMOND

Program.cs

```
int n = 4;
Console.WriteLine("Pyramid:");
for (int i = 1; i <= n; i++)
{
    Console.Write(new string(' ', n - i));        // n - i spaces
    Console.WriteLine(new string('*', 2 * i - 1)); // 2i - 1 stars
}
Console.WriteLine("Diamond:");
for (int i = 1; i <= n; i++)                      // top half (same as the pyramid)
{
    for (int s = 1; s <= n - i; s++) Console.Write(' ');
    for (int k = 1; k <= 2 * i - 1; k++) Console.Write('*');
    Console.WriteLine();
}
for (int i = n - 1; i >= 1; i--)                  // bottom half
{
    for (int s = 1; s <= n - i; s++) Console.Write(' ');
    for (int k = 1; k <= 2 * i - 1; k++) Console.Write('*');
    Console.WriteLine();
}
```

OUTPUT

```
Pyramid:
   *
  ***
 *****
*******
Diamond:
   *
  ***
 *****
*******
 *****
  ***
   *
```

6.4.2 Number patterns (Floyd's triangle, Pascal's triangle)

Floyd's triangle fills rows of increasing length with consecutive numbers. **Pascal's triangle** has 1 at both ends of each row, and every inner number is the sum of the two above it; the value at row r, position k is the binomial coefficient C(r, k), which can be computed from the previous value in the row: C(r, k) = C(r, k-1) × (r - k + 1) / k.

NUMBER PATTERNS

Program.cs

```
int rows = 5, number = 1;
Console.WriteLine("Floyd's triangle:");
for (int i = 1; i <= rows; i++)
{
    for (int j = 1; j <= i; j++) Console.Write($"{number++,3}");
    Console.WriteLine();
}

Console.WriteLine("Pascal's triangle:");
for (int r = 0; r < rows; r++)
{
    Console.Write(new string(' ', (rows - r - 1) * 2));
    long c = 1;
    for (int k = 0; k <= r; k++)
    {
        Console.Write($"{c,4}");
        c = c * (r - k) / (k + 1);               // next coefficient in the row
    }
    Console.WriteLine();
}

Console.WriteLine("Number triangle:");
for (int i = 1; i <= 4; i++)
{
    for (int j = 1; j <= i; j++) Console.Write(j);
    Console.WriteLine();
}
```

OUTPUT

```
Floyd's triangle:
  1
  2  3
  4  5  6
  7  8  9 10
 11 12 13 14 15
Pascal's triangle:
           1
         1   1
       1   2   1
     1   3   3   1
   1   4   6   4   1
Number triangle:
1
12
123
1234
```

6.5 Jump statements

6.5.1 break, continue, return

`break` leaves the innermost loop (or switch) immediately. `continue` skips the rest of the current iteration and moves on to the next one. `return` leaves the whole method (in top-level code it ends the program), optionally with a value. In nested loops, `break` only leaves the inner loop; to leave both, use a flag, put the loops in a method and `return`, or (rarely) `goto`.

BREAK, CONTINUE AND RETURN

Program.cs

```
Console.Write("break at 5: ");
for (int i = 1; i <= 10; i++)
{
    if (i == 5) break;              // stop the loop completely
    Console.Write($"{i} ");
}
Console.WriteLine();

Console.Write("skip multiples of 3: ");
for (int i = 1; i <= 10; i++)
{
    if (i % 3 == 0) continue;       // skip this one, carry on
    Console.Write($"{i} ");
}
Console.WriteLine();

int[,] grid = { { 4, 8, 15 }, { 16, 23, 42 } };
Console.WriteLine($"first value > 20 at {Find(grid, 20)}");

static string Find(int[,] g, int limit)
{
    for (int r = 0; r < g.GetLength(0); r++)
        for (int c = 0; c < g.GetLength(1); c++)
            if (g[r, c] > limit) return $"[{r},{c}] = {g[r, c]}";   // return leaves both loops
    return "nowhere";
}
```

OUTPUT

```
break at 5: 1 2 3 4
skip multiples of 3: 1 2 4 5 7 8 10
first value > 20 at [1,1] = 23
```

6.5.2 goto and why to avoid it

`goto label;` jumps to a labelled statement in the same method. Unrestricted jumps make code hard to follow ("spaghetti code"), so `goto` is almost never used in modern C#. The acceptable uses are `goto case` in a switch and, occasionally, escaping from deeply nested loops; even then, a method with `return` is usually clearer.

GOTO LEAVING NESTED LOOPS (WORKS, BUT PREFER A METHOD)

Program.cs

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 3; j++)
    {
        if (i * j == 4) goto Found;          // jump out of both loops
        Console.Write($"{i * j} ");
    }
}
Console.WriteLine("not found");
return;
Found:
Console.WriteLine("\nfound a product equal to 4");
```

OUTPUT

```
1 2 3 2
found a product equal to 4
```

6.6 Classic loop problems

These problems appear in every exam, placement test and first-round interview. Each combines loops with arithmetic tricks: `% 10` gives the last digit of a number, `/ 10` removes it.

6.6.1 Sum of digits, reverse a number, palindrome



EACH STEP: DIGIT = N % 10, N = N / 10

DIGIT PROBLEMS

Program.cs

```
int n = 4721;
int sum = 0, reversed = 0, temp = n;
while (temp > 0)
{
    int digit = temp % 10;            // last digit
    sum += digit;
    reversed = reversed * 10 + digit; // append it to the reversed number
    temp /= 10;                       // drop the last digit
}
Console.WriteLine($"{n}: digit sum {sum}, reversed {reversed}");

foreach (int x in new[] { 12321, 4554, 1230 })
{
    int t = x, rev = 0;
    while (t > 0) { rev = rev * 10 + t % 10; t /= 10; }
    Console.WriteLine($"{x} is {(rev == x ? "" : "not ")}a palindrome");
}
Console.WriteLine($"digits in 9876543: {CountDigits(9876543)}");
static int CountDigits(long v) { int c = 0; do { c++; v /= 10; } while (v != 0); return c; }
```

OUTPUT

```
4721: digit sum 14, reversed 1274
12321 is a palindrome
4554 is a palindrome
1230 is not a palindrome
digits in 9876543: 7
```

6.6.2 Factorial and Fibonacci

The **factorial** n! = 1 × 2 × … × n grows very fast: 20! is the largest that fits in a `long`, so use `System.Numerics.BigInteger` beyond that. In the **Fibonacci** sequence each number is the sum of the two before it: 0, 1, 1, 2, 3, 5, 8 …

FACTORIAL AND FIBONACCI

Program.cs

```
using System.Numerics;

long fact = 1;
for (int i = 1; i <= 10; i++) fact *= i;
Console.WriteLine($"10! = {fact}");

BigInteger big = 1;
for (int i = 1; i <= 30; i++) big *= i;
Console.WriteLine($"30! = {big}");

Console.Write("Fibonacci: ");
long a = 0, b = 1;
for (int i = 0; i < 12; i++)
{
    Console.Write($"{a} ");
    (a, b) = (b, a + b);           // tuple swap: next pair
}
Console.WriteLine();
```

OUTPUT

```
10! = 3628800
30! = 265252859812191058636308480000000
Fibonacci: 0 1 1 2 3 5 8 13 21 34 55 89
```

6.6.3 Prime check and primes in a range

A prime has exactly two divisors, 1 and itself. To test n, it is enough to try divisors up to **√n**: if n = a × b, one of a and b is at most √n. Skipping even numbers after 2 halves the work again. For all primes up to a limit, the **Sieve of Eratosthenes** is much faster: cross out the multiples of each prime.

PRIMES

Program.cs

```
static bool IsPrime(int n)
{
    if (n < 2) return false;
    if (n % 2 == 0) return n == 2;
    for (int d = 3; d * d <= n; d += 2)       // odd divisors up to sqrt(n)
        if (n % d == 0) return false;
    return true;
}

foreach (int x in new[] { 1, 2, 17, 21, 97, 7919 })
    Console.Write($"{x}:{(IsPrime(x) ? "prime" : "no")}  ");
Console.WriteLine();

Console.Write("primes 1-50: ");
for (int i = 1; i <= 50; i++)
    if (IsPrime(i)) Console.Write($"{i} ");
Console.WriteLine();

int limit = 100;                              // Sieve of Eratosthenes
bool[] composite = new bool[limit + 1];
int count = 0;
for (int p = 2; p <= limit; p++)
{
    if (composite[p]) continue;
    count++;
    for (int m = p * p; m <= limit; m += p) composite[m] = true;
}
Console.WriteLine($"primes up to {limit}: {count}");
```

OUTPUT

```
1:no  2:prime  17:prime  21:no  97:prime  7919:prime
primes 1-50: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
primes up to 100: 25
```

6.6.4 Armstrong, perfect and strong numbers

Number type	Definition	Example
Armstrong (narcissistic)	sum of its digits, each raised to the number of digits, equals the number	153 = 1³ + 5³ + 3³
Perfect	sum of its proper divisors equals the number	28 = 1 + 2 + 4 + 7 + 14
Strong	sum of the factorials of its digits equals the number	145 = 1! + 4! + 5!

SPECIAL NUMBERS

Program.cs

```
static bool IsArmstrong(int n)
{
    int digits = n.ToString().Length, sum = 0;
    for (int t = n; t > 0; t /= 10) sum += (int)Math.Pow(t % 10, digits);
    return sum == n;
}
static bool IsPerfect(int n)
{
    int sum = 1;                                  // 1 divides everything
    for (int d = 2; d * d <= n; d++)
        if (n % d == 0) sum += d + (d != n / d ? n / d : 0);   // add both divisors of the pair
    return n > 1 && sum == n;
}
static bool IsStrong(int n)
{
    int[] fact = { 1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880 };
    int sum = 0;
    for (int t = n; t > 0; t /= 10) sum += fact[t % 10];
    return sum == n;
}

Console.Write("Armstrong < 1000: ");
for (int i = 1; i < 1000; i++) if (IsArmstrong(i)) Console.Write($"{i} ");
Console.Write("\nPerfect < 10000: ");
for (int i = 1; i < 10000; i++) if (IsPerfect(i)) Console.Write($"{i} ");
Console.Write("\nStrong < 100000: ");
for (int i = 1; i < 100000; i++) if (IsStrong(i)) Console.Write($"{i} ");
Console.WriteLine();
```

OUTPUT

```
Armstrong < 1000: 1 2 3 4 5 6 7 8 9 153 370 371 407
Perfect < 10000: 6 28 496 8128
Strong < 100000: 1 2 145 40585
```

6.6.5 GCD, LCM and a number-guessing game

The greatest common divisor (GCD) is found quickly with **Euclid's algorithm**: gcd(a, b) = gcd(b, a % b), until b is 0. The least common multiple is lcm(a, b) = a / gcd(a, b) × b (divide first to avoid overflow). The guessing game combines a loop, input validation, comparisons and a counter.

GCD, LCM AND A GUESSING GAME

Program.cs

```
static int Gcd(int a, int b)
{
    while (b != 0) (a, b) = (b, a % b);       // Euclid
    return a;
}
Console.WriteLine($"gcd(84, 36) = {Gcd(84, 36)}, lcm(84, 36) = {84 / Gcd(84, 36) * 36}");
Console.WriteLine($"gcd(17, 5) = {Gcd(17, 5)} (coprime)");

int secret = 37;                              // in a real game: Random.Shared.Next(1, 101)
int tries = 0;
Console.WriteLine("Guess my number (1-100)");
while (true)
{
    Console.Write("Your guess: ");
    if (!int.TryParse(Console.ReadLine(), out int guess)) { Console.WriteLine("Numbers only!"); continue;
}
    tries++;
    if (guess < secret) Console.WriteLine("Too low");
    else if (guess > secret) Console.WriteLine("Too high");
    else { Console.WriteLine($"Correct! {tries} tries"); break; }
}
```

CONSOLE (user input in gold)

```
gcd(84, 36) = 12, lcm(84, 36) = 252
gcd(17, 5) = 1 (coprime)
Guess my number (1-100)
Your guess: 50
Too high
Your guess: 25
Too low
Your guess: forty
Numbers only!
Your guess: 37
Correct! 3 tries
```

Use if / else-if ladders for ranges and complex conditions, switch statements for many fixed values (no fall-through without goto case), and switch expressions with patterns (relational, tuple, type, when guards, _) to map values to results. while tests first, do-while runs at least once, for counts, foreach visits collections. Nested loops build patterns row by row; break leaves a loop, continue skips an iteration, return leaves the method. Digit tricks (% 10, / 10), √n prime tests, Euclid's GCD and the sieve solve the classic problems.

PRACTICE CHECKPOINT

A **menu-driven calculator using a switch expression inside a do-while loop**, plus **five pattern programs**.

SAMPLE SOLUTION

Program.cs

```
string choice;
do
{
    Console.WriteLine("\n1) Add  2) Subtract  3) Multiply  4) Divide  5) Patterns  0) Exit");
    Console.Write("Choice: ");
    choice = Console.ReadLine() ?? "0";
    if (choice is "1" or "2" or "3" or "4")
    {
        double a = ReadNumber("a = "), b = ReadNumber("b = ");
        string result = choice switch
```

```
        {
            "1" => $"{a} + {b} = {a + b}",
            "2" => $"{a} - {b} = {a - b}",
            "3" => $"{a} * {b} = {a * b}",
            "4" when b == 0 => "Cannot divide by zero",
            "4" => $"{a} / {b} = {a / b:F3}",
            _ => "?"
        };
        Console.WriteLine(result);
    }
    else if (choice == "5") Patterns(3);
    else if (choice != "0") Console.WriteLine("Invalid choice");
} while (choice != "0");
Console.WriteLine("Bye!");

static double ReadNumber(string prompt)
{
    double v;
    do Console.Write(prompt); while (!double.TryParse(Console.ReadLine(), out v));
    return v;
}

static void Patterns(int n)
{
    for (int i = 1; i <= n; i++) Console.WriteLine(new string('*', i));                       // 1
right triangle
    for (int i = n; i >= 1; i--) Console.WriteLine(new string('*', i));                       // 2
inverted
    for (int i = 1; i <= n; i++) Console.WriteLine(new string(' ', n - i) + new string('*', i)); // 3
right-aligned
    for (int i = 1; i <= n; i++) Console.WriteLine(new string(' ', n - i) + new string('*', 2 * i - 1));
// 4 pyramid
    for (int i = 1; i <= n; i++)                                                              // 5
hollow square
        Console.WriteLine(i == 1 || i == n ? new string('#', n) : "#" + new string(' ', n - 2) + "#");
}
```

CONSOLE (user input in gold)

```
1) Add  2) Subtract  3) Multiply  4) Divide  5) Patterns  0) Exit
Choice: 4
a = 22
b = 7
22 / 7 = 3.143

1) Add  2) Subtract  3) Multiply  4) Divide  5) Patterns  0) Exit
Choice: 3
a = 2.5
b = x
b = 4
2.5 * 4 = 10

1) Add  2) Subtract  3) Multiply  4) Divide  5) Patterns  0) Exit
Choice: 5
*
**
***
***
**
*
  *
 **
***
  *
 ***
*****
###
# #
###

1) Add  2) Subtract  3) Multiply  4) Divide  5) Patterns  0) Exit
Choice: 0
Bye!
```

MODULE 07 · BEGINNER

Arrays

Minimum duration **3 days** | Level **Beginner**

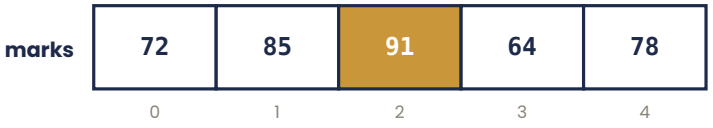
An array stores a fixed number of elements of one type in a single block of memory, accessed by index. This module covers single-dimensional arrays (including C# 12 collection expressions, indices and ranges), rectangular and jagged multi-dimensional arrays, arrays as reference types, the Array class, and the classic searching, sorting and matrix algorithms.

IN THIS MODULE

7.1 Single-dimensional arrays
7.2 Multi-dimensional arrays
7.3 Jagged arrays
7.4 Arrays as reference types
7.5 The Array class
7.6 Array algorithms

7.1 Single-dimensional arrays

Without arrays, storing the marks of 60 students would need 60 variables. An **array** holds many values of the same type under one name; each element is reached by its **index**, starting at 0. Arrays in C# are objects (reference types) and their length is fixed when they are created.



AN INT ARRAY OF LENGTH 5; MARKS[2] IS 91

7.1.1 Declaration and creation int[] a = new int[5];

The type of an array is written with square brackets after the element type: `int[]`, `string[]`, `double[]`. Declaring a variable does not create the array; `new int[5]` allocates five elements on the heap.

DECLARING AND CREATING ARRAYS — Program.cs

```
int[] marks;                // declaration: no array exists yet
marks = new int[5];         // creation: 5 ints, all 0
string[] names = new string[3];
double[] prices = new double[4];
Console.WriteLine($"{marks.Length} {names.Length} {prices.Length}");
marks[0] = 72;              // assign by index
marks[4] = 78;
Console.WriteLine($"first = {marks[0]}, last = {marks[4]}, middle = {marks[2]}");
int size = 3;
var dynamicSize = new char[size * 2];   // the size can be computed at run time
Console.WriteLine(dynamicSize.Length);
```

OUTPUT

```
5 3 4
first = 72, last = 78, middle = 0
6
```

7.1.2 Initialization { 1, 2, 3 } and collection expressions [1, 2, 3] (C# 12)

An **array initializer** creates and fills an array in one step; the length is taken from the number of values. C# 12 added **collection expressions** with square brackets, which work for arrays, lists, spans and other collections, and support the **spread** operator `..` to insert the elements of another collection.

INITIALIZERS AND COLLECTION EXPRESSIONS — Program.cs

```
int[] a = new int[] { 10, 20, 30 };
int[] b = new[] { 1, 2, 3 };            // element type inferred
int[] c = { 5, 6, 7 };                  // shortest form (declarations only)
int[] d = [100, 200, 300];              // C# 12 collection expression
int[] e = [.. c, .. d, 999];            // spread: copy elements of c and d
int[] empty = [];
string[] days = ["Mon", "Tue", "Wed"];
Console.WriteLine(string.Join(" ", a));
Console.WriteLine(string.Join(" ", e));
Console.WriteLine($"empty: {empty.Length}, days: {string.Join(",", days)}");
```

OUTPUT

```
10 20 30
5 6 7 100 200 300 999
empty: 0, days: Mon,Tue,Wed
```

7.1.3 Default values

Unlike local variables, array elements are always initialised automatically to their type's default value: 0 for numbers, `false` for bool, `'\0'` for char, `null` for strings and other reference types.

ELEMENTS START AT DEFAULT VALUES — Program.cs

```
int[] counts = new int[3];
bool[] seen = new bool[3];
string?[] names = new string?[3];
char[] letters = new char[2];
Console.WriteLine($"{counts[0]} {seen[1]} {names[2] ?? "null"} {(int)letters[0]}");
```

OUTPUT

```
0 False null 0
```

7.1.4 Length property

`array.Length` gives the number of elements. Valid indexes run from `0` to `Length - 1`. Length is fixed: to "grow" an array you must create a bigger one and copy (`Array.Resize` does this for you), or use a `List<T>` (Module 14).

7.1.5 Traversal with for and foreach

TRAVERSING AN ARRAY

Program.cs

```
int[] marks = { 72, 85, 91, 64, 78 };
for (int i = 0; i < marks.Length; i++)          // index needed
    Console.Write($"[{i}]={marks[i]} ");
Console.WriteLine();
for (int i = marks.Length - 1; i >= 0; i--)      // backwards
    Console.Write($"{marks[i]} ");
Console.WriteLine();
int total = 0;
foreach (int m in marks) total += m;              // no index needed
Console.WriteLine($"total {total}, average {(double)total / marks.Length}");
for (int i = 0; i < marks.Length; i++) marks[i] += 5;   // modifying needs for
Console.WriteLine(string.Join(", ", marks));
```

OUTPUT

```
[0]=72 [1]=85 [2]=91 [3]=64 [4]=78
78 64 91 85 72
total 390, average 78
77, 90, 96, 69, 83
```

7.1.6 IndexOutOfRangeException

Every array access is bounds-checked at run time. Using an index below 0 or at or above `Length` throws `IndexOutOfRangeException` instead of silently reading other memory, as happens in C. The classic cause is `i <= a.Length` instead of `i < a.Length`.

GOING PAST THE END

Program.cs

```
int[] a = { 1, 2, 3 };
try
{
    for (int i = 0; i <= a.Length; i++)          // bug: <= goes one too far
        Console.Write($"{a[i]} ");
}
catch (IndexOutOfRangeException e)
{
    Console.WriteLine($"\n{e.GetType().Name}: {e.Message}");
}
```

OUTPUT

```
1 2 3
IndexOutOfRangeException: Index was outside the bounds of the array.
```

7.1.7 Indices ^1 and ranges a[1..3]

As introduced in 4.7.4, `a[^1]` is the last element and `a[^k]` the k-th from the end; `a[x..y]` returns a **new array** containing the elements from x up to, but not including, y. Ranges are handy for "all but the first", "the last three" and splitting arrays.

INDICES AND RANGES ON ARRAYS

Program.cs

```
int[] a = [10, 20, 30, 40, 50, 60];
Console.WriteLine($"last {a[^1]}, second last {a[^2]}");
int[] firstThree = a[..3];
int[] lastTwo = a[^2..];
int[] middle = a[1..^1];
Console.WriteLine($"[{string.Join(",", firstThree)}] [{string.Join(",", lastTwo)}] [{string.Join(",",
middle)}]");
middle[0] = 999;                                    // a range on an array is a COPY
Console.WriteLine($"a[1] is still {a[1]}");
```

OUTPUT

```
last 60, second last 50
[10,20,30] [50,60] [20,30,40,50]
a[1] is still 20
```

7.2 Multi-dimensional arrays

7.2.1 Rectangular arrays int[,]

A **rectangular array** has two or more dimensions with a fixed size in each: `int[,]` is a table (rows × columns), `int[,,]` a cube. Elements are accessed with comma-separated indexes: `grid[row, col]`. All rows have the same length, and the whole table is one object.

	[0]	[1]	[2]	[3]
[0]	90	85	78	92
[1]	66	74	81	70
[2]	88	95	91	84

A 3 × 4 RECTANGULAR ARRAY; MARKS[1, 2] = 81

A TABLE OF MARKS Program.cs

```
int[,] marks =
{
    { 90, 85, 78, 92 },        // student 0
    { 66, 74, 81, 70 },        // student 1
    { 88, 95, 91, 84 }         // student 2
};
Console.WriteLine($"marks[1, 2] = {marks[1, 2]}");
marks[1, 2] = 83;

string[] subjects = { "Maths", "Science", "English", "History" };
Console.Write($"{"",-10}");
foreach (string s in subjects) Console.Write($"{s,9}");
Console.WriteLine($"{"Total",8}");
for (int r = 0; r < marks.GetLength(0); r++)
{
    Console.Write($"{"Student " + r,-10}");
    int total = 0;
    for (int c = 0; c < marks.GetLength(1); c++)
    {
        Console.Write($"{marks[r, c],9}");
        total += marks[r, c];
    }
    Console.WriteLine($"{total,8}");
}
```

OUTPUT

```
marks[1, 2] = 81
            Maths  Science  English  History   Total
Student 0      90       85       78       92     345
Student 1      66       74       83       70     293
Student 2      88       95       91       84     358
```

7.2.2 GetLength() and Rank

For multi-dimensional arrays, `Length` is the **total** number of elements; `GetLength(d)` gives the size of dimension d (0 = rows, 1 = columns), and `Rank` the number of dimensions. `foreach` visits all elements row by row.

DIMENSIONS OF RECTANGULAR ARRAYS Program.cs

```
int[,] grid = new int[3, 4];
int[,,] cube = new int[2, 3, 5];
Console.WriteLine($"grid: Rank {grid.Rank}, Length {grid.Length}, rows {grid.GetLength(0)}, cols
{grid.GetLength(1)}");
Console.WriteLine($"cube: Rank {cube.Rank}, Length {cube.Length}");
int[,] small = { { 1, 2 }, { 3, 4 } };
foreach (int v in small) Console.Write($"{v} ");      // row by row
Console.WriteLine();
```

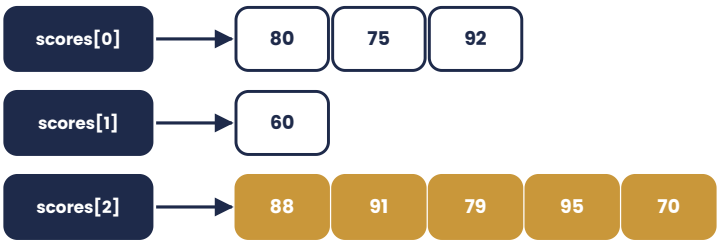
OUTPUT

```
grid: Rank 2, Length 12, rows 3, cols 4
cube: Rank 3, Length 30
1 2 3 4
```

7.3 Jagged arrays

7.3.1 Arrays of arrays int[][]

A **jagged array** is an array whose elements are themselves arrays, so each row can have a **different length**: `int[][]`. Each row is created separately, and elements are accessed as `a[row][col]`. Typical uses: marks of students who took different numbers of tests, triangles (such as Pascal's), or the rows of a CSV file.



A JAGGED ARRAY: EACH ROW IS A SEPARATE ARRAY OF ITS OWN LENGTH

JAGGED ARRAYS

Program.cs

```
int[][] scores = new int[3][];          // 3 rows, not created yet
scores[0] = new int[] { 80, 75, 92 };
scores[1] = new int[] { 60 };
scores[2] = [88, 91, 79, 95, 70];

for (int r = 0; r < scores.Length; r++)
{
    Console.Write($"student {r} ({scores[r].Length} tests): ");
    foreach (int s in scores[r]) Console.Write($"{s} ");
    Console.WriteLine($"| best {scores[r].Max()}");
}

int[][] triangle = new int[4][];        // rows of length 1, 2, 3, 4
for (int i = 0; i < triangle.Length; i++)
{
    triangle[i] = new int[i + 1];
    triangle[i][0] = triangle[i][i] = 1;
    for (int j = 1; j < i; j++) triangle[i][j] = triangle[i - 1][j - 1] + triangle[i - 1][j];
}
foreach (int[] row in triangle) Console.WriteLine(string.Join(" ", row));
```

OUTPUT

```
student 0 (3 tests): 80 75 92 | best 92
student 1 (1 tests): 60 | best 60
student 2 (5 tests): 88 91 79 95 70 | best 95
1
1 1
1 2 1
1 3 3 1
```

NOTE

`Max()` is a LINQ method (namespace `System.Linq`, imported by the implicit usings). Module 16 covers LINQ.

7.3.2 Rectangular vs jagged comparison

	Rectangular `int[,]`	Jagged `int[][]`
Shape	every row the same length	rows can differ

	Rectangular `int[,]`	Jagged `int[][]`
Access	`a[r, c]`	`a[r][c]`
Objects	one object	one array per row + the outer array
Creation	`new int[3, 4]`	`new int[3][]`, then each row
Row as an array	not directly	`a[r]` is an `int[]`
Performance	simple, compact	often faster to iterate in .NET; rows can be swapped cheaply
Use for	grids, matrices, boards	uneven data, triangles, lines of a file

7.4 Arrays as reference types

7.4.1 Passing arrays to methods

An array variable holds a **reference**. Passing it to a method passes a copy of the reference, so the method works on the **same** array: changes to elements are visible to the caller. Assigning a new array to the parameter inside the method, however, only changes the method's local copy of the reference.

ARRAYS ARE PASSED BY REFERENCE VALUE

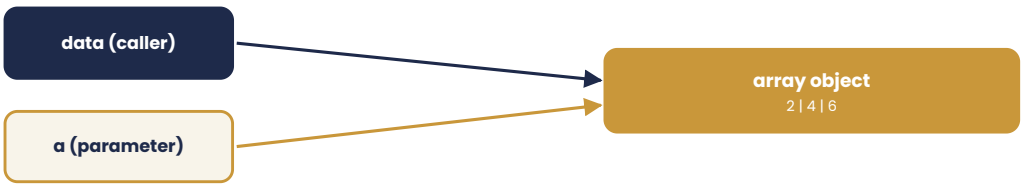
Program.cs

```
int[] data = { 1, 2, 3 };
Double(data);
Console.WriteLine(string.Join(" ", data));       // changed!
Replace(data);
Console.WriteLine(string.Join(" ", data));       // not replaced

static void Double(int[] a)
{
    for (int i = 0; i < a.Length; i++) a[i] *= 2;   // modifies the caller's array
}
static void Replace(int[] a)
{
    a = new int[] { 99, 99, 99 };                   // only the local reference changes
}
```

OUTPUT

```
2 4 6
2 4 6
```



BOTH REFERENCES POINT TO THE SAME ARRAY OBJECT

7.4.2 Copying: Clone(), Array.Copy(), CopyTo()

Because `b = a` copies only the reference, a real copy needs one of: `a.Clone()` (returns `object`, so cast it), `Array.Copy(source, dest, length)` or `a.CopyTo(dest, index)` into an existing array, a range `a[..]`, or LINQ's `a.ToArray()`. All of these are **shallow** copies: for an array of objects, the objects themselves are shared.

COPYING ARRAYS Program.cs

```
int[] original = { 1, 2, 3, 4, 5 };
int[] alias = original;                        // same array
int[] clone = (int[])original.Clone();         // new array
int[] partial = new int[3];
Array.Copy(original, 1, partial, 0, 3);        // copy 3 elements starting at index 1
int[] bigger = new int[7];
original.CopyTo(bigger, 2);                    // copy into position 2
int[] viaRange = original[..];

original[0] = 100;
Console.WriteLine($"alias[0] = {alias[0]}, clone[0] = {clone[0]}, viaRange[0] = {viaRange[0]}");
Console.WriteLine($"partial: {string.Join(" ", partial)}");
Console.WriteLine($"bigger:  {string.Join(" ", bigger)}");

string[][] rows = { new[] { "a" }, new[] { "b" } };
var shallow = (string[][])rows.Clone();
shallow[0][0] = "changed";                     // inner arrays are shared
Console.WriteLine($"rows[0][0] = {rows[0][0]}");
```

OUTPUT

```
alias[0] = 100, clone[0] = 1, viaRange[0] = 1
partial: 2 3 4
bigger:  0 0 1 2 3 4 5
rows[0][0] = changed
```

7.5 The Array class

7.5.1 Array.Sort, Reverse, IndexOf, BinarySearch, Resize, Fill, Exists, Find

`System.Array` is the base class of all arrays and offers static helper methods. Several take a **predicate**, a function that returns true or false, usually written as a lambda such as `x => x > 90` (Module 15).

Method	Purpose
`Array.Sort(a)`	sort ascending in place (introsort, O(n log n))
`Array.Reverse(a)`	reverse the order in place
`Array.IndexOf(a, x)` / `LastIndexOf`	position of x, or -1
`Array.BinarySearch(a, x)`	O(log n) search in a **sorted** array; negative if not found
`Array.Resize(ref a, n)`	create a new array of size n and copy
`Array.Fill(a, v)`	set every element to v
`Array.Exists(a, pred)`	is there an element matching the predicate?
`Array.Find` / `FindAll` / `FindIndex`	first match / all matches / index of first match
`Array.TrueForAll(a, pred)`	do all elements match?
`Array.Clear(a)`	reset elements to default values

ARRAY HELPER METHODS

Program.cs

```
int[] a = { 42, 7, 19, 88, 3, 56 };
Array.Sort(a);
Console.WriteLine($"sorted:   {string.Join(" ", a)}");
Array.Reverse(a);
Console.WriteLine($"reversed: {string.Join(" ", a)}");
Console.WriteLine($"IndexOf(19) = {Array.IndexOf(a, 19)}, IndexOf(5) = {Array.IndexOf(a, 5)}");

Array.Sort(a);
int pos = Array.BinarySearch(a, 56);
int missing = Array.BinarySearch(a, 50);
Console.WriteLine($"BinarySearch(56) = {pos}, BinarySearch(50) = {missing} (insert at {~missing})");

Array.Resize(ref a, 8);
Console.WriteLine($"resized:  {string.Join(" ", a)}");
int[] zeros = new int[5];
Array.Fill(zeros, 7);
Console.WriteLine($"filled:   {string.Join(" ", zeros)}");

int[] marks = { 72, 95, 38, 88, 61 };
Console.WriteLine($"any fail? {Array.Exists(marks, m => m < 40)}, all pass? {Array.TrueForAll(marks, m =>
m >= 40)}");
Console.WriteLine($"first > 80: {Array.Find(marks, m => m > 80)}, all > 80: {string.Join(",",
Array.FindAll(marks, m => m > 80))}, index of 38: {Array.FindIndex(marks, m => m == 38)}");
string[] names = { "ravi", "Asha", "meera", "Kiran" };
Array.Sort(names, StringComparer.OrdinalIgnoreCase);
Console.WriteLine(string.Join(" ", names));
```

OUTPUT

```
sorted:   3 7 19 42 56 88
reversed: 88 56 42 19 7 3
IndexOf(19) = 3, IndexOf(5) = -1
BinarySearch(56) = 4, BinarySearch(50) = -5 (insert at 4)
resized:  3 7 19 42 56 88 0 0
filled:   7 7 7 7 7
any fail? True, all pass? False
first > 80: 95, all > 80: 95,88, index of 38: 2
Asha Kiran meera ravi
```

NOTE

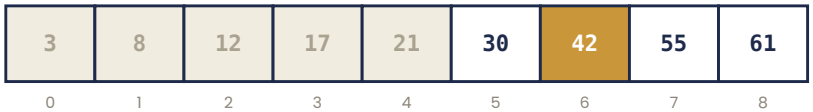
When `BinarySearch` does not find the value, it returns the bitwise complement of the index where the value would be inserted; `~missing` recovers that index (50 would go before 56, at index 4).

7.6 Array algorithms

Even with built-in methods available, implementing the basic algorithms yourself is essential practice and a standard interview requirement.

7.6.1 Linear and binary search

Linear search checks each element in turn: O(n), works on any array. **Binary search** needs a **sorted** array: compare with the middle element and discard the half that cannot contain the value, so it needs only O(log n) steps: at most 20 for a million elements.



BINARY SEARCH FOR 42: AFTER COMPARING WITH 21 (MIDDLE), THE LEFT HALF IS DISCARDED

LINEAR AND BINARY SEARCH

Program.cs

```
int[] sorted = { 3, 8, 12, 17, 21, 30, 42, 55, 61 };
Console.WriteLine($"linear: index of 42 = {LinearSearch(sorted, 42)}");
Console.WriteLine($"binary: index of 42 = {BinarySearch(sorted, 42)}, of 5 = {BinarySearch(sorted, 5)}");

static int LinearSearch(int[] a, int target)
{
    for (int i = 0; i < a.Length; i++)
        if (a[i] == target) return i;
    return -1;
}

static int BinarySearch(int[] a, int target)
{
    int lo = 0, hi = a.Length - 1, step = 0;
    while (lo <= hi)
    {
        int mid = lo + (hi - lo) / 2;            // avoids overflow of lo + hi
        Console.WriteLine($"  step {++step}: lo={lo} hi={hi} mid={mid} a[mid]={a[mid]}");
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1;       // target is in the right half
        else hi = mid - 1;                       // target is in the left half
    }
    return -1;
}
```

OUTPUT

```
linear: index of 42 = 6
  step 1: lo=0 hi=8 mid=4 a[mid]=21
  step 2: lo=5 hi=8 mid=6 a[mid]=42
  step 1: lo=0 hi=8 mid=4 a[mid]=21
  step 2: lo=0 hi=3 mid=1 a[mid]=8
  step 3: lo=0 hi=0 mid=0 a[mid]=3
binary: index of 42 = 6, of 5 = -1
```

NOTE

The step lines of both searches are printed before the "binary:" line, because both method calls inside the interpolated string are evaluated before that string is written.

7.6.2 Bubble, selection and insertion sort

The three simple O(n²) sorts each build the sorted result step by step. **Bubble sort** repeatedly swaps adjacent elements that are out of order, so the largest "bubbles" to the end in each pass. **Selection sort** finds the minimum of the unsorted part and swaps it into place. **Insertion sort** takes the next element and shifts it left into its place among the already-sorted elements, like sorting playing cards; it is fast for nearly sorted data and is used inside real-world sorts for small pieces.

THREE SIMPLE SORTS

Program.cs

```
int[] data = { 29, 10, 14, 37, 13 };

int[] b = (int[])data.Clone();                      // bubble sort
for (int pass = 0; pass < b.Length - 1; pass++)
{
    bool swapped = false;
    for (int j = 0; j < b.Length - 1 - pass; j++)
        if (b[j] > b[j + 1]) { (b[j], b[j + 1]) = (b[j + 1], b[j]); swapped = true; }
    Console.WriteLine($"bubble pass {pass + 1}: {string.Join(" ", b)}");
    if (!swapped) break;                             // already sorted: stop early
}

int[] s = (int[])data.Clone();                      // selection sort
for (int i = 0; i < s.Length - 1; i++)
{
    int min = i;
    for (int j = i + 1; j < s.Length; j++) if (s[j] < s[min]) min = j;
    (s[i], s[min]) = (s[min], s[i]);
}
Console.WriteLine($"selection: {string.Join(" ", s)}");

int[] ins = (int[])data.Clone();                    // insertion sort
for (int i = 1; i < ins.Length; i++)
{
    int key = ins[i], j = i - 1;
    while (j >= 0 && ins[j] > key) { ins[j + 1] = ins[j]; j--; }   // shift bigger elements right
    ins[j + 1] = key;
    Console.WriteLine($"insert {key,2}: {string.Join(" ", ins)}");
}
```

OUTPUT

```
bubble pass 1: 10 14 29 13 37
bubble pass 2: 10 14 13 29 37
bubble pass 3: 10 13 14 29 37
bubble pass 4: 10 13 14 29 37
selection: 10 13 14 29 37
insert 10: 10 29 14 37 13
insert 14: 10 14 29 37 13
insert 37: 10 14 29 37 13
insert 13: 10 13 14 29 37
```

Algorithm	Best	Average / worst	Stable	Notes
Bubble	O(n) with early exit	O(n²)	yes	simplest, slowest in practice
Selection	O(n²)	O(n²)	no	fewest swaps (n - 1)
Insertion	O(n)	O(n²)	yes	best of the three; great on nearly sorted data
Array.Sort	O(n log n)	O(n log n)	no	introsort: use it in real code

7.6.3 Common problems: max/min, second largest, reverse, rotate, remove duplicates, frequency count

CLASSIC ARRAY PROBLEMS

Program.cs

```
int[] a = { 12, 45, 7, 45, 23, 7, 89, 12, 3 };

int max = a[0], min = a[0];
foreach (int x in a) { if (x > max) max = x; if (x < min) min = x; }
Console.WriteLine($"max {max}, min {min}");

int first = int.MinValue, second = int.MinValue;          // second largest, one pass
foreach (int x in a)
{
    if (x > first) { second = first; first = x; }
    else if (x > second && x != first) second = x;
}
Console.WriteLine($"second largest {second}");

int[] r = (int[])a.Clone();                               // reverse in place with two indexes
for (int i = 0, j = r.Length - 1; i < j; i++, j--) (r[i], r[j]) = (r[j], r[i]);
Console.WriteLine($"reversed: {string.Join(" ", r)}");

int k = 3;                                                // rotate left by k
int[] rot = new int[a.Length];
for (int i = 0; i < a.Length; i++) rot[i] = a[(i + k) % a.Length];
Console.WriteLine($"rotated left by {k}: {string.Join(" ", rot)}");

int[] sorted = (int[])a.Clone();                          // remove duplicates from a sorted copy
Array.Sort(sorted);
int unique = 1;
for (int i = 1; i < sorted.Length; i++)
    if (sorted[i] != sorted[unique - 1]) sorted[unique++] = sorted[i];
Console.WriteLine($"unique: {string.Join(" ", sorted[..unique])}");

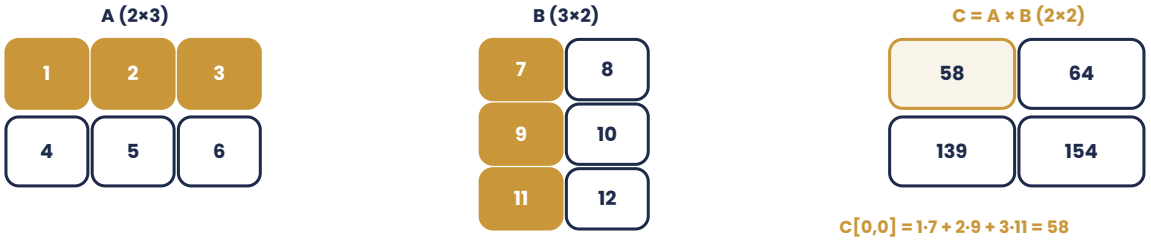
var freq = new Dictionary<int, int>();                    // frequency count
foreach (int x in a) freq[x] = freq.GetValueOrDefault(x) + 1;
foreach (var (value, count) in freq)
    if (count > 1) Console.Write($"{value} x{count}  ");
Console.WriteLine();
```

OUTPUT

```
max 89, min 3
second largest 45
reversed: 3 12 89 7 23 45 7 45 12
rotated left by 3: 45 23 7 89 12 3 12 45 7
unique: 3 7 12 23 45 89
12 x2  45 x2  7 x2
```

7.6.4 Matrix addition, multiplication and transpose

Matrices are stored in rectangular arrays. **Addition** adds corresponding elements (same sizes required). **Multiplication** of an m×n matrix A by an n×p matrix B gives an m×p matrix where C[i, j] is the sum over k of A[i, k] × B[k, j]: three nested loops, O(n³). The **transpose** swaps rows and columns.



ROW 0 OF A TIMES COLUMN 0 OF B GIVES C[0, 0]

MATRIX OPERATIONS

Program.cs

```
int[,] A = { { 1, 2, 3 }, { 4, 5, 6 } };
int[,] B = { { 7, 8 }, { 9, 10 }, { 11, 12 } };
int[,] S = { { 1, 1, 1 }, { 2, 2, 2 } };

Print("A + S", Add(A, S));
Print("A x B", Multiply(A, B));
Print("transpose(A)", Transpose(A));

static int[,] Add(int[,] x, int[,] y)
{
    int[,] r = new int[x.GetLength(0), x.GetLength(1)];
    for (int i = 0; i < x.GetLength(0); i++)
        for (int j = 0; j < x.GetLength(1); j++)
            r[i, j] = x[i, j] + y[i, j];
    return r;
}
static int[,] Multiply(int[,] x, int[,] y)
{
    int m = x.GetLength(0), n = x.GetLength(1), p = y.GetLength(1);
    if (y.GetLength(0) != n) throw new ArgumentException("columns of x must equal rows of y");
    int[,] r = new int[m, p];
    for (int i = 0; i < m; i++)
        for (int j = 0; j < p; j++)
            for (int k = 0; k < n; k++)
                r[i, j] += x[i, k] * y[k, j];
    return r;
}
static int[,] Transpose(int[,] x)
{
    int[,] t = new int[x.GetLength(1), x.GetLength(0)];
    for (int i = 0; i < x.GetLength(0); i++)
        for (int j = 0; j < x.GetLength(1); j++)
            t[j, i] = x[i, j];
    return t;
}
static void Print(string title, int[,] m)
{
    Console.WriteLine(title + ":");
    for (int i = 0; i < m.GetLength(0); i++)
    {
        for (int j = 0; j < m.GetLength(1); j++) Console.Write($"{m[i, j],5}");
        Console.WriteLine();
    }
}
```

OUTPUT

```
A + S:
    2    3    4
    6    7    8
A x B:
   58   64
  139  154
transpose(A):
    1    4
    2    5
    3    6
```

Arrays hold a fixed number of same-type elements indexed from 0 to Length - 1, with automatic default values and bounds checking. Create them with new T[n], initializers or [ ] collection expressions; use ^ and .. for end-relative indexes and range copies. Rectangular arrays (T[,]) are grids; jagged arrays (T[][]) allow rows of different lengths. Arrays are reference types: assignment and parameters share the array, so copy with Clone, Array.Copy or ranges. Prefer Array.Sort and BinarySearch in real code, but know linear and binary search and the simple sorts.

PRACTICE CHECKPOINT

Read **n numbers**, sort them with **insertion sort**, **binary-search** for a value, and **multiply two matrices** using rectangular arrays.

SAMPLE SOLUTION

Program.cs

```
Console.Write("How many numbers? ");
int n = int.Parse(Console.ReadLine()!);
int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    Console.Write($"  number {i + 1}: ");
    a[i] = int.Parse(Console.ReadLine()!);
}

for (int i = 1; i < a.Length; i++)                  // insertion sort
{
    int key = a[i], j = i - 1;
    while (j >= 0 && a[j] > key) { a[j + 1] = a[j]; j--; }
    a[j + 1] = key;
}
Console.WriteLine($"sorted: {string.Join(" ", a)}");

Console.Write("Search for: ");
int target = int.Parse(Console.ReadLine()!);
int lo = 0, hi = a.Length - 1, found = -1;
while (lo <= hi && found < 0)                        // binary search
{
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == target) found = mid;
    else if (a[mid] < target) lo = mid + 1;
    else hi = mid - 1;
}
Console.WriteLine(found >= 0 ? $"{target} found at index {found}" : $"{target} not found");

int[,] x = { { 2, 0, 1 }, { 1, 3, 2 } };             // 2 x 3
int[,] y = { { 1, 2 }, { 0, 1 }, { 4, 0 } };         // 3 x 2
int[,] product = new int[2, 2];
for (int i = 0; i < 2; i++)
    for (int j = 0; j < 2; j++)
        for (int k = 0; k < 3; k++)
            product[i, j] += x[i, k] * y[k, j];
Console.WriteLine("matrix product:");
for (int i = 0; i < 2; i++) Console.WriteLine($"{product[i, 0],4}{product[i, 1],4}");
```

CONSOLE (user input in gold)

```
How many numbers? 6
  number 1: 34
  number 2: 7
  number 3: 23
  number 4: 32
  number 5: 5
  number 6: 62
sorted: 5 7 23 32 34 62
Search for: 32
32 found at index 3
matrix product:
   6   4
   9   5
```

MODULE 08 · BEGINNER

Strings

Minimum duration **2 days** | Level **Beginner**

Text is everywhere: names, messages, file contents, web pages. A C# string is an immutable sequence of UTF-16 characters. This module covers how strings are stored, the many ways to write string literals, comparing strings correctly, the built-in string methods, efficient building with StringBuilder, character tests, a first look at regular expressions, and the classic string problems asked in interviews.

IN THIS MODULE

8.1 String basics
8.2 Comparing strings
8.3 String methods
8.4 StringBuilder
8.5 Characters
8.6 Regular expressions (introduction)
8.7 Classic string problems

8.1 String basics

A **string** is an ordered sequence of `char` values. Each `char` is a 16-bit UTF-16 code unit, so "Hello" is five chars stored one after another inside a string object on the heap. Strings are reference types, but they behave in many ways like values: they are compared by content and can never be changed after creation.



STRING S = "HELLO C#": EIGHT CHARS INDEXED FROM 0 TO 7

8.1.1 string vs System.String

`string` (lowercase) is a C# **keyword alias** for the .NET type `System.String`, exactly as `int` is an alias for `System.Int32`. Both names refer to the same type, so the choice is about style. The common convention, followed by the .NET team, is to use `string` for variables and parameters and `String` (or `string`) when calling static members such as `string.Join`.

STRING AND STRING ARE THE SAME TYPE Program.cs

```
string a = "hello";
String b = "hello";                    // needs using System; (implicit in modern projects)
Console.WriteLine(a.GetType());        // the real .NET type
Console.WriteLine(typeof(string) == typeof(String));
Console.WriteLine(a == b);

string empty = string.Empty;           // same as ""
string? nothing = null;                // no string object at all
Console.WriteLine($"[{empty}] length {empty.Length}, nothing is null: {nothing is null}");
```

OUTPUT

```
System.String
True
True
[] length 0, nothing is null: True
```

NOTE

An **empty string** "" is a real object with Length 0. A **null** string refers to no object; calling `nothing.Length` would throw a NullReferenceException. With nullable reference types enabled, the compiler warns when a `string?` might be null.

8.1.2 Immutability and string interning

Strings are **immutable**: once created, their characters can never change. Every method that seems to modify a string, such as `ToUpper`, `Replace` or `+=`, actually builds and returns a **new** string and leaves the original untouched. This makes strings safe to share between variables and threads and allows them to be used as dictionary keys.

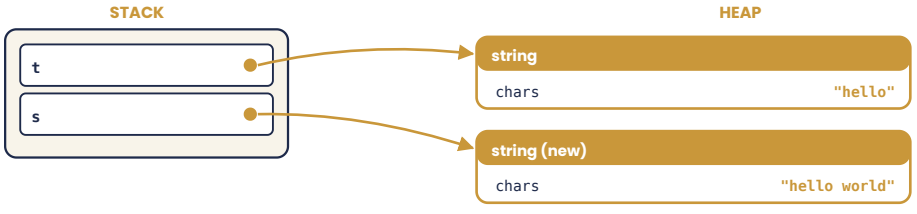
STRINGS NEVER CHANGE; METHODS RETURN NEW STRINGS Program.cs

```
string s = "hello";
string t = s;                  // t and s refer to the same object
s += " world";                 // creates a NEW string; s now refers to it
Console.WriteLine($"s = {s}");
Console.WriteLine($"t = {t}");   // unchanged

string name = "asha";
name.ToUpper();                // result thrown away: name does not change
Console.WriteLine(name);
name = name.ToUpper();         // keep the new string
Console.WriteLine(name);
```

OUTPUT

```
s = hello world
t = hello
asha
ASHA
```



AFTER S += " WORLD", S POINTS TO A NEW STRING; T STILL POINTS TO THE ORIGINAL

Because strings never change, the runtime can safely share one copy of identical text. The **intern pool** is a table of unique strings kept by the CLR. Every string **literal** in your program is interned automatically, so two variables initialised with the same literal refer to the same object. Strings built at run time (from input, concatenation of variables or `new string(...)`) are not interned unless you call `string.Intern`.

LITERALS ARE INTERNED Program.cs

```
string a = "LBF";
string b = "LBF";                                  // same literal: same interned object
string c = new string(new[] { 'L', 'B', 'F' });    // built at run time: a separate object

Console.WriteLine($"a == c: {a == c}");                                  // compares content
Console.WriteLine($"same object a, b: {ReferenceEquals(a, b)}");
Console.WriteLine($"same object a, c: {ReferenceEquals(a, c)}");

string d = string.Intern(c);                       // look up (or add) c in the pool
Console.WriteLine($"same object a, d: {ReferenceEquals(a, d)}");
```

OUTPUT

```
a == c: True
same object a, b: True
same object a, c: False
same object a, d: True
```

TIP

Interning is an automatic memory optimisation, not something to rely on in program logic. Always compare strings with `==` or `Equals`, which compare content, never with `ReferenceEquals`.

8.1.3 Escape sequences, verbatim strings and raw string literals

Inside a normal string literal, the backslash starts an **escape sequence** that stands for a character that is hard to type or would otherwise end the string.

Escape	Meaning	Escape	Meaning
`\n`	new line	`\\`	backslash
`\t`	horizontal tab	`\"`	double quote
`\r`	carriage return	`\'`	single quote (in char literals)
`\0`	null character	`\u0041`	UTF-16 code unit in hex (A)
`\a`	alert (beep)	`\e`	escape character (C# 13)

A **verbatim string** starts with `@`. Backslashes are taken literally and the string may span several lines, which is ideal for Windows paths and regular expressions; a double quote is written as two double quotes `""`. A **raw string literal** (C# 11) starts and ends with three or more double quotes. Nothing inside it is escaped, so it can contain quotes, backslashes and JSON freely; the indentation of the closing `"""` is removed from every line.

THREE KINDS OF STRING LITERAL Program.cs

```
string path1 = "C:\\Users\\asha\\notes.txt";        // escaped backslashes
string path2 = @"C:\Users\asha\notes.txt";          // verbatim: no escaping needed
Console.WriteLine(path1 == path2);
Console.WriteLine(path2);

Console.WriteLine("She said \"hi\"\nNext line \u2713");
Console.WriteLine(@"Verbatim ""quotes"" are doubled");

string json = """
    {
      "name": "Asha",
      "path": "C:\data"
    }
    """;
Console.WriteLine(json);
```

OUTPUT

```
True
C:\Users\asha\notes.txt
She said "hi"
Next line ✓
Verbatim "quotes" are doubled
{
  "name": "Asha",
  "path": "C:\data"
}
```

Raw strings can be interpolated too. With `$"""` the braces `{x}` are holes; when the text itself contains braces (JSON, CSS), use `$$"""` so that holes need two braces `{{x}}` and single braces are ordinary text.

INTERPOLATED RAW STRING Program.cs

```
string name = "Ravi";
int age = 22;
string json = $$"""
    { "name": "{{name}}", "age": {{age}} }
    """;
Console.WriteLine(json);
```

OUTPUT

```
{ "name": "Ravi", "age": 22 }
```

8.1.4 String interpolation in depth

An **interpolated string** starts with `$`; each hole `{expression}` is evaluated and converted to text. A hole can also carry an **alignment** (a field width; negative means left-aligned) and a **format string**: `{value,alignment:format}`. Module 5 introduced the format specifiers; here is the full hole syntax in one place.

THE PARTS OF AN INTERPOLATION HOLE

Program.cs

```
using System.Globalization;
CultureInfo.CurrentCulture = new CultureInfo("en-US");

string item = "Notebook";
double price = 1249.5;
int qty = 3;
DateTime d = new DateTime(2026, 3, 9);

Console.WriteLine($"{item} x{qty} = {price * qty:N2}");            // expression + format
Console.WriteLine($"[{item,12}] [{item,-12}]");                      // alignment
Console.WriteLine($"[{price,10:F1}] {0.183:P1} {255:X4}");           // alignment + format
Console.WriteLine($"Status: {(qty > 2 ? "bulk" : "single")}");       // conditional needs ( )
Console.WriteLine($"{d:dd MMM yyyy} was a {d:dddd}");
Console.WriteLine($"Braces: {{literal}} and {qty}");                 // {{ }} = a literal brace
Console.WriteLine($"Upper: {item.ToUpper()}, length {item.Length}");
```

OUTPUT

```
Notebook x3 = 3,748.50
[    Notebook] [Notebook    ]
[    1249.5] 18.3% 00FF
Status: bulk
09 Mar 2026 was a Monday
Braces: {literal} and 3
Upper: NOTEBOOK, length 8
```

Syntax	Meaning	Example result
`{x}`	value of x via ToString()	`3`
`{x,8}`	right-align in 8 characters	`" 3"`
`{x,-8}`	left-align in 8 characters	`"3 "`
`{x:N2}`	format string after the colon	`3.00`
`{x,8:N2}`	alignment then format	`" 3.00"`
`{{ }}`	literal braces	`{ }`
`{(c ? a : b)}`	a ternary must be in parentheses (the colon would start a format)	

NOTE

Interpolation is compiled into efficient code (an interpolated string handler) that avoids boxing and extra allocations, so it is both the most readable and one of the fastest ways to build a string from parts. Since C# 11, holes may also span multiple lines.

8.2 Comparing strings

Comparing text looks easy but hides traps: upper versus lower case, the sort order of different languages, and the difference between "same text" and "same object". C# gives two kinds of comparison: **equality** (are they the same?) and **ordering** (which comes first?).

8.2.1 == and Equals()

For strings, `==` is overloaded to compare **content**, character by character, case-sensitively (ordinal). `a.Equals(b)` does the same. The static `string.Equals(a, b)` is safe when either side may be null, and the overloads that take a `StringComparison` let you ignore case.

EQUALITY OF STRINGS

Program.cs

```
string a = "Hello";
string b = "hello";
string? n = null;

Console.WriteLine(a == b);                                          // case matters
Console.WriteLine(a.Equals(b));
Console.WriteLine(a.Equals(b, StringComparison.OrdinalIgnoreCase));     // ignore case
Console.WriteLine(string.Equals(n, a));                             // null-safe
Console.WriteLine(n == null);

object o1 = a;
object o2 = new string("Hello".ToCharArray());
Console.WriteLine(o1 == o2);              // object == object: compares REFERENCES
Console.WriteLine(o1.Equals(o2));         // virtual Equals: compares content
```

OUTPUT

```
False
False
True
False
True
False
True
```

WATCH OUT

When strings are held in variables of type `object`, `==` compares references, not content (the compiler warns when one side is typed `string` and the other `object`). Use `Equals` in that situation, or keep the static type `string`. Also avoid `a.ToLower() == b.ToLower()` for case-insensitive checks: it allocates two new strings and is wrong for some languages; use `StringComparison.OrdinalIgnoreCase`.

8.2.2 string.Compare(), CompareTo()

To find out which string comes first (for sorting or searching), use `string.Compare(a, b)` or `a.CompareTo(b)`. The result is an `int`: **negative** if a comes before b, **zero** if equal, **positive** if a comes after b. Only the sign is meaningful; never test for exactly -1 or 1.

ORDERING WITH COMPARE AND COMPARETO

Program.cs

```
using System.Globalization;
CultureInfo.CurrentCulture = new CultureInfo("en-US");

Console.WriteLine(Math.Sign(string.Compare("apple", "banana")));      // a before b
Console.WriteLine(Math.Sign(string.Compare("cherry", "banana")));     // c after b
Console.WriteLine(Math.Sign("kiwi".CompareTo("kiwi")));               // equal
Console.WriteLine(Math.Sign(string.Compare("apple", "Apple")));       // culture: lower first
Console.WriteLine(string.Compare("apple", "APPLE", ignoreCase: true));

string x = "mango", y = "melon";
if (string.Compare(x, y) < 0) Console.WriteLine($"{x} comes before {y}");
```

OUTPUT

```
-1
1
0
-1
0
mango comes before melon
```

8.2.3 StringComparison options (ordinal, ignore case, culture)

There are two ways to compare text. **Ordinal** comparison compares the numeric char codes, which is fast, exact and the same on every machine; uppercase letters (65 to 90) come before lowercase (97 to 122). **Culture-sensitive** comparison follows the alphabet rules of a language, where "apple" and "Apple" sort next to each other. Many methods accept a `StringComparison` value to choose.

StringComparison	Compares	Use for
`Ordinal`	char codes exactly	identifiers, keys, file names, protocols, passwords
`OrdinalIgnoreCase`	char codes, ignoring case	user names, commands, file extensions, HTTP headers
`CurrentCulture`	the current user's language rules	sorting text shown to the user
`CurrentCultureIgnoreCase`	same, ignoring case	user-facing search
`InvariantCulture`	culture-neutral linguistic rules	stable linguistic sorting across machines

ORDINAL VERSUS CULTURE COMPARISON Program.cs

```
using System.Globalization;
CultureInfo.CurrentCulture = new CultureInfo("en-US");

string[] fruits = { "banana", "Apple", "cherry", "apple" };

string[] ordinal = (string[])fruits.Clone();
Array.Sort(ordinal, StringComparer.Ordinal);
Console.WriteLine($"Ordinal: {string.Join(" ", ordinal)}");

string[] culture = (string[])fruits.Clone();
Array.Sort(culture, StringComparer.CurrentCulture);
Console.WriteLine($"Culture: {string.Join(" ", culture)}");

Console.WriteLine(Math.Sign(string.Compare("apple", "Apple", StringComparison.Ordinal)));
Console.WriteLine("report.PDF".EndsWith(".pdf", StringComparison.OrdinalIgnoreCase));
Console.WriteLine("Hello World".Contains("WORLD", StringComparison.OrdinalIgnoreCase));
```

OUTPUT

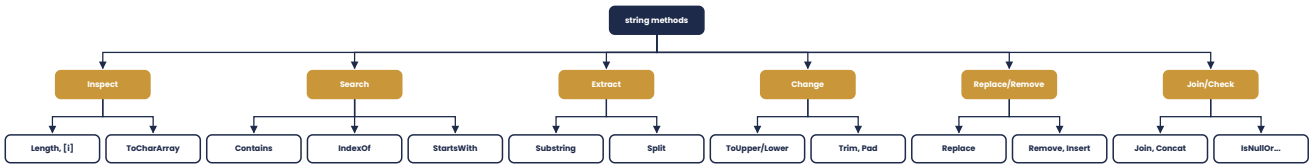
```
Ordinal: Apple apple banana cherry
Culture: apple Apple banana cherry
1
True
True
```

NOTE

A famous culture trap is the Turkish letter i: in the tr-TR culture, "i".ToUpper() is "i" (dotted capital I), so a case-insensitive culture comparison of "file" and "FILE" fails there. For program logic, choose an ordinal comparison; reserve culture comparisons for text shown to people.

8.3 String methods

`System.String` provides dozens of methods. Remember that none of them change the original string: each returns a new string (or a number or bool). The diagram groups the methods covered in this section.



THE STRING METHODS OF SECTION 8.3

8.3.1 Length, indexing, ToCharArray()

`Length` is the number of chars. The indexer `s[i]` reads one char (read-only, because strings are immutable); index-from-end `s[^1]` and ranges `s[a..b]` work just as with arrays, a range returning a new string. `ToCharArray()` copies the characters into a **mutable** char array, which you can change and turn back into a string with `new string(chars)`.

READING CHARACTERS Program.cs

```
string s = "Learn C#";
Console.WriteLine($"Length {s.Length}, first '{s[0]}', last '{s[^1]}'");
Console.WriteLine($"s[6..] = {s[6..]}, s[..5] = {s[..5]}, s[1..4] = {s[1..4]}");

foreach (char c in s) Console.Write($"{c}|");
Console.WriteLine();

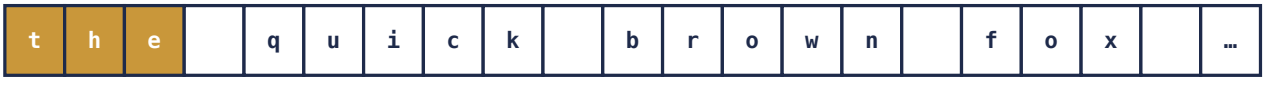
// s[0] = 'l';                          // error CS0200: indexer is read only
char[] chars = s.ToCharArray();          // a mutable copy
chars[0] = 'l';
Array.Reverse(chars);
Console.WriteLine(new string(chars));
Console.WriteLine(s);                    // original untouched
```

OUTPUT

```
Length 8, first 'L', last '#'
s[6..] = C#, s[..5] = Learn, s[1..4] = ear
L|e|a|r|n| |C|#|
#C nrael
Learn C#
```

8.3.2 Searching: Contains, IndexOf, LastIndexOf, StartsWith, EndsWith

`Contains`, `StartsWith` and `EndsWith` answer yes or no. `IndexOf` returns the position of the first occurrence (or -1), optionally starting from a given index; `LastIndexOf` searches from the end. All are case-sensitive unless you pass a `StringComparison`.



INDEXOF("THE") IS 0; INDEXOF('O') IS 12; THE SECOND "THE" STARTS AT INDEX 31

SEARCHING INSIDE A STRING

Program.cs

```
string text = "the quick brown fox jumps over the lazy dog";

Console.WriteLine(text.Contains("fox"));
Console.WriteLine($"IndexOf(\"the\") = {text.IndexOf("the")}, LastIndexOf(\"the\") =
{text.LastIndexOf("the")}");
Console.WriteLine($"IndexOf('o') = {text.IndexOf('o')}, LastIndexOf('o') = {text.LastIndexOf('o')}");
Console.WriteLine($"IndexOf(\"the\", 1) = {text.IndexOf("the", 1)}, IndexOf(\"cat\") =
{text.IndexOf("cat")}");
Console.WriteLine($"{text.StartsWith("The")} {text.StartsWith("The", StringComparison.OrdinalIgnoreCase)}
{text.EndsWith("dog")}");

int count = 0, pos = text.IndexOf('o');
while (pos != -1)                        // find every occurrence
{
    count++;
    pos = text.IndexOf('o', pos + 1);
}
Console.WriteLine($"'o' appears {count} times");
```

OUTPUT

```
True
IndexOf("the") = 0, LastIndexOf("the") = 31
IndexOf('o') = 12, LastIndexOf('o') = 41
IndexOf("the", 1) = 31, IndexOf("cat") = -1
False True True
'o' appears 4 times
```

8.3.3 Extracting: Substring, Split

`Substring(start)` returns the rest of the string from `start`; `Substring(start, length)` returns `length` chars. Combined with `IndexOf` it extracts parts of structured text. `Split` breaks a string into an array at separator characters or strings; the options `RemoveEmptyEntries` and `TrimEntries` clean up messy input.

SUBSTRING AND SPLIT

Program.cs

```
string email = "asha.k@example.com";
int at = email.IndexOf('@');
string user = email.Substring(0, at);
string domain = email.Substring(at + 1);
Console.WriteLine($"user = {user}, domain = {domain}");
Console.WriteLine(email[(at + 1)..]);                    // the same with a range

string csv = "Asha,  21 ,, Pune ,8.7";
string[] raw = csv.Split(',');
Console.WriteLine($"{raw.Length} parts: [{string.Join("][", raw)}]");
string[] clean = csv.Split(',', StringSplitOptions.RemoveEmptyEntries | StringSplitOptions.TrimEntries);
Console.WriteLine($"{clean.Length} parts: [{string.Join("][", clean)}]");

string sentence = "one two;three, four";
string[] words = sentence.Split(new[] { ' ', ';', ',' }, StringSplitOptions.RemoveEmptyEntries);
Console.WriteLine(string.Join("|", words));

string line = "ERROR 2026-03-09 Disk is almost full";
string[] parts = line.Split(' ', 3);                     // at most 3 pieces
Console.WriteLine($"level={parts[0]} date={parts[1]} message={parts[2]}");
```

OUTPUT

```
user = asha.k, domain = example.com
example.com
5 parts: [Asha][  21 ][][ Pune ][8.7]
4 parts: [Asha][21][Pune][8.7]
one|two|three|four
level=ERROR date=2026-03-09 message=Disk is almost full
```

WATCH OUT

`Substring` throws `ArgumentOutOfRangeException` if `start` or `start + length` goes past the end. Always check the result of `IndexOf` for -1 before using it as a position.

8.3.4 Changing: ToUpper, ToLower, Trim, TrimStart, TrimEnd, PadLeft, PadRight

`ToUpper` and `ToLower` change case (culture-aware; `ToUpperInvariant` is the culture-neutral version). `Trim` removes whitespace from both ends, `TrimStart` and `TrimEnd` from one end; all three can also remove specific characters. `PadLeft` and `PadRight` add characters to reach a total width, useful for fixed-width reports and leading zeros.

CASE, TRIMMING AND PADDING Program.cs

```
string raw = "   Hello, World!   ";
Console.WriteLine($"[{raw.ToUpper()}]");
Console.WriteLine($"[{raw.ToLower()}]");
Console.WriteLine($"[{raw.Trim()}]");
Console.WriteLine($"[{raw.TrimStart()}]");
Console.WriteLine($"[{raw.TrimEnd()}]");

Console.WriteLine("**price**".Trim('*'));               // remove chosen characters
Console.WriteLine("000420".TrimStart('0'));
Console.WriteLine("file.txt...".TrimEnd('.'));

Console.WriteLine($"[{"42".PadLeft(6)}] [{"42".PadRight(6)}] [{"42".PadLeft(6, '0')}]");
foreach (var (name, mark) in new[] { ("Asha", 91), ("Ravi", 7), ("Meera", 100) })
    Console.WriteLine(name.PadRight(8, '.') + mark.ToString().PadLeft(4));
```

OUTPUT

```
[   HELLO, WORLD!   ]
[   hello, world!   ]
[Hello, World!]
[Hello, World!   ]
[   Hello, World!]
price
420
file.txt
[    42] [42    ] [000042]
Asha....  91
Ravi....   7
Meera... 100
```

8.3.5 Replacing and removing: Replace, Remove, Insert

`Replace(old, new)` replaces **every** occurrence of a char or substring (an overload takes a `StringComparison`). `Remove(start)` deletes everything from `start`; `Remove(start, count)` deletes `count` chars. `Insert(index, text)` inserts text at a position.

REPLACE, REMOVE AND INSERT Program.cs

```
string s = "I like Java. Java is popular.";
Console.WriteLine(s.Replace("Java", "C#"));
Console.WriteLine(s.Replace('.', '!'));
Console.WriteLine(s.Replace("java", "C#", StringComparison.OrdinalIgnoreCase));
Console.WriteLine("a  b   c".Replace(" ", ""));              // delete all spaces

string phone = "9876543210";
Console.WriteLine(phone.Remove(5));                          // keep first 5
Console.WriteLine(phone.Remove(2, 6));                       // drop 6 chars from index 2
Console.WriteLine(phone.Insert(5, " "));                     // 98765 43210
Console.WriteLine(phone[..2] + new string('*', 6) + phone[^2..]);  // masking
```

OUTPUT

```
I like C#. C# is popular.
I like Java! Java is popular!
I like C#. C# is popular.
abc
98765
9810
98765 43210
98******10
```

8.3.6 Joining: string.Join, string.Concat

`string.Join(separator, values)` joins the items of an array or any collection with a separator between them; it accepts numbers and other objects too, calling `ToString` on each. `string.Concat` joins with no separator. The `+` operator on strings compiles into a call to `string.Concat`.

JOIN AND CONCAT Program.cs

```
string[] words = { "C#", "is", "fun" };
int[] marks = { 72, 85, 91 };
List<string> cities = ["Pune", "Delhi", "Chennai"];

Console.WriteLine(string.Join(" ", words));
Console.WriteLine(string.Join(", ", marks));
Console.WriteLine(string.Join(" -> ", cities));
Console.WriteLine(string.Join('/', "home", "asha", "docs"));     // params overload
Console.WriteLine(string.Concat(words));
Console.WriteLine(string.Concat("Total: ", 248, " marks"));
Console.WriteLine("ab" + 1 + 2);           // left to right: "ab1" then "ab12"
Console.WriteLine(1 + 2 + "ab");           // 3 first, then "3ab"
```

OUTPUT

```
C# is fun
72, 85, 91
Pune -> Delhi -> Chennai
home/asha/docs
C#isfun
Total: 248 marks
ab12
3ab
```

8.3.7 Checking: string.IsNullOrEmpty, string.IsNullOrWhiteSpace

Input validation constantly needs to know whether a string has real content. `string.IsNullOrEmpty(s)` is true for null or "". `string.IsNullOrWhiteSpace(s)` is also true for strings containing only spaces, tabs or new lines, which is usually what you want for user input.

CHECKING FOR MISSING TEXT Program.cs

```
string?[] inputs = { null, "", "   ", "\t\n", "Asha" };
Console.WriteLine($"{"input",-8}{"NullOrEmpty",-14}{"NullOrWhiteSpace"}");
foreach (string? s in inputs)
{
    string shown = s is null ? "null" : $"\"{s.Replace("\t", "\\t").Replace("\n", "\\n")}\"";
    Console.WriteLine($"{shown,-8}{string.IsNullOrEmpty(s),-14}{string.IsNullOrWhiteSpace(s)}");
}
```

OUTPUT

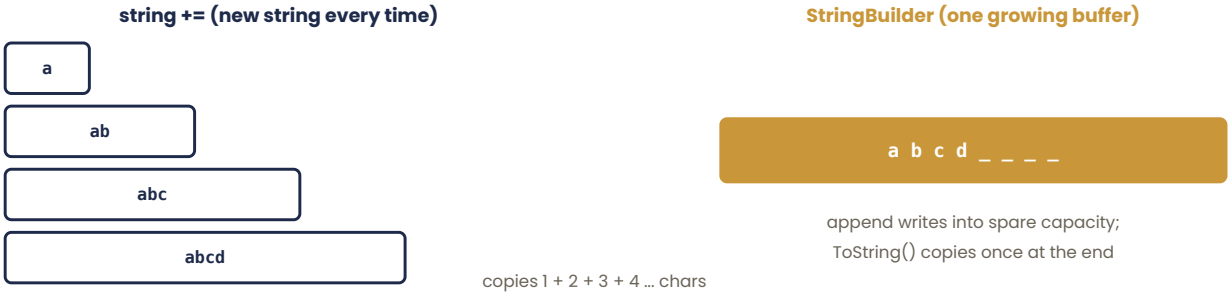
```
input   NullOrEmpty   NullOrWhiteSpace
null    True          True
""      True          True
"   "   False         True
"\t\n"  False         True
"Asha"  False         False
```

8.4 StringBuilder

`System.Text.StringBuilder` is a **mutable** string buffer. It keeps an internal char array with spare capacity; appending writes into that space and only occasionally grows the buffer. When you finish, `ToString()` creates the final string once.

8.4.1 Why StringBuilder (performance in loops)

Because strings are immutable, `s += x` in a loop creates a brand-new string each time and copies all the previous characters into it. Building a string of n pieces this way copies about $n^2/2$ characters: $O(n^2)$ time and a mountain of garbage for the GC. StringBuilder appends in amortised $O(1)$, so the whole loop is $O(n)$.



REPEATED CONCATENATION COPIES EVERYTHING AGAIN; STRINGBUILDER DOES NOT

CONCATENATION VERSUS STRINGBUILDER Program.cs

```
using System.Diagnostics;
using System.Text;

const int N = 100_000;

var sw = Stopwatch.StartNew();
string s = "";
for (int i = 0; i < N; i++) s += "x";            // 100,000 new strings
sw.Stop();
long concatMs = sw.ElapsedMilliseconds;

sw.Restart();
var sb = new StringBuilder();
for (int i = 0; i < N; i++) sb.Append('x');      // writes into the buffer
string built = sb.ToString();
sw.Stop();
long builderMs = sw.ElapsedMilliseconds;

Console.WriteLine($"lengths: {s.Length:N0} and {built.Length:N0}, equal: {s == built}");
Console.WriteLine($"StringBuilder was faster: {builderMs < concatMs}");
```

OUTPUT

```
lengths: 100,000 and 100,000, equal: True
StringBuilder was faster: True
```

The exact times depend on the machine, so the program prints only the comparison. On a typical laptop the `+=` loop takes on the order of a second (it copies about five billion characters in total), while the StringBuilder loop finishes in about a millisecond. Rule of thumb: use `+` or interpolation for a fixed handful of pieces, and StringBuilder (or `string.Join`) when concatenating inside a loop.

8.4.2 Append, AppendLine, Insert, Remove, Replace, ToString

Member	Effect
`Append(x)`	add the text of x (string, char, int, double, …) at the end
`AppendLine(x)`	append x followed by a new line (Environment.NewLine)
`AppendJoin(sep, items)`	append items joined by a separator
`Insert(i, x)`	insert text at index i

Member	Effect
`Remove(i, n)`	delete n chars from index i
`Replace(a, b)`	replace every occurrence in place
`sb[i]`	read or **write** one char
`Length`, `Capacity`, `Clear()`	current length, buffer size, empty the builder
`ToString()`	create the final immutable string

WORKING WITH STRINGBUILDER Program.cs

```
using System.Text;

var sb = new StringBuilder("Hello");
sb.Append(", ").Append("World").Append('!');          // methods return the builder: chaining
Console.WriteLine(sb);

sb.Insert(0, ">> ");
sb.Replace("World", "C#");
Console.WriteLine(sb.ToString());

sb.Remove(0, 3);                                      // remove ">> "
sb[0] = 'J';                                          // writable indexer
Console.WriteLine($"{sb} (length {sb.Length})");

var report = new StringBuilder();
report.AppendLine("REPORT");
report.AppendJoin(" | ", "Asha 91", "Ravi 78", "Meera 85").AppendLine();
for (int i = 1; i <= 3; i++) report.Append($"row {i}; ");
Console.Write(report);
Console.WriteLine();
report.Clear();
Console.WriteLine($"after Clear: length {report.Length}");
```

OUTPUT

```
Hello, World!
>> Hello, C#!
Jello, C#! (length 10)
REPORT
Asha 91 | Ravi 78 | Meera 85
row 1; row 2; row 3;
after Clear: length 0
```

TIP

If you know roughly how long the result will be, pass the capacity to the constructor, `new StringBuilder(10_000)`, so the buffer never has to grow.

8.5 Characters

A `char` holds one UTF-16 code unit and is written in single quotes: `'A'`. Chars are numeric underneath: `(int)'A'` is 65, and arithmetic such as `'7' - '0'` gives the digit value 7. Characters outside the Basic Multilingual Plane, such as many emoji, need two chars (a **surrogate pair**), which is why a string holding one smiley emoji, `"\uD83D\uDE00"`, has Length 2.

8.5.1 char.IsLetter, IsDigit, IsWhiteSpace, IsUpper, ToUpper

The static methods of `char` classify characters using Unicode rules, so they work for every alphabet, not just English.

Method	True for	Method	Result
`char.IsLetter(c)`	a–z, A–Z, other alphabets ('é', 'अ')	`char.ToUpper(c)`	uppercase version
`char.IsDigit(c)`	'0' to '9' (and other decimal digits)	`char.ToLower(c)`	lowercase version
`char.IsLetterOrDigit(c)`	either of the above	`char.GetNumericValue(c)`	numeric value as double ('7' gives 7)
`char.IsWhiteSpace(c)`	space, '\t', '\n', '\r'	`(int)c`	the UTF-16 code
`char.IsUpper(c)` / `IsLower(c)`	uppercase / lowercase letters	`(char)(c + 1)`	the next character
`char.IsPunctuation(c)`	'.', ',', '!', '?', …	`char.IsAsciiLetter(c)`	a–z, A–Z only (.NET 7+)

CLASSIFYING CHARACTERS Program.cs

```
string sample = "Ab3 !";
foreach (char c in sample)
    Console.WriteLine($"'{c}' code {(int)c,3}: letter {char.IsLetter(c),-5} digit {char.IsDigit(c),-5} "
+
                      $"space {char.IsWhiteSpace(c),-5} upper {char.IsUpper(c),-5} punct
{char.IsPunctuation(c)}");

Console.WriteLine($"{char.ToUpper('q')} {char.ToLower('Q')} {'7' - '0'} {(char)('A' + 2)}");

string password = "Secret@2026";
bool hasUpper = false, hasLower = false, hasDigit = false, hasSymbol = false;
foreach (char c in password)
{
    if (char.IsUpper(c)) hasUpper = true;
    else if (char.IsLower(c)) hasLower = true;
    else if (char.IsDigit(c)) hasDigit = true;
    else hasSymbol = true;
}
bool strong = password.Length >= 8 && hasUpper && hasLower && hasDigit && hasSymbol;
Console.WriteLine($"{password}: strong = {strong}");
```

OUTPUT

```
'A' code  65: letter True  digit False space False upper True  punct False
'b' code  98: letter True  digit False space False upper False punct False
'3' code  51: letter False digit True  space False upper False punct False
' ' code  32: letter False digit False space True  upper False punct False
'!' code  33: letter False digit False space False upper False punct True
Q q 7 C
Secret@2026: strong = True
```

8.6 Regular expressions (introduction)

A **regular expression** (regex) is a small pattern language for describing text: "three digits, a dash, four digits" or "something that looks like an email". The `System.Text.RegularExpressions.Regex` class tests, finds, extracts and replaces text that matches a pattern. Patterns are usually written as verbatim strings so that backslashes need no escaping.

Pattern	Matches	Pattern	Matches
`\d`	a digit	`^` `$`	start / end of the text
`\w`	a letter, digit or _	`*` `+` `?`	0 or more / 1 or more / 0 or 1
`\s`	whitespace	`{3}` `{2,4}`	exactly 3 / 2 to 4 times
`.`	any char except new line	`[aeiou]` `[^0-9]`	one of the set / not in the set
`\.`	a literal dot	`( )` `(?<name> )`	capture group / named group
`\b`	word boundary	`a|b`	a or b

8.6.1 Regex.IsMatch, Match, Matches, Replace

`Regex.IsMatch` answers whether the pattern occurs (use `^...$` to require the whole text to match). `Regex.Match` returns the first match with its captured **groups**; `Regex.Matches` returns all matches. `Regex.Replace` replaces every match, and the replacement can refer to groups as `$1` or `${name}`.

THE FOUR MAIN REGEX OPERATIONS

Program.cs

```
using System.Text.RegularExpressions;

string mobile = @"^[6-9]\d{9}$";                        // Indian mobile number
foreach (string m in new[] { "9876543210", "12345", "5876543210" })
    Console.WriteLine($"{m,-11} valid: {Regex.IsMatch(m, mobile)}");

string email = @"^[\w.+-]+@[\w-]+\.[\w.]+$";
Console.WriteLine($"email ok: {Regex.IsMatch("asha.k@example.com", email)}, {Regex.IsMatch("asha@",
email)}");

Match d = Regex.Match("Exam on 2026-03-09 at 10:00", @"(?<y>\d{4})-(?<m>\d{2})-(?<d>\d{2})");
if (d.Success)
    Console.WriteLine($"found '{d.Value}' at {d.Index}: day {d.Groups["d"].Value}, month
{d.Groups["m"].Value}, year {d.Groups["y"].Value}");

string order = "3 pens cost 45, 2 books cost 380 and 1 bag costs 1250";
MatchCollection numbers = Regex.Matches(order, @"\d+");
int sum = 0;
foreach (Match n in numbers) sum += int.Parse(n.Value);
Console.WriteLine($"numbers: {numbers.Count}, sum {sum}");

Console.WriteLine(Regex.Replace("too    many   spaces", @"\s+", " "));
Console.WriteLine(Regex.Replace("card 4111 2222 3333 4444", @"\d{4} (?=\d)", "**** "));
Console.WriteLine(Regex.Replace("2026-03-09", @"(\d+)-(\d+)-(\d+)", "$3/$2/$1"));
```

OUTPUT

```
9876543210  valid: True
12345       valid: False
5876543210  valid: False
email ok: True, False
found '2026-03-09' at 8: day 09, month 03, year 2026
numbers: 6, sum 1681
too many spaces
card **** **** **** 4444
09/03/2026
```

NOTE

`(?=\d)` in the card pattern is a **lookahead**: it requires a digit to follow but does not consume it, so the last group of four is left visible. For patterns used often, .NET 7+ can generate the matching code at compile time with the `[GeneratedRegex(@"...")]` attribute on a `partial` method, which is faster than building a `Regex` at run time.

TIP

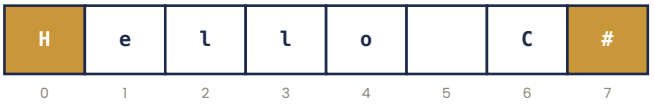
Regular expressions are powerful but easy to make unreadable. Prefer simple string methods (`Contains` , `Split` , `char.IsDigit`) when they do the job, and keep each regex small and commented.

8.7 Classic string problems

These problems appear in almost every beginner interview. Each one combines loops, indexing and the methods of this module; for each, think about the time complexity and about edge cases such as empty strings, spaces and mixed case.

8.7.1 Reverse a string and reverse words

A string cannot be reversed in place, so copy it into a char array, reverse the array (with `Array.Reverse` or two indexes swapping towards the middle) and build a new string. To reverse the **order of words**, split on spaces, reverse the array of words and join it again.



TWO-INDEX REVERSAL: SWAP S[I] AND S[J], THEN MOVE I RIGHT AND J LEFT UNTIL THEY MEET

REVERSING

Program.cs

```
string s = "Hello C#";

char[] a = s.ToCharArray();
for (int i = 0, j = a.Length - 1; i < j; i++, j--)
    (a[i], a[j]) = (a[j], a[i]);                    // swap with a tuple
Console.WriteLine(new string(a));

char[] b = s.ToCharArray();
Array.Reverse(b);                                   // built-in
Console.WriteLine(new string(b));

string sentence = "C# is fun to learn";
string[] words = sentence.Split(' ');
Array.Reverse(words);
Console.WriteLine(string.Join(" ", words));         // reverse word order

string[] each = sentence.Split(' ');
for (int i = 0; i < each.Length; i++)
{
    char[] w = each[i].ToCharArray();
    Array.Reverse(w);
    each[i] = new string(w);                        // reverse each word
}
Console.WriteLine(string.Join(" ", each));
```

OUTPUT

```
#C olleH
#C olleH
learn to fun is C#
#C si nuf ot nrael
```

8.7.2 Palindrome and anagram check

A **palindrome** reads the same forwards and backwards ("racecar", "Madam, I'm Adam" once punctuation and case are ignored). Two indexes start at both ends, skip characters that are not letters or digits, and compare lowercase versions while moving inwards: O(n) time, no extra memory.

PALINDROME WITH TWO POINTERS

Program.cs

```
foreach (string t in new[] { "racecar", "Madam, I'm Adam", "hello", "A man, a plan, a canal: Panama" })
    Console.WriteLine($"{t,-32} {IsPalindrome(t)}");

static bool IsPalindrome(string s)
{
    int i = 0, j = s.Length - 1;
    while (i < j)
    {
        if (!char.IsLetterOrDigit(s[i])) { i++; continue; }   // skip punctuation/spaces
        if (!char.IsLetterOrDigit(s[j])) { j--; continue; }
        if (char.ToLower(s[i]) != char.ToLower(s[j])) return false;
        i++; j--;
    }
    return true;
}
```

OUTPUT

```
racecar                          True
Madam, I'm Adam                  True
hello                            False
A man, a plan, a canal: Panama   True
```

Two words are **anagrams** if they contain the same letters the same number of times ("listen" and "silent"). Method 1: normalise both (lowercase, letters only), sort their chars and compare: O(n log n). Method 2: count letters in an array of 26 counters, adding for the first word and subtracting for the second; they are anagrams if every counter ends at zero: O(n).

TWO ANAGRAM CHECKS Program.cs

```
var pairs = new[] { ("Listen", "Silent"), ("Dusty", "Study"), ("hello", "world"), ("Dormitory", "Dirty
room") };
foreach (var (x, y) in pairs)
    Console.WriteLine($"{x,-10} {y,-11} sorted: {AnagramSort(x, y),-6} counting: {AnagramCount(x, y)}");

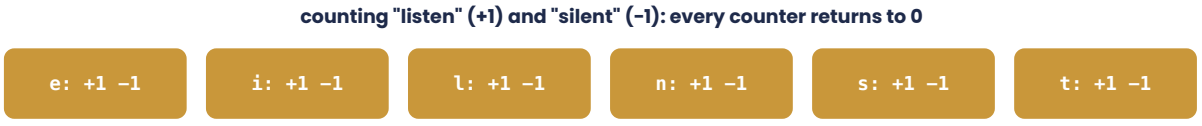
static string Normalize(string s)
{
    var chars = new List<char>();
    foreach (char c in s) if (char.IsLetter(c)) chars.Add(char.ToLower(c));
    return new string(chars.ToArray());
}

static bool AnagramSort(string a, string b)
{
    char[] x = Normalize(a).ToCharArray(), y = Normalize(b).ToCharArray();
    Array.Sort(x); Array.Sort(y);
    return new string(x) == new string(y);
}

static bool AnagramCount(string a, string b)
{
    int[] count = new int[26];
    foreach (char c in Normalize(a)) if (c is >= 'a' and <= 'z') count[c - 'a']++;
    foreach (char c in Normalize(b)) if (c is >= 'a' and <= 'z') count[c - 'a']--;
    foreach (int n in count) if (n != 0) return false;
    return true;
}
```

OUTPUT

```
Listen     Silent      sorted: True   counting: True
Dusty      Study       sorted: True   counting: True
hello      world       sorted: False  counting: False
Dormitory  Dirty room  sorted: True   counting: True
```



THE COUNTING METHOD USES ONE ARRAY OF 26 INTS

8.7.3 Count vowels, consonants, words

Loop over the characters once and classify each: a letter is a vowel if it appears in "aeiou" (after lowercasing), otherwise a consonant; digits, spaces and symbols count as neither. Words are the pieces left after splitting on whitespace with `RemoveEmptyEntries`, so several spaces in a row do not create empty words.

COUNTING CHARACTERS AND WORDS Program.cs

```
string text = "Learning  C# is fun and practical";
int vowels = 0, consonants = 0, others = 0;
foreach (char c in text)
{
    if (char.IsLetter(c))
    {
        if ("aeiou".Contains(char.ToLower(c))) vowels++;
        else consonants++;
    }
    else others++;
}
string[] words = text.Split((char[]?)null, StringSplitOptions.RemoveEmptyEntries);   // null = any
whitespace
Console.WriteLine($"vowels {vowels}, consonants {consonants}, other chars {others}");
Console.WriteLine($"words {words.Length}, longest \"{Longest(words)}\"");

static string Longest(string[] words)
{
    string best = "";
    foreach (string w in words) if (w.Length > best.Length) best = w;
    return best;
}
```

OUTPUT

```
vowels 9, consonants 17, other chars 7
words 6, longest "practical"
```

NOTE

The text deliberately has two spaces after "Learning". Splitting on `' '` without options would give an empty word there; passing `null` as the separator splits on any whitespace, and `RemoveEmptyEntries` drops the empty pieces. "C#" counts one consonant (C) and one other char (#).

8.7.4 Character frequency and first non-repeating character

Counting how often each character occurs is a job for a dictionary from `char` to `int` (Module 14 covers collections in detail). A `SortedDictionary` keeps the keys in order for printing. To find the **first non-repeating character**, count everything in a first pass, then scan the string again in its original order and return the first char whose count is 1: two passes, O(n).

FREQUENCY AND FIRST UNIQUE CHARACTER

Program.cs

```
string word = "programming";
var freq = new SortedDictionary<char, int>();
foreach (char c in word)
    freq[c] = freq.TryGetValue(c, out int n) ? n + 1 : 1;
foreach (var (ch, count) in freq) Console.Write($"{ch}:{count} ");
Console.WriteLine();

foreach (string s in new[] { "programming", "swiss", "aabbcc" })
{
    char? first = FirstUnique(s);
    Console.WriteLine($"{s}: {(first is null ? "none" : first.ToString())}");
}

static char? FirstUnique(string s)
{
    var counts = new Dictionary<char, int>();
    foreach (char c in s) counts[c] = counts.GetValueOrDefault(c) + 1;   // pass 1: count
    foreach (char c in s) if (counts[c] == 1) return c;                  // pass 2: original order
    return null;
}
```

OUTPUT

```
a:1 g:2 i:1 m:2 n:1 o:1 p:1 r:2
programming: p
swiss: w
aabbcc: none
```

Problem	Approach	Time	Extra space
Reverse string	char array, swap from both ends	O(n)	O(n) (new string)
Reverse words	Split, Array.Reverse, Join	O(n)	O(n)
Palindrome	two pointers, skip non-alphanumerics	O(n)	O(1)
Anagram	sort and compare / 26 counters	O(n log n) / O(n)	O(n) / O(1)
Vowels, words	one pass + Split	O(n)	O(words)
First unique char	count pass + scan pass	O(n)	O(distinct chars)

string is an alias of System.String: an immutable sequence of UTF-16 chars, so every "change" returns a new string, and literals are interned. Choose normal, verbatim (@) or raw (""") literals as convenient, and build text with interpolation {value,align:format}. Compare with == or Equals for content, and pass StringComparison.Ordinal or OrdinalIgnoreCase for program logic; culture comparisons are for text shown to people. Use the string methods to search, extract, change, replace, join and check text, StringBuilder when concatenating in loops, char methods to classify characters, and Regex for patterns.

PRACTICE CHECKPOINT

Build three small tools: a **word counter** that reports total and unique words with their frequencies, an **anagram checker** for two phrases, and a **timing comparison** of string concatenation against StringBuilder over 100,000 iterations.

SAMPLE SOLUTION

Program.cs

```
using System.Diagnostics;
using System.Text;

Console.Write("Enter a sentence: ");
string line = Console.ReadLine() ?? "";
string[] words = line.ToLower().Split((char[]?)null, StringSplitOptions.RemoveEmptyEntries);
var freq = new SortedDictionary<string, int>();
foreach (string w in words) freq[w] = freq.GetValueOrDefault(w) + 1;
Console.WriteLine($"words: {words.Length}, unique: {freq.Count}");
foreach (var (w, c) in freq) Console.WriteLine($"  {w,-6}{new string('#', c)} {c}");

Console.Write("First phrase: ");
string a = Console.ReadLine() ?? "";
Console.Write("Second phrase: ");
string b = Console.ReadLine() ?? "";
int[] count = new int[26];
foreach (char ch in a.ToLower()) if (ch is >= 'a' and <= 'z') count[ch - 'a']++;
foreach (char ch in b.ToLower()) if (ch is >= 'a' and <= 'z') count[ch - 'a']--;
bool anagram = Array.TrueForAll(count, n => n == 0);
Console.WriteLine(anagram ? "They are anagrams" : "Not anagrams");

const int N = 100_000;
var sw = Stopwatch.StartNew();
string s = "";
for (int i = 0; i < N; i++) s += "x";
long concatMs = sw.ElapsedMilliseconds;
sw.Restart();
var sb = new StringBuilder();
for (int i = 0; i < N; i++) sb.Append('x');
string t = sb.ToString();
long builderMs = sw.ElapsedMilliseconds;
Console.WriteLine($"same result: {s == t}, StringBuilder faster: {builderMs < concatMs}");
```

CONSOLE (user input in gold)

```
Enter a sentence: The cat and the hat and the bat
words: 8, unique: 5
  and   ## 2
  bat   # 1
  cat   # 1
  hat   # 1
  the   ### 3
First phrase: Listen
Second phrase: Silent
They are anagrams
same result: True, StringBuilder faster: True
```

MODULE 09 · BEGINNER

Methods and recursion

Minimum duration **3 days** | Level **Beginner**

A method is a named block of code that performs one task and can be called again and again. Methods remove repetition, give names to ideas and let large programs be built from small, testable pieces. This module covers defining and calling methods, the rich set of C# parameter kinds (value, ref, out, in, params, optional and named), returning several values with tuples, overloading, and recursion with its classic problems and memoization.

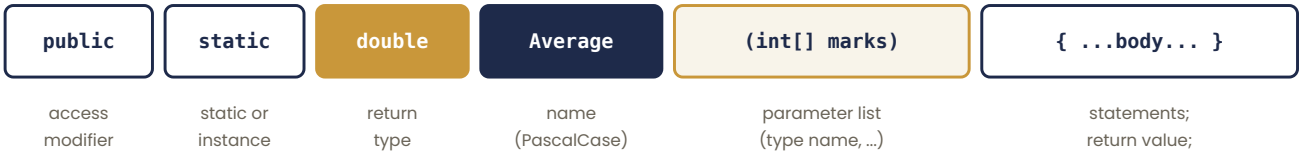
IN THIS MODULE

9.1 Method basics
9.2 Parameters
9.3 Returning multiple values
9.4 Method overloading
9.5 Recursion

9.1 Method basics

Every executable statement in C# lives inside a method. Even top-level statements in Program.cs are compiled into a hidden `Main` method. Writing your own methods follows the **DRY** principle (Don't Repeat Yourself): when the same logic is needed twice, give it a name and call it.

9.1.1 Defining methods: access modifier, return type, name, parameters



THE PARTS OF A METHOD DECLARATION; THE NAME PLUS THE PARAMETER TYPES FORM THE METHOD SIGNATURE

Part	Meaning
Access modifier	`public`, `private` (the default inside a class), `internal`, `protected`: who may call the method (Module 10)
`static`	belongs to the type itself and is called as `Type.Method()`; without it the method belongs to an object
Return type	the type of the value sent back, or `void` if nothing is returned
Name	a verb or verb phrase in PascalCase: `CalculateTax`, `IsValid`, `PrintReport`
Parameters	zero or more inputs, each with a type and a name; the values passed by the caller are the **arguments**
Body	statements in braces, or a single expression after `=>`

DEFINING AND CALLING METHODS

Program.cs

```
int[] marks = { 72, 85, 91, 64 };
double avg = Grades.Average(marks);                // call: arguments go in, a value comes back
Console.WriteLine($"Average {avg:F2}, grade {Grades.Letter(avg)}");

class Grades
{
    public static double Average(int[] marks)      // parameter: int[] marks
    {
        int total = 0;
        foreach (int m in marks) total += m;
        return (double)total / marks.Length;         // return value
    }

    public static char Letter(double score)
    {
        if (score >= 85) return 'A';
        if (score >= 70) return 'B';
        return 'C';
    }
}
```

OUTPUT

```
Average 78.00, grade B
```

9.1.2 Calling static and instance methods

A **static** method is called on the type name: `Math.Sqrt(16)`, `Grades.Average(m)`. It cannot use the fields of any particular object. An **instance** method is called on an object created with `new` and works with that object's own data: `account.Deposit(500)`. Inside the same class, either kind can be called by its name alone. In a top-level program, methods written after the statements are **local functions** of the hidden Main and can be called directly.

STATIC VERSUS INSTANCE METHODS

Program.cs

```
Console.WriteLine(MathUtil.Square(7));            // static: Type.Method()
Console.WriteLine(Math.Max(3, 9));                 // a static method of the BCL

var c1 = new Counter();                            // instance: needs an object
var c2 = new Counter();
c1.Increment(); c1.Increment(); c2.Increment();
Console.WriteLine($"c1 = {c1.Value}, c2 = {c2.Value}");   // each object has its own state

Greet("Asha");                                     // local function in Program.cs

static void Greet(string name) => Console.WriteLine($"Hello, {name}!");

class MathUtil
{
    public static int Square(int x) => x * x;
}

class Counter
{
    public int Value { get; private set; }
    public void Increment() => Value++;            // uses this object's Value
}
```

OUTPUT

```
49
9
c1 = 2, c2 = 1
Hello, Asha!
```

9.1.3 void methods and return values

A `void` method performs an action and returns nothing; `return;` may be used to leave it early. A method with a return type must return a value of that type on **every** path through its body; the compiler checks this. A `return` statement ends the method immediately, which makes "early return" (guard clauses) a clean way to handle special cases first.

VOID METHODS AND EARLY RETURN Program.cs

```
PrintLine('=', 20);
Console.WriteLine(Describe(-5));
Console.WriteLine(Describe(0));
Console.WriteLine(Describe(42));
PrintLine('-', 0);                              // prints nothing: returns early
PrintLine('=', 20);

static void PrintLine(char c, int n)
{
    if (n <= 0) return;                         // guard clause: leave early
    Console.WriteLine(new string(c, n));
}

static string Describe(int n)
{
    if (n < 0) return "negative";
    if (n == 0) return "zero";
    return n % 2 == 0 ? "positive even" : "positive odd";
}
```

OUTPUT

```
====================
negative
zero
positive even
====================
```

EVERY PATH MUST RETURN A VALUE Program.cs

```
static string Grade(int marks)
{
    if (marks >= 40) return "Pass";
    else if (marks < 40) return "Fail";
}
```

COMPILER OUTPUT

```
Program.cs(1,15): error CS0161: 'Grade(int)': not all code paths return a value
```

The compiler does not reason that the two conditions cover every int. Replace `else if (...)` with a plain `else`, or end with an unconditional `return`.

9.1.4 Expression-bodied methods =>

When a method body is a single expression, it can be written after `=>` instead of braces and `return`. The same syntax works for properties, constructors and operators. Use it for short, obvious methods; switch to a block body when the logic needs several statements.

EXPRESSION-BODIED MEMBERS

Program.cs

```
Console.WriteLine($"{Cube(3)} {IsEven(10)} {Max(4, 9)} {FullName("Asha", "Rao")}");
Log("saved");

static int Cube(int x) => x * x * x;
static bool IsEven(int n) => n % 2 == 0;
static int Max(int a, int b) => a > b ? a : b;
static string FullName(string first, string last) => $"{last.ToUpper()}, {first}";
static void Log(string msg) => Console.WriteLine($"[LOG] {msg}");    // void works too
```

OUTPUT

```
27 True 9 RAO, Asha
[LOG] saved
```

9.1.5 Local functions and static local functions

A **local function** is a method declared inside another method. It is visible only there, which keeps helper logic close to where it is used and out of the class's public surface. A local function can use the variables of the enclosing method (it **captures** them). Marking it `static` forbids capturing: it can only use its parameters, which makes data flow explicit and avoids hidden allocations.

LOCAL FUNCTIONS

Program.cs

```
Console.WriteLine(Report(new[] { 72, 85, 91, 30 }, passMark: 40));

static string Report(int[] marks, int passMark)
{
    int passed = 0;
    foreach (int m in marks) if (IsPass(m)) passed++;
    return $"{passed}/{marks.Length} passed, highest {Highest(marks)}";

    bool IsPass(int m) => m >= passMark;          // captures passMark from Report

    static int Highest(int[] values)              // static: may use only its parameters
    {
        int best = values[0];
        foreach (int v in values) if (v > best) best = v;
        return best;
    }
}
```

OUTPUT

```
3/4 passed, highest 91
```

NOTE

If `IsPass` were marked `static`, the compiler would reject it with error CS8421, because a static local function cannot reference the outer variable `passMark`. Local functions can also be called before they are declared, as above.

9.2 Parameters

C# gives fine control over how arguments travel between caller and method. The default is **pass by value**: the method receives a copy. Modifiers change that: `ref` (two-way), `out` (output only), `in` (read-only reference), and `params` (variable number of arguments).

Kind	Syntax	Caller must initialise?	Method may change caller's variable?	Typical use
value	`int x`	yes	no (works on a copy)	the normal case
ref	`ref int x`	yes	yes	swap, update in place
out	`out int x`	no	yes, must assign before returning	TryParse, extra results
in	`in BigStruct x`	yes	no (read-only reference)	large structs without copying
params	`params int[] xs`	n/a	n/a	any number of arguments

9.2.1 Value parameters

A value parameter is a new local variable initialised with a **copy** of the argument. Changing the parameter inside the method never affects the caller's variable.

A VALUE PARAMETER RECEIVES A COPY Program.cs

```
int score = 50;
AddBonus(score);
Console.WriteLine($"after AddBonus: score = {score}");     // unchanged

static void AddBonus(int s)
{
    s += 10;                                          // changes the copy only
    Console.WriteLine($"inside: s = {s}");
}
```

OUTPUT

```
inside: s = 60
after AddBonus: score = 50
```

9.2.2 ref parameters

With `ref`, the parameter becomes an **alias** for the caller's variable: both names refer to the same storage, so changes are visible to the caller. `ref` must be written both in the declaration and at the call site, which makes the possible change obvious to readers. The argument must be a variable that is already initialised.

REF: THE METHOD WORKS ON THE CALLER'S VARIABLE

Program.cs

```
int a = 3, b = 8;
Swap(ref a, ref b);
Console.WriteLine($"a = {a}, b = {b}");

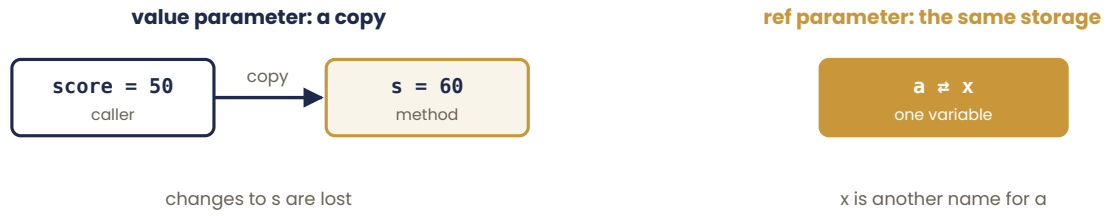
int balance = 1000;
ApplyInterest(ref balance, 5);
Console.WriteLine($"balance = {balance}");

static void Swap(ref int x, ref int y)
{
    int temp = x;
    x = y;
    y = temp;
}

static void ApplyInterest(ref int amount, int percent) => amount += amount * percent / 100;
```

OUTPUT

```
a = 8, b = 3
balance = 1050
```



VALUE PARAMETERS COPY; REF PARAMETERS ALIAS THE CALLER'S VARIABLE

9.2.3 out parameters and the TryParse pattern

An `out` parameter is a reference used only to **send a value back**. The caller does not need to initialise the variable, and the method **must** assign it before returning. Variables can be declared right in the call: `out int value`. The best-known use is the **Try pattern**: `bool TryXxx(input, out result)` returns whether it succeeded and gives the result through the out parameter, so no exception is needed for expected failures.

OUT PARAMETERS AND THE TRY PATTERN

Program.cs

```
foreach (string text in new[] { "42", "4x2", "-7" })
{
    if (int.TryParse(text, out int value))          // declare the out variable inline
        Console.WriteLine($"\"{text}\" -> {value}");
    else
        Console.WriteLine($"\"{text}\" is not a number");
}

if (TryDivide(17, 5, out int q, out int r)) Console.WriteLine($"17 / 5 = {q} remainder {r}");
if (!TryDivide(1, 0, out _, out _)) Console.WriteLine("cannot divide by zero");   // _ discards

static bool TryDivide(int a, int b, out int quotient, out int remainder)
{
    if (b == 0)
    {
        quotient = remainder = 0;                   // out parameters must always be assigned
        return false;
    }
    quotient = a / b;
    remainder = a % b;
    return true;
}
```

OUTPUT

```
"42" -> 42
"4x2" is not a number
"-7" -> -7
17 / 5 = 3 remainder 2
cannot divide by zero
```

9.2.4 in and ref readonly parameters

Passing a large **struct** by value copies all of its bytes on every call. An `in` parameter passes a **read-only reference** instead: no copy is made, and the method is not allowed to modify the argument. `ref readonly` parameters (C# 12) mean the same thing but require the caller to pass a variable (with `ref` or `in`), making the by-reference intent explicit. For small types such as `int` or `double`, plain value parameters are just as fast.

READ-ONLY REFERENCES TO A STRUCT

Program.cs

```
var a = new Point3D(1, 2, 3);
var b = new Point3D(4, 6, 3);
Console.WriteLine(Distance(in a, in b));          // 'in' at the call site is optional
Console.WriteLine(Length(ref a));

static double Distance(in Point3D p, in Point3D q)
{
    // p.X = 0;                                   // compile error: p is a read-only reference
    double dx = p.X - q.X, dy = p.Y - q.Y, dz = p.Z - q.Z;
    return Math.Sqrt(dx * dx + dy * dy + dz * dz);
}

static double Length(ref readonly Point3D p) => Math.Round(Math.Sqrt(p.X * p.X + p.Y * p.Y + p.Z * p.Z),
3);

readonly struct Point3D(double x, double y, double z)
{
    public double X { get; } = x;
    public double Y { get; } = y;
    public double Z { get; } = z;
}
```

OUTPUT

```
5
3.742
```

9.2.5 params arrays and collections

A `params` parameter accepts **any number** of arguments, which the compiler packs into an array; the caller may also pass an existing array. It must be the last parameter, and there can be only one. Since C# 13, `params` also works with collection types such as `ReadOnlySpan<T>`, `List<T>` and `IEnumerable<T>`; the span version avoids allocating an array. `Console.WriteLine(format, params object[] args)` and `string.Join` are familiar params methods.

PARAMS: A VARIABLE NUMBER OF ARGUMENTS

Program.cs

```
Console.WriteLine(Sum());                          // zero arguments
Console.WriteLine(Sum(5));
Console.WriteLine(Sum(1, 2, 3, 4));
int[] values = { 10, 20, 30 };
Console.WriteLine(Sum(values));                    // pass an array directly
PrintAll("Fruits", "mango", "apple");
Console.WriteLine(Average(90, 80, 70));

static int Sum(params int[] numbers)
{
    int total = 0;
    foreach (int n in numbers) total += n;
    return total;
}

static void PrintAll(string title, params string[] items) =>
    Console.WriteLine($"{title} ({items.Length}): {string.Join(", ", items)}");

static double Average(params ReadOnlySpan<int> xs)   // C# 13: no array allocated
{
    int total = 0;
    foreach (int x in xs) total += x;
    return (double)total / xs.Length;
}
```

OUTPUT

```
0
5
10
60
Fruits (2): mango, apple
80
```

9.2.6 Optional parameters with default values

A parameter can have a **default value**, making the argument optional. Optional parameters must come after all required ones, and the default must be a compile-time constant (a literal, `const`, `null` or `default`). They replace many trivial overloads.

OPTIONAL PARAMETERS

Program.cs

```
Console.WriteLine(Price(1000));                  // both defaults
Console.WriteLine(Price(1000, 10));              // discount given, default tax
Console.WriteLine(Price(1000, 10, 5));           // everything given
Greet();
Greet("Ravi", "Good evening");

static double Price(double amount, double discountPct = 0, double taxPct = 18)
{
    double afterDiscount = amount * (1 - discountPct / 100);
    return afterDiscount * (1 + taxPct / 100);
}

static void Greet(string name = "guest", string greeting = "Hello") =>
    Console.WriteLine($"{greeting}, {name}!");
```

OUTPUT

```
1180
1062
945
Hello, guest!
Good evening, Ravi!
```

9.2.7 Named arguments

Arguments can be passed by **name**: `name: value`. Named arguments make calls self-explanatory (especially for bool and number literals), may be written in any order, and let you skip optional parameters in the middle. Positional arguments may be mixed with named ones as long as each is in its correct position.

NAMED ARGUMENTS

Program.cs

```
Console.WriteLine(Price(1000, taxPct: 5));               // skip discountPct
Console.WriteLine(Price(taxPct: 12, amount: 500));         // any order
CreateUser("asha", isAdmin: true, sendEmail: false);       // clearer than (..., true, false)

static double Price(double amount, double discountPct = 0, double taxPct = 18) =>
    amount * (1 - discountPct / 100) * (1 + taxPct / 100);

static void CreateUser(string name, bool isAdmin = false, bool sendEmail = true) =>
    Console.WriteLine($"user {name}: admin={isAdmin}, email={sendEmail}");
```

OUTPUT

```
1050
560
user asha: admin=True, email=False
```

9.2.8 Passing reference types vs value types

"Pass by value" copies the **variable**. For a value type (int, struct) the variable holds the data, so the method gets its own copy of the data. For a reference type (array, class object, List) the variable holds a **reference**, so the method gets a copy of the reference: both point to the same object. The method can therefore change the object's contents, but reassigning the parameter to a new object does not affect the caller. With `ref`, even reassignment is visible.

REFERENCE TYPES AND VALUE TYPES AS ARGUMENTS

Program.cs

```
int[] marks = { 50, 60 };
var p = new Point { X = 1 };

Modify(marks, p);
Console.WriteLine($"after Modify:   marks[0] = {marks[0]}, p.X = {p.X}");   // contents changed

Replace(marks);
Console.WriteLine($"after Replace:  marks[0] = {marks[0]}");               // still the old array

ReplaceRef(ref marks);
Console.WriteLine($"after ReplaceRef: marks[0] = {marks[0]}");             // caller's variable changed

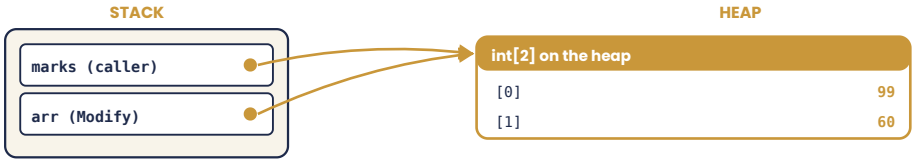
var s = new PointStruct { X = 1 };
ModifyStruct(s);
Console.WriteLine($"struct copy:    s.X = {s.X}");

static void Modify(int[] arr, Point pt) { arr[0] = 99; pt.X = 42; }
static void Replace(int[] arr) { arr = new[] { -1, -1 }; }          // only the copy of the reference
static void ReplaceRef(ref int[] arr) { arr = new[] { 7, 7 }; }
static void ModifyStruct(PointStruct ps) { ps.X = 500; }

class Point { public int X; }
struct PointStruct { public int X; }
```

OUTPUT

```
after Modify:   marks[0] = 99, p.X = 42
after Replace:  marks[0] = 99
after ReplaceRef: marks[0] = 7
struct copy:    s.X = 1
```



BOTH THE CALLER'S VARIABLE AND THE PARAMETER REFER TO THE SAME ARRAY

9.3 Returning multiple values

A method has one return type, but often needs to produce several results: minimum and maximum, quotient and remainder, success and value. C# offers out parameters, tuples, and (later) small classes or records. Tuples are usually the cleanest choice for a few related values.

9.3.1 Out parameters

TWO RESULTS THROUGH OUT PARAMETERS

Program.cs

```
int[] data = { 42, 7, 19, 88, 3 };
MinMax(data, out int min, out int max);
Console.WriteLine($"min {min}, max {max}");

static void MinMax(int[] a, out int min, out int max)
{
    min = max = a[0];
    foreach (int x in a)
    {
        if (x < min) min = x;
        if (x > max) max = x;
    }
}
```

OUTPUT

```
min 3, max 88
```

9.3.2 Tuples (int min, int max) and deconstruction

A **tuple** groups several values into one lightweight value: `(int min, int max)`. The elements can have names, and a tuple can be **deconstructed** into separate variables in one statement. Tuples are value types (`System.ValueTuple`), compare element by element with `==`, and are ideal for returning two or three related results.

TUPLES AND DECONSTRUCTION

Program.cs

```
int[] data = { 42, 7, 19, 88, 3 };

var result = MinMax(data);                          // a named tuple
Console.WriteLine($"min {result.min}, max {result.max}, whole: {result}");

(int lo, int hi) = MinMax(data);                    // deconstruction into new variables
Console.WriteLine($"range {hi - lo}");

var (_, biggest) = MinMax(data);                    // discard what you don't need
Console.WriteLine($"biggest {biggest}");

var stats = Stats(data);
Console.WriteLine($"sum {stats.Sum}, avg {stats.Average:F1}, count {stats.Count}");

int x = 1, y = 2;
(x, y) = (y, x);                                    // tuple swap
Console.WriteLine($"x={x} y={y}, equal tuples: {(1, "a") == (1, "a")}");

static (int min, int max) MinMax(int[] a)
{
    int min = a[0], max = a[0];
    foreach (int v in a) { if (v < min) min = v; if (v > max) max = v; }
    return (min, max);
}

static (int Sum, double Average, int Count) Stats(int[] a)
{
    int sum = 0;
    foreach (int v in a) sum += v;
    return (sum, (double)sum / a.Length, a.Length);
}
```

OUTPUT

```
min 3, max 88, whole: (3, 88)
range 85
biggest 88
sum 159, avg 31.8, count 5
x=2 y=1, equal tuples: True
```

TIP

Prefer tuples for private helpers and small results. When the same group of values is passed around widely or needs behaviour, define a `record` (Module 12) instead, so the type has a real name.

9.4 Method overloading

Overloading means declaring several methods with the same name but different parameter lists in the same type. Callers use one natural name (`Area`, `Print`, `Console.WriteLine`), and the compiler picks the right version from the arguments. `Console.WriteLine` alone has around 18 overloads.

9.4.1 Rules of overloading

- Overloads must differ in their **signature**: the number of parameters, their types, their order, or their ref kind (`ref` / `out` / `in` versus value).
- The **return type alone** does not distinguish overloads: `int Get()` and `double Get()` cannot coexist.
- Parameter **names** and the `params` modifier alone do not distinguish overloads.
- Overloads that differ only in `ref` versus `out` are not allowed.

- Keep overloads consistent: they should all do "the same thing" for different inputs.

OVERLOADED AREA METHODS Program.cs

```
Console.WriteLine($"circle    {Shapes.Area(2.0):F2}");
Console.WriteLine($"square    {Shapes.Area(4)}");
Console.WriteLine($"rectangle {Shapes.Area(4, 5)}");
Console.WriteLine($"triangle  {Shapes.Area(3.0, 4.0, 5.0):F1}");

static class Shapes
{
    public static double Area(double radius) => Math.PI * radius * radius;      // circle
    public static int Area(int side) => side * side;                            // square
    public static int Area(int length, int width) => length * width;            // rectangle
    public static double Area(double a, double b, double c)                     // triangle (Heron)
    {
        double s = (a + b + c) / 2;
        return Math.Sqrt(s * (s - a) * (s - b) * (s - c));
    }
}
```

OUTPUT

```
circle    12.57
square    16
rectangle 20
triangle  6.0
```

NOTE

Overloading applies to methods of a class or struct. **Local functions cannot be overloaded**: two local functions named `Area` in Program.cs give error CS0128 ("A local variable or function named 'Area' is already defined in this scope"). That is why the overloads above live in a `static class`.

9.4.2 Overload resolution

For each call, the compiler collects the overloads that **can** accept the arguments (directly or through implicit conversions) and picks the **best** one: an exact type match beats a conversion, and a conversion to a "closer" type beats one to a wider type (int to long is better than int to double, and any numeric conversion is better than boxing to object). If no single overload is best, the call is **ambiguous** and does not compile.

WHICH OVERLOAD IS CHOSEN?

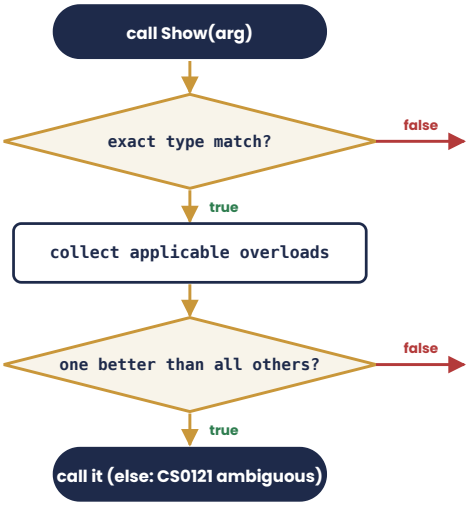
Program.cs

```
P.Show(5);          // int: exact match
P.Show(5L);         // long: exact match
P.Show(5.0f);       // float -> double (better than boxing to object)
P.Show('A');        // char -> int: the closest implicit numeric conversion
P.Show("hi");       // string -> object
P.Show(5m);         // decimal has no implicit conversion to double -> object

static class P
{
    public static void Show(int x) => Console.WriteLine($"int    {x}");
    public static void Show(long x) => Console.WriteLine($"long   {x}");
    public static void Show(double x) => Console.WriteLine($"double {x}");
    public static void Show(object x) => Console.WriteLine($"object {x}");
}
```

OUTPUT

```
int    5
long   5
double 5
int    65
object hi
object 5
```



OVERLOAD RESOLUTION IN SHORT

AN AMBIGUOUS CALL

Program.cs

```
P.Print(1, 2);      // both overloads need one int -> double conversion

static class P
{
    public static void Print(int a, double b) => Console.WriteLine("int, double");
    public static void Print(double a, int b) => Console.WriteLine("double, int");
}
```

COMPILER OUTPUT

```
Program.cs(1,3): error CS0121: The call is ambiguous between the following methods or properties:
'P.Print(int, double)' and 'P.Print(double, int)'
```

9.5 Recursion

A method is **recursive** when it calls itself. Recursion fits problems that are defined in terms of smaller versions of themselves: n! = n × (n - 1)!, a folder contains folders, a tree contains subtrees. Every recursive method needs a way to stop.

9.5.1 Base case and recursive case

A recursive method has two parts. The **base case** answers the smallest problem directly, without recursion. The **recursive case** breaks the problem into a smaller one, calls itself, and combines the result. Each call must move closer to the base case; otherwise the recursion never ends.

BASE CASE AND RECURSIVE CASE

Program.cs

```
Console.WriteLine($"5! = {Factorial(5)}");
Console.WriteLine($"sum of digits of 9045 = {DigitSum(9045)}");
CountDown(3);

static long Factorial(int n)
{
    if (n <= 1) return 1;                  // base case
    return n * Factorial(n - 1);           // recursive case: smaller problem
}

static int DigitSum(int n) => n < 10 ? n : n % 10 + DigitSum(n / 10);

static void CountDown(int n)
{
    if (n == 0) { Console.WriteLine("Go!"); return; }
    Console.Write($"{n}... ");
    CountDown(n - 1);
}
```

OUTPUT

```
5! = 120
sum of digits of 9045 = 18
3... 2... 1... Go!
```

9.5.2 The call stack and StackOverflowException

Every call pushes a **stack frame** (parameters, local variables, return address) onto the thread's call stack; returning pops it. A recursive call to `Factorial(4)` pushes frames for 4, 3, 2 and 1; the base case returns first, then each frame multiplies and returns in reverse order. The trace below makes the winding and unwinding visible.

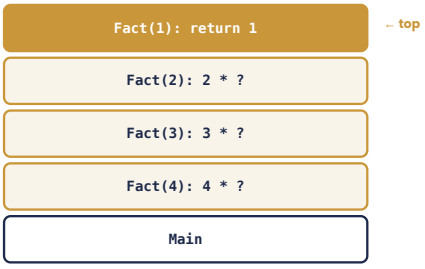
WATCHING THE CALL STACK Program.cs

```
Console.WriteLine($"result {Fact(4, 0)}");

static long Fact(int n, int depth)
{
    string pad = new string(' ', depth * 2);
    Console.WriteLine($"{pad}Fact({n}) called");
    if (n <= 1)
    {
        Console.WriteLine($"{pad}Fact(1) returns 1");
        return 1;
    }
    long r = n * Fact(n - 1, depth + 1);
    Console.WriteLine($"{pad}Fact({n}) returns {r}");
    return r;
}
```

OUTPUT

```
Fact(4) called
  Fact(3) called
    Fact(2) called
      Fact(1) called
      Fact(1) returns 1
    Fact(2) returns 2
  Fact(3) returns 6
Fact(4) returns 24
result 24
```



THE DEEPEST FRAME (THE BASE CASE) IS ON TOP AND RETURNS FIRST

The stack is limited (about 1 MB per thread by default). If recursion has no base case, or goes too deep (for example a million levels), the stack fills up and the runtime throws a `StackOverflowException`. Unlike other exceptions it **cannot be caught**: the process is terminated with the message "Stack overflow." So always check that the base case is reachable, and use a loop for very deep linear recursion.

INFINITE RECURSION C#

```
static int Broken(int n) => Broken(n - 1);    // no base case: crashes the process with "Stack overflow."
```

9.5.3 Types: direct, indirect, tail, tree recursion

Type	Description	Example
Direct	the method calls itself	`Factorial` calls `Factorial`
Indirect (mutual)	A calls B, B calls A	`IsEven` calls `IsOdd` and back
Tail	the recursive call is the very last action; nothing is left to do after it returns	`FactTail(n - 1, acc * n)`

Type	Description	Example
Tree (multiple)	each call makes two or more recursive calls, so the calls form a tree	`Fib(n - 1) + Fib(n - 2)`

KINDS OF RECURSION

Program.cs

```
Console.WriteLine($"IsEven(10) {IsEven(10)}, IsOdd(7) {IsOdd(7)}");
Console.WriteLine($"FactTail(10) {FactTail(10, 1)}");
Console.WriteLine($"Fib(10) {Fib(10)}");

static bool IsEven(int n) => n == 0 || IsOdd(n - 1);       // indirect recursion
static bool IsOdd(int n) => n != 0 && IsEven(n - 1);

static long FactTail(int n, long acc) =>                   // tail recursion: accumulator
    n <= 1 ? acc : FactTail(n - 1, acc * n);

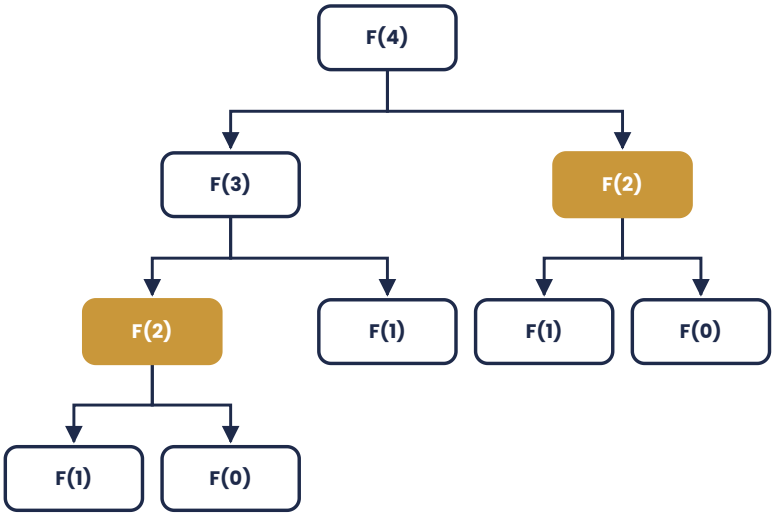
static int Fib(int n) => n < 2 ? n : Fib(n - 1) + Fib(n - 2);   // tree recursion
```

OUTPUT

```
IsEven(10) True, IsOdd(7) True
FactTail(10) 3628800
Fib(10) 55
```

NOTE

C# does not guarantee tail-call optimisation: even a tail-recursive method uses one stack frame per call (the JIT only sometimes removes them). For deep linear recursion in C#, convert it to a loop, which the tail form makes easy.



TREE RECURSION FOR FIB(4): F(2) IS COMPUTED TWICE; THE WASTE GROWS EXPONENTIALLY

9.5.4 Classic problems: factorial, Fibonacci, power, reverse a string, Tower of Hanoi, binary search, permutations

Power by repeated squaring uses $x^n = (x^{n/2})^2$, halving n at each step: O(log n) calls instead of n. **Reverse a string**: the reverse of s is the reverse of the rest followed by the first char. **Binary search** calls itself on one half. **Permutations** choose each remaining char in turn as the next one and recurse on what is left.

POWER, REVERSE, BINARY SEARCH AND PERMUTATIONS

Program.cs

```
Console.WriteLine($"Power(2, 10) = {Power(2, 10)}, Power(3, 5) = {Power(3, 5)}");
Console.WriteLine($"Reverse(\"recursion\") = {Reverse("recursion")}");

int[] sorted = { 3, 8, 12, 17, 21, 30, 42 };
Console.WriteLine($"BinarySearch 21 -> {BinarySearch(sorted, 21, 0, sorted.Length - 1)}, 5 ->
{BinarySearch(sorted, 5, 0, sorted.Length - 1)}");

var perms = new List<string>();
Permute("", "abc", perms);
Console.WriteLine($"{perms.Count} permutations: {string.Join(" ", perms)}");

static long Power(long x, int n)
{
    if (n == 0) return 1;
    long half = Power(x, n / 2);
    return n % 2 == 0 ? half * half : half * half * x;
}

static string Reverse(string s) => s.Length <= 1 ? s : Reverse(s[1..]) + s[0];

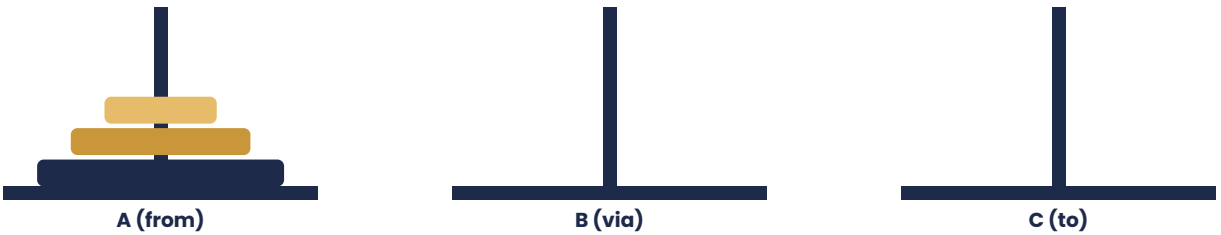
static int BinarySearch(int[] a, int target, int lo, int hi)
{
    if (lo > hi) return -1;                                  // base case: empty range
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == target) return mid;
    return a[mid] < target ? BinarySearch(a, target, mid + 1, hi)
                           : BinarySearch(a, target, lo, mid - 1);
}

static void Permute(string prefix, string rest, List<string> result)
{
    if (rest.Length == 0) { result.Add(prefix); return; }    // nothing left to place
    for (int i = 0; i < rest.Length; i++)
        Permute(prefix + rest[i], rest.Remove(i, 1), result);
}
```

OUTPUT

```
Power(2, 10) = 1024, Power(3, 5) = 243
Reverse("recursion") = noisrucer
BinarySearch 21 -> 4, 5 -> -1
6 permutations: abc acb bac bca cab cba
```

The **Tower of Hanoi** moves n disks from peg A to peg C using peg B, never placing a larger disk on a smaller one. The recursive idea: move the top n - 1 disks out of the way to B, move the largest disk to C, then move the n - 1 disks from B onto it. It needs $2^n - 1$ moves.



THREE DISKS START ON A; THE GOAL IS TO MOVE THEM ALL TO C

TOWER OF HANOI

Program.cs

```
int moves = 0;
Hanoi(3, 'A', 'C', 'B');
Console.WriteLine($"Total moves: {moves}");

void Hanoi(int n, char from, char to, char via)       // captures 'moves'
{
    if (n == 0) return;
    Hanoi(n - 1, from, via, to);                       // 1. clear the way
    Console.WriteLine($"Move disk {n} from {from} to {to}");
    moves++;
    Hanoi(n - 1, via, to, from);                       // 3. stack back on top
}
```

OUTPUT

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Total moves: 7
```

9.5.5 Memoization with a Dictionary

Tree recursion often recomputes the same subproblems: naive `Fib(30)` makes 2,692,537 calls although there are only 31 distinct values. **Memoization** stores each result the first time it is computed, typically in a `Dictionary`, and returns the stored value on later calls. The running time drops from exponential $O(\varphi^n)$ to linear $O(n)$. This is the top-down form of **dynamic programming**.

MEMOIZATION TURNS EXPONENTIAL INTO LINEAR

Program.cs

```
long calls = 0;
Console.WriteLine($"naive Fib(30) = {FibNaive(30)}, calls: {calls:N0}");

calls = 0;
var memo = new Dictionary<int, long>();
Console.WriteLine($"memo  Fib(30) = {FibMemo(30)}, calls: {calls:N0}");
calls = 0;
Console.WriteLine($"memo  Fib(50) = {FibMemo(50)}, calls: {calls:N0}");   // reuses earlier results too

long FibNaive(int n)
{
    calls++;
    return n < 2 ? n : FibNaive(n - 1) + FibNaive(n - 2);
}

long FibMemo(int n)
{
    calls++;
    if (n < 2) return n;
    if (memo.TryGetValue(n, out long known)) return known;      // already computed
    long result = FibMemo(n - 1) + FibMemo(n - 2);
    memo[n] = result;                                            // remember it
    return result;
}
```

OUTPUT

```
naive Fib(30) = 832040, calls: 2,692,537
memo  Fib(30) = 832040, calls: 59
memo  Fib(50) = 12586269025, calls: 41
```

NOTE

The second memoized call reuses the values up to Fib(30) already in the dictionary, so it needs only 41 calls to reach Fib(50). `calls:N0` assumes the en-US culture for the thousands separator.



CALLS NEEDED FOR FIB(30): THE MEMOIZED BAR IS TOO SHORT TO SEE

A method has an access modifier, optional static, a return type, a name and parameters; static methods are called on the type, instance methods on an object. Every path of a non-void method must return. Use expression bodies for one-liners and local functions for private helpers. Arguments are passed by value unless marked ref (two-way), out (results, Try pattern) or in (read-only reference); params takes any number of arguments, and optional plus named arguments make calls flexible and clear. Return several values with tuples and deconstruction. Overloads differ by parameter list, never by return type alone. Recursion needs a reachable base case; deep recursion overflows the stack, and memoization removes repeated work.

PRACTICE CHECKPOINT

Write a **MinMax** method returning a tuple, a **TryDivide** method using out, **overloaded Area** methods, and a recursive **Tower of Hanoi** for a number of disks entered by the user.

SAMPLE SOLUTION

Program.cs

```
int[] data = { 34, 7, 23, 32, 5, 62 };
var (min, max) = Tools.MinMax(data);
```

```
Console.WriteLine($"min {min}, max {max}");

Console.WriteLine(Tools.TryDivide(22, 7, out double q) ? $"22 / 7 = {q:F3}" : "cannot divide");
Console.WriteLine(Tools.TryDivide(5, 0, out _) ? "?" : "5 / 0: cannot divide");

Console.WriteLine($"circle r=1.5: {Tools.Area(1.5):F2}, square 6: {Tools.Area(6)}, rect 3x7:
{Tools.Area(3, 7)}");

Console.Write("Number of disks: ");
int n = int.Parse(Console.ReadLine()!);
int moves = Tools.Hanoi(n, 'A', 'C', 'B');
Console.WriteLine($"{moves} moves (2^{n} - 1 = {(1 << n) - 1})");

static class Tools
{
    public static (int Min, int Max) MinMax(int[] a)
    {
        int min = a[0], max = a[0];
        foreach (int x in a) { if (x < min) min = x; if (x > max) max = x; }
        return (min, max);
    }

    public static bool TryDivide(double a, double b, out double result)
    {
        result = 0;
        if (b == 0) return false;
        result = a / b;
        return true;
    }

    public static double Area(double radius) => Math.PI * radius * radius;
    public static int Area(int side) => side * side;
    public static int Area(int length, int width) => length * width;

    public static int Hanoi(int n, char from, char to, char via)
    {
        if (n == 0) return 0;
        int count = Hanoi(n - 1, from, via, to);
        Console.WriteLine($"  disk {n}: {from} -> {to}");
        return count + 1 + Hanoi(n - 1, via, to, from);
    }
}
```

CONSOLE (user input in gold)

```
min 5, max 62
22 / 7 = 3.143
5 / 0: cannot divide
circle r=1.5: 7.07, square 6: 36, rect 3x7: 21
Number of disks: 3
  disk 1: A -> C
  disk 2: A -> B
  disk 1: C -> B
  disk 3: A -> C
  disk 1: B -> A
  disk 2: B -> C
  disk 1: A -> C
7 moves (2^3 - 1 = 7)
```

LEVEL 2

Intermediate level

Outcome: design classes with OOP, use structs, records, exceptions, generics, collections, delegates, events, LINQ and files.

8 MODULES · 25 DAYS MINIMUM

Module 10 · Object-oriented programming: classes and objects	**4 days**
Module 11 · Inheritance, polymorphism, abstraction and interfaces	**4 days**
Module 12 · Structs, enums, records and nullable reference types	**2 days**
Module 13 · Exception handling	**2 days**
Module 14 · Generics and collections	**4 days**
Module 15 · Delegates, events and lambda expressions	**3 days**
Module 16 · LINQ (Language Integrated Query)	**4 days**
Module 17 · File handling and serialization	**2 days**

MODULE 10 · INTERMEDIATE

Object-oriented programming: classes and objects

Minimum duration **4 days** Level **Intermediate**

C# is built around classes. A class bundles data (fields and properties) with the code that works on that data (methods), and objects are the individual instances created from it. This module covers the ideas of OOP, defining classes, fields, every kind of property including C# 11 required members and the C# 14 field keyword, constructors including primary constructors, access modifiers and encapsulation, the other members a type can have, partial and nested classes, extension methods and C# 14 extension blocks, and the members every type inherits from System.Object.

IN THIS MODULE

10.1 OOP concepts
10.2 Classes and objects
10.3 Fields
10.4 Properties
10.5 Constructors
10.6 Access modifiers and encapsulation
10.7 Other class members
10.8 Partial classes and methods
10.9 Nested classes
10.10 Extension methods and extension members
10.11 System.Object members

10.1 OOP concepts

10.1.1 Procedural vs object-oriented programming

In **procedural** programming, a program is a sequence of procedures (methods) that operate on data stored separately, often in shared variables. This works well for small programs, but as they grow, any method can change any data, and a change in the data layout ripples through the whole code. **Object-oriented programming** (OOP) groups data and the operations on it into **objects**. Each object guards its own data and offers a small set of operations; the rest of the program talks to it only through those operations.

Aspect	Procedural	Object-oriented
Unit of organisation	functions/procedures	classes and objects
Data	separate from functions, often shared	kept inside objects, hidden behind members
Change	a data change affects every function using it	a change stays inside the class
Reuse	copy or call functions	inheritance, composition, interfaces
Models	steps of a process	things of the problem domain: Account, Order, Student
Examples	C, early BASIC	C#, Java, C++, Python

THE SAME IDEA IN TWO STYLES

Program.cs

```
// Procedural style: data and functions are separate; anyone can break the balance
decimal balance = 1000m;
balance = Withdraw(balance, 300m);
balance = -50m;                                  // nothing stops this
Console.WriteLine($"procedural balance: {balance}");

// Object-oriented style: the account protects its own balance
var acc = new Account(1000m);
acc.Withdraw(300m);
// acc.Balance = -50m;                           // compile error: setter is private
Console.WriteLine($"object balance: {acc.Balance}");

static decimal Withdraw(decimal bal, decimal amount) => bal - amount;

class Account
{
    public decimal Balance { get; private set; }
    public Account(decimal opening) => Balance = opening;
    public void Withdraw(decimal amount)
    {
        if (amount <= 0 || amount > Balance) throw new InvalidOperationException("Invalid amount");
        Balance -= amount;
    }
}
```

OUTPUT

```
procedural balance: -50
object balance: 700
```

10.1.2 The four pillars: encapsulation, abstraction, inheritance, polymorphism



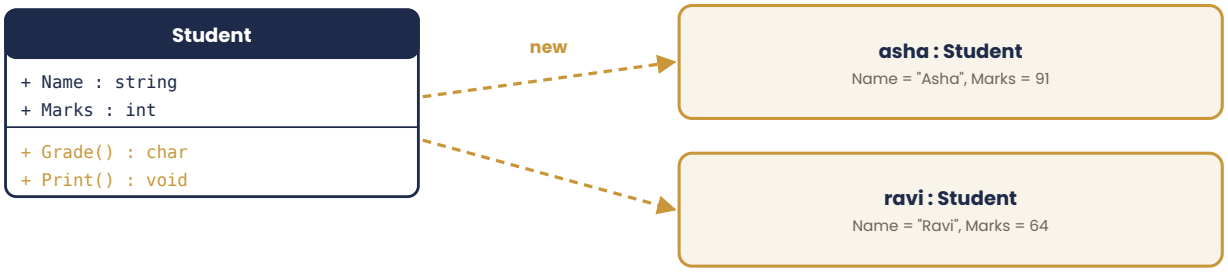
Module 10 focuses on encapsulation; Module 11 covers inheritance, polymorphism and abstraction in depth

THE FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

Pillar	Idea	C# features
Encapsulation	bundle data with methods and restrict direct access, so objects stay valid	private fields, properties, access modifiers
Abstraction	expose only the essential operations; hide details	public methods, interfaces, abstract classes
Inheritance	derive a specialised class from a general one and reuse its code	`class Dog : Animal`, `base`
Polymorphism	treat different objects through a common type; each responds in its own way	`virtual` / `override`, interfaces, overloading

10.2 Classes and objects

A **class** is a blueprint: it describes what data each object will hold and what it can do. An **object** (or instance) is one concrete thing built from that blueprint, with its own values. The class `Student` is written once; `asha` and `ravi` are two objects of it.



ONE CLASS, MANY OBJECTS: EACH OBJECT HAS ITS OWN COPY OF THE INSTANCE DATA

10.2.1 Defining a class

A class is declared with the `class` keyword, a name in PascalCase and a body containing its **members**: fields, properties, constructors, methods, events, indexers and nested types. By convention each public class goes in its own file named after it (Student.cs), although several small classes may share a file while learning.

A FIRST CLASS Program.cs

```
var s = new Student();
s.Name = "Asha";
s.Marks = 91;
s.Print();
Console.WriteLine($"grade {s.Grade()}");

public class Student                        // class declaration
{
    public string Name = "";                // fields: the object's data
    public int Marks;

    public char Grade() => Marks >= 85 ? 'A' : Marks >= 60 ? 'B' : 'C';   // methods: behaviour

    public void Print() => Console.WriteLine($"{Name} scored {Marks}");
}
```

OUTPUT

```
Asha scored 91
grade A
```

NOTE

Public fields are used here only to keep the first example short. Real classes keep fields private and expose properties (section 10.4), which is what encapsulation means in practice.

10.2.2 Creating objects with new and target-typed new()

The `new` operator allocates the object on the managed heap, runs a constructor and returns a **reference** to it. When the type is already known from the variable, field or parameter, C# 9 allows the shorter **target-typed** `new()`.

THREE WAYS TO WRITE NEW

Program.cs

```
Student a = new Student();                  // explicit type on both sides
var b = new Student();                      // type inferred from the right side
Student c = new();                          // target-typed new: type taken from the left side
List<Student> list = new() { a, b, c };     // works with any known target type

PrintStudent(new() { Name = "Meera", Marks = 77 });   // target type comes from the parameter
Console.WriteLine($"{list.Count} students, a and b are different objects: {!ReferenceEquals(a, b)}");

static void PrintStudent(Student s) => Console.WriteLine($"{s.Name}: {s.Marks}");

public class Student { public string Name = ""; public int Marks; }
```

OUTPUT

```
Meera: 77
3 students, a and b are different objects: True
```

10.2.3 Reference semantics and null

Classes are **reference types**. A variable of a class type holds a reference (an address) to an object, not the object itself. Assigning one variable to another copies the reference, so both refer to the **same** object, and a change made through one is visible through the other. A reference can also be `null`, meaning it refers to no object; accessing a member through null throws `NullReferenceException`.

TWO VARIABLES, ONE OBJECT

Program.cs

```
var asha = new Student { Name = "Asha", Marks = 91 };
var alias = asha;                           // copies the reference, not the object
alias.Marks = 95;
Console.WriteLine($"asha.Marks = {asha.Marks}");    // 95: same object

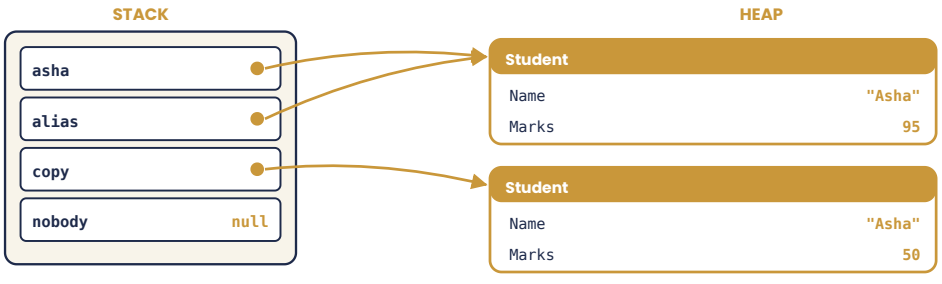
var copy = new Student { Name = asha.Name, Marks = asha.Marks };   // a real copy
copy.Marks = 50;
Console.WriteLine($"asha.Marks = {asha.Marks}, copy.Marks = {copy.Marks}");

Student? nobody = null;
Console.WriteLine(nobody?.Name ?? "(no student)");  // ?. and ?? guard against null
try { Console.WriteLine(nobody!.Name); }
catch (NullReferenceException) { Console.WriteLine("NullReferenceException"); }

public class Student { public string Name = ""; public int Marks; }
```

OUTPUT

```
asha.Marks = 95
asha.Marks = 95, copy.Marks = 50
(no student)
NullReferenceException
```



ASHA AND ALIAS SHARE ONE OBJECT; COPY IS A SEPARATE OBJECT; NOBODY REFERS TO NOTHING

10.3 Fields

A **field** is a variable declared directly in a class. Fields hold the state of an object (or of the class itself). They are normally `private` and named in camelCase, often with a leading underscore: `_balance`. Fields that are not explicitly initialised get their default value (0, false, null).

10.3.1 Instance and static fields

An **instance field** exists once per object: every Student has its own Name. A **static field** exists once per class, shared by all objects; it is accessed through the class name. Static fields are used for counters, caches and shared configuration.

INSTANCE FIELDS VERSUS A SHARED STATIC FIELD — Program.cs

```
var a = new Ticket("Asha");
var b = new Ticket("Ravi");
var c = new Ticket("Meera");
Console.WriteLine($"{a} | {b} | {c}");
Console.WriteLine($"tickets issued: {Ticket.Issued}");

public class Ticket
{
    private static int _next = 100;        // static: one shared counter for the class
    public static int Issued;              // static: shared

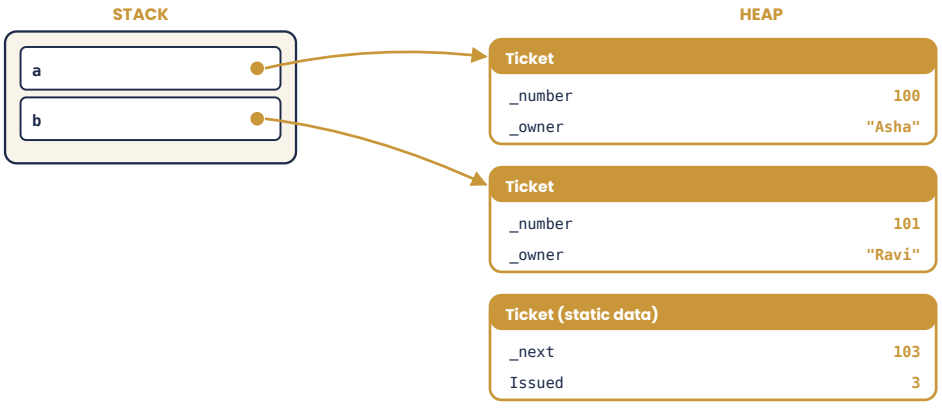
    private readonly int _number;          // instance: one per object
    private readonly string _owner;

    public Ticket(string owner)
    {
        _owner = owner;
        _number = _next++;                 // each object takes the next number
        Issued++;
    }

    public override string ToString() => $"#{_number} {_owner}";
}
```

OUTPUT

```
#100 Asha | #101 Ravi | #102 Meera
tickets issued: 3
```



INSTANCE FIELDS LIVE IN EACH OBJECT; STATIC FIELDS LIVE ONCE, WITH THE TYPE

10.3.2 readonly and const fields

A `const` field is a compile-time constant: its value is written into the code wherever it is used, it must be a literal-like value (numbers, bool, char, string, enum, null), and it is implicitly static. A `readonly` field can be assigned only in its declaration or in a constructor; after construction it never changes. readonly works for any type and its value can be computed at run time.

	`const`	`readonly`
Assigned	at the declaration only	at the declaration or in a constructor
Value known	at compile time	at run time
Types	primitives, string, enum, null	any type
Per object?	no, implicitly static	instance or static (`static readonly`)
Example	`const int MaxSeats = 40;`	`readonly DateTime _created = DateTime.Now;`

CONST AND READONLY Program.cs

```
var c = new Circle(2);
Console.WriteLine($"PI = {Circle.Pi}, area = {c.Area():F2}, id = {c.Id}");
Console.WriteLine($"max radius {Circle.MaxRadius}");

public class Circle
{
    public const double Pi = 3.14159;                       // compile-time constant
    public static readonly double MaxRadius = Math.Sqrt(10_000);   // computed once at run time
    public readonly string Id;                               // set once per object
    private readonly double _radius;

    public Circle(double radius)
    {
        _radius = radius;                                    // allowed: inside the constructor
        Id = $"C-{radius}";
    }

    public double Area() => Pi * _radius * _radius;
    // public void Grow() { _radius *= 2; }                  // error CS0191: readonly field
}
```

OUTPUT

```
PI = 3.14159, area = 12.57, id = C-2
max radius 100
```

WATCH OUT

A public `const` is copied into every assembly that uses it. If you later change its value in a library, programs compiled against the old version keep the old number until they are recompiled. Use `static readonly` for values that might change between versions.

10.4 Properties

A **property** looks like a field to the caller (`acc.Balance`) but is really a pair of methods: a `get` accessor that returns a value and a `set` accessor that receives a new value in the implicit parameter `value`. Properties are the standard way to expose data in C# because the class keeps control: it can validate, compute, log or restrict access without changing the calling code.

10.4.1 Full properties with backing fields

A full property stores its value in a private **backing field** and adds logic in the accessors.

A PROPERTY WITH A BACKING FIELD AND VALIDATION Program.cs

```
var p = new Product();
p.Name = "  Notebook  ";
p.Price = 120m;
Console.WriteLine($"[{p.Name}] {p.Price}");
try { p.Price = -5m; }
catch (ArgumentOutOfRangeException e) { Console.WriteLine(e.Message); }

public class Product
{
    private string _name = "";             // backing fields
    private decimal _price;

    public string Name
    {
        get { return _name; }
        set { _name = value.Trim(); }      // 'value' is the incoming value
    }

    public decimal Price
    {
        get => _price;
        set
        {
            if (value < 0) throw new ArgumentOutOfRangeException(nameof(value), "Price cannot be
negative.");
            _price = value;
        }
    }
}
```

OUTPUT

```
[Notebook] 120
Price cannot be negative. (Parameter 'value')
```

10.4.2 Auto-implemented properties

When the accessors would only read and write the field, an **auto-property** `{ get; set; }` lets the compiler generate the hidden backing field. It can have an initial value, and each accessor can have its own access level, for example `{ get; private set; }` so that only the class itself can change the value.

AUTO-IMPLEMENTED PROPERTIES

Program.cs

```
var s = new Student { Name = "Asha" };
s.AddMarks(40); s.AddMarks(45);
Console.WriteLine($"{s.Name}: {s.Total}, city {s.City}");
// s.Total = 1000;                          // error CS0272: the set accessor is inaccessible

public class Student
{
    public string Name { get; set; } = "";             // auto-property with initializer
    public string City { get; set; } = "Pune";
    public int Total { get; private set; }             // public read, private write

    public void AddMarks(int m) => Total += m;
}
```

OUTPUT

```
Asha: 85, city Pune
```

10.4.3 Read-only and init-only properties (init)

A property with only `get` is **read-only**: it can be assigned in the constructor or initializer and never again. An `init` accessor (C# 9) allows the value to be set in an **object initializer** (or constructor) but not afterwards, giving immutable objects that are still convenient to create.

GET-ONLY AND INIT PROPERTIES

Program.cs

```
var b = new Book("978-0134685991") { Title = "Effective Java", Year = 2018 };
Console.WriteLine($"{b.Isbn} {b.Title} ({b.Year}), created {b.CreatedYear}");
// b.Title = "Other";                       // error CS8852: init-only property
// b.Isbn = "x";                            // error CS0200: read-only property

public class Book
{
    public Book(string isbn) => Isbn = isbn;

    public string Isbn { get; }                        // read-only: constructor only
    public string Title { get; init; } = "";           // init-only: initializer or constructor
    public int Year { get; init; }
    public int CreatedYear { get; } = 2026;            // read-only with initializer
}
```

OUTPUT

```
978-0134685991 Effective Java (2018), created 2026
```

10.4.4 required members (C# 11)

An init property can still be forgotten in the initializer. Marking a property or field `required` makes the compiler insist that every object initializer sets it. A constructor that does set them can be marked `[SetsRequiredMembers]` so callers using it are not forced to repeat them.

REQUIRED MEMBERS

Program.cs

```
using System.Diagnostics.CodeAnalysis;

var u = new User { Email = "asha@example.com", Name = "Asha" };
Console.WriteLine($"{u.Name} <{u.Email}> role={u.Role}");
var v = new User("ravi@example.com", "Ravi");
Console.WriteLine($"{v.Name} <{v.Email}>");
// var w = new User { Name = "X" };       // error CS9035: required member 'User.Email' must be set

public class User
{
    public required string Email { get; init; }
    public required string Name { get; init; }
    public string Role { get; init; } = "student";

    public User() { }

    [SetsRequiredMembers]
    public User(string email, string name) { Email = email; Name = name; }
}
```

OUTPUT

```
Asha <asha@example.com> role=student
Ravi <ravi@example.com>
```

10.4.5 Computed (expression-bodied) properties

A property does not have to store anything: a **computed property** calculates its value from other data each time it is read. With an expression body `=>` it fits on one line. Computed properties never go out of date, because they are recalculated on every read.

COMPUTED PROPERTIES

Program.cs

```
var r = new Rect { Width = 4, Height = 2.5 };
Console.WriteLine($"area {r.Area}, perimeter {r.Perimeter}, square? {r.IsSquare}");
r.Width = 2.5;
Console.WriteLine($"area {r.Area}, perimeter {r.Perimeter}, square? {r.IsSquare}");

public class Rect
{
    public double Width { get; set; }
    public double Height { get; set; }

    public double Area => Width * Height;                // computed on every read
    public double Perimeter => 2 * (Width + Height);
    public bool IsSquare => Width == Height;
}
```

OUTPUT

```
area 10, perimeter 13, square? False
area 6.25, perimeter 10, square? True
```

10.4.6 The field keyword in property accessors (C# 14)

Before C# 14, adding even one line of logic to an auto-property (such as trimming input) forced you to write a full property with a declared backing field. In C# 14 the contextual keyword `field` refers to the compiler-generated backing field inside an accessor, so you can keep the short auto-property form and add logic to only the accessor that needs it.

FIELD: LOGIC WITHOUT DECLARING A BACKING FIELD Program.cs

```
var c = new Contact { Name = "  Asha Rao ", Phone = "98765-43210" };
Console.WriteLine($"[{c.Name}] [{c.Phone}]");
c.Age = 21;
try { c.Age = -3; } catch (ArgumentOutOfRangeException) { Console.WriteLine("age rejected"); }
Console.WriteLine(c.Age);

public class Contact
{
    public string Name { get; set => field = value.Trim(); } = "";          // C# 14

    public string Phone { get; set => field = value.Replace("-", ""); } = "";

    public int Age
    {
        get;
        set => field = value >= 0 ? value : throw new ArgumentOutOfRangeException(nameof(value));
    }
}
```

OUTPUT

```
[Asha Rao] [9876543210]
age rejected
21
```

NOTE

If a class already has a member called `field`, write `@field` or `this.field` to refer to it inside the accessor. The field keyword needs C# 14 (.NET 10 SDK).

10.5 Constructors

A **constructor** is a special method that runs when an object is created with `new`. It has the same name as the class and no return type. Its job is to put the new object into a valid initial state.

10.5.1 Default and parameterized constructors

If a class declares no constructor at all, the compiler provides a public **parameterless** (default) constructor that leaves fields at their initial values. As soon as you write any constructor, that default one is no longer generated. A **parameterized** constructor takes the values needed to build a valid object.

DEFAULT AND PARAMETERIZED CONSTRUCTORS

Program.cs

```
var a = new Point();                        // parameterless constructor
var b = new Point(3, 4);                    // parameterized constructor
Console.WriteLine($"{a} {b}");

public class Point
{
    public int X { get; }
    public int Y { get; }

    public Point() { X = 0; Y = 0; }                        // explicit default constructor
    public Point(int x, int y) { X = x; Y = y; }

    public override string ToString() => $"({X}, {Y})";
}
```

OUTPUT

```
(0, 0) (3, 4)
```

10.5.2 Constructor overloading and chaining with this(...)

A class may have several constructors with different parameter lists (overloading). To avoid repeating initialisation code, one constructor can call another of the same class with `: this(...)` written before its body. The called constructor runs first. Usually the simpler constructors chain to the most complete one.

CONSTRUCTOR CHAINING

Program.cs

```
var a = new Employee("Asha");
var b = new Employee("Ravi", "Sales");
var c = new Employee("Meera", "IT", 65000m);
foreach (var e in new[] { a, b, c }) Console.WriteLine(e);

public class Employee
{
    public string Name { get; }
    public string Dept { get; }
    public decimal Salary { get; }

    public Employee(string name) : this(name, "General") { }                // chains
    public Employee(string name, string dept) : this(name, dept, 30000m) { } // chains

    public Employee(string name, string dept, decimal salary)               // does the work
    {
        Console.WriteLine($"  main constructor for {name}");
        Name = name; Dept = dept; Salary = salary;
    }

    public override string ToString() => $"{Name} / {Dept} / {Salary}";
}
```

OUTPUT

```
  main constructor for Asha
  main constructor for Ravi
  main constructor for Meera
Asha / General / 30000
Ravi / Sales / 30000
Meera / IT / 65000
```

TIP

Optional parameters (Module 9) are an alternative to many chained constructors: `Employee(string name, string dept = "General", decimal salary = 30000m)`. Chaining remains useful when the overloads take genuinely different inputs.

10.5.3 Static constructors

A **static constructor** initialises static data. It has no access modifier and no parameters, runs automatically **once**, before the first use of the class (creating an instance or accessing a static member), and cannot be called directly. It is used for setup such as loading configuration or building lookup tables.

A STATIC CONSTRUCTOR RUNS ONCE Program.cs

```
Console.WriteLine("program starts");
Console.WriteLine($"rate = {Config.TaxRate}");      // triggers the static constructor
Console.WriteLine($"rate = {Config.TaxRate}");      // already initialised: does not run again
var c = new Config();

public class Config
{
    public static readonly decimal TaxRate;

    static Config()                                 // static constructor
    {
        Console.WriteLine("static constructor runs");
        TaxRate = 0.18m;
    }

    public Config() => Console.WriteLine("instance constructor runs");
}
```

OUTPUT

```
program starts
static constructor runs
rate = 0.18
rate = 0.18
instance constructor runs
```

10.5.4 Primary constructors (C# 12)

A **primary constructor** puts the parameters directly after the class name: `class Account(string owner, decimal opening)`. The parameters are in scope throughout the class body, so they can initialise properties or be used in methods. Any other constructor must chain to the primary one with `this(...)`. Note that primary constructor parameters are not properties: to expose one, assign it to a property.

A CLASS WITH A PRIMARY CONSTRUCTOR Program.cs

```
var acc = new Account("Asha", 1000m);
acc.Deposit(500m);
Console.WriteLine($"{acc.Owner}: {acc.Balance}");
var guest = new Account("Guest");
Console.WriteLine($"{guest.Owner}: {guest.Balance}");

public class Account(string owner, decimal opening)       // primary constructor
{
    public string Owner { get; } = owner;                  // expose as a property
    public decimal Balance { get; private set; } = opening;

    public Account(string owner) : this(owner, 0m) { }     // must chain to the primary

    public void Deposit(decimal amount)
    {
        if (amount <= 0) throw new ArgumentException($"{Owner}: deposit must be positive");
        Balance += amount;
    }
}
```

OUTPUT

```
Asha: 1500
Guest: 0
```

NOTE

Primary constructor parameters are mutable captured variables, not readonly fields. Use them to initialise members, and avoid modifying them inside methods. For simple immutable data types, a `record` with a primary constructor (Module 12) generates the properties automatically.

10.5.5 Object initializers

An **object initializer** sets accessible properties and fields right after construction, in braces: `new Student { Name = "Asha", Marks = 91 }`. It runs after the constructor, works with constructor arguments too, and can be nested. **Collection initializers** fill collections the same way.

OBJECT AND COLLECTION INITIALIZERS

Program.cs

```
var order = new Order(101)
{
    Customer = "Asha",
    Address = new Address { City = "Pune", Pin = "411001" },     // nested initializer
    Items = { "pen", "notebook" }                                 // adds to the existing list
};
Console.WriteLine($"#{order.Id} {order.Customer}, {order.Address.City} {order.Address.Pin}, items:
{string.Join(", ", order.Items)}");

var scores = new Dictionary<string, int> { ["Asha"] = 91, ["Ravi"] = 78 };   // index initializer
Console.WriteLine($"Ravi: {scores["Ravi"]}");

public class Order(int id)
{
    public int Id { get; } = id;
    public string Customer { get; set; } = "";
    public Address Address { get; set; } = new();
    public List<string> Items { get; } = new();
}

public class Address { public string City { get; set; } = ""; public string Pin { get; set; } = ""; }
```

OUTPUT

```
#101 Asha, Pune 411001, items: pen, notebook
Ravi: 78
```

10.5.6 Private constructors

A `private` constructor prevents code outside the class from using `new`. This is used to control how objects are created: through a **static factory method** that can validate, cache or choose the type, or for the **Singleton** pattern that allows only one instance. (A class containing only static members should be a `static class` instead, see 10.7.2.)

FACTORY METHODS AND A SINGLETON

Program.cs

```
var t1 = Temperature.FromCelsius(25);
var t2 = Temperature.FromFahrenheit(212);
Console.WriteLine($"{t1.Celsius}°C, {t2.Celsius}°C");
// var t3 = new Temperature(5);            // error CS0122: inaccessible due to its protection level

var log1 = Logger.Instance;
var log2 = Logger.Instance;
log1.Write("started");
Console.WriteLine($"same logger: {ReferenceEquals(log1, log2)}");

public class Temperature
{
    public double Celsius { get; }
    private Temperature(double c) => Celsius = c;          // only the factories can call this

    public static Temperature FromCelsius(double c) => new(c);
    public static Temperature FromFahrenheit(double f) => new((f - 32) * 5 / 9);
}

public sealed class Logger
{
    public static Logger Instance { get; } = new();         // the single instance
    private Logger() { }
    public void Write(string msg) => Console.WriteLine($"[log] {msg}");
}
```

OUTPUT

```
25°C, 100°C
[log] started
same logger: True
```

10.6 Access modifiers and encapsulation

Encapsulation means an object keeps its data private and lets the outside world change it only through members that keep it valid. Access modifiers are the tool: they decide which code may see each type and member.

10.6.1 public, private, protected, internal, protected internal, private protected, file

Modifier	Accessible from
`public`	any code in any assembly
`private`	only the containing class or struct
`protected`	the containing class and classes derived from it
`internal`	any code in the same assembly (project)
`protected internal`	the same assembly OR derived classes anywhere
`private protected`	derived classes that are also in the same assembly
`file`	a **type** visible only inside its source file (C# 11)



FROM MOST OPEN TO MOST RESTRICTED

10.6.2 Default access levels

When no modifier is written, C# chooses the most restrictive sensible level. Writing modifiers explicitly is still good style, because it shows the decision was intentional.

Declaration	Default access	Allowed
Top-level class, struct, interface, enum, record	`internal`	public or internal (or file)
Class or struct members (fields, methods, ...)	`private`	all modifiers (structs: no protected)
Nested types	`private`	all modifiers
Interface members	`public`	public, and others since C# 8
Enum members	`public`	always public

10.6.3 Validation inside properties

Because every change to the data goes through a property setter or method, that is the single place to enforce the rules. The object can then never be in an invalid state: a negative age, an empty name or a marks value of 150 is rejected at the moment someone tries to set it.

THE OBJECT GUARDS ITS OWN RULES

Program.cs

```
var s = new Student("Asha", 20);
s.Marks = 88;
Console.WriteLine(s);
foreach (Action bad in new Action[] { () => s.Marks = 150, () => s.Name = " ", () => s.Age = 3 })
{
    try { bad(); }
    catch (ArgumentException e) { Console.WriteLine($"rejected: {e.Message}"); }
}
Console.WriteLine(s);                       // still valid

public class Student
{
    private string _name = "";
    private int _age, _marks;

    public Student(string name, int age) { Name = name; Age = age; }   // reuse the validation

    public string Name
    {
        get => _name;
        set => _name = string.IsNullOrWhiteSpace(value) ? throw new ArgumentException("Name is required")
: value.Trim();
    }
    public int Age
    {
        get => _age;
        set => _age = value is >= 16 and <= 100 ? value : throw new ArgumentException($"Age {value} is
out of range");
    }
    public int Marks
    {
        get => _marks;
        set => _marks = value is >= 0 and <= 100 ? value : throw new ArgumentException($"Marks {value}
must be 0-100");
    }
    public override string ToString() => $"{Name}, {Age}, marks {Marks}";
}
```

OUTPUT

```
Asha, 20, marks 88
rejected: Marks 150 must be 0-100
rejected: Name is required
rejected: Age 3 is out of range
Asha, 20, marks 88
```

10.7 Other class members

Besides fields, properties, constructors and methods, a class can declare indexers, operators, conversion operators and a finalizer, and can refer to itself with `this`.

10.7.1 this keyword

Inside an instance member, `this` is a reference to the current object. It is used to tell a field apart from a parameter with the same name, to pass the current object to another method, to return the object for **fluent** method chaining, and (as `this(...)`) to chain constructors or declare indexers.

USES OF THIS

Program.cs

```
var b = new HtmlBuilder("ul");
b.Add("Mango").Add("Apple").Add("Kiwi");           // fluent chaining: each call returns this
Console.WriteLine(b.Build());

var team = new Team("Blue");
var p = new Player("Asha");
p.Join(team);
Console.WriteLine($"{team.Name} has {string.Join(", ", team.Members)}");

public class HtmlBuilder
{
    private readonly string tag;
    private readonly List<string> items = new();

    public HtmlBuilder(string tag) => this.tag = tag;      // this.tag is the field, tag the parameter

    public HtmlBuilder Add(string item)
    {
        items.Add($"<li>{item}</li>");
        return this;                                       // return the current object
    }

    public string Build() => $"<{tag}>{string.Concat(items)}</{tag}>";
}

public class Team(string name)
{
    public string Name { get; } = name;
    public List<string> Members { get; } = new();
    public void Register(Player p) => Members.Add(p.Name);
}

public class Player(string name)
{
    public string Name { get; } = name;
    public void Join(Team t) => t.Register(this);          // pass the current object
}
```

OUTPUT

```
<ul><li>Mango</li><li>Apple</li><li>Kiwi</li></ul>
Blue has Asha
```

10.7.2 Static members and static classes

Static methods, properties and fields belong to the type rather than to any object; they cannot use `this` or instance members directly. A `static class` can contain only static members, cannot be instantiated or inherited, and is the right home for utility functions such as `Math`, `Console` or `File`. The directive `using static` imports a type's static members so they can be called without the type name.

A STATIC UTILITY CLASS

Program.cs

```
using static System.Math;                  // call Sqrt, PI, Max without "Math."

Console.WriteLine($"{Converter.KmToMiles(42.195):F2} miles");
Console.WriteLine($"{Converter.CToF(37)} F, conversions done: {Converter.Count}");
Console.WriteLine($"hypotenuse {Sqrt(3 * 3 + 4 * 4)}, circle area {PI * 2 * 2:F2}, max {Max(7, 3)}");
// var c = new Converter();                // error CS0712: cannot create an instance of a static class

public static class Converter
{
    public static int Count { get; private set; }
    private const double MilesPerKm = 0.621371;

    public static double KmToMiles(double km) { Count++; return km * MilesPerKm; }
    public static double CToF(double c) { Count++; return c * 9 / 5 + 32; }
}
```

OUTPUT

```
26.22 miles
98.6 F, conversions done: 2
hypotenuse 5, circle area 12.57, max 7
```

10.7.3 Indexers

An **indexer** lets objects be indexed with square brackets like an array: `gradebook["Asha"]`. It is declared like a property named `this` with parameters in square brackets, and it can be overloaded on the parameter types. Collections such as `List<T>` and `Dictionary<K,V>` are built on indexers.

AN INDEXER WITH INT AND STRING KEYS

Program.cs

```
var week = new WeekTemps();
week["Mon"] = 31.5;
week[2] = 29.0;                          // Wed by number
week["Fri"] = 33.2;
Console.WriteLine($"Mon {week[0]}, Wed {week["Wed"]}, Fri {week["fri"]}");
try { week["Xyz"] = 1; } catch (ArgumentException e) { Console.WriteLine(e.Message); }

public class WeekTemps
{
    private static readonly string[] Days = { "Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun" };
    private readonly double[] temps = new double[7];

    public double this[int day]                      // indexer by position
    {
        get => temps[day];
        set => temps[day] = value;
    }

    public double this[string day]                   // overloaded indexer by name
    {
        get => temps[IndexOf(day)];
        set => temps[IndexOf(day)] = value;
    }

    private static int IndexOf(string day)
    {
        int i = Array.FindIndex(Days, d => d.Equals(day, StringComparison.OrdinalIgnoreCase));
        return i >= 0 ? i : throw new ArgumentException($"Unknown day '{day}'");
    }
}
```

OUTPUT

```
Mon 31.5, Wed 29, Fri 33.2
Unknown day 'Xyz'
```

10.7.4 Operator overloading

A class or struct can define what operators such as `+`, `-`, `*`, `==` and `<` mean for its objects, so that mathematical types read naturally: `a + b` instead of `a.Add(b)`. An operator is a `public static` method named `operator +`. Some operators must be overloaded in pairs: `==` with `!=`, and `<` with `>`. When overloading `==`, also override `Equals` and `GetHashCode` (10.11.2).

A FRACTION TYPE WITH OPERATORS

Program.cs

```
var half = new Fraction(1, 2);
var third = new Fraction(1, 3);
Console.WriteLine($"{half} + {third} = {half + third}");
Console.WriteLine($"{half} - {third} = {half - third}");
Console.WriteLine($"{half} * {third} = {half * third}");
Console.WriteLine($"-{half} = {-half}");
Console.WriteLine($"2/4 == 1/2: {new Fraction(2, 4) == half}, 1/2 > 1/3: {half > third}");

public class Fraction
{
    public int Num { get; }
    public int Den { get; }

    public Fraction(int num, int den)
    {
        if (den == 0) throw new DivideByZeroException();
        int g = Gcd(Math.Abs(num), Math.Abs(den));
        int sign = den < 0 ? -1 : 1;
        Num = sign * num / g;                          // always stored in lowest terms
        Den = sign * den / g;
    }

    private static int Gcd(int a, int b) => b == 0 ? Math.Max(a, 1) : Gcd(b, a % b);

    public static Fraction operator +(Fraction a, Fraction b) => new(a.Num * b.Den + b.Num * a.Den, a.Den
* b.Den);
    public static Fraction operator -(Fraction a, Fraction b) => new(a.Num * b.Den - b.Num * a.Den, a.Den
* b.Den);
    public static Fraction operator *(Fraction a, Fraction b) => new(a.Num * b.Num, a.Den * b.Den);
    public static Fraction operator -(Fraction a) => new(-a.Num, a.Den);          // unary minus

    public static bool operator ==(Fraction a, Fraction b) => a.Num == b.Num && a.Den == b.Den;
    public static bool operator !=(Fraction a, Fraction b) => !(a == b);
    public static bool operator <(Fraction a, Fraction b) => a.Num * b.Den < b.Num * a.Den;
    public static bool operator >(Fraction a, Fraction b) => b < a;

    public override bool Equals(object? obj) => obj is Fraction f && this == f;
    public override int GetHashCode() => HashCode.Combine(Num, Den);
    public override string ToString() => Den == 1 ? $"{Num}" : $"{Num}/{Den}";
}
```

OUTPUT

```
1/2 + 1/3 = 5/6
1/2 - 1/3 = 1/6
1/2 * 1/3 = 1/6
-1/2 = -1/2
2/4 == 1/2: True, 1/2 > 1/3: True
```

Can be overloaded	Cannot be overloaded
unary `+ - ! ~ ++ -- true false`	`= . ?: ?? ??= => new is as typeof sizeof`
binary `+ - * / % & | ^ << >> >>>`	`&&` and `||` (they use `&`, `|`, true, false)
comparison `== != < > <= >=` (in pairs)	`[]` (use an indexer instead)
compound assignment `+=` etc. (C# 14; otherwise derived from `+`)	`()` casts (use conversion operators)

10.7.5 Conversion operators (implicit and explicit)

A type can define conversions to and from other types. An `implicit` conversion happens automatically and must be safe: it never loses information or throws (int to Fraction). An `explicit` conversion needs a cast and is used when information may be lost (Fraction to int truncates).

USER-DEFINED CONVERSIONS Program.cs

```
Fraction f = 3;                                  // implicit: int -> Fraction
Fraction g = new(3, 4);
double d = (double)g;                            // explicit: Fraction -> double
int whole = (int)new Fraction(7, 2);             // explicit: truncates 3.5 to 3
Console.WriteLine($"{f}  {g} = {d}  7/2 -> {whole}");

Celsius body = new(37);
Fahrenheit inF = body;                           // implicit between temperature types
Console.WriteLine($"{body.Degrees} C = {inF.Degrees} F");

public class Fraction(int num, int den)
{
    public int Num { get; } = num;
    public int Den { get; } = den;
    public static implicit operator Fraction(int n) => new(n, 1);
    public static explicit operator double(Fraction f) => (double)f.Num / f.Den;
    public static explicit operator int(Fraction f) => f.Num / f.Den;
    public override string ToString() => $"{Num}/{Den}";
}

public record Celsius(double Degrees)
{
    public static implicit operator Fahrenheit(Celsius c) => new(c.Degrees * 9 / 5 + 32);
}
public record Fahrenheit(double Degrees);
```

OUTPUT

```
3/1  3/4 = 0.75  7/2 -> 3
37 C = 98.6 F
```

10.7.6 Finalizers (destructors) and why they are rare

A **finalizer**, written `~ClassName()`, runs when the garbage collector reclaims an object. You cannot call it and cannot know **when** (or, at process exit, whether) it will run. Objects with finalizers also survive an extra GC cycle, which costs performance. Since the GC already frees memory, finalizers are needed only as a last-resort safety net for **unmanaged** resources such as raw OS handles, and even then .NET provides `SafeHandle` to do it for you.

FINALIZER SYNTAX C#

```
class NativeBuffer
{
    ~NativeBuffer()                    // finalizer: runs at an unpredictable time on the GC's thread
    {
        // release an unmanaged resource here as a safety net
    }
}
```

For deterministic clean-up of files, network connections and database connections, implement `IDisposable` and use a `using` statement or declaration, which calls `Dispose()` at a known moment, even when an exception occurs (Modules 11 and 13 cover this fully).

IDISPOSABLE INSTEAD OF A FINALIZER

Program.cs

```
using (var r = new Resource("file A"))         // using statement: disposed at the closing brace
{
    r.Use();
}
using var conn = new Resource("connection");     // using declaration: disposed at end of scope
conn.Use();
Console.WriteLine("end of program");

public class Resource(string name) : IDisposable
{
    public void Use() => Console.WriteLine($"using {name}");
    public void Dispose() => Console.WriteLine($"disposed {name}");
}
```

OUTPUT

```
using file A
disposed file A
using connection
end of program
disposed connection
```

10.8 Partial classes and methods

The `partial` keyword lets one class be split across several declarations, usually in different files; the compiler merges them into a single type. It is how designers and source generators work: generated code goes in one file (Form1.Designer.cs, a generated regex or JSON serializer) and your hand-written code in another, so regenerating never overwrites your work. All parts must use `partial` and be in the same assembly and namespace.

A **partial method** is declared in one part and implemented in another. A classic partial method (no access modifier, void return) may be left unimplemented, in which case the compiler removes the declaration and every call to it. A partial method with an access modifier, a return value or out parameters (C# 9) **must** be implemented. C# 13 extends the idea to **partial properties** and indexers, which source generators use.

ONE CLASS IN TWO PARTS

Program.cs

```
var c = new Customer { Name = "Asha" };
c.Validate();
Console.WriteLine(c.Describe());

// --- Customer.Generated.cs (pretend a tool wrote this part) ---
public partial class Customer
{
    public string Name { get; set; } = "";
    partial void OnValidating();                    // hook: optional to implement
    public partial string Describe();               // has an access modifier: must be implemented

    public void Validate()
    {
        OnValidating();
        Console.WriteLine($"validated {Name}");
    }
}

// --- Customer.cs (your hand-written part) ---
public partial class Customer
{
    partial void OnValidating() => Console.WriteLine("custom check before validation");
    public partial string Describe() => $"Customer: {Name}";
}
```

OUTPUT

```
custom check before validation
validated Asha
Customer: Asha
```

10.9 Nested classes

A class declared inside another class is a **nested type**. It is private by default, so it can be a hidden implementation detail of the outer class, and it can access the outer class's **private** members (given a reference to an outer object). Use nested types for helpers that make sense only in the context of their outer class, such as the Node of a linked list or an Enumerator. If the nested type is public, outside code refers to it as `Outer.Inner`.

NESTED CLASSES

Program.cs

```
var list = new IntList();
list.Add(10); list.Add(20); list.Add(30);
Console.WriteLine($"{list} (count {list.Count})");

var setting = new Car.Settings { Mode = "eco" };     // public nested type: Outer.Inner
Console.WriteLine($"mode {setting.Mode}");

public class IntList
{
    private Node? head, tail;
    public int Count { get; private set; }

    private class Node(int value)                      // private nested class: hidden detail
    {
        public int Value { get; } = value;
        public Node? Next { get; set; }
    }

    public void Add(int value)
    {
        var node = new Node(value);
        if (tail is null) head = tail = node;
        else { tail.Next = node; tail = node; }
        Count++;
    }

    public override string ToString()
    {
        var parts = new List<string>();
        for (var n = head; n is not null; n = n.Next) parts.Add(n.Value.ToString());
        return string.Join(" -> ", parts);
    }
}

public class Car
{
    public class Settings { public string Mode { get; set; } = "normal"; }
}
```

OUTPUT

```
10 -> 20 -> 30 (count 3)
mode eco
```

10.10 Extension methods and extension members

Sometimes you want to add a method to a type you cannot change, such as `string`, `int` or a class from a library. **Extension methods** let you write a static method that can be called as if it were an instance method of that type. LINQ (Module 16) is built entirely from extension methods on `IEnumerable<T>`.

10.10.1 Classic extension methods with this parameter

An extension method is a `static` method in a non-nested, non-generic `static class`, whose first parameter is marked with `this`. The type of that parameter is the type being extended. The namespace containing the class must be in scope (via `using`). Extension methods cannot access private members, and an instance method with the same signature always wins over an extension.

EXTENSION METHODS

Program.cs

```
Console.WriteLine("the quick  brown fox".WordCount());
Console.WriteLine("Madam".IsPalindrome());
Console.WriteLine($"{7.IsEven()} {10.IsEven()}");
Console.WriteLine("hello world".Capitalize());
int[] marks = { 72, 85, 91 };
Console.WriteLine(marks.Describe());
Console.WriteLine(StringExtensions.WordCount("called as a normal static method"));

public static class StringExtensions
{
    public static int WordCount(this string s) =>
        s.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length;

    public static bool IsPalindrome(this string s)
    {
        string t = s.ToLower();
        for (int i = 0, j = t.Length - 1; i < j; i++, j--) if (t[i] != t[j]) return false;
        return true;
    }

    public static string Capitalize(this string s) =>
        string.Join(" ", s.Split(' ').Select(w => w.Length == 0 ? w : char.ToUpper(w[0]) + w[1..]));
}

public static class NumberExtensions
{
    public static bool IsEven(this int n) => n % 2 == 0;
    public static string Describe(this int[] a) => $"{a.Length} values, max {a.Max()}";
}
```

OUTPUT

```
4
True
False True
Hello World
3 values, max 91
6
```

10.10.2 Extension blocks with extension properties (C# 14)

Classic extension methods can only add **methods**. C# 14 introduces **extension blocks**: inside a static class, `extension(Type receiver) { ... }` groups members that extend that type, and those members may now be **properties** as well as methods. A block whose receiver has no name, `extension(Type)`, declares **static** extension members that are called on the type itself. Existing classic extension methods keep working and can live alongside blocks.

EXTENSION BLOCKS (C# 14)

Program.cs

```
string title = "  extension members in csharp 14 ";
Console.WriteLine($"[{title.Tidy}] words: {title.WordCount}, empty: {"".IsBlank}");
Console.WriteLine(title.Shout());
Console.WriteLine($"{12.IsEven} {12.Squared} {7.IsEven}");
Console.WriteLine(string.Repeat("ab", 3));              // static extension member on string

public static class Extensions
{
    extension(string s)                                 // receiver: an instance of string
    {
        public string Tidy => s.Trim();                 // extension PROPERTY
        public int WordCount => s.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length;
        public bool IsBlank => string.IsNullOrWhiteSpace(s);
        public string Shout() => s.Trim().ToUpper() + "!"; // extension method
    }

    extension(int n)
    {
        public bool IsEven => n % 2 == 0;
        public int Squared => n * n;
    }

    extension(string)                                   // no receiver name: static members
    {
        public static string Repeat(string text, int count) => string.Concat(Enumerable.Repeat(text,
count));
    }
}
```

OUTPUT

```
[extension members in csharp 14] words: 5, empty: True
EXTENSION MEMBERS IN CSHARP 14!
True 144 False
ababab
```

10.11 System.Object members

Every type in .NET derives, directly or indirectly, from `System.Object` (C# keyword `object`). It provides a handful of members that every object therefore has. Three of them are `virtual` and are commonly overridden.

10.11.1 ToString(), Equals(), GetHashCode(), GetType()

Member	Default behaviour (classes)	Typical override
`ToString()`	returns the full type name, e.g. "Point"	a readable description, used by interpolation and Console
`Equals(object)`	**reference** equality: true only for the same object	value equality: same field values
`GetHashCode()`	a number based on object identity	combine the fields used by Equals
`GetType()`	the exact run-time Type (not virtual)	cannot be overridden
`ReferenceEquals(a, b)`	static: true if both refer to the same object	cannot be overridden
`MemberwiseClone()`	protected: a shallow copy	used to implement cloning

WHAT OBJECT PROVIDES BY DEFAULT

Program.cs

```
var p1 = new PlainPoint { X = 1, Y = 2 };
var p2 = new PlainPoint { X = 1, Y = 2 };
Console.WriteLine(p1.ToString());                    // default: the type name
Console.WriteLine($"Equals: {p1.Equals(p2)}, ==: {p1 == p2}");   // default: reference equality
Console.WriteLine($"type {p1.GetType().Name}, base {p1.GetType().BaseType!.Name}");

object o = 42;
Console.WriteLine($"{o.GetType().FullName}, {"hi".GetType().Name}, {new int[3].GetType().Name}");

public class PlainPoint { public int X; public int Y; }
```

OUTPUT

```
PlainPoint
Equals: False, ==: False
type PlainPoint, base Object
System.Int32, String, Int32[]
```

10.11.2 Overriding Equals and GetHashCode together

To make two different objects with the same data count as equal, override `Equals`. You must then also override `GetHashCode` so that **equal objects return equal hash codes**. Hash-based collections (`HashSet`, `Dictionary` keys) first compare hash codes and only call Equals for objects whose hashes match; if the hashes disagree, equal objects are treated as different. `HashCode.Combine` builds a good hash from several fields. Implementing `IEquatable<T>` adds a fast, type-safe `Equals(T)`.

EQUALS AND GETHASHCODE GO TOGETHER

Program.cs

```
var a = new Point(1, 2);
var b = new Point(1, 2);
var c = new Point(3, 4);
Console.WriteLine($"a.Equals(b) {a.Equals(b)}, a == b {a == b}, a.Equals(c) {a.Equals(c)}");
Console.WriteLine($"same hash for a and b: {a.GetHashCode() == b.GetHashCode()}");

var set = new HashSet<Point> { a, b, c };
Console.WriteLine($"set count {set.Count}, contains (3,4): {set.Contains(new Point(3, 4))}");

var bad = new HashSet<BadPoint> { new BadPoint(1, 2), new BadPoint(1, 2) };
Console.WriteLine($"BadPoint set count {bad.Count}");     // duplicates: hash codes differ

public class Point(int x, int y) : IEquatable<Point>
{
    public int X { get; } = x;
    public int Y { get; } = y;

    public bool Equals(Point? other) => other is not null && X == other.X && Y == other.Y;
    public override bool Equals(object? obj) => Equals(obj as Point);
    public override int GetHashCode() => HashCode.Combine(X, Y);    // same fields as Equals

    public static bool operator ==(Point? l, Point? r) => l is null ? r is null : l.Equals(r);
    public static bool operator !=(Point? l, Point? r) => !(l == r);
    public override string ToString() => $"({X}, {Y})";
}

public class BadPoint(int x, int y)                  // overrides Equals but NOT GetHashCode
{
    public int X { get; } = x;
    public int Y { get; } = y;
    public override bool Equals(object? obj) => obj is BadPoint p && p.X == X && p.Y == Y;
}
```

OUTPUT

```
a.Equals(b) True, a == b True, a.Equals(c) False
same hash for a and b: True
set count 2, contains (3,4): True
BadPoint set count 2
```

WATCH OUT

The compiler warns (CS0659) when a class overrides Equals without GetHashCode, as BadPoint does. The BadPoint set contains two "equal" objects because their default, identity-based hash codes differ. Only use immutable fields in GetHashCode: if a field changes while the object is in a HashSet, it can no longer be found. For data-centric types, a `record` (Module 12) generates correct Equals, GetHashCode, == and ToString automatically.

A class is a blueprint of data and behaviour; objects are instances created with new (or target-typed new()) and handled through references that may be null. Keep fields private; use static fields for shared data and readonly/const for values that must not change. Expose data through properties: full, auto, get-only, init, required, computed, and C# 14 field-backed. Constructors establish a valid state; chain them with this(…), use static constructors for static setup, primary constructors for brevity, object initializers for convenience and private constructors for factories and singletons. Access modifiers enforce encapsulation. Classes can also have indexers, operators, conversions, partial parts, nested types and extension members, and they should override ToString, and Equals with GetHashCode together, when value semantics matter.

PRACTICE CHECKPOINT

Write a **BankAccount** class with an encapsulated balance, **constructor chaining**, a **static account counter**, an overridden **ToString()**, and an **extension method on decimal** for currency formatting.

SAMPLE SOLUTION

Program.cs

```
using System.Globalization;

var a = new BankAccount("Asha", 5000m);
var b = new BankAccount("Ravi");
a.Deposit(1500m);
a.Withdraw(2000m);
b.Deposit(750.5m);
try { b.Withdraw(10_000m); }
catch (InvalidOperationException e) { Console.WriteLine($"Error: {e.Message}"); }

Console.WriteLine(a);
Console.WriteLine(b);
Console.WriteLine($"Accounts opened: {BankAccount.Count}");
Console.WriteLine(1234567.891m.ToRupees());

public class BankAccount
{
    private static int _count;
    public static int Count => _count;

    public string AccountNo { get; }
    public string Owner { get; }
    public decimal Balance { get; private set; }         // only the class can change it

    public BankAccount(string owner) : this(owner, 0m) { }   // chaining

    public BankAccount(string owner, decimal opening)
    {
        if (opening < 0) throw new ArgumentOutOfRangeException(nameof(opening));
        Owner = owner;
        Balance = opening;
        AccountNo = $"LBF{++_count:D4}";
    }

    public void Deposit(decimal amount)
    {
        if (amount <= 0) throw new ArgumentException("Deposit must be positive");
        Balance += amount;
    }

    public void Withdraw(decimal amount)
    {
        if (amount <= 0) throw new ArgumentException("Withdrawal must be positive");
        if (amount > Balance) throw new InvalidOperationException($"Insufficient funds in {AccountNo}");
        Balance -= amount;
    }

    public override string ToString() => $"{AccountNo} {Owner,-5} balance {Balance.ToRupees()}";
}

public static class CurrencyExtensions
{
    public static string ToRupees(this decimal amount) =>
        "Rs " + amount.ToString("N2", CultureInfo.InvariantCulture);
}
```

OUTPUT

```
Error: Insufficient funds in LBF0002
LBF0001 Asha  balance Rs 4,500.00
LBF0002 Ravi  balance Rs 750.50
Accounts opened: 2
Rs 1,234,567.89
```

MODULE 11 · INTERMEDIATE

Inheritance, polymorphism, abstraction and interfaces

Minimum duration **4 days** | Level **Intermediate**

Module 10 built single classes. Real designs are families of related types: a SavingsAccount is a kind of Account, a Circle and a Rectangle are both Shapes, and many unrelated classes can all be compared, printed or disposed. This module covers reuse through inheritance, runtime polymorphism with virtual and override, casting and type patterns, abstract classes, interfaces including default and static abstract members, the standard .NET interfaces, and how to choose between inheritance and composition.

IN THIS MODULE

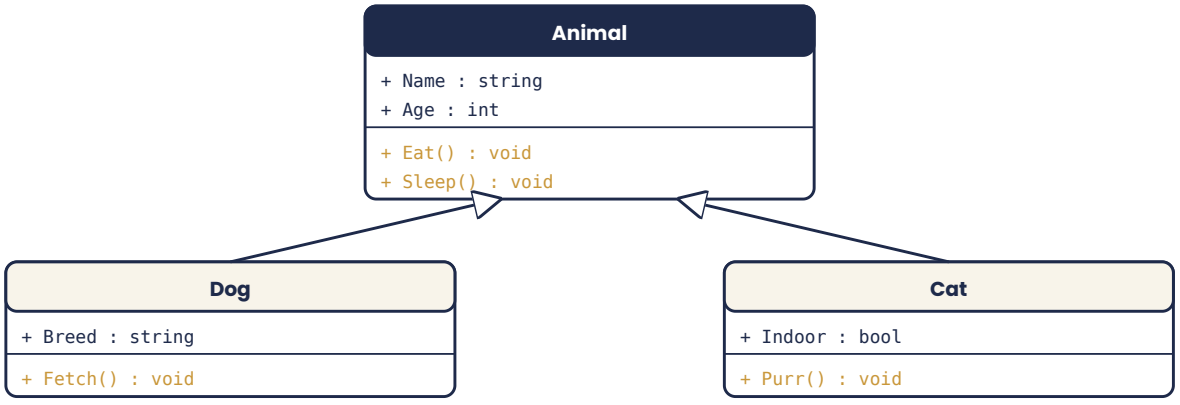
11.1 Inheritance
11.2 Polymorphism
11.3 Abstraction
11.4 Interfaces
11.5 Relationships between classes

11.1 Inheritance

Inheritance lets a new class (the **derived** class, child or subclass) be built from an existing one (the **base** class, parent or superclass). The derived class automatically gets the base class's members and can add new members or change inherited behaviour. Inheritance models an **IS-A** relationship: a Dog is an Animal, a SavingsAccount is an Account.

11.1.1 Base and derived classes class Dog : Animal

The base class is named after a colon: `class Dog : Animal`. A Dog object contains everything an Animal has (Name, Eat) plus what Dog adds (Breed, Fetch). Anywhere an Animal is expected, a Dog can be used.



DOG AND CAT INHERIT NAME, AGE, EAT AND SLEEP FROM ANIMAL AND ADD THEIR OWN MEMBERS

A DERIVED CLASS

Program.cs

```
var d = new Dog { Name = "Bruno", Age = 3, Breed = "Labrador" };
d.Eat();                                    // inherited from Animal
d.Fetch();                                  // declared in Dog
Console.WriteLine($"{d.Name} is a {d.Breed}, age {d.Age}");

Animal a = d;                               // a Dog IS an Animal
a.Sleep();
Console.WriteLine($"d is Animal: {d is Animal}, a is Dog: {a is Dog}");

public class Animal
{
    public string Name { get; set; } = "";
    public int Age { get; set; }
    public void Eat() => Console.WriteLine($"{Name} is eating");
    public void Sleep() => Console.WriteLine($"{Name} is sleeping");
}

public class Dog : Animal                   // Dog derives from Animal
{
    public string Breed { get; set; } = "";
    public void Fetch() => Console.WriteLine($"{Name} fetches the ball");
}
```

OUTPUT

```
Bruno is eating
Bruno fetches the ball
Bruno is a Labrador, age 3
Bruno is sleeping
d is Animal: True, a is Dog: True
```

11.1.2 Single inheritance of classes only

A C# class can have only **one** direct base class. This avoids the "diamond problem" of multiple inheritance in C++, where two parents may supply conflicting versions of the same member. A class can, however, implement any number of **interfaces** (11.4). Inheritance can go several levels deep (Animal → Dog → GuideDog), and every chain ends at `System.Object`. Structs cannot inherit from classes or other structs.

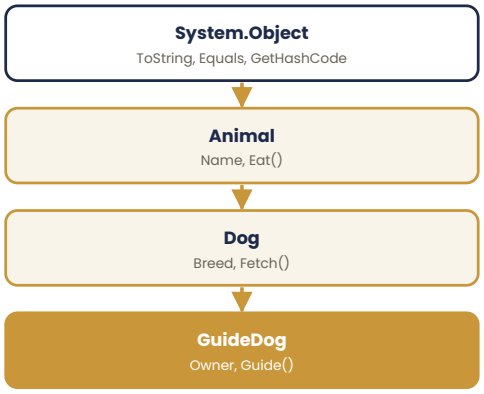
ONLY ONE BASE CLASS IS ALLOWED

Program.cs

```
class Animal { }
class Pet { }
class Dog : Animal, Pet { }       // two base classes
```

COMPILER OUTPUT

```
Program.cs(3,21): error CS1721: Class 'Dog' cannot have multiple base classes: 'Animal' and 'Pet'
```



A MULTI-LEVEL INHERITANCE CHAIN ALWAYS ENDS AT SYSTEM.OBJECT

11.1.3 base keyword for constructors and members

Constructors are **not** inherited. A derived constructor must make sure a base constructor runs, written `: base(arguments)` before its body; if omitted, the base's parameterless constructor is called (and it is an error if there is none). Inside the derived class, `base.Member` calls the base class's version of a member, typically to extend rather than replace it.

BASE(...) AND BASE.MEMBER

Program.cs

```
var s = new SavingsAccount("Asha", 10_000m, 4m);
s.AddInterest();
Console.WriteLine(s.Describe());

public class Account
{
    public string Owner { get; }
    public decimal Balance { get; protected set; }

    public Account(string owner, decimal opening)
    {
        Owner = owner;
        Balance = opening;
    }

    public virtual string Describe() => $"{Owner}: balance {Balance:F2}";
}

public class SavingsAccount : Account
{
    public decimal RatePct { get; }

    public SavingsAccount(string owner, decimal opening, decimal ratePct)
        : base(owner, opening)                          // run the base constructor first
    {
        RatePct = ratePct;
    }

    public void AddInterest() => Balance += Balance * RatePct / 100;   // protected setter

    public override string Describe() => base.Describe() + $" (savings @ {RatePct}%)";   // extend base
}
```

OUTPUT

```
Asha: balance 10400.00 (savings @ 4%)
```

11.1.4 Constructor execution order

Creating a derived object runs code in a precise order. **Field initializers** run first, from the most derived class up to the base; then the **constructor bodies** run from the base class down to the most derived. So by the time a derived constructor body runs, the base part of the object is fully built.

WATCHING THE ORDER OF CONSTRUCTION Program.cs

```
new Puppy();

static int Log(string msg) { Console.WriteLine(msg); return 0; }

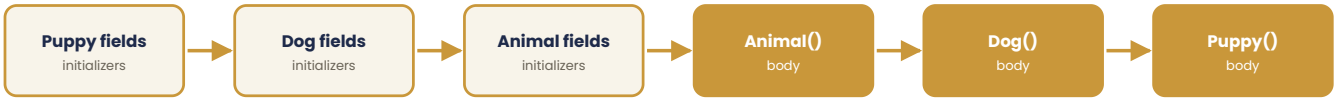
class Animal
{
    private int _a = Log("3. Animal field initializer");
    public Animal() => Console.WriteLine("4. Animal constructor body");
}

class Dog : Animal
{
    private int _d = Log("2. Dog field initializer");
    public Dog() => Console.WriteLine("5. Dog constructor body");
}

class Puppy : Dog
{
    private int _p = Log("1. Puppy field initializer");
    public Puppy() => Console.WriteLine("6. Puppy constructor body");
}
```

OUTPUT

```
1. Puppy field initializer
2. Dog field initializer
3. Animal field initializer
4. Animal constructor body
5. Dog constructor body
6. Puppy constructor body
```



INITIALIZERS RUN DERIVED TO BASE, CONSTRUCTOR BODIES RUN BASE TO DERIVED

WATCH OUT

Do not call virtual methods from a constructor. The override in the derived class would run before the derived constructor body has initialised its fields.

11.1.5 protected members

A `protected` member is accessible inside its own class and in any derived class, but not from outside code. It lets derived classes reuse internal helpers or state without making them public. Use it sparingly: every protected member is part of the contract with all future subclasses. `private protected` limits that to derived classes in the same assembly.

PROTECTED MEMBERS SHARED WITH SUBCLASSES

Program.cs

```
var e = new Manager("Asha", 80_000m);
Console.WriteLine(e.Payslip());
// Console.WriteLine(e.BaseSalary);        // error CS0122: inaccessible due to its protection level

public class Employee(string name, decimal baseSalary)
{
    public string Name { get; } = name;
    protected decimal BaseSalary { get; } = baseSalary;      // visible to subclasses only
    protected decimal Tax(decimal amount) => amount * 0.1m;

    public virtual string Payslip() => $"{Name}: {BaseSalary - Tax(BaseSalary):N0}";
}

public class Manager(string name, decimal baseSalary) : Employee(name, baseSalary)
{
    private decimal Bonus => BaseSalary * 0.2m;              // uses the protected property
    public override string Payslip()
    {
        decimal gross = BaseSalary + Bonus;
        return $"{Name} (manager): {gross - Tax(gross):N0}";
    }
}
```

OUTPUT

```
Asha (manager): 86,400
```

11.1.6 sealed classes and methods

A `sealed` class cannot be inherited from. Seal classes that were not designed for extension (and to let the JIT optimise calls); `string` is sealed. A `sealed override` method stops further overriding down the chain while the class itself remains inheritable.

SEALED CLASSES AND SEALED OVERRIDES

Program.cs

```
Console.WriteLine(new Puppy().Sound());

class Animal { public virtual string Sound() => "..."; }
class Dog : Animal { public sealed override string Sound() => "Woof"; }   // no further overrides
class Puppy : Dog { /* public override string Sound() => "Yip";  error CS0239 */ }

sealed class Config { }
// class MyConfig : Config { }              // error CS0509: cannot derive from sealed type 'Config'
```

OUTPUT

```
Woof
```

11.2 Polymorphism

Polymorphism ("many forms") means one piece of code can work with objects of different types, and each object responds in its own way. C# supports two kinds: compile-time polymorphism, where the compiler picks among overloads, and runtime polymorphism, where the method to run is chosen while the program runs, based on the actual object.

11.2.1 Compile-time: method and operator overloading

Overloading (Module 9) and operator overloading (Module 10) are resolved by the **compiler** from the static types of the arguments. This is also called static or early binding: the decision is fixed in the compiled code.

THE COMPILER CHOOSES THE OVERLOAD Program.cs

```
Console.WriteLine(Printer.Format(42));
Console.WriteLine(Printer.Format(3.14159));
Console.WriteLine(Printer.Format(new DateTime(2026, 3, 9)));
Console.WriteLine(Printer.Format("hi", 3));

static class Printer
{
    public static string Format(int n) => $"int: {n:N0}";
    public static string Format(double d) => $"double: {d:F2}";
    public static string Format(DateTime d) => $"date: {d:dd-MM-yyyy}";
    public static string Format(string s, int times) => $"string: {string.Concat(Enumerable.Repeat(s,
times))}";
}
```

OUTPUT

```
int: 42
double: 3.14
date: 09-03-2026
string: hihihi
```

11.2.2 Runtime: virtual and override

A base class marks a method `virtual` to allow derived classes to replace it; a derived class replaces it with `override`. When the method is called through a base-class reference, the runtime looks at the **actual object** and runs the most derived override (dynamic or late binding). This lets you write code against the base type that automatically works with every current and future subclass.

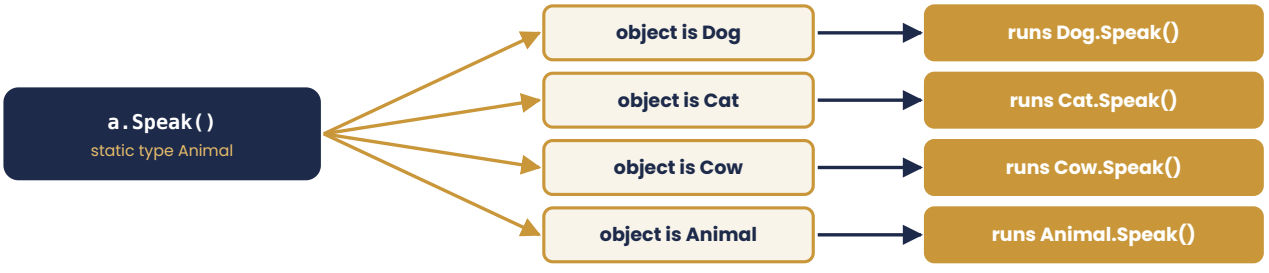
ONE CALL, MANY BEHAVIOURS Program.cs

```
List<Animal> zoo = [new Dog("Bruno"), new Cat("Misty"), new Cow("Gauri"), new Animal("Generic")];
foreach (Animal a in zoo)                  // static type Animal, run-time type varies
    Console.WriteLine($"{a.Name,-8} ({a.GetType().Name,-6}) says {a.Speak()}");

public class Animal(string name)
{
    public string Name { get; } = name;
    public virtual string Speak() => "...";                // can be overridden
}
public class Dog(string name) : Animal(name) { public override string Speak() => "Woof"; }
public class Cat(string name) : Animal(name) { public override string Speak() => "Meow"; }
public class Cow(string name) : Animal(name) { public override string Speak() => "Moo"; }
```

OUTPUT

```
Bruno    (Dog   ) says Woof
Misty    (Cat   ) says Meow
Gauri    (Cow   ) says Moo
Generic  (Animal) says ...
```



VIRTUAL DISPATCH: THE RUN-TIME TYPE OF THE OBJECT DECIDES WHICH OVERRIDE RUNS

NOTE

Only methods, properties, indexers and events can be virtual. Static members and fields cannot. An override must have the same signature and accessibility as the virtual member, and can call `base.Speak()` to reuse the base behaviour.

11.2.3 Method hiding with new vs overriding

If a derived class declares a member with the same name as a non-virtual base member, it **hides** the base member; writing `new` on the declaration states that this is intentional and silences warning CS0108. Hiding is resolved by the **static** type of the variable, not the object, so it does not give polymorphism. Overriding is resolved by the run-time type. Hiding is rarely what you want.

OVERRIDE VERSUS NEW Program.cs

```
Base b = new Derived();
Derived d = (Derived)b;
Console.WriteLine($"through Base:    {b.Overridden()} | {b.Hidden()}");
Console.WriteLine($"through Derived: {d.Overridden()} | {d.Hidden()}");

class Base
{
    public virtual string Overridden() => "Base.Overridden";
    public string Hidden() => "Base.Hidden";
}

class Derived : Base
{
    public override string Overridden() => "Derived.Overridden";   // replaces for every caller
    public new string Hidden() => "Derived.Hidden";                // only for Derived-typed variables
}
```

OUTPUT

```
through Base:    Derived.Overridden | Base.Hidden
through Derived: Derived.Overridden | Derived.Hidden
```

	`override`	`new` (hiding)
Requires base member to be	virtual, abstract or override	anything
Decided by	run-time type of the object	compile-time type of the variable
Polymorphic?	yes	no
Use when	specialising behaviour (almost always)	a base class you do not control later adds a member with your name

11.2.4 Upcasting and downcasting

Upcasting converts a derived reference to a base type. It is always safe and implicit: every Dog is an Animal. Through the base-type variable only base members are visible, although overrides still run. **Downcasting** converts a base reference back to a derived type. It needs an explicit cast and can fail at run time with `InvalidCastException` if the object is not actually of that type. Neither cast copies or changes the object; only the view of it changes.

UPCASTING AND DOWNCASTING

Program.cs

```
Dog dog = new Dog { Name = "Bruno" };
Animal animal = dog;                        // upcast: implicit, always safe
Console.WriteLine(animal.Name);
// animal.Fetch();                           // error: Animal has no Fetch method

Dog again = (Dog)animal;                    // downcast: explicit
again.Fetch();
Console.WriteLine(ReferenceEquals(dog, again));     // still the very same object

Animal other = new Cat { Name = "Misty" };
try
{
    Dog wrong = (Dog)other;                 // it is a Cat, not a Dog
}
catch (InvalidCastException) { Console.WriteLine("InvalidCastException: a Cat is not a Dog"); }

class Animal { public string Name { get; set; } = ""; }
class Dog : Animal { public void Fetch() => Console.WriteLine($"{Name} fetches"); }
class Cat : Animal { }
```

OUTPUT

```
Bruno
Bruno fetches
True
InvalidCastException: a Cat is not a Dog
```

11.2.5 Type checking with is, as and type patterns

To downcast safely, check the type first. The `is` operator tests a type, and the **type pattern** `is Dog d` tests and assigns in one step. The `as` operator attempts the cast and returns `null` instead of throwing. A `switch` expression with type patterns handles several types neatly, and property patterns can test members at the same time.

IS, AS AND SWITCH WITH TYPE PATTERNS

Program.cs

```
Animal[] animals = { new Dog { Name = "Bruno" }, new Cat { Name = "Misty", Indoor = true }, new Cat {
Name = "Tom" } };

foreach (Animal a in animals)
{
    if (a is Dog d) d.Fetch();                                  // type pattern: test + variable
    Cat? c = a as Cat;                                           // null if not a Cat
    if (c != null) Console.WriteLine($"{c.Name} indoor: {c.Indoor}");
}

foreach (Animal a in animals)
{
    string info = a switch
    {
        Dog dog => $"{dog.Name}: dog, walk twice a day",
        Cat { Indoor: true } cat => $"{cat.Name}: indoor cat",    // type + property pattern
        Cat cat => $"{cat.Name}: outdoor cat",
        _ => "unknown animal"
    };
    Console.WriteLine(info);
}

class Animal { public string Name { get; set; } = ""; }
class Dog : Animal { public void Fetch() => Console.WriteLine($"{Name} fetches"); }
class Cat : Animal { public bool Indoor { get; set; } }
```

OUTPUT

```
Bruno fetches
Misty indoor: True
Tom indoor: False
Bruno: dog, walk twice a day
Misty: indoor cat
Tom: outdoor cat
```

TIP

Frequent type checks on a hierarchy often mean a virtual method is missing: instead of `if (a is Dog) ... else if (a is Cat) ...`, give Animal a virtual method and let each class override it. Type patterns are best for data you do not control or for one-off decisions.

11.3 Abstraction

Abstraction means working with the essential idea of a thing while hiding the details. In C#, an **abstract class** represents a general concept (Shape, Payment, Employee) that is too incomplete to exist on its own, while defining what every concrete kind must provide.

11.3.1 Abstract classes and abstract members

An `abstract` class cannot be instantiated with `new`. It may contain normal members with code, fields and constructors, and also **abstract members**: methods or properties declared without a body. Every non-abstract derived class **must** override all abstract members; the compiler enforces this. Abstract members are implicitly virtual.

AN ABSTRACT SHAPE WITH CONCRETE SUBCLASSES

Program.cs

```
List<Shape> shapes = [new Circle(1.5), new Rectangle(4, 2.5), new Triangle(3, 4, 5)];
foreach (Shape s in shapes) Console.WriteLine(s.Describe());
Console.WriteLine($"total area {shapes.Sum(s => s.Area()):F2}");
// var x = new Shape("?");                // error CS0144: cannot create an instance of an abstract type

public abstract class Shape(string name)
{
    public string Name { get; } = name;                      // normal member
    public abstract double Area();                           // no body: subclasses must implement
    public abstract double Perimeter();
    public string Describe() => $"{Name,-9} area {Area(),6:F2}  perimeter {Perimeter(),6:F2}";   // uses
abstract members
}

public class Circle(double r) : Shape("Circle")
{
    public override double Area() => Math.PI * r * r;
    public override double Perimeter() => 2 * Math.PI * r;
}

public class Rectangle(double w, double h) : Shape("Rectangle")
{
    public override double Area() => w * h;
    public override double Perimeter() => 2 * (w + h);
}

public class Triangle(double a, double b, double c) : Shape("Triangle")
{
    public override double Area() { double s = (a + b + c) / 2; return Math.Sqrt(s * (s - a) * (s - b) *
(s - c)); }
    public override double Perimeter() => a + b + c;
}
```

OUTPUT

```
Circle    area   7.07  perimeter   9.42
Rectangle area  10.00  perimeter  13.00
Triangle  area   6.00  perimeter  12.00
total area 23.07
```

ABSTRACT MEMBERS MUST BE IMPLEMENTED

Program.cs

```
abstract class Shape { public abstract double Area(); }
class Square : Shape { }          // forgot to implement Area
```

COMPILER OUTPUT

```
Program.cs(2,7): error CS0534: 'Square' does not implement inherited abstract member 'Shape.Area()'
```

11.3.2 When to use an abstract class

Choose an abstract class when several closely related classes share **state and code** (fields, constructors, common methods) and also have parts that each must define differently. A common use is the **Template Method** pattern: the base class fixes the steps of an algorithm in a non-virtual method and leaves individual steps abstract.

TEMPLATE METHOD WITH AN ABSTRACT CLASS

Program.cs

```
new CsvReport().Generate();
new HtmlReport().Generate();

public abstract class Report
{
    public void Generate()                                  // template: fixed sequence of steps
    {
        Console.Write(Header());
        foreach (var (name, marks) in Data()) Console.Write(Row(name, marks));
        Console.WriteLine(Footer());
    }
    private static (string, int)[] Data() => [("Asha", 91), ("Ravi", 78)];
    protected abstract string Header();                     // steps each format must supply
    protected abstract string Row(string name, int marks);
    protected virtual string Footer() => "";                // optional step with a default
}

public class CsvReport : Report
{
    protected override string Header() => "name,marks;";
    protected override string Row(string n, int m) => $"{n},{m};";
}

public class HtmlReport : Report
{
    protected override string Header() => "<table>";
    protected override string Row(string n, int m) => $"<tr><td>{n}</td><td>{m}</td></tr>";
    protected override string Footer() => "</table>";
}
```

OUTPUT

```
name,marks;Asha,91;Ravi,78;
<table><tr><td>Asha</td><td>91</td></tr><tr><td>Ravi</td><td>78</td></tr></table>
```

11.4 Interfaces

An **interface** is a contract: a named set of members that a type promises to provide, with no say in how. Any class or struct, from any hierarchy, can implement an interface, and code written against the interface works with all of them. Interfaces describe **capabilities** ("can be compared", "can be printed", "can be disposed") rather than identity.

11.4.1 Declaring and implementing interfaces

Interfaces are declared with `interface` and by convention named with a leading **I**: `IPrintable`, `IComparable`. Members are implicitly public and normally have no body. A class implements the interface by listing it after the colon (after the base class, if any) and providing a public member for each interface member. A variable of interface type can refer to any implementing object.

ONE INTERFACE, SEVERAL IMPLEMENTATIONS

Program.cs

```
IPayment[] payments = { new CardPayment("4111222233334444"), new UpiPayment("asha@okbank"), new
CashPayment() };
foreach (IPayment p in payments)
    Console.WriteLine($"{p.Method,-5} -> {p.Pay(499.00m)}");

public interface IPayment
{
    string Method { get; }                  // property contract
    string Pay(decimal amount);              // method contract
}

public class CardPayment(string number) : IPayment
{
    public string Method => "Card";
    public string Pay(decimal amount) => $"charged {amount} to card ending {number[^4..]}";
}

public class UpiPayment(string vpa) : IPayment
{
    public string Method => "UPI";
    public string Pay(decimal amount) => $"collect request of {amount} sent to {vpa}";
}

public class CashPayment : IPayment
{
    public string Method => "Cash";
    public string Pay(decimal amount) => $"collect {amount} in cash";
}
```

OUTPUT

```
Card  -> charged 499.00 to card ending 4444
UPI   -> collect request of 499.00 sent to asha@okbank
Cash  -> collect 499.00 in cash
```

11.4.2 Implementing multiple interfaces

A class may have only one base class but can implement **any number** of interfaces, listed after the base class and separated by commas. This is how C# gets the benefits of multiple inheritance (one object, several roles) without its problems. An interface can also extend other interfaces.

ONE CLASS, SEVERAL INTERFACES

Program.cs

```
var phone = new SmartPhone();
phone.Call("98765 43210");
phone.TakePhoto();
phone.Play("Raag Yaman");

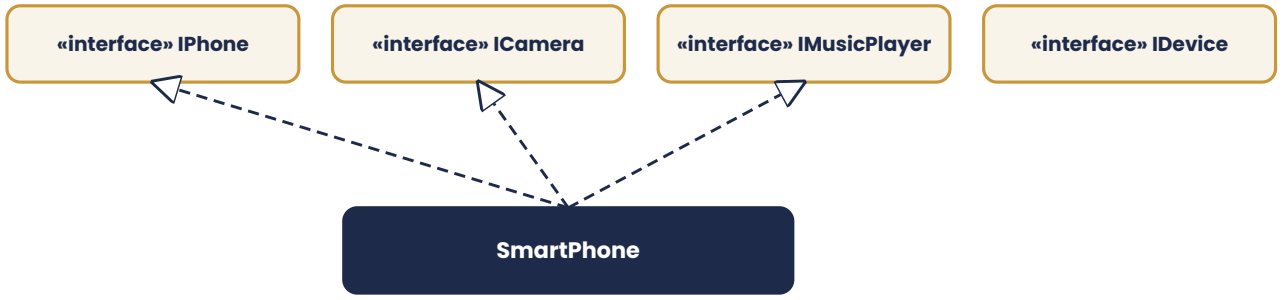
ICamera cam = phone;                        // the same object seen through one role
cam.TakePhoto();
Console.WriteLine($"phone is IMusicPlayer: {phone is IMusicPlayer}, is IDevice: {phone is IDevice}");

public interface IDevice { string Model { get; } }
public interface IPhone : IDevice { void Call(string number); }      // interface inheritance
public interface ICamera { void TakePhoto(); }
public interface IMusicPlayer { void Play(string track); }

public class SmartPhone : IPhone, ICamera, IMusicPlayer
{
    public string Model => "LBF-1";
    public void Call(string number) => Console.WriteLine($"{Model} calling {number}");
    public void TakePhoto() => Console.WriteLine("click!");
    public void Play(string track) => Console.WriteLine($"playing {track}");
}
```

OUTPUT

```
LBF-1 calling 98765 43210
click!
playing Raag Yaman
click!
phone is IMusicPlayer: True, is IDevice: True
```



SMARTPHONE REALISES THREE INTERFACES (DASHED LINES); IPHONE ITSELF EXTENDS IDEVICE

11.4.3 Explicit interface implementation

Sometimes two interfaces declare members with the same signature but different meanings, or you want an interface member to be available only through the interface. An **explicit implementation** names the interface: `string IFrench.Greet()`. It has no access modifier and can be called only through a variable of that interface type.

EXPLICIT INTERFACE IMPLEMENTATION Program.cs

```
var p = new Translator();
Console.WriteLine(p.Greet());              // the class's own public method
Console.WriteLine(((IEnglish)p).Greet());  // explicit implementations through the interface
IFrench f = p;
Console.WriteLine(f.Greet());

public interface IEnglish { string Greet(); }
public interface IFrench { string Greet(); }

public class Translator : IEnglish, IFrench
{
    public string Greet() => "Namaste";
    string IEnglish.Greet() => "Hello";        // explicit: no access modifier
    string IFrench.Greet() => "Bonjour";
}
```

OUTPUT

```
Namaste
Hello
Bonjour
```

11.4.4 Default interface methods (C# 8)

Since C# 8, an interface member can have a **default implementation**. Classes that implement the interface get it for free and may provide their own version. The main purpose is to let library authors **add members to a published interface** without breaking every existing implementation. A default method is available only through the interface type, not through the class.

A DEFAULT INTERFACE METHOD Program.cs

```
ILogger[] loggers = { new ConsoleLogger(), new AlertLogger() };
foreach (ILogger log in loggers)
{
    log.Log("backup started");
    log.LogError("disk almost full");          // default unless the class overrides it
}

public interface ILogger
{
    void Log(string message);                                    // must implement
    void LogError(string message) => Log($"ERROR: {message}");   // default implementation
}

public class ConsoleLogger : ILogger
{
    public void Log(string message) => Console.WriteLine($"[console] {message}");
}

public class AlertLogger : ILogger
{
    public void Log(string message) => Console.WriteLine($"[alert] {message}");
    public void LogError(string message) => Console.WriteLine($"[alert] !!! {message.ToUpper()} !!!");
// own version
}
```

OUTPUT

```
[console] backup started
[console] ERROR: disk almost full
[alert] backup started
[alert] !!! DISK ALMOST FULL !!!
```

11.4.5 Static abstract members in interfaces (C# 11)

An interface can require implementing types to provide **static** members, including operators, with `static abstract`. They are used through a generic type parameter constrained to the interface: inside the method, `T.Zero` or `a + b` calls the static member of whatever type T is. This feature powers .NET **generic math**: `INumber<T>` lets one method work with int, double, decimal and every other numeric type.

STATIC ABSTRACT MEMBERS AND GENERIC MATH Program.cs

```
using System.Numerics;

Console.WriteLine(Sum(new[] { 1, 2, 3, 4 }));            // T = int
Console.WriteLine(Sum(new[] { 1.5, 2.25 }));             // T = double
Console.WriteLine(Sum(new[] { 0.1m, 0.2m }));            // T = decimal
Console.WriteLine(Info<Square>());
Console.WriteLine(Info<Triangle>());

static T Sum<T>(T[] values) where T : INumber<T>
{
    T total = T.Zero;                       // static member of T
    foreach (T v in values) total += v;     // operator + of T
    return total;
}

static string Info<T>() where T : IShapeInfo => $"a {T.Kind} has {T.Sides} sides";

public interface IShapeInfo
{
    static abstract string Kind { get; }
    static abstract int Sides { get; }
}
public class Square : IShapeInfo { public static string Kind => "square"; public static int Sides => 4; }
public class Triangle : IShapeInfo { public static string Kind => "triangle"; public static int Sides =>
3; }
```

```
OUTPUT
10
3.75
0.3
a square has 4 sides
a triangle has 3 sides
```

11.4.6 Common .NET interfaces: IComparable, IComparer, IEquatable, IEnumerable, IDisposable, ICloneable

Interface	Member	Purpose	Used by
`IComparable<T>`	`int CompareTo(T other)`	the natural order of a type	Array.Sort, List.Sort, SortedSet
`IComparer<T>`	`int Compare(T x, T y)`	an external, alternative order	Sort(comparer), SortedDictionary
`IEquatable<T>`	`bool Equals(T other)`	type-safe value equality	List.Contains, Dictionary, HashSet
`IEnumerable<T>`	`GetEnumerator()`	can be iterated with foreach	foreach, LINQ
`IDisposable`	`void Dispose()`	release resources deterministically	using statements
`ICloneable`	`object Clone()`	make a copy (shallow or deep is unspecified)	rarely; prefer copy methods

ICOMPARABLE, ICOMPARER AND IEQUATABLE

Program.cs

```
var staff = new List<Employee> { new("Ravi", 52000), new("Asha", 61000), new("Meera", 47000) };

staff.Sort();                                          // uses IComparable<Employee>: by salary
Console.WriteLine("by salary: " + string.Join(", ", staff));

staff.Sort(new ByName());                              // uses an IComparer<Employee>
Console.WriteLine("by name:   " + string.Join(", ", staff));

staff.Sort((a, b) => b.Salary.CompareTo(a.Salary));    // a lambda as a Comparison: descending
Console.WriteLine("highest:   " + staff[0]);

Console.WriteLine($"contains Asha 61000: {staff.Contains(new Employee("Asha", 61000))}");   // IEquatable

public class Employee(string name, decimal salary) : IComparable<Employee>, IEquatable<Employee>
{
    public string Name { get; } = name;
    public decimal Salary { get; } = salary;

    public int CompareTo(Employee? other) => other is null ? 1 : Salary.CompareTo(other.Salary);
    public bool Equals(Employee? other) => other is not null && Name == other.Name && Salary ==
other.Salary;
    public override bool Equals(object? obj) => Equals(obj as Employee);
    public override int GetHashCode() => HashCode.Combine(Name, Salary);
    public override string ToString() => $"{Name} {Salary}";
}

public class ByName : IComparer<Employee>
{
    public int Compare(Employee? x, Employee? y) => string.Compare(x?.Name, y?.Name,
StringComparison.Ordinal);
}
```

OUTPUT

```
by salary: Meera 47000, Ravi 52000, Asha 61000
by name:   Asha 61000, Meera 47000, Ravi 52000
highest:   Asha 61000
contains Asha 61000: True
```

IENUMERABLE, IDISPOSABLE AND ICLONEABLE

Program.cs

```
foreach (int n in new Countdown(5)) Console.Write($"{n} ");
Console.WriteLine();

using (var timer = new Section("load data")) { Console.WriteLine("  ...working"); }

var original = new Team("Blue", new List<string> { "Asha", "Ravi" });
var shallow = (Team)original.Clone();
var deep = original.DeepCopy();
original.Members.Add("Meera");
Console.WriteLine($"shallow sees {shallow.Members.Count} members, deep sees {deep.Members.Count}");

public class Countdown(int from) : IEnumerable<int>        // can be used in foreach
{
    public IEnumerator<int> GetEnumerator()
    {
        for (int i = from; i >= 1; i--) yield return i;   // the compiler builds the enumerator
    }
    System.Collections.IEnumerator System.Collections.IEnumerable.GetEnumerator() => GetEnumerator();
}

public class Section : IDisposable
{
    private readonly string name;
    public Section(string name) { this.name = name; Console.WriteLine($"begin {name}"); }
    public void Dispose() => Console.WriteLine($"end {name}");
}

public class Team(string name, List<string> members) : ICloneable
{
    public string Name { get; } = name;
    public List<string> Members { get; } = members;
    public object Clone() => MemberwiseClone();                          // shallow: shares the list
    public Team DeepCopy() => new(Name, new List<string>(Members));      // copies the list too
}
```

OUTPUT

```
5 4 3 2 1
begin load data
  ...working
end load data
shallow sees 3 members, deep sees 2
```

NOTE

ICloneable does not say whether Clone is shallow or deep and returns `object`, so Microsoft's design guidelines recommend not using it in public APIs. A clearly named copy method, a copy constructor, or a record's `with` expression (Module 12) is clearer. `yield return` is covered in depth in Module 19.

11.4.7 Abstract class vs interface

	Abstract class	Interface
Purpose	shared identity and code for a family (IS-A)	a capability or contract (CAN-DO)
How many per class	one base class	any number
Fields / state	yes: instance fields	no instance fields (static only)
Constructors	yes	no

	Abstract class	Interface
Member bodies	any mix of abstract, virtual and normal	abstract by default; default methods since C# 8
Access modifiers	any	public by default
Structs can use it	no	yes
Adding a member later	easy if it has a body	breaks implementers unless a default body is given
Example	`Stream`, `Shape`	`IDisposable`, `IComparable<T>`

TIP

A common, flexible design uses both: an interface defines the contract (`IShape`), and an optional abstract base class (`ShapeBase : IShape`) supplies shared code for implementers that want it. Code that consumes shapes depends only on the interface.

11.5 Relationships between classes

Object-oriented designs are networks of classes that relate to each other. Naming the kind of relationship helps choose the right code structure.

11.5.1 IS-A and HAS-A

IS-A is inheritance or interface implementation: a Car *is a* Vehicle. **HAS-A** is containment: a Car *has an* Engine, stored as a field or property. A quick test: if the sentence "X is a Y" is true in every situation, inheritance may fit; if "X has a Y" or "X uses a Y" sounds right, use a field.

IS-A AND HAS-A IN ONE CLASS — Program.cs

```
var car = new Car("Nexon", new Engine(120));
car.Start();
Console.WriteLine($"car is Vehicle: {car is Vehicle}, engine power {car.Engine.Power} hp");

public class Vehicle(string model) { public string Model { get; } = model; }

public class Engine(int power)
{
    public int Power { get; } = power;
    public void Ignite() => Console.WriteLine($"engine ({Power} hp) running");
}

public class Car(string model, Engine engine) : Vehicle(model)     // IS-A Vehicle
{
    public Engine Engine { get; } = engine;                          // HAS-A Engine
    public void Start() { Console.Write($"{Model}: "); Engine.Ignite(); }
}
```

OUTPUT

```
Nexon: engine (120 hp) running
car is Vehicle: True, engine power 120 hp
```

11.5.2 Association, aggregation and composition

HAS-A relationships differ in how strongly the objects are tied together.

Relationship	Meaning	Lifetime	Example	UML
Association	objects know or use each other	independent	a Teacher teaches Students	plain line
Aggregation	a whole groups parts that exist on their own ("has, but does not own")	parts outlive the whole	a Department has Professors	hollow diamond
Composition	a whole owns its parts; they are created and destroyed with it	parts die with the whole	a House has Rooms; an Order has OrderLines	filled diamond



HOLLOW DIAMOND: AGGREGATION; FILLED DIAMOND: COMPOSITION

AGGREGATION AND COMPOSITION IN CODE

Program.cs

```
var asha = new Professor("Dr. Asha");
Department? cs = new Department("CS", new List<Professor> { asha });      // aggregation: professor made outside
cs = null;                                                                 // department gone...
Console.WriteLine($"{asha.Name} still exists");                            // ...professor lives on

var house = new House(3);                                                  // composition: house creates its rooms
Console.WriteLine($"house with {house.RoomCount} rooms: {house.Describe()}");

public class Professor(string name) { public string Name { get; } = name; }
public class Department(string name, List<Professor> staff)               // receives existing parts
{
    public string Name { get; } = name;
    public List<Professor> Staff { get; } = staff;
}

public class House
{
    private readonly List<Room> rooms = new();                             // owned, never exposed
    public House(int n) { for (int i = 1; i <= n; i++) rooms.Add(new Room($"Room {i}")); }
    public int RoomCount => rooms.Count;
    public string Describe() => string.Join(", ", rooms.Select(r => r.Label));
    private class Room(string label) { public string Label { get; } = label; }
}
```

OUTPUT

```
Dr. Asha still exists
house with 3 rooms: Room 1, Room 2, Room 3
```

11.5.3 Composition over inheritance

Inheritance is the tightest coupling in OOP: a subclass depends on its base's internals, it cannot change its base at run time, and deep hierarchies explode in size when behaviours combine (FlyingSwimmingDuck, NonFlyingSwimmingDuck, …). The design principle **"favour composition over inheritance"** says: build behaviour by giving an object parts (usually typed by interfaces) that it delegates to. Behaviours can then be mixed freely and even swapped at run time. Use inheritance only for a genuine, stable IS-A relationship.

BEHAVIOURS COMPOSED FROM PARTS (STRATEGY PATTERN)

Program.cs

```
var mallard = new Duck("Mallard", new WingFlying(), new LoudQuack());
var rubber = new Duck("Rubber duck", new NoFlying(), new Squeak());
mallard.Show();
rubber.Show();
rubber.Fly = new RocketFlying();           // behaviour replaced at run time
rubber.Show();

public interface IFly { string Fly(); }
public interface IQuack { string Quack(); }
public class WingFlying : IFly { public string Fly() => "flies with wings"; }
public class NoFlying : IFly { public string Fly() => "cannot fly"; }
public class RocketFlying : IFly { public string Fly() => "flies with a rocket!"; }
public class LoudQuack : IQuack { public string Quack() => "Quack!"; }
public class Squeak : IQuack { public string Quack() => "Squeak"; }

public class Duck(string name, IFly fly, IQuack quack)
{
    public IFly Fly { get; set; } = fly;          // HAS-A flying behaviour
    public IQuack Quack { get; set; } = quack;
    public void Show() => Console.WriteLine($"{name}: {Fly.Fly()}, says {Quack.Quack()}");   // delegate
}
```

OUTPUT

```
Mallard: flies with wings, says Quack!
Rubber duck: cannot fly, says Squeak
Rubber duck: flies with a rocket!, says Squeak
```

Inheritance (class Dog : Animal) reuses and extends a single base class; constructors chain with base(...), initializers run derived-to-base and constructor bodies base-to-derived, protected exposes members to subclasses, and sealed stops inheritance. Polymorphism is compile-time (overloading) or runtime (virtual/override, chosen by the object's real type); new merely hides. Upcasts are implicit; downcast safely with is, as and type patterns. Abstract classes share state and code while forcing subclasses to implement abstract members. Interfaces are contracts a type can implement many of, with explicit, default and static abstract members; know IComparable, IComparer, IEquatable, IEnumerable, IDisposable and ICloneable. Model IS-A with inheritance, HAS-A with fields, and prefer composition for combining behaviours.

PRACTICE CHECKPOINT

Create an **abstract Shape** class with **Circle** and **Rectangle**, an **IPrintable** interface, a **List<Shape>** processed polymorphically, and an **Employee** class sorted with **IComparable**.

SAMPLE SOLUTION

Program.cs

```
List<Shape> shapes = [new Circle(2), new Rectangle(3, 4), new Circle(0.5), new Rectangle(5, 1.5)];
double total = 0;
foreach (Shape s in shapes)
{
    s.Print();                                  // IPrintable, implemented in Shape
    total += s.Area();                          // runtime polymorphism
}
Console.WriteLine($"Total area: {total:F2}");
Shape largest = shapes[0];
foreach (Shape s in shapes) if (s.Area() > largest.Area()) largest = s;
Console.WriteLine($"Largest: {largest.Name}");

var team = new List<Employee> { new("Ravi", 52000), new("Asha", 61000), new("Meera", 47000), new("Kiran",
55000) };
team.Sort();                                    // IComparable<Employee>
```

```
foreach (Employee e in team) Console.WriteLine(e);

public interface IPrintable { void Print(); }

public abstract class Shape : IPrintable
{
    public abstract string Name { get; }
    public abstract double Area();
    public abstract double Perimeter();
    public void Print() => Console.WriteLine($"{Name,-14} area {Area(),7:F2}  perimeter
{Perimeter(),6:F2}");
}

public class Circle(double radius) : Shape
{
    public override string Name => $"Circle r={radius}";
    public override double Area() => Math.PI * radius * radius;
    public override double Perimeter() => 2 * Math.PI * radius;
}

public class Rectangle(double width, double height) : Shape
{
    public override string Name => $"Rect {width}x{height}";
    public override double Area() => width * height;
    public override double Perimeter() => 2 * (width + height);
}

public class Employee(string name, decimal salary) : IComparable<Employee>
{
    public string Name { get; } = name;
    public decimal Salary { get; } = salary;
    public int CompareTo(Employee? other) => other is null ? 1 : Salary.CompareTo(other.Salary);
    public override string ToString() => $"{Name,-6} {Salary,8:N0}";
}
```

OUTPUT

```
Circle r=2      area   12.57  perimeter  12.57
Rect 3x4        area   12.00  perimeter  14.00
Circle r=0.5    area    0.79  perimeter   3.14
Rect 5x1.5      area    7.50  perimeter  13.00
Total area: 32.85
Largest: Circle r=2
Meera    47,000
Ravi     52,000
Kiran    55,000
Asha     61,000
```

MODULE 12 · INTERMEDIATE

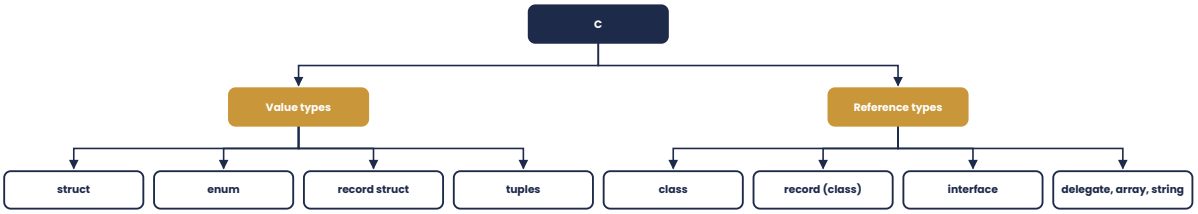
Structs, enums, records and nullable reference types

Minimum duration **2 days** | Level **Intermediate**

Classes are not the only kind of type in C#. Structs give lightweight value types, enums give names to fixed sets of values, records give concise data types with value equality, tuples group values without declaring a type, and anonymous types give throwaway shapes for queries. This module covers each of them and then the nullable reference types feature, with which the compiler tracks null safety and warns about possible NullReferenceExceptions before the program runs.

IN THIS MODULE

12.1 Structs
12.2 Enums
12.3 Records (C# 9)
12.4 Tuples
12.5 Nullable reference types (C# 8)
12.6 Anonymous types



THE KINDS OF TYPES COVERED IN MODULES 10 TO 12

12.1 Structs

A **struct** is a **value type**: a variable of a struct type contains the data itself rather than a reference to an object elsewhere. All the built-in numeric types, `bool`, `char`, `DateTime`, `TimeSpan` and `Guid` are structs. Structs are ideal for small, simple values that behave like numbers: a point, a colour, a money amount.

12.1.1 Defining structs

A struct is declared like a class but with the `struct` keyword. It can have fields, properties, methods, constructors (including a parameterless one since C# 10), operators and indexers, and it can implement interfaces. It cannot inherit from another struct or class and cannot be inherited from. A struct variable is usable without `new` once all its fields are assigned, and `default` gives a struct with every field at its default value.

A SIMPLE STRUCT Program.cs

```
Point a = new Point(3, 4);
Point b = default;                          // all fields zero
Point c;                                    // unassigned: must set fields before use
c.X = 1; c.Y = 2;
Console.WriteLine($"{a} length {a.Length}, {b}, {c}");

a.Move(1, 1);                               // mutating method on a struct variable
Console.WriteLine(a);
Console.WriteLine(new Point().Equals(default(Point)));   // value equality by default

public struct Point
{
    public int X;
    public int Y;

    public Point(int x, int y) { X = x; Y = y; }

    public double Length => Math.Sqrt(X * X + Y * Y);
    public void Move(int dx, int dy) { X += dx; Y += dy; }
    public override string ToString() => $"({X}, {Y})";
}
```

OUTPUT

```
(3, 4) length 5, (0, 0), (1, 2)
(4, 5)
True
```

12.1.2 Struct vs class (value vs reference semantics)

The essential difference is what assignment copies. Assigning a struct copies **all of its data**, so the two variables are independent. Assigning a class variable copies only the **reference**, so both variables share one object. The same applies to passing arguments and returning values.

COPY SEMANTICS Program.cs

```
PointS s1 = new() { X = 1 };
PointS s2 = s1;                             // copies the data
s2.X = 99;
Console.WriteLine($"struct: s1.X = {s1.X}, s2.X = {s2.X}");

PointC c1 = new() { X = 1 };
PointC c2 = c1;                             // copies the reference
c2.X = 99;
Console.WriteLine($"class:  c1.X = {c1.X}, c2.X = {c2.X}");

var list = new List<PointS> { new() { X = 5 } };
// list[0].X = 10;                          // error CS1612: cannot modify the return value (it is a copy)
var tmp = list[0]; tmp.X = 10; list[0] = tmp;   // copy out, change, copy back
Console.WriteLine($"list[0].X = {list[0].X}");

struct PointS { public int X; }
class PointC { public int X; }
```

OUTPUT

```
struct: s1.X = 1, s2.X = 99
class:  c1.X = 99, c2.X = 99
list[0].X = 10
```



TWO INDEPENDENT STRUCT COPIES VERSUS TWO REFERENCES TO ONE OBJECT

	struct	class
Kind	value type	reference type
Variable holds	the data	a reference to an object on the heap
Assignment / argument	copies all fields	copies the reference
Can be null	no (unless `Nullable<T>`, written `T?`)	yes
Inheritance	none (can implement interfaces)	single inheritance
Default equality	compares field values	compares references
Memory	inline: on the stack for locals, inside the containing object	a separate heap object managed by the GC
Boxing	boxed when converted to object or an interface	never

12.1.3 readonly struct and record struct

Mutable structs cause surprises (as the list example shows) because you often modify a copy by mistake. Mark a struct `readonly` to make the compiler require that every field is readonly and every property get-only or init: the value can never change after construction. A `record struct` (C# 10) is a struct for which the compiler generates value equality, `==`, `ToString`, deconstruction and `with` support; `readonly record struct` combines both.

READONLY STRUCT AND READONLY RECORD STRUCT Program.cs

```
var m = new Money(250.50m, "INR");
var total = m.Add(new Money(100m, "INR"));        // returns a new value
Console.WriteLine($"{m} + 100 = {total}");

var p1 = new Point3(1, 2, 3);
var p2 = p1 with { Z = 10 };                       // copy with one change
Console.WriteLine($"{p1} | {p2} | equal: {p1 == new Point3(1, 2, 3)}");
var (x, y, _) = p2;                                // deconstruction
Console.WriteLine($"x={x}, y={y}");

public readonly struct Money(decimal amount, string currency)
{
    public decimal Amount { get; } = amount;
    public string Currency { get; } = currency;
    // public void Clear() { Amount = 0; }        // error: readonly struct members cannot modify state
    public Money Add(Money other) =>
        other.Currency == Currency ? new(Amount + other.Amount, Currency) : throw new
InvalidOperationException("Currency mismatch");
    public override string ToString() => $"{Amount:F2} {Currency}";
}

public readonly record struct Point3(int X, int Y, int Z);    // all members generated
```

OUTPUT

```
250.50 INR + 100 = 350.50 INR
Point3 { X = 1, Y = 2, Z = 3 } | Point3 { X = 1, Y = 2, Z = 10 } | equal: True
x=1, y=2
```

12.1.4 ref struct (overview)

A `ref struct` is a struct that can **only live on the stack**. It cannot be boxed, stored in a field of a class, captured by a lambda, or used across an `await`. In exchange it can safely hold references into memory, including stack memory. The most important ref structs are `Span<T>` and `ReadOnlySpan<T>`, which represent a window over an array, a string or a block of stack memory without copying it. They are a performance tool covered further in Module 19.

SPAN<T>: THE BEST-KNOWN REF STRUCT Program.cs

```
int[] data = { 10, 20, 30, 40, 50, 60 };
Span<int> middle = data.AsSpan(1, 4);          // a view of elements 1..4: no copy
middle[0] = 99;                                // writes through to the array
Console.WriteLine(string.Join(" ", data));

ReadOnlySpan<char> text = "2026-03-09".AsSpan();
int year = int.Parse(text[..4]);               // parse a slice without creating a substring
Console.WriteLine($"year {year}, month {int.Parse(text.Slice(5, 2))}");

Span<int> buffer = stackalloc int[4];          // memory on the stack, freed automatically
for (int i = 0; i < buffer.Length; i++) buffer[i] = i * i;
Console.WriteLine(string.Join(",", buffer.ToArray()));
```

OUTPUT

```
10 99 30 40 50 60
year 2026, month 3
0,1,4,9
```

12.1.5 When to use a struct

- The type represents a **single small value**, like a primitive: coordinates, a complex number, a date range, money.
- Its size is small, typically **16 bytes or less**, because every assignment and argument copies it.
- It is **immutable** (make it `readonly`).
- It will not need to be **boxed** frequently (stored as `object` or an interface), which would allocate.
- Many instances are created, for example millions of points in an array, where avoiding a heap object per item matters.

In every other case, especially for entities with identity (Customer, Order) or anything large or mutable, use a class or a record class.

12.2 Enums

An **enumeration** (enum) is a value type that defines a set of named constants: the days of the week, order statuses, log levels. Enums make code self-documenting (`OrderStatus.Shipped` instead of the magic number 2), and the compiler catches typos.

12.2.1 Defining enums and underlying types

Each enum member has an integral value. By default the type is `int`, the first member is 0 and each next member is one more; values can also be assigned explicitly. A different **underlying type** (byte, short, long, …) can be given after a colon to save space or match an external format.

DEFINING AND USING ENUMS — Program.cs

```
OrderStatus s = OrderStatus.Shipped;
Console.WriteLine($"{s} = {(int)s}");

Day today = Day.Wed;
if (today is Day.Sat or Day.Sun) Console.WriteLine("weekend");
else Console.WriteLine($"{today} is a working day ({(int)today})");

string message = s switch
{
    OrderStatus.Pending => "waiting for payment",
    OrderStatus.Shipped => "on the way",
    OrderStatus.Delivered => "delivered",
    _ => "unknown"
};
Console.WriteLine(message);
Console.WriteLine($"Level.High: underlying type {Enum.GetUnderlyingType(typeof(Level)).Name}, value
{(byte)Level.High}");

enum OrderStatus { Pending, Paid, Shipped, Delivered, Cancelled }      // 0, 1, 2, 3, 4
enum Day { Mon = 1, Tue, Wed, Thu, Fri, Sat, Sun }                      // 1 to 7
enum Level : byte { Low = 10, Medium = 20, High = 30 }                  // underlying type byte
```

OUTPUT

```
Shipped = 2
Wed is a working day (3)
on the way
Level.High: underlying type Byte, value 30
```

12.2.2 Converting between enums, ints and strings

An enum converts to its number with a cast `(int)status` and back with `(OrderStatus)2`. `ToString()` gives the member name. Casting is not checked: any number of the underlying type can be cast to an enum, even one with no matching member, so validate values that come from outside with `Enum.IsDefined`.

ENUM CONVERSIONS Program.cs

```
var s = (OrderStatus)3;                          // int -> enum
Console.WriteLine($"{s}, {(int)OrderStatus.Cancelled}, {OrderStatus.Paid.ToString().ToUpper()}");

var weird = (OrderStatus)42;                     // compiles and runs!
Console.WriteLine($"weird = {weird}, defined: {Enum.IsDefined(weird)}");
Console.WriteLine(OrderStatus.Paid < OrderStatus.Shipped);    // enums compare by value

enum OrderStatus { Pending, Paid, Shipped, Delivered, Cancelled }
```

OUTPUT

```
Delivered, 4, PAID
weird = 42, defined: False
True
```

12.2.3 Enum.Parse, Enum.GetValues

`Enum.Parse<T>(text)` converts a name (or number) to the enum and throws if it is invalid; `Enum.TryParse<T>` returns false instead and can ignore case. `Enum.GetValues<T>()` and `Enum.GetNames<T>()` list all members, which is handy for menus and drop-down lists.

PARSING AND LISTING ENUMS Program.cs

```
Day d = Enum.Parse<Day>("Fri");
Console.WriteLine($"{d} = {(int)d}");

foreach (string input in new[] { "sun", "Funday" })
    Console.WriteLine(Enum.TryParse(input, ignoreCase: true, out Day day) ? $"{input} -> {day}" : $"
{input} is not a day");

Console.WriteLine(string.Join(", ", Enum.GetNames<Day>()));
foreach (Day x in Enum.GetValues<Day>())
    if (x >= Day.Fri) Console.Write($"{x}={(int)x} ");
Console.WriteLine();

enum Day { Mon = 1, Tue, Wed, Thu, Fri, Sat, Sun }
```

OUTPUT

```
Fri = 5
sun -> Sun
Funday is not a day
Mon, Tue, Wed, Thu, Fri, Sat, Sun
Fri=5 Sat=6 Sun=7
```

WATCH OUT

Enum.TryParse also accepts numeric strings: `Enum.TryParse<Day>("42", out var d)` succeeds and gives `(Day)42`. When parsing user input, follow it with `Enum.IsDefined`.

12.2.4 Flags enums with [Flags]

Sometimes a value is a **combination** of options: file permissions, font styles, days on which a class meets. A **flags enum** gives each member a distinct bit (1, 2, 4, 8, ...), combines them with bitwise OR `|`, tests them with

`HasFlag` or `&`, and removes them with `& ~`. The `[Flags]` attribute makes `ToString` print the combination as a list of names.



Read | Write = 0001 | 0010 = 0011 = 3

each permission occupies one bit, so any combination fits in one number

FLAG VALUES ARE POWERS OF TWO

A [FLAGS] ENUM

Program.cs

```
Permission p = Permission.Read | Permission.Write;       // combine
Console.WriteLine($"{p} = {(int)p}");

p |= Permission.Execute;                                 // add
Console.WriteLine($"{p}, can write: {p.HasFlag(Permission.Write)}, can delete: {(p & Permission.Delete)
!= 0}");

p &= ~Permission.Write;                                  // remove
Console.WriteLine($"{p} = {(int)p}");

Permission admin = Permission.All;
Console.WriteLine($"{admin} = {(int)admin}, {(Permission)6}, {Permission.None}");
Console.WriteLine(Enum.Parse<Permission>("Read, Delete"));

[Flags]
enum Permission
{
    None = 0,
    Read = 1,          // 0001
    Write = 2,         // 0010
    Execute = 4,       // 0100
    Delete = 8,        // 1000
    All = Read | Write | Execute | Delete
}
```

OUTPUT

```
Read, Write = 3
Read, Write, Execute, can write: True, can delete: False
Read, Execute = 5
All = 15, Write, Execute, None
Read, Delete
```

12.3 Records (C# 9)

Many classes exist only to **hold data**: a DTO from an API, a configuration entry, a row of a report. For them, the important questions are "do two objects contain the same data?" and "give me a copy with one field changed". A **record** is a class (or struct) for which the compiler generates exactly that: value-based equality, a readable `ToString`, `with` expressions, and deconstruction.

12.3.1 Positional records

The shortest form is the **positional record**: `record Student(string Name, int Marks);`. From that one line the compiler generates a primary constructor, a public `init` property for each parameter, `Equals`, `GetHashCode`, `==` and `!=`, `ToString`, a `Deconstruct` method and copy support for `with`. A record can still have a body with extra members.

POSITIONAL AND NOMINAL RECORDS Program.cs

```
var s = new Student("Asha", 91);
Console.WriteLine(s);                                   // generated ToString
Console.WriteLine($"{s.Name} has grade {s.Grade}");
// s.Marks = 95;                                        // error CS8852: init-only property

var p = new Person { FirstName = "Ravi", LastName = "Kumar" };   // nominal record with init properties
Console.WriteLine($"{p} -> {p.FullName}");

public record Student(string Name, int Marks)           // positional record
{
    public char Grade => Marks >= 85 ? 'A' : 'B';       // extra members are allowed
}

public record Person
{
    public required string FirstName { get; init; }
    public required string LastName { get; init; }
    public string FullName => $"{FirstName} {LastName}";
}
```

OUTPUT

```
Student { Name = Asha, Marks = 91, Grade = A }
Asha has grade A
Person { FirstName = Ravi, LastName = Kumar, FullName = Ravi Kumar } -> Ravi Kumar
```

NOTE

The generated ToString includes every public property, computed ones such as Grade and FullName too. Override `PrintMembers` or `ToString` to change it.

12.3.2 Value-based equality

Two record instances are equal when they are of the same type and all their fields are equal, even though they are different objects. Classes, by contrast, compare references by default. This makes records natural as dictionary keys and in sets, and easy to compare in tests.

RECORDS COMPARE BY VALUE Program.cs

```
var a = new Student("Asha", 91);
var b = new Student("Asha", 91);
Console.WriteLine($"records: a == b {a == b}, Equals {a.Equals(b)}, same object {ReferenceEquals(a,
b)}");
Console.WriteLine($"hash codes equal: {a.GetHashCode() == b.GetHashCode()}");

var c1 = new StudentClass("Asha", 91);
var c2 = new StudentClass("Asha", 91);
Console.WriteLine($"classes: c1 == c2 {c1 == c2}");

var set = new HashSet<Student> { a, b, new("Ravi", 78) };
Console.WriteLine($"set holds {set.Count} distinct students");

public record Student(string Name, int Marks);
public class StudentClass(string name, int marks) { public string Name = name; public int Marks = marks;
}
```

OUTPUT

```
records: a == b True, Equals True, same object False
hash codes equal: True
classes: c1 == c2 False
set holds 2 distinct students
```

WATCH OUT

Equality of a record compares its fields with their own Equals. A record holding a `List<string>` therefore compares the list **references**, not the items: two records with different list objects containing the same items are not equal.

12.3.3 Non-destructive mutation with with

Records are usually immutable. To "change" one, a `with` expression makes a **copy** with some properties replaced, leaving the original untouched. This is called non-destructive mutation. `with` also works on record structs, ordinary structs and anonymous types.

COPIES WITH WITH

Program.cs

```
var original = new Student("Asha", 91, "Pune");
var moved = original with { City = "Delhi" };            // copy, change one property
var retest = original with { Marks = 95, City = "Mumbai" };
Console.WriteLine(original);
Console.WriteLine(moved);
Console.WriteLine(retest);
Console.WriteLine($"copy equals original: {(original with { }) == original}");

public record Student(string Name, int Marks, string City);
```

OUTPUT

```
Student { Name = Asha, Marks = 91, City = Pune }
Student { Name = Asha, Marks = 91, City = Delhi }
Student { Name = Asha, Marks = 95, City = Mumbai }
copy equals original: True
```

NOTE

`with` makes a **shallow** copy: reference-type properties (such as a list) are shared between the original and the copy.

12.3.4 Deconstruction

Positional records get a generated `Deconstruct` method, so their values can be unpacked into separate variables, with discards for unwanted ones, and used in positional patterns.

DECONSTRUCTING A RECORD Program.cs

```
var s = new Student("Ravi", 78, "Chennai");
var (name, marks, city) = s;                              // deconstruct
Console.WriteLine($"{name} from {city} scored {marks}");
var (_, m, _) = s;                                        // discard what you don't need
Console.WriteLine(m);

string result = s switch
{
    (_, >= 85, _) => "distinction",
    (_, >= 60, "Chennai") => "first class, Chennai centre",          // positional pattern
    _ => "pass"
};
Console.WriteLine(result);

public record Student(string Name, int Marks, string City);
```

OUTPUT

```
Ravi from Chennai scored 78
78
first class, Chennai centre
```

12.3.5 Record class vs record struct

`record` (or `record class`) is a reference type; `record struct` is a value type. Positional properties of a record class are `init`-only, while a plain record struct generates **mutable** `{ get; set; }` properties; add `readonly` to make them init-only.

	`record` / `record class`	`record struct`	`readonly record struct`
Kind	reference type	value type	value type
Positional properties	init-only	get; set; (mutable)	init-only
Can be null	yes	no	no
Inheritance	records can inherit from records	no	no
Good for	DTOs, events, API results	small mutable values	small immutable values: points, money

RECORD CLASS AND RECORD STRUCT Program.cs

```
var rc = new PointClass(1, 2);
var rs = new PointStruct(1, 2);
rs.X = 50;                                // allowed: record struct properties are settable
Console.WriteLine($"{rc} | {rs}");
Console.WriteLine($"{rc.GetType().IsValueType} {rs.GetType().IsValueType}");

public record PointClass(int X, int Y);
public record struct PointStruct(int X, int Y);
```

OUTPUT

```
PointClass { X = 1, Y = 2 } | PointStruct { X = 50, Y = 2 }
False True
```

12.3.6 Records vs classes

Question	Choose a record	Choose a class
Is it mainly data?	yes: DTOs, messages, value objects	no: it has behaviour and changing state
Equality means	same data	same identity (the same customer, even if details change)
Mutability	immutable, copy with `with`	mutable state is natural
Examples	`Money`, `Address`, `OrderPlaced` event, API response	`BankAccount`, `DbContext`, a game character
EF Core entities	usually not (identity matters)	yes

12.4 Tuples

Module 9 used tuples to return several values. Here is the full picture of the type behind them.

12.4.1 ValueTuple and named elements

Tuple syntax `(string, int)` is shorthand for the struct `System.ValueTuple<string, int>`. Its elements are public mutable **fields** called `Item1`, `Item2`, …; element names such as `(string Name, int Marks)` are a compile-time convenience that the compiler maps onto those fields. Tuples have value equality, a ToString like `(Asha, 91)`, and element names can be inferred from variable names.

VALUETUPLE IN PRACTICE

Program.cs

```
(string, int) t1 = ("Asha", 91);
Console.WriteLine($"{t1.Item1} {t1.Item2} {t1}");

(string Name, int Marks) t2 = ("Ravi", 78);          // named elements
Console.WriteLine($"{t2.Name} {t2.Marks} (Item1 still works: {t2.Item1})");

var t3 = (City: "Pune", Pin: 411001);                // names in the literal
string course = "C#"; int days = 71;
var t4 = (course, days);                             // names inferred from variables
Console.WriteLine($"{t3.City} {t4.course} {t4.days}");

t2.Marks += 5;                                       // tuple fields are mutable
Console.WriteLine($"{t2} equals (\"Ravi\", 83): {t2 == ("Ravi", 83)}, type {t2.GetType().Name}");

var lookup = new Dictionary<(int Row, int Col), string> { [(0, 0)] = "start", [(2, 3)] = "goal" };  //
tuple keys
Console.WriteLine(lookup[(2, 3)]);
```

OUTPUT

```
Asha 91 (Asha, 91)
Ravi 78 (Item1 still works: Ravi)
Pune C# 71
(Ravi, 83) equals ("Ravi", 83): True, type ValueTuple`2
goal
```

12.4.2 Deconstruction and discards _

Deconstruction unpacks a tuple (or any type with a `Deconstruct` method) into variables, either new ones with `var (a, b)` or existing ones with `(a, b) = ...`. The **discard** `_` ignores elements you do not need;

discards also work for out parameters, lambda parameters and patterns. Any class can support deconstruction by adding a `Deconstruct` method with out parameters.

DECONSTRUCTION EVERYWHERE Program.cs

```
var (min, max, avg) = Stats(new[] { 72, 85, 91, 64 });
Console.WriteLine($"{min} {max} {avg}");

(_, int highest, _) = Stats(new[] { 5, 9, 2 });      // discards
Console.WriteLine(highest);

int a = 1, b = 2;
(a, b) = (b, a);                                     // deconstruct into existing variables
Console.WriteLine($"a={a} b={b}");

var (r, g, bl) = new Color(255, 165, 0);                  // a class with a Deconstruct method
Console.WriteLine($"r={r} g={g} b={bl}");

foreach (var (name, marks) in new Dictionary<string, int> { ["Asha"] = 91, ["Ravi"] = 78 })   //
KeyValuePair has Deconstruct
    Console.WriteLine($"{name}: {marks}");

static (int Min, int Max, double Avg) Stats(int[] v) => (v.Min(), v.Max(), v.Average());

class Color(byte r, byte g, byte b)
{
    public void Deconstruct(out byte red, out byte green, out byte blue) { red = r; green = g; blue = b;
}
}
```

OUTPUT

```
64 91 78
9
a=2 b=1
r=255 g=165 b=0
Asha: 91
Ravi: 78
```

12.5 Nullable reference types (C# 8)

Tony Hoare, who invented the null reference, called it his "billion-dollar mistake": any reference might be null, and forgetting to check causes a `NullReferenceException` at run time. **Nullable reference types** (NRT) let the compiler help. You declare which references may be null (`string?`) and which may not (`string`), and the compiler tracks the **null state** of every variable and warns when you might dereference null or store null where it is not allowed. NRT changes only compile-time analysis: no run-time checks are added.

12.5.1 Enabling the nullable context

New projects created with the .NET SDK have NRT enabled in the project file. It can also be switched on or off for part of a file with a directive, which is useful when migrating old code gradually. Treating nullable warnings as errors makes the protection strict.

MYAPP.CSPROJ xml

```
<PropertyGroup>
  <TargetFramework>net10.0</TargetFramework>
  <Nullable>enable</Nullable>                          <!-- on for the whole project -->
  <WarningsAsErrors>nullable</WarningsAsErrors>        <!-- optional: make them errors -->
</PropertyGroup>
```

PER-FILE DIRECTIVES

C#

```
#nullable enable      // from here to the end of the file (or the next directive)
#nullable disable     // old behaviour: no annotations, no warnings
#nullable restore     // back to the project setting
```

12.5.2 string vs string?

With the nullable context enabled, `string` means "never null" and `string?` means "may be null". The same applies to every reference type: `Student` versus `Student?`. For value types, `int?` is different: it is the real type `Nullable<int>` with `HasValue` and `Value`, and has existed since C# 2.

NULLABLE AND NON-NULLABLE REFERENCES

Program.cs

```
string name = "Asha";            // must never be null
string? nickname = null;         // allowed to be null
Console.WriteLine(name.Length);
Console.WriteLine(nickname?.Length ?? 0);        // must be guarded

int? age = null;                  // Nullable<int>: a value type with HasValue
Console.WriteLine($"age.HasValue {age.HasValue}, default {age.GetValueOrDefault()}, or {age ?? 18}");
age = 21;
Console.WriteLine($"age.Value {age.Value}");
```

OUTPUT

```
4
0
age.HasValue False, default 0, or 18
age.Value 21
```

12.5.3 Null-state analysis and warnings

The compiler follows every path through your code and knows, at each point, whether a variable is **not-null** or **maybe-null**. After a check such as `if (s != null)` or `if (s is null) return;` the variable is known to be not-null. The most common warnings are listed below; each one points at a possible NullReferenceException.

TYPICAL NULLABLE WARNINGS

Program.cs

```
string? FindCity(int id) => id == 1 ? "Pune" : null;

string? city = FindCity(2);
Console.WriteLine(city.Length);            // may be null here

string mustHave = FindCity(1);             // assigning maybe-null to non-nullable
PrintLength(city);

static void PrintLength(string s) => Console.WriteLine(s.Length);

class Student { public string Name { get; set; } }     // never initialised
```

COMPILER OUTPUT

```
Program.cs(4,19): warning CS8602: Dereference of a possibly null reference.
Program.cs(6,19): warning CS8600: Converting null literal or possible null value to non-nullable type.
Program.cs(7,13): warning CS8604: Possible null reference argument for parameter 's' in 'void
PrintLength(string s)'.
Program.cs(11,31): warning CS8618: Non-nullable property 'Name' must contain a non-null value when
exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.
```

Warning	Meaning	Typical fix
CS8602	dereferencing a maybe-null value	check for null, use `?.`, or prove it is not null
CS8600	assigning maybe-null to a non-nullable variable	make the variable nullable or supply a fallback with `??`
CS8604	passing maybe-null to a non-nullable parameter	check first, or make the parameter nullable
CS8618	non-nullable member not initialised by the constructor	initialise it, set it in the constructor, mark it `required`, or make it nullable
CS8625	assigning the null literal to a non-nullable reference	use a nullable type or a real value

12.5.4 Null checks and the null-forgiving operator

The cleanest way to remove a warning is to handle null for real: an `if` check, a pattern `is not null`, the null-conditional `?.`, the null-coalescing `??` and `??=`, or an early `return`/`throw`. When you know better than the compiler (for example right after a validation the compiler cannot see), the **null-forgiving operator** `!` suppresses the warning: `value!.Length`. It does nothing at run time, so if you are wrong, the exception still happens. Use it sparingly.

HANDLING NULL PROPERLY

Program.cs

```
string? input = GetInput(true);

if (input is not null) Console.WriteLine($"length {input.Length}");    // flow analysis: not-null here
Console.WriteLine(input?.ToUpper() ?? "(none)");                       // ?. and ??

string? cached = null;
cached ??= "computed once";                                             // assign only if null
Console.WriteLine(cached.Length);                                       // known not-null now

string? config = GetInput(false);
string safe = config ?? throw new InvalidOperationException("missing config");   // throw expression
Console.WriteLine(safe);

string? fromDb = GetInput(true);
Console.WriteLine(fromDb!.Length);       // "trust me": suppresses CS8602, no run-time effect

static string? GetInput(bool good) => good ? "hello" : "default-config";
```

OUTPUT

```
length 5
HELLO
13
default-config
5
```

TIP

Attributes such as `[NotNullWhen(true)]` (used by `string.IsNullOrEmpty` and `Dictionary.TryGetValue`), `[MaybeNull]` and `[MemberNotNull]` tell the analysis about your own helper methods, so callers get accurate null states without needing `!`.

12.6 Anonymous types

An **anonymous type** is a class without a name, created on the spot with `new { Name = ..., Age = ... }`. The compiler generates a sealed class with read-only properties, value-based `Equals` and `GetHashCode`, and a readable `ToString`. Because the type has no name, it can only be stored in `var` variables and is practical only inside one method. Anonymous types are used mostly as temporary results of LINQ queries (Module 16), where tuples or records are the alternatives.

ANONYMOUS TYPES Program.cs

```
var person = new { Name = "Asha", Age = 21 };
Console.WriteLine(person);
Console.WriteLine($"{person.Name} is {person.Age}");
// person.Age = 22;                       // error CS0200: properties are read-only

var same = new { Name = "Asha", Age = 21 };
Console.WriteLine($"Equals {person.Equals(same)}, == {person == same}");    // value Equals, reference ==

var older = person with { Age = 22 };                    // with works on anonymous types (C# 10)
Console.WriteLine(older);

string city = "Pune";
var projected = new { person.Name, city, Initial = person.Name[0] };     // names inferred from members
Console.WriteLine(projected);

var students = new[] { new { Name = "Ravi", Marks = 78 }, new { Name = "Meera", Marks = 85 } };
foreach (var s in students.Where(s => s.Marks > 80)) Console.WriteLine($"{s.Name} passed with
distinction");
```

OUTPUT

```
{ Name = Asha, Age = 21 }
Asha is 21
Equals True, == False
{ Name = Asha, Age = 22 }
{ Name = Asha, city = Pune, Initial = A }
Meera passed with distinction
```

	Anonymous type	Tuple	Record
Has a name	no	no (element names optional)	yes
Kind	class (reference)	struct (value)	class or struct
Mutable	no	yes (fields)	init-only by default
Usable as return type	no (only object or dynamic)	yes	yes
Best for	LINQ projections inside a method	returning a few values	data shared across the program

Structs are value types copied on assignment; keep them small and readonly, use record struct for generated equality, and know ref structs like Span<T> exist for high-performance code. Enums name a set of integral constants; convert with casts, Parse/TryParse and GetValues, validate with IsDefined, and use [Flags] with power-of-two values for combinations. Records give concise data types with value equality, ToString, with copies and deconstruction, as reference (record) or value (record struct) types. Tuples are ValueTuple structs with optional element names. With nullable reference types enabled, string? may be null and string must not, and the compiler warns about every possible null dereference. Anonymous types are nameless read-only classes for local projections.

PRACTICE CHECKPOINT

Create a **Point readonly struct**, a **Student positional record** copied with **with**, a **[Flags] permissions enum**, and write the whole program with **nullable reference types enabled** so that it compiles with zero warnings.

SAMPLE SOLUTION

Program.cs

```
#nullable enable

var a = new Point(3, 4);
var b = a.Translate(2, -1);
Console.WriteLine($"{a} -> {b}, distance {a.DistanceTo(b):F3}");

var s1 = new Student("Asha", 88, null);
var s2 = s1 with { Marks = 93, Mentor = "Dr. Rao" };
Console.WriteLine(s1);
Console.WriteLine(s2);
Console.WriteLine($"mentor: {s1.Mentor ?? "not assigned"}, {s2.Mentor?.ToUpper()}");
Console.WriteLine($"same student? {s1 == s2}, after reverting: {s1 == s2 with { Marks = 88, Mentor = null
}}");

Access user = Access.Read | Access.Comment;
user |= Access.Write;
Console.WriteLine($"user: {user} ({(int)user}), can delete: {user.HasFlag(Access.Delete)}");
user &= ~Access.Comment;
Console.WriteLine($"user: {user}, admin: {Access.Admin}");

Student? found = Find("Ravi");
Console.WriteLine(found is null ? "Ravi not found" : found.Name);

static Student? Find(string name) => name == "Asha" ? new Student("Asha", 88, null) : null;

public readonly struct Point(double x, double y)
{
    public double X { get; } = x;
    public double Y { get; } = y;
    public Point Translate(double dx, double dy) => new(X + dx, Y + dy);
    public double DistanceTo(Point o) => Math.Sqrt(Math.Pow(X - o.X, 2) + Math.Pow(Y - o.Y, 2));
    public override string ToString() => $"({X}, {Y})";
}

public record Student(string Name, int Marks, string? Mentor);

[Flags]
public enum Access { None = 0, Read = 1, Write = 2, Comment = 4, Delete = 8, Admin = Read | Write |
Comment | Delete }
```

OUTPUT

```
(3, 4) -> (5, 3), distance 2.236
Student { Name = Asha, Marks = 88, Mentor =  }
Student { Name = Asha, Marks = 93, Mentor = Dr. Rao }
mentor: not assigned, DR. RAO
same student? False, after reverting: True
user: Read, Write, Comment (7), can delete: False
user: Read, Write, admin: Admin
Ravi not found
```

MODULE 13 · INTERMEDIATE

Exception handling

Minimum duration **2 days** | Level **Intermediate**

Programs meet problems they cannot prevent: a file is missing, the network drops, the user types letters where a number was expected. .NET reports such run-time errors as exceptions. This module explains what exceptions are and how they are organised, how to catch them with try, catch, finally and filters, how to throw them correctly (including the difference between throw; and throw ex;), how to define custom exceptions, how to clean up resources reliably, and the best practices that keep applications stable and their errors informative.

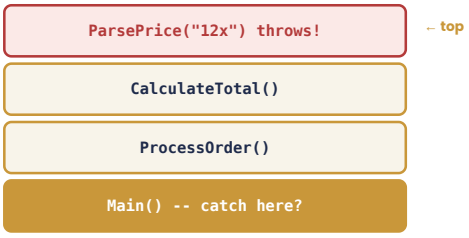
IN THIS MODULE

13.1 Basics
13.2 Handling exceptions
13.3 Throwing exceptions
13.4 Custom exceptions
13.5 Resource cleanup
13.6 Best practices

13.1 Basics

13.1.1 What an exception is

An **exception** is an object that describes an error, created and **thrown** at the point where the problem is detected. Throwing stops the normal flow: the runtime leaves the current method and walks back up the **call stack**, method by method, looking for a matching `catch` block. If it finds one, execution continues there; if it reaches the top without finding one, the exception is **unhandled**, the runtime prints it and terminates the process.



THE EXCEPTION TRAVELS UP THE CALL STACK UNTIL A MATCHING CATCH IS FOUND

AN EXCEPTION UNWINDS THE STACK

Program.cs

```
Console.WriteLine("start");
try
{
    int total = CalculateTotal(new[] { "120", "80", "12x", "50" });
    Console.WriteLine($"total {total}");            // skipped: never reached
}
catch (FormatException e)
{
    Console.WriteLine($"caught in Main: {e.Message}");
}
Console.WriteLine("program continues");

static int CalculateTotal(string[] prices)
{
    int sum = 0;
    foreach (string p in prices)
    {
        sum += int.Parse(p);                        // throws on "12x"
        Console.WriteLine($"  added {p}");
    }
    return sum;
}
```

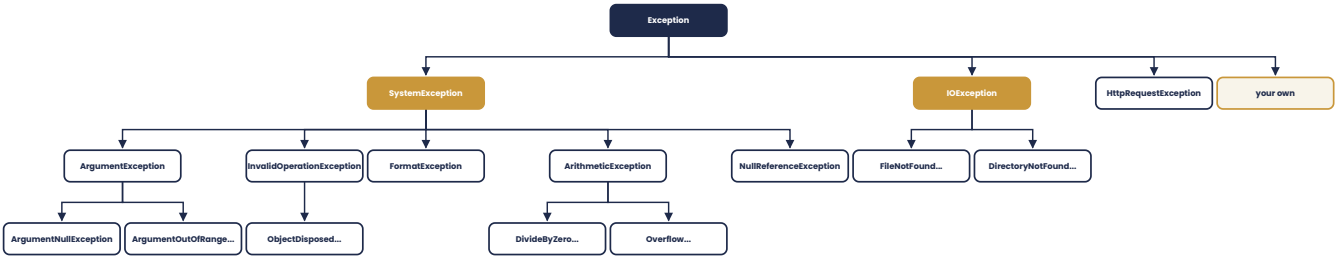
OUTPUT

```
start
  added 120
  added 80
caught in Main: The input string '12x' was not in a correct format.
program continues
```

Without the try/catch, the same program would stop at "12x" with an unhandled-exception report: the exception type, its message and the **stack trace** listing the methods that were active, and the remaining lines would never run.

13.1.2 Exception hierarchy: System.Exception and derived types

All exceptions derive from `System.Exception`. The base class library groups them into families, and a `catch` for a base type also catches all of its derived types. `SystemException` is the historical base of most runtime exceptions; application code should derive custom exceptions directly from `Exception`.



PART OF THE .NET EXCEPTION HIERARCHY (IOEXCEPTION ACTUALLY DERIVES FROM SYSTEMEXCEPTION TOO)

13.1.3 Common exceptions: NullReferenceException, ArgumentException, ArgumentNullException, InvalidOperationException, FormatException, DivideByZeroException, IndexOutOfRangeException, FileNotFoundException

Exception	Thrown when	Typical cause
`NullReferenceException`	a member is accessed through a null reference	missing null check (a bug)

Exception	Thrown when	Typical cause
`ArgumentException`	an argument is invalid	bad input to a method
`ArgumentNullException`	a required argument is null	caller passed null
`ArgumentOutOfRangeException`	an argument is outside its allowed range	negative amount, bad index for a List
`InvalidOperationException`	the call is invalid for the object's current state	reading an empty sequence, modifying a list during foreach
`FormatException`	text has the wrong format	int.Parse("abc")
`DivideByZeroException`	integer or decimal division by zero	divisor not checked (double gives Infinity instead)
`IndexOutOfRangeException`	array index outside 0 to Length - 1	off-by-one loop
`KeyNotFoundException`	a dictionary key does not exist	use TryGetValue
`FileNotFoundException`	a file does not exist	wrong path, file deleted
`OverflowException`	checked arithmetic overflows	`checked` block, Convert.ToInt32 of a huge value

TRIGGERING THE COMMON EXCEPTIONS

Program.cs

```
Action[] faults =
{
    () => { string? s = null; _ = s!.Length; },
    () => int.Parse("abc"),
    () => { int zero = 0; _ = 10 / zero; },
    () => { int[] a = new int[3]; _ = a[3]; },
    () => new List<int>().First(),
    () => new Dictionary<string, int>()["x"].ToString(),
    () => { int big = int.MaxValue; _ = checked(big + 1); },
    () => File.ReadAllText("missing-file.txt"),
};
foreach (Action fault in faults)
{
    try { fault(); }
    catch (Exception e) when (e is not FileNotFoundException)
    {
        Console.WriteLine($"{e.GetType().Name,-28}{e.Message}");
    }
    catch (FileNotFoundException e)
    {
        Console.WriteLine($"{e.GetType().Name,-28}file name ends with missing-file.txt:
{e.FileName!.EndsWith("missing-file.txt")}");
    }
}
```

OUTPUT

```
NullReferenceException      Object reference not set to an instance of an object.
FormatException             The input string 'abc' was not in a correct format.
DivideByZeroException       Attempted to divide by zero.
IndexOutOfRangeException    Index was outside the bounds of the array.
InvalidOperationException   Sequence contains no elements
KeyNotFoundException        The given key 'x' was not present in the dictionary.
OverflowException           Arithmetic operation resulted in an overflow.
FileNotFoundException       file name ends with missing-file.txt: True
```

NOTE

The FileNotFoundException message contains the full path of the missing file, which differs on every machine, so the program prints only whether the file name matches.

13.2 Handling exceptions

13.2.1 try, catch, finally

Code that may fail goes in a `try` block. A `catch` block names the exception type it handles and optionally a variable for the exception object. A `finally` block runs **always** when control leaves the try, whether it completed normally, an exception was caught, an exception is propagating further, or a `return` was executed. finally is the place for clean-up such as closing files or releasing locks. A try must have at least one catch or a finally.

TRY, CATCH AND FINALLY Program.cs

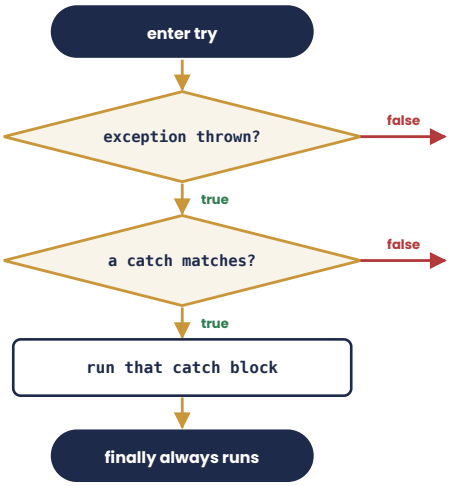
```
Divide(10, 2);
Divide(10, 0);
Console.WriteLine($"returned {Compute()}");

static void Divide(int a, int b)
{
    try
    {
        Console.WriteLine($"{a} / {b} = {a / b}");
    }
    catch (DivideByZeroException e)
    {
        Console.WriteLine($"error: {e.Message}");
    }
    finally
    {
        Console.WriteLine("finally runs either way");
    }
}

static int Compute()
{
    try { return 42; }                                  // finally still runs before the method returns
    finally { Console.WriteLine("finally before return"); }
}
```

OUTPUT

```
10 / 2 = 5
finally runs either way
error: Attempted to divide by zero.
finally runs either way
finally before return
returned 42
```



CONTROL FLOW THROUGH TRY, CATCH AND FINALLY

13.2.2 Multiple catch blocks and their order

A try may be followed by several catch blocks. The runtime checks them **from top to bottom** and runs only the **first** whose type matches (the same type or a base type). Therefore the most specific types must come first and general ones such as `Exception` last; the compiler reports error CS0160 if a catch can never be reached because an earlier one already handles its type.

ORDERING CATCH BLOCKS Program.cs

```
foreach (string input in new[] { "25", "abc", "99999999999", "" , "0" })
{
    try
    {
        int n = int.Parse(input);
        Console.WriteLine($"100 / {n} = {100 / n}");
    }
    catch (FormatException)                         // most specific first
    {
        Console.WriteLine($"'{input}': not a number");
    }
    catch (OverflowException)
    {
        Console.WriteLine($"'{input}': too large for int");
    }
    catch (ArithmeticException e)                   // base of DivideByZero (and Overflow)
    {
        Console.WriteLine($"'{input}': arithmetic error ({e.GetType().Name})");
    }
    catch (Exception e)                             // anything else, last
    {
        Console.WriteLine($"'{input}': unexpected {e.GetType().Name}");
    }
}
```

OUTPUT

```
100 / 25 = 4
'abc': not a number
'99999999999': too large for int
'': not a number
'0': arithmetic error (DivideByZeroException)
```

A GENERAL CATCH BEFORE A SPECIFIC ONE

Program.cs

```
try { int.Parse("x"); }
catch (Exception) { }
catch (FormatException) { }      // already caught above
```

COMPILER OUTPUT

```
Program.cs(3,8): error CS0160: A previous catch clause already catches all exceptions of this or of a
super type ('Exception')
```

13.2.3 Exception filters with when

An **exception filter** adds a condition to a catch: `catch (HttpRequestException e) when (e.StatusCode == HttpStatusCode.NotFound)`. If the condition is false the catch is skipped as if it did not match, and the search continues. Filters run **before** the stack unwinds, so a debugger or crash dump still sees the original state, and they avoid catching and rethrowing just to inspect an exception.

FILTERING WITH WHEN

Program.cs

```
foreach (int code in new[] { 404, 503, 500 })
{
    try
    {
        CallServer(code);
    }
    catch (ServerException e) when (e.StatusCode == 404)
    {
        Console.WriteLine("404: resource not found, show a friendly page");
    }
    catch (ServerException e) when (e.StatusCode >= 500 && IsRetryable(e))
    {
        Console.WriteLine($"{e.StatusCode}: retrying later");
    }
    catch (ServerException e)
    {
        Console.WriteLine($"{e.StatusCode}: giving up");
    }
}

static bool IsRetryable(ServerException e) => e.StatusCode == 503;
static void CallServer(int code) => throw new ServerException(code);

class ServerException(int statusCode) : Exception($"Server returned {statusCode}")
{
    public int StatusCode { get; } = statusCode;
}
```

OUTPUT

```
404: resource not found, show a friendly page
503: retrying later
500: giving up
```

TIP

A filter that logs and returns false, `catch (Exception e) when (Log(e))` with `Log` returning false, records every exception at the point where it occurs without handling it.

13.2.4 Exception properties: Message, StackTrace, InnerException

Property	Contents
`Message`	a human-readable description of the error
`StackTrace`	the chain of method calls from where it was thrown to where it was caught (with file and line numbers when debug symbols are present)
`InnerException`	the original exception that caused this one, or null
`Source`	the name of the assembly that threw it
`TargetSite`	the method that threw it
`Data`	a dictionary for extra key/value details
`HResult`	a numeric code, mainly for interop
`ToString()`	type, message, inner exceptions and stack trace together: the best thing to log

INSPECTING AN EXCEPTION Program.cs

```
try
{
    Orders.Process();
}
catch (InvalidOperationException e)
{
    Console.WriteLine($"Type:    {e.GetType().FullName}");
    Console.WriteLine($"Message: {e.Message}");
    Console.WriteLine($"Inner:   {e.InnerException?.GetType().Name}: {e.InnerException?.Message}");
    Console.WriteLine($"Thrown in: {e.TargetSite?.Name}");
    Console.WriteLine($"Trace mentions Orders.Load: {e.StackTrace!.Contains("Orders.Load")}");
    Console.WriteLine($"Data[\"OrderId\"] = {e.Data["OrderId"]}");
}

static class Orders
{
    public static void Process() => Load("A-17");

    public static void Load(string id)
    {
        try
        {
            int qty = int.Parse("seven");
        }
        catch (FormatException fe)
        {
            var ex = new InvalidOperationException($"Order {id} could not be loaded", fe);   // wrap
            ex.Data["OrderId"] = id;
            throw ex;
        }
    }
}
```

OUTPUT

```
Type:    System.InvalidOperationException
Message: Order A-17 could not be loaded
Inner:   FormatException: The input string 'seven' was not in a correct format.
Thrown in: Load
Trace mentions Orders.Load: True
Data["OrderId"] = A-17
```

A stack trace printed with `Console.WriteLine(e)` looks like the sample below. Read it from the top: the first "at" line is where the exception was thrown; lower lines are the callers. Exact method decorations, paths and line numbers depend on the build.

Sample output of Console.WriteLine(e)

```
System.InvalidOperationException: Order A-17 could not be loaded
 ---> System.FormatException: The input string 'seven' was not in a correct format.
   at System.Number.ThrowFormatException[TChar](ReadOnlySpan`1 value)
   at System.Int32.Parse(String s)
   at Orders.Load(String id) in /src/Demo/Program.cs:line 22
   --- End of inner exception stack trace ---
   at Orders.Load(String id) in /src/Demo/Program.cs:line 28
   at Orders.Process() in /src/Demo/Program.cs:line 15
   at Program.<Main>$(String[] args) in /src/Demo/Program.cs:line 3
```

13.3 Throwing exceptions

13.3.1 throw statement

Your own methods should throw an exception when they cannot do what their name promises, typically when arguments are invalid or the object is in the wrong state. Create an exception object of the most appropriate type with a clear message and `throw` it. Prefer existing types (`ArgumentException`, `InvalidOperationException`, `NotSupportedException`) when they fit; for arguments, pass the parameter name with `nameof`.

THROWING FROM YOUR OWN CODE

Program.cs

```
var acc = new Account(500m);
foreach (decimal amt in new[] { 200m, -50m, 1000m })
{
    try
    {
        acc.Withdraw(amt);
        Console.WriteLine($"withdrew {amt}, balance {acc.Balance}");
    }
    catch (ArgumentOutOfRangeException e) { Console.WriteLine($"bad argument '{e.ParamName}'"); }
    catch (InvalidOperationException e) { Console.WriteLine(e.Message); }
}

class Account(decimal opening)
{
    public decimal Balance { get; private set; } = opening;

    public void Withdraw(decimal amount)
    {
        if (amount <= 0)
            throw new ArgumentOutOfRangeException(nameof(amount), amount, "Amount must be positive.");
        if (amount > Balance)
            throw new InvalidOperationException($"Cannot withdraw {amount}; balance is only {Balance}.");
        Balance -= amount;
    }
}
```

OUTPUT

```
withdrew 200, balance 300
bad argument 'amount'
Cannot withdraw 1000; balance is only 300.
```

13.3.2 throw; vs throw ex; (preserving the stack trace)

Inside a catch block you may want to log an exception and then let it continue upward. Write `throw;` on its own: it **rethrows the same exception with its original stack trace**. Writing `throw ex;` throws it again as if it started at that line, **resetting the stack trace** and hiding where the error really happened. Code analysis rule CA2200 warns about `throw ex;`.

THROW; KEEPS THE ORIGINAL STACK TRACE Program.cs

```
foreach (bool good in new[] { true, false })
{
    try
    {
        if (good) Rethrow.Good(); else Rethrow.Bad();
    }
    catch (InvalidOperationException e)
    {
        bool hasOrigin = e.StackTrace!.Contains("Rethrow.DeepWork");
        Console.WriteLine($"{(good ? "throw;    " : "throw ex;")} trace still shows DeepWork:
{hasOrigin}");
    }
}

static class Rethrow
{
    public static void DeepWork() => throw new InvalidOperationException("failure deep inside");

    public static void Good()
    {
        try { DeepWork(); }
        catch (InvalidOperationException) { Console.WriteLine("  logged, rethrowing with throw;"); throw;
}
    }

    public static void Bad()
    {
        try { DeepWork(); }
        catch (InvalidOperationException ex) { Console.WriteLine("  logged, rethrowing with throw ex;");
throw ex; }
    }
}
```

OUTPUT

```
  logged, rethrowing with throw;
throw;    trace still shows DeepWork: True
  logged, rethrowing with throw ex;
throw ex; trace still shows DeepWork: False
```

NOTE

Run in the Debug configuration (the default for `dotnet run`), small methods are not inlined, so DeepWork appears in the trace exactly as written. To add context instead of rethrowing unchanged, throw a **new** exception with the original as its `InnerException` (13.4.2).

13.3.3 Throw expressions

Since C# 7, `throw` can be used as an **expression** in three places: the right side of `??`, the branches of the conditional operator `?:`, and expression-bodied members and lambdas. This makes guard code compact.

THROW EXPRESSIONS

Program.cs

```
Console.WriteLine(new Customer("Asha", "asha@example.com").Email);
try { new Customer("Ravi", null); }
catch (ArgumentNullException e) { Console.WriteLine($"{e.GetType().Name}: {e.ParamName}"); }
try { Console.WriteLine(Customer.Discount(-1)); }
catch (ArgumentOutOfRangeException e) { Console.WriteLine($"{e.GetType().Name}: {e.ParamName}"); }

class Customer(string name, string? email)
{
    public string Name { get; } = name;
    public string Email { get; } = email ?? throw new ArgumentNullException(nameof(email));   // with ??

    public static int Discount(int years) =>
        years >= 0 ? Math.Min(years * 2, 20) : throw new ArgumentOutOfRangeException(nameof(years));   //
with ?:

    public string Tier => throw new NotImplementedException();   // expression body
}
```

OUTPUT

```
asha@example.com
ArgumentNullException: email
ArgumentOutOfRangeException: years
```

13.3.4 Guard helpers: ArgumentNullException.ThrowIfNull, ArgumentOutOfRangeException.ThrowIfNegative

Modern .NET provides static **guard helpers** that check a condition and throw the right exception with the right parameter name, captured automatically by `[CallerArgumentExpression]`. They make validation one line per rule.

Helper	Throws when
`ArgumentNullException.ThrowIfNull(x)`	x is null
`ArgumentException.ThrowIfNullOrEmpty(s)`	s is null or ""
`ArgumentException.ThrowIfNullOrWhiteSpace(s)`	s is null, empty or only spaces
`ArgumentOutOfRangeException.ThrowIfNegative(n)`	n < 0
`ArgumentOutOfRangeException.ThrowIfNegativeOrZero(n)`	n <= 0
`ArgumentOutOfRangeException.ThrowIfGreaterThan(n, max)`	n > max (also LessThan, Equal, NotEqual…)
`ObjectDisposedException.ThrowIf(disposed, this)`	the object was disposed

GUARD CLAUSES IN ONE LINE EACH

Program.cs

```
Register("Asha", 21, 85);
foreach (Action bad in new Action[] { () => Register(null!, 21, 85), () => Register("  ", 21, 85), () =>
Register("Ravi", -3, 85), () => Register("Meera", 20, 140) })
{
    try { bad(); }
    catch (ArgumentException e) { Console.WriteLine($"{e.GetType().Name,-28} parameter: {e.ParamName}");
}
}

static void Register(string name, int age, int marks)
{
    ArgumentException.ThrowIfNullOrWhiteSpace(name);            // also handles null
    ArgumentOutOfRangeException.ThrowIfNegative(age);
    ArgumentOutOfRangeException.ThrowIfGreaterThan(marks, 100);
    Console.WriteLine($"registered {name}, {age}, {marks}");
}
```

OUTPUT

```
registered Asha, 21, 85
ArgumentNullException        parameter: name
ArgumentException            parameter: name
ArgumentOutOfRangeException  parameter: age
ArgumentOutOfRangeException  parameter: marks
```

13.4 Custom exceptions

When none of the built-in types describes a failure of **your domain** well (insufficient funds, seat already booked, invalid course code), define your own exception class. Callers can then catch exactly that situation, and the exception can carry extra data that helps them react.

13.4.1 Deriving from Exception

A custom exception derives from `Exception` (or a more specific existing exception), has a name ending in **Exception**, and by convention provides the three standard constructors: parameterless, with a message, and with a message and inner exception. Add read-only properties for the extra details.

A CUSTOM EXCEPTION WITH EXTRA DATA

Program.cs

```
var seats = new SeatBooking(totalSeats: 3);
foreach (string name in new[] { "Asha", "Ravi", "Meera", "Kiran" })
{
    try
    {
        int seat = seats.Book(name);
        Console.WriteLine($"{name} -> seat {seat}");
    }
    catch (SeatsFullException e)
    {
        Console.WriteLine($"{e.Message} (capacity {e.Capacity}, waiting list for {e.Customer})");
    }
}

public class SeatsFullException : Exception
{
    public int Capacity { get; }
    public string? Customer { get; }

    public SeatsFullException() { }
    public SeatsFullException(string message) : base(message) { }
    public SeatsFullException(string message, Exception inner) : base(message, inner) { }

    public SeatsFullException(int capacity, string customer)
        : base($"All {capacity} seats are booked")
    {
        Capacity = capacity;
        Customer = customer;
    }
}

public class SeatBooking(int totalSeats)
{
    private int booked;
    public int Book(string customer) =>
        booked < totalSeats ? ++booked : throw new SeatsFullException(totalSeats, customer);
}
```

OUTPUT

```
Asha -> seat 1
Ravi -> seat 2
Meera -> seat 3
All 3 seats are booked (capacity 3, waiting list for Kiran)
```

TIP

Do not create a custom exception for every error. Make one only when callers are expected to catch and handle that case differently. Keep them in a small hierarchy (e.g. `BookingException` with `SeatsFullException` and `PaymentDeclinedException`) so callers can catch the whole family.

13.4.2 Inner exceptions

Low-level exceptions (a FormatException, an IOException) often mean little at a higher level. **Exception wrapping** catches the low-level exception and throws a higher-level one that explains what operation failed, passing the original as the `innerException` argument. Nothing is lost: the inner exception, its message and its stack trace remain available, and `ToString()` prints the whole chain.

WRAPPING WITH AN INNER EXCEPTION Program.cs

```
try
{
    var cfg = ConfigLoader.Load(new Dictionary<string, string> { ["port"] = "80a" });
}
catch (ConfigException e)
{
    Console.WriteLine($"Error: {e.Message}");
    for (Exception? inner = e.InnerException; inner is not null; inner = inner.InnerException)
        Console.WriteLine($"  caused by {inner.GetType().Name}: {inner.Message}");
    Console.WriteLine($"root cause: {e.GetBaseException().GetType().Name}");
}

public class ConfigException(string message, Exception inner) : Exception(message, inner);

public static class ConfigLoader
{
    public static int Load(Dictionary<string, string> settings)
    {
        try
        {
            return int.Parse(settings["port"]);
        }
        catch (FormatException ex)
        {
            throw new ConfigException("Setting 'port' must be a whole number", ex);   // wrap, keep the
cause
        }
    }
}
```

OUTPUT

```
Error: Setting 'port' must be a whole number
  caused by FormatException: The input string '80a' was not in a correct format.
root cause: FormatException
```

13.5 Resource cleanup

Memory is reclaimed by the garbage collector, but other resources are not: open files, network sockets, database connections, locks. They must be released as soon as they are no longer needed, and also when an exception occurs, or the program leaks handles until it fails.

13.5.1 IDisposable and Dispose()

Types that hold such resources implement `IDisposable`, whose single method `Dispose()` releases them. `FileStream`, `StreamReader`, `HttpClient`, `SqlConnection` and `Timer` are examples. Without language help, correct use requires a try/finally so that Dispose runs even on failure. Dispose should be safe to call more than once.

DISPOSE IN FINALLY

Program.cs

```
var log = new LogFile("app.log");
try
{
    log.Write("starting");
    throw new InvalidOperationException("crash while working");
}
catch (InvalidOperationException e) { Console.WriteLine($"caught: {e.Message}"); }
finally
{
    log.Dispose();                    // always released
}
log.Dispose();                        // second call does nothing

public class LogFile(string path) : IDisposable
{
    private bool disposed;
    public void Write(string text)
    {
        ObjectDisposedException.ThrowIf(disposed, this);
        Console.WriteLine($"[{path}] {text}");
    }
    public void Dispose()
    {
        if (disposed) return;         // idempotent
        disposed = true;
        Console.WriteLine($"closed {path}");
    }
}
```

OUTPUT

```
[app.log] starting
caught: crash while working
closed app.log
```

13.5.2 using statements and using declarations

The `using` statement is shorthand for exactly that try/finally: the object is disposed when the block ends, however it ends. The C# 8 **using declaration** `using var x = ...;` needs no block: the object is disposed at the end of the enclosing scope. Several resources are disposed in **reverse** order of creation.

USING STATEMENTS AND DECLARATIONS Program.cs

```
try
{
    using (var a = new Res("A"))                // using statement with a block
    {
        Console.WriteLine("working with A");
        throw new Exception("failure inside using");
    }                                           // A disposed here, before the catch runs
}
catch (Exception e) { Console.WriteLine($"caught: {e.Message}"); }

Copy();

static void Copy()
{
    using var source = new Res("source");       // using declarations
    using var target = new Res("target");
    Console.WriteLine("copying");
}                                               // disposed in reverse order: target, source

class Res(string name) : IDisposable
{
    public void Dispose() => Console.WriteLine($"dispose {name}");
}
```

OUTPUT

```
working with A
dispose A
caught: failure inside using
copying
dispose target
dispose source
```

THE USING STATEMENT EXPANDED C#

```
// what the compiler generates for:  using (var a = new Res("A")) { ... }
var a = new Res("A");
try
{
    ...
}
finally
{
    a?.Dispose();
}
```

13.6 Best practices

13.6.1 Catch specific exceptions

Catch only exceptions you can actually **handle**: recover, retry, use a default, or report to the user meaningfully. Catching `Exception` everywhere hides bugs such as NullReferenceException that should be fixed, not swallowed. An empty catch block is almost always wrong.

SPECIFIC VERSUS CATCH-ALL

Program.cs

```
// BAD: swallows everything, including bugs
static int ParseBad(string s)
{
    try { return int.Parse(s.Trim()); }
    catch { return 0; }                       // a null s (a bug) also becomes 0, silently
}

// GOOD: handle the expected failure, let bugs surface
static int ParseGood(string s)
{
    try { return int.Parse(s.Trim()); }
    catch (FormatException) { return 0; }     // expected: user typed letters
}

Console.WriteLine($"{ParseBad("x")} {ParseGood("x")} {ParseBad(null!)}");
try { ParseGood(null!); }
catch (NullReferenceException) { Console.WriteLine("ParseGood exposes the bug: NullReferenceException");
}
```

OUTPUT

```
0 0 0
ParseGood exposes the bug: NullReferenceException
```

13.6.2 Do not use exceptions for normal control flow

Throwing and catching is **expensive** compared with an if: the runtime captures a stack trace and unwinds frames, costing microseconds instead of nanoseconds. It also makes code harder to follow. Exceptions are for **exceptional** situations. Expected outcomes (a user typing invalid input, a key that may be absent, an empty list) should be checked with conditions.

CHECK INSTEAD OF CATCHING

Program.cs

```
var stock = new Dictionary<string, int> { ["pen"] = 40, ["book"] = 0 };

// BAD: exception as an if
int Get1(string item) { try { return stock[item]; } catch (KeyNotFoundException) { return -1; } }

// GOOD: test first
int Get2(string item) => stock.TryGetValue(item, out int qty) ? qty : -1;

Console.WriteLine($"{Get1("pen")} {Get1("bag")} | {Get2("pen")} {Get2("bag")}");
```

OUTPUT

```
40 -1 | 40 -1
```

13.6.3 The TryXxx pattern as an alternative

When failure is a normal, expected outcome, offer a **Try** method that returns `bool` and delivers the result through an `out` parameter, next to (or instead of) the throwing version. The BCL does this everywhere: `int.TryParse`, `Dictionary.TryGetValue`, `Enum.TryParse`, `Queue.TryDequeue`.

OFFERING A TRY METHOD Program.cs

```
var bank = new Bank();
if (bank.TryWithdraw(300m, out decimal left)) Console.WriteLine($"ok, left {left}");
if (!bank.TryWithdraw(5000m, out left)) Console.WriteLine($"declined, still {left}");
try { bank.Withdraw(5000m); }
catch (InvalidOperationException e) { Console.WriteLine($"throwing version: {e.Message}"); }

class Bank
{
    private decimal balance = 1000m;

    public bool TryWithdraw(decimal amount, out decimal remaining)   // expected failure: no exception
    {
        if (amount <= 0 || amount > balance) { remaining = balance; return false; }
        balance -= amount;
        remaining = balance;
        return true;
    }

    public void Withdraw(decimal amount)                          // throwing version built on it
    {
        if (!TryWithdraw(amount, out _)) throw new InvalidOperationException("insufficient funds");
    }
}
```

OUTPUT

```
ok, left 700
declined, still 700
throwing version: insufficient funds
```

13.6.4 Logging exceptions

When an exception is handled at a boundary (a request handler, a background job, the top of the program), **log it**, including the full `ToString()` with the stack trace and inner exceptions, plus context such as the user, order id or input. In real applications use `ILogger` from Microsoft.Extensions.Logging (built into ASP.NET Core and the generic host) or a library such as Serilog, which pass the exception object as a separate argument so that structured log stores keep every detail.

LOGGING WITH ILOGGER (ASP.NET CORE / GENERIC HOST) C#

```
public class OrderService(ILogger<OrderService> logger)
{
    public void Place(Order order)
    {
        try
        {
            // ... save the order, charge the card ...
        }
        catch (PaymentDeclinedException ex)
        {
            logger.LogWarning(ex, "Payment declined for order {OrderId}", order.Id);   // expected:
warning
            throw;
        }
        catch (Exception ex)
        {
            logger.LogError(ex, "Unexpected failure placing order {OrderId}", order.Id); // full
exception logged
            throw;
        }
    }
}
```

NOTE

Log an exception once, where it is handled, not at every level it passes through; otherwise the same error appears many times. Never log secrets such as passwords or card numbers in exception messages.

13.6.5 Global exception handling (overview)

Some exceptions will always escape. A **global handler** at the top of the application logs them and shows a friendly message instead of a crash dialog. Each application model has its own hook.

Application type	Global hook
Console app	a try/catch around the code in Main; `AppDomain.CurrentDomain.UnhandledException` to log before the process ends
ASP.NET Core	`app.UseExceptionHandler(...)` middleware or an `IExceptionHandler` implementation (.NET 8+), returning a ProblemDetails response
Background tasks	`TaskScheduler.UnobservedTaskException`; always await tasks or observe their exceptions
WPF / WinForms / MAUI	`Application.DispatcherUnhandledException`, `Application.ThreadException`, MAUI unhandled-exception events

A TOP-LEVEL HANDLER IN A CONSOLE APP — Program.cs

```
AppDomain.CurrentDomain.UnhandledException += (_, e) =>
    Console.WriteLine($"[fatal] logged: {((Exception)e.ExceptionObject).Message}");

try
{
    RunApplication();
}
catch (Exception ex)                                    // last line of defence in Main
{
    Console.WriteLine($"Sorry, something went wrong ({ex.GetType().Name}). Details were logged.");
    Environment.ExitCode = 1;
}

static void RunApplication()
{
    Console.WriteLine("application running");
    throw new InvalidOperationException("unexpected state");
}
```

OUTPUT

```
application running
Sorry, something went wrong (InvalidOperationException). Details were logged.
```

An exception is an object thrown when an operation fails; it unwinds the call stack until a matching catch is found. All exceptions derive from System.Exception. Use try/catch for errors you can handle, ordering catch blocks from specific to general, add when filters for conditions, and put clean-up in finally or, better, using statements and declarations for IDisposable objects. Throw the most appropriate exception type with a clear message, use guard helpers like ThrowIfNull, rethrow with throw; (never throw ex;) and wrap low-level errors with an InnerException. Define custom exceptions for domain failures callers will handle. Do not use exceptions for normal control flow: offer TryXxx methods, log exceptions once at boundaries, and add a global handler.

PRACTICE CHECKPOINT

Write an **ATM withdrawal** program with a custom **InsufficientFundsException**, an **exception filter**, and a **finally** block that always logs the transaction attempt.

SAMPLE SOLUTION

Program.cs

```
var atm = new Atm(5000m);
decimal[] requests = { 2000m, 3200m, 10_000m, -100m, 1550m, 1500m };
int attempt = 0;

foreach (decimal amount in requests)
{
    attempt++;
    bool ok = false;
    try
    {
        atm.Withdraw(amount);
        ok = true;
        Console.WriteLine($"Dispensed {amount}. Balance {atm.Balance}");
    }
    catch (InsufficientFundsException e) when (e.Shortfall <= 500m)      // filter: nearly enough
    {
        Console.WriteLine($"Short by {e.Shortfall}. You can withdraw up to {e.Available}.");
    }
    catch (InsufficientFundsException e)
    {
        Console.WriteLine($"Insufficient funds: requested {e.Requested}, available {e.Available}.");
    }
    catch (ArgumentException e)
    {
        Console.WriteLine($"Invalid amount: {e.Message}");
    }
    finally
    {
        Console.WriteLine($"  [log] attempt #{attempt}: withdraw {amount} -> {(ok ? "OK" : "FAILED")}");
    }
}

public class InsufficientFundsException : Exception
{
    public decimal Requested { get; }
    public decimal Available { get; }
    public decimal Shortfall => Requested - Available;

    public InsufficientFundsException(decimal requested, decimal available)
        : base($"Requested {requested} but only {available} is available")
    {
        Requested = requested;
        Available = available;
    }
}

public class Atm(decimal opening)
{
    public decimal Balance { get; private set; } = opening;

    public void Withdraw(decimal amount)
    {
        if (amount <= 0) throw new ArgumentException("must be positive", nameof(amount));
        if (amount % 100 != 0) throw new ArgumentException("must be a multiple of 100", nameof(amount));
        if (amount > Balance) throw new InsufficientFundsException(amount, Balance);
        Balance -= amount;
```

```
    }
}
```

OUTPUT

```
Dispensed 2000. Balance 3000
  [log] attempt #1: withdraw 2000 -> OK
Short by 200. You can withdraw up to 3000.
  [log] attempt #2: withdraw 3200 -> FAILED
Insufficient funds: requested 10000, available 3000.
  [log] attempt #3: withdraw 10000 -> FAILED
Invalid amount: must be positive (Parameter 'amount')
  [log] attempt #4: withdraw -100 -> FAILED
Invalid amount: must be a multiple of 100 (Parameter 'amount')
  [log] attempt #5: withdraw 1550 -> FAILED
Dispensed 1500. Balance 1500
  [log] attempt #6: withdraw 1500 -> OK
```

MODULE 14 · INTERMEDIATE

Generics and collections

Minimum duration **4 days** | Level **Intermediate**

Almost every program stores groups of objects: the students in a course, the products in a cart, the pending jobs in a queue. Generics let one piece of code work with any type while keeping full type safety, and the .NET collections library builds on them to provide lists, dictionaries, sets, queues, stacks and more. This module covers generic methods, classes and constraints, variance, the legacy collections to avoid, every major generic collection, the collection interfaces, immutable, frozen and concurrent collections, sorting, iterators with yield, collection expressions, and how to choose the right collection by its Big-O costs.

IN THIS MODULE

14.1 Generics
14.2 Non-generic collections (legacy)
14.3 Generic collections
14.4 Collection interfaces
14.5 Specialised collections
14.6 Sorting and comparing
14.7 Iterators
14.8 Collection expressions and spread
14.9 Choosing the right collection

14.1 Generics

Generics let you write a class, interface, method or delegate with a **type parameter**, a placeholder such as `T`, that is filled in when the code is used: `List<int>`, `List<string>`, `List<Student>`. The same code then works for every type, and the compiler checks every use.

14.1.1 Why generics (type safety, no boxing, reuse)

Before generics (C# 1), general-purpose containers stored everything as `object`. That had three problems: **no type safety** (any object could be added, and mistakes surfaced only as InvalidCastException at run time), **casting** on every read, and **boxing**: storing a value type such as int as object allocates a heap box for every element. Generics solve all three, and one implementation serves every type.

OBJECT VERSUS GENERIC Program.cs

```
// Without generics: an object-based box
var ob = new ObjectBox();
ob.Value = 42;                          // int boxed into object
int n = (int)ob.Value;                  // cast (and unbox) needed
ob.Value = "oops";                      // compiles: no type safety
try { int bad = (int)ob.Value; }
catch (InvalidCastException) { Console.WriteLine("run-time InvalidCastException"); }

// With generics: type checked at compile time, no cast, no boxing
var gb = new Box<int> { Value = 42 };
int m = gb.Value;
// gb.Value = "oops";                   // error CS0029: cannot convert string to int
var sb = new Box<string> { Value = "same code, other type" };
Console.WriteLine($"{n} {m} {sb.Value} {sb}");

class ObjectBox { public object? Value; }

class Box<T>                            // T is the type parameter
{
    public T? Value { get; set; }
    public override string ToString() => $"Box<{typeof(T).Name}>";
}
```

OUTPUT

```
run-time InvalidCastException
42 42 same code, other type Box<String>
```

14.1.2 Generic methods

A **generic method** declares its own type parameters after its name: `static void Swap<T>(ref T a, ref T b)`. The caller can name the type (`Swap<int>(...)`), but usually the compiler **infers** it from the arguments. Generic methods can live in non-generic classes.

GENERIC METHODS WITH TYPE INFERENCE Program.cs

```
int a = 1, b = 2;
Swap(ref a, ref b);                                  // T inferred as int
string x = "left", y = "right";
Swap<string>(ref x, ref y);                          // T given explicitly
Console.WriteLine($"{a} {b} {x} {y}");

Console.WriteLine(Max(3, 9));                        // int
Console.WriteLine(Max("mango", "apple"));            // string
Console.WriteLine(First(new[] { 'q', 'r' }, '?'));
Console.WriteLine(First(Array.Empty<double>(), -1.0));
PrintAll(new List<bool> { true, false });

static void Swap<T>(ref T a, ref T b) => (a, b) = (b, a);

static T Max<T>(T a, T b) where T : IComparable<T> => a.CompareTo(b) >= 0 ? a : b;   // constraint: see 14.1.4

static T First<T>(T[] items, T fallback) => items.Length > 0 ? items[0] : fallback;

static void PrintAll<T>(IEnumerable<T> items) => Console.WriteLine($"{typeof(T).Name}: {string.Join(", ", items)}");
```

OUTPUT

```
2 1 right left
9
mango
q
-1
Boolean: True, False
```

14.1.3 Generic classes and interfaces

A generic class lists its type parameters after its name and uses them for fields, properties, parameters and return types. There may be several: `Pair<TFirst, TSecond>`. Each distinct set of type arguments produces a distinct **constructed type**: `Pair<int, string>` and `Pair<string, int>` are different types. Interfaces can be generic too, and a class may implement a generic interface for a specific type or stay generic itself.

GENERIC CLASSES AND A GENERIC INTERFACE Program.cs

```
var stack = new SimpleStack<string>();
stack.Push("first"); stack.Push("second"); stack.Push("third");
Console.WriteLine($"{stack.Pop()} {stack.Peek()} count {stack.Count}");

var p = new Pair<string, int>("Asha", 91);
var q = p.Swap();                                    // Pair<int, string>
Console.WriteLine($"{p} -> {q}");

IRepository<string> repo = new MemoryRepository<string>();
repo.Add("C#"); repo.Add("Java");
Console.WriteLine(string.Join(", ", repo.GetAll()));

public class SimpleStack<T>
{
    private T[] items = new T[4];
    public int Count { get; private set; }

    public void Push(T item)
    {
        if (Count == items.Length) Array.Resize(ref items, items.Length * 2);   // grow when full
        items[Count++] = item;
    }
    public T Pop() => Count > 0 ? items[--Count] : throw new InvalidOperationException("empty");
```

```
    public T Peek() => Count > 0 ? items[Count - 1] : throw new InvalidOperationException("empty");
}

public record Pair<TFirst, TSecond>(TFirst First, TSecond Second)
{
    public Pair<TSecond, TFirst> Swap() => new(Second, First);
}

public interface IRepository<T>
{
    void Add(T item);
    IEnumerable<T> GetAll();
}

public class MemoryRepository<T> : IRepository<T>
{
    private readonly List<T> store = new();
    public void Add(T item) => store.Add(item);
    public IEnumerable<T> GetAll() => store;
}
```

OUTPUT

```
third second count 2
Pair { First = Asha, Second = 91 } -> Pair { First = 91, Second = Asha }
C#, Java
```

14.1.4 Constraints: where T : class, struct, new(), base class, interface, notnull, unmanaged

Inside a generic method, the compiler knows nothing about T, so only the members of `object` are available. A **constraint** restricts which types may be used for T and, in return, lets the code use what the constraint guarantees: interface members, base class members, a parameterless constructor, or comparison with null.

Constraint	T must be	Allows
`where T : class`	a reference type (`class?` allows nullable)	null comparisons, `as`
`where T : struct`	a non-nullable value type	use in `T?` as Nullable<T>
`where T : new()`	a type with a public parameterless constructor	`new T()`
`where T : BaseClass`	BaseClass or derived from it	the base class's members
`where T : IInterface`	a type implementing the interface	the interface's members
`where T : U`	the same as or derived from another type parameter U	treating T as U
`where T : notnull`	a non-nullable type (value or reference)	safe dictionary keys
`where T : unmanaged`	a value type containing no references	pointers, `sizeof`, raw memory
`where T : allows ref struct`	may be a ref struct (C# 13)	generic code over Span<T>

CONSTRAINTS UNLOCK MEMBERS

Program.cs

```
var cache = new Factory<Student>();
Student s = cache.Create();
Console.WriteLine($"created {s.Name}");

Console.WriteLine(Largest(new[] { 4, 17, 9 }));
Console.WriteLine(Largest(new[] { "pear", "fig", "plum" }));
Console.WriteLine(TotalPay(new List<Manager> { new() { Salary = 50_000 }, new() { Salary = 70_000 } }));
Console.WriteLine($"{SizeOf<int>()} {SizeOf<double>()} {SizeOf<Point2>()}");
Console.WriteLine(FirstOrNull(new List<string>()) ?? "(null)");

static T Largest<T>(T[] items) where T : IComparable<T>          // interface constraint
{
    T best = items[0];
    foreach (T item in items) if (item.CompareTo(best) > 0) best = item;
    return best;
}

static decimal TotalPay<T>(IEnumerable<T> staff) where T : Employee   // base class constraint
{
    decimal sum = 0;
    foreach (T e in staff) sum += e.Salary;                          // Employee members usable
    return sum;
}

static unsafe int SizeOf<T>() where T : unmanaged => sizeof(T);      // needs AllowUnsafeBlocks

static T? FirstOrNull<T>(List<T> list) where T : class => list.Count > 0 ? list[0] : null;   // class:
null allowed

class Factory<T> where T : new()                                     // new() constraint
{
    public T Create() => new T();
}

class Student { public string Name { get; set; } = "new student"; }
class Employee { public decimal Salary { get; set; } }
class Manager : Employee { }
struct Point2 { public int X, Y; }
```

OUTPUT

```
created new student
17
plum
120000
4 8 8
(null)
```

NOTE

The `unsafe` SizeOf example requires `<AllowUnsafeBlocks>true</AllowUnsafeBlocks>` in the project file. Several constraints can be combined: `where T : class, IComparable<T>, new()` (new() must be last), and each type parameter gets its own where clause.

14.1.5 Default values with default(T)

Generic code often needs "no value" for T: the result when a search finds nothing, or the initial value of a field. Since T may be a reference type (default null) or a value type (default 0, false, …), C# offers `default(T)`, or simply `default` when the type is known from context.

DEFAULT FOR ANY TYPE

Program.cs

```
Console.WriteLine($"[{default(int)}] [{default(bool)}] [{default(string) ?? "null"}]
[{default(DateTime):yyyy-MM-dd}]");
Console.WriteLine(FindOrDefault(new[] { 3, 8, 12 }, x => x > 10));   // found: 12
Console.WriteLine(FindOrDefault(new[] { 3, 8 }, x => x > 10));       // not found: default(int) = 0
Console.WriteLine(FindOrDefault(new[] { "a", "bb" }, s => s.Length > 5) ?? "null");

static T? FindOrDefault<T>(T[] items, Func<T, bool> match)
{
    foreach (T item in items) if (match(item)) return item;
    return default;                                  // null for reference types, 0/false/... for value
types
}
```

OUTPUT

```
[0] [False] [null] [0001-01-01]
12
0
null
```

WATCH OUT

For an unconstrained `T?` with T = int, `T?` is just int (not Nullable<int>), so "not found" and "found 0" look the same. When that matters, return a bool with an out parameter (the Try pattern) or constrain T.

14.1.6 Covariance (out) and contravariance (in)

Variance answers: if a Dog is an Animal, is a collection of Dogs a collection of Animals? For generic interfaces and delegates, C# allows it when it is safe. A type parameter marked `out` is **covariant**: it appears only in output positions, so `IEnumerable<Dog>` can be used as `IEnumerable<Animal>`. A type parameter marked `in` is **contravariant**: it appears only as input, so an `Action<Animal>` can be used where an `Action<Dog>` is needed. Classes such as `List<T>` are **invariant**, because T is used for both input and output.

VARIANCE IN ACTION

Program.cs

```
IEnumerable<Dog> dogs = new List<Dog> { new("Bruno"), new("Rocky") };
IEnumerable<Animal> animals = dogs;                  // covariance: IEnumerable<out T>
foreach (Animal a in animals) Console.Write($"{a.Name} ");
Console.WriteLine();

Action<Animal> greetAny = a => Console.WriteLine($"hello {a.Name}");
Action<Dog> greetDog = greetAny;                     // contravariance: Action<in T>
greetDog(new Dog("Tommy"));

Func<Dog> makeDog = () => new Dog("Puppy");
Func<Animal> makeAnimal = makeDog;                   // Func<out TResult> is covariant
Console.WriteLine(makeAnimal().Name);

// List<Animal> list = new List<Dog>();              // error CS0029: List<T> is invariant
// otherwise list.Add(new Cat(...)) would put a Cat into a list of Dogs

IProducer<Animal> producer = new DogShelter();       // our own covariant interface
Console.WriteLine(producer.Produce().Name);

record Animal(string Name);
record Dog(string Name) : Animal(Name);
interface IProducer<out T> { T Produce(); }          // T only returned: may be 'out'
class DogShelter : IProducer<Dog> { public Dog Produce() => new("Shelter dog"); }
```

OUTPUT

```
Bruno Rocky
hello Tommy
Puppy
Shelter dog
```

	Covariant `out T`	Contravariant `in T`	Invariant `T`
T used as	return values only	parameters only	both
Allows	Producer<Dog> as Producer<Animal>	Consumer<Animal> as Consumer<Dog>	exact type only
Examples	IEnumerable<out T>, IReadOnlyList<out T>, Func<out TResult>	IComparer<in T>, Action<in T>, IEqualityComparer<in T>	List<T>, IList<T>, Dictionary

14.1.7 Generic math with static abstract interface members (overview)

Module 11 introduced static abstract interface members. .NET uses them for **generic math**: interfaces such as `INumber<T>`, `IAdditionOperators<T,T,T>` and `IFloatingPoint<T>` are implemented by every numeric type, so one generic algorithm can use operators and constants on T.

ONE ALGORITHM FOR EVERY NUMERIC TYPE

Program.cs

```
using System.Numerics;

Console.WriteLine(Average(new[] { 2, 4, 9 }));              // int arithmetic
Console.WriteLine(Average(new[] { 1.5, 2.5, 4.0 }));        // double
Console.WriteLine(Average(new[] { 10.25m, 20.75m }));       // decimal
Console.WriteLine(Clamp(150, 0, 100));
Console.WriteLine(Clamp(-2.5f, 0f, 1f));

static T Average<T>(T[] values) where T : INumber<T>
{
    T sum = T.Zero;
    foreach (T v in values) sum += v;
    return sum / T.CreateChecked(values.Length);           // convert the int count to T
}

static T Clamp<T>(T value, T min, T max) where T : INumber<T> => T.Max(min, T.Min(value, max));
```

OUTPUT

```
5
2.6666666666666665
15.50
100
0
```

NOTE

With int, the average 15/3 = 5 uses integer division, exactly as the int type would; with double the full fraction is kept. Generic math is mainly for library authors; everyday code rarely needs it.

14.2 Non-generic collections (legacy)

14.2.1 ArrayList and Hashtable and why to avoid them

The namespace `System.Collections` still contains the C# 1 collections: `ArrayList` (a resizable list of object), `Hashtable` (a dictionary of object to object), and non-generic `Queue`, `Stack` and `SortedList`. They exist only for backward compatibility. Each element is stored as object, so value types are boxed, every read needs a cast, and type errors appear only at run time. Always use the generic equivalents from `System.Collections.Generic`.

WHY ARRAYLIST AND HASHTABLE ARE LEGACY

Program.cs

```
using System.Collections;

var list = new ArrayList { 10, 20, "thirty" };          // anything goes
int total = 0;
try
{
    foreach (object o in list) total += (int)o;         // cast each element
}
catch (InvalidCastException) { Console.WriteLine($"InvalidCastException after adding up {total}"); }

var table = new Hashtable { ["Asha"] = 91, [42] = "a number key" };
Console.WriteLine($"{table["Asha"]} {table[42]} {table["missing"] ?? "null (no exception)"}");

var modern = new List<int> { 10, 20 };                  // generic replacement
// modern.Add("thirty");                                // compile-time error instead
```

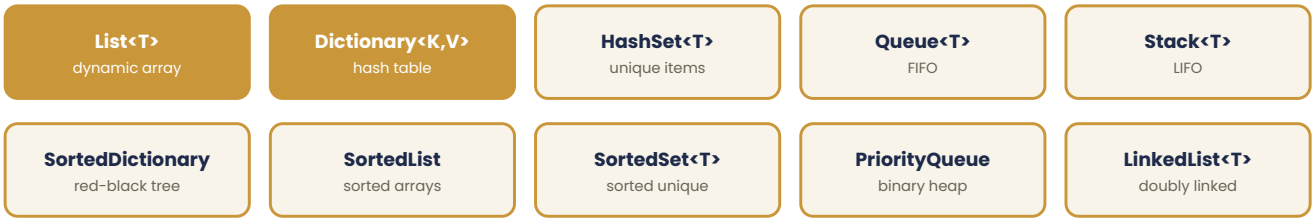
OUTPUT

```
InvalidCastException after adding up 30
91 a number key null (no exception)
```

Legacy (avoid)	Generic replacement
`ArrayList`	`List<T>`
`Hashtable`	`Dictionary<TKey, TValue>`
`Queue`, `Stack`	`Queue<T>`, `Stack<T>`
`SortedList`	`SortedList<TKey, TValue>`
`CollectionBase`	`Collection<T>`

14.3 Generic collections

The namespace `System.Collections.Generic` (imported automatically in modern projects) contains the everyday collections. Each one is a different data structure with different strengths, which section 14.9 summarises.



THE MAIN GENERIC COLLECTIONS AND THE DATA STRUCTURE BEHIND EACH

14.3.1 List<T>: Add, Insert, Remove, Contains, IndexOf, Sort, Find, FindAll, ForEach, capacity

`List<T>` is a **dynamic array**: elements are stored contiguously, accessed by index in O(1), and the list grows automatically. It is the default choice for an ordered group of items. Adding at the end is amortised O(1); inserting or removing in the middle shifts the following elements, O(n); searching with Contains or IndexOf is O(n).

WORKING WITH LIST<T>

Program.cs

```
var fruits = new List<string> { "mango", "apple" };
fruits.Add("kiwi");                                  // at the end
fruits.Insert(1, "banana");                          // at an index
fruits.AddRange(new[] { "fig", "grape" });
Console.WriteLine(string.Join(" ", fruits));

Console.WriteLine($"Remove apple: {fruits.Remove("apple")}, Remove plum: {fruits.Remove("plum")}");
fruits.RemoveAt(0);                                  // removes "mango"
Console.WriteLine($"{string.Join(" ", fruits)} | count {fruits.Count}, [1] = {fruits[1]}");
Console.WriteLine($"Contains fig {fruits.Contains("fig")}, IndexOf kiwi {fruits.IndexOf("kiwi")}");

fruits.Sort();
Console.WriteLine($"sorted: {string.Join(" ", fruits)}");
Console.WriteLine($"Find g*: {fruits.Find(f => f.StartsWith('g'))}, FindAll short: {string.Join(",",
fruits.FindAll(f => f.Length <= 4))}");
Console.WriteLine($"FindIndex k*: {fruits.FindIndex(f => f[0] == 'k')}, Exists x*: {fruits.Exists(f =>
f[0] == 'x')}");
int removed = fruits.RemoveAll(f => f.Contains('i'));        // fig, kiwi
Console.Write($"RemoveAll i: {removed} -> ");
fruits.ForEach(f => Console.Write(f.ToUpper() + " "));
Console.WriteLine();
```

OUTPUT

```
mango banana apple kiwi fig grape
Remove apple: True, Remove plum: False
banana kiwi fig grape | count 4, [1] = kiwi
Contains fig True, IndexOf kiwi 1
sorted: banana fig grape kiwi
Find g*: grape, FindAll short: fig,kiwi
FindIndex k*: 3, Exists x*: False
RemoveAll i: 2 -> BANANA GRAPE
```

Count is the number of elements; **Capacity** is the size of the internal array. When Count reaches Capacity, the list allocates an array twice as large and copies the elements. If you know the final size, pass it to the constructor to avoid repeated resizing; `TrimExcess` shrinks the array afterwards.

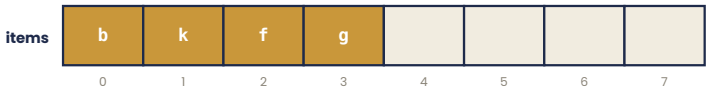
CAPACITY DOUBLES AS THE LIST GROWS

Program.cs

```
var nums = new List<int>();
int lastCapacity = -1;
for (int i = 1; i <= 20; i++)
{
    nums.Add(i);
    if (nums.Capacity != lastCapacity)
    {
        Console.WriteLine($"Count {nums.Count,2} -> Capacity {nums.Capacity}");
        lastCapacity = nums.Capacity;
    }
}
nums.TrimExcess();
Console.WriteLine($"after TrimExcess: Capacity {nums.Capacity}");
var sized = new List<int>(1000);                    // pre-allocated: no growth while adding 1000 items
Console.WriteLine($"pre-sized: Count {sized.Count}, Capacity {sized.Capacity}");
```

OUTPUT

```
Count  1 -> Capacity 4
Count  5 -> Capacity 8
Count  9 -> Capacity 16
Count 17 -> Capacity 32
after TrimExcess: Capacity 20
pre-sized: Count 0, Capacity 1000
```



INTERNALLY A LIST<T> IS AN ARRAY WITH SPARE CAPACITY (COUNT 4, CAPACITY 8)

14.3.2 Dictionary<TKey, TValue>: Add, TryGetValue, ContainsKey, Remove, iterating KeyValuePair

A `Dictionary<TKey, TValue>` maps unique **keys** to values using a **hash table**. Lookup, insertion and removal by key are O(1) on average, however many entries there are. Keys must be unique and should not change while in the dictionary; their `GetHashCode` and `Equals` decide equality (a custom comparer such as `StringComparer.OrdinalIgnoreCase` can be passed to the constructor).

DICTIONARY BASICS Program.cs

```
var marks = new Dictionary<string, int>
{
    ["Asha"] = 91,
    ["Ravi"] = 78
};
marks.Add("Meera", 85);                          // Add throws if the key exists
try { marks.Add("Asha", 50); }
catch (ArgumentException e) { Console.WriteLine(e.Message); }

marks["Ravi"] = 80;                              // indexer: update (or add if missing)
Console.WriteLine($"TryAdd Asha: {marks.TryAdd("Asha", 10)}, TryAdd Kiran: {marks.TryAdd("Kiran", 66)}");

if (marks.TryGetValue("Meera", out int m)) Console.WriteLine($"Meera -> {m}");   // one lookup
Console.WriteLine($"ContainsKey Zoya {marks.ContainsKey("Zoya")}, GetValueOrDefault Zoya
{marks.GetValueOrDefault("Zoya", -1)}");
try { _ = marks["Zoya"]; } catch (KeyNotFoundException) { Console.WriteLine("indexer read of a missing
key throws"); }

foreach (KeyValuePair<string, int> pair in marks) Console.Write($"{pair.Key}={pair.Value} ");
Console.WriteLine();
Console.WriteLine($"Remove Ravi: {marks.Remove("Ravi")}, count {marks.Count}");
foreach (var (name, score) in marks) Console.Write($"{name}:{score} ");           // deconstructed pairs
Console.WriteLine();
Console.WriteLine($"keys: {string.Join(",", marks.Keys)}  values: {string.Join(",", marks.Values)}");

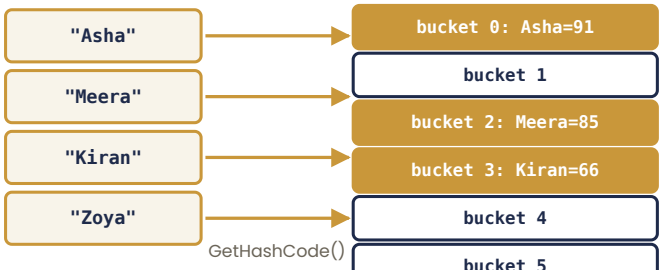
var ci = new Dictionary<string, string>(StringComparer.OrdinalIgnoreCase) { ["IN"] = "India" };
Console.WriteLine(ci["in"]);                                                      // case-insensitive
keys
```

OUTPUT

```
An item with the same key has already been added. Key: Asha
TryAdd Asha: False, TryAdd Kiran: True
Meera -> 85
ContainsKey Zoya False, GetValueOrDefault Zoya -1
indexer read of a missing key throws
Asha=91 Ravi=80 Meera=85 Kiran=66
Remove Ravi: True, count 3
Asha:91 Meera:85 Kiran:66
keys: Asha,Meera,Kiran  values: 91,85,66
India
```

NOTE

Enumeration currently follows insertion order as long as nothing has been removed, but the documentation does not guarantee any order: after removals, new entries may fill the gaps. When order matters, use SortedDictionary, or sort the keys when printing.



hash code → bucket index → entry

O(1) average for lookup, add, remove

collisions share a bucket chain

A HASH TABLE FINDS THE BUCKET FROM THE KEY'S HASH CODE (BUCKET NUMBERS ARE ILLUSTRATIVE)

14.3.3 HashSet<T> and set operations

A `HashSet<T>` stores **unique** items with O(1) Add, Remove and Contains. Adding an item that is already present simply returns false. It supports mathematical set operations that modify the set in place (UnionWith, IntersectWith, ExceptWith, SymmetricExceptWith) and comparisons (IsSubsetOf, Overlaps, SetEquals).

SET OPERATIONS Program.cs

```
var python = new HashSet<string> { "Asha", "Ravi", "Meera", "Kiran" };
var csharp = new HashSet<string> { "Ravi", "Kiran", "Zoya" };
Console.WriteLine($"Add Asha again: {python.Add("Asha")}, count {python.Count}");

static string Show(IEnumerable<string> s) => "{" + string.Join(", ", s.Order()) + "}";

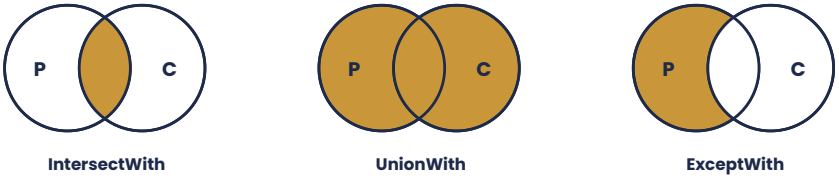
var both = new HashSet<string>(python); both.IntersectWith(csharp);
var either = new HashSet<string>(python); either.UnionWith(csharp);
var onlyPython = new HashSet<string>(python); onlyPython.ExceptWith(csharp);
var exactlyOne = new HashSet<string>(python); exactlyOne.SymmetricExceptWith(csharp);

Console.WriteLine($"both:        {Show(both)}");
Console.WriteLine($"either:      {Show(either)}");
Console.WriteLine($"only python: {Show(onlyPython)}");
Console.WriteLine($"exactly one: {Show(exactlyOne)}");
Console.WriteLine($"both is subset of csharp: {both.IsSubsetOf(csharp)}, overlap:
{python.Overlaps(csharp)}");

int[] withDuplicates = { 4, 2, 4, 9, 2, 7 };
Console.WriteLine($"distinct count: {new HashSet<int>(withDuplicates).Count}");
```

OUTPUT

```
Add Asha again: False, count 4
both:        {Kiran, Ravi}
either:      {Asha, Kiran, Meera, Ravi, Zoya}
only python: {Asha, Meera}
exactly one: {Asha, Meera, Zoya}
both is subset of csharp: True, overlap: True
distinct count: 4
```



THREE OF THE SET OPERATIONS (SHADED: THE RESULT OF P OP C)

14.3.4 SortedList, SortedDictionary, SortedSet

The sorted collections keep their contents **ordered by key** at all times, so enumeration is always in sorted order and minimum and maximum are easy to get. `SortedDictionary<K,V>` is a balanced binary search tree

(red-black tree): O(log n) insert, remove and lookup. `SortedList<K,V>` stores two sorted arrays: lookups are O(log n) by binary search and it uses less memory and allows access by index, but inserting in the middle is O(n). `SortedSet<T>` is a sorted set of unique items, also a red-black tree, with range queries.

SORTED COLLECTIONS Program.cs

```
var rank = new SortedDictionary<string, int> { ["Ravi"] = 80, ["Asha"] = 91, ["Meera"] = 85 };
rank["Kiran"] = 66;
foreach (var (name, score) in rank) Console.Write($"{name}:{score} ");            // always sorted by
key
Console.WriteLine();

var byYear = new SortedList<int, string> { [2002] = "C# 1.0", [2023] = "C# 12", [2005] = "C# 2.0", [2025]
= "C# 14" };
Console.WriteLine($"first: {byYear.Keys[0]} {byYear.Values[0]}, last: {byYear.Keys[^1]}
{byYear.Values[^1]}");
Console.WriteLine($"index of 2023: {byYear.IndexOfKey(2023)}");

var scores = new SortedSet<int> { 72, 45, 91, 60, 88, 45, 30 };
Console.WriteLine($"set: {string.Join(" ", scores)}  min {scores.Min}, max {scores.Max}");
Console.WriteLine($"between 50 and 90: {string.Join(" ", scores.GetViewBetween(50, 90))}");
Console.WriteLine($"descending: {string.Join(" ", scores.Reverse())}");
```

OUTPUT

```
Asha:91 Kiran:66 Meera:85 Ravi:80
first: 2002 C# 1.0, last: 2025 C# 14
index of 2023: 2
set: 30 45 60 72 88 91  min 30, max 91
between 50 and 90: 60 72 88
descending: 91 88 72 60 45 30
```

14.3.5 Queue<T>, Stack<T>, PriorityQueue<TElement, TPriority>

A `Queue<T>` is **first in, first out** (FIFO): Enqueue adds at the back, Dequeue removes from the front, like a line at a ticket counter. A `Stack<T>` is **last in, first out** (LIFO): Push and Pop work at the top, like a stack of plates; undo histories and expression evaluation use stacks. A `PriorityQueue<TElement, TPriority>` (.NET 6) always dequeues the element with the **lowest priority value** first; it is a binary min-heap with O(log n) Enqueue and Dequeue. All three operate in O(1) or O(log n) and offer TryDequeue, TryPop and Peek.

QUEUE, STACK AND PRIORITYQUEUE

Program.cs

```
var line = new Queue<string>();
line.Enqueue("Asha"); line.Enqueue("Ravi"); line.Enqueue("Meera");
Console.WriteLine($"next: {line.Peek()}, served: {line.Dequeue()}, {line.Dequeue()}, waiting:
{line.Count}");

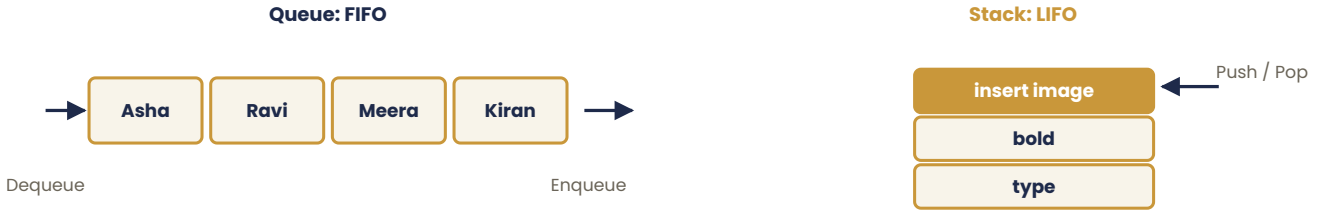
var undo = new Stack<string>();
undo.Push("type 'Hello'"); undo.Push("bold"); undo.Push("insert image");
Console.WriteLine($"undo: {undo.Pop()}, then {undo.Pop()}, top now: {undo.Peek()}");
Console.WriteLine($"balanced (a[b]{{c}}): {Balanced("(a[b]{c})")}, (]: {Balanced("(]")}");

var er = new PriorityQueue<string, int>();          // lower number = more urgent
er.Enqueue("sprained ankle", 3);
er.Enqueue("heart attack", 1);
er.Enqueue("common cold", 5);
er.Enqueue("broken arm", 2);
while (er.TryDequeue(out string? patient, out int priority))
    Console.Write($"[{priority}] {patient}  ");
Console.WriteLine();

static bool Balanced(string s)                       // classic stack problem
{
    var open = new Stack<char>();
    foreach (char c in s)
    {
        if ("([{".Contains(c)) open.Push(c);
        else if (")]}".Contains(c))
        {
            if (open.Count == 0) return false;
            char o = open.Pop();
            if ("([{".IndexOf(o) != ")]}".IndexOf(c)) return false;
        }
    }
    return open.Count == 0;
}
```

OUTPUT

```
next: Asha, served: Asha, Ravi, waiting: 1
undo: insert image, then bold, top now: type 'Hello'
balanced (a[b]{c}): True, (]: False
[1] heart attack  [2] broken arm  [3] sprained ankle  [5] common cold
```



A QUEUE REMOVES FROM THE FRONT; A STACK REMOVES FROM THE TOP

14.3.6 LinkedList<T>

`LinkedList<T>` is a **doubly linked list**: each element sits in a `LinkedListNode<T>` that points to the previous and next nodes. Given a node, inserting or removing next to it is O(1), with no shifting; but there is no indexer, and finding an element is O(n). In practice List<T> is faster for most workloads because of memory locality; LinkedList suits cases with many insertions and removals in the middle while holding node references, such as an LRU cache.

A DOUBLY LINKED LIST

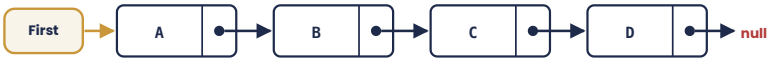
Program.cs

```
var playlist = new LinkedList<string>();
playlist.AddLast("Song B");
playlist.AddFirst("Song A");
LinkedListNode<string> c = playlist.AddLast("Song C");
playlist.AddBefore(c, "Ad break");                  // O(1) given the node
playlist.AddAfter(c, "Song D");
Console.WriteLine(string.Join(" <-> ", playlist));

playlist.Remove("Ad break");                        // O(n) search, then O(1) unlink
Console.WriteLine($"first {playlist.First!.Value}, last {playlist.Last!.Value}, count {playlist.Count}");
Console.Write("backwards: ");
for (var node = playlist.Last; node is not null; node = node.Previous) Console.Write($"{node.Value}; ");
Console.WriteLine();
```

OUTPUT

```
Song A <-> Song B <-> Ad break <-> Song C <-> Song D
first Song A, last Song D, count 4
backwards: Song D; Song C; Song B; Song A;
```

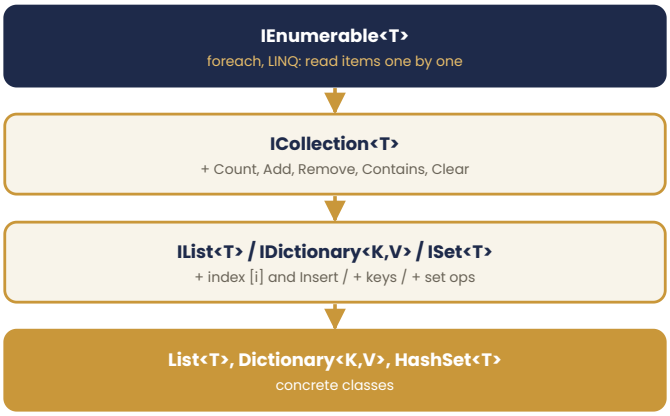


NODES LINKED IN BOTH DIRECTIONS (ONLY FORWARD LINKS DRAWN)

14.4 Collection interfaces

The collections implement a family of interfaces. Methods should **accept** the most general interface they need (so callers can pass any collection) and **return** an interface that exposes exactly what callers may do.

14.4.1 IEnumerable<T>, ICollection<T>, IList<T>, IDictionary<TKey, TValue>



THE MAIN GENERIC COLLECTION INTERFACES, FROM MOST GENERAL TO CONCRETE

Interface	Adds	Implemented by
`IEnumerable<T>`	GetEnumerator: can be iterated	every collection, arrays, LINQ results, iterators
`ICollection<T>`	Count, Add, Remove, Contains, Clear, CopyTo	List, HashSet, LinkedList, Dictionary (of pairs)
`IList<T>`	indexer, IndexOf, Insert, RemoveAt	List<T>, arrays
`IDictionary<K,V>`	key indexer, Keys, Values, TryGetValue	Dictionary, SortedDictionary, SortedList
`ISet<T>`	set operations	HashSet, SortedSet

PROGRAM TO INTERFACES Program.cs

```
int[] array = { 5, 3, 8 };
var list = new List<int> { 1, 2 };
var set = new HashSet<int> { 7, 7, 9 };
Console.WriteLine($"{Sum(array)} {Sum(list)} {Sum(set)} {Sum(Enumerable.Range(1, 4))}");   // anything
enumerable

AddDefaults(list);
AddDefaults(set);
Console.WriteLine($"{string.Join(",", list)} | {string.Join(",", set.Order())}");

static int Sum(IEnumerable<int> items)            // the most general input type
{
    int total = 0;
    foreach (int i in items) total += i;
    return total;
}

static void AddDefaults(ICollection<int> target)  // needs Add: ICollection is enough
{
    target.Add(0);
    target.Add(100);
}
```

OUTPUT

```
16 3 16 10
1,2,0,100 | 0,7,9,100
```

14.4.2 IReadOnlyList<T>, IReadOnlyDictionary<TKey, TValue>

When a class exposes its internal collection, returning `List<T>` lets callers add or delete items behind the class's back, breaking encapsulation. Returning `IReadOnlyList<T>`, `IReadOnlyCollection<T>` or `IReadOnlyDictionary<K,V>` gives read access (indexer, Count, enumeration) without mutation methods. Note that read-only views are not immutable: the owner can still change the underlying collection, and callers see the change.

EXPOSING READ-ONLY VIEWS

Program.cs

```
var course = new Course();
course.Enrol("Asha"); course.Enrol("Ravi");
IReadOnlyList<string> view = course.Students;
Console.WriteLine($"{view.Count} students, first {view[0]}");
// view.Add("Hacker");                          // error CS1061: IReadOnlyList has no Add
course.Enrol("Meera");
Console.WriteLine($"view now sees {view.Count}");     // a live view, not a copy
Console.WriteLine($"Asha's grade: {course.Grades["Asha"]}");

class Course
{
    private readonly List<string> students = new();
    private readonly Dictionary<string, char> grades = new();

    public IReadOnlyList<string> Students => students;                  // read-only view
    public IReadOnlyDictionary<string, char> Grades => grades;

    public void Enrol(string name)
    {
        students.Add(name);
        grades[name] = 'B';
    }
}
```

OUTPUT

```
2 students, first Asha
view now sees 3
Asha's grade: B
```

14.5 Specialised collections

14.5.1 Immutable collections (System.Collections.Immutable)

An **immutable collection** can never change after it is created. "Modifying" methods such as `Add` or `Remove` return a **new** collection and leave the original intact, sharing most of the internal structure so that copies are cheap. Immutable collections are inherently thread-safe and ideal for snapshots, configuration and functional-style code. For building one in many steps, use a **builder** and then freeze it.

IMMUTABLE COLLECTIONS

Program.cs

```
using System.Collections.Immutable;

ImmutableList<string> v1 = ImmutableList.Create("Asha", "Ravi");
ImmutableList<string> v2 = v1.Add("Meera");             // new list; v1 unchanged
ImmutableList<string> v3 = v2.Remove("Asha").SetItem(0, "RAVI");
Console.WriteLine($"v1: {string.Join(",", v1)} | v2: {string.Join(",", v2)} | v3: {string.Join(",",
v3)}");

var prices = ImmutableDictionary<string, decimal>.Empty.Add("pen", 10m).Add("book", 250m);
var updated = prices.SetItem("pen", 12m);
Console.WriteLine($"old pen {prices["pen"]}, new pen {updated["pen"]}");

ImmutableArray<int>.Builder builder = ImmutableArray.CreateBuilder<int>();
for (int i = 1; i <= 5; i++) builder.Add(i * i);       // mutable while building
ImmutableArray<int> squares = builder.ToImmutable();   // then frozen
Console.WriteLine(string.Join(" ", squares));

ImmutableHashSet<int> set = [3, 1, 2];                  // collection expressions work too
Console.WriteLine($"{set.Count} {set.Contains(2)}");
```

OUTPUT

```
v1: Asha,Ravi | v2: Asha,Ravi,Meera | v3: RAVI,Meera
old pen 10, new pen 12
1 4 9 16 25
3 True
```

Type	Mutable counterpart	Notes
`ImmutableList<T>`	List<T>	balanced tree: O(log n) index and update
`ImmutableArray<T>`	T[]	fastest reads; every change copies the array
`ImmutableDictionary<K,V>`	Dictionary	O(log n); use ImmutableSortedDictionary for ordering
`ImmutableHashSet<T>`	HashSet	
`ImmutableQueue<T>`, `ImmutableStack<T>`	Queue, Stack	persistent linked structures

14.5.2 Frozen collections (FrozenDictionary, FrozenSet)

.NET 8 added **frozen collections** in `System.Collections.Frozen`. They are created once from existing data with `ToFrozenDictionary()` or `ToFrozenSet()`; the creation step analyses the keys and builds an optimised layout, which costs more up front, but every lookup afterwards is faster than with a normal Dictionary. They are read-only. Use them for lookup tables that are built at start-up and read millions of times: country codes, keywords, routing tables, feature flags.

FROZEN LOOKUP TABLES

Program.cs

```
using System.Collections.Frozen;

var source = new Dictionary<string, string>
{
    ["IN"] = "India", ["US"] = "United States", ["JP"] = "Japan", ["DE"] = "Germany"
};
FrozenDictionary<string, string> countries = source.ToFrozenDictionary(StringComparer.OrdinalIgnoreCase);
Console.WriteLine($"{countries["in"]}, {countries.Count} entries, has FR:
{countries.ContainsKey("FR")}");

FrozenSet<string> keywords = new[] { "class", "struct", "record", "interface", "enum" }.ToFrozenSet();
foreach (string word in new[] { "record", "banana" })
    Console.WriteLine($"{word}: keyword = {keywords.Contains(word)}");
// countries.Add("FR", "France");          // not available: frozen collections are read-only
```

OUTPUT

```
India, 4 entries, has FR: False
record: keyword = True
banana: keyword = False
```

	Dictionary	ImmutableDictionary	FrozenDictionary
Can change	yes	returns modified copies	no
Creation cost	low	medium	high (optimises once)
Lookup speed	fast	slower (tree)	fastest
Best for	general use	snapshots that evolve over time	build-once, read-many tables

14.5.3 Concurrent collections (detailed in Module 18)

The standard collections are **not thread-safe**: if two threads add to a List or Dictionary at the same time, the data can be corrupted. `System.Collections.Concurrent` provides collections designed for simultaneous access from many threads without explicit locks: `ConcurrentDictionary<K,V>`, `ConcurrentQueue<T>`, `ConcurrentStack<T>`, `ConcurrentBag<T>` and `BlockingCollection<T>` for producer-consumer pipelines. Module 18 covers them with threads and tasks.

A THREAD-SAFE DICTIONARY UNDER PARALLEL UPDATES

Program.cs

```
using System.Collections.Concurrent;

var wordCounts = new ConcurrentDictionary<string, int>();
string[] words = { "apple", "kiwi", "apple", "mango", "kiwi", "apple" };

Parallel.For(0, 1000, i =>                                  // many threads at once
{
    string w = words[i % words.Length];
    wordCounts.AddOrUpdate(w, 1, (_, old) => old + 1);      // atomic insert-or-update
});
foreach (var (w, c) in wordCounts.OrderBy(p => p.Key)) Console.Write($"{w}={c} ");
Console.WriteLine();

var queue = new ConcurrentQueue<int>();
Parallel.For(0, 100, i => queue.Enqueue(i));
Console.WriteLine($"queued {queue.Count}, TryDequeue succeeded: {queue.TryDequeue(out _)}");
```

OUTPUT

```
apple=500 kiwi=333 mango=167
queued 100, TryDequeue succeeded: True
```

NOTE

With a plain Dictionary and `counts[w]++` from many threads, the totals would usually come out wrong or the dictionary would throw. The counts above are exact: 1000 iterations over 6 words give apple 3 × 167 − 1 = 500, kiwi 333 and mango 167.

14.6 Sorting and comparing

14.6.1 IComparable<T> and IComparer<T>

Module 11 introduced both interfaces. To summarise: `IComparable<T>` is implemented **by the type itself** and defines its single natural order (numbers by value, strings alphabetically, employees by id). `IComparer<T>` is a **separate object** that defines an alternative order, and any number of them can exist. `Comparer<T>.Create` turns a lambda into an IComparer, which sorted collections accept in their constructors.

NATURAL ORDER AND ALTERNATIVE ORDERS

Program.cs

```
var books = new List<Book> { new("Clean Code", 2008, 464), new("C# in Depth", 2019, 528),
new("Refactoring", 1999, 448) };

books.Sort();                                                  // natural order: IComparable (by year)
Console.WriteLine(string.Join(" | ", books));

books.Sort(new ByPages());                                     // an IComparer class
Console.WriteLine(string.Join(" | ", books.Select(b => $"{b.Title} {b.Pages}p")));

var byTitle = new SortedSet<Book>(Comparer<Book>.Create((a, b) => string.Compare(a.Title, b.Title,
StringComparison.Ordinal)));
foreach (var b in books) byTitle.Add(b);
Console.WriteLine(string.Join(" | ", byTitle.Select(b => b.Title)));

record Book(string Title, int Year, int Pages) : IComparable<Book>
{
    public int CompareTo(Book? other) => other is null ? 1 : Year.CompareTo(other.Year);
    public override string ToString() => $"{Title} ({Year})";
}

class ByPages : IComparer<Book>
{
    public int Compare(Book? x, Book? y) => (x?.Pages ?? 0).CompareTo(y?.Pages ?? 0);
}
```

OUTPUT

```
Refactoring (1999) | Clean Code (2008) | C# in Depth (2019)
Refactoring 448p | Clean Code 464p | C# in Depth 528p
C# in Depth | Clean Code | Refactoring
```

14.6.2 Sorting with lambdas and Comparison<T>

`List<T>.Sort` and `Array.Sort` also accept a `Comparison<T>` delegate, usually written as a lambda `(a, b) => ...` returning negative, zero or positive. Reverse the operands for descending order, and chain a second comparison for ties. Be aware that `List.Sort` and `Array.Sort` are **unstable** (equal elements may change their relative order); LINQ's `OrderBy` / `ThenBy` is stable and does not modify the source.

SORTING WITH LAMBDAS Program.cs

```
var people = new List<(string Name, string City, int Age)>
{
    ("Ravi", "Pune", 30), ("Asha", "Delhi", 25), ("Meera", "Pune", 25), ("Kiran", "Delhi", 30), ("Zoya",
"Pune", 22)
};

people.Sort((a, b) => a.Age.CompareTo(b.Age));                          // ascending by age (ties: no
guaranteed order)
Console.WriteLine("ages:        " + string.Join(", ", people.Select(p => p.Age)));

people.Sort((a, b) => b.Age.CompareTo(a.Age) != 0 ? b.Age.CompareTo(a.Age)        // age descending
                                                  : string.Compare(a.Name, b.Name,
StringComparison.Ordinal));   // then name
Console.WriteLine("age desc, name: " + string.Join(", ", people.Select(p => $"{p.Name}{p.Age}")));

var ordered = people.OrderBy(p => p.City).ThenBy(p => p.Name);       // LINQ: stable, returns a new
sequence
Console.WriteLine("city, name:  " + string.Join(", ", ordered.Select(p => $"{p.City}:{p.Name}")));

string[] words = { "banana", "Fig", "apple", "Cherry" };
Array.Sort(words, StringComparer.OrdinalIgnoreCase);
Console.WriteLine(string.Join(" ", words));
```

OUTPUT

```
ages:        22, 25, 25, 30, 30
age desc, name: Kiran30, Ravi30, Asha25, Meera25, Zoya22
city, name:  Delhi:Asha, Delhi:Kiran, Pune:Meera, Pune:Ravi, Pune:Zoya
apple banana Cherry Fig
```

14.7 Iterators

14.7.1 IEnumerable<T> and IEnumerator<T>

`foreach` works with any type that has a `GetEnumerator()` method, normally by implementing `IEnumerable<T>`. The enumerator (`IEnumerator<T>`) is a cursor with `MoveNext()` (advance; returns false at the end), `Current` (the element at the cursor) and `Dispose()`. The compiler translates every foreach into this pattern.

FOREACH IS BUILT ON IENUMERATOR

Program.cs

```
var colours = new List<string> { "red", "green", "blue" };

foreach (string c in colours) Console.Write($"{c} ");     // what you write
Console.WriteLine();

using (IEnumerator<string> e = colours.GetEnumerator())   // what the compiler generates
{
    while (e.MoveNext())
    {
        string c = e.Current;
        Console.Write($"[{c}] ");
    }
}
Console.WriteLine();

try
{
    foreach (string c in colours) if (c == "green") colours.Remove(c);   // modifying during foreach
}
catch (InvalidOperationException ex) { Console.WriteLine(ex.Message); }
```

OUTPUT

```
red green blue
[red] [green] [blue]
Collection was modified; enumeration operation may not execute.
```

14.7.2 yield return and yield break

Writing an enumerator class by hand is tedious. An **iterator** method returns `IEnumerable<T>` and uses `yield return value;` to produce elements one at a time; the compiler generates the enumerator state machine. `yield break;` ends the sequence early. Iterators are **lazy**: the method body runs only as the caller asks for the next element, and pauses at each yield. This allows infinite sequences and processing of huge data without holding it all in memory.

ITERATORS WITH YIELD

Program.cs

```
foreach (int n in Evens(10)) Console.Write($"{n} ");
Console.WriteLine();

foreach (int f in Fibonacci())                // infinite sequence: stop when we have enough
{
    if (f > 100) break;
    Console.Write($"{f} ");
}
Console.WriteLine();

Console.WriteLine(string.Join(",", ReadUntilBlank(new[] { "a", "b", "", "c" })));

IEnumerable<int> lazy = Traced(3);
Console.WriteLine("iterator created, nothing has run yet");
foreach (int x in lazy) Console.WriteLine($"  consumer got {x}");

static IEnumerable<int> Evens(int max)
{
    for (int i = 0; i <= max; i += 2) yield return i;
}

static IEnumerable<long> Fibonacci()
{
    long a = 0, b = 1;
    while (true)
```

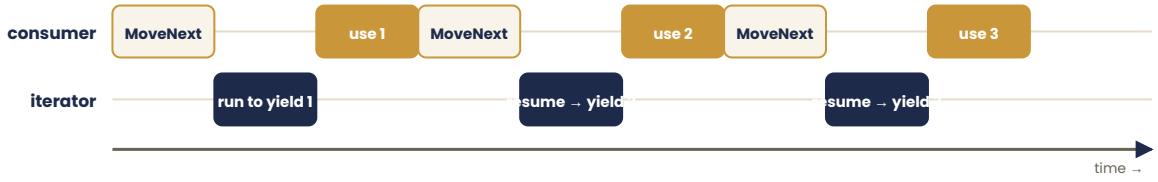
```
    {
        yield return a;
        (a, b) = (b, a + b);
    }
}

static IEnumerable<string> ReadUntilBlank(string[] lines)
{
    foreach (string line in lines)
    {
        if (line.Length == 0) yield break;    // stop the sequence
        yield return line.ToUpper();
    }
}

static IEnumerable<int> Traced(int count)
{
    for (int i = 1; i <= count; i++)
    {
        Console.WriteLine($"  producing {i}");
        yield return i;                       // pause here until the next MoveNext
    }
    Console.WriteLine("  done producing");
}
```

OUTPUT

```
0 2 4 6 8 10
0 1 1 2 3 5 8 13 21 34 55 89
A,B
iterator created, nothing has run yet
  producing 1
  consumer got 1
  producing 2
  consumer got 2
  producing 3
  consumer got 3
  done producing
```



EXECUTION ALTERNATES BETWEEN THE CONSUMER AND THE PAUSED ITERATOR

NOTE

Iterator methods cannot have ref or out parameters, and argument checks inside them run only on the first MoveNext. To validate eagerly, put the checks in a normal method that then calls a local iterator function.

14.8 Collection expressions and spread `..` (C# 12)

A **collection expression** creates a collection with square brackets: `[1, 2, 3]`. The same syntax works for arrays, `List<T>`, `Span<T>`, `HashSet<T>`, immutable collections, interfaces such as `IEnumerable<T>` and any type marked with the CollectionBuilder attribute; the target type decides what is built. The **spread element** `..source` inserts all elements of another collection. `[]` is an empty collection of the target type.

COLLECTION EXPRESSIONS WITH SPREAD

Program.cs

```
int[] a = [1, 2, 3];
List<int> b = [4, 5];
HashSet<string> tags = ["csharp", "dotnet", "csharp"];        // duplicates removed by the set
IEnumerable<int> seq = [7, 8];                                // compiler picks a suitable type
int[] empty = [];

int[] all = [0, .. a, .. b, 6, .. seq];                       // spread several sources
Console.WriteLine(string.Join(" ", all));
Console.WriteLine($"tags {tags.Count}, empty {empty.Length}");

List<string> Merge(List<string> x, List<string> y) => [.. x, "--", .. y];
Console.WriteLine(string.Join(" ", Merge(["a", "b"], ["c"])));

Span<int> span = [10, 20, 30];                                // can even live on the stack
Console.WriteLine(span[^1]);
Print([3.5, 4.5]);                                            // target type from the parameter

static void Print(double[] values) => Console.WriteLine($"{values.Length} values: {string.Join(", ",
values)}");
```

OUTPUT

```
0 1 2 3 4 5 6 7 8
tags 2, empty 0
a b -- c
30
2 values: 3.5, 4.5
```

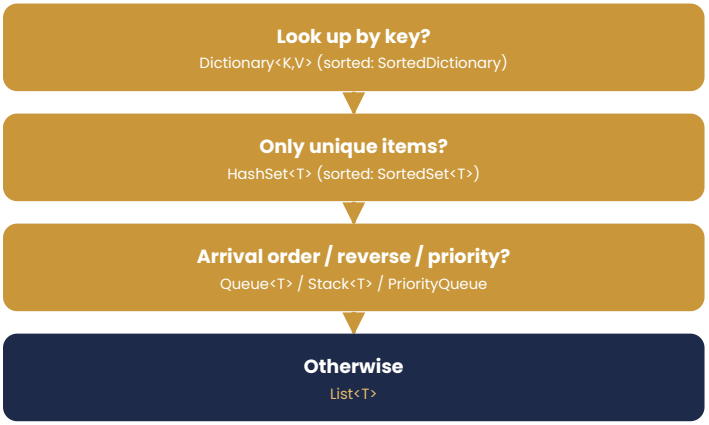
14.9 Choosing the right collection and Big-O summary

Choose a collection by the **operations you perform most**. The table gives the typical (average) costs; n is the number of elements.

Collection	Index [i]	Search by value / key	Add	Insert / remove in middle	Ordered?	Typical use
`T[]`	O(1)	O(n)	fixed size	O(n)	insertion	fixed-size data, performance
`List<T>`	O(1)	O(n)	O(1)*	O(n)	insertion	default ordered collection
`Dictionary<K,V>`	n/a	O(1)	O(1)*	O(1) by key	no	lookup by key
`HashSet<T>`	n/a	O(1)	O(1)*	O(1)	no	uniqueness, membership
`SortedDictionary`	n/a	O(log n)	O(log n)	O(log n)	by key	sorted lookups with many changes
`SortedList`	O(1)	O(log n)	O(n)	O(n)	by key	sorted, mostly read, low memory
`SortedSet<T>`	n/a	O(log n)	O(log n)	O(log n)	sorted	sorted unique items, ranges
`Queue<T>`	n/a	O(n)	O(1)*	n/a	FIFO	tasks in arrival order
`Stack<T>`	n/a	O(n)	O(1)*	n/a	LIFO	undo, backtracking

Collection	Index [i]	Search by value / key	Add	Insert / remove in middle	Ordered?	Typical use
`PriorityQueue`	n/a	n/a	O(log n)	n/a	by priority	scheduling, Dijkstra
`LinkedList<T>`	O(n)	O(n)	O(1)	O(1) at a node	insertion	frequent mid-list edits

* amortised: occasionally the internal array grows, costing O(n) for that one call.



A QUICK GUIDE TO CHOOSING A COLLECTION: ASK THE QUESTIONS FROM TOP TO BOTTOM

Generics give type-safe, reusable code without casts or boxing: generic methods (with type inference), classes and interfaces, constraints (class, struct, new(), base class, interface, notnull, unmanaged) that unlock members, default(T), out/in variance, and generic math. Avoid ArrayList and Hashtable. List<T> is the default ordered collection, Dictionary<K,V> gives O(1) lookup by key, HashSet<T> gives uniqueness and set algebra, the Sorted* collections keep order, Queue/Stack/PriorityQueue serve FIFO, LIFO and priority processing, and LinkedList suits node-based edits. Accept and return interfaces (IEnumerable, ICollection, IReadOnlyList). Use immutable, frozen and concurrent collections for snapshots, read-mostly tables and multi-threading. Sort with IComparable, IComparer or lambdas, build lazy sequences with yield, and create collections with [..] expressions.

PRACTICE CHECKPOINT

Build a generic **Repository<T>** class with constraints, a **Student Management System** using **List<T>** and **Dictionary<int, Student>**, and a custom iterator using **yield return**.

SAMPLE SOLUTION

Program.cs

```
var repo = new Repository<Student>();
repo.Add(new Student(101, "Asha", "CSE", 91));
repo.Add(new Student(102, "Ravi", "ECE", 78));
repo.Add(new Student(103, "Meera", "CSE", 85));
repo.Add(new Student(104, "Kiran", "MECH", 58));
try { repo.Add(new Student(102, "Duplicate", "CSE", 0)); }
catch (ArgumentException e) { Console.WriteLine($"Error: {e.Message}"); }

Console.WriteLine($"Find 103: {repo.GetById(103)?.Name ?? "none"}, Find 999: {repo.GetById(999)?.Name ??
"none"}");
repo.Update(new Student(102, "Ravi", "ECE", 82));
Console.WriteLine($"Removed 104: {repo.Remove(104)}, total {repo.Count}");

Console.WriteLine("All students by marks:");
foreach (Student s in repo.All().OrderByDescending(s => s.Marks))
    Console.WriteLine($"  {s.Id} {s.Name,-6} {s.Dept,-5} {s.Marks}");
```

```
var byDept = new Dictionary<string, List<Student>>();
foreach (Student s in repo.All())
{
    if (!byDept.TryGetValue(s.Dept, out var list)) byDept[s.Dept] = list = new List<Student>();
    list.Add(s);
}
foreach (var (dept, list) in byDept)
    Console.WriteLine($"{dept}: {string.Join(", ", list.Select(s => s.Name))}");

Console.Write("Toppers (iterator): ");
foreach (Student s in repo.Where(s => s.Marks >= 85)) Console.Write($"{s.Name} ");
Console.WriteLine();

public interface IEntity { int Id { get; } }

public record Student(int Id, string Name, string Dept, int Marks) : IEntity;

public class Repository<T> where T : class, IEntity            // constraints
{
    private readonly Dictionary<int, T> byId = new();           // O(1) lookup by id
    private readonly List<T> ordered = new();                    // keeps insertion order

    public int Count => ordered.Count;

    public void Add(T item)
    {
        if (!byId.TryAdd(item.Id, item)) throw new ArgumentException($"Id {item.Id} already exists");
        ordered.Add(item);
    }

    public T? GetById(int id) => byId.GetValueOrDefault(id);

    public void Update(T item)
    {
        if (!byId.ContainsKey(item.Id)) throw new KeyNotFoundException($"Id {item.Id}");
        ordered[ordered.IndexOf(byId[item.Id])] = item;
        byId[item.Id] = item;
    }

    public bool Remove(int id) => byId.Remove(id, out T? item) && ordered.Remove(item);

    public IReadOnlyList<T> All() => ordered;

    public IEnumerable<T> Where(Func<T, bool> predicate)       // custom iterator
    {
        foreach (T item in ordered)
            if (predicate(item)) yield return item;
    }
}
```

OUTPUT

```
Error: Id 102 already exists
Find 103: Meera, Find 999: none
Removed 104: True, total 3
All students by marks:
  101 Asha   CSE   91
  103 Meera  CSE   85
  102 Ravi   ECE   82
CSE: Asha, Meera
ECE: Ravi
Toppers (iterator): Asha Meera
```

MODULE 15 · INTERMEDIATE

Delegates, events and lambda expressions

Minimum duration **3 days** | Level **Intermediate**

So far, methods have been called by name. Often, though, the code that knows WHEN something should happen is not the code that knows WHAT should happen: a sort routine needs a comparison supplied by the caller, a button needs to run whatever the application decides on a click. Delegates are type-safe references to methods that make this possible. This module covers declaring and invoking delegates, multicast delegates, the built-in Action, Func and Predicate types, anonymous methods and lambdas with closures, events and the publisher-subscriber model, and a first look at expression trees.

IN THIS MODULE

15.1 Delegates
15.2 Built-in delegates
15.3 Anonymous methods and lambdas
15.4 Events
15.5 Expression trees (overview)

15.1 Delegates

A **delegate** is a type that represents a method signature: a return type and a parameter list. A variable of a delegate type can hold a reference to any method (static or instance) with a matching signature, and calling the variable calls that method. Delegates are objects, so they can be stored in fields, passed as arguments and returned from methods. They are the C# version of function pointers, but type-safe.

15.1.1 Declaring and invoking a delegate

A delegate type is declared with the `delegate` keyword followed by a signature. An instance is created by assigning a method group (the method name without parentheses). It is invoked like a method, or explicitly with `Invoke`.

A DELEGATE TYPE AND ITS INSTANCES

Program.cs

```
MathOp op = Add;                          // method group conversion
Console.WriteLine(op(6, 3));              // calls Add
op = Multiply;                            // the same variable, another method
Console.WriteLine(op.Invoke(6, 3));       // explicit Invoke
op = new Calculator(10).AddBase;          // an instance method: the delegate remembers the object
Console.WriteLine(op(6, 3));
Console.WriteLine($"target method: {op.Method.Name}, target object: {op.Target}");

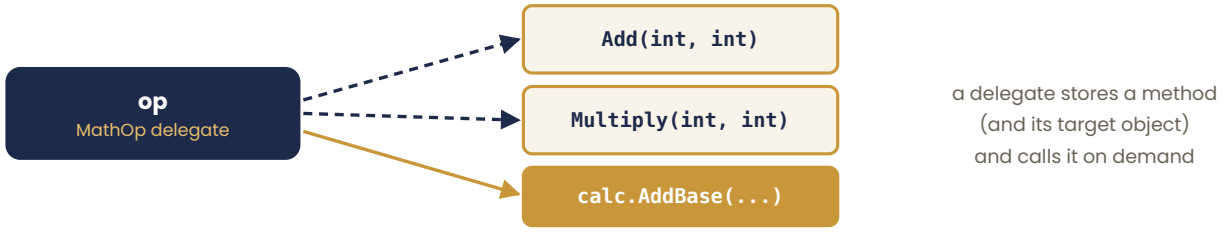
static int Add(int a, int b) => a + b;
static int Multiply(int a, int b) => a * b;

delegate int MathOp(int a, int b);        // the delegate type: int (int, int)

class Calculator(int baseValue)
{
    public int AddBase(int a, int b) => a + b + baseValue;
    public override string ToString() => $"Calculator(base {baseValue})";
}
```

OUTPUT

```
9
18
19
target method: AddBase, target object: Calculator(base 10)
```



ONE DELEGATE VARIABLE CAN REFER TO ANY METHOD WITH THE MATCHING SIGNATURE

15.1.2 Passing methods as parameters

The main power of delegates is passing **behaviour** into a method. The method contains the fixed part of an algorithm and calls the delegate for the variable part. This is how `List.Sort(Comparison)`, `List.FindAll(Predicate)` and all of LINQ work.

BEHAVIOUR PASSED AS AN ARGUMENT

Program.cs

```
int[] marks = { 72, 45, 91, 38, 85, 60 };

Console.WriteLine(string.Join(" ", Filter(marks, IsPass)));        // pass a named method
Console.WriteLine(string.Join(" ", Filter(marks, IsDistinction)));
Console.WriteLine(string.Join(" ", Transform(marks, AddGrace)));

static int[] Filter(int[] items, Check test)          // the algorithm is fixed...
{
    var result = new List<int>();
    foreach (int x in items) if (test(x)) result.Add(x); // ...the decision is supplied
    return result.ToArray();
}

static int[] Transform(int[] items, Change f)
{
    int[] result = new int[items.Length];
    for (int i = 0; i < items.Length; i++) result[i] = f(items[i]);
    return result;
}

static bool IsPass(int m) => m >= 40;
static bool IsDistinction(int m) => m >= 85;
static int AddGrace(int m) => Math.Min(100, m + 5);

delegate bool Check(int value);
delegate int Change(int value);
```

OUTPUT

```
72 45 91 85 60
91 85
77 50 96 43 90 65
```

15.1.3 Multicast delegates

A delegate can hold a list of methods, an **invocation list**. `+=` adds a method and `-=` removes one; invoking the delegate calls all of them in order. If the delegate returns a value, the caller receives only the **last** method's result, so multicast is used mainly with `void` delegates such as notifications. If one method throws, the remaining ones are not called.

A MULTICAST DELEGATE

Program.cs

```
Notify notify = SendEmail;
notify += SendSms;                          // add to the invocation list
notify += WriteLog;
notify("Order #101 shipped");

Console.WriteLine($"{notify.GetInvocationList().Length} handlers");
notify -= SendSms;                          // remove one
notify("Order #102 shipped");

Func<int> numbers = () => 1;
numbers += () => 2;
numbers += () => 3;
Console.WriteLine($"return value of a multicast call: {numbers()}");   // only the last

static void SendEmail(string msg) => Console.WriteLine($"  email: {msg}");
static void SendSms(string msg) => Console.WriteLine($"  sms:   {msg}");
static void WriteLog(string msg) => Console.WriteLine($"  log:   {msg}");

delegate void Notify(string message);
```

OUTPUT

```
  email: Order #101 shipped
  sms:   Order #101 shipped
  log:   Order #101 shipped
3 handlers
  email: Order #102 shipped
  log:   Order #102 shipped
return value of a multicast call: 3
```

NOTE

Delegates are immutable: `+=` creates a new delegate with a longer invocation list and assigns it to the variable. This is why removing a handler while the delegate is being invoked is safe.

15.2 Built-in delegates

Declaring a new delegate type for every signature is unnecessary. .NET provides generic delegate types that cover almost every case, and modern code uses them in preference to custom delegate types.

Delegate	Signature	Use
`Action`	`void ()`	a callback with no data
`Action<T1, ..., T16>`	`void (T1, ...)`	perform something with arguments
`Func<TResult>`	`TResult ()`	produce a value
`Func<T1, ..., T16, TResult>`	`TResult (T1, ...)`	compute a value from arguments (the last type is the result)
`Predicate<T>`	`bool (T)`	test a condition (older APIs such as List.FindAll)
`Comparison<T>`	`int (T, T)`	compare two values for sorting
`EventHandler<TArgs>`	`void (object?, TArgs)`	events (15.4)

15.2.1 Action and Action<T>

`Action` represents a method that returns nothing. The generic versions `Action<T>`, `Action<T1, T2>` and so on take up to 16 parameters.

ACTION DELEGATES Program.cs

```
Action hello = () => Console.WriteLine("Hello!");
Action<string> greet = name => Console.WriteLine($"Hello, {name}!");
Action<string, int> repeat = (text, n) => Console.WriteLine(string.Concat(Enumerable.Repeat(text, n)));

hello();
greet("Asha");
repeat("ab", 3);
RunTwice(hello);

new List<int> { 1, 2, 3 }.ForEach(n => Console.Write($"{n * n} "));    // List.ForEach takes Action<T>
Console.WriteLine();

static void RunTwice(Action work) { work(); work(); }
```

OUTPUT

```
Hello!
Hello, Asha!
ababab
Hello!
Hello!
1 4 9
```

15.2.2 Func<T, TResult>

`Func` represents a method that returns a value. The **last** type argument is always the return type: `Func<int, int, bool>` takes two ints and returns a bool. Func is the delegate used throughout LINQ.

FUNC DELEGATES Program.cs

```
Func<int> dice = () => 4;                                  // no input, returns int
Func<double, double> square = x => x * x;                  // double -> double
Func<int, int, bool> divisible = (a, b) => a % b == 0;     // (int, int) -> bool
Func<string, string, string> join = (a, b) => $"{a}-{b}";

Console.WriteLine($"{dice()} {square(1.5)} {divisible(12, 4)} {join("C", "Sharp")}");

Func<double, double> addTax = p => p * 1.18;
Func<double, double> discount = p => p * 0.9;
Func<double, double> both = Compose(discount, addTax);    // build new functions from old ones
Console.WriteLine(both(1000));

static Func<T, T> Compose<T>(Func<T, T> first, Func<T, T> second) => x => second(first(x));
```

OUTPUT

```
4 2.25 True C-Sharp
1062
```

15.2.3 Predicate<T>

`Predicate<T>` is a function from T to bool. It predates `Func<T, bool>` and is used by older collection methods: `List.Find`, `FindAll`, `RemoveAll`, `Exists`, `TrueForAll` and `Array.Find`. Although the two have identical signatures, they are different types and cannot be assigned to each other directly.

PREDICATE<T> WITH LIST METHODS Program.cs

```
var cities = new List<string> { "Pune", "Delhi", "Patna", "Mumbai", "Panaji" };
Predicate<string> startsWithP = c => c.StartsWith('P');

Console.WriteLine(string.Join(", ", cities.FindAll(startsWithP)));
Console.WriteLine($"{cities.Exists(c => c.Length > 5)} {cities.TrueForAll(c => c.Length >= 4)}");
Console.WriteLine($"removed {cities.RemoveAll(startsWithP)}, left: {string.Join(", ", cities)}");

Func<string, bool> asFunc = c => c.StartsWith('P');
// Predicate<string> p2 = asFunc;           // error CS0029: different delegate types
Predicate<string> p2 = new Predicate<string>(asFunc);     // wrap it explicitly
Console.WriteLine(p2("Pune"));
```

OUTPUT

```
Pune, Patna, Panaji
True True
removed 3, left: Delhi, Mumbai
True
```

15.3 Anonymous methods and lambdas

Writing a separate named method for every little comparison or filter is clumsy. C# lets you write the method **inline**, where the delegate is needed.

15.3.1 Anonymous methods with delegate

C# 2 introduced **anonymous methods**: `delegate (parameters) { body }`. They are still valid, and one unique feature remains: the parameter list can be omitted entirely, matching any signature. In practice lambdas (C# 3) have replaced them.

ANONYMOUS METHODS Program.cs

```
Func<int, int> twice = delegate (int x) { return x * 2; };
Action<string> shout = delegate (string s) { Console.WriteLine(s.ToUpper() + "!"); };
Action<int, int> ignoreArgs = delegate { Console.WriteLine("parameters ignored"); };   // no parameter
list

Console.WriteLine(twice(21));
shout("anonymous method");
ignoreArgs(1, 2);
```

OUTPUT

```
42
ANONYMOUS METHOD!
parameters ignored
```

15.3.2 Expression lambdas and statement lambdas

A **lambda expression** has parameters, the arrow `=>` ("goes to") and a body. An **expression lambda** has a single expression as its body and returns its value. A **statement lambda** has a block in braces and uses `return` if it returns a value. Parameter types are usually inferred from the target delegate; parentheses can be omitted for exactly one untyped parameter, and `_` discards unused parameters.

Form	Example
no parameters	`() => DateTime.Now.Year`
one parameter	`x => x * x`

Form	Example
several parameters	`(a, b) => a + b`
explicit types	`(int a, int b) => a + b`
statement body	`s => { var t = s.Trim(); return t.Length; }`
discards	`(_, _) => 0`
explicit return type (C# 10)	`int (string s) => s.Length`

LAMBDA FORMS Program.cs

```
Func<int, int> square = x => x * x;                          // expression lambda
Func<int, int, int> add = (a, b) => a + b;
Func<string, int> wordCount = text =>                         // statement lambda
{
    if (string.IsNullOrWhiteSpace(text)) return 0;
    return text.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length;
};
Func<int, int, int> first = (a, _) => a;                      // discard

Console.WriteLine($"{square(9)} {add(2, 3)} {wordCount("lambdas are  short")} {first(7, 8)}");

var words = new List<string> { "kiwi", "banana", "fig", "apple" };
words.Sort((a, b) => a.Length != b.Length ? a.Length.CompareTo(b.Length) : string.CompareOrdinal(a, b));
Console.WriteLine(string.Join(" ", words));
```

OUTPUT

```
81 5 3 7
fig kiwi apple banana
```

15.3.3 Captured variables and closures

A lambda can use local variables and parameters of the method in which it is written. It **captures** the variables themselves, not copies of their values: the lambda sees later changes, and changes made by the lambda are visible outside. The compiler implements this by moving the captured variables into a hidden object (a **closure**) that lives as long as the delegate does, even after the enclosing method has returned.

CLOSURES CAPTURE VARIABLES

Program.cs

```
int threshold = 50;
Func<int, bool> above = m => m > threshold;    // captures the variable threshold
Console.WriteLine(above(60));
threshold = 70;                                // the lambda sees the change
Console.WriteLine(above(60));

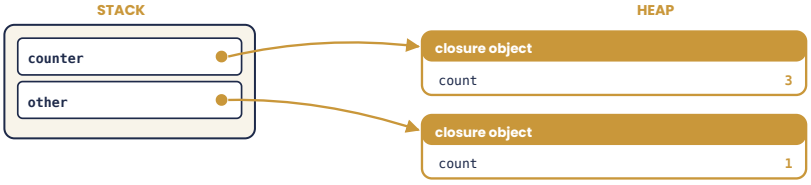
Func<int> counter = MakeCounter();
Func<int> other = MakeCounter();
Console.WriteLine($"{counter()} {counter()} {counter()} | {other()}");   // each closure has its own
count

var actions = new List<Action>();
for (int i = 0; i < 3; i++) actions.Add(() => Console.Write($"for:{i} "));   // ONE shared i
foreach (int j in new[] { 0, 1, 2 }) actions.Add(() => Console.Write($"foreach:{j} "));  // new j each
time
foreach (Action a in actions) a();
Console.WriteLine();

static Func<int> MakeCounter()
{
    int count = 0;                             // lives on in the closure after the method returns
    return () => ++count;
}
```

OUTPUT

```
True
False
1 2 3 | 1
for:3 for:3 for:3 foreach:0 foreach:1 foreach:2
```



EACH CALL OF MAKECOUNTER CREATES A NEW CLOSURE OBJECT HOLDING ITS OWN COUNT

WATCH OUT

The `for` loop has a single variable i shared by all iterations, so all three lambdas see its final value 3. `foreach` creates a fresh variable per iteration (since C# 5). To capture the current value in a `for` loop, copy it first: `int copy = i; actions.Add(() => Use(copy));`.

15.3.4 Static lambdas

Capturing is convenient but has a cost: a closure object is allocated, and an accidental capture can keep large objects alive. Marking a lambda `static` (C# 9) forbids it from capturing any local variable, parameter or `this`; the compiler then guarantees that no closure is needed and can cache the delegate. Constants and static members can still be used.

STATIC LAMBDAS CANNOT CAPTURE Program.cs

```
const double Tax = 0.18;
int discount = 10;

Func<double, double> withTax = static price => price * (1 + Tax);     // const is fine
// Func<double, double> bad = static price => price - discount;       // error CS8820: static lambda
cannot capture 'discount'
Func<double, double, double> applyDiscount = static (price, pct) => price * (1 - pct / 100);   // pass
data as parameters

Console.WriteLine(withTax(100));
Console.WriteLine(applyDiscount(500, discount));
```

OUTPUT

```
118
450
```

15.3.5 Natural type of lambdas and default parameters in lambdas (C# 12)

Since C# 10, a lambda has a **natural type** when its parameter types are known: `var f = (int x) => x * 2;` infers `Func<int, int>`, and a method group works too when it has a single overload, as in `var read = Console.ReadLine;` (for `int.Parse`, which has many overloads, the type cannot be inferred: error CS8917). C# 12 lets lambda parameters have **default values** and `params` arrays, exactly like methods; such lambdas get a compiler-generated delegate type, so use `var` for them.

NATURAL TYPES, DEFAULT VALUES AND PARAMS Program.cs

```
var doubleIt = (int x) => x * 2;                  // natural type Func<int, int>
var isEven = (int n) => n % 2 == 0;               // Func<int, bool>
var log = (string msg) => Console.WriteLine($"[log] {msg}");   // Action<string>
Console.WriteLine($"{doubleIt(21)} {isEven(7)} {doubleIt.GetType().Name} {log.GetType().Name}");
log("natural types");

var greet = (string name = "guest", string greeting = "Hello") => $"{greeting}, {name}!";   // defaults
(C# 12)
Console.WriteLine(greet());
Console.WriteLine(greet("Asha"));
Console.WriteLine(greet("Ravi", "Welcome"));

var total = (params int[] values) => values.Sum();    // params in a lambda (C# 12)
Console.WriteLine($"{total()} {total(5)} {total(1, 2, 3, 4)}");
```

OUTPUT

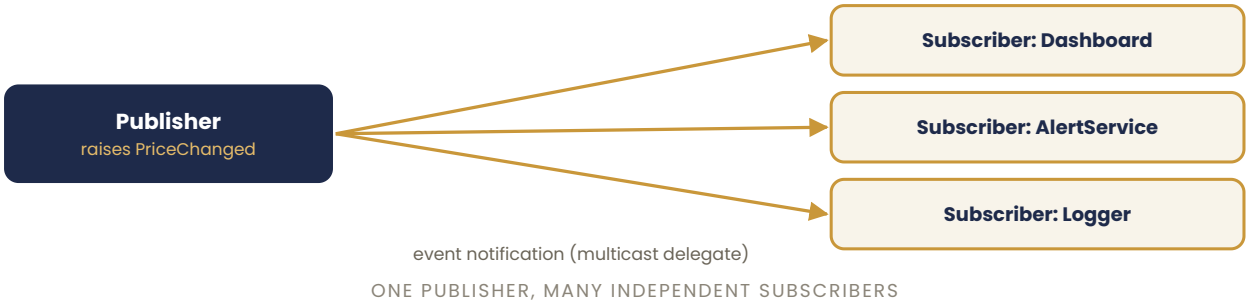
```
42 False Func`2 Action`1
[log] natural types
Hello, guest!
Hello, Asha!
Welcome, Ravi!
0 5 10
```

15.4 Events

15.4.1 The publisher-subscriber model

Many objects need to react when something happens to another object: when a button is clicked, a file finishes downloading or a stock price changes. Hard-coding those reactions into the object would tie it to every class that cares. In the **publisher-subscriber** (observer) model, the **publisher** only announces "this

happened"; any number of **subscribers** register handler methods and are notified. The publisher does not know or care who is listening.



ONE PUBLISHER, MANY INDEPENDENT SUBSCRIBERS

15.4.2 Declaring events with event

An **event** is a delegate field wrapped with the `event` keyword. From outside the class, the only operations allowed are `+=` (subscribe) and `-=` (unsubscribe). Only the declaring class can **raise** (invoke) the event or replace its subscriber list. With a plain public delegate field, any code could invoke it or wipe out all subscribers with `=`; the event keyword prevents both.

AN EVENT KEEPS CONTROL WITH THE PUBLISHER Program.cs

```
var timer = new Countdown();
timer.Finished += () => Console.WriteLine("  subscriber 1: ring the bell");
timer.Finished += () => Console.WriteLine("  subscriber 2: show a message");
timer.Run(3);
// timer.Finished();                          // error CS0070: can only appear on the left of += or -=
// timer.Finished = null;                     // error CS0070 as well

class Countdown
{
    public event Action? Finished;            // the event

    public void Run(int from)
    {
        for (int i = from; i > 0; i--) Console.WriteLine(i);
        Finished?.Invoke();                    // only the publisher can raise it
    }
}
```

OUTPUT

```
3
2
1
  subscriber 1: ring the bell
  subscriber 2: show a message
```

15.4.3 EventHandler and EventHandler<TEventArgs>

.NET events follow a standard pattern. The delegate type is `EventHandler` (no data) or `EventHandler<TEventArgs>`, whose handlers receive two parameters: `object? sender`, the object that raised the event, and an **event-args** object carrying the details. Data classes conventionally end in **EventArgs** and derive from `System.EventArgs` (or are records). A protected virtual `OnXxx` method raises the event so that subclasses can customise it.

THE STANDARD EVENTHANDLER PATTERN

Program.cs

```
var d = new Downloader();
d.ProgressChanged += (sender, e) => Console.WriteLine($"  {e.FileName}: {e.Percent}%");
d.Completed += OnCompleted;
d.Download("notes.pdf");

static void OnCompleted(object? sender, EventArgs e) =>
    Console.WriteLine($"  done (raised by {sender?.GetType().Name})");

public class ProgressEventArgs(string fileName, int percent) : EventArgs
{
    public string FileName { get; } = fileName;
    public int Percent { get; } = percent;
}

public class Downloader
{
    public event EventHandler<ProgressEventArgs>? ProgressChanged;   // with data
    public event EventHandler? Completed;                            // without data

    public void Download(string file)
    {
        Console.WriteLine($"downloading {file}");
        for (int p = 25; p <= 100; p += 25) OnProgressChanged(new ProgressEventArgs(file, p));
        OnCompleted();
    }

    protected virtual void OnProgressChanged(ProgressEventArgs e) => ProgressChanged?.Invoke(this, e);
    protected virtual void OnCompleted() => Completed?.Invoke(this, EventArgs.Empty);
}
```

OUTPUT

```
downloading notes.pdf
  notes.pdf: 25%
  notes.pdf: 50%
  notes.pdf: 75%
  notes.pdf: 100%
  done (raised by Downloader)
```

15.4.4 Raising events safely with ?.Invoke

If nobody has subscribed, the event field is `null`, and calling it would throw a NullReferenceException. The null-conditional call `Event?.Invoke(this, e)` reads the field **once**, checks it and invokes it. This is also thread-safe: in older code, `if (Event != null) Event(this, e);` could fail if another thread unsubscribed the last handler between the check and the call.

RAISING AN EVENT SAFELY

Program.cs

```
var sensor = new Sensor();
sensor.Read(31);                                  // no subscribers: nothing happens, no exception
sensor.TooHot += (_, t) => Console.WriteLine($"warning: {t} °C");
sensor.Read(28);
sensor.Read(45);

class Sensor
{
    public event EventHandler<int>? TooHot;       // EventHandler<T> accepts any T since .NET 4.5

    public void Read(int celsius)
    {
        Console.WriteLine($"reading {celsius}");
        if (celsius > 40) TooHot?.Invoke(this, celsius);
        // old style, NOT thread-safe:  if (TooHot != null) TooHot(this, celsius);
    }
}
```

OUTPUT

```
reading 31
reading 28
reading 45
warning: 45 °C
```

15.4.5 Subscribing and unsubscribing (+=, -=)

`+=` adds a handler; `-=` removes it. To unsubscribe you must pass the **same** delegate: a named method works naturally, but a lambda written twice creates two different delegates, so `-=` with a new lambda removes nothing. Keep a lambda in a variable if you need to unsubscribe it later.

UNSUBSCRIBING REQUIRES THE SAME DELEGATE

Program.cs

```
var button = new Button();
EventHandler logClick = (_, _) => Console.WriteLine("  lambda kept in a variable");

button.Click += SayHello;
button.Click += logClick;
button.Click += (_, _) => Console.WriteLine("  anonymous lambda");
button.PressAndReport();

button.Click -= SayHello;                                   // works: same method
button.Click -= logClick;                                   // works: same delegate instance
button.Click -= (_, _) => Console.WriteLine("  anonymous lambda");   // removes NOTHING: a new delegate
button.PressAndReport();

static void SayHello(object? s, EventArgs e) => Console.WriteLine("  named method");

class Button
{
    public event EventHandler? Click;
    public void PressAndReport()
    {
        Console.WriteLine($"click ({Click?.GetInvocationList().Length ?? 0} handlers)");
        Click?.Invoke(this, EventArgs.Empty);
    }
}
```

OUTPUT

```
click (3 handlers)
  named method
  lambda kept in a variable
  anonymous lambda
click (1 handlers)
  anonymous lambda
```

15.4.6 Memory leaks from forgotten event subscriptions

Subscribing stores a delegate in the publisher, and the delegate holds a reference to the subscriber object (its Target). As long as the publisher is alive, the subscriber **cannot be garbage collected**, even if nothing else uses it. When the publisher is long-lived (a static event, a singleton service, the main window) and short-lived objects subscribe without unsubscribing, they pile up in memory and keep receiving events: a classic .NET memory leak.

A LEAK CAUSED BY A STATIC EVENT

Program.cs

```
for (int i = 1; i <= 3; i++)
{
    var page = new Page($"page {i}");
    page.Open();
    if (i == 3) page.Close();               // only the last page unsubscribes
}                                            // pages 1 and 2 are unreachable for us... but not for the
GC
Settings.ChangeTheme("dark");
Console.WriteLine($"subscribers still attached: {Settings.SubscriberCount}");

static class Settings                        // a long-lived publisher
{
    public static event Action<string>? ThemeChanged;
    public static int SubscriberCount => ThemeChanged?.GetInvocationList().Length ?? 0;
    public static void ChangeTheme(string theme) => ThemeChanged?.Invoke(theme);
}

class Page(string name)
{
    public void Open() => Settings.ThemeChanged += Apply;
    public void Close() => Settings.ThemeChanged -= Apply;          // the fix: always unsubscribe
    private void Apply(string theme) => Console.WriteLine($"  {name} repaints in {theme}");
}
```

OUTPUT

```
  page 1 repaints in dark
  page 2 repaints in dark
subscribers still attached: 2
```

TIP

Unsubscribe in `Dispose`, `Close` or the matching lifecycle method; subscribe and unsubscribe in pairs. For long-lived publishers with many short-lived listeners, consider weak-event patterns (for example WeakReferenceMessenger in the CommunityToolkit.Mvvm library).

15.5 Expression trees (overview)

A lambda assigned to a delegate type is compiled into executable code. The same lambda assigned to `Expression<TDelegate>` is compiled into an **expression tree**: a data structure that describes the code (parameters, operators, constants, method calls) and can be inspected, modified, translated or compiled at run time. This is how **LINQ providers** such as Entity Framework Core turn `q.Where(s => s.Marks > 80)` into SQL: they read the tree instead of running the lambda.

CODE AS DATA

Program.cs

```
using System.Linq.Expressions;

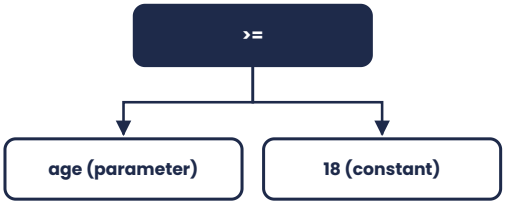
Func<int, bool> asCode = age => age >= 18;                    // compiled code
Expression<Func<int, bool>> asData = age => age >= 18;        // a tree describing the code

Console.WriteLine(asCode(20));
Console.WriteLine(asData);                                     // prints the expression
var body = (BinaryExpression)asData.Body;
Console.WriteLine($"node {body.NodeType}, left {body.Left}, right {body.Right}");
Console.WriteLine(asData.Compile()(15));                       // turn data into code

ParameterExpression a = Expression.Parameter(typeof(int), "a");  // build a tree by hand
ParameterExpression b = Expression.Parameter(typeof(int), "b");
BinaryExpression expr = Expression.Add(Expression.Multiply(a, b), Expression.Constant(1));
var lambda = Expression.Lambda<Func<int, int, int>>(expr, a, b);
Console.WriteLine($"{lambda} gives {lambda.Compile()(3, 4)}");
```

OUTPUT

```
True
age => (age >= 18)
node GreaterThanOrEqual, left age, right 18
False
(a, b) => ((a * b) + 1) gives 13
```



THE TREE FOR AGE => AGE >= 18

A delegate is a type-safe reference to one or more methods with a given signature; it lets you pass behaviour as data, and multicast delegates call several methods in turn. Prefer the built-in Action (no result), Func (last type is the result) and Predicate types. Write the behaviour inline with lambdas: expression or statement bodies, captured variables (closures), static lambdas to forbid capture, natural types, and default parameters (C# 12). Events wrap delegates so only the publisher can raise them; use EventHandler<TEventArgs>, raise with ?.Invoke, and always unsubscribe to avoid leaks. Expression<TDelegate> turns a lambda into an inspectable tree, the basis of LINQ providers.

PRACTICE CHECKPOINT

Write a **Calculator** that takes its operations as **Func<double, double, double>**, and a **Stock** class that raises a **PriceChanged** event with custom event args.

SAMPLE SOLUTION

Program.cs

```
var calc = new Calculator();
calc.Register("^", Math.Pow);                                  // add a new operation at run time
calc.Register("avg", (a, b) => (a + b) / 2);
foreach (string op in new[] { "+", "-", "*", "/", "^", "avg", "%" })
{
    try { Console.WriteLine($"8 {op} 2 = {calc.Calculate(op, 8, 2)}"); }
    catch (KeyNotFoundException e) { Console.WriteLine(e.Message); }
}
```

```
var stock = new Stock("LBF", 250.00m);
stock.PriceChanged += (sender, e) =>
    Console.WriteLine($"{((Stock)sender!).Symbol}: {e.OldPrice} -> {e.NewPrice}
({e.ChangePercent:+0.00;-0.00}%)");
stock.PriceChanged += (_, e) =>
{
    if (Math.Abs(e.ChangePercent) >= 10) Console.WriteLine($"  ALERT: moved {e.ChangePercent:F2}%");
};
stock.Price = 262.50m;
stock.Price = 236.25m;
stock.Price = 236.25m;                                          // no change: no event

public class Calculator
{
    private readonly Dictionary<string, Func<double, double, double>> ops = new()
    {
        ["+"] = (a, b) => a + b,
        ["-"] = (a, b) => a - b,
        ["*"] = (a, b) => a * b,
        ["/"] = (a, b) => b == 0 ? throw new DivideByZeroException() : a / b,
    };

    public void Register(string symbol, Func<double, double, double> op) => ops[symbol] = op;

    public double Calculate(string symbol, double a, double b) =>
        ops.TryGetValue(symbol, out var op) ? op(a, b) : throw new KeyNotFoundException($"Unknown
operation '{symbol}'");
}

public class PriceChangedEventArgs(decimal oldPrice, decimal newPrice) : EventArgs
{
    public decimal OldPrice { get; } = oldPrice;
    public decimal NewPrice { get; } = newPrice;
    public decimal ChangePercent => (NewPrice - OldPrice) / OldPrice * 100;
}

public class Stock(string symbol, decimal price)
{
    public string Symbol { get; } = symbol;
    public event EventHandler<PriceChangedEventArgs>? PriceChanged;

    public decimal Price
    {
        get => price;
        set
        {
            if (value == price) return;
            var args = new PriceChangedEventArgs(price, value);
            price = value;
            PriceChanged?.Invoke(this, args);
        }
    }
}
```

OUTPUT

```
8 + 2 = 10
8 - 2 = 6
8 * 2 = 16
8 / 2 = 4
8 ^ 2 = 64
8 avg 2 = 5
Unknown operation '%'
LBF: 250.00 -> 262.50 (+5.00%)
LBF: 262.50 -> 236.25 (-10.00%)
  ALERT: moved -10.00%
```

MODULE 16 · INTERMEDIATE

LINQ (Language Integrated Query)

Minimum duration **4 days** | Level **Intermediate**

LINQ lets you query collections, databases and XML with one consistent, strongly typed syntax built into C#. Instead of writing loops that filter, sort, group and total data by hand, you describe what you want and LINQ does the work. This module explains what LINQ is, the two syntaxes, deferred execution, and then every major operator family: filtering and projection, sorting, element operators, aggregation, quantifiers, grouping and joining (including left joins and the .NET 9 CountBy and AggregateBy), set operations, partitioning and generation, and conversion. It ends with the different LINQ providers, IEnumerable versus IQueryable, and performance pitfalls such as multiple enumeration.

IN THIS MODULE

16.1 Introduction
16.2 Filtering and projection
16.3 Sorting
16.4 Element operators
16.5 Aggregation
16.6 Quantifiers
16.7 Grouping and joining
16.8 Set operations
16.9 Partitioning and generation
16.10 Conversion
16.11 LINQ providers
16.12 Performance considerations

Most examples in this module use the same small data set. It is shown once here; in the examples it is abbreviated as `var emps = Data.Employees;`.

DATA.CS: THE SAMPLE EMPLOYEES

C#

```
public record Emp(int Id, string Name, string Dept, decimal Salary, int? ManagerId);

public static class Data
{
    public static readonly List<Emp> Employees =
    [
        new(1, "Asha",   "IT",    95000, null),
        new(2, "Ravi",   "IT",    72000, 1),
        new(3, "Meera",  "HR",    61000, null),
        new(4, "Kiran",  "IT",    58000, 1),
        new(5, "Zoya",   "Sales", 67000, null),
        new(6, "Arjun",  "Sales", 52000, 5),
        new(7, "Neha",   "HR",    48000, 3),
        new(8, "Vikram", "Sales", 80000, 5),
    ];
}
```

Id	Name	Dept	Salary	ManagerId
1	Asha	IT	95,000	-
2	Ravi	IT	72,000	1
3	Meera	HR	61,000	-

Id	Name	Dept	Salary	ManagerId
4	Kiran	IT	58,000	1
5	Zoya	Sales	67,000	-
6	Arjun	Sales	52,000	5
7	Neha	HR	48,000	3
8	Vikram	Sales	80,000	5

16.1 Introduction

16.1.1 What LINQ is and why it was introduced

Before LINQ (C# 3, 2007), every kind of data had its own query technique: loops for collections, SQL strings for databases, XPath for XML. SQL strings were not checked by the compiler, and typos showed up only at run time. **LINQ** (Language Integrated Query) added one set of query operators, available on any data source, with full IntelliSense and compile-time type checking. It is built from C# features you already know: extension methods, lambdas, generics, anonymous types and iterators.

THE SAME QUERY WITH A LOOP AND WITH LINQ — Program.cs

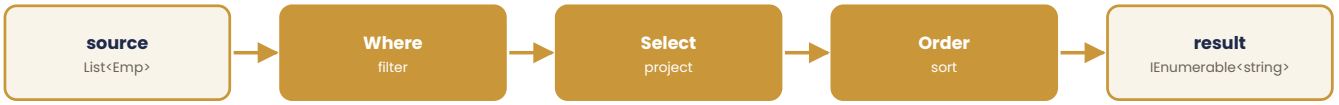
```
var emps = Data.Employees;

// Without LINQ: a loop, a temporary list, a manual sort
var result = new List<string>();
foreach (var e in emps)
    if (e.Dept == "IT" && e.Salary > 60000) result.Add(e.Name);
result.Sort();
Console.WriteLine(string.Join(", ", result));

// With LINQ: say WHAT you want
var names = emps.Where(e => e.Dept == "IT" && e.Salary > 60000)
                .Select(e => e.Name)
                .Order();
Console.WriteLine(string.Join(", ", names));
```

OUTPUT

```
Asha, Ravi
Asha, Ravi
```



A LINQ QUERY IS A PIPELINE: EACH OPERATOR TAKES A SEQUENCE AND RETURNS A NEW ONE

16.1.2 Query syntax vs method syntax

LINQ can be written in two styles. **Method syntax** chains extension methods with lambdas. **Query syntax** looks like SQL, starting with `from` and ending with `select` or `group`; the compiler translates it into method calls, so both produce identical code. Query syntax reads well for joins, grouping and `let`; method syntax is more common and offers every operator (Count, Take, Distinct and many others have no query keyword). Mixing is fine.

TWO SYNTAXES, ONE RESULT

Program.cs

```
var emps = Data.Employees;

var q1 = from e in emps                          // query syntax
         where e.Salary >= 60000
         orderby e.Salary descending
         select new { e.Name, e.Salary };

var q2 = emps.Where(e => e.Salary >= 60000)      // method syntax: the same query
              .OrderByDescending(e => e.Salary)
              .Select(e => new { e.Name, e.Salary });

Console.WriteLine(string.Join(" ", q1.Select(x => x.Name)));
Console.WriteLine(string.Join(" ", q2.Select(x => x.Name)));

var q3 = from e in emps                          // 'let' introduces a helper variable
         let bonus = e.Salary * 0.1m
         where bonus > 7000
         select $"{e.Name}:{bonus:N0}";
Console.WriteLine(string.Join(" ", q3));
Console.WriteLine((from e in emps where e.Dept == "HR" select e).Count());   // mixing both styles
```

OUTPUT

```
Asha Vikram Ravi Zoya Meera
Asha Vikram Ravi Zoya Meera
Asha:9,500 Ravi:7,200 Vikram:8,000
2
```

Query keyword	Method
`from x in source`	(the source sequence)
`where cond`	`Where(x => cond)`
`select expr`	`Select(x => expr)`
`orderby k` / `descending`, `k2`	`OrderBy` / `OrderByDescending`, `ThenBy`
`group x by k`	`GroupBy(x => k)`
`join y in s on k1 equals k2`	`Join(...)`
`join ... into g`	`GroupJoin(...)`
`from y in x.Items` (second from)	`SelectMany(...)`
`let v = expr`	`Select` into an anonymous type

16.1.3 Deferred vs immediate execution

Most LINQ operators use **deferred execution**: calling `Where` or `Select` does not run anything; it only builds a description of the query. The query runs when it is **enumerated**, by foreach or by an operator that needs the results, and it runs **again** every time it is enumerated, seeing the current data. Operators that return a single value or a materialised collection (`Count`, `Sum`, `First`, `ToList`, `ToArray`, `ToDictionary`) execute **immediately**.

DEFERRED EXECUTION

Program.cs

```
var numbers = new List<int> { 1, 2, 3, 4 };

var evens = numbers.Where(n =>
{
    Console.Write($"[check {n}] ");
    return n % 2 == 0;
});                                          // nothing runs here
Console.WriteLine("query defined");

numbers.Add(6);                              // change the source after defining the query
foreach (int n in evens) Console.Write($"=> {n} ");   // runs now, sees 6
Console.WriteLine();

List<int> snapshot = numbers.Where(n => n > 2).ToList();   // immediate: runs once, now
numbers.Add(8);
Console.WriteLine($"snapshot: {string.Join(",", snapshot)}  live count: {numbers.Count(n => n > 2)}");
```

OUTPUT

```
query defined
[check 1] [check 2] => 2 [check 3] [check 4] => 4 [check 6] => 6
snapshot: 3,4,6  live count: 4
```

NOTE

Elements flow through a deferred pipeline **one at a time**: element 2 is checked and printed before element 3 is even looked at. This streaming makes LINQ memory-efficient for large sources, but also means side effects in lambdas happen at enumeration time, possibly more than once.

Deferred (streaming)	Deferred (buffering whole input first)	Immediate
Where, Select, SelectMany, Skip, Take, Concat, Zip, Distinct, OfType, Cast, Chunk	OrderBy, ThenBy, Reverse, GroupBy, Join, GroupJoin, Except, Intersect	Count, Sum, Min, Max, Average, Aggregate, First, Single, Any, All, Contains, ToList, ToArray, ToDictionary, ToHashSet, ToLookup

16.2 Filtering and projection

16.2.1 Where

`Where` keeps the elements for which a predicate returns true. Conditions can be combined with `&&` and `||`, or by chaining several Where calls. An overload also passes the element's **index**.

FILTERING WITH WHERE

Program.cs

```
var emps = Data.Employees;

var itHigh = emps.Where(e => e.Dept == "IT" && e.Salary > 60000);
Console.WriteLine(string.Join(", ", itHigh.Select(e => e.Name)));

var noManager = emps.Where(e => e.ManagerId is null).Where(e => e.Salary < 90000);   // chained
Console.WriteLine(string.Join(", ", noManager.Select(e => e.Name)));

string[] words = { "alpha", "beta", "gamma", "delta", "epsilon" };
var evenPositions = words.Where((w, index) => index % 2 == 0);    // index overload
Console.WriteLine(string.Join(", ", evenPositions));
```

OUTPUT

```
Asha, Ravi
Meera, Zoya
alpha, gamma, epsilon
```

16.2.2 Select and anonymous types

`Select` **projects** each element into a new form: one property, a computed value, a tuple, a record or an **anonymous type** with just the fields you need. The result can be of a completely different type from the source. An overload provides the index.

PROJECTING WITH SELECT

Program.cs

```
var emps = Data.Employees;

IEnumerable<string> names = emps.Select(e => e.Name.ToUpper());
Console.WriteLine(string.Join(" ", names.Take(3)));

var cards = emps.Where(e => e.Dept == "HR")
                .Select(e => new { e.Name, Monthly = Math.Round(e.Salary / 12), Initial = e.Name[0] });
foreach (var c in cards) Console.WriteLine(c);

var ranked = emps.OrderByDescending(e => e.Salary)
                 .Select((e, i) => $"{i + 1}. {e.Name}")      // index overload
                 .Take(3);
Console.WriteLine(string.Join("  ", ranked));

var tuples = emps.Take(2).Select(e => (e.Name, Grade: e.Salary > 70000 ? 'A' : 'B'));   // tuples
Console.WriteLine(string.Join(" ", tuples));
```

OUTPUT

```
ASHA RAVI MEERA
{ Name = Meera, Monthly = 5083, Initial = M }
{ Name = Neha, Monthly = 4000, Initial = N }
1. Asha  2. Vikram  3. Ravi
(Asha, A) (Ravi, A)
```

16.2.3 SelectMany

When each element contains a **collection** (an order with lines, a student with courses), `Select` would give a sequence of sequences. `SelectMany` **flattens** them into one sequence. An overload keeps a reference to the parent so results can combine both. In query syntax it is a second `from`.

FLATTENING WITH SELECTMANY

Program.cs

```
var students = new[]
{
    new { Name = "Asha", Courses = new[] { "C#", "SQL" } },
    new { Name = "Ravi", Courses = new[] { "Java" } },
    new { Name = "Meera", Courses = new[] { "C#", "Python", "SQL" } },
};

var nested = students.Select(s => s.Courses);                  // IEnumerable<string[]>
Console.WriteLine(nested.Count());
var all = students.SelectMany(s => s.Courses);                 // IEnumerable<string>: flattened
Console.WriteLine(string.Join(", ", all));
Console.WriteLine(string.Join(", ", all.Distinct()));

var pairs = students.SelectMany(s => s.Courses, (s, c) => $"{s.Name}-{c}");   // keep the parent
Console.WriteLine(string.Join(" ", pairs));

var grid = from x in new[] { 1, 2 } from y in new[] { 'a', 'b' } select $"{x}{y}";   // two froms
Console.WriteLine(string.Join(" ", grid));
```

OUTPUT

```
3
C#, SQL, Java, C#, Python, SQL
C#, SQL, Java, Python
Asha-C# Asha-SQL Ravi-Java Meera-C# Meera-Python Meera-SQL
1a 1b 2a 2b
```



SELECTMANY TURNS A SEQUENCE OF COLLECTIONS INTO ONE FLAT SEQUENCE

16.2.4 OfType, Cast

`OfType<T>()` keeps only the elements that are of type T (skipping others and nulls) and returns them typed as T. `Cast<T>()` converts every element to T and throws InvalidCastException if one is not. Both are mainly used with mixed or non-generic collections such as `ArrayList` or older UI APIs.

OFTYPE AND CAST

Program.cs

```
object[] mixed = { 1, "two", 3.0, 4, null!, "five", 6 };
var ints = mixed.OfType<int>();
var strings = mixed.OfType<string>();
Console.WriteLine($"ints: {string.Join(",", ints)}  strings: {string.Join(",", strings)}  sum
{ints.Sum()}");

System.Collections.ArrayList legacy = new() { 10, 20, 30 };
IEnumerable<int> typed = legacy.Cast<int>();            // makes LINQ available on a legacy list
Console.WriteLine(typed.Max());

try { Console.WriteLine(mixed.Cast<int>().Sum()); }
catch (InvalidCastException) { Console.WriteLine("Cast<int> failed on \"two\""); }

Shape[] shapes = { new Circle(), new Square(), new Circle() };
Console.WriteLine($"circles: {shapes.OfType<Circle>().Count()}");

class Shape { }
class Circle : Shape { }
class Square : Shape { }
```

OUTPUT

```
ints: 1,4,6  strings: two,five  sum 11
30
Cast<int> failed on "two"
circles: 2
```

16.3 Sorting

16.3.1 OrderBy, OrderByDescending, ThenBy

`OrderBy` and `OrderByDescending` sort by a key; `ThenBy` and `ThenByDescending` add secondary keys used only when earlier keys are equal. LINQ sorting is **stable** (equal elements keep their original order) and returns a new sequence without changing the source. A custom `IComparer` can be passed as a second argument.

MULTI-KEY SORTING

Program.cs

```
var emps = Data.Employees;

var byDeptThenSalary = emps.OrderBy(e => e.Dept)
                           .ThenByDescending(e => e.Salary);
foreach (var e in byDeptThenSalary) Console.Write($"{e.Dept}:{e.Name} ");
Console.WriteLine();

var q = from e in emps
        orderby e.Name.Length, e.Name descending        // query syntax with two keys
        select e.Name;
Console.WriteLine(string.Join(" ", q));

var ci = new[] { "banana", "Apple", "cherry" }.OrderBy(s => s, StringComparer.Ordinal);
Console.WriteLine(string.Join(" ", ci));
```

OUTPUT

```
HR:Meera HR:Neha IT:Asha IT:Ravi IT:Kiran Sales:Vikram Sales:Zoya Sales:Arjun
Zoya Ravi Neha Asha Meera Kiran Arjun Vikram
Apple banana cherry
```

16.3.2 Order and Reverse

.NET 7 added `Order()` and `OrderDescending()` to sort elements by their own natural order without writing `x => x`. `Reverse()` returns the elements in reverse order of the source; it does not sort.

ORDER AND REVERSE Program.cs

```
int[] marks = { 72, 45, 91, 60 };
Console.WriteLine(string.Join(" ", marks.Order()));
Console.WriteLine(string.Join(" ", marks.OrderDescending()));
Console.WriteLine(string.Join(" ", marks.Reverse()));          // just reversed: 60 91 45 72
Console.WriteLine(string.Join(" ", new[] { "pear", "fig", "apple" }.Order()));
```

OUTPUT

```
45 60 72 91
91 72 60 45
60 91 45 72
apple fig pear
```

16.4 Element operators

16.4.1 First, FirstOrDefault, Last, Single, ElementAt

Element operators return **one** element and execute immediately. Each has an `OrDefault` variant that returns `default(T)` (null for reference types) instead of throwing when no element qualifies; since .NET 6 you can supply your own default value. `Single` additionally insists that **exactly one** element matches, which makes it a correctness check, for example when looking up by a unique id.

Operator	Returns	Throws InvalidOperationException when
`First(pred)`	the first match	no match
`FirstOrDefault(pred)`	first match or default	never
`Last(pred)` / `LastOrDefault`	the last match	no match (Last)
`Single(pred)`	the only match	zero or more than one match
`SingleOrDefault(pred)`	the only match or default	more than one match
`ElementAt(i)` / `ElementAtOrDefault`	the element at index i (also `^1`)	index out of range (ArgumentOutOfRangeException)

PICKING SINGLE ELEMENTS

Program.cs

```
var emps = Data.Employees;

Console.WriteLine(emps.First().Name);
Console.WriteLine(emps.First(e => e.Dept == "Sales").Name);
Console.WriteLine(emps.Last(e => e.Dept == "IT").Name);
Console.WriteLine(emps.FirstOrDefault(e => e.Dept == "Legal")?.Name ?? "no Legal employee");
Console.WriteLine(emps.Single(e => e.Id == 7).Name);                    // exactly one expected
Console.WriteLine(emps.ElementAt(2).Name + " / " + emps.ElementAt(^1).Name);

int[] empty = [];
Console.WriteLine(empty.FirstOrDefault());                              // 0
Console.WriteLine(empty.FirstOrDefault(-1));                            // custom default (.NET 6)

foreach (Func<Emp> attempt in new Func<Emp>[] { () => emps.First(e => e.Salary > 100000), () =>
emps.Single(e => e.Dept == "HR") })
{
    try { attempt(); }
    catch (InvalidOperationException ex) { Console.WriteLine(ex.Message); }
}
```

OUTPUT

```
Asha
Zoya
Kiran
no Legal employee
Neha
Meera / Vikram
0
-1
Sequence contains no matching element
Sequence contains more than one matching element
```

16.5 Aggregation

16.5.1 Count, Sum, Min, Max, Average, Aggregate

Aggregation operators reduce a sequence to a single value. `Count` (and `LongCount`) count elements, optionally matching a predicate. `Sum`, `Min`, `Max` and `Average` work on numbers or on a selected numeric property. `Average`, `Min` and `Max` throw on an empty sequence of non-nullable values. `Aggregate` is the general fold: it carries an accumulator through the sequence with a function you provide.

AGGREGATION OPERATORS

Program.cs

```
var emps = Data.Employees;

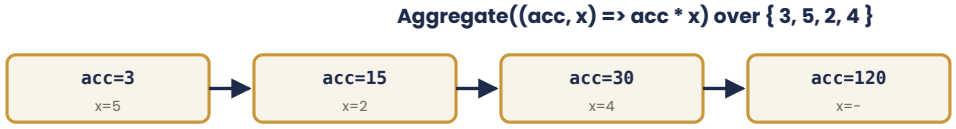
Console.WriteLine($"count {emps.Count}, IT count {emps.Count(e => e.Dept == "IT")}");
Console.WriteLine($"total {emps.Sum(e => e.Salary):N0}, average {emps.Average(e => e.Salary):N2}");
Console.WriteLine($"min {emps.Min(e => e.Salary):N0}, max {emps.Max(e => e.Salary):N0}");
Console.WriteLine($"longest name length {emps.Max(e => e.Name.Length)}");

int[] n = { 3, 5, 2, 4 };
int product = n.Aggregate((acc, x) => acc * x);                          // 3*5*2*4
string csv = n.Aggregate("", (acc, x) => acc.Length == 0 ? $"{x}" : $"{acc};{x}");   // with a seed
int maxGap = n.Zip(n.Skip(1), (a, b) => Math.Abs(b - a)).Max();
Console.WriteLine($"product {product}, csv {csv}, max gap {maxGap}");

try { Console.WriteLine(Array.Empty<int>().Average()); }
catch (InvalidOperationException e) { Console.WriteLine($"Average of empty: {e.Message}"); }
```

OUTPUT

```
count 8, IT count 3
total 533,000, average 66,625.00
min 48,000, max 95,000
longest name length 6
product 120, csv 3;5;2;4, max gap 3
Average of empty: Sequence contains no elements
```



AGGREGATE THREADS AN ACCUMULATOR THROUGH THE SEQUENCE

16.5.2 MinBy, MaxBy

`Max(e => e.Salary)` returns the highest **salary**, but usually you want the **employee** who earns it. .NET 6 added `MinBy` and `MaxBy`, which return the element with the smallest or largest key, in one pass and without sorting.

MINBY AND MAXBY

Program.cs

```
var emps = Data.Employees;

Emp top = emps.MaxBy(e => e.Salary)!;
Emp lowest = emps.MinBy(e => e.Salary)!;
Console.WriteLine($"top earner {top.Name} ({top.Salary:N0}), lowest {lowest.Name} ({lowest.Salary:N0})");

var shortestName = emps.MinBy(e => e.Name.Length)!;       // ties: the first one wins
Console.WriteLine(shortestName.Name);

var oldWay = emps.OrderByDescending(e => e.Salary).First();   // works, but sorts everything: O(n log n)
Console.WriteLine(oldWay.Name);
```

OUTPUT

```
top earner Asha (95,000), lowest Neha (48,000)
Asha
Asha
```

16.6 Quantifiers

16.6.1 Any, All, Contains

Quantifiers answer yes-or-no questions about a sequence and stop as soon as the answer is known. `Any()` is true if the sequence has any element; `Any(pred)` if any element matches. `All(pred)` is true if every element matches (and is true for an empty sequence). `Contains(x)` checks for a specific value using equality (or a supplied comparer).

ANY, ALL AND CONTAINS

Program.cs

```
var emps = Data.Employees;

Console.WriteLine($"any HR: {emps.Any(e => e.Dept == "HR")}, any Legal: {emps.Any(e => e.Dept ==
"Legal")}");
Console.WriteLine($"all earn > 40k: {emps.All(e => e.Salary > 40000)}, all in IT: {emps.All(e => e.Dept
== "IT")}");
Console.WriteLine($"names contain Zoya: {emps.Select(e => e.Name).Contains("Zoya")}");
Console.WriteLine($"empty list Any: {new List<int>().Any()}, All: {new List<int>().All(x => x > 100)}");

string[] banned = { "admin", "root" };
foreach (string user in new[] { "asha", "Root" })
    Console.WriteLine($"{user} banned: {banned.Contains(user, StringComparer.OrdinalIgnoreCase)}");
```

OUTPUT

```
any HR: True, any Legal: False
all earn > 40k: True, all in IT: False
names contain Zoya: True
empty list Any: False, All: True
asha banned: False
Root banned: True
```

TIP

To check whether a sequence has elements, use `Any()` rather than `Count() > 0`: Any stops at the first element, while Count may walk the whole sequence (for a List, Count is a property and both are fast).

16.7 Grouping and joining

16.7.1 GroupBy

`GroupBy` splits a sequence into groups of elements that share a key. The result is a sequence of `IGrouping<TKey, TElement>`, each of which has a `Key` and is itself a sequence of its elements. Groups appear in the order their keys first occur. Aggregating each group gives "per-department" style reports.

GROUPING

Program.cs

```
var emps = Data.Employees;

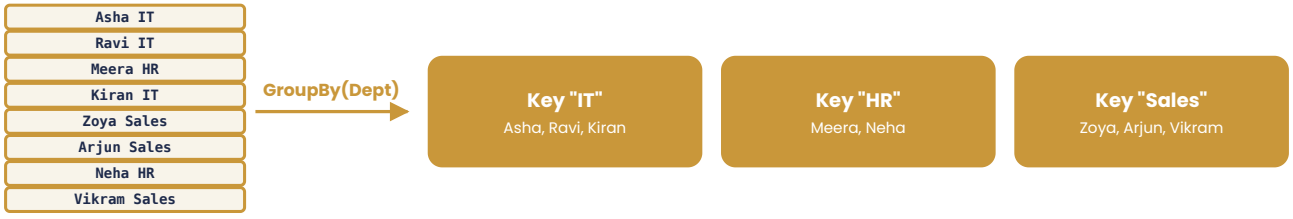
foreach (IGrouping<string, Emp> g in emps.GroupBy(e => e.Dept))
    Console.WriteLine($"{g.Key}: {string.Join(", ", g.Select(e => e.Name))}");

var stats = from e in emps
            group e by e.Dept into g                        // query syntax
            orderby g.Key
            select new { Dept = g.Key, Count = g.Count(), Avg = g.Average(e => e.Salary), Top = g.MaxBy(e
=> e.Salary)!.Name };
foreach (var s in stats) Console.WriteLine($"{s.Dept,-6} {s.Count} staff, avg {s.Avg,9:N2}, top
{s.Top}");

var bands = emps.GroupBy(e => e.Salary >= 70000 ? "70k+" : e.Salary >= 55000 ? "55-70k" : "<55k",
                         e => e.Name);                       // element selector
foreach (var b in bands) Console.WriteLine($"{b.Key}: {string.Join(" ", b)}");
```

OUTPUT

```
IT: Asha, Ravi, Kiran
HR: Meera, Neha
Sales: Zoya, Arjun, Vikram
HR     2 staff, avg 54,500.00, top Meera
IT     3 staff, avg 75,000.00, top Asha
Sales  3 staff, avg 66,333.33, top Vikram
70k+: Asha Ravi Vikram
55-70k: Meera Kiran Zoya
<55k: Arjun Neha
```



EACH GROUP HAS A KEY AND CONTAINS ITS ELEMENTS

16.7.2 Join and GroupJoin

`Join` combines two sequences by matching keys, like an SQL INNER JOIN: each pair of elements with equal keys produces one result, and elements without a partner are dropped. `GroupJoin` pairs each outer element with the **group** of matching inner elements (possibly empty), which is ideal for one-to-many relationships; in query syntax it is `join ... into`.

INNER JOIN AND GROUP JOIN Program.cs

```
var emps = Data.Employees;
var depts = new[]
{
    new { Code = "IT", Title = "Information Technology", Floor = 3 },
    new { Code = "HR", Title = "Human Resources", Floor = 1 },
    new { Code = "FIN", Title = "Finance", Floor = 2 },          // no employees
};

var inner = from e in emps                                        // Join (query syntax)
            join d in depts on e.Dept equals d.Code
            select $"{e.Name}@{d.Floor}";
Console.WriteLine(string.Join(" ", inner));                       // Sales staff dropped: no Sales dept
row

var sameWithMethods = emps.Join(depts, e => e.Dept, d => d.Code, (e, d) => e.Name);
Console.WriteLine(sameWithMethods.Count());

var perDept = depts.GroupJoin(emps, d => d.Code, e => e.Dept,      // GroupJoin: one-to-many
                              (d, staff) => new { d.Title, Names = staff.Select(s => s.Name) });
foreach (var x in perDept) Console.WriteLine($"{x.Title}: {(x.Names.Any() ? string.Join(", ", x.Names) :
"(none)")}");
```

```
OUTPUT
Asha@3 Ravi@3 Meera@1 Kiran@3 Neha@1
5
Information Technology: Asha, Ravi, Kiran
Human Resources: Meera, Neha
Finance: (none)
```

16.7.3 Left joins

A **left outer join** keeps **every** element of the left sequence, with a default (null) when there is no match on the right. Classic LINQ builds it from GroupJoin plus `DefaultIfEmpty()` and SelectMany; in query syntax: `join ... into g from x in g.DefaultIfEmpty()`. .NET 10 adds dedicated `LeftJoin` and `RightJoin` operators that do the same in one call. A self-join on the same list finds each employee's manager, and employees with no manager.

LEFT JOINS Program.cs

```
var emps = Data.Employees;

var withManager = from e in emps                                  // classic left join (self-join)
                  join m in emps on e.ManagerId equals m.Id into mgrs
                  from m in mgrs.DefaultIfEmpty()                  // null when no manager
                  select new { e.Name, Manager = m?.Name ?? "(none)" };
foreach (var x in withManager) Console.Write($"{x.Name}->{x.Manager}  ");
Console.WriteLine();

var noManager = withManager.Where(x => x.Manager == "(none)").Select(x => x.Name);
Console.WriteLine($"no manager: {string.Join(", ", noManager)}");

var viaLeftJoin = emps.LeftJoin(emps, e => e.ManagerId, m => (int?)m.Id,      // .NET 10 LeftJoin
                                (e, m) => $"{e.Name}:{m?.Name ?? "-"}");
Console.WriteLine(string.Join(" ", viaLeftJoin));
```

```
OUTPUT
Asha->(none)  Ravi->Asha  Meera->(none)  Kiran->Asha  Zoya->(none)  Arjun->Zoya  Neha->Meera  Vikram-
>Zoya
no manager: Asha, Meera, Zoya
Asha:- Ravi:Asha Meera:- Kiran:Asha Zoya:- Arjun:Zoya Neha:Meera Vikram:Zoya
```

NOTE

The join key types must match exactly: `e.ManagerId` is `int?`, so the classic query compares it with `m.Id` converted to int? automatically in query syntax, and the method call casts `(int?)m.Id`. Null keys never match anything.

16.7.4 ToLookup

`ToLookup` is the **immediate** version of GroupBy: it runs once and builds an `ILookup<TKey, TElement>`, a dictionary from each key to its group. Unlike a Dictionary, looking up a missing key returns an **empty** sequence rather than throwing. Use it when you will look groups up by key many times.

TOLOOKUP Program.cs

```
var emps = Data.Employees;
ILookup<string, string> byDept = emps.ToLookup(e => e.Dept, e => e.Name);

Console.WriteLine(string.Join(", ", byDept["Sales"]));
Console.WriteLine($"Legal: {byDept["Legal"].Count()} people (no exception)");
Console.WriteLine($"{byDept.Count} keys, contains HR: {byDept.Contains("HR")}");
foreach (var g in byDept) Console.Write($"{g.Key}={g.Count()} ");
Console.WriteLine();
```

OUTPUT

```
Zoya, Arjun, Vikram
Legal: 0 people (no exception)
3 keys, contains HR: True
IT=3 HR=2 Sales=3
```

16.7.5 CountBy and AggregateBy (.NET 9)

Counting or totalling per key used to require `GroupBy(...).Select(g => ...)`, which builds every group in memory. .NET 9 added `CountBy` and `AggregateBy`, which compute one value per key directly and return `KeyValuePair`s. .NET 9 also added `Index()` (see 16.9.2).

COUNTBY AND AGGREGATEBY Program.cs

```
var emps = Data.Employees;

foreach (var (dept, count) in emps.CountBy(e => e.Dept))
    Console.Write($"{dept}:{count} ");
Console.WriteLine();

var payroll = emps.AggregateBy(e => e.Dept, seed: 0m, (total, e) => total + e.Salary);
foreach (var (dept, total) in payroll) Console.Write($"{dept}:{total:N0} ");
Console.WriteLine();

string text = "the cat and the hat and the bat";
var top = text.Split(' ').CountBy(w => w).OrderByDescending(p => p.Value).First();
Console.WriteLine($"most frequent word: '{top.Key}' x{top.Value}");
```

OUTPUT

```
IT:3 HR:2 Sales:3
IT:225,000 HR:109,000 Sales:199,000
most frequent word: 'the' x3
```

16.8 Set operations

16.8.1 Distinct, DistinctBy, Union, Intersect, Except

Set operators treat sequences as sets, using element equality (or a comparer). `Distinct` removes duplicates; `Union` combines two sequences without duplicates; `Intersect` keeps elements present in both; `Except` keeps elements of the first that are not in the second. The **By** variants (.NET 6) compare by a key selector instead of the whole element: `DistinctBy`, `UnionBy`, `IntersectBy`, `ExceptBy`. Results keep the order of first appearance.

SET OPERATORS

Program.cs

```
int[] a = { 1, 2, 2, 3, 4, 4, 5 };
int[] b = { 4, 5, 6, 7 };

Console.WriteLine($"Distinct:  {string.Join(" ", a.Distinct())}");
Console.WriteLine($"Union:     {string.Join(" ", a.Union(b))}");
Console.WriteLine($"Intersect: {string.Join(" ", a.Intersect(b))}");
Console.WriteLine($"Except:    {string.Join(" ", a.Except(b))}");
Console.WriteLine($"Concat:    {string.Join(" ", a.Take(3).Concat(b.Take(2)))}   (keeps duplicates)");

var emps = Data.Employees;
var onePerDept = emps.DistinctBy(e => e.Dept).Select(e => e.Name);        // first employee of each dept
Console.WriteLine($"DistinctBy: {string.Join(", ", onePerDept)}");

int[] promotedIds = { 2, 6 };
var notPromoted = emps.ExceptBy(promotedIds, e => e.Id).Select(e => e.Id);  // by key
Console.WriteLine($"ExceptBy:  {string.Join(",", notPromoted)}");

string[] x = { "Apple", "apple", "Mango" };
Console.WriteLine(string.Join(" ", x.Distinct(StringComparer.OrdinalIgnoreCase)));
```

OUTPUT

```
Distinct:  1 2 3 4 5
Union:     1 2 3 4 5 6 7
Intersect: 4 5
Except:    1 2 3
Concat:    1 2 2 4 5   (keeps duplicates)
DistinctBy: Asha, Meera, Zoya
ExceptBy:  1,3,4,5,7,8
Apple Mango
```

16.9 Partitioning and generation

16.9.1 Skip, Take, SkipWhile, TakeWhile, Chunk

`Take(n)` returns the first n elements and `Skip(n)` everything after them; together they implement **paging**. Ranges work too: `Take(2..5)`, `TakeLast`, `SkipLast`. `TakeWhile` and `SkipWhile` take or skip while a condition holds and then stop checking. `Chunk(size)` (.NET 6) splits a sequence into arrays of at most `size` elements, useful for batching.

PARTITIONING

Program.cs

```
int[] n = Enumerable.Range(1, 10).ToArray();                // 1..10

Console.WriteLine($"Take 3: {string.Join(" ", n.Take(3))} | Skip 7: {string.Join(" ", n.Skip(7))}");
Console.WriteLine($"Take(2..5): {string.Join(" ", n.Take(2..5))} | TakeLast 2: {string.Join(" ",
n.TakeLast(2))}");

int pageSize = 4;
for (int page = 1; page <= 3; page++)
    Console.WriteLine($"page {page}: {string.Join(" ", n.Skip((page - 1) * pageSize).Take(pageSize))}");

int[] temps = { 18, 21, 25, 31, 28, 22, 35 };
Console.WriteLine($"TakeWhile < 30: {string.Join(" ", temps.TakeWhile(t => t < 30))}");
Console.WriteLine($"SkipWhile < 30: {string.Join(" ", temps.SkipWhile(t => t < 30))}");   // stops
checking after 31

foreach (int[] batch in n.Chunk(4)) Console.Write($"[{string.Join(",", batch)}] ");
Console.WriteLine();
```

OUTPUT

```
Take 3: 1 2 3 | Skip 7: 8 9 10
Take(2..5): 3 4 5 | TakeLast 2: 9 10
page 1: 1 2 3 4
page 2: 5 6 7 8
page 3: 9 10
TakeWhile < 30: 18 21 25
SkipWhile < 30: 31 28 22 35
[1,2,3,4] [5,6,7,8] [9,10]
```

16.9.2 Range, Repeat, Zip, Index

`Enumerable.Range(start, count)` generates consecutive integers, `Enumerable.Repeat(value, count)` repeats a value, and `Enumerable.Empty<T>()` gives an empty sequence. `Zip` pairs up elements of two (or three) sequences by position, stopping at the shorter one. `Index()` (.NET 9) pairs each element with its position as a tuple `(Index, Item)`.

GENERATING AND PAIRING SEQUENCES

Program.cs

```
Console.WriteLine(string.Join(" ", Enumerable.Range(5, 4)));                  // 5 6 7 8
Console.WriteLine(string.Join(" ", Enumerable.Range(1, 5).Select(i => i * i)));
Console.WriteLine(string.Join("", Enumerable.Repeat("ab", 3)));

string[] names = { "Asha", "Ravi", "Meera" };
int[] marks = { 91, 78, 85, 60 };                                             // one extra: ignored
foreach (var (name, mark) in names.Zip(marks)) Console.Write($"{name}={mark} ");
Console.WriteLine();
var report = names.Zip(marks, (n, m) => $"{n}:{(m >= 80 ? "A" : "B")}");
Console.WriteLine(string.Join(" ", report));

foreach (var (i, name) in names.Index()) Console.Write($"{i + 1}.{name} ");    // .NET 9
Console.WriteLine();
var table = from r in Enumerable.Range(1, 3) from c in Enumerable.Range(1, 3) select r * c;
Console.WriteLine(string.Join(" ", table));
```

OUTPUT

```
5 6 7 8
1 4 9 16 25
ababab
Asha=91 Ravi=78 Meera=85
Asha:A Ravi:B Meera:A
1.Asha 2.Ravi 3.Meera
1 2 3 2 4 6 3 6 9
```

16.10 Conversion

16.10.1 ToList, ToArray, ToDictionary, ToHashSet

Conversion operators execute the query **immediately** and store the results in a collection: a snapshot that will not change when the source does and that can be enumerated many times cheaply. `ToDictionary` needs a key selector (and optionally a value selector) and throws if two elements produce the same key. `ToHashSet` removes duplicates.

MATERIALISING QUERY RESULTS

Program.cs

```
var emps = Data.Employees;

List<string> names = emps.Select(e => e.Name).ToList();
string[] depts = emps.Select(e => e.Dept).Distinct().ToArray();
Dictionary<int, string> byId = emps.ToDictionary(e => e.Id, e => e.Name);
HashSet<string> deptSet = emps.Select(e => e.Dept).ToHashSet();

Console.WriteLine($"{names.Count} names, depts [{string.Join(",", depts)}], byId[5] = {byId[5]}, set
{deptSet.Count}");

try { emps.ToDictionary(e => e.Dept); }
catch (ArgumentException e) { Console.WriteLine(e.Message); }

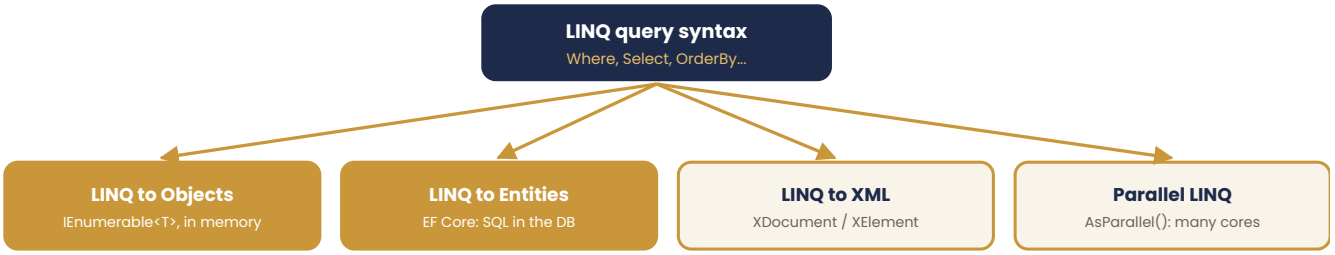
var totals = emps.GroupBy(e => e.Dept).ToDictionary(g => g.Key, g => g.Sum(e => e.Salary));
Console.WriteLine($"Sales total {totals["Sales"]:N0}");
```

OUTPUT

```
8 names, depts [IT,HR,Sales], byId[5] = Zoya, set 3
An item with the same key has already been added. Key: IT
Sales total 199,000
```

16.11 LINQ providers

The same query operators can run against very different data sources. A **LINQ provider** decides what "executing a query" means for its source: looping in memory, generating SQL, walking an XML tree. Your query code looks the same in every case.



ONE QUERY LANGUAGE, SEVERAL PROVIDERS

16.11.1 LINQ to Objects

Every example so far was **LINQ to Objects**: queries over any `IEnumerable<T>` in memory (arrays, lists, dictionaries, strings, iterator methods). The operators are extension methods in `System.Linq.Enumerable`; they take delegates (`Func<T, bool>`) and execute them in your process. **PLINQ** (`AsParallel()`) runs LINQ to Objects queries across several CPU cores for heavy computations.

LINQ TO OBJECTS AND PLINQ — Program.cs

```
string text = "LINQ works on any sequence, even strings";
var vowels = text.ToLower().Where(c => "aeiou".Contains(c)).CountBy(c => c).OrderBy(p => p.Key);
Console.WriteLine(string.Join(" ", vowels.Select(p => $"{p.Key}{p.Value}")));

var files = new Dictionary<string, long> { ["a.pdf"] = 2_400_000, ["b.txt"] = 1_200, ["c.mp4"] =
85_000_000 };
var big = files.Where(f => f.Value > 1_000_000).Select(f => f.Key);
Console.WriteLine(string.Join(", ", big));

long primes = Enumerable.Range(2, 200_000).AsParallel().Count(IsPrime);      // PLINQ: uses all cores
Console.WriteLine($"primes below 200002: {primes}");

static bool IsPrime(int n)
{
    if (n < 2) return false;
    for (int d = 2; (long)d * d <= n; d++) if (n % d == 0) return false;
    return true;
}
```

OUTPUT

```
a1 e5 i2 o2 u1
a.pdf, c.mp4
primes below 200002: 17984
```

16.11.2 LINQ to Entities (EF Core, Module 21)

With **Entity Framework Core**, a `DbSet<T>` such as `db.Students` is a queryable table. A LINQ query against it is **not** executed in C#: EF Core translates the whole expression into SQL, sends it to the database, and materialises the rows into objects. Only the data you ask for crosses the network. Module 21 covers EF Core in detail.

A LINQ TO ENTITIES QUERY AND ITS SQL C#

```
var toppers = await db.Students
    .Where(s => s.Marks >= 85 && s.City == "Pune")
    .OrderByDescending(s => s.Marks)
    .Select(s => new { s.Name, s.Marks })
    .Take(10)
    .ToListAsync();

-- SQL generated by EF Core (SQL Server), roughly:
-- SELECT TOP(10) [s].[Name], [s].[Marks]
-- FROM [Students] AS [s]
-- WHERE [s].[Marks] >= 85 AND [s].[City] = N'Pune'
-- ORDER BY [s].[Marks] DESC
```

16.11.3 LINQ to XML

LINQ to XML (`System.Xml.Linq`) represents XML as an object tree of `XDocument`, `XElement` and `XAttribute`. Trees can be built with **functional construction** (nested constructors, often fed by a LINQ query), loaded with `XElement.Parse` or `XDocument.Load`, and queried with the usual operators plus axis methods such as `Elements()`, `Descendants()` and `Attribute()`.

BUILDING AND QUERYING XML Program.cs

```
using System.Xml.Linq;

var emps = Data.Employees;
XElement xml = new XElement("staff",                                  // functional construction
    from e in emps where e.Dept == "HR"
    select new XElement("employee",
        new XAttribute("id", e.Id),
        new XElement("name", e.Name),
        new XElement("salary", e.Salary)));
Console.WriteLine(xml);

XElement doc = XElement.Parse("""
    <courses>
      <course code="CS101" seats="60"><title>C#</title></course>
      <course code="CS102" seats="40"><title>SQL</title></course>
      <course code="CS201" seats="75"><title>ASP.NET Core</title></course>
    </courses>
    """);
var large = from c in doc.Elements("course")
            where (int)c.Attribute("seats")! >= 50
            select $"{(string)c.Attribute("code")!}:{(string)c.Element("title")!}";
Console.WriteLine(string.Join(", ", large));
```

OUTPUT

```
<staff>
  <employee id="3">
    <name>Meera</name>
    <salary>61000</salary>
  </employee>
  <employee id="7">
    <name>Neha</name>
    <salary>48000</salary>
  </employee>
</staff>
CS101:C#, CS201:ASP.NET Core
```

16.11.4 IEnumerable<T> vs IQueryable<T>

`IEnumerable<T>` queries use `Enumerable` methods that take **delegates**: compiled code that runs in memory. `IQueryable<T>` queries use `Queryable` methods that take **expression trees** (Module 15): data describing the query, which the provider translates (to SQL, for example). The difference matters: once a query is treated as IEnumerable, every following operator runs in memory, so filtering after `AsEnumerable()` or `ToList()` loads the whole table first.

	`IEnumerable<T>`	`IQueryable<T>`
Namespace methods	`System.Linq.Enumerable`	`System.Linq.Queryable`
Lambdas become	delegates (Func)	expression trees (Expression<Func>)
Executes	in your process, in memory	in the provider (database, service)
Typical source	List, array, iterator	EF Core DbSet, OData, remote APIs
Can call any C# method in a lambda	yes	only what the provider can translate

DELEGATES VERSUS EXPRESSION TREES Program.cs

```
var list = Data.Employees;
IEnumerable<Emp> inMemory = list.Where(e => e.Salary > 60000);
IQueryable<Emp> queryable = list.AsQueryable().Where(e => e.Salary > 60000);

Console.WriteLine($"in-memory query is IQueryable: {inMemory is IQueryable<Emp>}");
Console.WriteLine(queryable.Expression.NodeType);               // the query is an expression tree
Console.WriteLine(queryable.Count());                           // provider executes it (here in
memory)
```

```
OUTPUT
in-memory query is IQueryable: False
Call
5
```

WHERE THE FILTER RUNS MATTERS C#

```
// BAD with EF Core: ToList() loads every student row, then filters in memory
var bad = db.Students.ToList().Where(s => s.Marks > 85);

// GOOD: the filter becomes part of the SQL WHERE clause
var good = db.Students.Where(s => s.Marks > 85).ToList();
```

16.12 Performance considerations and multiple enumeration

LINQ is concise, but its convenience hides costs. The most common problem is **multiple enumeration**: a deferred query is re-executed every time it is enumerated. If its source is expensive (a database, a file, a slow computation, an iterator with side effects), calling `Count()`, then `Any()`, then a foreach repeats the work three times. Materialise once with `ToList()` when you need the results more than once. IDEs flag this with the warning "Possible multiple enumeration".

THE COST OF MULTIPLE ENUMERATION

Program.cs

```
int calls = 0;
IEnumerable<int> expensive = Enumerable.Range(1, 5).Select(n => { calls++; return n * n; });

int count = expensive.Count();                  // enumerates (5 calls)
int sum = expensive.Sum();                      // enumerates again (5 more)
bool any = expensive.Any(x => x > 10);          // again, stops at 16 (4 more)
Console.WriteLine($"count {count}, sum {sum}, any {any}: selector ran {calls} times");

calls = 0;
List<int> once = expensive.ToList();            // execute once
Console.WriteLine($"count {once.Count}, sum {once.Sum()}, any {once.Any(x => x > 10)}: selector ran
{calls} times");
```

OUTPUT

```
count 5, sum 55, any True: selector ran 14 times
count 5, sum 55, any True: selector ran 5 times
```

Tip	Why
Materialise (ToList/ToArray) a query you enumerate more than once	avoid repeating work and side effects
Use `Any()` instead of `Count() > 0`	stops at the first element
Filter (Where) before sorting or projecting	less data flows through later stages
Use MinBy/MaxBy instead of OrderBy().First()	O(n) instead of O(n log n)
Use a Dictionary/HashSet/ToLookup for repeated lookups inside a loop	O(1) lookup instead of O(n) Where/First each time
With EF Core, keep queries IQueryable until the end; project only needed columns	filtering and paging happen in the database
In hot paths, a plain loop can still be faster	LINQ allocates enumerators and delegates; measure first

LINQ adds query operators to any data source, in method syntax (extension methods with lambdas) or SQL-like query syntax. Most operators are deferred and run on enumeration; Count, Sum, First, ToList and friends execute immediately. Filter with Where, project with Select and SelectMany, convert types with OfType and Cast, sort with OrderBy/ThenBy/Order, pick elements with First/Single/ElementAt (OrDefault variants avoid exceptions), aggregate with Count/Sum/Average/Aggregate and MinBy/MaxBy, test with Any/All/Contains, group with GroupBy/ToLookup/CountBy/AggregateBy, combine with Join, GroupJoin and left joins, use set and partitioning operators, and materialise with ToList/ToDictionary. IQueryable queries become expression trees that providers like EF Core translate; avoid multiple enumeration of expensive queries.

PRACTICE CHECKPOINT

On a list of employees, use LINQ to find the **top 3 earners per department**, the **average salary by department**, and the **employees with no manager using a left join**.

SAMPLE SOLUTION

Program.cs

```
List<Emp> emps =
[
    new(1, "Asha", "IT", 95000, null),    new(2, "Ravi", "IT", 72000, 1),
    new(3, "Meera", "HR", 61000, null),   new(4, "Kiran", "IT", 58000, 1),
    new(5, "Zoya", "Sales", 67000, null), new(6, "Arjun", "Sales", 52000, 5),
    new(7, "Neha", "HR", 48000, 3),       new(8, "Vikram", "Sales", 80000, 5),
    new(9, "Isha", "IT", 88000, 1),       new(10, "Dev", "Sales", 71000, 5),
    new(11, "Pooja", "HR", 53000, 3),     new(12, "Rahul", "IT", 64000, null),
];

Console.WriteLine("Top 3 earners per department:");
var top3 = from e in emps
           group e by e.Dept into g
           orderby g.Key
           select new { Dept = g.Key, Top = g.OrderByDescending(e => e.Salary).Take(3) };
foreach (var d in top3)
    Console.WriteLine($"  {d.Dept,-6}{string.Join(", ", d.Top.Select(e => $"{e.Name} {e.Salary /
1000}k"))}");

Console.WriteLine("Average salary by department:");
foreach (var (dept, avg) in emps.GroupBy(e => e.Dept)
                                .Select(g => (Dept: g.Key, Avg: g.Average(e => e.Salary)))
                                .OrderByDescending(x => x.Avg))
    Console.WriteLine($"  {dept,-6}{avg,10:N0}");

var noManager = from e in emps
                join m in emps on e.ManagerId equals m.Id into managers
                from m in managers.DefaultIfEmpty()
                where m is null
                select e.Name;
Console.WriteLine($"No manager: {string.Join(", ", noManager)}");

public record Emp(int Id, string Name, string Dept, decimal Salary, int? ManagerId);
```

OUTPUT

```
Top 3 earners per department:
  HR    Meera 61k, Pooja 53k, Neha 48k
  IT    Asha 95k, Isha 88k, Ravi 72k
  Sales Vikram 80k, Dev 71k, Zoya 67k
Average salary by department:
  IT        75,400
  Sales     67,500
  HR        54,000
No manager: Asha, Meera, Zoya, Rahul
```

MODULE 17 · INTERMEDIATE

File handling and serialization

Minimum duration **2 days** | Level **Intermediate**

Data that lives only in variables disappears when the program ends. To keep it, programs write files: logs, reports, settings, saved records. This module covers the System.IO namespace (files, directories and paths), reading and writing text files, streams for binary and in-memory data, an overview of asynchronous file I/O, serialization of objects to JSON with System.Text.Json (and an overview of Newtonsoft.Json, XML and CSV), and watching folders for changes.

IN THIS MODULE

17.1 The System.IO namespace
17.2 Reading and writing text files
17.3 Streams
17.4 Async file I/O (overview)
17.5 Serialization
17.6 Watching the file system

NOTE

The outputs in this module were traced for Linux and macOS, where the directory separator is `/`. On Windows the same programs print `\\` in paths. Examples use relative paths, so files are created in the folder the program runs from (for `dotnet run`, the project folder).

17.1 The System.IO namespace

`System.IO` (imported automatically in modern projects) contains the types for working with the file system. Static classes such as `File`, `Directory` and `Path` offer one-call operations; the instance classes `FileInfo` and `DirectoryInfo` represent one specific file or folder and are more efficient when you perform several operations on the same item.

Type	Purpose
`File` / `FileInfo`	create, read, write, copy, move, delete files; size and timestamps
`Directory` / `DirectoryInfo`	create, list, move and delete folders
`Path`	build and take apart path strings (no disk access)
`StreamReader` / `StreamWriter`	read and write text through a stream
`FileStream`, `MemoryStream`, `BufferedStream`	byte streams over a file, memory, or another stream
`BinaryReader` / `BinaryWriter`	read and write primitive values in binary form
`FileSystemWatcher`	raise events when files change

17.1.1 File and FileInfo

FILE AND FILEINFO

Program.cs

```
File.WriteAllText("hello.txt", "Hello, files!");              // create (or overwrite)
Console.WriteLine($"exists: {File.Exists("hello.txt")}, missing.txt exists:
{File.Exists("missing.txt")}");

File.Copy("hello.txt", "copy.txt", overwrite: true);
File.Move("copy.txt", "renamed.txt", overwrite: true);          // rename or move
Console.WriteLine($"copy.txt: {File.Exists("copy.txt")}, renamed.txt: {File.Exists("renamed.txt")}");

var info = new FileInfo("renamed.txt");                          // one object, many questions
Console.WriteLine($"{info.Name}  ext {info.Extension}  {info.Length} bytes  read-only:
{info.IsReadOnly}");
Console.WriteLine($"modified today: {info.LastWriteTime.Date == DateTime.Today}");

info.Delete();
File.Delete("hello.txt");
File.Delete("does-not-exist.txt");                               // no exception if the file is missing
Console.WriteLine($"after delete: {File.Exists("hello.txt")}");
```

OUTPUT

```
exists: True, missing.txt exists: False
copy.txt: False, renamed.txt: True
renamed.txt  ext .txt  13 bytes  read-only: False
modified today: True
after delete: False
```

WATCH OUT

Checking `File.Exists` and then opening the file is a race: another process may delete it in between. For robust code, simply try the operation and catch `FileNotFoundException`, `UnauthorizedAccessException` or `IOException`.

17.1.2 Directory and DirectoryInfo

WORKING WITH FOLDERS

Program.cs

```
Directory.CreateDirectory("reports/2026/march");              // creates every missing level
File.WriteAllText("reports/summary.txt", "s");
File.WriteAllText("reports/2026/jan.csv", "j");
File.WriteAllText("reports/2026/march/week1.csv", "w1");
File.WriteAllText("reports/2026/march/week2.csv", "w2");

string[] top = Directory.GetFiles("reports");                  // this folder only
Console.WriteLine($"top level: {string.Join(", ", top)}");

var csvs = Directory.EnumerateFiles("reports", "*.csv", SearchOption.AllDirectories)
                    .Select(Path.GetFileName).Order();         // recursive, lazy, sorted for display
Console.WriteLine($"all csv: {string.Join(", ", csvs)}");

var dir = new DirectoryInfo("reports/2026");
Console.WriteLine($"{dir.Name}: {dir.GetDirectories().Length} sub-folder(s), parent {dir.Parent!.Name}");
long total = dir.EnumerateFiles("*", SearchOption.AllDirectories).Sum(f => f.Length);
Console.WriteLine($"bytes under 2026: {total}");

Directory.Delete("reports", recursive: true);                  // delete the folder and its contents
Console.WriteLine($"reports exists: {Directory.Exists("reports")}");
```

OUTPUT

```
top level: reports/summary.txt
all csv: jan.csv, week1.csv, week2.csv
2026: 1 sub-folder(s), parent reports
bytes under 2026: 5
reports exists: False
```

NOTE

The order in which `GetFiles` and `EnumerateFiles` return entries is not guaranteed (it depends on the file system), so sort the results when order matters. Prefer `EnumerateFiles` for large folders: it yields names as they are found instead of building the whole array first.

17.1.3 Path helpers

Never build paths by concatenating strings with "/" or "\\": the separator differs between operating systems and it is easy to get double or missing separators. The `Path` class combines and splits paths correctly. Its methods work purely on strings and never touch the disk.

THE PATH CLASS

Program.cs

```
string path = Path.Combine("data", "2026", "marks.csv");          // OS-correct separators
Console.WriteLine(path);
Console.WriteLine($"file {Path.GetFileName(path)}, name {Path.GetFileNameWithoutExtension(path)}, ext
{Path.GetExtension(path)}");
Console.WriteLine($"folder {Path.GetDirectoryName(path)}, changed {Path.ChangeExtension(path,
".json")}");
Console.WriteLine($"rooted: {Path.IsPathRooted(path)}, {Path.IsPathRooted("/etc/hosts")}");
Console.WriteLine($"separator '{Path.DirectorySeparatorChar}', join: {Path.Join("logs", "app.log")}");

string full = Path.GetFullPath(path);                             // absolute, based on the current
folder
Console.WriteLine($"full path ends with the relative one: {full.EndsWith(path)}");
string temp = Path.Combine(Path.GetTempPath(), Path.GetRandomFileName());   // a unique temp file name
Console.WriteLine($"temp name has extension: {Path.HasExtension(temp)}");
Console.WriteLine($"'/' is invalid in a file name: {Path.GetInvalidFileNameChars().Contains('/')}");
```

OUTPUT

```
data/2026/marks.csv
file marks.csv, name marks, ext .csv
folder data/2026, changed data/2026/marks.json
rooted: False, True
separator '/', join: logs/app.log
full path ends with the relative one: True
temp name has extension: True
'/' is invalid in a file name: True
```

TIP

Special folders such as Documents or AppData are found with `Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments)`; the program's own folder is `AppContext.BaseDirectory`.

17.2 Reading and writing text files

17.2.1 File.ReadAllText, ReadAllLines, WriteAllText, AppendAllText

For small and medium files, the one-call methods of `File` are the simplest choice: they open the file, do the work and close it, even if an exception occurs. Text is UTF-8 by default; an `Encoding` argument selects another.

Method	Effect
`File.WriteAllText(path, text)`	create or overwrite with the text
`File.WriteAllLines(path, lines)`	write each string followed by a newline
`File.AppendAllText(path, text)` / `AppendAllLines`	add to the end (create if missing)
`File.ReadAllText(path)`	the whole file as one string
`File.ReadAllLines(path)`	a string[] of lines (newline characters removed)
`File.ReadAllBytes` / `WriteAllBytes`	the whole file as a byte[]

ONE-CALL TEXT I/O

Program.cs

```
string file = "notes.txt";
File.WriteAllText(file, "Line 1\nLine 2\n");
File.AppendAllText(file, "Line 3\n");
File.AppendAllLines(file, new[] { "Line 4", "Line 5" });

string all = File.ReadAllText(file);
string[] lines = File.ReadAllLines(file);
Console.WriteLine($"{all.Length} chars, {lines.Length} lines, last: {lines[^1]}");

File.WriteAllLines("marks.txt", new[] { "Asha,91", "Ravi,78", "Meera,85" });
var parsed = File.ReadAllLines("marks.txt").Select(l => l.Split(',')).Select(p => (Name: p[0], Mark:
int.Parse(p[1])));
Console.WriteLine($"average {parsed.Average(p => p.Mark):F1}, best {parsed.MaxBy(p => p.Mark).Name}");

File.Delete(file); File.Delete("marks.txt");
```

OUTPUT

```
35 chars, 5 lines, last: Line 5
average 84.7, best Asha
```

17.2.2 StreamReader and StreamWriter

For writing many pieces over time, or reading in controlled steps, use `StreamWriter` and `StreamReader`. They wrap a file stream with a text encoding and a buffer. Always create them in a `using` statement or declaration so the file is flushed and closed; data written to a StreamWriter that is never disposed may stay in its buffer and never reach the disk.

STREAMWRITER AND STREAMREADER

Program.cs

```
using (var writer = new StreamWriter("log.txt"))                // append: new StreamWriter(path,
append: true)
{
    writer.WriteLine("REPORT");
    for (int i = 1; i <= 3; i++) writer.WriteLine($"entry {i}: ok");
    writer.Write("end");                                          // no newline
}                                                                 // flushed and closed here

using (var reader = new StreamReader("log.txt"))
{
    Console.WriteLine($"header: {reader.ReadLine()}");
    string? line;
    while ((line = reader.ReadLine()) != null)                    // null at end of file
        Console.WriteLine($"  read: {line}");
    Console.WriteLine($"at end: {reader.EndOfStream}");
}

using var r2 = new StreamReader("log.txt");
Console.WriteLine($"first char '{(char)r2.Read()}', next peek '{(char)r2.Peek()}', rest length
{r2.ReadToEnd().Length}");
```

OUTPUT

```
header: REPORT
  read: entry 1: ok
  read: entry 2: ok
  read: entry 3: ok
  read: end
at end: True
first char 'R', next peek 'E', rest length 45
```

17.2.3 Reading large files line by line with File.ReadLines

`ReadAllLines` loads the entire file into memory before returning, which fails or is slow for a multi-gigabyte log. `File.ReadLines` returns a **lazy** `IEnumerable<string>` that reads one line at a time as you iterate, so memory use stays constant and LINQ queries can stop early.

STREAMING A LARGE FILE Program.cs

```
using (var w = new StreamWriter("big.log"))                    // simulate a large log file
    for (int i = 1; i <= 100_000; i++)
        w.WriteLine(i % 25_000 == 0 ? $"{i} ERROR disk full" : $"{i} INFO ok");

int errors = File.ReadLines("big.log").Count(l => l.Contains("ERROR"));         // streams the file
string firstError = File.ReadLines("big.log").First(l => l.Contains("ERROR"));  // stops after line 25000
Console.WriteLine($"errors: {errors}, first: {firstError}");

foreach (string line in File.ReadLines("big.log").Where(l => l.Contains("ERROR")).Take(2))
    Console.WriteLine($"  {line}");
File.Delete("big.log");
```

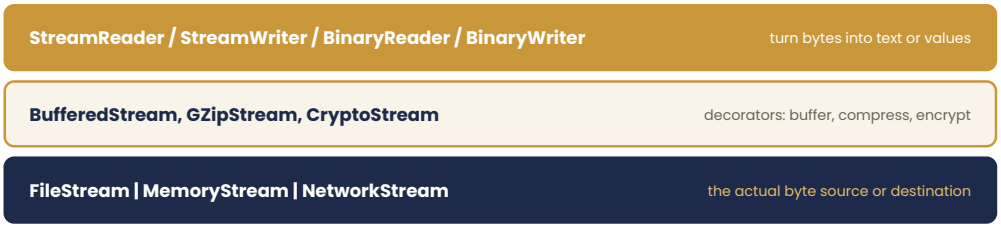
```
OUTPUT
errors: 4, first: 25000 ERROR disk full
  25000 ERROR disk full
  50000 ERROR disk full
```

	`ReadAllLines`	`ReadLines`
Returns	string[] (all lines at once)	IEnumerable<string> (lazy)
Memory	whole file	one line at a time
Can stop early	no, already read everything	yes (First, Take, break)
Enumerate twice	free (it is an array)	reads the file again

17.3 Streams

17.3.1 Stream, FileStream, MemoryStream

A **stream** is a sequence of bytes that can be read, written and possibly repositioned. The abstract class `Stream` defines `Read`, `Write`, `Seek`, `Position`, `Length`, `Flush` and the capabilities `CanRead`, `CanWrite`, `CanSeek`. Concrete streams provide the bytes: `FileStream` (a file), `MemoryStream` (an in-memory buffer), `NetworkStream`, and wrappers such as `GZipStream` and `CryptoStream` that transform another stream. Because all share one base type, code written against `Stream` works with any of them.



STREAMS ARE LAYERED: READERS AND WRITERS SIT ON DECORATORS THAT SIT ON A BASE STREAM

FILESTREAM AND MEMORYSTREAM

Program.cs

```
byte[] data = { 72, 105, 33, 10, 200 };
using (var fs = new FileStream("data.bin", FileMode.Create, FileAccess.Write))
{
    fs.Write(data, 0, data.Length);
    fs.WriteByte(255);
    Console.WriteLine($"written {fs.Length} bytes, position {fs.Position}");
}

using (var fs = new FileStream("data.bin", FileMode.Open, FileAccess.Read))
{
    byte[] buffer = new byte[3];
    int n = fs.Read(buffer, 0, buffer.Length);                // returns how many bytes were read
    Console.WriteLine($"read {n}: {string.Join(" ", buffer)} -> \"
{System.Text.Encoding.ASCII.GetString(buffer)}\"");
    fs.Seek(-1, SeekOrigin.End);                               // jump to the last byte
    Console.WriteLine($"last byte {fs.ReadByte()}, then {fs.ReadByte()} (end of stream)");
}

using var ms = new MemoryStream();                             // a stream in memory
using (var w = new StreamWriter(ms, leaveOpen: true)) w.Write("in memory");
Console.WriteLine($"memory stream: {ms.Length} bytes, CanSeek {ms.CanSeek}");
ms.Position = 0;
Console.WriteLine(new StreamReader(ms).ReadToEnd());
File.Delete("data.bin");
```

OUTPUT

```
written 6 bytes, position 6
read 3: 72 105 33 -> "Hi!"
last byte 255, then -1 (end of stream)
memory stream: 9 bytes, CanSeek True
in memory
```

17.3.2 BinaryReader and BinaryWriter

Text files are readable but large and slow to parse. `BinaryWriter` writes primitive values in their compact binary form (an int is always 4 bytes, a double 8), and `BinaryReader` reads them back. Values must be read in exactly the same order and types as they were written. Strings are stored with a length prefix.

BINARY DATA

Program.cs

```
using (var w = new BinaryWriter(File.Create("student.dat")))
{
    w.Write(101);           // int: 4 bytes
    w.Write("Asha");        // string: 1-byte length prefix + 4 UTF-8 bytes
    w.Write(91.5);          // double: 8 bytes
    w.Write(true);          // bool: 1 byte
}
Console.WriteLine($"file size: {new FileInfo("student.dat").Length} bytes");

using (var r = new BinaryReader(File.OpenRead("student.dat")))
{
    int id = r.ReadInt32();
    string name = r.ReadString();
    double marks = r.ReadDouble();
    bool active = r.ReadBoolean();
    Console.WriteLine($"{id} {name} {marks} {active}");
}
File.Delete("student.dat");
```

OUTPUT

```
file size: 18 bytes
101 Asha 91.5 True
```

17.3.3 Buffered streams

Every read or write on a raw stream can turn into an operating-system call, which is expensive. A **buffer** collects many small operations into a few large ones. `FileStream` already has an internal buffer (4 KB by default, adjustable with the bufferSize argument), and StreamReader/StreamWriter add their own. `BufferedStream` adds buffering to streams that lack it, such as some network streams. Remember to `Flush` or dispose so the last buffered bytes are written.

BUFFERING MANY SMALL WRITES

Program.cs

```
using (var fs = new FileStream("buffered.txt", FileMode.Create, FileAccess.Write, FileShare.None,
bufferSize: 64 * 1024))
using (var buffered = new BufferedStream(fs, 32 * 1024))
{
    for (int i = 0; i < 10_000; i++) buffered.WriteByte((byte)('a' + i % 26));   // 10,000 tiny writes
    Console.WriteLine($"before flush, file length on disk: {new FileInfo("buffered.txt").Length}");
    buffered.Flush();
    Console.WriteLine($"after flush: {new FileInfo("buffered.txt").Length}");
}
File.Delete("buffered.txt");
```

OUTPUT

```
before flush, file length on disk: 0
after flush: 10000
```

17.4 Async file I/O (overview, detailed in Module 18)

File and network operations spend most of their time waiting for the device. **Asynchronous** methods (`ReadAllTextAsync`, `WriteAllTextAsync`, `ReadLinesAsync`, `StreamReader.ReadLineAsync`, `Stream.CopyToAsync`) return a `Task` that you `await`; while the I/O is in progress, the thread is free to do other work, which keeps UIs responsive and lets web servers handle more requests. Module 18 explains async and await in depth.

ASYNCHRONOUS FILE ACCESS

Program.cs

```
await File.WriteAllTextAsync("async.txt", "first line\nsecond line\nthird line\n");
string text = await File.ReadAllTextAsync("async.txt");
Console.WriteLine($"{text.Length} chars read asynchronously");

await foreach (string line in File.ReadLinesAsync("async.txt"))      // .NET 7+: lazy and async
    Console.WriteLine($"  {line}");

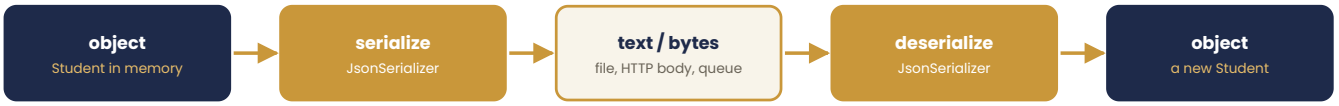
await using (var fs = File.OpenRead("async.txt"))
await using (var copy = File.Create("async-copy.txt"))
    await fs.CopyToAsync(copy);
Console.WriteLine($"copy size {new FileInfo("async-copy.txt").Length}");
File.Delete("async.txt"); File.Delete("async-copy.txt");
```

OUTPUT

```
34 chars read asynchronously
  first line
  second line
  third line
copy size 34
```

17.5 Serialization

Serialization converts an object into a format that can be stored or sent (text such as JSON, XML or CSV, or bytes); **deserialization** rebuilds the object from that format. It is how applications save settings and documents, call web APIs, cache data and exchange messages.



SERIALIZATION ROUND TRIP

17.5.1 JSON with System.Text.Json: JsonSerializer.Serialize and Deserialize

`System.Text.Json` is the built-in, high-performance JSON library of .NET. `JsonSerializer.Serialize(obj)` returns a JSON string from an object's public properties; `JsonSerializer.Deserialize<T>(json)` creates a T from JSON. Records and classes with constructors are supported, as are collections, dictionaries, nullable values, `DateTime`, `DateOnly` and nested objects. By default property names are written as declared and reading is case-sensitive.

SERIALIZE AND DESERIALIZE

Program.cs

```
using System.Text.Json;

var s = new Student(101, "Asha", 91.5, ["C#", "SQL"], new DateOnly(2026, 3, 9));

string compact = JsonSerializer.Serialize(s);
Console.WriteLine(compact);

string pretty = JsonSerializer.Serialize(s, new JsonSerializerOptions { WriteIndented = true });
Console.WriteLine(pretty);

Student back = JsonSerializer.Deserialize<Student>(compact)!;
Console.WriteLine($"{back.Name} joined {back.Joined:dd MMM yyyy}, courses {string.Join("+",
back.Courses)}");

File.WriteAllText("student.json", pretty);                                    // save...
var loaded = JsonSerializer.Deserialize<Student>(File.ReadAllText("student.json"));   // ...and load
Console.WriteLine($"loaded from file: {loaded?.Id} {loaded?.Name}");

var list = JsonSerializer.Deserialize<List<int>>("[3, 1, 4]")!;               // any type
Console.WriteLine(list.Sum());
File.Delete("student.json");

record Student(int Id, string Name, double Marks, List<string> Courses, DateOnly Joined);
```

OUTPUT

```
{"Id":101,"Name":"Asha","Marks":91.5,"Courses":["C#","SQL"],"Joined":"2026-03-09"}
{
  "Id": 101,
  "Name": "Asha",
  "Marks": 91.5,
  "Courses": [
    "C#",
    "SQL"
  ],
  "Joined": "2026-03-09"
}
Asha joined 09 Mar 2026, courses C#+SQL
loaded from file: 101 Asha
8
```

TIP

Create one `JsonSerializerOptions` instance and reuse it (for example in a static field); building new options for every call wastes the metadata cache. For the best performance and trimming support, **source generation** (`[JsonSerializable(typeof(Student))]` on a partial JsonSerializerContext) generates the serialization code at compile time.

17.5.2 Serialization options and attributes (JsonPropertyName, JsonIgnore)

Options control the format globally; attributes customise individual properties.

Option / attribute	Effect
`WriteIndented = true`	pretty-print with indentation
`PropertyNamingPolicy = JsonNamingPolicy.CamelCase`	Name becomes name (also SnakeCaseLower, KebabCaseLower)
`PropertyNameCaseInsensitive = true`	accept NAME, name, Name when reading
`DefaultIgnoreCondition = WhenWritingNull`	omit null properties

Option / attribute	Effect
`Converters = { new JsonStringEnumConverter() }`	write enums as names instead of numbers
`new JsonSerializerOptions(JsonSerializerDefaults.Web)`	camelCase, case-insensitive, numbers may be quoted: the ASP.NET Core defaults
`[JsonPropertyName("x")]`	use this exact JSON name for the property
`[JsonIgnore]`	never serialize the property
`[JsonRequired]`	deserialization fails if the property is missing
`[JsonPropertyOrder(n)]`	control the output order

OPTIONS AND ATTRIBUTES

Program.cs

```
using System.Text.Json;
using System.Text.Json.Serialization;

var options = new JsonSerializerOptions
{
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
    DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull,
    Converters = { new JsonStringEnumConverter() }
};

var pen = new Product { Name = "Pen", Price = 10.5m, InternalCode = "SECRET-1", Category =
Category.Stationery };
Console.WriteLine(JsonSerializer.Serialize(pen, options));

string fromApi = """{ "ID": 7, "NAME": "Ravi", "Marks": "78.5" }""";
var web = new JsonSerializerOptions(JsonSerializerDefaults.Web);              // case-insensitive,
quoted numbers OK
var r = JsonSerializer.Deserialize<Result>(fromApi, web)!;
Console.WriteLine(r);

try { JsonSerializer.Deserialize<Result>(fromApi); }                          // default options: strict
catch (JsonException e) { Console.WriteLine($"strict read failed: {e.GetType().Name}"); }

enum Category { Stationery, Books, Electronics }

class Product
{
    [JsonPropertyName("product_name")] public string Name { get; set; } = "";
    public decimal Price { get; set; }
    [JsonIgnore] public string InternalCode { get; set; } = "";
    public string? Description { get; set; }                                  // null: omitted
    public Category Category { get; set; }
}

record Result(int Id, string Name, double Marks);
```

OUTPUT

```
{"product_name":"Pen","price":10.5,"category":"Stationery"}
Result { Id = 7, Name = Ravi, Marks = 78.5 }
strict read failed: JsonException
```

NOTE

With default options, "ID" and "NAME" do not match Id and Name (reading is case-sensitive) and are simply skipped; reading the quoted number "78.5" into the double Marks is what throws the JsonException. For quick, schema-free access, `JsonNode.Parse(json)["name"]` and `JsonDocument` read JSON without defining classes.

17.5.3 Newtonsoft.Json (overview)

Newtonsoft.Json (Json.NET, package Newtonsoft.Json) was the standard JSON library for .NET for over a decade and is still found in many existing projects. Its API is similar: `JsonConvert.SerializeObject` and `JsonConvert.DeserializeObject<T>`, with attributes `[JsonProperty]` and `[JsonIgnore]` from the Newtonsoft namespace, and the dynamic `JObject` API. It is more permissive and feature-rich (comments, loose parsing, many converters), while System.Text.Json is faster, allocates less and is built in. New projects should use System.Text.Json unless they need a Newtonsoft-only feature.

NEWTONSOFT.JSON AT A GLANCE C#

```
// dotnet add package Newtonsoft.Json
using Newtonsoft.Json;
using Newtonsoft.Json.Linq;

string json = JsonConvert.SerializeObject(student, Formatting.Indented);
Student s = JsonConvert.DeserializeObject<Student>(json)!;

JObject obj = JObject.Parse(json);              // dynamic access
string name = (string)obj["Name"]!;
```

17.5.4 XML serialization (overview)

`System.Xml.Serialization.XmlSerializer` converts public classes with a parameterless constructor and public read/write properties to and from XML. Attributes such as `[XmlElement]`, `[XmlAttribute]`, `[XmlIgnore]` and `[XmlRoot]` shape the document. XML is still common in configuration files, SOAP services, office formats and government or banking data exchange. For hand-built or queried XML, LINQ to XML (Module 16) is usually simpler.

XMLSERIALIZER ROUND TRIP Program.cs

```
using System.Xml.Serialization;

var course = new Course { Code = "CS101", Title = "C# Programming", Seats = 60, Topics = ["OOP", "LINQ"]
};
var serializer = new XmlSerializer(typeof(Course));

using (var writer = new StreamWriter("course.xml")) serializer.Serialize(writer, course);
string xml = File.ReadAllText("course.xml");
Console.WriteLine($"has code attribute: {xml.Contains("code=\"CS101\"")}, has <Title>: {xml.Contains("
<Title>C# Programming</Title>")}");

using (var reader = new StreamReader("course.xml"))
{
    var back = (Course)serializer.Deserialize(reader)!;
    Console.WriteLine($"{back.Code} {back.Title} {back.Seats} [{string.Join(", ", back.Topics)}]");
}
File.Delete("course.xml");

public class Course
{
    [XmlAttribute("code")] public string Code { get; set; } = "";
    public string Title { get; set; } = "";
    public int Seats { get; set; }
    [XmlArrayItem("Topic")] public List<string> Topics { get; set; } = new();
}
```

OUTPUT

```
has code attribute: True, has <Title>: True
CS101 C# Programming 60 [OOP, LINQ]
```

17.5.5 CSV handling

CSV (comma-separated values) is the lingua franca of spreadsheets and data exports. Simple CSV can be written with string.Join and read with Split, but real data contains commas, quotes and line breaks inside values. The rule (RFC 4180): a field containing a comma, quote or newline is wrapped in double quotes, and a quote inside it is doubled. For production use, a library such as **CsvHelper** handles all edge cases and maps rows to classes automatically.

WRITING AND READING CSV SAFELY

Program.cs

```
var rows = new[]
{
    (Id: 101, Name: "Asha", City: "Pune, MH", Marks: 91.5),
    (Id: 102, Name: "Ravi \"RK\" Kumar", City: "Delhi", Marks: 78.0),
};

var lines = new List<string> { "Id,Name,City,Marks" };
lines.AddRange(rows.Select(r => string.Join(",", r.Id, Esc(r.Name), Esc(r.City), r.Marks)));
File.WriteAllLines("students.csv", lines);
Console.WriteLine(File.ReadAllText("students.csv").TrimEnd());

foreach (string line in File.ReadLines("students.csv").Skip(1))            // skip the header
    Console.WriteLine(string.Join(" | ", ParseLine(line)));
File.Delete("students.csv");

static string Esc(string v) =>
    v.IndexOfAny(new[] { ',', '"', '\n' }) >= 0 ? $"\"{v.Replace("\"", "\"\"")}\"" : v;

static List<string> ParseLine(string line)                                  // handles quoted fields
{
    var fields = new List<string>();
    var current = new System.Text.StringBuilder();
    bool inQuotes = false;
    for (int i = 0; i < line.Length; i++)
    {
        char c = line[i];
        if (inQuotes)
        {
            if (c == '"' && i + 1 < line.Length && line[i + 1] == '"') { current.Append('"'); i++; }   //
"" -> "
            else if (c == '"') inQuotes = false;
            else current.Append(c);
        }
        else if (c == '"') inQuotes = true;
        else if (c == ',') { fields.Add(current.ToString()); current.Clear(); }
        else current.Append(c);
    }
    fields.Add(current.ToString());
    return fields;
}
```

OUTPUT

```
Id,Name,City,Marks
101,Asha,"Pune, MH",91.5
102,"Ravi ""RK"" Kumar",Delhi,78
101 | Asha | Pune, MH | 91.5
102 | Ravi "RK" Kumar | Delhi | 78
```

NOTE

Numbers are written with the current culture by string.Join. In cultures that use a comma as the decimal separator (de-DE, fr-FR), 91.5 would become "91,5" and break the file, so production code should format numbers with `CultureInfo.InvariantCulture`.

17.6 Watching the file system

17.6.1 FileSystemWatcher

`FileSystemWatcher` monitors a folder and raises events when files are created, changed, renamed or deleted. It is used for hot-reloading configuration, processing files dropped into an "inbox" folder, and synchronisation tools. Set `Path`, an optional `Filter` such as "*.csv", `NotifyFilter` for which changes matter, and `IncludeSubdirectories`; subscribe to the events; then set `EnableRaisingEvents = true`.

WATCHING A FOLDER FOR CSV FILES C#

```
using var watcher = new FileSystemWatcher("inbox")
{
    Filter = "*.csv",
    NotifyFilter = NotifyFilters.FileName | NotifyFilters.LastWrite | NotifyFilters.Size,
    IncludeSubdirectories = false
};

watcher.Created += (_, e) => Console.WriteLine($"created: {e.Name}");
watcher.Changed += (_, e) => Console.WriteLine($"changed: {e.Name}");
watcher.Renamed += (_, e) => Console.WriteLine($"renamed: {e.OldName} -> {e.Name}");
watcher.Deleted += (_, e) => Console.WriteLine($"deleted: {e.Name}");
watcher.Error   += (_, e) => Console.WriteLine($"error: {e.GetException().Message}");

watcher.EnableRaisingEvents = true;             // start watching
Console.WriteLine("Watching inbox/ for CSV files. Press Enter to stop.");
Console.ReadLine();
```

Example session (events depend on how other programs write the files)

```
Watching inbox/ for CSV files. Press Enter to stop.
created: orders.csv
changed: orders.csv
changed: orders.csv
renamed: orders.csv -> orders-2026-03.csv
deleted: orders-2026-03.csv
```

WATCH OUT

Events arrive on thread-pool threads, and one save by an editor can raise several Changed events (or a Created plus Renamed sequence), so handlers should be idempotent and quick. Under heavy activity the internal buffer can overflow and events are lost (the Error event reports it). A file that has just been created may still be locked by the program writing it; retry briefly before reading.

System.IO provides File/FileInfo and Directory/DirectoryInfo for disk operations and Path for building portable paths. Use the File.ReadAll/WriteAll/Append methods for small text files, StreamReader/StreamWriter inside using for incremental text I/O, and File.ReadLines for huge files. Streams (FileStream, MemoryStream, buffered and decorating streams) move bytes; BinaryReader/Writer store values compactly; async methods free threads while waiting for I/O. System.Text.Json serializes objects to JSON with options and attributes for naming, ignoring and enums; Newtonsoft.Json, XmlSerializer and CSV (quote fields properly or use CsvHelper) cover other formats. FileSystemWatcher raises events when files change.

PRACTICE CHECKPOINT

Extend the **Student Management System** to **save and load students as JSON** and to **export a CSV report**.

SAMPLE SOLUTION

Program.cs

```
using System.Globalization;
using System.Text;
using System.Text.Json;

var store = new StudentStore("students.json");
store.Add(new Student(101, "Asha", "CSE", 91.5));
store.Add(new Student(102, "Ravi", "ECE", 78));
store.Add(new Student(103, "Meera, K", "CSE", 85.25));
store.Save();
Console.WriteLine(File.ReadAllText("students.json"));

var reloaded = new StudentStore("students.json");
reloaded.Load();
Console.WriteLine($"loaded {reloaded.All.Count} students, top: {reloaded.All.MaxBy(s =>
s.Marks)!.Name}");

reloaded.ExportCsv("report.csv");
Console.WriteLine(File.ReadAllText("report.csv").TrimEnd());

public record Student(int Id, string Name, string Dept, double Marks);

public class StudentStore(string path)
{
    private static readonly JsonSerializerOptions Options = new() { WriteIndented = true,
PropertyNamingPolicy = JsonNamingPolicy.CamelCase };
    private List<Student> students = new();
    public IReadOnlyList<Student> All => students;

    public void Add(Student s) => students.Add(s);

    public void Save() => File.WriteAllText(path, JsonSerializer.Serialize(students, Options));

    public void Load() =>
        students = File.Exists(path) ? JsonSerializer.Deserialize<List<Student>>(File.ReadAllText(path),
Options) ?? new() : new();

    public void ExportCsv(string csvPath)
    {
        var sb = new StringBuilder("Id,Name,Dept,Marks,Result\n");
        foreach (var s in students.OrderByDescending(s => s.Marks))
        {
            string name = s.Name.Contains(',') ? $"\"{s.Name}\"" : s.Name;
            string marks = s.Marks.ToString("F2", CultureInfo.InvariantCulture);
            sb.Append($"{s.Id},{name},{s.Dept},{marks},{(s.Marks >= 80 ? "Distinction" : "Pass")}\n");
        }
        File.WriteAllText(csvPath, sb.ToString());
    }
}
```

OUTPUT

```
[
  {
    "id": 101,
    "name": "Asha",
    "dept": "CSE",
    "marks": 91.5
  },
  {
    "id": 102,
    "name": "Ravi",
    "dept": "ECE",
    "marks": 78
  },
  {
    "id": 103,
    "name": "Meera, K",
    "dept": "CSE",
    "marks": 85.25
  }
]
loaded 3 students, top: Asha
Id,Name,Dept,Marks,Result
101,Asha,CSE,91.50,Distinction
103,"Meera, K",CSE,85.25,Distinction
102,Ravi,ECE,78.00,Pass
```

LEVEL 3

Advanced level

Outcome: async and multithreading, advanced language features, CLR internals, EF Core, ASP.NET Core Web APIs, testing, design patterns and interview readiness.

7 MODULES · 29 DAYS MINIMUM

Module 18 · Asynchronous programming and multithreading	**5 days**
Module 19 · Advanced C# language features	**4 days**
Module 20 · .NET runtime internals and memory management	**2 days**
Module 21 · Data access with ADO.NET and Entity Framework Core	**4 days**
Module 22 · Web development with ASP.NET Core	**6 days**
Module 23 · Testing, debugging, design patterns and best practices	**3 days**
Module 24 · Platforms, data structures, projects and interviews	**5 days**

MODULE 18 · ADVANCED

Asynchronous programming and multithreading

Minimum duration **5 days** | Level **Advanced**

Modern applications must stay responsive while they wait for networks and disks, and must use every CPU core for heavy computation. This module covers the concepts (processes, threads, concurrency, parallelism, asynchrony, I/O-bound versus CPU-bound work), raw threads and the thread pool, synchronisation with lock, the C# 13 Lock type, Monitor, Mutex, semaphores and Interlocked, deadlocks, the Task Parallel Library, async and await in depth (return types, async void, deadlocks, ConfigureAwait, cancellation, progress, async streams and exceptions), concurrent collections and Channels, and timers.

IN THIS MODULE

18.1 Concepts
18.2 Threads
18.3 Synchronisation
18.4 Task Parallel Library (TPL)
18.5 async and await
18.6 Concurrent collections
18.7 Timers

NOTE

Concurrent programs are not fully deterministic: the operating system decides when each thread runs. The examples in this module are written so that their printed output is predictable (results are collected and printed in a fixed order, and timings are printed as comparisons rather than exact milliseconds). Where a result can vary between runs, the text says so.

18.1 Concepts

18.1.1 Process vs thread

A **process** is a running program with its own isolated memory, handles and security context; one process cannot read another's memory directly. A **thread** is a path of execution inside a process. Every process starts with one thread (the main thread) and can create more; all threads of a process **share** its heap memory, which makes communication cheap but requires care, since two threads may change the same data at the same time. Each thread has its own stack and instruction pointer.

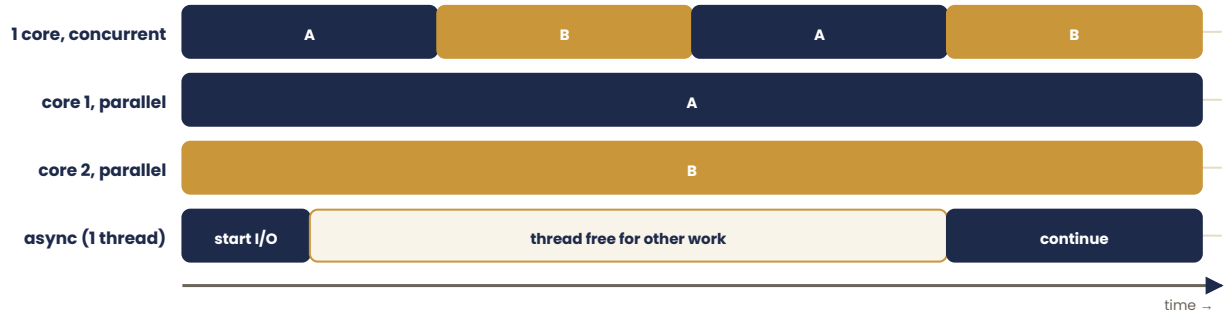


THREADS SHARE THEIR PROCESS'S HEAP; PROCESSES ARE ISOLATED FROM EACH OTHER

	Process	Thread
Memory	own, isolated address space	shares the process heap; own stack (about 1 MB)
Creation cost	high (milliseconds)	lower, but still significant: reuse via the thread pool
Communication	pipes, sockets, files, shared memory	shared objects (needs synchronisation)
Failure	a crash affects only that process	an unhandled exception in any thread ends the whole process
C# types	`System.Diagnostics.Process`	`Thread`, `ThreadPool`, `Task`

18.1.2 Concurrency vs parallelism vs asynchrony

These three words are often mixed up. **Concurrency** means several tasks are in progress during the same period, possibly by interleaving on a single core. **Parallelism** means tasks literally run at the same instant on different cores, to finish CPU work faster. **Asynchrony** means starting an operation and not blocking while it completes: the thread is released and the code continues when the result is ready. A web server handling many requests with async I/O is concurrent and asynchronous even on one core; an image filter split across eight cores is parallel.



INTERLEAVING, SIMULTANEOUS EXECUTION, AND NON-BLOCKING WAITING

18.1.3 I/O-bound vs CPU-bound work

Choosing the right tool depends on what the work spends its time on. **I/O-bound** work mostly waits: for a web service, a database, a disk. Adding threads does not make the network faster; the right tool is **async/await**, which waits without occupying a thread. **CPU-bound** work keeps a core busy computing: image processing, compression, simulations. The right tool is **parallelism** (Task.Run, Parallel, PLINQ) to spread it across cores, while keeping the UI or request thread free.

	I/O-bound	CPU-bound
Spends time	waiting for devices or remote systems	executing instructions
Examples	HTTP calls, SQL queries, file reads	hashing, sorting millions of items, rendering
Use	`await` the async API (`GetStringAsync`, `ToListAsync`)	`Task.Run`, `Parallel.For`, `AsParallel()`
Avoid	wrapping sync I/O in Task.Run on servers	running it on the UI thread

18.2 Threads

18.2.1 Thread class, Start, Join, Sleep

`System.Threading.Thread` represents an operating-system thread. Construct it with a method (or lambda) to run, call `Start()` to begin, and `Join()` to wait until it finishes. `Thread.Sleep(ms)` pauses the current thread. Creating threads directly is the lowest-level approach; modern code almost always uses tasks (18.4), but understanding threads explains everything built on top of them.

STARTING AND JOINING THREADS Program.cs

```
int[] results = new int[3];

var workers = new Thread[3];
for (int i = 0; i < 3; i++)
{
    int id = i;                                     // capture a copy for the lambda
    workers[i] = new Thread(() =>
    {
        Thread.Sleep(100 * (3 - id));               // simulate work of different lengths
        results[id] = (id + 1) * 100;               // each thread writes its own slot
    });
    workers[i].Name = $"worker-{id}";
    workers[i].Start();                             // runs concurrently from now on
}
Console.WriteLine($"main thread continues while {workers.Count(w => w.IsAlive)} workers run");

foreach (Thread t in workers) t.Join();             // wait for every worker
Console.WriteLine($"all done: {string.Join(", ", results)}");
Console.WriteLine($"{workers[0].Name} alive: {workers[0].IsAlive}");
```

OUTPUT

```
main thread continues while 3 workers run
all done: 100, 200, 300
worker-0 alive: False
```

18.2.2 Foreground vs background threads

A **foreground** thread keeps the process alive: the program does not exit until all foreground threads have finished. A **background** thread (`IsBackground = true`) is terminated abruptly when the last foreground thread ends, so it suits helper work that may be abandoned, such as a heartbeat or cache refresher. Threads created with `new Thread` are foreground by default; thread-pool threads (and therefore tasks) are background.

THE PROCESS WAITS ONLY FOR FOREGROUND THREADS Program.cs

```
var fg = new Thread(() => { Thread.Sleep(200); Console.WriteLine("foreground thread finished"); });
var bg = new Thread(() => { Thread.Sleep(5000); Console.WriteLine("never printed"); }) { IsBackground =
true };

Console.WriteLine($"fg background: {fg.IsBackground}, bg background: {bg.IsBackground}");
fg.Start();
bg.Start();
Console.WriteLine($"pool thread is background: {Task.Run(() =>
Thread.CurrentThread.IsBackground).Result}");
Console.WriteLine("Main ends now; the process waits for fg but not for bg");
```

OUTPUT

```
fg background: False, bg background: True
pool thread is background: True
Main ends now; the process waits for fg but not for bg
foreground thread finished
```

18.2.3 The thread pool

Creating and destroying a thread for every small job is wasteful. The **thread pool** keeps a set of reusable background worker threads; work items are queued and executed by whichever worker is free, and the pool adds or removes threads based on load. `ThreadPool.QueueUserWorkItem` queues work directly, but in practice `Task.Run` is the standard way to use the pool. Avoid blocking pool threads for long periods (with Thread.Sleep or synchronous I/O), because the pool then has to inject extra threads slowly, which is called **thread-pool starvation**.

QUEUING WORK TO THE THREAD POOL Program.cs

```
var squares = new int[5];
using var done = new CountdownEvent(squares.Length);       // signals when the count reaches zero

for (int i = 0; i < squares.Length; i++)
{
    ThreadPool.QueueUserWorkItem(state =>
    {
        int n = (int)state!;
        squares[n] = n * n;
        done.Signal();
    }, i);
}
done.Wait();                                               // block until all 5 items signalled
Console.WriteLine(string.Join(" ", squares));

ThreadPool.GetMinThreads(out int minWorkers, out _);
Console.WriteLine($"pool keeps at least as many workers as cores: {minWorkers >=
Environment.ProcessorCount}");
```

OUTPUT

```
0 1 4 9 16
pool keeps at least as many workers as cores: True
```

18.3 Synchronisation

18.3.1 Race conditions

A **race condition** occurs when the result depends on the unpredictable timing of threads. The classic case is `count++` from several threads. It looks like one step but is three: read count, add one, write it back. If two

threads read the same value before either writes, one increment is lost. Races are dangerous because they appear rarely, under load, and disappear when you debug.

TWO THREADS INCREMENTING ONE COUNTER Program.cs

```
int unsafeCount = 0;
var t1 = new Thread(() => { for (int i = 0; i < 1_000_000; i++) unsafeCount++; });
var t2 = new Thread(() => { for (int i = 0; i < 1_000_000; i++) unsafeCount++; });
t1.Start(); t2.Start();
t1.Join(); t2.Join();

Console.WriteLine($"expected 2,000,000; unsafe result correct: {unsafeCount == 2_000_000}");
Console.WriteLine($"lost updates: {2_000_000 - unsafeCount > 0}");
```

OUTPUT

```
expected 2,000,000; unsafe result correct: False
lost updates: True
```

The exact count changes from run to run (a typical run on a multi-core machine gives somewhere around 1.1 to 1.6 million). On rare runs, or on a single-core machine, the threads may not overlap and the result can happen to be correct, which is exactly what makes races hard to find.



BOTH THREADS READ 41 AND BOTH WRITE 42: ONE INCREMENT IS LOST

18.3.2 lock statement and the Lock type (C# 13)

The `lock` statement allows only one thread at a time into a block of code (a **critical section**). A thread that reaches a lock held by another thread waits until it is released. Lock on a private, read-only object dedicated to that purpose, never on `this`, a type or a string, which other code could also lock. Keep the locked section short and never `await` inside it (the compiler forbids it). C# 13 with .NET 9 introduces `System.Threading.Lock`, a dedicated lock type that is faster than locking on a plain object; the `lock` statement recognises it automatically.

LOCK MAKES THE COUNTER CORRECT

Program.cs

```
var counter = new SafeCounter();
var threads = Enumerable.Range(0, 4).Select(_ => new Thread(() =>
{
    for (int i = 0; i < 250_000; i++) counter.Increment();
})).ToList();
threads.ForEach(t => t.Start());
threads.ForEach(t => t.Join());
Console.WriteLine($"safe count: {counter.Value:N0}");

var account = new Account(1000);
Parallel.For(0, 10, _ => account.Withdraw(150));          // 10 attempts, only 6 can succeed
Console.WriteLine($"balance {account.Balance}, successful withdrawals {account.Successes}");

class SafeCounter
{
    private readonly Lock gate = new();                    // C# 13 / .NET 9 Lock type
    private int value;
    public int Value { get { lock (gate) return value; } }
    public void Increment() { lock (gate) value++; }       // one thread at a time
}

class Account(int balance)
{
    private readonly object sync = new();                  // classic: lock on a private object
    public int Balance => balance;
    public int Successes { get; private set; }

    public void Withdraw(int amount)
    {
        lock (sync)                                        // check-then-act must be atomic
        {
            if (balance < amount) return;
            balance -= amount;
            Successes++;
        }
    }
}
```

```
OUTPUT
safe count: 1,000,000
balance 100, successful withdrawals 6
```

18.3.3 Monitor, Mutex, Semaphore, SemaphoreSlim

The `lock` statement compiles to `Monitor.Enter` and `Monitor.Exit` in a try/finally. Monitor also offers `TryEnter` with a timeout and `Wait` / `Pulse` for signalling. A `Mutex` is like a lock but can be **named** and shared between processes, for example to ensure only one instance of an application runs. A **semaphore** limits how many threads may enter at once (N instead of 1): `Semaphore` is the cross-process OS version and `SemaphoreSlim` the lightweight in-process version with async support (`WaitAsync`), the usual way to throttle concurrent HTTP calls or database connections.

Primitive	Allows at once	Cross-process	Async-friendly	Typical use
`lock` / `Monitor`	1	no	no	protect shared fields
`Lock` (.NET 9)	1	no	no	same, faster
`Mutex`	1	yes (named)	no	single-instance apps
`Semaphore`	N	yes (named)	no	limit resource use across processes

Primitive	Allows at once	Cross-process	Async-friendly	Typical use
`SemaphoreSlim`	N	no	yes (WaitAsync)	throttle async operations

MONITOR, MUTEX AND SEMAPHORESLIM

Program.cs

```
var sync = new object();
bool gotIt = Monitor.TryEnter(sync, TimeSpan.FromMilliseconds(100));    // with a timeout
try { Console.WriteLine($"Monitor.TryEnter: {gotIt}"); }
finally { if (gotIt) Monitor.Exit(sync); }

using var mutex = new Mutex(initiallyOwned: false, name: "LBF-Demo-SingleInstance");
bool first = mutex.WaitOne(0);                                  // false if another instance holds it
Console.WriteLine($"first instance: {first}");
if (first) mutex.ReleaseMutex();

var throttle = new SemaphoreSlim(3);                            // at most 3 at a time
int running = 0, maxSeen = 0;
var jobs = Enumerable.Range(1, 10).Select(async i =>
{
    await throttle.WaitAsync();
    try
    {
        int now = Interlocked.Increment(ref running);
        InterlockedMax(ref maxSeen, now);
        await Task.Delay(50);                                   // simulated I/O
        Interlocked.Decrement(ref running);
    }
    finally { throttle.Release(); }
});
await Task.WhenAll(jobs);
Console.WriteLine($"10 jobs done, never more than 3 at once: {maxSeen <= 3}");

static void InterlockedMax(ref int target, int value)
{
    int current;
    while ((current = target) < value && Interlocked.CompareExchange(ref target, value, current) !=
current) { }
}
```

OUTPUT

```
Monitor.TryEnter: True
first instance: True
10 jobs done, never more than 3 at once: True
```

18.3.4 Interlocked

For single operations on a number or reference, locks are overkill. The `Interlocked` class performs them **atomically** using special CPU instructions: `Increment`, `Decrement`, `Add`, `Exchange` (swap in a new value) and `CompareExchange` (set a new value only if the current value equals an expected one, the building block of lock-free algorithms).

ATOMIC OPERATIONS WITH INTERLOCKED Program.cs

```
int hits = 0;
long bytes = 0;
Parallel.For(0, 100_000, i =>
{
    Interlocked.Increment(ref hits);                // atomic ++
    Interlocked.Add(ref bytes, 512);                // atomic +=
});
Console.WriteLine($"hits {hits:N0}, bytes {bytes:N0}");

string status = "idle";
string old = Interlocked.Exchange(ref status, "busy");                    // swap, return old
Console.WriteLine($"was {old}, now {status}");

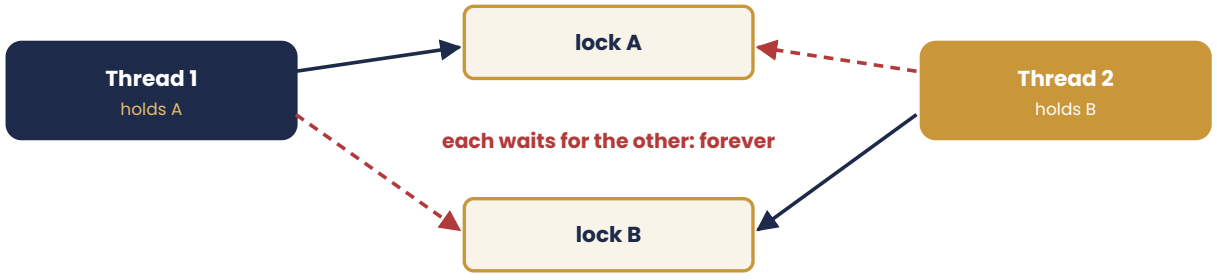
int flag = 0;
bool won1 = Interlocked.CompareExchange(ref flag, 1, 0) == 0;             // 0 -> 1: succeeds
bool won2 = Interlocked.CompareExchange(ref flag, 1, 0) == 0;             // already 1: fails
Console.WriteLine($"first claim {won1}, second claim {won2}");
```

OUTPUT

```
hits 100,000, bytes 51,200,000
was idle, now busy
first claim True, second claim False
```

18.3.5 Deadlocks

A **deadlock** happens when two or more threads each hold a lock the other needs, so all of them wait forever. Thread 1 locks A then wants B; thread 2 locks B then wants A. Four conditions together cause it: mutual exclusion, hold-and-wait, no preemption and a circular wait. Breaking any one prevents it. The simplest rule is to **always acquire locks in the same global order**; others are to hold one lock at a time, use timeouts (`Monitor.TryEnter`) and avoid calling unknown code while holding a lock.



A CIRCULAR WAIT BETWEEN TWO THREADS

HOW A DEADLOCK HAPPENS C#

```
// DEADLOCK-PRONE: the two methods take the locks in opposite orders
void Transfer1() { lock (accountA) { Thread.Sleep(10); lock (accountB) { /* move money */ } } }
void Transfer2() { lock (accountB) { Thread.Sleep(10); lock (accountA) { /* move money */ } } }
// run both on different threads: each grabs its first lock, then waits for the other forever
```

AVOIDING DEADLOCK WITH A FIXED LOCK ORDER

Program.cs

```
var a = new BankAccount(1, 1000);
var b = new BankAccount(2, 1000);

var t1 = Task.Run(() => { for (int i = 0; i < 1000; i++) BankAccount.Transfer(a, b, 1); });
var t2 = Task.Run(() => { for (int i = 0; i < 1000; i++) BankAccount.Transfer(b, a, 2); });
bool finished = Task.WaitAll(new[] { t1, t2 }, TimeSpan.FromSeconds(5));
Console.WriteLine($"finished without deadlock: {finished}; balances {a.Balance} + {b.Balance} =
{a.Balance + b.Balance}");

class BankAccount(int id, int balance)
{
    public int Id { get; } = id;
    public int Balance { get; private set; } = balance;
    private readonly Lock gate = new();

    public static void Transfer(BankAccount from, BankAccount to, int amount)
    {
        var (first, second) = from.Id < to.Id ? (from, to) : (to, from);   // global order: lower Id
first
        lock (first.gate)
        lock (second.gate)
        {
            from.Balance -= amount;
            to.Balance += amount;
        }
    }
}
```

OUTPUT

```
finished without deadlock: True; balances 2000 + 0 = 2000
```

18.4 Task Parallel Library (TPL)

The **Task Parallel Library** (System.Threading.Tasks) raises the level of abstraction from threads to **tasks**: units of work whose result will be available later. Tasks run on the thread pool, can return values, propagate exceptions, be combined and be cancelled, and they are what async/await is built on.

18.4.1 Task and Task<T>

A `Task` represents an operation that may still be running; `Task<T>` also produces a result of type T. You can check `Status` and `IsCompleted`, wait for it (preferably with `await`), read `Result` (which blocks until it is available) and attach continuations. A task's exception is stored and rethrown when the task is awaited.

TASK AND TASK<T>

Program.cs

```
Task<long> sumTask = Task.Run(() =>                    // CPU work on a pool thread
{
    long sum = 0;
    for (int i = 1; i <= 10_000_000; i++) sum += i;
    return sum;
});
Console.WriteLine("task started; Main is free to do other work meanwhile");
long total = await sumTask;                              // wait without blocking a thread
Console.WriteLine($"sum = {total:N0}, status {sumTask.Status}");

Task failing = Task.Run(() => throw new InvalidOperationException("boom"));
try { await failing; }
catch (InvalidOperationException e) { Console.WriteLine($"caught from task: {e.Message}, status
{failing.Status}"); }

Task<string> done = Task.FromResult("already have it");   // a completed task, no thread used
Console.WriteLine(await done);
```

OUTPUT

```
task started; Main is free to do other work meanwhile
sum = 50,000,005,000,000, status RanToCompletion
caught from task: boom, status Faulted
already have it
```

Status	Meaning
`Created` / `WaitingToRun`	not started yet / queued
`Running`	executing
`WaitingForActivation`	an async operation waiting for something
`RanToCompletion`	finished successfully
`Faulted`	finished with an exception (Exception property)
`Canceled`	cancelled through a CancellationToken

18.4.2 Task.Run, Task.WhenAll, Task.WhenAny

`Task.Run` queues CPU-bound work to the thread pool and returns its task. `Task.WhenAll(tasks)` returns a task that completes when **all** given tasks have completed, and for `Task<T>` its result is an array of results in the **same order** as the input. `Task.WhenAny(tasks)` completes when the **first** one completes and returns that task, which is used for timeouts and "fastest wins" patterns. Starting several I/O operations and then awaiting WhenAll runs them concurrently, so the total time is roughly that of the slowest, not the sum.

RUNNING I/O OPERATIONS CONCURRENTLY Program.cs

```
using System.Diagnostics;

var sw = Stopwatch.StartNew();
int a = await FetchPrice("A", 300);                    // one after another: about 300 + 300 + 300 ms
int b = await FetchPrice("B", 300);
int c = await FetchPrice("C", 300);
long sequential = sw.ElapsedMilliseconds;

sw.Restart();
int[] prices = await Task.WhenAll(FetchPrice("A", 300), FetchPrice("B", 300), FetchPrice("C", 300));   //
concurrently
long concurrent = sw.ElapsedMilliseconds;
Console.WriteLine($"sequential sum {a + b + c}, concurrent results [{string.Join(", ", prices)}]");
Console.WriteLine($"sequential about 900 ms: {sequential >= 880}, concurrent under 500 ms: {concurrent <
500}");

Task<int> fast = FetchPrice("mirror-1", 100);
Task<int> slow = FetchPrice("mirror-2", 400);
Task<int> winner = await Task.WhenAny(fast, slow);      // first to finish
Console.WriteLine($"fastest mirror answered: {(winner == fast ? "mirror-1" : "mirror-2")}, price {await
winner}");

Task<int> request = FetchPrice("slow-api", 2000);
Task timeout = Task.Delay(500);
if (await Task.WhenAny(request, timeout) == timeout) Console.WriteLine("timed out after 500 ms");

static async Task<int> FetchPrice(string source, int delayMs)
{
    await Task.Delay(delayMs);                         // stands in for an HTTP call
    return source.Length * 10;
}
```

OUTPUT

```
sequential sum 30, concurrent results [10, 10, 10]
sequential about 900 ms: True, concurrent under 500 ms: True
fastest mirror answered: mirror-1, price 80
timed out after 500 ms
```

NOTE

.NET 9 added `Task.WhenEach`, an async stream that yields tasks in the order they complete: `await foreach (var t in Task.WhenEach(tasks))`. `Task.WaitAll` and `Task.WaitAny` are the blocking equivalents of WhenAll and WhenAny; avoid them in async code.

18.4.3 Parallel.For, Parallel.ForEach, Parallel.ForEachAsync

The `Parallel` class splits CPU-bound loops across cores. `Parallel.For(from, to, body)` and `Parallel.ForEach(source, body)` run iterations in parallel and return when all are done; iterations run in no particular order, so each must be independent, writing to its own slot or using thread-safe structures. `ParallelOptions.MaxDegreeOfParallelism` limits the number of concurrent iterations. `Parallel.ForEachAsync` (.NET 6) is the async version for I/O-bound work with a concurrency limit.

PARALLEL LOOPS

Program.cs

```
using System.Diagnostics;

double[] results = new double[2_000_000];
var sw = Stopwatch.StartNew();
for (int i = 0; i < results.Length; i++) results[i] = Math.Sqrt(i) * Math.Sin(i);
long seqMs = sw.ElapsedMilliseconds;

sw.Restart();
Parallel.For(0, results.Length, i => results[i] = Math.Sqrt(i) * Math.Sin(i));   // each i writes its own
slot
long parMs = sw.ElapsedMilliseconds;
Console.WriteLine($"same values: {results[1_000_000] == Math.Sqrt(1_000_000) * Math.Sin(1_000_000)}");
Console.WriteLine($"parallel no slower on a multi-core machine: {Environment.ProcessorCount == 1 || parMs
<= seqMs * 1.5}");

string[] files = { "a.txt", "b.txt", "c.txt", "d.txt" };
var lengths = new System.Collections.Concurrent.ConcurrentDictionary<string, int>();
Parallel.ForEach(files, new ParallelOptions { MaxDegreeOfParallelism = 2 }, f => lengths[f] = f.Length *
100);
Console.WriteLine(string.Join(", ", lengths.OrderBy(p => p.Key).Select(p => $"{p.Key}={p.Value}")));

int[] ids = Enumerable.Range(1, 6).ToArray();
var fetched = new System.Collections.Concurrent.ConcurrentBag<int>();
await Parallel.ForEachAsync(ids, new ParallelOptions { MaxDegreeOfParallelism = 3 }, async (id, ct) =>
{
    await Task.Delay(50, ct);                              // async I/O per item, at most 3 at once
    fetched.Add(id * id);
});
Console.WriteLine($"ForEachAsync results: {string.Join(" ", fetched.Order())}");
```

OUTPUT

```
same values: True
parallel no slower on a multi-core machine: True
a.txt=500, b.txt=500, c.txt=500, d.txt=500
ForEachAsync results: 1 4 9 16 25 36
```

18.4.4 PLINQ with AsParallel

PLINQ parallelises a LINQ to Objects query by inserting `AsParallel()`. The query is partitioned across cores and the partial results are merged. Results are unordered unless you add `AsOrdered()` (which costs some speed). PLINQ helps only when each element involves real CPU work; for cheap operations the coordination overhead makes it slower.

PARALLEL LINQ

Program.cs

```
var numbers = Enumerable.Range(1, 2_000_000);

int primeCount = numbers.AsParallel().Count(IsPrime);
Console.WriteLine($"primes up to 2,000,000: {primeCount:N0}");

var firstPrimes = numbers.AsParallel().AsOrdered()          // keep the original order
                         .Where(IsPrime).Take(10);
Console.WriteLine(string.Join(" ", firstPrimes));

var limited = numbers.AsParallel().WithDegreeOfParallelism(2).Where(n => n % 100_000 == 0).Sum(n =>
(long)n);
Console.WriteLine($"sum of multiples of 100,000: {limited:N0}");

static bool IsPrime(int n)
{
    if (n < 2) return false;
    for (int d = 2; (long)d * d <= n; d++) if (n % d == 0) return false;
    return true;
}
```

OUTPUT

```
primes up to 2,000,000: 148,933
2 3 5 7 11 13 17 19 23 29
sum of multiples of 100,000: 21,000,000
```

18.5 async and await

`async` and `await` let you write asynchronous code that reads like ordinary sequential code. An `async` method can contain `await` expressions; at each await on an operation that has not finished yet, the method **returns to its caller** (handing back a Task), and the rest of the method is scheduled to resume when the awaited operation completes. No thread sits blocked while waiting.

18.5.1 How async/await works (state machine overview)

The compiler rewrites an async method into a **state machine**: a struct with a field for each local variable and a `MoveNext` method containing the code split at each await. When an awaited task is incomplete, the state machine registers a continuation with it and returns. When the task completes, the continuation calls MoveNext again, which jumps to the saved state and carries on. If the awaited task is already complete, execution simply continues synchronously without any switching.

THE ORDER OF EXECUTION AROUND AWAIT

Program.cs

```
Console.WriteLine($"1. Main before call");
Task<int> pending = ComputeAsync();
Console.WriteLine($"3. Main got a Task back immediately (completed: {pending.IsCompleted})");
int value = await pending;
Console.WriteLine($"5. Main resumed with {value}");

static async Task<int> ComputeAsync()
{
    Console.WriteLine("2. ComputeAsync starts, runs synchronously until the first incomplete await");
    await Task.Delay(200);                          // returns to the caller here
    Console.WriteLine("4. ComputeAsync resumed after the delay");
    return 42;
}
```

OUTPUT

```
1. Main before call
2. ComputeAsync starts, runs synchronously until the first incomplete await
3. Main got a Task back immediately (completed: False)
4. ComputeAsync resumed after the delay
5. Main resumed with 42
```



AT AN INCOMPLETE AWAIT THE METHOD RETURNS; IT RESUMES LATER FROM WHERE IT LEFT OFF

18.5.2 Async methods returning Task, Task<T>, ValueTask

Return type	Use when
`Task`	the async method produces no value
`Task<T>`	it produces a value of type T
`ValueTask` / `ValueTask<T>`	the result is often available synchronously (cache hits); avoids allocating a Task. Await it only once.
`IAsyncEnumerable<T>`	it produces a stream of values asynchronously (18.5.8)
`void`	only for event handlers (18.5.3)

TASK, TASK<T> AND VALUETASK

Program.cs

```
var cache = new PriceCache();
Console.WriteLine(await cache.GetAsync("pen"));          // miss: really awaits
Console.WriteLine(await cache.GetAsync("pen"));          // hit: completes synchronously, no Task
allocated
await cache.ClearAsync();
Console.WriteLine($"entries after clear: {cache.Count}");

class PriceCache
{
    private readonly Dictionary<string, decimal> data = new();
    public int Count => data.Count;

    public ValueTask<decimal> GetAsync(string item) =>
        data.TryGetValue(item, out var price)
            ? ValueTask.FromResult(price)                // fast path
            : new ValueTask<decimal>(LoadAsync(item));   // slow path wraps a Task

    private async Task<decimal> LoadAsync(string item)
    {
        await Task.Delay(100);                           // pretend to call a service
        return data[item] = item.Length * 2.5m;
    }

    public async Task ClearAsync()                       // Task: no value
    {
        await Task.Yield();
        data.Clear();
    }
}
```

OUTPUT

```
7.5
7.5
entries after clear: 0
```

18.5.3 Avoiding async void (except event handlers)

An `async void` method cannot be awaited, so the caller cannot know when it finishes, and an exception thrown inside it cannot be caught by the caller: it is raised on the synchronization context or thread pool and typically **crashes the process**. Always return `Task`. The only legitimate use of async void is an event handler, whose signature is fixed by the event (for example a button Click handler); wrap its body in try/catch.

ASYNC VOID VERSUS ASYNC TASK

C#

```
// BAD: the caller cannot await it, and the exception escapes any try/catch around the call
async void SaveBad() { await Task.Delay(10); throw new IOException("disk full"); }

try { SaveBad(); }                       // returns immediately; the catch below never sees the exception
catch (IOException) { /* never reached: the process crashes later instead */ }

// GOOD
async Task SaveGood() { await Task.Delay(10); throw new IOException("disk full"); }
try { await SaveGood(); } catch (IOException e) { Console.WriteLine(e.Message); }   // caught

// The one exception: event handlers
saveButton.Click += async (sender, e) =>
{
    try { await SaveGood(); }
    catch (Exception ex) { ShowError(ex.Message); }   // always handle inside
};
```

18.5.4 Avoiding deadlocks with .Result and .Wait()

Blocking on a task with `.Result`, `.Wait()` or `.GetAwaiter().GetResult()` ("sync over async") wastes a thread and can **deadlock**. In apps with a single-threaded synchronization context (WinForms, WPF, MAUI, old ASP.NET), the UI thread blocks waiting for the task, while the task's continuation needs that same UI thread to finish: neither can proceed. In console apps and ASP.NET Core there is no such context, so it does not deadlock, but it still ties up threads and can starve the thread pool under load. The rule: **async all the way**, awaiting from top to bottom.

THE CLASSIC UI DEADLOCK C#

```
// WPF button handler: DEADLOCK
private void LoadButton_Click(object sender, RoutedEventArgs e)
{
    string data = DownloadAsync().Result;   // UI thread blocks here...
    Output.Text = data;
}

private async Task<string> DownloadAsync()
{
    string s = await http.GetStringAsync(url);   // ...this continuation wants the UI thread back
    return s.ToUpper();                            // never runs: both wait for each other
}

// FIX: make the handler async and await
private async void LoadButton_Click(object sender, RoutedEventArgs e)
{
    Output.Text = await DownloadAsync();
}
```

18.5.5 ConfigureAwait

By default, after an await the method resumes on the **captured context**: the UI thread in a desktop app. `await task.ConfigureAwait(false)` says "I do not need the original context", letting the continuation run on any thread-pool thread. Library code that does not touch UI elements should use `ConfigureAwait(false)` on every await: it is slightly faster and prevents the deadlock above if a caller blocks. Application code that updates the UI after the await must not use it. ASP.NET Core and console apps have no synchronization context, so there it makes no practical difference. .NET 8 added `ConfigureAwaitOptions`, such as `SuppressThrowing`.

CONFIGUREAWAIT(FALSE) IN LIBRARY CODE Program.cs

```
Console.WriteLine($"synchronization context in a console app:
{SynchronizationContext.Current?.GetType().Name ?? "none"}");
string text = await LibraryCode.ReadConfigAsync();
Console.WriteLine(text);

static class LibraryCode
{
    public static async Task<string> ReadConfigAsync()
    {
        await Task.Delay(50).ConfigureAwait(false);           // library: don't capture the context
        string raw = "  timeout=30  ";
        await Task.Yield();
        return raw.Trim();
    }
}
```

OUTPUT

```
synchronization context in a console app: none
timeout=30
```

18.5.6 Cancellation with CancellationToken

Long operations should be cancellable: the user clicks Cancel, an HTTP request is aborted, a timeout expires. .NET uses **cooperative cancellation**. A `CancellationTokenSource` creates a `CancellationToken` that is passed down to every async method. The operation checks it (`token.ThrowIfCancellationRequested()`) or passes it on to APIs that accept it (`Task.Delay`, `HttpClient`, EF Core). Calling `cts.Cancel()` or `cts.CancelAfter(timeout)` requests cancellation, and the operation ends with an `OperationCanceledException`.

COOPERATIVE CANCELLATION Program.cs

```
using var cts = new CancellationTokenSource();
var progressLog = new List<int>();

try
{
    await ProcessFilesAsync(10, cts.Token, step =>
    {
        progressLog.Add(step);
        if (step == 3) cts.Cancel();                // e.g. the user pressed Cancel
    });
}
catch (OperationCanceledException)
{
    Console.WriteLine($"cancelled after steps {string.Join(",", progressLog)}");
}

using var timeoutCts = new CancellationTokenSource(TimeSpan.FromMilliseconds(150));   // automatic
timeout
try { await Task.Delay(5000, timeoutCts.Token); }
catch (TaskCanceledException) { Console.WriteLine($"timed out, token cancelled:
{timeoutCts.IsCancellationRequested}"); }

static async Task ProcessFilesAsync(int count, CancellationToken token, Action<int> onStep)
{
    for (int i = 1; i <= count; i++)
    {
        token.ThrowIfCancellationRequested();       // check before each unit of work
        await Task.Delay(20, token);                // pass the token on
        onStep(i);
    }
}
```

OUTPUT

```
cancelled after steps 1,2,3
timed out, token cancelled: True
```

TIP

Accept a `CancellationToken cancellationToken = default` as the last parameter of every public async method and pass it to every async call inside. ASP.NET Core supplies one per request (`HttpContext.RequestAborted`), triggered when the client disconnects. `CancellationTokenSource.CreateLinkedTokenSource` combines several tokens.

18.5.7 Progress reporting with IProgress<T>

`IProgress<T>` is the standard way for an async operation to report progress without knowing who listens. The operation calls `progress?.Report(value)`. The built-in `Progress<T>` class captures the synchronization context when created, so in a UI app its callback runs on the UI thread and can update a progress bar safely. (In a console app it runs callbacks on the thread pool, possibly out of order, so the example uses a small synchronous implementation to keep the output ordered.)

REPORTING PROGRESS

Program.cs

```
IProgress<int> progress = new ConsoleProgress();
int copied = await CopyFilesAsync(5, progress);
Console.WriteLine($"copied {copied} files");

static async Task<int> CopyFilesAsync(int total, IProgress<int>? progress = null)
{
    for (int i = 1; i <= total; i++)
    {
        await Task.Delay(30);                       // copy one file
        progress?.Report(i * 100 / total);          // percent done
    }
    return total;
}

class ConsoleProgress : IProgress<int>
{
    public void Report(int percent) => Console.WriteLine($"[{new string('#', percent / 10),-10}]
{percent}%");
}
```

OUTPUT

```
[##        ] 20%
[####      ] 40%
[######    ] 60%
[########  ] 80%
[##########] 100%
copied 5 files
```

PROGRESS<T> IN A UI APPLICATION

C#

```
// In a WPF/WinForms/MAUI handler: Progress<T> marshals each report to the UI thread
var progress = new Progress<int>(percent => progressBar.Value = percent);
await CopyFilesAsync(500, progress);
```

18.5.8 Async streams with IAsyncEnumerable<T> and await foreach

An iterator can be asynchronous: an `async` method returning `IAsyncEnumerable<T>` may both `await` and `yield return`. The consumer uses `await foreach`, receiving items as they become available, which suits paged APIs, database cursors, log tails and message streams. Cancellation is passed with `WithCancellation(token)` and received through a parameter marked `[EnumeratorCancellation]`.

ASYNC STREAMS

Program.cs

```
using System.Runtime.CompilerServices;

await foreach (var page in FetchPagesAsync(totalItems: 7, pageSize: 3))
    Console.WriteLine($"page {page.Number}: {string.Join(", ", page.Items)}");

using var cts = new CancellationTokenSource();
int seen = 0;
try
{
    await foreach (int reading in SensorReadingsAsync().WithCancellation(cts.Token))
    {
        Console.Write($"{reading} ");
        if (++seen == 4) cts.Cancel();                      // stop an endless stream
    }
}
catch (OperationCanceledException) { Console.WriteLine("| stream cancelled"); }

static async IAsyncEnumerable<(int Number, int[] Items)> FetchPagesAsync(int totalItems, int pageSize)
{
    for (int start = 1, number = 1; start <= totalItems; start += pageSize, number++)
    {
        await Task.Delay(50);                                // one "HTTP request" per page
        yield return (number, Enumerable.Range(start, Math.Min(pageSize, totalItems - start +
1)).ToArray());
    }
}

static async IAsyncEnumerable<int> SensorReadingsAsync([EnumeratorCancellation] CancellationToken token =
default)
{
    for (int t = 20; ; t++)
    {
        await Task.Delay(20, token);
        yield return t;
    }
}
```

OUTPUT

```
page 1: 1, 2, 3
page 2: 4, 5, 6
page 3: 7
20 21 22 23 | stream cancelled
```

18.5.9 Exception handling in async code

An exception thrown in an async method is captured in its Task and **rethrown at the await**, so ordinary try/catch works around `await`. With `Task.WhenAll`, if several tasks fail, awaiting rethrows only the **first** exception; all of them remain available through the combined task's `Exception.InnerExceptions` (an `AggregateException`). Blocking calls (`.Wait()`, `.Result`) wrap exceptions in AggregateException, another reason to prefer await.

EXCEPTIONS THROUGH AWAIT AND WHENALL

Program.cs

```
try
{
    await LoadAsync("profile", fail: true);
}
catch (InvalidOperationException e) { Console.WriteLine($"caught: {e.Message}"); }

Task all = Task.WhenAll(LoadAsync("a", fail: false), LoadAsync("b", fail: true), LoadAsync("c", fail:
true));
try
{
    await all;
}
catch (Exception first)
{
    Console.WriteLine($"await rethrew one: {first.Message}");
    Console.WriteLine($"all failures: {string.Join(", ", all.Exception!.InnerExceptions.Select(x =>
x.Message))}");
}

try { LoadAsync("x", fail: true).Wait(); }
catch (AggregateException ae) { Console.WriteLine($"Wait() wraps it: {ae.GetType().Name} ->
{ae.InnerException!.Message}"); }

static async Task<string> LoadAsync(string name, bool fail)
{
    await Task.Delay(name == "b" ? 50 : 100);
    if (fail) throw new InvalidOperationException($"{name} failed");
    return name;
}
```

OUTPUT

```
caught: profile failed
await rethrew one: b failed
all failures: b failed, c failed
Wait() wraps it: AggregateException -> x failed
```

NOTE

The order of InnerExceptions follows the order of the tasks passed to WhenAll, so "b failed" comes first regardless of which task failed first in time.

18.6 Concurrent collections

18.6.1 ConcurrentDictionary, ConcurrentQueue, ConcurrentBag, BlockingCollection

Module 14 introduced the thread-safe collections of `System.Collections.Concurrent`. They use fine-grained locking or lock-free techniques internally, so many threads can use them at once without external locks. Their APIs are designed around **atomic** operations, because "check, then act" sequences such as `if (!dict.ContainsKey(k)) dict.Add(k, v)` are races even on a thread-safe collection.

Collection	Ordering	Key atomic operations
`ConcurrentDictionary<K,V>`	none	TryAdd, TryRemove, TryUpdate, GetOrAdd, AddOrUpdate
`ConcurrentQueue<T>`	FIFO	Enqueue, TryDequeue, TryPeek
`ConcurrentStack<T>`	LIFO	Push, TryPop, PushRange
`ConcurrentBag<T>`	unordered	Add, TryTake (fast when the same thread adds and takes)

Collection	Ordering	Key atomic operations
`BlockingCollection<T>`	wraps one of the above (FIFO by default)	Add, Take (blocks when empty), CompleteAdding, GetConsumingEnumerable, bounded capacity

THREAD-SAFE COLLECTIONS Program.cs

```
using System.Collections.Concurrent;

var cache = new ConcurrentDictionary<int, string>();
int factoryCalls = 0;
Parallel.For(0, 100, i =>
{
    cache.GetOrAdd(i % 5, key => { Interlocked.Increment(ref factoryCalls); return $"value-{key}"; });
});
Console.WriteLine($"{cache.Count} entries, cache[3] = {cache[3]}, factory ran at least 5 times:
{factoryCalls >= 5}");

var bag = new ConcurrentBag<int>();
Parallel.For(1, 101, i => bag.Add(i));
Console.WriteLine($"bag count {bag.Count}, sum {bag.Sum()}");

using var queue = new BlockingCollection<int>(boundedCapacity: 2);    // producer-consumer
var consumer = Task.Run(() =>
{
    foreach (int job in queue.GetConsumingEnumerable())                // blocks until items arrive; ends
after CompleteAdding
        Console.WriteLine($"  consumed job {job}");
});
for (int j = 1; j <= 5; j++) queue.Add(j);                             // blocks when 2 items are waiting
queue.CompleteAdding();
await consumer;
Console.WriteLine("consumer finished");
```

OUTPUT

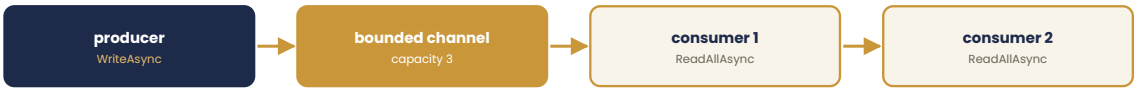
```
5 entries, cache[3] = value-3, factory ran at least 5 times: True
bag count 100, sum 5050
  consumed job 1
  consumed job 2
  consumed job 3
  consumed job 4
  consumed job 5
consumer finished
```

NOTE

`GetOrAdd` guarantees that only one value is stored per key, but under contention the value factory may run more than once for the same key (the extra results are discarded). If the factory is expensive or has side effects, store `Lazy<T>` values.

18.6.2 System.Threading.Channels for producer-consumer

Channels (`System.Threading.Channels`) are the modern, async-first producer-consumer primitive. A channel has a `Writer` that producers use (`WriteAsync`, `Complete`) and a `Reader` that consumers use (`ReadAllAsync`, `ReadAsync`, `TryRead`). A **bounded** channel has a capacity: when full, `WriteAsync` waits asynchronously, which provides **back-pressure** so a fast producer cannot exhaust memory. Unlike BlockingCollection, nothing blocks a thread. Channels underpin pipelines such as "read files, parse, store" and background job queues in ASP.NET Core.



A BOUNDED CHANNEL BETWEEN A PRODUCER AND TWO CONSUMERS

A PRODUCER-CONSUMER PIPELINE WITH CHANNELS

Program.cs

```
using System.Threading.Channels;

var channel = Channel.CreateBounded<Order>(new BoundedChannelOptions(3)
{
    FullMode = BoundedChannelFullMode.Wait                    // producer waits when 3 items are queued
});

var producer = Task.Run(async () =>
{
    for (int id = 1; id <= 8; id++)
        await channel.Writer.WriteAsync(new Order(id, id * 125m));
    channel.Writer.Complete();                                // no more items
});

async Task<(int Count, decimal Total)> Consume()
{
    int count = 0; decimal total = 0;
    await foreach (Order o in channel.Reader.ReadAllAsync())  // ends when the writer completes and the
channel is empty
    {
        await Task.Delay(10);                                 // process the order
        count++; total += o.Amount;
    }
    return (count, total);
}

var results = await Task.WhenAll(Consume(), Consume());       // two consumers share the work
await producer;
Console.WriteLine($"orders processed: {results.Sum(r => r.Count)}, total {results.Sum(r =>
r.Total):N2}");
Console.WriteLine($"both consumers did some work: {results.All(r => r.Count > 0)}");

record Order(int Id, decimal Amount);
```

OUTPUT

```
orders processed: 8, total 4,500.00
both consumers did some work: True
```

NOTE

How the 8 orders are split between the two consumers varies from run to run; the totals do not. For a single consumer, items are read in exactly the order they were written.

18.7 Timers

18.7.1 PeriodicTimer and other timers

.NET has several timers. `PeriodicTimer` (.NET 6) is designed for async code: `await timer.WaitForNextTickAsync(token)` in a loop, so each tick's work finishes before the next wait, ticks never overlap, and cancellation is simple. It is the recommended choice for background loops such as polling or heartbeats.

A PERIODICTIMER LOOP

Program.cs

```
using var timer = new PeriodicTimer(TimeSpan.FromMilliseconds(100));
using var cts = new CancellationTokenSource();
int tick = 0;
try
{
    while (await timer.WaitForNextTickAsync(cts.Token))
    {
        tick++;
        Console.WriteLine($"tick {tick}: checking for new orders");
        if (tick == 3) cts.Cancel();                        // stop after three ticks
    }
}
catch (OperationCanceledException) { Console.WriteLine("timer stopped"); }
```

OUTPUT

```
tick 1: checking for new orders
tick 2: checking for new orders
tick 3: checking for new orders
timer stopped
```

Timer	Namespace	Callback runs on	Use for
`PeriodicTimer`	System.Threading	your async loop (no callback)	async background loops: the modern default
`System.Threading.Timer`	System.Threading	a thread-pool thread	lightweight callbacks; callbacks may overlap
`System.Timers.Timer`	System.Timers	thread pool (or a SynchronizingObject)	Elapsed event with AutoReset; older component-style code
`DispatcherTimer` / WinForms `Timer`	WPF / WinForms	the UI thread	updating the UI periodically
`Task.Delay`	System.Threading.Tasks	continuation after await	one-off delays and timeouts
`BackgroundService` + PeriodicTimer	Microsoft.Extensions.Hosting	hosted service	scheduled jobs in ASP.NET Core / worker services

A process isolates memory; its threads share the heap. Concurrency interleaves work, parallelism runs it simultaneously, asynchrony waits without blocking: use async/await for I/O-bound work and Task.Run/Parallel/PLINQ for CPU-bound work. Threads and the thread pool are the foundation; protect shared state with lock (or the .NET 9 Lock type), Monitor, Mutex, SemaphoreSlim or Interlocked, and avoid deadlocks by locking in a fixed order. Tasks represent future results: combine them with WhenAll and WhenAny. Async methods return Task, Task<T> or ValueTask (never async void except for event handlers), should be awaited all the way instead of blocked on with .Result, use ConfigureAwait(false) in libraries, accept CancellationTokens, report through IProgress<T>, stream with IAsyncEnumerable, and rethrow exceptions at the await. Use concurrent collections and Channels for producer-consumer designs and PeriodicTimer for background loops.

PRACTICE CHECKPOINT

Download **10 web pages** sequentially and with **Task.WhenAll** and compare the timings, add **cancellation** support, and build a **producer-consumer pipeline with Channels**.

SAMPLE SOLUTION

Program.cs

```
using System.Diagnostics;
using System.Threading.Channels;

string[] urls = Enumerable.Range(1, 10).Select(i => $"https://example.com/page{i}").ToArray();
var downloader = new Downloader();

var sw = Stopwatch.StartNew();
var sequential = new List<int>();
foreach (string url in urls) sequential.Add(await downloader.GetLengthAsync(url));
var seqTime = sw.Elapsed;

sw.Restart();
int[] concurrent = await Task.WhenAll(urls.Select(u => downloader.GetLengthAsync(u)));
var parTime = sw.Elapsed;

Console.WriteLine($"sequential: {sequential.Sum():N0} bytes in about {Math.Round(seqTime.TotalSeconds)}
s");
Console.WriteLine($"WhenAll:    {concurrent.Sum():N0} bytes, at least 5x faster:
{parTime.TotalMilliseconds * 5 < seqTime.TotalMilliseconds}");

using var cts = new CancellationTokenSource();
int completed = 0;
try
{
    foreach (string url in urls)
    {
        await downloader.GetLengthAsync(url, cts.Token);
        if (++completed == 3) cts.Cancel();                    // user presses Cancel after 3 pages
    }
}
catch (OperationCanceledException) { Console.WriteLine($"cancelled after {completed} of {urls.Length}
pages"); }

var channel = Channel.CreateBounded<string>(4);
var producer = Task.Run(async () =>
{
    foreach (string url in urls) await channel.Writer.WriteAsync(url);
    channel.Writer.Complete();
});
var lengths = new System.Collections.Concurrent.ConcurrentBag<int>();
var consumers = Enumerable.Range(1, 3).Select(_ => Task.Run(async () =>
{
    await foreach (string url in channel.Reader.ReadAllAsync())
        lengths.Add(await downloader.GetLengthAsync(url));
})).ToArray();
await Task.WhenAll(consumers.Append(producer));
Console.WriteLine($"pipeline: {lengths.Count} pages, {lengths.Sum():N0} bytes with 3 consumers");

class Downloader
{
    // A real implementation: return (await http.GetStringAsync(url, token)).Length;
    // simulated here so the timings and sizes are repeatable
    public async Task<int> GetLengthAsync(string url, CancellationToken token = default)
    {
        await Task.Delay(200, token);                          // network latency
        return 1000 + int.Parse(url[(url.LastIndexOf("page") + 4)..]) * 100;
```

```
    }
}
```

OUTPUT

```
sequential: 15,500 bytes in about 2 s
WhenAll:    15,500 bytes, at least 5x faster: True
cancelled after 3 of 10 pages
pipeline: 10 pages, 15,500 bytes with 3 consumers
```

MODULE 19 · ADVANCED

Advanced C# language features

Minimum duration **4 days** | Level **Advanced**

This module covers the features that experienced C# developers use to write expressive and high-performance code: pattern matching in all its forms, Span<T>, Memory<T>, stackalloc, ref returns and ArrayPool for allocation-free code, reflection and attributes, source generators and analyzers, dynamic, unsafe code and native interop, preprocessor directives, and a tour of what each recent C# version (10 to 14) added.

IN THIS MODULE

19.1 Pattern matching in depth
19.2 High-performance types
19.3 Reflection and attributes
19.4 Compile-time code generation
19.5 Other advanced features
19.6 Recent language versions

19.1 Pattern matching in depth

A **pattern** tests whether a value has a certain shape and, optionally, extracts parts of it into variables. Patterns are used with `is`, in `switch` statements and in `switch` expressions, and they can be nested and combined. Since C# 7 each version has added new pattern kinds; together they replace long chains of if/else and casts with declarative, compiler-checked code.

Pattern	Example	Matches when
Constant	`x is 0`, `s is null`	equal to the constant
Type / declaration	`o is string s`	of the type (and assigns s)
Relational	`t is > 30`	comparison holds
Logical	`and`, `or`, `not`	combination holds
Property	`{ Age: >= 18 }`	properties match their sub-patterns
Positional	`(0, 0)`, `Point(var x, _)`	deconstructed parts match
List (C# 11)	`[1, .., var last]`	elements match by position
var	`var x`	always (captures the value)
Discard	`_`	always (ignores the value)

19.1.1 Type, constant, relational and logical patterns (and, or, not)

Relational patterns (`<`, `<=`, `>`, `>=`) compare with a constant. **Logical** patterns combine patterns with `and`, `or` and `not`, with parentheses for grouping. `is not null` is the idiomatic null check: unlike `!= null`, it cannot be changed by an overloaded operator. The compiler checks switch expressions for **exhaustiveness** (warning CS8509 if some input is not handled) and for unreachable arms.

COMBINING BASIC PATTERNS

Program.cs

```
foreach (object? o in new object?[] { 42, -7, 3.5, "hello", "", null, 'x', new[] { 1, 2 } })
    Console.WriteLine($"{o ?? "null",-8} -> {Describe(o)}");

foreach (int t in new[] { -5, 12, 24, 38, 45 })
    Console.Write($"{t}:{Weather(t)}  ");
Console.WriteLine();
Console.WriteLine($"{IsLetter('q')} {IsLetter('7')} {IsWeekend(DayOfWeek.Sunday)}");

static string Describe(object? o) => o switch
{
    null => "nothing",                                  // constant pattern
    int n when n < 0 => "negative int",                 // type pattern + when guard
    int n => $"int {n}",
    double d => $"double {d:F1}",
    string { Length: 0 } => "empty string",             // type + property pattern
    string s => $"string of {s.Length}",
    int[] arr => $"array of {arr.Length}",
    _ => $"other ({o.GetType().Name})"                  // discard: everything else
};

static string Weather(int celsius) => celsius switch
{
    < 0 => "freezing",                                  // relational patterns
    >= 0 and < 15 => "cold",                            // and
    >= 15 and < 30 => "pleasant",
    30 or 31 or 32 or 33 or 34 or 35 or 36 or 37 or 38 or 39 => "hot",   // or
    _ => "extreme"
};

static bool IsLetter(char c) => c is (>= 'a' and <= 'z') or (>= 'A' and <= 'Z');
static bool IsWeekend(DayOfWeek d) => d is DayOfWeek.Saturday or DayOfWeek.Sunday;
```

OUTPUT

```
42       -> int 42
-7       -> negative int
3.5      -> double 3.5
hello    -> string of 5
         -> empty string
null     -> nothing
x        -> other (Char)
System.Int32[] -> array of 2
-5:freezing  12:cold  24:pleasant  38:hot  45:extreme
True False True
```

19.1.2 Property patterns and extended property patterns

A **property pattern** `{ Prop: pattern, ... }` matches an object whose properties match the given sub-patterns; it also implies a non-null check. Property patterns can be nested, and C# 10 **extended property patterns** let you reach into nested properties with a dot: `{ Address.City: "Pune" }` instead of `{ Address: { City: "Pune" } }`. Property patterns can be combined with a type pattern and can capture the matched object in a variable.

PROPERTY PATTERNS Program.cs

```
var orders = new[]
{
    new Order(1, 12000m, new Customer("Asha", true, new Address("Pune", "IN"))),
    new Order(2, 800m, new Customer("Ravi", false, new Address("Delhi", "IN"))),
    new Order(3, 2500m, new Customer("Emma", false, new Address("Berlin", "DE"))),
    new Order(4, 3000m, new Customer("Kiran", true, new Address("Pune", "IN"))),
};

foreach (var o in orders) Console.WriteLine($"order {o.Id}: shipping {Shipping(o)}, discount
{Discount(o):P0}");

static string Shipping(Order o) => o switch
{
    { Customer.Address.Country: not "IN" } => "international",          // extended property pattern
    { Customer.Address.City: "Pune" or "Mumbai" } => "same-day",
    { Total: >= 1000 } => "free standard",
    _ => "standard (Rs 50)"
};

static decimal Discount(Order o) => o switch
{
    { Customer: { IsPremium: true }, Total: > 10000 } => 0.15m,          // nested property pattern
    { Customer.IsPremium: true } => 0.10m,
    { Total: > 2000 } => 0.05m,
    _ => 0m
};

record Address(string City, string Country);
record Customer(string Name, bool IsPremium, Address Address);
record Order(int Id, decimal Total, Customer Customer);
```

OUTPUT

```
order 1: shipping same-day, discount 15%
order 2: shipping standard (Rs 50), discount 0%
order 3: shipping international, discount 5%
order 4: shipping same-day, discount 10%
```

19.1.3 Positional patterns with deconstruction

A **positional pattern** deconstructs a value (tuples, positional records, or any type with a Deconstruct method) and matches each part: `(0, 0)`, `Point(var x, 0)`. Matching on a **tuple of several values** is an elegant way to express decisions that depend on combinations, such as state machines or game rules.

POSITIONAL AND TUPLE PATTERNS Program.cs

```
foreach (var p in new[] { new Point(0, 0), new Point(3, 0), new Point(0, -2), new Point(4, 5), new
Point(-1, -1) })
    Console.WriteLine($"{p} -> {Locate(p)}");

foreach (var (a, b) in new[] { ("rock", "scissors"), ("paper", "paper"), ("rock", "paper") })
    Console.WriteLine($"{a} vs {b}: {Play(a, b)}");

static string Locate(Point p) => p switch
{
    (0, 0) => "origin",
    (_, 0) => "on the x-axis",
    (0, _) => "on the y-axis",
    ( > 0, > 0) => "quadrant I",
    var (x, y) when x == y => $"on the diagonal at {x}",
    _ => "elsewhere"
};

static string Play(string a, string b) => (a, b) switch           // tuple pattern
{
    _ when a == b => "draw",
    ("rock", "scissors") or ("scissors", "paper") or ("paper", "rock") => "first player wins",
    _ => "second player wins"
};

record Point(int X, int Y);
```

OUTPUT

```
Point { X = 0, Y = 0 } -> origin
Point { X = 3, Y = 0 } -> on the x-axis
Point { X = 0, Y = -2 } -> on the y-axis
Point { X = 4, Y = 5 } -> quadrant I
Point { X = -1, Y = -1 } -> on the diagonal at -1
rock vs scissors: first player wins
paper vs paper: draw
rock vs paper: second player wins
```

19.1.4 List patterns (C# 11)

List patterns match arrays, lists and any countable, indexable type element by element: `[1, 2, 3]`. A **slice pattern** `..` matches zero or more elements and can capture them: `[var first, .., var last]` or `[_, .. var middle, _]`. They are excellent for parsing command-line arguments, tokens or fixed record layouts.

LIST AND SLICE PATTERNS

Program.cs

```
int[][] samples = { new int[0], new[] { 7 }, new[] { 1, 2 }, new[] { 1, 5, 9, 3 }, new[] { 0, 0, 0 } };
foreach (int[] s in samples) Console.WriteLine($"[{string.Join(",", s)}] -> {Shape(s)}");

foreach (string line in new[] { "add Asha 91", "remove 17", "list", "add", "help me now" })
    Console.WriteLine($"{line,-12} => {Execute(line.Split(' '))}");

static string Shape(int[] a) => a switch
{
    [] => "empty",
    [var only] => $"one element {only}",
    [0, 0, ..] => "starts with two zeros",
    [var first, var second] => $"pair {first},{second}",
    [var first, .. var middle, var last] => $"first {first}, last {last}, {middle.Length} in the middle"
};

static string Execute(string[] args) => args switch
{
    ["add", var name, var marks] when int.TryParse(marks, out int m) => $"added {name} with {m}",
    ["add", ..] => "usage: add <name> <marks>",
    ["remove", var id] => $"removed #{id}",
    ["list"] => "listing all",
    [var cmd, ..] => $"unknown command '{cmd}'",
    [] => "no command"
};
```

OUTPUT

```
[] -> empty
[7] -> one element 7
[1,2] -> pair 1,2
[1,5,9,3] -> first 1, last 3, 2 in the middle
[0,0,0] -> starts with two zeros
add Asha 91  => added Asha with 91
remove 17    => removed #17
list         => listing all
add          => usage: add <name> <marks>
help me now  => unknown command 'help'
```

19.1.5 Discard and var patterns

The **discard pattern** `_` matches anything and is used as the default arm of a switch expression or to ignore parts of a positional or list pattern. The **var pattern** `var x` also always matches but captures the value, including null, which is useful for naming an intermediate result inside a larger pattern or a `when` guard.

DISCARD AND VAR PATTERNS

Program.cs

```
foreach (var (name, marks) in new[] { ("Asha", 91), ("Ravi", 34), ("Meera", 67) })
{
    string result = (name, marks) switch
    {
        (_, >= 85) => "distinction",                        // discard the name
        (var n, < 40) when n.StartsWith('R') => $"{n}: re-exam scheduled",   // var captures
        _ => "pass"
    };
    Console.WriteLine($"{name}: {result}");
}

if (GetScore() is var s && s > 50) Console.WriteLine($"score {s} recorded");   // var pattern names a
computed value
static int GetScore() => 73;
```

OUTPUT

```
Asha: distinction
Ravi: Ravi: re-exam scheduled
Meera: pass
score 73 recorded
```

19.2 High-performance types

Allocation is cheap in .NET, but in hot paths (parsers, serializers, web servers handling thousands of requests per second) millions of small allocations cause frequent garbage collections. The following types let you work with slices of memory without copying or allocating.

19.2.1 Span<T> and ReadOnlySpan<T>

`Span<T>` is a **view** over a contiguous region of memory: an array, part of an array, a string (as `ReadOnlySpan<char>`), stack memory or native memory. Slicing a span is O(1) and copies nothing. Spans are `ref struct`s, so they live only on the stack: they cannot be fields of classes, boxed, captured by lambdas or used across `await`. Many APIs accept spans: `int.Parse`, `string.Concat`, `Encoding.GetBytes`, `MemoryExtensions` such as `IndexOf`, `Split`, `Trim`.

SLICING WITHOUT ALLOCATING

Program.cs

```
string line = "2026-03-09;Asha;91.5;Pune";
ReadOnlySpan<char> span = line;                        // no copy

int sep1 = span.IndexOf(';');
ReadOnlySpan<char> datePart = span[..sep1];          // slicing: no new string
DateOnly date = DateOnly.Parse(datePart, System.Globalization.CultureInfo.InvariantCulture);
ReadOnlySpan<char> rest = span[(sep1 + 1)..];
int sep2 = rest.IndexOf(';');
ReadOnlySpan<char> name = rest[..sep2];
Console.WriteLine($"{date:dd MMM yyyy}, name {name.ToString()}, length {name.Length}");

int count = 0;
foreach (Range r in span.Split(';')) count++;          // .NET 8+: allocation-free split into ranges
Console.WriteLine($"fields: {count}");

int[] numbers = { 5, 1, 4, 2, 3 };
Span<int> middle = numbers.AsSpan(1, 3);               // view over elements 1..3
middle.Sort();                                         // sorts the underlying array in place
middle.Reverse();
Console.WriteLine(string.Join(" ", numbers));
Console.WriteLine($"contains 4: {middle.Contains(4)}, index of 2: {middle.IndexOf(2)}");
```

OUTPUT

```
09 Mar 2026, name Asha, length 4
fields: 4
5 4 2 1 3
contains 4: True, index of 2: 1
```



name = span.Slice(11, 4): a pointer and a length into the same string, no copy

A READONLYSPAN<CHAR> SLICE REFERS TO PART OF THE ORIGINAL STRING

19.2.2 Memory<T>

Because a Span cannot be stored on the heap, it cannot be kept in a field or used across an await. `Memory<T>` (and `ReadOnlyMemory<T>`) is its heap-friendly counterpart: a regular struct that represents the same kind of slice and can be stored, passed to async methods and later converted to a span with `.Span` for the actual processing.

MEMORY<T> CAN BE STORED AND AWAITED

Program.cs

```
byte[] buffer = new byte[16];
Memory<byte> memory = buffer.AsMemory();
int written = await FillAsync(memory[..8]);              // a slice passed across an await
Console.WriteLine($"wrote {written} bytes: {string.Join(" ", buffer.Take(10))}");

var parser = new LineParser("alpha,beta,gamma".AsMemory());     // stored in a field
Console.WriteLine(parser.FirstField());

static async Task<int> FillAsync(Memory<byte> target)
{
    await Task.Delay(10);                                // Span<byte> could not be used here
    Span<byte> s = target.Span;                          // get a span for the synchronous work
    for (int i = 0; i < s.Length; i++) s[i] = (byte)(i * 2);
    return s.Length;
}

class LineParser(ReadOnlyMemory<char> text)
{
    private readonly ReadOnlyMemory<char> data = text;    // a Span field would not compile (CS8345)
    public string FirstField()
    {
        ReadOnlySpan<char> s = data.Span;
        return s[..s.IndexOf(',')].ToString();
    }
}
```

OUTPUT

```
wrote 8 bytes: 0 2 4 6 8 10 12 14 0 0
alpha
```

19.2.3 stackalloc

`stackalloc` allocates a block of memory on the **stack**; it is freed automatically when the method returns, so the GC never sees it. Assigned to a `Span<T>` it needs no unsafe code. Use it only for small, bounded sizes (a few hundred bytes up to about 1 KB); large or unbounded stackallocs risk a StackOverflowException. A common idiom uses the stack for small inputs and falls back to an array (or ArrayPool) for large ones.

STACKALLOC INTO A SPAN

Program.cs

```
Console.WriteLine(ToHex(255));
Console.WriteLine(ToHex(48879));
Console.WriteLine(ReverseWords("span is fast", stackLimit: 64));

static string ToHex(int value)
{
    Span<char> digits = stackalloc char[8];              // 8 chars on the stack, no heap allocation
    int pos = digits.Length;
    do { digits[--pos] = "0123456789ABCDEF"[value % 16]; value /= 16; } while (value > 0);
    return new string(digits[pos..]);                    // one allocation for the result only
}

static string ReverseWords(string text, int stackLimit)
{
    Span<char> buf = text.Length <= stackLimit ? stackalloc char[text.Length] : new char[text.Length];
// small: stack, large: heap
    text.AsSpan().CopyTo(buf);
    buf.Reverse();                                       // reverse everything...
    int start = 0;
    for (int i = 0; i <= buf.Length; i++)
        if (i == buf.Length || buf[i] == ' ') { buf[start..i].Reverse(); start = i + 1; }   // ...then
each word back
    return new string(buf);
}
```

OUTPUT

```
FF
BEEF
fast is span
```

19.2.4 ref locals, ref returns and ref struct

A **ref return** returns a reference to a storage location (an array element, a field) instead of a copy of its value; the caller can store it in a **ref local** and read or write the original through it. This avoids copying large structs and allows in-place updates. A `ref struct` is a struct that must stay on the stack (like Span), which allows it to contain `ref` fields (C# 11) pointing at other memory.

WORKING WITH REFERENCES

Program.cs

```
int[] scores = { 40, 95, 70, 88 };
ref int best = ref FindMax(scores);          // a reference to the array element
best = 100;                                  // writes straight into the array
Console.WriteLine(string.Join(" ", scores));

var grid = new Grid(3, 3);
ref int cell = ref grid.At(1, 2);
cell = 7;
grid.At(0, 0) = 5;                           // assign through a ref return directly
Console.WriteLine($"{grid.At(1, 2)} {grid.At(0, 0)}");

var counter = new RefCounter(ref scores[0]);   // ref struct holding a ref field
counter.Add(2);
Console.WriteLine(scores[0]);

static ref int FindMax(int[] values)
{
    int index = 0;
    for (int i = 1; i < values.Length; i++) if (values[i] > values[index]) index = i;
    return ref values[index];
}

class Grid(int rows, int cols)
{
    private readonly int[] cells = new int[rows * cols];
    public ref int At(int r, int c) => ref cells[r * cols + c];
}

ref struct RefCounter
{
    private ref int value;                    // ref field (C# 11), allowed only in a ref struct
    public RefCounter(ref int target) => value = ref target;
    public void Add(int n) => value += n;
}
```

OUTPUT

```
40 100 70 88
7 5
42
```

19.2.5 ArrayPool<T>

When a method needs a temporary array that is too large for stackalloc, renting from `ArrayPool<T>.Shared` avoids allocating a new array each call. `Rent(minimumLength)` returns an array of **at least** that length (often larger, rounded up to a power of two) that may contain old data; `Return(array)` gives it back for reuse. Always return it in a finally block, never use it after returning, and work only with the length you asked for.

RENTING A TEMPORARY BUFFER

Program.cs

```
using System.Buffers;

string text = string.Concat(Enumerable.Repeat("pool ", 50));      // 250 chars
char[] rented = ArrayPool<char>.Shared.Rent(text.Length);
try
{
    Console.WriteLine($"asked for {text.Length}, got {rented.Length} (at least as many: {rented.Length >=
text.Length})");
    Span<char> work = rented.AsSpan(0, text.Length);               // use only what you asked for
    text.AsSpan().CopyTo(work);
    int vowels = 0;
    foreach (char c in work) if ("aeiou".Contains(c)) vowels++;
    Console.WriteLine($"vowels: {vowels}");
}
finally
{
    ArrayPool<char>.Shared.Return(rented);                          // give it back for reuse
}
```

OUTPUT

```
asked for 250, got 256 (at least as many: True)
vowels: 100
```

19.2.6 Implicit span conversions (C# 14)

Arrays and strings have always converted to spans, but before C# 14 those conversions were ordinary user-defined operators that the compiler ignored in some places, most notably when calling **extension methods** declared on spans and in generic type inference. C# 14 makes span conversions **first-class**: `T[]` converts implicitly to `Span<T>` and `ReadOnlySpan<T>`, `Span<T>` to `ReadOnlySpan<T>`, and `string` to `ReadOnlySpan<char>`, in every context including extension receivers. Libraries can now offer one span-based extension method instead of separate overloads for arrays, lists and strings.

SPAN EXTENSION METHODS ON ARRAYS AND STRINGS

Program.cs

```
int[] marks = { 72, 85, 91, 85, 60 };
Console.WriteLine(marks.CountOf(85));                  // C# 14: array -> ReadOnlySpan<int> as extension
receiver
Console.WriteLine("banana".CountOf('a'));              // string -> ReadOnlySpan<char>
Span<int> slice = marks.AsSpan(0, 3);
Console.WriteLine(slice.CountOf(85));                  // Span<int> -> ReadOnlySpan<int>
Console.WriteLine(Sum(marks));                         // ordinary parameter conversion (worked before
too)

static int Sum(ReadOnlySpan<int> values)
{
    int s = 0;
    foreach (int v in values) s += v;
    return s;
}

static class SpanExtensions
{
    public static int CountOf<T>(this ReadOnlySpan<T> span, T value) where T : IEquatable<T>
    {
        int n = 0;
        foreach (T item in span) if (item.Equals(value)) n++;
        return n;
    }
}
```

OUTPUT

```
2
3
1
393
```

NOTE

With C# 13 or earlier, `marks.CountOf(85)` fails to compile (CS1929) because the array-to-span conversion was not considered for an extension method receiver; you had to write `marks.AsSpan().CountOf(85)`.

19.3 Reflection and attributes

Every .NET assembly carries **metadata**: a complete description of its types, members, parameters and attributes. **Reflection** (System.Reflection) reads that metadata at run time and can create objects, read and write properties and invoke methods of types that were not known when the code was compiled. Serializers, ORMs such as EF Core, dependency-injection containers, test frameworks and ASP.NET Core model binding are all built on it. **Attributes** attach extra, declarative metadata to code elements that reflection (or the compiler) can read.

19.3.1 Type, GetProperties, GetMethods, Activator.CreateInstance

A `System.Type` object describes a type. Get it with `typeof(T)`, `obj.GetType()` or `Type.GetType("Name")`. Its methods list members: `GetProperties`, `GetMethods`, `GetFields`, `GetConstructors`, filtered with `BindingFlags`. The resulting `PropertyInfo`, `MethodInfo` and `FieldInfo` objects can read, write and invoke. `Activator.CreateInstance(type)` creates an object from a Type.

INSPECTING AND USING A TYPE AT RUN TIME

Program.cs

```
using System.Reflection;

Type t = typeof(Student);
Console.WriteLine($"{t.Name}: class {t.IsClass}, base {t.BaseType!.Name}, public {t.IsPublic}");
Console.WriteLine("properties: " + string.Join(", ", t.GetProperties().Select(p => $"
{p.PropertyType.Name} {p.Name}")));
var methods = t.GetMethods(BindingFlags.Public | BindingFlags.Instance | BindingFlags.DeclaredOnly)
               .Where(m => !m.IsSpecialName);                // skip get_/set_ accessor methods
Console.WriteLine("methods: " + string.Join(", ", methods.Select(m => m.Name)));

object obj = Activator.CreateInstance(t)!;                   // create without 'new Student()'
t.GetProperty("Name")!.SetValue(obj, "Asha");                // set a property by name
t.GetProperty("Marks")!.SetValue(obj, 91);
object? greeting = t.GetMethod("Greet")!.Invoke(obj, new object[] { "Ravi" });   // call a method by name
Console.WriteLine(greeting);

FieldInfo secret = t.GetField("attempts", BindingFlags.NonPublic | BindingFlags.Instance)!;
Console.WriteLine($"private field read by reflection: {secret.GetValue(obj)}");

public class Student
{
    private int attempts = 3;
    public string Name { get; set; } = "";
    public int Marks { get; set; }
    public string Greet(string other) => $"{Name} ({Marks}) greets {other}";
    public void Promote() => Marks += 5;
}
```

OUTPUT

```
Student: class True, base Object, public True
properties: String Name, Int32 Marks
methods: Greet, Promote
Asha (91) greets Ravi
private field read by reflection: 3
```

WATCH OUT

Reflection is slow compared with direct calls (lookups by string, boxing, security checks), bypasses compile-time checking (a renamed property breaks silently), can break encapsulation, and is a problem for trimming and Native AOT. Cache `PropertyInfo` / `MethodInfo` objects, prefer generics or source generators where possible, and never use reflection for ordinary logic.

19.3.2 Built-in attributes: [Obsolete], [Serializable], [Conditional], caller info attributes

An attribute is written in square brackets before the element it describes, and is a class deriving from `System.Attribute` (the "Attribute" suffix may be omitted). Some attributes are read by the **compiler** and change how code is compiled.

Attribute	Effect
`[Obsolete("message")]`	compiler warning CS0618 wherever it is used; `[Obsolete("msg", true)]` makes it an error
`[Serializable]`	marks a type for legacy runtime serialization (BinaryFormatter, now removed); ignored by System.Text.Json
`[Conditional("DEBUG")]`	calls to the method are removed by the compiler unless the symbol is defined
`[CallerMemberName]`	the compiler fills in the calling member's name

Attribute	Effect
`[CallerFilePath]`, `[CallerLineNumber]`	the caller's source file and line
`[CallerArgumentExpression("p")]`	the source text of the argument passed for parameter p (C# 10)
`[Flags]`, `[DllImport]`, `[MethodImpl]`, `[Experimental]`	enum formatting, native interop, JIT hints, experimental API warnings

[OBSOLETE] PRODUCES A WARNING AT EVERY USE

Program.cs

```
var legacy = new PaymentGateway();
legacy.Charge(100);

class PaymentGateway
{
    [Obsolete("Use ChargeAsync instead; Charge will be removed in v3.")]
    public void Charge(decimal amount) { }
    public Task ChargeAsync(decimal amount) => Task.CompletedTask;
}
```

COMPILER OUTPUT

```
Program.cs(2,1): warning CS0618: 'PaymentGateway.Charge(decimal)' is obsolete: 'Use ChargeAsync instead;
Charge will be removed in v3.'
```

CALLER INFORMATION AND [CONDITIONAL] Program.cs

```
using System.Diagnostics;
using System.Runtime.CompilerServices;

new OrderService().Save();
int age = -3;
try { Guard.Check(age >= 0); }
catch (ArgumentException e) { Console.WriteLine(e.Message); }
DebugLog.Trace("this line exists only in Debug builds");

class OrderService
{
    public void Save() => Logger.Log("order saved");
}

static class Logger
{
    public static void Log(string message,
                           [CallerMemberName] string member = "",
                           [CallerFilePath] string file = "",
                           [CallerLineNumber] int line = 0) =>
        Console.WriteLine($"[{Path.GetFileName(file)}:{line} {member}] {message}");
}

static class Guard
{
    public static void Check(bool condition, [CallerArgumentExpression(nameof(condition))] string expr =
"")
    {
        if (!condition) throw new ArgumentException($"Check failed: {expr}");
    }
}

static class DebugLog
{
    [Conditional("DEBUG")]                                  // the call above vanishes in Release builds
    public static void Trace(string msg) => Console.WriteLine($"TRACE: {msg}");
}
```

OUTPUT

```
[Program.cs:12 Save] order saved
Check failed: age >= 0
TRACE: this line exists only in Debug builds
```

19.3.3 Creating and reading custom attributes

Define your own attribute by deriving from `Attribute`. `[AttributeUsage]` says where it may be applied (classes, properties, methods…) and whether it may be repeated. Constructor parameters become positional arguments, and public settable properties become named arguments. Attributes do nothing on their own: some code must read them with `GetCustomAttribute<T>()` or `GetCustomAttributes` and act on them, which is exactly how ASP.NET Core reads `[HttpGet]` and EF Core reads `[Key]`.

DEFINING AND READING CUSTOM ATTRIBUTES

Program.cs

```
using System.Reflection;

foreach (Type t in new[] { typeof(InvoiceService), typeof(ReportService) })
{
    var info = t.GetCustomAttribute<ModuleInfoAttribute>();
    Console.WriteLine(info is null ? $"{t.Name}: no module info" : $"{t.Name}: owner {info.Owner},
v{info.Version}, beta {info.Beta}");
    foreach (MethodInfo m in t.GetMethods().Where(m => m.IsDefined(typeof(AuditedAttribute))))
        Console.WriteLine($"  audited method: {m.Name} ({m.GetCustomAttribute<AuditedAttribute>
()!.Level})");
}

[AttributeUsage(AttributeTargets.Class, AllowMultiple = false)]
class ModuleInfoAttribute(string owner) : Attribute
{
    public string Owner { get; } = owner;          // positional argument
    public int Version { get; set; } = 1;          // named arguments
    public bool Beta { get; set; }
}

[AttributeUsage(AttributeTargets.Method)]
class AuditedAttribute(string level = "info") : Attribute
{
    public string Level { get; } = level;
}

[ModuleInfo("Asha", Version = 3)]
class InvoiceService
{
    [Audited("critical")] public void Cancel() { }
    public void Print() { }
    [Audited] public void Create() { }
}

class ReportService { }
```

OUTPUT

```
InvoiceService: owner Asha, v3, beta False
  audited method: Cancel (critical)
  audited method: Create (info)
ReportService: no module info
```

19.4 Compile-time code generation

19.4.1 Source generators (overview)

A **source generator** is a compiler plug-in (built on the Roslyn compiler APIs) that inspects your code during compilation and **adds new C# source files** to the build. It moves work that frameworks used to do with reflection at run time to compile time, giving faster start-up, no reflection cost, and compatibility with trimming and Native AOT. You use generators by applying attributes to `partial` members; the generator writes the other part. .NET itself ships several.

USING A BUILT-IN SOURCE GENERATOR Program.cs

```
using System.Text.RegularExpressions;

foreach (string pin in new[] { "411001", "41100", "4110012" })
    Console.WriteLine($"{pin}: {Validators.IsPin(pin)}");

static partial class Validators
{
    [GeneratedRegex(@"^[1-9]\d{5}$")]                    // the regex engine code is generated at compile
time
    private static partial Regex PinRegex();

    public static bool IsPin(string s) => PinRegex().IsMatch(s);
}
```

OUTPUT

```
411001: True
41100: False
4110012: False
```

Generator	Attribute you write	Replaces run-time
Regex	`[GeneratedRegex]` on a partial method	regex parsing and compilation
System.Text.Json	`[JsonSerializable(typeof(T))]` on a partial JsonSerializerContext	reflection-based serialization
Logging	`[LoggerMessage]` on a partial method	message template parsing and boxing
P/Invoke	`[LibraryImport]`	run-time marshalling stubs
Configuration binding, options validation, COM interop	various	reflection

Writing your own generator means creating a .NET Standard 2.0 class library that implements `IIncrementalGenerator` and is referenced as an analyzer; the generated files are visible in the IDE under Dependencies > Analyzers.

19.4.2 Roslyn analyzers (overview)

Analyzers are compiler plug-ins that inspect code and report **diagnostics** (warnings, errors, suggestions), often with automatic **code fixes**. The .NET SDK includes the CAxxxx code-quality analyzers and IDExxxx style rules; packages add more (for example xUnit analyzers, security analyzers, StyleCop). Severity is configured per rule in an `.editorconfig` file, and `<TreatWarningsAsErrors>` can enforce them in CI.

.EDITORCONFIG RULE CONFIGURATION ini

```
# .editorconfig
[*.cs]
dotnet_diagnostic.CA2200.severity = error      # rethrow with 'throw ex;' loses the stack trace
dotnet_diagnostic.CA1822.severity = suggestion # member can be marked static
dotnet_diagnostic.IDE0005.severity = warning   # unnecessary using directive
csharp_style_namespace_declarations = file_scoped:warning
```

TURNING ANALYZERS UP IN THE PROJECT FILE xml

```xml
<PropertyGroup>
  <AnalysisLevel>latest-recommended</AnalysisLevel>
  <EnforceCodeStyleInBuild>true</EnforceCodeStyleInBuild>
  <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
</PropertyGroup>
```

19.5 Other advanced features

19.5.1 dynamic and the DLR

A variable declared `dynamic` skips compile-time type checking: member access, calls and operators are resolved at **run time** by the Dynamic Language Runtime (DLR). If the member does not exist, a `RuntimeBinderException` is thrown. dynamic is useful for interop with dynamic languages (IronPython), COM automation (Office), and data whose shape is only known at run time; `ExpandoObject` creates objects whose members are added on the fly. In normal code, prefer static typing: dynamic loses IntelliSense, refactoring safety and performance.

DYNAMIC AND EXPANDOOBJECT Program.cs

```
using System.Dynamic;

dynamic d = 10;
Console.WriteLine(d + 5);                    // resolved at run time: int + int
d = "hello";
Console.WriteLine($"{d.ToUpper()} {d.Length}");   // now string members
try { d.Fly(); }
catch (Microsoft.CSharp.RuntimeBinder.RuntimeBinderException e) { Console.WriteLine(e.Message); }

dynamic person = new ExpandoObject();        // add members at run time
person.Name = "Asha";
person.Age = 21;
person.Describe = (Func<string>)(() => $"{person.Name}, {person.Age}");
Console.WriteLine(person.Describe());
var asDict = (IDictionary<string, object?>)person;
Console.WriteLine($"members: {string.Join(", ", asDict.Keys)}");
```

OUTPUT

```
15
HELLO 5
'string' does not contain a definition for 'Fly'
Asha, 21
members: Name, Age, Describe
```

19.5.2 Unsafe code and pointers (overview)

Inside an `unsafe` block (enabled with `<AllowUnsafeBlocks>true</AllowUnsafeBlocks>`) C# supports C-style **pointers**: `int* p`, address-of `&x`, dereference `*p`, pointer arithmetic, and `fixed` to pin a managed object so the GC cannot move it while a pointer refers to it. There are no bounds checks, so mistakes corrupt memory. Today Span<T> covers most needs safely; unsafe code remains for interop and extreme optimisation.

POINTERS IN AN UNSAFE CONTEXT Program.cs

```
int[] values = { 10, 20, 30, 40 };
Console.WriteLine(SumWithPointers(values));

int x = 5;
unsafe
{
    int* p = &x;                                // address of a local
    *p = 42;                                    // write through the pointer
    Console.WriteLine($"x = {x}, sizeof(long) = {sizeof(long)}");
}

static unsafe int SumWithPointers(int[] data)
{
    int sum = 0;
    fixed (int* start = data)                   // pin the array while we use raw pointers
    {
        for (int* p = start; p < start + data.Length; p++) sum += *p;   // pointer arithmetic
    }
    return sum;
}
```

OUTPUT

```
100
x = 42, sizeof(long) = 8
```

19.5.3 Interop with native code (P/Invoke, overview)

Platform Invoke (P/Invoke) calls functions exported by native libraries: the Windows API (user32.dll, kernel32.dll), libc on Linux and macOS, or your own C/C++ libraries. Declare a `static extern` method with `[DllImport]`, or in modern .NET a `static partial` method with `[LibraryImport]`, for which a source generator creates the marshalling code at compile time. The runtime loads the library, converts (marshals) arguments such as strings, and calls the function.

CALLING NATIVE FUNCTIONS C#

```
using System.Runtime.InteropServices;

static partial class Native
{
    [LibraryImport("libc", EntryPoint = "getpid")]          // Linux / macOS
    internal static partial int GetPid();

    [LibraryImport("user32.dll", EntryPoint = "MessageBoxW", StringMarshalling =
StringMarshalling.Utf16)]
    internal static partial int MessageBox(nint hWnd, string text, string caption, uint type);   //
Windows
}

if (OperatingSystem.IsWindows()) Native.MessageBox(0, "Hello from C#", "P/Invoke", 0);
else Console.WriteLine($"process id from libc: {Native.GetPid()}  (same as {Environment.ProcessId})");
```

NOTE

Native interop is inherently platform-specific: guard calls with `OperatingSystem.IsWindows()` and similar checks. COM interop and C++/CLI are related Windows technologies. Wrapping unmanaged handles in `SafeHandle` (Module 20) ensures they are released.

19.5.4 Preprocessor directives #if, #region, #nullable

Preprocessor directives are processed before compilation and start with `#`. `#if`, `#elif`, `#else` and `#endif` include or exclude code depending on **symbols**: DEBUG (defined in Debug builds), RELEASE, TRACE, target-framework symbols such as NET10_0_OR_GREATER, symbols from `<DefineConstants>`, or your own `#define` (which must appear at the top of the file). `#region` / `#endregion` mark collapsible areas in the editor; `#nullable` controls nullable analysis; `#warning` and `#error` emit diagnostics; `#pragma warning disable/restore` suppresses specific warnings.

CONDITIONAL COMPILATION (DEBUG BUILD, .NET 10) Program.cs

```
#define VERBOSE                          // must come before any code in the file

#region Configuration
#if DEBUG
Console.WriteLine("Debug build: extra checks enabled");
#else
Console.WriteLine("Release build");
#endif

#if VERBOSE
Console.WriteLine("verbose logging on");
#endif

#if NET10_0_OR_GREATER
Console.WriteLine("compiled for .NET 10 or later");
#elif NET8_0_OR_GREATER
Console.WriteLine("compiled for .NET 8 or 9");
#endif
#endregion

#pragma warning disable CS0168               // 'variable declared but never used'
int unused;
#pragma warning restore CS0168
#warning TODO: remove the demo variable above
```

OUTPUT

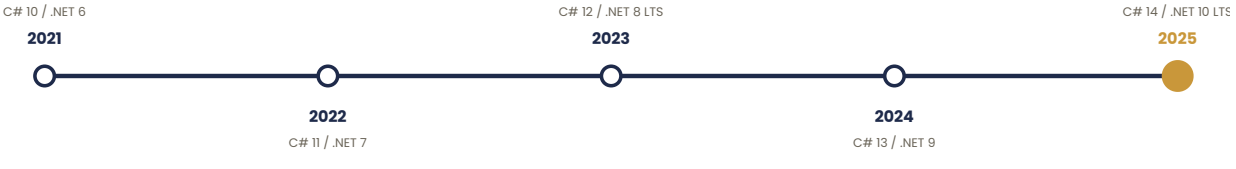
```
Debug build: extra checks enabled
verbose logging on
compiled for .NET 10 or later
```

NOTE

The `#warning` line produces a compiler warning ("warning CS1030: #warning: 'TODO: remove the demo variable above'") during the build, not program output. Use conditional compilation sparingly: code inside an inactive #if block is not compiled at all, so it can silently stop building.

19.6 Recent language versions

C# gets a new version every November together with .NET. The language version defaults to the newest one supported by the target framework: C# 12 with .NET 8, C# 13 with .NET 9, C# 14 with .NET 10. Many features of these versions have appeared throughout this book; this section collects them in one place.



ONE MAJOR C# VERSION EACH NOVEMBER

19.6.1 C# 10 to 12 recap: global usings, file-scoped namespaces, raw strings, required members, primary constructors, collection expressions

Version	Feature	Example	Covered in
C# 10	global usings / implicit usings	`global using System.Text.Json;`	Module 2
C# 10	file-scoped namespaces	`namespace Lbf.School;`	Module 2
C# 10	record structs, extended property patterns, lambda natural types	`record struct P(int X);`	12, 19, 15
C# 11	raw string literals	`"""{ "a": 1 }"""`	Module 8
C# 11	required members	`public required string Name { get; init; }`	Module 10
C# 11	list patterns, static abstract interface members, generic math	`[var first, ..]`	19, 11, 14
C# 11	file-local types, UTF-8 literals, ref fields	`file class Helper`, `"abc"u8`	10, 19
C# 12	primary constructors for classes and structs	`class Account(string owner)`	Module 10
C# 12	collection expressions and spread	`int[] a = [1, ..b];`	Module 14
C# 12	default lambda parameters, alias any type	`(int x = 1) => x`, `using Point = (int, int);`	Module 15

19.6.2 C# 13: params collections, new Lock type, partial properties

Feature	Example
params collections	`void Log(params ReadOnlySpan<string> parts)`
New `System.Threading.Lock` type	`private readonly Lock gate = new(); lock (gate) { ... }`
Partial properties and indexers	`public partial string Name { get; set; }` (implemented by a generator)
`\e` escape for the ESC character	`"\e[1mbold\e[0m"` in terminal output
Implicit index access in object initializers	`new Buffer { Data = { [^1] = 9 } }`
ref locals and unsafe code in async methods and iterators	span-based parsing inside async code
`allows ref struct` anti-constraint	`where T : allows ref struct`
Overload resolution priority	`[OverloadResolutionPriority(1)]` for library authors

19.6.3 C# 14: extension members, field keyword, null-conditional assignment, implicit span conversions

C# 14 (November 2025, .NET 10) introduced extension blocks with extension properties and static extension members (Module 10), the `field` keyword in property accessors (Module 10), first-class implicit span conversions (19.2.6), and several smaller features shown below: **null-conditional assignment** (`a?.B = value` assigns only when a is not null), `nameof` with unbound generic types, parameter modifiers in lambdas without explicit types, user-defined compound assignment operators, and partial constructors and events.

SMALLER C# 14 FEATURES

Program.cs

```
Customer? none = null;
none?.Name = "ignored";                          // C# 14: no-op because 'none' is null (no exception)
var c = new Customer();
c?.Name = "Asha";                                // assigns
c.Tags?.Add("new");                              // the right side is evaluated only when the target is
not null
c?.Points += 10;                                 // compound assignment works too
Console.WriteLine($"{c.Name} {c.Points} tags:{c.Tags?.Count ?? 0}");

Console.WriteLine(nameof(List<>));               // C# 14: unbound generic type in nameof
Console.WriteLine(nameof(Dictionary<,>));

TryParser<int> parse = (text, out result) => int.TryParse(text, out result);   // C# 14: 'out' without
parameter types
Console.WriteLine(parse("123", out int n) ? n * 2 : -1);

delegate bool TryParser<T>(string text, out T result);

class Customer
{
    public string Name { get; set; } = "";
    public int Points { get; set; }
    public List<string>? Tags { get; set; }
}
```

OUTPUT

```
Asha 10 tags:0
List
Dictionary
246
```

19.6.4 Following "What's new in C#" for each release

The language keeps evolving, so part of being an advanced C# developer is keeping up. Useful habits:

- Read the official **"What's new in C#"** page on Microsoft Learn for each version, and the .NET release notes and blog posts each November.
- Follow the **dotnet/csharplang** repository on GitHub, where every proposal, design meeting note and the language feature status page are public; previews of the next version arrive in .NET preview SDKs months early (enable with `<LangVersion>preview</LangVersion>`).
- Use **SharpLab** to see what new syntax compiles into (lowered C# and IL).
- Let the IDE and analyzers suggest modern idioms (IDE0090 for target-typed new, IDE0305 for collection expressions, and so on) and adopt them gradually across a code base.
- Remember that most features are syntax: the same IL can usually be written the older way, so upgrading the SDK and LangVersion is low risk, while upgrading the target framework (runtime) needs testing.

Patterns (type, constant, relational, logical, property, positional, list, var, discard) make complex conditions declarative and compiler-checked. Span<T>, ReadOnlySpan<T>, Memory<T>, stackalloc, ref returns and ArrayPool remove allocations in hot paths, and C# 14 makes span conversions first-class. Reflection reads metadata to create objects and call members by name; attributes add declarative metadata, read by the compiler (Obsolete, Conditional, caller info) or by your own reflection code. Source generators and analyzers extend the compiler. dynamic, unsafe pointers, P/Invoke and preprocessor directives serve special cases. Each yearly C# version (10 to 14) adds features; follow What's new in C# and the csharplang repository.

PRACTICE CHECKPOINT

Write a **shape-pricing function** with property and list patterns, a **parser using ReadOnlySpan<char>** that avoids allocations, and a **validation framework** driven by custom attributes and reflection.

SAMPLE SOLUTION

Program.cs

```
using System.Reflection;

Shape[] cart = [new Circle(0.5), new Circle(3), new Rect(2, 2), new Rect(10, 2), new Triangle(3, 4, 5)];
foreach (Shape s in cart) Console.WriteLine($"{s,-34} Rs {Pricing.Price(s),6:F2}");
Console.WriteLine($"bundle: {Pricing.Bundle(cart)}");

Console.WriteLine($"sum of \"x=10;y=-4;z=25\": {SpanParser.SumValues("x=10;y=-4;z=25")}");

var good = new Student { Name = "Asha", Marks = 91, Email = "asha@example.com" };
var bad = new Student { Name = "", Marks = 140, Email = "this-is-a-very-long-address@example.com" };
foreach (var st in new[] { good, bad })
{
    var errors = Validator.Validate(st);
    Console.WriteLine(errors.Count == 0 ? "valid" : "invalid: " + string.Join("; ", errors));
}

abstract record Shape;
record Circle(double Radius) : Shape;
record Rect(double W, double H) : Shape;
record Triangle(double A, double B, double C) : Shape;

static class Pricing
{
    public static double Price(Shape s) => s switch
    {
        Circle { Radius: <= 1 } => 10,                                  // property patterns
        Circle { Radius: var r } => 10 + Math.PI * r * r,
        Rect { W: var w, H: var h } when w == h => 5 + w * h,           // squares are cheaper
        Rect { W: > 5 } or Rect { H: > 5 } => 50,                       // oversize flat rate
        Rect r => 8 + r.W * r.H,
        Triangle(var a, var b, var c) => 12 + (a + b + c),              // positional pattern
        _ => throw new ArgumentException("unknown shape")
    };

    public static string Bundle(Shape[] items) => items switch       // list patterns
    {
        [] => "empty cart",
        [Circle, ..] and [.., Triangle] => "circle-to-triangle bundle: 10% off",
        [_, _, ..] => "multi-item cart",
        [_] => "single item"
    };
}

static class SpanParser
{
    public static int SumValues(ReadOnlySpan<char> text)               // no Split, no Substring
    {
        int sum = 0;
        while (!text.IsEmpty)
        {
            int end = text.IndexOf(';');
            ReadOnlySpan<char> pair = end >= 0 ? text[..end] : text;
            sum += int.Parse(pair[(pair.IndexOf('=') + 1)..]);
            text = end >= 0 ? text[(end + 1)..] : ReadOnlySpan<char>.Empty;
        }
        return sum;
    }
}
```

```
[AttributeUsage(AttributeTargets.Property)] class RequiredAttribute : Attribute;
[AttributeUsage(AttributeTargets.Property)] class BetweenAttribute(int min, int max) : Attribute
{
    public int Min { get; } = min;
    public int Max { get; } = max;
}
[AttributeUsage(AttributeTargets.Property)] class MaxLengthAttribute(int length) : Attribute
{
    public int Length { get; } = length;
}

static class Validator
{
    public static List<string> Validate(object obj)
    {
        var errors = new List<string>();
        foreach (PropertyInfo p in obj.GetType().GetProperties())
        {
            object? value = p.GetValue(obj);
            if (p.GetCustomAttribute<RequiredAttribute>() is not null &&
string.IsNullOrWhiteSpace(value?.ToString()))
                errors.Add($"{p.Name} is required");
            if (p.GetCustomAttribute<BetweenAttribute>() is { } range && value is int n && (n < range.Min
|| n > range.Max))
                errors.Add($"{p.Name} must be {range.Min}-{range.Max}");
            if (p.GetCustomAttribute<MaxLengthAttribute>() is { } max && value is string s && s.Length >
max.Length)
                errors.Add($"{p.Name} longer than {max.Length}");
        }
        return errors;
    }
}

class Student
{
    [Required] public string Name { get; set; } = "";
    [Between(0, 100)] public int Marks { get; set; }
    [Required, MaxLength(30)] public string Email { get; set; } = "";
}
```

OUTPUT

```
Circle { Radius = 0.5 }           Rs  10.00
Circle { Radius = 3 }             Rs  38.27
Rect { W = 2, H = 2 }             Rs   9.00
Rect { W = 10, H = 2 }            Rs  50.00
Triangle { A = 3, B = 4, C = 5 }  Rs  24.00
bundle: circle-to-triangle bundle: 10% off
sum of "x=10;y=-4;z=25": 31
valid
invalid: Name is required; Marks must be 0-100; Email longer than 30
```

MODULE 20 · ADVANCED

.NET runtime internals and memory management

Minimum duration **2 days** Level **Advanced**

Writing correct C# does not require knowing how the runtime works, but writing fast, scalable and leak-free C# does. This module looks under the hood of the Common Language Runtime (CLR): assemblies, metadata and IL, JIT compilation, tiered compilation, ReadyToRun and Native AOT; how memory is laid out on the stack and the managed heap; how the garbage collector works with generations, compaction and its different modes; deterministic clean-up with Dispose, finalizers, SafeHandle and IAsyncDisposable; common memory leaks; and the tools used to measure performance.

IN THIS MODULE

20.1 The CLR
20.2 Memory
20.3 Garbage collection
20.4 Deterministic cleanup
20.5 Memory leaks in .NET
20.6 Performance tools

20.1 The CLR

20.1.1 Assemblies, metadata and IL

The C# compiler does not produce machine code. It produces an **assembly** (a .dll or .exe file) containing **IL** (Intermediate Language, also called CIL or MSIL), a CPU-independent instruction set for a stack machine, together with **metadata** describing every type, member, attribute and referenced assembly, and a **manifest** naming the assembly, its version and dependencies. At run time the **CLR** loads assemblies, verifies them, compiles IL to native code, and provides garbage collection, type safety, exception handling and threading.



FROM C# SOURCE TO NATIVE INSTRUCTIONS

Part of an assembly	Contains
PE header	standard Windows/portable executable format, even on Linux
Manifest	name, version, culture, public key, list of referenced assemblies
Metadata tables	every type, method, field, property, parameter and attribute
IL code	method bodies as stack-machine instructions
Resources	embedded files, strings, images

20.1.2 Viewing IL (ILSpy, SharpLab)

Looking at IL shows what C# features really compile into: properties become get_/set_ methods, lambdas become hidden classes, foreach becomes enumerator calls, async methods become state machines. **ILSpy** (and the dotPeek and JetBrains Rider equivalents) decompiles any assembly back to C# and IL. **SharpLab** (sharplab.io) shows the IL, lowered C# and JIT assembly for a snippet as you type. The SDK tool `ildasm` (via the dotnet-ildasm global tool) prints IL on the command line.

A METHOD AND ITS IL C#

```
static int Add(int a, int b) => a + b;

// IL produced in a Release build:
.method private hidebysig static int32 Add(int32 a, int32 b) cil managed
{
    .maxstack 8
    IL_0000: ldarg.0      // push argument a onto the evaluation stack
    IL_0001: ldarg.1      // push argument b
    IL_0002: add          // pop both, push a + b
    IL_0003: ret          // return the value on top of the stack
}
```

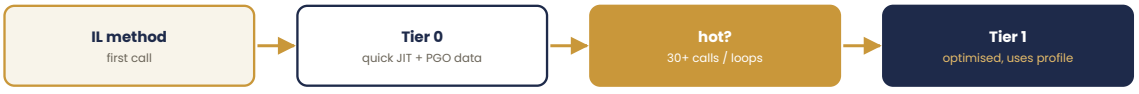
WHAT AN AUTO-PROPERTY REALLY IS C#

```
public string Name { get; set; }

// compiles to a hidden field and two methods:
.field private string '<Name>k__BackingField'
.method public hidebysig specialname instance string get_Name() cil managed { ... }
.method public hidebysig specialname instance void set_Name(string 'value') cil managed { ... }
```

20.1.3 JIT, tiered compilation and ReadyToRun

The **JIT** (just-in-time) compiler, RyuJIT, translates each method's IL to native code the first time the method is called, then reuses that code. JIT compilation takes time, which hurts start-up. **Tiered compilation** (on by default) balances start-up and speed: methods are first compiled quickly with few optimisations (tier 0); methods called often (30 calls or containing hot loops, via on-stack replacement) are recompiled in the background with full optimisations (tier 1). **Dynamic PGO** (profile-guided optimisation, default since .NET 8) instruments tier-0 code to learn which types and branches actually occur and uses that to optimise tier 1, for example by inlining the virtual call that is almost always made. **ReadyToRun** (R2R) pre-compiles IL to native code at publish time and stores both in the assembly, so start-up skips most JIT work while tiering can still re-optimise hot methods.



TIERED COMPILATION WITH DYNAMIC PGO

READYTORUN AND TIERING SWITCHES C#

```
dotnet publish -c Release -r linux-x64 -p:PublishReadyToRun=true      # pre-compiled for faster start-up
dotnet run -c Release -p:TieredPGO=false                                # experiment: turn dynamic PGO
off
```

20.1.4 Native AOT

Native AOT (ahead-of-time) compiles the whole application, including the parts of the runtime it uses, into a single native executable at publish time. There is no JIT at run time and no need for the .NET runtime on the

target machine. The results: very fast start-up, smaller memory use and small self-contained binaries, ideal for command-line tools, serverless functions and containers. The price: code generation must be known at compile time, so unrestricted reflection, `Assembly.Load` of plugins, runtime code generation and some libraries do not work; the compiler emits trimming and AOT warnings for such code. Source generators (Module 19) are the AOT-friendly replacement for reflection.

PUBLISHING WITH NATIVE AOT C#

```
<PropertyGroup>
  <PublishAot>true</PublishAot>
  <InvariantGlobalization>true</InvariantGlobalization>   <!-- optional: smaller, no culture data -->
</PropertyGroup>

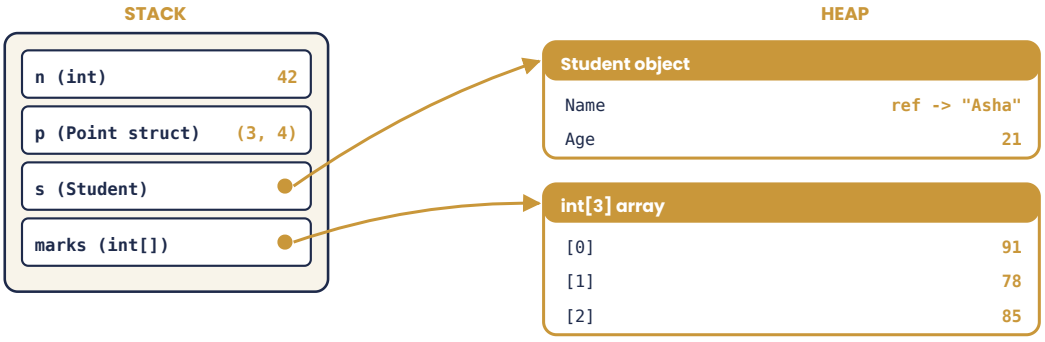
dotnet publish -c Release -r linux-x64        # produces one native executable
```

	JIT (default)	ReadyToRun	Native AOT
Start-up	slowest	faster	fastest
Peak performance	best (tiering + PGO)	best (rejits hot code)	very good, no PGO
Needs .NET runtime installed	yes (or self-contained)	yes (or self-contained)	no
Reflection / dynamic loading	full	full	restricted
Binary size	small IL	larger	single native file

20.2 Memory

20.2.1 Stack and managed heap

Each thread has a **stack**: a fixed-size region (1 MB by default on Windows, 8 MB on Linux for the main thread) holding a **frame** per active method call, with its parameters, local variables and return address. Allocation is just moving a pointer, and a frame disappears the moment its method returns. The **managed heap** holds all objects created with `new` for reference types (class instances, arrays, strings, boxed values, closures); objects live as long as something references them and are reclaimed by the garbage collector. A reference-type local variable is a reference on the stack pointing to an object on the heap.



LOCALS LIVE IN THE STACK FRAME; OBJECTS LIVE ON THE HEAP

	Stack	Managed heap
Holds	method frames: locals, parameters, return addresses	objects: class instances, arrays, strings, boxes

	Stack	Managed heap
Allocation	move a pointer: practically free	bump-pointer allocation: very fast, but creates GC work
Freed	automatically when the method returns	by the garbage collector when unreachable
Size	small, fixed per thread	grows as needed
Shared between threads	no	yes

20.2.2 Small Object Heap and Large Object Heap

The heap is divided by object size. Objects smaller than **85,000 bytes** go to the **Small Object Heap** (SOH), which is organised in generations and compacted. Objects of 85,000 bytes or more (in practice big arrays and long strings) go to the **Large Object Heap** (LOH). Moving large objects is expensive, so the LOH is collected only with generation 2 and is not compacted by default, which can lead to **fragmentation**. A **Pinned Object Heap** (POH, .NET 5) holds objects that must never move, such as buffers shared with native code.

WHERE OBJECTS ARE ALLOCATED

Program.cs

```
var small = new byte[1_000];
var large = new byte[100_000];                    // >= 85,000 bytes: Large Object Heap

Console.WriteLine($"small array generation: {GC.GetGeneration(small)}");
Console.WriteLine($"large array generation: {GC.GetGeneration(large)} (the LOH is reported as generation
2)");
Console.WriteLine($"max generation: {GC.MaxGeneration}");

var pinned = GC.AllocateArray<byte>(4096, pinned: true);   // Pinned Object Heap (.NET 5+)
Console.WriteLine($"pinned buffer length: {pinned.Length}");
```

OUTPUT

```
small array generation: 0
large array generation: 2 (the LOH is reported as generation 2)
max generation: 2
pinned buffer length: 4096
```

TIP

Repeatedly allocating large temporary arrays is costly: each one starts in the LOH and triggers expensive generation-2 collections. Rent them from `ArrayPool<T>.Shared` instead (Module 19).

20.2.3 Value types on the stack vs inside objects

A common oversimplification is "value types live on the stack, reference types on the heap". More precisely, a value type is stored **wherever its variable is**: a local int is in the stack frame, but an int field of a class lives **inside the object on the heap**, and the elements of an `int[]` or `Point[]` are stored inline in the array object. A value type is copied to the heap as a separate object only when it is **boxed** (converted to object or an interface) or captured by a lambda or async method, which moves locals into a heap-allocated closure or state machine.

VALUE TYPES LIVE WHERE THEIR CONTAINER LIVES

Program.cs

```
int local = 5;                                  // in the stack frame
var holder = new Holder { Count = 7 };          // Count is stored inside the heap object
Point[] points = { new(1, 2), new(3, 4) };      // structs stored inline in the array: one allocation

long before = GC.GetAllocatedBytesForCurrentThread();
object boxed = local;                           // boxing: a new heap object holding a copy
long boxingBytes = GC.GetAllocatedBytesForCurrentThread() - before;
Console.WriteLine($"boxing allocated memory: {boxingBytes > 0}");

local = 99;
Console.WriteLine($"box is a copy: {(int)boxed}");
Console.WriteLine($"{holder.Count} {points[1].X}");

class Holder { public int Count; }
record struct Point(int X, int Y);
```

OUTPUT

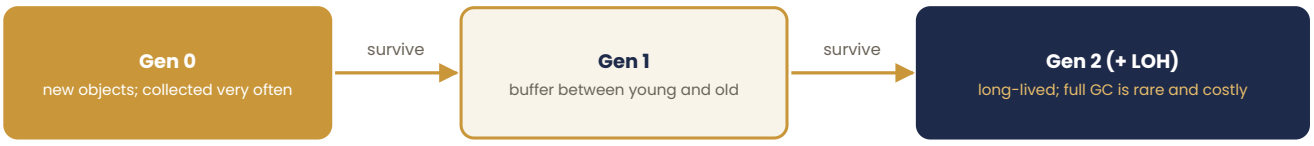
```
boxing allocated memory: True
box is a copy: 5
7 3
```

20.3 Garbage collection

The **garbage collector** (GC) automatically reclaims heap memory occupied by objects that the program can no longer reach. It starts from the **roots** (static fields, local variables and parameters of active methods on every thread, CPU registers, GC handles), follows every reference, and treats everything it cannot reach as garbage. This removes whole classes of bugs (dangling pointers, double frees, most leaks) at the cost of occasional pauses.

20.3.1 Generations 0, 1 and 2

The GC relies on the **generational hypothesis**: most objects die young (temporary strings, LINQ iterators, request DTOs) while objects that survive a while tend to live long (caches, singletons). The SOH is split into three generations. New objects go to **generation 0**, which is small and collected very often and cheaply. Survivors are **promoted** to generation 1 (a buffer) and then to **generation 2**, which holds long-lived data and is collected rarely, since a full collection is the most expensive.



most objects die in gen 0, which is cheap to collect

OBJECTS ARE PROMOTED AS THEY SURVIVE COLLECTIONS

WATCHING AN OBJECT GET PROMOTED

Program.cs

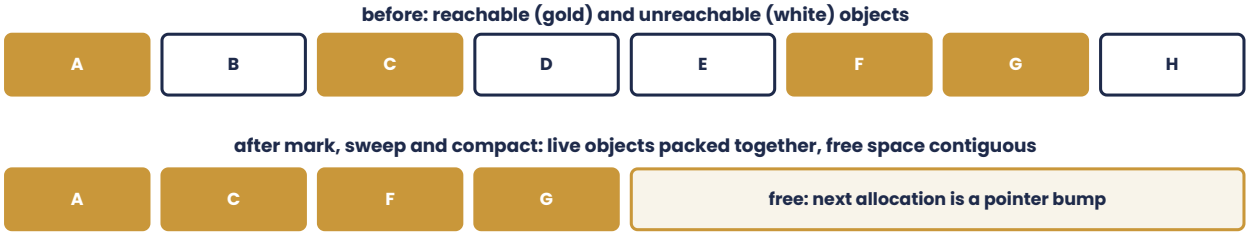
```
object survivor = new object();
Console.WriteLine($"new object: gen {GC.GetGeneration(survivor)}");
GC.Collect();                                   // forced here only to demonstrate promotion
Console.WriteLine($"after 1 collection: gen {GC.GetGeneration(survivor)}");
GC.Collect();
Console.WriteLine($"after 2 collections: gen {GC.GetGeneration(survivor)}");
GC.Collect();
Console.WriteLine($"after 3 collections: gen {GC.GetGeneration(survivor)} (cannot go higher)");
GC.KeepAlive(survivor);
```

OUTPUT

```
new object: gen 0
after 1 collection: gen 1
after 2 collections: gen 2
after 3 collections: gen 2 (cannot go higher)
```

20.3.2 Mark, sweep and compact

A collection works in phases. First all managed threads are paused at safe points (for the blocking parts). **Mark**: starting from the roots, the GC visits every reachable object and marks it live. **Sweep / plan**: everything unmarked is garbage. **Compact**: live objects are slid together to remove the gaps, and every reference to a moved object is updated, so free space is one contiguous block and new allocations can again be a simple pointer bump. To collect only gen 0 without scanning all of gen 2, the GC uses a **card table** that records which old objects were written with references to young ones. Since .NET 4.5, gen-2 collections run mostly concurrently with the application (**background GC**), keeping pauses short.



COMPACTION REMOVES FRAGMENTATION

20.3.3 Workstation vs server GC

	Workstation GC	Server GC
Default for	desktop and console apps	ASP.NET Core apps
Heaps and GC threads	one heap; collection on the triggering thread	one heap and one dedicated GC thread per core
Optimised for	low memory, responsiveness	throughput on multi-core servers
Memory use	lower	higher (several heaps)
Setting	`<ServerGarbageCollector>false`	`<ServerGarbageCollector>true`

Both run concurrent (background) gen-2 collections by default (`<ConcurrentGarbageCollection>`). For containers, the GC respects memory limits, and .NET 8 introduced **DATAS** (Dynamic Adaptation To Application Sizes), enabled by default for Server GC from .NET 9, which grows and shrinks the number of heaps with the actual load to cut memory use.

INSPECTING THE GC MODE OF A CONSOLE APP Program.cs

```
Console.WriteLine($"server GC: {System.Runtime.GCSettings.IsServerGC}");
Console.WriteLine($"latency mode: {System.Runtime.GCSettings.LatencyMode}");
```

OUTPUT

```
server GC: False
latency mode: Interactive
```

20.3.4 GC class (and why not to call GC.Collect)

`System.GC` exposes the collector: `GetGeneration`, `CollectionCount(gen)`, `GetTotalMemory`, `GetGCMemoryInfo`, `GetAllocatedBytesForCurrentThread`, `AddMemoryPressure` for unmanaged memory, `KeepAlive`, `SuppressFinalize`, and `Collect`. Calling `GC.Collect()` in application code is almost always a mistake: the GC is self-tuning, a forced full collection pauses the program, promotes short-lived objects into older generations (making later collections more expensive) and usually does not fix the real problem. Legitimate uses are rare: tests and benchmarks, or right after unloading a huge one-off data set.

QUERYING THE GC (INSTEAD OF FORCING IT) Program.cs

```
long before = GC.GetAllocatedBytesForCurrentThread();
var list = new List<string>();
for (int i = 0; i < 1000; i++) list.Add(i.ToString());       // many small allocations
long allocated = GC.GetAllocatedBytesForCurrentThread() - before;
Console.WriteLine($"allocated more than 20 KB: {allocated > 20_000}");

GCMemoryInfo info = GC.GetGCMemoryInfo();
Console.WriteLine($"GC knows the available memory: {info.TotalAvailableMemoryBytes > 0}");
Console.WriteLine($"gen0 collections so far are counted: {GC.CollectionCount(0) >= 0}");
```

OUTPUT

```
allocated more than 20 KB: True
GC knows the available memory: True
gen0 collections so far are counted: True
```

20.3.5 Weak references

A normal (strong) reference keeps an object alive. A `WeakReference<T>` refers to an object **without** keeping it alive: if no strong references remain, the GC may collect it, and `TryGetTarget` then returns false. Weak references suit caches of data that can be recreated and observer lists that must not keep subscribers alive. `ConditionalWeakTable` attaches extra data to objects without extending their lifetime.

WEAK REFERENCES LET THE GC RECLAIM OBJECTS

Program.cs

```
using System.Runtime.CompilerServices;

WeakReference<byte[]> weak = CreateWeakCache();
GC.Collect();                                          // demonstration only
GC.WaitForPendingFinalizers();
Console.WriteLine($"after GC, still alive: {weak.TryGetTarget(out _)}");

byte[] strong = new byte[1024];
var weak2 = new WeakReference<byte[]>(strong);
GC.Collect();
Console.WriteLine($"while a strong reference exists: {weak2.TryGetTarget(out var target)}
({target!.Length} bytes)");
GC.KeepAlive(strong);

[MethodImpl(MethodImplOptions.NoInlining)]             // keep the only strong reference inside this
method
static WeakReference<byte[]> CreateWeakCache() => new(new byte[10_000]);
```

OUTPUT

```
after GC, still alive: False
while a strong reference exists: True (1024 bytes)
```

20.4 Deterministic cleanup

20.4.1 The Dispose pattern

Memory is managed, but **unmanaged resources** (file handles, sockets, database connections, GDI handles, native memory) are not, and they are scarce. Types that own them implement `IDisposable` so callers can release them deterministically with `using` (Module 13). The full **Dispose pattern**, used by non-sealed classes that own resources, has a public `Dispose()` that calls a `protected virtual Dispose(bool disposing)` and then `GC.SuppressFinalize(this)`; derived classes override `Dispose(bool)` and call the base. `disposing` is true when called from Dispose (managed objects may be cleaned up) and false when called from a finalizer (only unmanaged resources may be touched).

THE DISPOSE PATTERN

Program.cs

```
using (var log = new AuditLog("audit.txt"))
{
    log.Write("login");
}
var other = new AuditLog("second.txt");
other.Dispose();
other.Dispose();                                       // safe to call twice
try { other.Write("too late"); }
catch (ObjectDisposedException e) { Console.WriteLine($"ObjectDisposedException: {e.ObjectName}"); }

public class AuditLog : IDisposable
{
    private StreamWriter? writer;                       // a managed object that owns an OS handle
    private bool disposed;

    public AuditLog(string path) { writer = new StreamWriter(path); Console.WriteLine($"opened {path}");
}

    public void Write(string text)
    {
        ObjectDisposedException.ThrowIf(disposed, this);
        writer!.WriteLine(text);
    }
```

```
    public void Dispose()
    {
        Dispose(disposing: true);
        GC.SuppressFinalize(this);                          // no finalizer work needed any more
    }

    protected virtual void Dispose(bool disposing)
    {
        if (disposed) return;
        if (disposing)
        {
            writer?.Dispose();                              // release managed resources
            writer = null;
            Console.WriteLine("disposed");
        }
        // release unmanaged resources here (none in this class)
        disposed = true;
    }
}
```

```
OUTPUT
opened audit.txt
disposed
opened second.txt
disposed
ObjectDisposedException: AuditLog
```

20.4.2 Finalizers and SafeHandle

A **finalizer** (`~ClassName()`, Module 10) is a safety net that runs on the GC's finalizer thread if the object was not disposed. Finalizable objects are expensive: they are registered at creation, survive at least one extra collection while waiting in the finalization queue, and their finalizer runs at an unpredictable time. Since .NET 2.0 the recommended way to own an OS handle is a `SafeHandle` subclass (such as `SafeFileHandle`), which contains the finalizer and reliable release logic; your class then only implements `IDisposable` and disposes the SafeHandle. As a result, you should almost never need to write a finalizer yourself.

SAFEHANDLE OWNS THE UNMANAGED RESOURCE C#

```
using Microsoft.Win32.SafeHandles;
using System.Runtime.InteropServices;

// Wrap a native handle: the SafeHandle holds the finalizer, your class just disposes it
sealed class NativeBufferHandle : SafeHandleZeroOrMinusOneIsInvalid
{
    public NativeBufferHandle(int size) : base(ownsHandle: true) =>
SetHandle(Marshal.AllocHGlobal(size));
    protected override bool ReleaseHandle() { Marshal.FreeHGlobal(handle); return true; }
}

sealed class NativeBuffer(int size) : IDisposable
{
    private readonly NativeBufferHandle handle = new(size);
    public void Dispose() => handle.Dispose();      // no finalizer needed in this class
}
```

20.4.3 IAsyncDisposable and await using

Some resources need asynchronous work to close properly: flushing a network stream, committing a database transaction, closing a message channel. Such types implement `IAsyncDisposable` with `ValueTask`

`DisposeAsync()`, and callers use `await using`, which awaits DisposeAsync at the end of the scope. Many types implement both interfaces (FileStream, DbConnection, Utf8JsonWriter).

AWAIT USING Program.cs

```
await using (var upload = new UploadSession("report.pdf"))
{
    await upload.SendChunkAsync(1);
    await upload.SendChunkAsync(2);
}
Console.WriteLine("upload complete");

class UploadSession(string file) : IAsyncDisposable
{
    public async Task SendChunkAsync(int n) { await Task.Delay(10); Console.WriteLine($"sent chunk {n} of
{file}"); }

    public async ValueTask DisposeAsync()
    {
        await Task.Delay(10);                              // e.g. send the final "commit" message
        Console.WriteLine($"session for {file} closed asynchronously");
    }
}
```

OUTPUT

```
sent chunk 1 of report.pdf
sent chunk 2 of report.pdf
session for report.pdf closed asynchronously
upload complete
```

20.5 Memory leaks in .NET

20.5.1 Event handler leaks, static references, undisposed resources

With a garbage collector, a "leak" means objects that are still **reachable** although the program no longer needs them. Memory then grows steadily until the process slows down (more and longer gen-2 collections) or runs out of memory. The usual causes:

Cause	How it keeps objects alive	Fix
Event subscriptions	a long-lived publisher's delegate references each subscriber (Module 15)	unsubscribe in Dispose; weak events
Static fields and collections	statics are GC roots for the life of the app: a static List or Dictionary used as a cache grows forever	bounded caches (MemoryCache with size limits and expiration), clear entries
Captured variables	a lambda stored somewhere long-lived captures a large object or `this`	capture only what is needed; static lambdas
Undisposed resources	unmanaged memory and handles, timers that keep their callbacks alive	using / await using; dispose timers and HttpResponseMessage
Long-lived timers / background tasks	the timer references its callback target	dispose timers; use cancellation for background loops
Wrong DI lifetimes	a singleton holding scoped or transient objects	respect lifetimes (Module 22)
Unbounded queues	producers faster than consumers	bounded channels (Module 18)

A STATIC COLLECTION THAT ONLY GROWS Program.cs

```
for (int i = 0; i < 100; i++) Cache.Items.Add(new byte[10_000]);           // ~1 MB that never goes away
GC.Collect();
long withLeak = GC.GetTotalMemory(forceFullCollection: true);
Cache.Items.Clear();                                                       // remove the references
long afterFix = GC.GetTotalMemory(forceFullCollection: true);
Console.WriteLine($"memory dropped by more than 900 KB after clearing the static cache: {withLeak -
afterFix > 900_000}");

static class Cache { public static readonly List<byte[]> Items = new(); }   // a static field is a GC
root
```

OUTPUT

```
memory dropped by more than 900 KB after clearing the static cache: True
```

20.6 Performance tools

The first rule of performance work is **measure, do not guess**. Intuition about what is slow is often wrong, and an "optimisation" that is not measured may make things worse or more complex for no gain.

20.6.1 BenchmarkDotNet

BenchmarkDotNet (NuGet package BenchmarkDotNet) is the standard micro-benchmarking library for .NET, used by the .NET team itself. It handles everything that makes naive Stopwatch timing unreliable: JIT warm-up, tiering, running many iterations, removing outliers, computing statistics and isolating each benchmark in its own process. With `[MemoryDiagnoser]` it also reports allocations and GC collections per operation. Benchmarks must run in **Release** mode.

A BENCHMARKDOTNET BENCHMARK CLASS C#

```
// dotnet add package BenchmarkDotNet
using BenchmarkDotNet.Attributes;
using BenchmarkDotNet.Running;

BenchmarkRunner.Run<JoinBenchmarks>();      // dotnet run -c Release

[MemoryDiagnoser]
public class JoinBenchmarks
{
    private readonly string[] words = Enumerable.Range(0, 1000).Select(i => $"w{i}").ToArray();

    [Benchmark(Baseline = true)]
    public string PlusConcat()
    {
        string s = "";
        foreach (string w in words) s += w + ",";
        return s;
    }

    [Benchmark]
    public string StringJoin() => string.Join(",", words);
}
```

Example result table (shape of the report; numbers vary by machine)

```
| Method     | Mean       | Ratio | Gen0      | Allocated   | Alloc Ratio |
|----------- |-----------:|------:|----------:|------------:|------------:|
| PlusConcat | 512.40 us  | 1.000 | 1490.2344 | 9,140.48 KB |       1.000 |
| StringJoin |   7.15 us  | 0.014 |    2.5940 |    15.95 KB |       0.002 |
```

Read the **Mean** time per operation, the **Ratio** against the baseline, how many gen-0 collections occur per 1,000 operations, and the bytes **Allocated** per operation. Here the loop with += is about 70 times slower and allocates over 500 times more memory, for the reasons explained in Module 8.

20.6.2 Visual Studio diagnostic tools, dotnet-counters, dotnet-trace

Tool	What it shows	When
Visual Studio Diagnostic Tools / Performance Profiler	CPU usage, memory snapshots and diffs, allocation tracking, async and database activity	during development on Windows
JetBrains dotTrace / dotMemory, Rider	CPU profiling and memory analysis, cross-platform	development
`dotnet-counters`	live metrics: CPU, GC heap size, gen collections, allocation rate, thread pool queue, exceptions	quick health check, also in production
`dotnet-trace`	records an EventPipe trace (CPU samples, GC events) to open in PerfView, Visual Studio or speedscope	investigating slowness
`dotnet-dump` / `dotnet-gcdump`	process or heap snapshots to analyse which objects use memory and what roots them	investigating leaks and crashes
PerfView	free deep analysis of traces and GC behaviour	advanced investigations

DIAGNOSTIC COMMAND-LINE TOOLS

C#

```
dotnet tool install --global dotnet-counters
dotnet tool install --global dotnet-trace
dotnet tool install --global dotnet-gcdump

dotnet-counters ps                                      # list .NET processes
dotnet-counters monitor -p 12345 --counters System.Runtime
dotnet-trace collect -p 12345 --duration 00:00:30        # writes a .nettrace file
dotnet-gcdump collect -p 12345                           # heap snapshot for leak analysis
```

20.6.3 Reducing allocations

In throughput-critical code, every allocation eventually costs GC time. Common techniques, most of which appeared earlier in the book:

Technique	Instead of
StringBuilder, string.Join, string.Create, interpolation handlers	string += in loops
Span<T>, ReadOnlySpan<char> slicing, stackalloc	Substring, Split, ToArray for temporary data
ArrayPool<T>, ObjectPool<T>	new large arrays or expensive objects each call
Structs (readonly) for small values; avoid boxing	tiny class objects; passing value types as object or non-generic interfaces

Technique	Instead of
Pre-sized collections (`new List<T>(n)`)	growing and copying repeatedly
static lambdas, cached delegates	lambdas that capture variables in hot loops
ValueTask for often-synchronous async methods	Task allocations on cache hits
Frozen collections, compiled/generated Regex and JSON	repeated parsing or reflection

MEASURING ALLOCATIONS OF THREE APPROACHES

Program.cs

```
const int N = 1000;
long Measure(Action a) { long b = GC.GetAllocatedBytesForCurrentThread(); a(); return
GC.GetAllocatedBytesForCurrentThread() - b; }

long concat = Measure(() => { string s = ""; for (int i = 0; i < N; i++) s += "x"; });
long builder = Measure(() => { var sb = new System.Text.StringBuilder(N); for (int i = 0; i < N; i++)
sb.Append('x'); _ = sb.ToString(); });
long create = Measure(() => _ = string.Create(N, 'x', (span, ch) => span.Fill(ch)));

Console.WriteLine($"+= allocates over 100x more than StringBuilder: {concat > builder * 100}");
Console.WriteLine($"string.Create allocates no more than StringBuilder: {create <= builder}");
```

OUTPUT

```
+= allocates over 100x more than StringBuilder: True
string.Create allocates no more than StringBuilder: True
```

C# compiles to IL plus metadata in assemblies; the CLR JIT-compiles IL with tiering and dynamic PGO, and ReadyToRun or Native AOT trade flexibility for start-up speed. Locals live in stack frames, objects on the managed heap (SOH, LOH for objects of 85,000 bytes or more, POH for pinned buffers), and value types live inside whatever contains them unless boxed or captured. The generational GC marks from roots, sweeps and compacts, collecting gen 0 often and gen 2 rarely, in workstation or server mode; do not call GC.Collect. Release unmanaged resources with the Dispose pattern, SafeHandle and await using. Leaks come from events, statics, captures, undisposed resources and unbounded queues. Measure with BenchmarkDotNet and the dotnet diagnostic tools, then reduce allocations.

PRACTICE CHECKPOINT

Benchmark two implementations of a string-processing method with BenchmarkDotNet, and **find and fix an event-handler memory leak**.

SAMPLE SOLUTION

Program.cs

```
using System.Runtime.CompilerServices;

// Part 1: the two implementations to benchmark (run them under BenchmarkDotNet in Release:
// [MemoryDiagnoser] public class SlugBench { [Benchmark(Baseline = true)] public string A() =>
Slug.Naive(Input);
//                                          [Benchmark] public string B() => Slug.Fast(Input); })
string input = "  Hello, World! C# Is FUN  ";
Console.WriteLine($"naive: '{Slug.Naive(input)}'  fast: '{Slug.Fast(input)}'  equal: {Slug.Naive(input)
== Slug.Fast(input)}");

// Part 2: the leak
var bus = new MessageBus();                          // long-lived publisher
WeakReference leaked = SubscribeAndForget(bus, unsubscribe: false);
WeakReference fixedRef = SubscribeAndForget(bus, unsubscribe: true);
```

```
GC.Collect(); GC.WaitForPendingFinalizers(); GC.Collect();
Console.WriteLine($"without unsubscribe, view still alive: {leaked.IsAlive}");
Console.WriteLine($"with Dispose/unsubscribe, view still alive: {fixedRef.IsAlive}");
Console.WriteLine($"handlers attached to the bus: {bus.HandlerCount}");
GC.KeepAlive(bus);

[MethodImpl(MethodImplOptions.NoInlining)]
static WeakReference SubscribeAndForget(MessageBus bus, bool unsubscribe)
{
    var view = new DashboardView(bus);
    if (unsubscribe) view.Dispose();                // the fix
    return new WeakReference(view);
}

static class Slug
{
    public static string Naive(string s)           // many temporary strings
    {
        string result = "";
        foreach (string word in s.Trim().ToLower().Split(' ', StringSplitOptions.RemoveEmptyEntries))
        {
            string clean = "";
            foreach (char c in word) if (char.IsLetterOrDigit(c)) clean += c;
            if (clean.Length > 0) result += (result.Length > 0 ? "-" : "") + clean;
        }
        return result;
    }

    public static string Fast(ReadOnlySpan<char> s) // one pass, one result string
    {
        Span<char> buffer = s.Length <= 256 ? stackalloc char[s.Length] : new char[s.Length];
        int len = 0;
        bool pendingDash = false;
        foreach (char c in s)
        {
            if (char.IsLetterOrDigit(c))
            {
                if (pendingDash && len > 0) buffer[len++] = '-';
                buffer[len++] = char.ToLowerInvariant(c);
                pendingDash = false;
            }
            else if (c == ' ') pendingDash = true;
        }
        return new string(buffer[..len]);
    }
}

class MessageBus
{
    public event Action<string>? MessageArrived;
    public int HandlerCount => MessageArrived?.GetInvocationList().Length ?? 0;
}

class DashboardView : IDisposable
{
    private readonly MessageBus bus;
    private readonly byte[] bigUiState = new byte[1_000_000];      // makes the leak expensive
    public DashboardView(MessageBus bus) { this.bus = bus; bus.MessageArrived += OnMessage; }
    private void OnMessage(string m) => _ = bigUiState.Length;
    public void Dispose() => bus.MessageArrived -= OnMessage;       // unsubscribe
}
```

OUTPUT

```
naive: 'hello-world-c-is-fun'  fast: 'hello-world-c-is-fun'  equal: True
without unsubscribe, view still alive: True
with Dispose/unsubscribe, view still alive: False
handlers attached to the bus: 1
```

MODULE 21 · ADVANCED

Data access with ADO.NET and Entity Framework Core

Minimum duration **4 days** | Level **Advanced**

Almost every business application stores its data in a relational database. This module connects C# programs to databases, first with ADO.NET, the low-level API on which everything else is built (connections, commands, readers, parameters, transactions), then with the Dapper micro-ORM, and finally with Entity Framework Core, the full object-relational mapper: DbContext, code-first models, migrations, CRUD, LINQ queries, relationships, loading strategies, change tracking, scaffolding and performance. It closes with the Repository and Unit of Work patterns and layered architecture.

IN THIS MODULE

21.1 Database refresher
21.2 ADO.NET
21.3 Entity Framework Core
21.4 Architecture

NOTE

The runnable examples use **SQLite** (package Microsoft.Data.Sqlite or Microsoft.EntityFrameworkCore.Sqlite), often with an in-memory database, so they need no server and give the same output everywhere. The same code works with SQL Server, PostgreSQL or MySQL by changing the provider package, the connection class and the connection string.

21.1 Database refresher

21.1.1 Relational databases and SQL basics

A **relational database** stores data in **tables** of rows and columns. Each table has a **primary key** that identifies a row uniquely, and **foreign keys** that refer to rows of other tables, expressing relationships. **SQL** (Structured Query Language) defines tables (DDL: CREATE, ALTER, DROP) and manipulates data (DML: SELECT, INSERT, UPDATE, DELETE). The database guarantees **ACID** transactions: Atomic, Consistent, Isolated and Durable.

TABLES, KEYS, JOINS AND GROUPING

SQL

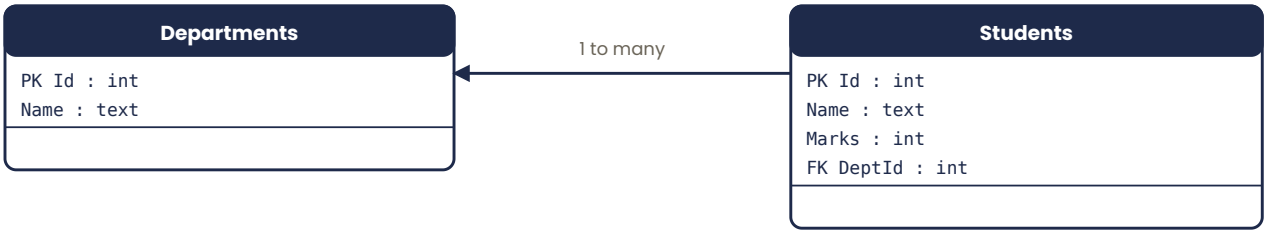
```sql
CREATE TABLE Departments (
    Id   INTEGER PRIMARY KEY,
    Name TEXT NOT NULL
);
CREATE TABLE Students (
    Id     INTEGER PRIMARY KEY,
    Name   TEXT NOT NULL,
    Marks  INTEGER,
    DeptId INTEGER REFERENCES Departments(Id)        -- foreign key
);
INSERT INTO Departments VALUES (1, 'CSE'), (2, 'ECE'), (3, 'MECH');
INSERT INTO Students VALUES (101, 'Asha', 91, 1), (102, 'Ravi', 78, 2), (103, 'Meera', 85, 1), (104,
'Kiran', 58, NULL);

SELECT s.Name, d.Name AS Dept, s.Marks
FROM Students s JOIN Departments d ON d.Id = s.DeptId
WHERE s.Marks >= 60
ORDER BY s.Marks DESC;

SELECT d.Name, COUNT(s.Id) AS Students, AVG(s.Marks) AS Avg
FROM Departments d LEFT JOIN Students s ON s.DeptId = d.Id
GROUP BY d.Name ORDER BY d.Name;
```

Name	Dept	Marks
Asha	CSE	91
Meera	CSE	85
Ravi	ECE	78

Name	Students	Avg
CSE	2	88.0
ECE	1	78.0
MECH	0	NULL



ONE DEPARTMENT HAS MANY STUDENTS; STUDENTS.DEPTID IS THE FOREIGN KEY

21.1.2 SQL Server, PostgreSQL, MySQL and SQLite

Database	Type	ADO.NET provider package	EF Core provider	Typical use
SQL Server / Azure SQL	client-server, commercial (free Express/Developer editions)	Microsoft.Data.SqlClient	Microsoft.EntityFrameworkCore.SqlServer	enterprise .NET applications

Database	Type	ADO.NET provider package	EF Core provider	Typical use
PostgreSQL	client-server, open source	Npgsql	Npgsql.EntityFrameworkCore.PostgreSQL	cloud and open-source stacks
MySQL / MariaDB	client-server, open source	MySqlConnector	Pomelo.EntityFrameworkCore.MySql	web hosting, LAMP heritage
SQLite	embedded, a single file, no server	Microsoft.Data.Sqlite	Microsoft.EntityFrameworkCore.Sqlite	mobile, desktop, tests, small sites

21.2 ADO.NET

ADO.NET (System.Data) is the foundation of all database access in .NET. It defines abstract classes (`DbConnection`, `DbCommand`, `DbDataReader`, `DbParameter`, `DbTransaction`) that each provider implements: `SqlConnection` for SQL Server, `NpgsqlConnection` for PostgreSQL, `SqliteConnection` for SQLite. Dapper and EF Core both use it underneath. Working with it directly gives full control and the best performance, at the cost of writing SQL and mapping by hand.

21.2.1 Connection strings

A **connection string** is a semicolon-separated list of key=value settings that tells the provider where the database is and how to log in. Never hard-code connection strings containing passwords in source code: store them in configuration (`appsettings.json`, environment variables, user secrets in development, Azure Key Vault in production) and read them through `IConfiguration` (Module 22).

TYPICAL CONNECTION STRINGS C#

```
// SQL Server, Windows authentication, local instance
"Server=localhost;Database=School;Trusted_Connection=True;TrustServerCertificate=True"

// SQL Server / Azure SQL with a user and password
"Server=tcp:lbf.database.windows.net,1433;Database=School;User Id=app;Password=<secret>;Encrypt=True"

// PostgreSQL
"Host=localhost;Port=5432;Database=school;Username=app;Password=<secret>"

// SQLite: a file, or a private in-memory database
"Data Source=school.db"
"Data Source=:memory:"
```

21.2.2 SqlConnection, SqlCommand, SqlDataReader

The basic flow is always the same: **open** a connection, create a **command** with SQL text, **execute** it, and for queries iterate a **data reader** row by row (forward-only, read-only, very fast). Connections, commands and readers are IDisposable, so wrap them in `using`; connection pooling (on by default) makes opening a connection cheap, so open late and close early. Async versions (`OpenAsync`, `ExecuteReaderAsync`, `ReadAsync`) should be used in web applications.

THE ADO.NET PATTERN WITH SQL SERVER C#

```
using Microsoft.Data.SqlClient;

await using var conn = new SqlConnection(connectionString);
await conn.OpenAsync();
await using var cmd = new SqlCommand("SELECT Id, Name, Marks FROM Students ORDER BY Id", conn);
await using SqlDataReader reader = await cmd.ExecuteReaderAsync();
while (await reader.ReadAsync())
{
    int id = reader.GetInt32(0);
    string name = reader.GetString(reader.GetOrdinal("Name"));
    int? marks = reader.IsDBNull(2) ? null : reader.GetInt32(2);
    Console.WriteLine($"{id} {name} {marks}");
}
```

READING ROWS WITH A DATA READER Program.cs

```
using Microsoft.Data.Sqlite;

using var conn = new SqliteConnection("Data Source=:memory:");    // same API, SQLite provider
conn.Open();

using (var create = conn.CreateCommand())
{
    create.CommandText = """
        CREATE TABLE Students (Id INTEGER PRIMARY KEY, Name TEXT NOT NULL, Marks INTEGER);
        INSERT INTO Students (Id, Name, Marks) VALUES (101, 'Asha', 91), (102, 'Ravi', 78), (103,
'Meera', NULL);
        """;
    create.ExecuteNonQuery();
}

using var cmd = conn.CreateCommand();
cmd.CommandText = "SELECT Id, Name, Marks FROM Students ORDER BY Id";
using SqliteDataReader reader = cmd.ExecuteReader();
Console.WriteLine($"columns: {reader.FieldCount}, first column name: {reader.GetName(0)}");
while (reader.Read())                                          // false after the last row
{
    int id = reader.GetInt32(0);
    string name = reader.GetString(1);
    string marks = reader.IsDBNull(2) ? "absent" : reader.GetInt32(2).ToString();   // NULL check
    Console.WriteLine($"{id} {name,-6} {marks}");
}
```

OUTPUT

```
columns: 3, first column name: Id
101 Asha   91
102 Ravi   78
103 Meera  absent
```

21.2.3 Parameterized queries and SQL injection

Building SQL by concatenating user input is the cause of **SQL injection**, consistently among the most dangerous web vulnerabilities. If a user types `' OR '1'='1` into a name box, the concatenated WHERE clause becomes always true and returns every row; worse inputs can delete tables. The cure is **parameters**: the SQL text contains placeholders (`@name`) and values are sent separately, so they are always treated as data, never as SQL. Parameters also let the database reuse the query plan.

SQL INJECTION AND THE PARAMETERIZED FIX

Program.cs

```
using Microsoft.Data.Sqlite;

using var conn = new SqliteConnection("Data Source=:memory:");
conn.Open();
var setup = conn.CreateCommand();
setup.CommandText = "CREATE TABLE Users (Name TEXT, Password TEXT); INSERT INTO Users VALUES
('asha','s3cret'),('ravi','pa55'),('admin','root!');";
setup.ExecuteNonQuery();

string userInput = "' OR '1'='1";                                   // malicious input

var unsafeCmd = conn.CreateCommand();
unsafeCmd.CommandText = "SELECT COUNT(*) FROM Users WHERE Name = '" + userInput + "'";   // NEVER do this
Console.WriteLine($"concatenated SQL: {unsafeCmd.CommandText}");
Console.WriteLine($"rows matched: {unsafeCmd.ExecuteScalar()}");

var safeCmd = conn.CreateCommand();
safeCmd.CommandText = "SELECT COUNT(*) FROM Users WHERE Name = @name";                  // placeholder
safeCmd.Parameters.AddWithValue("@name", userInput);                                    // value sent
separately
Console.WriteLine($"parameterized rows matched: {safeCmd.ExecuteScalar()}");
```

OUTPUT

```
concatenated SQL: SELECT COUNT(*) FROM Users WHERE Name = '' OR '1'='1'
rows matched: 3
parameterized rows matched: 0
```

WATCH OUT

Never build SQL from user input with + or string interpolation, including in EF Core raw SQL. In EF Core, `FromSql($"... {value}")` and `ExecuteSql` turn interpolated values into parameters safely, while `FromSqlRaw` with concatenated strings is just as vulnerable as ADO.NET.

21.2.4 ExecuteNonQuery, ExecuteScalar, ExecuteReader

Method	Returns	Use for
`ExecuteNonQuery()`	number of rows affected	INSERT, UPDATE, DELETE, DDL
`ExecuteScalar()`	the first column of the first row (object)	COUNT, SUM, MAX, a new identity value
`ExecuteReader()`	a DbDataReader over the result rows	SELECT returning rows

THE THREE EXECUTE METHODS

Program.cs

```
using Microsoft.Data.Sqlite;

using var conn = new SqliteConnection("Data Source=:memory:");
conn.Open();
Exec("CREATE TABLE Products (Id INTEGER PRIMARY KEY AUTOINCREMENT, Name TEXT, Price REAL, Stock
INTEGER)");

var insert = conn.CreateCommand();
insert.CommandText = "INSERT INTO Products (Name, Price, Stock) VALUES (@n, @p, @s); SELECT
last_insert_rowid();";
var pn = insert.Parameters.Add("@n", SqliteType.Text);
var pp = insert.Parameters.Add("@p", SqliteType.Real);
var ps = insert.Parameters.Add("@s", SqliteType.Integer);
foreach (var (name, price, stock) in new[] { ("Pen", 10.5, 100), ("Notebook", 45.0, 0), ("Bag", 899.0,
12) })
{
    pn.Value = name; pp.Value = price; ps.Value = stock;          // reuse one command with new values
    Console.WriteLine($"inserted {name} with id {insert.ExecuteScalar()}");
}

Console.WriteLine($"rows updated: {Exec("UPDATE Products SET Price = Price * 1.1 WHERE Stock > 0")}");
Console.WriteLine($"rows deleted: {Exec("DELETE FROM Products WHERE Stock = 0")}");

var scalar = conn.CreateCommand();
scalar.CommandText = "SELECT ROUND(SUM(Price * Stock), 2) FROM Products";
Console.WriteLine($"stock value: {Convert.ToDouble(scalar.ExecuteScalar()):N2}");

int Exec(string sql)
{
    using var c = conn.CreateCommand();
    c.CommandText = sql;
    return c.ExecuteNonQuery();                                  // rows affected
}
```

OUTPUT

```
inserted Pen with id 1
inserted Notebook with id 2
inserted Bag with id 3
rows updated: 2
rows deleted: 1
stock value: 13,021.80
```

21.2.5 Transactions

A **transaction** groups several statements into one atomic unit: either all of them take effect (`Commit`) or none do (`Rollback`). The classic example is a money transfer, which must not debit one account without crediting the other. In ADO.NET, begin a transaction on the connection, attach it to every command, commit at the end, and roll back if anything fails (disposing an uncommitted transaction also rolls it back).

AN ATOMIC MONEY TRANSFER

Program.cs

```
using Microsoft.Data.Sqlite;

using var conn = new SqliteConnection("Data Source=:memory:");
conn.Open();
Run(null, "CREATE TABLE Accounts (Id INTEGER PRIMARY KEY, Owner TEXT, Balance REAL CHECK (Balance >=
0))");
Run(null, "INSERT INTO Accounts VALUES (1, 'Asha', 5000), (2, 'Ravi', 1000)");

Transfer(1, 2, 1500);          // succeeds
Transfer(2, 1, 9000);          // the CHECK constraint fails on the debit: everything is rolled back
Show();
```

```
void Transfer(int from, int to, double amount)
{
    using SqliteTransaction tx = conn.BeginTransaction();
    try
    {
        Run(tx, $"UPDATE Accounts SET Balance = Balance + {amount} WHERE Id = {to}");    // credit first
        Run(tx, $"UPDATE Accounts SET Balance = Balance - {amount} WHERE Id = {from}");  // may violate
CHECK
        tx.Commit();
        Console.WriteLine($"transfer {from}->{to} of {amount}: committed");
    }
    catch (SqliteException)
    {
        tx.Rollback();                                          // undo the credit as well
        Console.WriteLine($"transfer {from}->{to} of {amount}: rolled back");
    }
}

void Show()
{
    using var c = conn.CreateCommand();
    c.CommandText = "SELECT Owner, Balance FROM Accounts ORDER BY Id";
    using var r = c.ExecuteReader();
    while (r.Read()) Console.WriteLine($"{r.GetString(0)}: {r.GetDouble(1)}");
}

void Run(SqliteTransaction? tx, string sql)
{
    using var c = conn.CreateCommand();
    c.Transaction = tx;
    c.CommandText = sql;                    // fine here: values are numbers from our own code, not user
input
    c.ExecuteNonQuery();
}
```

OUTPUT

```
transfer 1->2 of 1500: committed
transfer 2->1 of 9000: rolled back
Asha: 3500
Ravi: 2500
```

21.2.6 Dapper micro-ORM (overview)

Writing readers and parameter code by hand is repetitive. **Dapper** (NuGet package Dapper, created at Stack Overflow) is a **micro-ORM**: a set of extension methods on `IDbConnection` that run your SQL and map result columns to object properties by name, with parameters taken from an anonymous object. You keep full control of the SQL and get performance close to hand-written ADO.NET, without EF Core's change tracking, migrations or LINQ translation.

DAPPER: SQL IN, OBJECTS OUT

Program.cs

```
using Dapper;
using Microsoft.Data.Sqlite;

using var conn = new SqliteConnection("Data Source=:memory:");
conn.Open();
conn.Execute("CREATE TABLE Students (Id INTEGER PRIMARY KEY, Name TEXT, Marks INTEGER)");
int added = conn.Execute("INSERT INTO Students (Id, Name, Marks) VALUES (@Id, @Name, @Marks)",
    new[] { new Student(1, "Asha", 91), new Student(2, "Ravi", 78), new Student(3, "Meera", 85) });  // runs once per item
Console.WriteLine($"inserted {added}");

IEnumerable<Student> toppers = conn.Query<Student>(
    "SELECT Id, Name, Marks FROM Students WHERE Marks >= @min ORDER BY Marks DESC", new { min = 80 });
foreach (var s in toppers) Console.WriteLine(s);

int count = conn.ExecuteScalar<int>("SELECT COUNT(*) FROM Students");
Student? one = conn.QuerySingleOrDefault<Student>("SELECT * FROM Students WHERE Id = @id", new { id = 2 });
Console.WriteLine($"{count} students; id 2 is {one?.Name}");

public record Student(long Id, string Name, long Marks);
```

OUTPUT

```
inserted 3
Student { Id = 1, Name = Asha, Marks = 91 }
Student { Id = 3, Name = Meera, Marks = 85 }
3 students; id 2 is Ravi
```

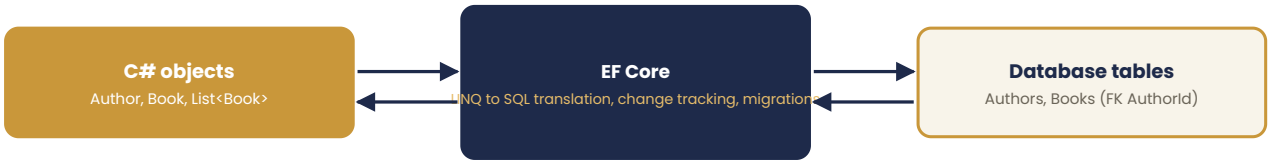
NOTE

SQLite returns integers as 64-bit values, so the record uses `long` to match Dapper's constructor mapping exactly; with SQL Server an `int` column maps to `int`.

21.3 Entity Framework Core

21.3.1 What an ORM is

Relational databases think in tables, rows and foreign keys; C# thinks in objects, references and collections. Translating between them by hand (writing SQL, reading columns, setting properties) is repetitive and error-prone; this gap is called the **object-relational impedance mismatch**. An **ORM** (Object-Relational Mapper) does the translation automatically: you work with C# classes and LINQ, and the ORM generates the SQL, materialises objects, tracks changes and saves them. **Entity Framework Core** (EF Core) is Microsoft's open-source, cross-platform ORM and the standard choice in ASP.NET Core applications.



EF CORE MAPS BETWEEN OBJECTS AND TABLES IN BOTH DIRECTIONS

	ADO.NET	Dapper	EF Core
You write	SQL + mapping code	SQL	LINQ (SQL generated)

	ADO.NET	Dapper	EF Core
Mapping	manual	automatic for results	automatic, both directions
Change tracking / SaveChanges	no	no	yes
Schema migrations	no	no	yes
Performance	fastest	near ADO.NET	good; excellent with care
Best for	special cases, bulk work	read-heavy, SQL-savvy teams	most business applications

21.3.2 Installing EF Core packages

EF Core is distributed as NuGet packages: one **provider** package for your database, plus design-time packages for migrations. The `dotnet ef` command-line tool is installed separately as a .NET tool.

PACKAGES AND TOOLS

C#

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite       # or .SqlServer / Npgsql.EntityFrameworkCore.PostgreSQL
dotnet add package Microsoft.EntityFrameworkCore.Design       # needed by migrations and scaffolding
dotnet tool install --global dotnet-ef                        # the 'dotnet ef' command
dotnet ef --version
```

21.3.3 DbContext and DbSet<T>

A **DbContext** represents a session with the database: it holds the configuration (provider, connection string), the model (which classes map to which tables), the **change tracker** and the methods to query and save. Each `DbSet<T>` property represents a table and is the starting point for LINQ queries. A context is lightweight and meant to be short-lived: one per unit of work (in ASP.NET Core, one per request via dependency injection), always disposed, and never shared between threads.

A DBCONTEXT

C#

```
using Microsoft.EntityFrameworkCore;

public class LibraryContext : DbContext
{
    public DbSet<Author> Authors => Set<Author>();      // table Authors
    public DbSet<Book> Books => Set<Book>();            // table Books

    protected override void OnConfiguring(DbContextOptionsBuilder options) =>
        options.UseSqlite("Data Source=library.db");    // provider + connection string

    // In ASP.NET Core, prefer constructor injection of options instead:
    // public LibraryContext(DbContextOptions<LibraryContext> options) : base(options) { }
    // builder.Services.AddDbContext<LibraryContext>(o => o.UseSqlServer(connStr));
}
```

21.3.4 Code-first: entities, conventions, data annotations and Fluent API

In **code-first** development you write plain C# classes (**entities**) and EF Core derives the database schema from them. It applies **conventions**: a property named `Id` or `<Type>Id` is the primary key (an int key is auto-generated); a navigation property plus a `<Navigation>Id` property form a foreign key; non-nullable reference types become NOT NULL columns; table names come from the DbSet names. Where conventions

are not enough, configure with **data annotations** (attributes on the entity) or the **Fluent API** in `OnModelCreating`, which is more powerful and keeps entities clean.

ENTITIES CONFIGURED BY CONVENTION, ANNOTATIONS AND FLUENT API C#

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

public class Author
{
    public int Id { get; set; }                          // convention: primary key, identity
    [Required, MaxLength(100)] public string Name { get; set; } = "";   // data annotations
    public string? Country { get; set; }                 // nullable -> NULL allowed
    public List<Book> Books { get; set; } = new();       // navigation: one author, many books
}

public class Book
{
    public int Id { get; set; }
    public string Title { get; set; } = "";
    public int Year { get; set; }
    [Column(TypeName = "decimal(8,2)")] public decimal Price { get; set; }
    public int AuthorId { get; set; }                    // convention: foreign key to Author
    public Author Author { get; set; } = null!;          // navigation back to the author
}

// Fluent API in the DbContext: same kind of configuration, kept out of the entities
protected override void OnModelCreating(ModelBuilder model)
{
    model.Entity<Book>(b =>
    {
        b.ToTable("Books");
        b.Property(x => x.Title).IsRequired().HasMaxLength(200);
        b.HasIndex(x => x.Title);
        b.HasOne(x => x.Author).WithMany(a => a.Books).HasForeignKey(x =>
x.AuthorId).OnDelete(DeleteBehavior.Cascade);
    });
    model.Entity<Author>().HasData(new Author { Id = 1, Name = "R. K. Narayan", Country = "India" });
// seed data
}
```

21.3.5 Migrations: dotnet ef migrations add, database update

As the model changes, the database schema must follow without losing data. A **migration** is a generated C# class with an `Up` method (apply the change) and a `Down` method (revert it), created by comparing the current model with a snapshot of the previous one. `dotnet ef database update` applies pending migrations; the database records applied ones in the `__EFMigrationsHistory` table. Migrations are source code: review them, commit them, and apply them in deployment (via update, an idempotent SQL script, or a migration bundle).

Creating and applying migrations (timestamps vary)

```
$ dotnet ef migrations add InitialCreate
Build started...
Build succeeded.
Done. To undo this action, use 'ef migrations remove'

$ dotnet ef database update
Build started...
Build succeeded.
Applying migration '20260309101500_InitialCreate'.
Done.

$ dotnet ef migrations add AddBookPrice          # after adding Price to Book
$ dotnet ef migrations script --idempotent -o deploy.sql
```

A GENERATED MIGRATION C#

```
public partial class AddBookPrice : Migration
{
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.AddColumn<decimal>(
            name: "Price", table: "Books", type: "decimal(8,2)", nullable: false, defaultValue: 0m);
    }

    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropColumn(name: "Price", table: "Books");
    }
}
```

TIP

For quick experiments and tests, `context.Database.EnsureCreated()` creates the schema directly without migrations (the examples below use it with an in-memory SQLite database). Do not mix EnsureCreated with migrations on a real database.

21.3.6 CRUD operations and SaveChanges

Create: `Add` an entity. **Read**: query a DbSet with LINQ or `Find(key)`. **Update**: change properties of a tracked entity. **Delete**: `Remove` it. None of these touch the database until `SaveChanges()` (or `SaveChangesAsync`), which sends all pending inserts, updates and deletes in one transaction and fills in generated keys. EF Core 7+ also offers `ExecuteUpdate` and `ExecuteDelete` for set-based changes that run immediately without loading entities.

CREATE, READ, UPDATE, DELETE Program.cs

```
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;

using var conn = new SqliteConnection("Data Source=:memory:");
conn.Open();
var options = new DbContextOptionsBuilder<LibraryContext>().UseSqlite(conn).Options;
using (var db = new LibraryContext(options)) db.Database.EnsureCreated();

using (var db = new LibraryContext(options))
{
    var narayan = new Author { Name = "R. K. Narayan", Country = "India" };              // CREATE
    narayan.Books.Add(new Book { Title = "Malgudi Days", Year = 1943, Price = 299 });
    narayan.Books.Add(new Book { Title = "The Guide", Year = 1958, Price = 350 });
    db.Authors.Add(narayan);                                          // the books are added with it
```

```
    db.Authors.Add(new Author { Name = "Ruskin Bond", Country = "India" });
    int saved = db.SaveChanges();                                  // one transaction
    Console.WriteLine($"rows saved: {saved}, Narayan's id: {narayan.Id}, first book id:
{narayan.Books[0].Id}");
}

using (var db = new LibraryContext(options))
{
    Book guide = db.Books.Single(b => b.Title == "The Guide");    // READ
    guide.Price = 399;                                             // UPDATE (tracked change)
    Author bond = db.Authors.Find(2)!;
    db.Authors.Remove(bond);                                       // DELETE
    Console.WriteLine($"changes saved: {db.SaveChanges()}");

    int raised = db.Books.Where(b => b.Year < 1950).ExecuteUpdate(s => s.SetProperty(b => b.Price, b =>
b.Price + 50));   // set-based
    Console.WriteLine($"ExecuteUpdate changed {raised} row(s)");
}

using (var db = new LibraryContext(options))
    foreach (var b in db.Books.OrderBy(b => b.Year)) Console.WriteLine($"{b.Id} {b.Title} ({b.Year}) Rs
{b.Price}");
Console.WriteLine($"authors left: {new LibraryContext(options).Authors.Count()}");

public class LibraryContext(DbContextOptions<LibraryContext> options) : DbContext(options)
{
    public DbSet<Author> Authors => Set<Author>();
    public DbSet<Book> Books => Set<Book>();
}
public class Author { public int Id { get; set; } public string Name { get; set; } = ""; public string?
Country { get; set; } public List<Book> Books { get; set; } = new(); }
public class Book { public int Id { get; set; } public string Title { get; set; } = ""; public int Year {
get; set; } public double Price { get; set; } public int AuthorId { get; set; } public Author Author {
get; set; } = null!; }
```

OUTPUT

```
rows saved: 4, Narayan's id: 1, first book id: 1
changes saved: 2
ExecuteUpdate changed 1 row(s)
1 Malgudi Days (1943) Rs 349
2 The Guide (1958) Rs 399
authors left: 1
```

NOTE

Price is a `double` here because SQLite has no native decimal type (EF Core stores decimal as text there and cannot translate arithmetic or ordering on it). With SQL Server or PostgreSQL use `decimal` for money. The later examples reuse this LibraryContext, Author and Book.

21.3.7 Querying with LINQ

LINQ queries against a DbSet are `IQueryable` (Module 16): EF Core translates the expression tree to SQL when the query is enumerated or a terminal operator (`ToList`, `First`, `Count`, `Sum`) runs, and only the requested rows and columns cross the network. `ToQueryString()` shows the generated SQL. Methods with no SQL translation cause an exception (or must be applied after `AsEnumerable()`, in memory).

LINQ QUERIES TRANSLATED TO SQL

Program.cs

```
// assumes the LibraryContext, Author and Book types and data from 21.3.6 (before the delete), seeded
with:
// Narayan: Malgudi Days (1943, 299), The Guide (1958, 350); Ruskin Bond: The Blue Umbrella (1980, 150),
Rusty (1955, 199)
using var db = Library.CreateSeeded();

var titles = db.Books.Where(b => b.Year > 1950).OrderBy(b => b.Year).Select(b => b.Title).ToList();
Console.WriteLine(string.Join(", ", titles));

var perAuthor = db.Authors
    .Select(a => new { a.Name, Count = a.Books.Count, Newest = a.Books.Max(b => b.Year) })   //
projection across a relationship
    .OrderBy(x => x.Name)
    .ToList();
foreach (var x in perAuthor) Console.WriteLine($"{x.Name}: {x.Count} books, newest {x.Newest}");

double total = db.Books.Sum(b => b.Price);
bool anyOld = db.Books.Any(b => b.Year < 1945);
var page2 = db.Books.OrderBy(b => b.Title).Skip(2).Take(2).Select(b => b.Title).ToList();     // paging
in SQL
Console.WriteLine($"total {total}, any before 1945: {anyOld}, page 2: {string.Join(", ", page2)}");

Console.WriteLine(db.Books.Where(b => b.Year > 1950).Select(b => b.Title).ToQueryString());
```

OUTPUT

```
Rusty, The Guide, The Blue Umbrella
R. K. Narayan: 2 books, newest 1958
Ruskin Bond: 2 books, newest 1980
total 998, any before 1945: True, page 2: The Blue Umbrella, The Guide
SELECT "b"."Title"
FROM "Books" AS "b"
WHERE "b"."Year" > 1950
```

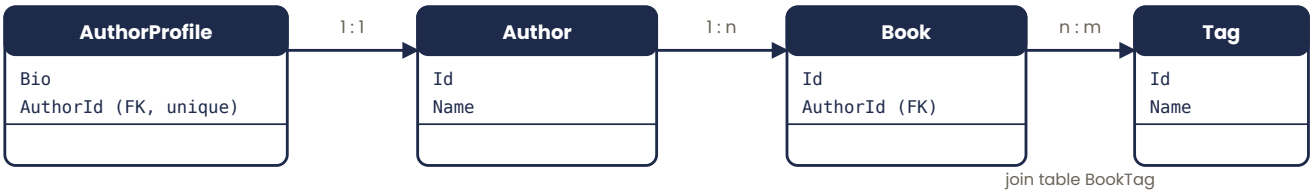
NOTE

`Library.CreateSeeded()` stands for the setup code of 21.3.6 (open the in-memory connection, EnsureCreated, add the two authors and four books) wrapped in a helper; it is omitted to save space.

21.3.8 Relationships: one-to-one, one-to-many, many-to-many

Relationships are expressed with **navigation properties** and foreign keys. EF Core discovers most of them by convention and can be configured with the Fluent API (`HasOne` / `HasMany`, `WithOne` / `WithMany`).

Relationship	Example	In the classes	In the database
One-to-many	an Author has many Books	`List<Book> Books` in Author; `Author Author` + `int AuthorId` in Book	Books.AuthorId foreign key
One-to-one	an Author has one Profile	`AuthorProfile? Profile` in Author; `int AuthorId` + navigation in AuthorProfile	unique foreign key in AuthorProfiles
Many-to-many	Books have many Tags, Tags label many Books	`List<Tag> Tags` in Book and `List<Book> Books` in Tag	a join table BookTag (created automatically)



THE THREE KINDS OF RELATIONSHIPS

USING ONE-TO-ONE AND MANY-TO-MANY RELATIONSHIPS

Program.cs

```
using var db = Library.CreateSeeded();                          // the context from 21.3.7 plus Profile
and Tag entities

var guide = db.Books.Single(b => b.Title == "The Guide");
var classic = new Tag { Name = "classic" };
var fiction = new Tag { Name = "fiction" };
guide.Tags.AddRange([classic, fiction]);                         // many-to-many: no join entity needed
db.Books.Single(b => b.Title == "Rusty").Tags.Add(fiction);
db.Authors.Single(a => a.Name == "R. K. Narayan").Profile = new AuthorProfile { Bio = "Author of Malgudi"
};   // one-to-one
db.SaveChanges();

foreach (var t in db.Tags.OrderBy(t => t.Name).Select(t => new { t.Name, Books = t.Books.Select(b =>
b.Title).OrderBy(x => x) }))
    Console.WriteLine($"{t.Name}: {string.Join(", ", t.Books)}");
Console.WriteLine(db.Authors.Where(a => a.Profile != null).Select(a => a.Name + ": " +
a.Profile!.Bio).Single());

public class Tag { public int Id { get; set; } public string Name { get; set; } = ""; public List<Book>
Books { get; set; } = new(); }
public class AuthorProfile { public int Id { get; set; } public string Bio { get; set; } = ""; public int
AuthorId { get; set; } }
// Book gains:   public List<Tag> Tags { get; set; } = new();
// Author gains: public AuthorProfile? Profile { get; set; }
```

OUTPUT

```
classic: The Guide
fiction: Rusty, The Guide
R. K. Narayan: Author of Malgudi
```

21.3.9 Loading related data: eager (Include), explicit, lazy

By default a query returns only the entities you asked for; navigation properties stay empty (null or an empty list) unless their data is loaded. EF Core offers three strategies. **Eager loading** with `Include` (and `ThenInclude` for deeper levels) loads related data in the same query, the usual choice. **Explicit loading** loads a navigation later on demand with `Entry(x).Collection(...).Load()`. **Lazy loading** (package Microsoft.EntityFrameworkCore.Proxies, virtual navigation properties) loads automatically the first time a navigation is accessed, convenient but a common cause of the N+1 problem (21.3.12). Projections with `Select` avoid the question entirely by fetching exactly what is needed.

EAGER AND EXPLICIT LOADING

Program.cs

```
using var db = Library.CreateSeeded();

var plain = db.Authors.First(a => a.Name == "Ruskin Bond");
Console.WriteLine($"without Include: {plain.Books.Count} books loaded");

db.ChangeTracker.Clear();                                          // forget tracked entities for a fair
comparison
var eager = db.Authors.Include(a => a.Books).First(a => a.Name == "Ruskin Bond");   // eager: JOIN in one
query
Console.WriteLine($"with Include: {eager.Books.Count} books: {string.Join(", ", eager.Books.OrderBy(b =>
b.Year).Select(b => b.Title))}");

db.ChangeTracker.Clear();
var expl = db.Authors.First(a => a.Name == "R. K. Narayan");
db.Entry(expl).Collection(a => a.Books).Query().Where(b => b.Year > 1950).Load();   // explicit, even
filtered
Console.WriteLine($"explicitly loaded: {string.Join(", ", expl.Books.Select(b => b.Title))}");

var book = db.Books.Include(b => b.Author).First(b => b.Title == "Rusty");          // reference
navigation
Console.WriteLine($"{book.Title} by {book.Author.Name}");
```

OUTPUT

```
without Include: 0 books loaded
with Include: 2 books: Rusty, The Blue Umbrella
explicitly loaded: The Guide
Rusty by Ruskin Bond
```

NOTE

If related entities are already being tracked by the same context, EF Core "fixes up" navigations automatically, so results can differ from what a fresh query would load. That is why the example clears the change tracker between steps.

21.3.10 Change tracking and AsNoTracking

Every entity returned by a query (or added) is **tracked** by the context, which keeps a snapshot of its original values. On `SaveChanges`, EF Core compares current values with the snapshots (`DetectChanges`) and generates UPDATE statements only for modified properties. Each tracked entity has a **state**: Added, Unchanged, Modified, Deleted, or Detached when not tracked. Tracking costs memory and time, so read-only queries (lists for display, reports, API GET endpoints) should use `AsNoTracking()`.

ENTITY STATES

Program.cs

```
using var db = Library.CreateSeeded();

var book = new Book { Title = "Swami and Friends", Year = 1935, Price = 250, AuthorId = 1 };
Console.WriteLine($"new object:       {db.Entry(book).State}");
db.Books.Add(book);
Console.WriteLine($"after Add:        {db.Entry(book).State}");
db.SaveChanges();
Console.WriteLine($"after SaveChanges: {db.Entry(book).State}");
book.Price = 275;
Console.WriteLine($"after edit:       {db.Entry(book).State}, modified: {string.Join(",",
db.Entry(book).Properties.Where(p => p.IsModified).Select(p => p.Metadata.Name))}");
db.Books.Remove(book);
Console.WriteLine($"after Remove:     {db.Entry(book).State}");
db.SaveChanges();
Console.WriteLine($"after delete:     {db.Entry(book).State}");

var readOnly = db.Books.AsNoTracking().Where(b => b.Year > 1950).ToList();     // not tracked
Console.WriteLine($"no-tracking entity state: {db.Entry(readOnly[0]).State}, tracked entries:
{db.ChangeTracker.Entries().Count()}");
```

OUTPUT

```
new object:       Detached
after Add:        Added
after SaveChanges: Unchanged
after edit:       Modified, modified: Price
after Remove:     Deleted
after delete:     Detached
no-tracking entity state: Detached, tracked entries: 0
```

The tracked-entries count is 0 here because `Library.CreateSeeded()` returns a fresh context after seeding and the only entity tracked later was deleted. To update a disconnected entity received from an API, either load and modify it, call `Update(entity)` (marks all properties modified), or use `ExecuteUpdate`.

21.3.11 Database-first scaffolding

When the database already exists (a legacy system, or a schema owned by DBAs), **database-first** development generates the entity classes and DbContext from it with `dotnet ef dbcontext scaffold`. Generated classes are partial so that your additions survive re-scaffolding.

SCAFFOLDING A CONTEXT FROM AN EXISTING DATABASE

C#

```
dotnet ef dbcontext scaffold
"Server=localhost;Database=Legacy;Trusted_Connection=True;TrustServerCertificate=True" \
    Microsoft.EntityFrameworkCore.SqlServer \
    --output-dir Models --context LegacyContext --table Customers --table Orders --data-annotations --no-
onconfiguring
```

21.3.12 Performance tips and the N+1 problem

The most common EF Core performance bug is the **N+1 query problem**: one query loads N parent rows, then code touches a navigation property for each row, triggering one more query per row (with lazy loading or explicit loads in a loop). 100 authors become 101 round trips. Fix it by loading everything needed in one query with `Include` or, better, a `Select` projection.

THE N+1 PROBLEM AND ITS FIX

C#

```
// N+1: 1 query for authors + 1 query per author (lazy loading or Load() in the loop)
foreach (var a in db.Authors.ToList())
    Console.WriteLine($"{a.Name}: {a.Books.Count}");        // each access hits the database

// 1 query: project exactly what the page needs
var rows = db.Authors.Select(a => new { a.Name, Count = a.Books.Count }).ToList();
```

Tip	Why
Project with `Select` to DTOs	fetch only needed columns; no tracking overhead
`AsNoTracking()` for read-only queries	no snapshots or identity resolution
Filter, sort and page in the database (Where, OrderBy, Skip/Take before ToList)	avoid loading whole tables into memory
Avoid N+1: Include or projection; consider `AsSplitQuery()` for several collection Includes	fewer round trips; avoid cartesian explosion
`ExecuteUpdate` / `ExecuteDelete` for bulk changes	one SQL statement instead of loading entities
Use async methods (ToListAsync, SaveChangesAsync) in web apps	free threads while waiting
Add indexes (`HasIndex`) for filtered and joined columns	database-side speed
Log generated SQL (`LogTo(Console.WriteLine)`) during development	see what really runs
Compiled queries and DbContext pooling (`AddDbContextPool`) for hot paths	less per-query overhead

21.4 Architecture

21.4.1 Repository and Unit of Work patterns

The **Repository** pattern hides data access behind an interface shaped like an in-memory collection of domain objects (`GetById`, `Add`, `Find`), so business logic does not depend on EF Core or SQL and can be tested with fakes. The **Unit of Work** pattern groups the changes of one business operation and commits them together. In EF Core, `DbSet<T>` already is a repository and `DbContext` already is a unit of work, so many teams use the context directly in services; a thin custom repository is still useful to centralise complex queries, enforce aggregate boundaries, or isolate the domain layer.

REPOSITORY AND UNIT OF WORK INTERFACES C#

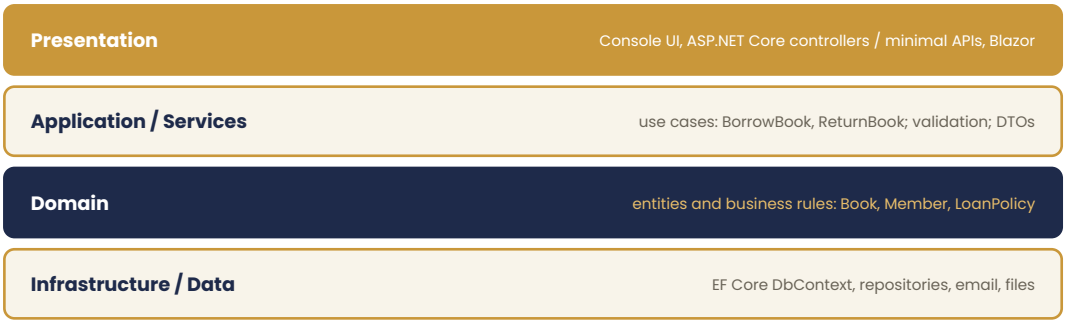
```
public interface IBookRepository
{
    Task<Book?> GetByIdAsync(int id, CancellationToken ct = default);
    Task<List<Book>> GetByAuthorAsync(string author, CancellationToken ct = default);
    void Add(Book book);
    void Remove(Book book);
}

public interface IUnitOfWork
{
    IBookRepository Books { get; }
    IAuthorRepository Authors { get; }
    Task<int> SaveChangesAsync(CancellationToken ct = default);   // commits everything together
}

public class EfBookRepository(LibraryContext db) : IBookRepository
{
    public Task<Book?> GetByIdAsync(int id, CancellationToken ct = default) =>
        db.Books.Include(b => b.Author).FirstOrDefaultAsync(b => b.Id == id, ct);
    public Task<List<Book>> GetByAuthorAsync(string author, CancellationToken ct = default) =>
        db.Books.AsNoTracking().Where(b => b.Author.Name == author).OrderBy(b => b.Year).ToListAsync(ct);
    public void Add(Book book) => db.Books.Add(book);
    public void Remove(Book book) => db.Books.Remove(book);
}
```

21.4.2 Layered architecture

Larger applications separate responsibilities into **layers**, each depending only on the one below (or, in Clean Architecture, on inner abstractions). This keeps business rules independent of the UI and the database, makes each part testable, and lets you change one technology without rewriting the rest.



A TYPICAL LAYERED .NET APPLICATION

Layer	Project (example)	Depends on
Presentation	Library.Api / Library.Console	Application
Application	Library.Application	Domain (and repository interfaces)
Domain	Library.Domain	nothing
Infrastructure	Library.Infrastructure	Application/Domain interfaces, EF Core

Relational databases store tables linked by keys and are queried with SQL. ADO.NET connects with connection strings, executes commands with ExecuteNonQuery/Scalar/Reader, reads rows with data readers, must always use parameters to prevent SQL injection, and groups statements in transactions. Dapper maps SQL results to objects with little overhead. EF Core maps classes to tables through a DbContext and DbSets, built code-first with conventions, annotations or the Fluent API and evolved with migrations (or scaffolded from an existing database). Add/modify/Remove then SaveChanges; query with LINQ; model one-to-one, one-to-many and many-to-many relationships; load related data with Include, explicit or lazy loading; understand change tracking and use AsNoTracking for reads; avoid N+1 queries. Repositories, units of work and layers organise data access in bigger applications.

PRACTICE CHECKPOINT

Build a **Library Management** console app with **EF Core** and SQLite (or SQL Server), using **migrations**, a **one-to-many Author-Book** relationship and full **CRUD**.

SAMPLE SOLUTION

Program.cs

```
// dotnet add package Microsoft.EntityFrameworkCore.Sqlite ; dotnet add package
Microsoft.EntityFrameworkCore.Design
// dotnet ef migrations add InitialCreate ; dotnet ef database update     (creates library.db)
using Microsoft.EntityFrameworkCore;

using var db = new LibraryContext();
db.Database.Migrate();                                  // applies pending migrations at start-up
db.Books.ExecuteDelete(); db.Authors.ExecuteDelete();   // start clean for this demo run

var service = new LibraryService(db);
int narayan = service.AddAuthor("R. K. Narayan", "India");
int austen = service.AddAuthor("Jane Austen", "UK");
service.AddBook(narayan, "Malgudi Days", 1943, 3);
service.AddBook(narayan, "The Guide", 1958, 1);
service.AddBook(austen, "Pride and Prejudice", 1813, 2);
service.PrintCatalogue();

service.Borrow("The Guide");
service.Borrow("The Guide");                            // no copies left
service.UpdateYear("Malgudi Days", 1942);
service.DeleteAuthor(austen);                           // cascade deletes her books
service.PrintCatalogue();

public class LibraryService(LibraryContext db)
{
    public int AddAuthor(string name, string country)
    {
        var a = new Author { Name = name, Country = country };
        db.Authors.Add(a);
        db.SaveChanges();
        return a.Id;
    }

    public void AddBook(int authorId, string title, int year, int copies)
    {
        db.Books.Add(new Book { AuthorId = authorId, Title = title, Year = year, Copies = copies });
        db.SaveChanges();
    }

    public void Borrow(string title)
    {
        var book = db.Books.Single(b => b.Title == title);
        if (book.Copies == 0) { Console.WriteLine($"'{title}' is not available"); return; }
        book.Copies--;
```

```
        db.SaveChanges();
        Console.WriteLine($"borrowed '{title}', {book.Copies} left");
    }

    public void UpdateYear(string title, int year) =>
        Console.WriteLine($"updated {db.Books.Where(b => b.Title == title).ExecuteUpdate(s =>
s.SetProperty(b => b.Year, year))} book(s)");

    public void DeleteAuthor(int id)
    {
        db.Authors.Remove(db.Authors.Include(a => a.Books).Single(a => a.Id == id));
        Console.WriteLine($"deleted author and books: {db.SaveChanges()} row(s)");
    }

    public void PrintCatalogue()
    {
        Console.WriteLine("--- catalogue ---");
        foreach (var a in db.Authors.AsNoTracking().Include(a => a.Books).OrderBy(a => a.Name))
        {
            Console.WriteLine($"{a.Name} ({a.Country})");
            foreach (var b in a.Books.OrderBy(b => b.Year)) Console.WriteLine($"   {b.Title}, {b.Year},
copies {b.Copies}");
        }
    }
}

public class LibraryContext : DbContext
{
    public DbSet<Author> Authors => Set<Author>();
    public DbSet<Book> Books => Set<Book>();
    protected override void OnConfiguring(DbContextOptionsBuilder o) => o.UseSqlite("Data
Source=library.db");
    protected override void OnModelCreating(ModelBuilder m) =>
        m.Entity<Book>().HasOne(b => b.Author).WithMany(a => a.Books).HasForeignKey(b =>
b.AuthorId).OnDelete(DeleteBehavior.Cascade);
}

public class Author { public int Id { get; set; } public string Name { get; set; } = ""; public string
Country { get; set; } = ""; public List<Book> Books { get; set; } = new(); }
public class Book { public int Id { get; set; } public string Title { get; set; } = ""; public int Year {
get; set; } public int Copies { get; set; } public int AuthorId { get; set; } public Author Author { get;
set; } = null!; }
```

OUTPUT

```
--- catalogue ---
Jane Austen (UK)
   Pride and Prejudice, 1813, copies 2
R. K. Narayan (India)
   Malgudi Days, 1943, copies 3
   The Guide, 1958, copies 1
borrowed 'The Guide', 0 left
'The Guide' is not available
updated 1 book(s)
deleted author and books: 2 row(s)
--- catalogue ---
R. K. Narayan (India)
   Malgudi Days, 1942, copies 3
   The Guide, 1958, copies 0
```

MODULE 22 · ADVANCED

Web development with ASP.NET Core

Minimum duration **6 days** | Level **Advanced**

ASP.NET Core is the cross-platform, open-source framework for building web APIs, websites and real-time services in C#, and it is where most professional C# developers work. This module starts with web fundamentals (HTTP and REST), then covers the structure of an ASP.NET Core application (host, builder, middleware, configuration, logging), dependency injection, building APIs with minimal APIs and controllers (routing, model binding, validation, DTOs, results, OpenAPI, error handling), security (authentication, JWT, Identity, authorization, CORS, HTTPS), the web UI options, and further topics: caching, background services, SignalR, gRPC and deployment.

IN THIS MODULE

22.1 Web fundamentals
22.2 ASP.NET Core basics
22.3 Dependency injection
22.4 Building APIs
22.5 Security
22.6 Web UI options (overview)
22.7 Going further

NOTE

Web examples are complete Program.cs files for a project created with `dotnet new web` or `dotnet new webapi` on .NET 10. Their "output" is shown as HTTP exchanges made with curl or an .http file against the local development server; response headers that change on every request (Date) are omitted, and ports differ per project.

22.1 Web fundamentals

22.1.1 HTTP methods, status codes, headers

HTTP is a request-response protocol. A client (browser, mobile app, another service) sends a **request**: a method, a URL path, headers and an optional body. The server returns a **response**: a status code, headers and an optional body. HTTP is stateless: each request carries everything the server needs (for example an authentication token).

An HTTP request and its response

```
GET /api/students/101 HTTP/1.1
Host: localhost:5080
Accept: application/json
Authorization: Bearer eyJhbGciOi...

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 41

{"id":101,"name":"Asha","marks":91}
```

Method	Meaning	Safe	Idempotent	Typical response
GET	read a resource	yes	yes	200 OK with body
POST	create a resource / perform an action	no	no	201 Created + Location header
PUT	replace a resource completely	no	yes	204 No Content or 200
PATCH	change part of a resource	no	not necessarily	200 / 204
DELETE	remove a resource	no	yes	204 No Content
HEAD / OPTIONS	headers only / allowed methods (CORS preflight)	yes	yes	200 / 204

Class	Common codes
1xx informational	101 Switching Protocols (WebSockets)
2xx success	200 OK, 201 Created, 202 Accepted, 204 No Content
3xx redirection	301 Moved Permanently, 302 Found, 304 Not Modified
4xx client error	400 Bad Request, 401 Unauthorized (not authenticated), 403 Forbidden (not allowed), 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 429 Too Many Requests
5xx server error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Important **headers**: `Content-Type` (format of the body, e.g. application/json), `Accept` (formats the client wants), `Authorization` (credentials such as a Bearer token), `Location` (URL of a newly created resource), `Cache-Control` and `ETag` (caching), `Origin` and the `Access-Control-*` family (CORS).

22.1.2 REST principles

REST (Representational State Transfer) is an architectural style for web APIs. Its core ideas: everything is a **resource** identified by a URL (`/api/students/101`); clients manipulate resources through **representations** (usually JSON) using the standard HTTP methods with their standard meanings; interactions are **stateless**; responses declare whether they are **cacheable**; and the interface is **uniform**. Good REST APIs use plural nouns for collections, nesting for relationships, query strings for filtering and paging, and proper status codes.

Operation	Method + URL	Success response
list students (filter, page)	`GET /api/students?dept=CSE&page=2&size=20`	200 + array
get one	`GET /api/students/101`	200 + object, or 404
create	`POST /api/students` with JSON body	201 + Location: /api/students/105
replace	`PUT /api/students/101`	204
delete	`DELETE /api/students/101`	204
related resources	`GET /api/students/101/courses`	200 + array
action that is not CRUD	`POST /api/students/101/promote`	200 / 202

22.1.3 JSON APIs

Web APIs exchange data as **JSON**. ASP.NET Core uses System.Text.Json (Module 17) with the **web defaults**: property names in camelCase, case-insensitive reading, and numbers accepted in quotes. Returning a C# object from an endpoint serializes it automatically; a parameter of a complex type is read from the JSON request body. Errors are returned in the standard **ProblemDetails** format (RFC 9457), `application/problem+json`, so clients can handle them uniformly (22.4.8).

22.2 ASP.NET Core basics

22.2.1 Creating a project with dotnet new webapi

Creating and running a web API project

```
$ dotnet new webapi -n StudentApi           # minimal API template with OpenAPI
$ cd StudentApi
$ dotnet run
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5080
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development

$ dotnet new webapi -n StudentApi --use-controllers     # controller-based variant
$ dotnet watch                                          # restart / hot reload on file changes
```

File	Purpose
`Program.cs`	builds and configures the app: services, middleware, endpoints
`appsettings.json`, `appsettings.Development.json`	configuration per environment
`Properties/launchSettings.json`	local URLs and environment variables for dotnet run
`StudentApi.http`	sample requests to send from Visual Studio / VS Code / Rider
`StudentApi.csproj`	Sdk="Microsoft.NET.Sdk.Web", target framework, packages

22.2.2 Program.cs, the host and the builder

`WebApplication.CreateBuilder(args)` creates a **builder** preconfigured with the generic host: configuration sources, logging providers, the dependency-injection container and the **Kestrel** web server. You register **services** on `builder.Services`, call `Build()` to get the `WebApplication`, configure the **middleware pipeline** and map **endpoints** on it, and finally `Run()`, which starts listening for requests until the process is stopped.

THE STRUCTURE OF PROGRAM.CS

Program.cs

```
var builder = WebApplication.CreateBuilder(args);       // 1. host: config, logging, DI, Kestrel

builder.Services.AddOpenApi();                          // 2. register services
builder.Services.AddSingleton<IClock, SystemClock>();

var app = builder.Build();                              // 3. build the application

if (app.Environment.IsDevelopment()) app.MapOpenApi();  // 4. middleware and endpoints
app.UseHttpsRedirection();

app.MapGet("/", () => "Student API is running");
app.MapGet("/time", (IClock clock) => new { utc = clock.UtcNow.ToString("yyyy-MM-dd") });   // service
injected

app.Run();                                              // 5. start listening

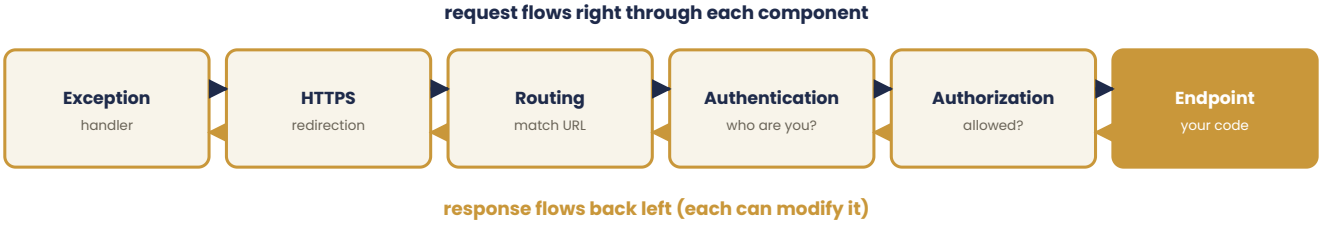
public interface IClock { DateTime UtcNow { get; } }
public class SystemClock : IClock { public DateTime UtcNow => new(2026, 3, 9, 0, 0, 0, DateTimeKind.Utc);
}   // fixed for the demo
```

OUTPUT

```
$ curl http://localhost:5080/
Student API is running
$ curl http://localhost:5080/time
{"utc":"2026-03-09"}
```

22.2.3 The middleware pipeline

Every request passes through a **pipeline** of **middleware** components, each of which can inspect or change the request, call the next component (or short-circuit and respond itself), and then inspect or change the response on the way back. The **order** in which middleware is added is the order in which it runs, and it matters: exception handling first, then HTTPS redirection, static files, routing, CORS, authentication, authorization, and finally the endpoints.



THE ASP.NET CORE MIDDLEWARE PIPELINE

WRITING MIDDLEWARE

Program.cs

```
var app = WebApplication.CreateBuilder(args).Build();

app.Use(async (context, next) =>                          // custom inline middleware
{
    var start = DateTime.UtcNow;
    Console.WriteLine($"--> {context.Request.Method} {context.Request.Path}");
    await next(context);                                   // call the rest of the pipeline
    Console.WriteLine($"<-- {context.Response.StatusCode}");
});

app.Use(async (context, next) =>
{
    if (context.Request.Headers.ContainsKey("X-Block"))
    {
        context.Response.StatusCode = 403;                 // short-circuit: next is never called
        await context.Response.WriteAsync("blocked");
        return;
    }
    await next(context);
});

app.UseMiddleware<RequestIdMiddleware>();                  // class-based middleware
app.MapGet("/hello", (HttpContext ctx) => $"hello, request {ctx.Items["RequestId"]}");
app.Run();

public class RequestIdMiddleware(RequestDelegate next)
{
    private static int counter;
    public async Task InvokeAsync(HttpContext context)
    {
        string id = $"R{Interlocked.Increment(ref counter):D3}";
        context.Items["RequestId"] = id;
        context.Response.Headers["X-Request-Id"] = id;
        await next(context);
    }
}
```

OUTPUT

```
$ curl http://localhost:5080/hello
hello, request R001
$ curl -H "X-Block: 1" http://localhost:5080/hello
blocked

server console:
--> GET /hello
<-- 200
--> GET /hello
<-- 403
```

22.2.4 Configuration (appsettings.json, environments, user secrets)

Configuration comes from several **providers**, read in order, with later ones overriding earlier ones: `appsettings.json`, then `appsettings.{Environment}.json`, then **user secrets** (Development only), then **environment variables**, then command-line arguments. The environment is chosen by the `ASPNETCORE_ENVIRONMENT` variable (Development, Staging, Production). Values are read through `IConfiguration` with ":"-separated keys, or bound to strongly typed classes with the **Options pattern**. Secrets such as connection-string passwords and JWT keys must never be committed to source control: use user secrets locally and environment variables or a vault (Azure Key Vault) in production.

APPSETTINGS.JSON json

```
{
  "ConnectionStrings": { "School": "Data Source=school.db" },
  "Jwt": { "Issuer": "lbf-api", "Audience": "lbf-clients", "ExpiryMinutes": 60 },
  "Features": { "EnableRegistration": true },
  "Logging": { "LogLevel": { "Default": "Information", "Microsoft.AspNetCore": "Warning" } }
}
```

READING CONFIGURATION Program.cs

```
var builder = WebApplication.CreateBuilder(args);

string? conn = builder.Configuration.GetConnectionString("School");          // ConnectionStrings:School
int expiry = builder.Configuration.GetValue<int>("Jwt:ExpiryMinutes");
builder.Services.Configure<JwtOptions>(builder.Configuration.GetSection("Jwt"));   // Options pattern

var app = builder.Build();
app.MapGet("/config", (Microsoft.Extensions.Options.IOptions<JwtOptions> jwt, IWebHostEnvironment env) =>
    new { env = env.EnvironmentName, conn, expiry, issuer = jwt.Value.Issuer });
app.Run();

public class JwtOptions
{
    public string Issuer { get; set; } = "";
    public string Audience { get; set; } = "";
    public int ExpiryMinutes { get; set; }
}
```

OUTPUT

```
$ curl http://localhost:5080/config
{"env":"Development","conn":"Data Source=school.db","expiry":60,"issuer":"lbf-api"}
```

USER SECRETS AND ENVIRONMENT VARIABLES C#

```
dotnet user-secrets init
dotnet user-secrets set "Jwt:Key" "a-long-random-development-signing-key-123456"
# production (Linux): environment variables use __ instead of :
export Jwt__Key="..."  ConnectionStrings__School="Server=...;Password=..."
```

22.2.5 Logging

ASP.NET Core has logging built in through `ILogger<T>`, injected anywhere. Messages have a **level** (Trace, Debug, Information, Warning, Error, Critical), a **category** (usually the class name) and a **message template** with named placeholders, which structured logging stores as separate fields (searchable in tools like Seq, Elasticsearch or Application Insights). Minimum levels per category are set in the Logging section of configuration. Providers include the console, debug output, EventSource and OpenTelemetry; Serilog and NLog are popular third-party options.

STRUCTURED LOGGING WITH ILOGGER Program.cs

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapPost("/orders/{id:int}", (int id, ILogger<Program> logger) =>
{
    logger.LogInformation("Placing order {OrderId} at {Time:HH:mm}", id, new DateTime(2026, 3, 9, 10, 30,
0));   // structured: OrderId is a field
    if (id <= 0)
    {
        logger.LogWarning("Rejected invalid order id {OrderId}", id);
        return Results.BadRequest();
    }
    return Results.Ok(new { id, status = "placed" });
});
app.Run();
```

OUTPUT

```
$ curl -X POST http://localhost:5080/orders/7
{"id":7,"status":"placed"}
$ curl -X POST http://localhost:5080/orders/0 -w "%{http_code}"
400

server console:
info: Program[0]
      Placing order 7 at 10:30
info: Program[0]
      Placing order 0 at 10:30
warn: Program[0]
      Rejected invalid order id 0
```

22.3 Dependency injection

22.3.1 What DI is and why

Dependency injection (DI) means a class receives the objects it depends on (its dependencies) from outside, usually through its constructor, instead of creating them itself with `new`. The class depends on an **abstraction** (an interface), and a **container** decides which concrete implementation to supply. Benefits: classes are loosely coupled, implementations can be swapped through configuration (a fake email sender in tests, SMTP in production), lifetimes and disposal are managed centrally, and unit testing becomes easy. ASP.NET Core is built around a DI container (`IServiceCollection` / `IServiceProvider`); controllers, minimal API handlers, middleware and hosted services all receive their dependencies from it.

CONSTRUCTOR INJECTION VERSUS NEW C#

```
// WITHOUT DI: tightly coupled, hard to test
public class OrderService
{
    private readonly SmtpEmailSender email = new SmtpEmailSender("smtp.example.com");   // hard-wired
    public void Place(Order o) { /* ... */ email.Send(o.CustomerEmail, "Order placed"); }
}

// WITH DI: depends on an abstraction, receives it from the container
public class OrderService(IEmailSender email)
{
    public void Place(Order o) { /* ... */ email.Send(o.CustomerEmail, "Order placed"); }
}

builder.Services.AddScoped<IEmailSender, SmtpEmailSender>();   // production wiring
// in a unit test: new OrderService(new FakeEmailSender())
```

22.3.2 Service lifetimes: transient, scoped, singleton

Lifetime	Registration	One instance per	Use for
Transient	`AddTransient<I, T>()`	every injection (new each time)	lightweight, stateless services
Scoped	`AddScoped<I, T>()`	HTTP request (scope)	DbContext, unit of work, per-request state
Singleton	`AddSingleton<I, T>()`	application lifetime	caches, configuration, HttpClient factories; must be thread-safe

COMPARING LIFETIMES Program.cs

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddTransient<TransientId>();
builder.Services.AddScoped<ScopedId>();
builder.Services.AddSingleton<SingletonId>();
var app = builder.Build();

app.MapGet("/ids", (TransientId t1, TransientId t2, ScopedId s1, ScopedId s2, SingletonId g) =>
    $"transient {t1.Id}/{t2.Id}  scoped {s1.Id}/{s2.Id}  singleton {g.Id}");
app.Run();

public abstract class Numbered { private static int next; public int Id { get; } =
Interlocked.Increment(ref next); }
public class TransientId : Numbered;
public class ScopedId : Numbered;
public class SingletonId : Numbered;
```

OUTPUT

```
$ curl http://localhost:5080/ids
transient 1/2  scoped 3/3  singleton 4
$ curl http://localhost:5080/ids
transient 5/6  scoped 7/7  singleton 4
```

NOTE

The numbers come from one shared counter in creation order. Within a request the two transient parameters get different instances, the two scoped parameters share one, and the singleton is the same across requests.

WATCH OUT

A **captive dependency** is a longer-lived service holding a shorter-lived one, e.g. a singleton that receives a scoped DbContext: the DbContext would then be shared by all requests and threads. ASP.NET Core validates scopes in Development and throws at start-up for this mistake. A singleton that needs scoped services should create a scope with `IServiceScopeFactory`.

22.3.3 Registering and injecting services

Register services in Program.cs before `Build()`, mapping an interface to an implementation with a lifetime. Many features come with extension methods that register everything they need (`AddDbContext`, `AddControllers`, `AddAuthentication`, `AddHttpClient`). Services are injected through constructors (controllers, services, middleware), handler parameters (minimal APIs) or, rarely, resolved from `app.Services`. .NET 8 added **keyed services** for several implementations of one interface.

REGISTERING SERVICES AND INJECTING THEM — Program.cs

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddSingleton<IGradePolicy, StandardGradePolicy>();
builder.Services.AddScoped<IStudentService, StudentService>();
builder.Services.AddKeyedSingleton<INotifier, EmailNotifier>("email");     // keyed services (.NET 8)
builder.Services.AddKeyedSingleton<INotifier, SmsNotifier>("sms");
var app = builder.Build();

app.MapGet("/grade/{marks:int}", (int marks, IStudentService svc) => svc.Describe(marks));
app.MapPost("/notify/{channel}", (string channel, IServiceProvider sp) =>
    sp.GetRequiredKeyedService<INotifier>(channel).Send("Results published"));
app.Run();

public interface IGradePolicy { char Grade(int marks); }
public class StandardGradePolicy : IGradePolicy { public char Grade(int m) => m >= 85 ? 'A' : m >= 60 ?
'B' : 'C'; }

public interface IStudentService { string Describe(int marks); }
public class StudentService(IGradePolicy policy, ILogger<StudentService> log) : IStudentService   //
constructor injection
{
    public string Describe(int marks)
    {
        log.LogDebug("grading {Marks}", marks);
        return $"marks {marks} -> grade {policy.Grade(marks)}";
    }
}

public interface INotifier { string Send(string msg); }
public class EmailNotifier : INotifier { public string Send(string m) => $"email sent: {m}"; }
public class SmsNotifier : INotifier { public string Send(string m) => $"sms sent: {m}"; }
```

OUTPUT

```
$ curl http://localhost:5080/grade/91
marks 91 -> grade A
$ curl -X POST http://localhost:5080/notify/sms
sms sent: Results published
```

22.4 Building APIs

ASP.NET Core offers two programming models for APIs. **Minimal APIs** map routes directly to lambdas or methods in Program.cs or in grouped extension methods: little ceremony, fast, and the default in the templates since .NET 8. **Controllers** group related actions in classes deriving from `ControllerBase`, with

attribute routing and filters; they suit large APIs and teams used to MVC. Both share routing, model binding, DI, validation, OpenAPI and the middleware pipeline, and can be mixed in one application.

22.4.1 Minimal APIs

A COMPLETE CRUD MINIMAL API

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddSingleton<StudentStore>();
var app = builder.Build();

var students = app.MapGroup("/api/students");                    // a route group with a common prefix

students.MapGet("/", (StudentStore store) => store.All());
students.MapGet("/{id:int}", (int id, StudentStore store) =>
    store.Find(id) is { } s ? Results.Ok(s) : Results.NotFound());
students.MapPost("/", (Student input, StudentStore store) =>
{
    var created = store.Add(input with { Id = 0 });
    return Results.Created($"/api/students/{created.Id}", created);
});
students.MapPut("/{id:int}", (int id, Student input, StudentStore store) =>
    store.Update(id, input) ? Results.NoContent() : Results.NotFound());
students.MapDelete("/{id:int}", (int id, StudentStore store) =>
    store.Delete(id) ? Results.NoContent() : Results.NotFound());

app.Run();

public record Student(int Id, string Name, int Marks);

public class StudentStore                                        // in-memory store; thread-safe with a
lock
{
    private readonly List<Student> items = [new(1, "Asha", 91), new(2, "Ravi", 78)];
    private readonly Lock gate = new();
    public List<Student> All() { lock (gate) return items.ToList(); }
    public Student? Find(int id) { lock (gate) return items.FirstOrDefault(s => s.Id == id); }
    public Student Add(Student s) { lock (gate) { var n = s with { Id = items.Max(x => x.Id) + 1 };
items.Add(n); return n; } }
    public bool Update(int id, Student s) { lock (gate) { int i = items.FindIndex(x => x.Id == id); if (i
< 0) return false; items[i] = s with { Id = id }; return true; } }
    public bool Delete(int id) { lock (gate) return items.RemoveAll(x => x.Id == id) > 0; }
}
```

OUTPUT

```
$ curl http://localhost:5080/api/students
[{"id":1,"name":"Asha","marks":91},{"id":2,"name":"Ravi","marks":78}]
$ curl -i -X POST http://localhost:5080/api/students -H "Content-Type: application/json" -d
'{"name":"Meera","marks":85}'
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Location: /api/students/3

{"id":3,"name":"Meera","marks":85}
$ curl -X PUT http://localhost:5080/api/students/2 -H "Content-Type: application/json" -d
'{"name":"Ravi","marks":82}' -w "%{http_code}"
204
$ curl http://localhost:5080/api/students/9 -w "%{http_code}"
404
$ curl -X DELETE http://localhost:5080/api/students/1 -w "%{http_code}"
204
```

22.4.2 Controller-based APIs

A controller is a class (conventionally named `XxxController`) deriving from `ControllerBase` and marked `[ApiController]`, which enables API conventions: attribute routing is required, invalid models automatically return 400 with ProblemDetails, and complex parameters bind from the body. Each public method is an **action**; HTTP method attributes (`[HttpGet]`, `[HttpPost("{id}")]`) map routes. Register with `AddControllers()` and `MapControllers()`.

A CONTROLLER WITH ATTRIBUTE ROUTING C#

```
builder.Services.AddControllers();
builder.Services.AddScoped<IStudentService, StudentService>();
// ...
app.MapControllers();

[ApiController]
[Route("api/[controller]")]                          // -> api/students
public class StudentsController(IStudentService service) : ControllerBase
{
    [HttpGet]
    public async Task<ActionResult<List<StudentDto>>> GetAll([FromQuery] string? dept, CancellationToken
ct) =>
        await service.ListAsync(dept, ct);

    [HttpGet("{id:int}")]
    public async Task<ActionResult<StudentDto>> Get(int id, CancellationToken ct) =>
        await service.GetAsync(id, ct) is { } dto ? dto : NotFound();

    [HttpPost]
    public async Task<ActionResult<StudentDto>> Create(CreateStudentRequest request, CancellationToken
ct)
    {
        StudentDto created = await service.CreateAsync(request, ct);
        return CreatedAtAction(nameof(Get), new { id = created.Id }, created);   // 201 + Location
    }

    [HttpDelete("{id:int}")]
    public async Task<IActionResult> Delete(int id, CancellationToken ct) =>
        await service.DeleteAsync(id, ct) ? NoContent() : NotFound();
}
```

	Minimal APIs	Controllers
Where endpoints live	lambdas/methods mapped in code (often grouped per feature)	action methods in controller classes
Ceremony	very little	more structure, conventions
Filters	endpoint filters	action, result, exception filters
Validation	built-in (.NET 10 AddValidation) or filters	automatic with [ApiController]
Performance / AOT	slightly faster; Native AOT supported	not AOT-compatible
Good for	microservices, new projects	large APIs, MVC experience

22.4.3 Routing and route parameters

Routing matches the request path to an endpoint. Route templates contain literal segments and **parameters** in braces, optionally with **constraints** (`{id:int}`, `{slug:alpha}`, `{date:datetime}`, `{code:length(6)}`, `{n:min(1)}`), optional parameters (`{page?}`), defaults (`{lang=en}`) and catch-alls (`{*path}`). If a constraint fails, the route does not match and another endpoint (or 404) handles the request.

ROUTE TEMPLATES AND CONSTRAINTS

Program.cs

```
var app = WebApplication.CreateBuilder(args).Build();

app.MapGet("/products/{id:int}", (int id) => $"product by id {id}");
app.MapGet("/products/{slug:alpha}", (string slug) => $"product by slug '{slug}'");
app.MapGet("/reports/{year:int:min(2000)}/{month:range(1,12)?}", (int year, int? month) =>
    month is null ? $"annual report {year}" : $"report {year}-{month:D2}");
app.MapGet("/files/{*path}", (string path) => $"file path: {path}");
app.MapGet("/search", (string q, int page = 1) => $"search '{q}' page {page}");    // query string
parameters

app.Run();
```

OUTPUT

```
$ curl http://localhost:5080/products/42
product by id 42
$ curl http://localhost:5080/products/notebook
product by slug 'notebook'
$ curl http://localhost:5080/reports/2026/3
report 2026-03
$ curl http://localhost:5080/reports/2026
annual report 2026
$ curl http://localhost:5080/reports/1999 -w "%{http_code}"
404
$ curl http://localhost:5080/files/docs/2026/notes.pdf
file path: docs/2026/notes.pdf
$ curl "http://localhost:5080/search?q=csharp&page=2"
search 'csharp' page 2
```

22.4.4 Model binding and validation

Model binding creates handler parameters from the request: simple types from the route, then the query string; complex types from the JSON body; special types (`HttpContext`, `CancellationToken`, `ClaimsPrincipal`, `IFormFile`) and registered services from DI. Attributes override the source: `[FromRoute]`, `[FromQuery]`, `[FromBody]`, `[FromHeader]`, `[FromForm]`, `[FromServices]`, and `[AsParameters]` groups several into one object. **Validation** checks the bound values: data annotations (`[Required]`, `[Range]`, `[EmailAddress]`, `[StringLength]`) are validated automatically for controllers with [ApiController] and, since .NET 10, for minimal APIs after `AddValidation()`; FluentValidation is a popular library for complex rules.

BINDING SOURCES AND VALIDATION

Program.cs

```
using System.ComponentModel.DataAnnotations;
using Microsoft.AspNetCore.Mvc;                                  // [FromHeader]

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddValidation();                                // .NET 10: validate minimal API
parameters
var app = builder.Build();

app.MapPost("/api/students", (CreateStudentRequest req) => Results.Ok($"valid: {req.Name},
{req.Marks}"));
app.MapGet("/api/students", ([AsParameters] StudentQuery q) => $"dept={q.Dept ?? "all"} page={q.Page}
size={q.Size}");
app.MapGet("/whoami", ([FromHeader(Name = "X-Client")] string? client) => $"client: {client ??
"unknown"}");
app.Run();

public record CreateStudentRequest(
    [property: Required, StringLength(50, MinimumLength = 2)] string Name,
    [property: Range(0, 100)] int Marks,
    [property: EmailAddress] string? Email);

public record StudentQuery(string? Dept, int Page = 1, int Size = 20);
```

OUTPUT

```
$ curl -X POST http://localhost:5080/api/students -H "Content-Type: application/json" -d
'{"name":"Asha","marks":91}'
"valid: Asha, 91"
$ curl -X POST http://localhost:5080/api/students -H "Content-Type: application/json" -d
'{"name":"A","marks":140,"email":"x"}'
{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.1","title":"One or more validation errors
occurred.","status":400,
 "errors":{"Name":["The field Name must be a string with a minimum length of 2 and a maximum length of
50."],
           "Marks":["The field Marks must be between 0 and 100."],"Email":["The Email field is not a
valid e-mail address."]}}
$ curl "http://localhost:5080/api/students?dept=CSE&page=2"
dept=CSE page=2 size=20
$ curl -H "X-Client: mobile" http://localhost:5080/whoami
client: mobile
```

NOTE

The validation response is shown wrapped over several lines for readability (the server sends it on one line), and the exact key casing in "errors" can differ between versions and configurations. A string returned through `Results.Ok` is serialized as JSON, hence the quotes.

22.4.5 DTOs and mapping

A **DTO** (Data Transfer Object) is a simple type that defines exactly what an API sends or receives, separate from the database entity. Exposing EF Core entities directly is risky: it leaks internal fields (password hashes, foreign keys), allows **over-posting** (a client setting IsAdmin or Id), couples the API contract to the database schema, and can create serialization cycles through navigation properties. Use request DTOs for input (`CreateStudentRequest`), response DTOs for output (`StudentDto`), and map between them, by hand (simple and explicit), with LINQ projections, or with a library such as Mapster or AutoMapper.

MAPPING BETWEEN ENTITIES AND DTOS

Program.cs

```
var entity = new StudentEntity { Id = 7, Name = "Asha Rao", Marks = 91, PasswordHash = "x9f$...", DeptId
= 2, CreatedAt = new DateTime(2026, 3, 9) };

StudentDto dto = entity.ToDto();                              // never expose PasswordHash or DeptId
Console.WriteLine(System.Text.Json.JsonSerializer.Serialize(dto, new
System.Text.Json.JsonSerializerOptions(System.Text.Json.JsonSerializerDefaults.Web)));

var request = new CreateStudentRequest("Ravi Kumar", 78);
StudentEntity newEntity = request.ToEntity();                  // the client cannot set Id or
PasswordHash
Console.WriteLine($"{newEntity.Name} {newEntity.Marks} id={newEntity.Id}");

public class StudentEntity
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public int Marks { get; set; }
    public string PasswordHash { get; set; } = "";
    public int DeptId { get; set; }
    public DateTime CreatedAt { get; set; }
}

public record StudentDto(int Id, string Name, int Marks, string Grade);       // response DTO
public record CreateStudentRequest(string Name, int Marks);                   // request DTO

public static class StudentMapping
{
    public static StudentDto ToDto(this StudentEntity e) => new(e.Id, e.Name, e.Marks, e.Marks >= 85 ?
"A" : "B");
    public static StudentEntity ToEntity(this CreateStudentRequest r) => new() { Name = r.Name.Trim(),
Marks = r.Marks };
}
```

OUTPUT

```
{"id":7,"name":"Asha Rao","marks":91,"grade":"A"}
Ravi Kumar 78 id=0
```

22.4.6 Returning results and status codes

Handlers can return plain objects (serialized with 200 OK), strings (text/plain), or **IResult** values that control the status code, headers and body: `Results.Ok`, `Created`, `NoContent`, `BadRequest`, `NotFound`, `Conflict`, `Unauthorized`, `Forbid`, `Problem`, `ValidationProblem`, `File`, `Redirect`, `Stream`. **TypedResults** returns the same with concrete types, and declaring the union return type `Results<Ok<T>, NotFound>` lets OpenAPI describe every possible response automatically. Controllers use the equivalent helper methods and `ActionResult<T>`.

TYPEDRESULTS AND STATUS CODES Program.cs

```
using Microsoft.AspNetCore.Http.HttpResults;

var app = WebApplication.CreateBuilder(args).Build();
var books = new Dictionary<int, string> { [1] = "Malgudi Days" };

app.MapGet("/books/{id:int}", Results<Ok<BookDto>, NotFound> (int id) =>
    books.TryGetValue(id, out var title) ? TypedResults.Ok(new BookDto(id, title)) :
TypedResults.NotFound());

app.MapPost("/books", Results<Created<BookDto>, Conflict<string>, ValidationProblem> (BookDto b) =>
{
    if (string.IsNullOrWhiteSpace(b.Title))
        return TypedResults.ValidationProblem(new Dictionary<string, string[]> { ["title"] = ["Title is
required"] });
    if (!books.TryAdd(b.Id, b.Title)) return TypedResults.Conflict($"book {b.Id} already exists");
    return TypedResults.Created($"/books/{b.Id}", b);
});

app.MapGet("/books/{id:int}/cover", (int id) => Results.File(new byte[] { 0x89, 0x50, 0x4E, 0x47 },
"image/png"));
app.Run();

public record BookDto(int Id, string Title);
```

OUTPUT

```
$ curl http://localhost:5080/books/1
{"id":1,"title":"Malgudi Days"}
$ curl -X POST http://localhost:5080/books -H "Content-Type: application/json" -d
'{"id":1,"title":"Again"}' -w " %{http_code}"
"book 1 already exists" 409
$ curl -X POST http://localhost:5080/books -H "Content-Type: application/json" -d '{"id":2,"title":""}' -
w " %{http_code}"
{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.1","title":"One or more validation errors
occurred.","status":400,"errors":{"title":["Title is required"]}} 400
```

22.4.7 API documentation with OpenAPI

OpenAPI (formerly Swagger) is the standard machine-readable description of an HTTP API: endpoints, parameters, request and response schemas, status codes and security. Since .NET 9, ASP.NET Core generates it natively with `Microsoft.AspNetCore.OpenApi`: `AddOpenApi()` and `MapOpenApi()` serve the document at `/openapi/v1.json` (OpenAPI 3.1 in .NET 10). Interactive UIs such as **Scalar** or **Swagger UI** (packages Scalar.AspNetCore, Swashbuckle) display it so developers can try requests, and tools like NSwag or Kiota generate typed client code from it. Endpoint metadata (`WithName`, `WithSummary`, `Produces`, XML comments) enriches the document.

GENERATING AND EXPOSING OPENAPI DOCUMENTATION C#

```
builder.Services.AddOpenApi();
// ...
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();                          // GET /openapi/v1.json
    app.MapScalarApiReference();               // interactive UI at /scalar (package Scalar.AspNetCore)
}

app.MapGet("/api/students/{id:int}", GetStudent)
   .WithName("GetStudent")
   .WithSummary("Get one student by id")
   .WithTags("Students")
   .Produces<StudentDto>(StatusCodes.Status200OK)
   .ProducesProblem(StatusCodes.Status404NotFound);
```

EXCERPT OF A GENERATED /OPENAPI/V1.JSON json

```
{
  "openapi": "3.1.1",
  "info": { "title": "StudentApi | v1", "version": "1.0.0" },
  "paths": {
    "/api/students/{id}": {
      "get": {
        "tags": [ "Students" ],
        "summary": "Get one student by id",
        "operationId": "GetStudent",
        "parameters": [ { "name": "id", "in": "path", "required": true, "schema": { "type": "integer",
"format": "int32" } } ],
        "responses": { "200": { "description": "OK", "content": { "application/json": { "schema": {
"$ref": "#/components/schemas/StudentDto" } } } } }
      }
    }
  }
}
```

22.4.8 Global error handling and ProblemDetails

Unhandled exceptions must never leak stack traces to clients. In production, the **exception handler middleware** catches them, logs them and returns a safe 500 response. `AddProblemDetails()` makes all error responses (exceptions, 404s from `UseStatusCodePages`, validation failures) use the standard ProblemDetails JSON. Since .NET 8, classes implementing `IExceptionHandler` can map specific exception types to specific status codes, for example a domain `NotFoundException` to 404. In Development, the developer exception page shows full details instead.

MAPPING EXCEPTIONS TO PROBLEMDETAILS (PRODUCTION ENVIRONMENT) Program.cs

```
using Microsoft.AspNetCore.Diagnostics;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddProblemDetails();
builder.Services.AddExceptionHandler<DomainExceptionHandler>();
var app = builder.Build();

app.UseExceptionHandler();                                    // catches exceptions from everything
after it
app.UseStatusCodePages();                                     // ProblemDetails bodies for empty 4xx
responses

app.MapGet("/students/{id:int}", (int id) => id == 7 ? "Asha" : throw new NotFoundException($"Student
{id} not found"));
app.MapGet("/crash", () => { throw new InvalidOperationException("secret internal detail"); });
app.Run();

public class NotFoundException(string message) : Exception(message);

public class DomainExceptionHandler(IProblemDetailsService problems, ILogger<DomainExceptionHandler> log)
: IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext ctx, Exception ex, CancellationToken ct)
    {
        if (ex is not NotFoundException) return false;             // let the default handler produce a
500
        log.LogInformation("Not found: {Message}", ex.Message);
        ctx.Response.StatusCode = StatusCodes.Status404NotFound;
        return await problems.TryWriteAsync(new() { HttpContext = ctx, ProblemDetails = { Title = "Not
found", Detail = ex.Message } });
    }
}
```

OUTPUT

```
$ curl http://localhost:5080/students/7
Asha
$ curl http://localhost:5080/students/9
{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.5","title":"Not
found","status":404,"detail":"Student 9 not found","traceId":"00-...-00"}
$ curl http://localhost:5080/crash
{"type":"https://tools.ietf.org/html/rfc9110#section-15.6.1","title":"An error occurred while processing
your request.","status":500,"traceId":"00-...-00"}
```

NOTE

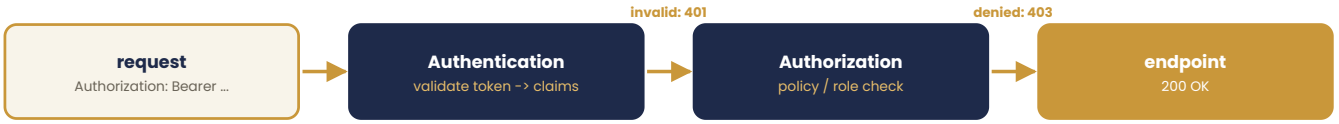
The traceId (shortened here) differs on every request and links the response to the server logs. Notice that the 500 response does not reveal "secret internal detail".

22.5 Security

22.5.1 Authentication vs authorization

Authentication answers "who are you?": it verifies credentials (a password, a token, a certificate) and produces a `ClaimsPrincipal`, the user with a set of **claims** (name, id, email, roles). **Authorization** answers "what are you allowed to do?": it checks the authenticated user's claims against requirements on an endpoint. A request without valid credentials to a protected endpoint gets **401 Unauthorized**; an authenticated user who lacks permission gets **403 Forbidden**. In the pipeline, `UseAuthentication()` must

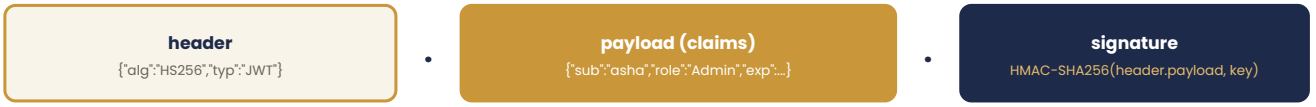
come before `UseAuthorization()` (both are added automatically when services are registered, but explicit calls control the order).



AUTHENTICATION THEN AUTHORIZATION

22.5.2 JWT bearer authentication

APIs used by SPAs, mobile apps and other services usually authenticate with **JSON Web Tokens** (JWT). After a successful login, the server (or an identity provider such as Microsoft Entra ID, Auth0 or Keycloak) issues a token: three Base64Url parts, **header.payload.signature**. The payload contains claims and an expiry; the signature, made with a secret key or a private key, lets the API verify that the token was issued by a trusted party and not modified, without a database lookup. The client sends it in every request as `Authorization: Bearer <token>`. Tokens should be short-lived and sent only over HTTPS; JWT payloads are encoded, not encrypted, so never put secrets in them.



THE THREE PARTS OF A JWT

ISSUING AND VALIDATING JWTS

Program.cs

```
using System.IdentityModel.Tokens.Jwt;          // package System.IdentityModel.Tokens.Jwt
using System.Security.Claims;
using System.Text;
using Microsoft.AspNetCore.Authentication.JwtBearer;   // package
Microsoft.AspNetCore.Authentication.JwtBearer
using Microsoft.IdentityModel.Tokens;

var builder = WebApplication.CreateBuilder(args);
var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!));   // from
user secrets

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o => o.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true, ValidIssuer = "lbf-api",
        ValidateAudience = true, ValidAudience = "lbf-clients",
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true, IssuerSigningKey = key
    });
builder.Services.AddAuthorization();
var app = builder.Build();
app.UseAuthentication();
app.UseAuthorization();

app.MapPost("/login", (LoginRequest req) =>
{
    if (req is not { User: "asha", Password: "P@ssw0rd" }) return Results.Unauthorized();   // demo check
only
    var token = new JwtSecurityToken(
        issuer: "lbf-api", audience: "lbf-clients",
        claims: [new Claim(ClaimTypes.Name, req.User), new Claim(ClaimTypes.Role, "Teacher")],
        expires: DateTime.UtcNow.AddMinutes(60),
        signingCredentials: new SigningCredentials(key, SecurityAlgorithms.HmacSha256));
    return Results.Ok(new { token = new JwtSecurityTokenHandler().WriteToken(token) });
});

app.MapGet("/api/profile", (ClaimsPrincipal user) => $"hello {user.Identity!.Name}, roles:
{string.Join(",", user.FindAll(ClaimTypes.Role).Select(c => c.Value))}")
   .RequireAuthorization();                                          // needs a valid token
app.MapGet("/api/public", () => "anyone can see this");
app.Run();

public record LoginRequest(string User, string Password);
```

OUTPUT

```
$ curl http://localhost:5080/api/profile -w "%{http_code}"
401
$ curl -X POST http://localhost:5080/login -H "Content-Type: application/json" -d
'{"user":"asha","password":"P@ssw0rd"}'
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bmlxdWVfbmFtZSI6ImFzaGEi...(shortened)"}
$ curl http://localhost:5080/api/profile -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIs...(the token)"
hello asha, roles: Teacher
```

WATCH OUT

The hard-coded password check is only for the demo. Real applications verify hashed passwords (ASP.NET Core Identity, 22.5.3) or, better, delegate login to an identity provider using OpenID Connect, and the API only validates tokens. The signing key must be long (at least 256 bits for HS256), secret and never committed.

22.5.3 ASP.NET Core Identity (overview)

ASP.NET Core Identity is a membership system: user and role storage (with EF Core), secure password hashing (PBKDF2), account confirmation, password reset, lockout after failed attempts, two-factor authentication and external logins (Google, Microsoft). It provides UI for Razor Pages apps and, since .NET 8, ready-made **API endpoints** (`MapIdentityApi`) for registration, login and token refresh in SPAs and mobile apps. For larger systems, a dedicated identity server (Microsoft Entra ID, Duende IdentityServer, Keycloak, Auth0) issues tokens for many applications.

IDENTITY WITH API ENDPOINTS C#

```
builder.Services.AddDbContext<AppDbContext>(o => o.UseSqlite("Data Source=app.db"));
builder.Services.AddIdentityApiEndpoints<IdentityUser>()        // users, password hashing, tokens
       .AddRoles<IdentityRole>()
       .AddEntityFrameworkStores<AppDbContext>();
builder.Services.AddAuthorization();
// ...
app.MapIdentityApi<IdentityUser>();     // POST /register, /login, /refresh, /manage/info, ...

public class AppDbContext(DbContextOptions<AppDbContext> o) : IdentityDbContext<IdentityUser>(o);
```

22.5.4 Role and policy-based authorization

Role-based authorization allows an endpoint only for users in given roles: `[Authorize(Roles = "Admin,Teacher")]` or `RequireAuthorization(p => p.RequireRole("Admin"))`. **Policy-based** authorization is more flexible: a named policy combines requirements on claims, roles or custom logic (minimum age, same department, resource ownership), defined once in `AddAuthorization` and applied by name. Custom requirements are implemented with `IAuthorizationHandler`.

ROLES AND POLICIES C#

```
builder.Services.AddAuthorizationBuilder()
    .AddPolicy("TeachersOnly", p => p.RequireRole("Teacher", "Admin"))
    .AddPolicy("CseStaff", p => p.RequireClaim("department", "CSE"))
    .AddPolicy("CanPublishResults", p => p.RequireAssertion(ctx =>
        ctx.User.IsInRole("Admin") || ctx.User.HasClaim("permission", "results:publish")));

app.MapPost("/api/results/publish", () => "published").RequireAuthorization("CanPublishResults");
app.MapDelete("/api/students/{id:int}", (int id) =>
Results.NoContent()).RequireAuthorization("TeachersOnly");

[Authorize(Roles = "Admin")]                // controller-level: all actions
public class AdminController : ControllerBase
{
    [AllowAnonymous] [HttpGet("health")] public string Health() => "ok";   // exception for one action
}
```

Authenticated but not authorized: 403

```
# token for asha (role Teacher, no publish permission)
$ curl -X DELETE http://localhost:5080/api/students/3 -H "Authorization: Bearer <asha-token>" -w "%
{http_code}"
204
$ curl -X POST http://localhost:5080/api/results/publish -H "Authorization: Bearer <asha-token>" -w "%
{http_code}"
403
```

22.5.5 CORS and HTTPS

Browsers enforce the **same-origin policy**: JavaScript on `https://app.example.com` may not read responses from `https://api.example.com` unless the API explicitly allows that origin with **CORS** (Cross-Origin Resource Sharing) headers. CORS does not protect the API from non-browser clients; it only relaxes browser restrictions, so allow specific origins, never `AllowAnyOrigin` together with credentials. **HTTPS** encrypts traffic and is mandatory for anything with tokens or personal data: `UseHttpsRedirection()` redirects HTTP to HTTPS and `UseHsts()` tells browsers to use HTTPS only; `dotnet dev-certs https --trust` sets up a local development certificate.

CORS POLICY AND HTTPS C#

```
builder.Services.AddCors(o => o.AddPolicy("Frontend", p => p
    .WithOrigins("https://app.example.com", "http://localhost:5173")    // the SPA origins
    .AllowAnyHeader()
    .WithMethods("GET", "POST", "PUT", "DELETE")
    .AllowCredentials()));
// ...
if (!app.Environment.IsDevelopment()) app.UseHsts();
app.UseHttpsRedirection();
app.UseCors("Frontend");                  // before authentication/authorization and endpoints
app.UseAuthentication();
app.UseAuthorization();
```

22.6 Web UI options (overview)

22.6.1 Razor Pages and MVC

For server-rendered websites, ASP.NET Core offers **Razor**, a syntax mixing HTML with C# (`@Model.Name`, `@foreach`). **MVC** (Model-View-Controller) separates controllers that handle requests, models with data, and views that render HTML; it suits large sites with many related actions. **Razor Pages** organises the UI by page: each page is a `.cshtml` file with a `PageModel` class holding its handlers (`OnGet`, `OnPost`), which is simpler for page-focused applications such as admin portals and forms.

A RAZOR PAGE AND ITS PAGEMODEL html

```
@page "/students"
@model StudentsModel
<h1>Students (@Model.Students.Count)</h1>
<table>
  @foreach (var s in Model.Students)
  {
    <tr><td>@s.Name</td><td>@s.Marks</td></tr>
  }
</table>

// Students.cshtml.cs
public class StudentsModel(SchoolDb db) : PageModel
{
    public List<Student> Students { get; private set; } = [];
    public async Task OnGetAsync() => Students = await db.Students.AsNoTracking().OrderBy(s =>
s.Name).ToListAsync();
}
```

22.6.2 Blazor (server and WebAssembly)

Blazor builds interactive web UIs with C# instead of JavaScript, using reusable **Razor components** (`.razor` files). Components can be rendered with different **render modes**: static server rendering (plain HTML, like Razor Pages), **Interactive Server** (UI events travel over a SignalR connection and the component runs on the

server), **Interactive WebAssembly** (the .NET runtime and your code run in the browser), or **Auto** (server first, then WebAssembly once downloaded). The Blazor Web App template (.NET 8+) mixes them per page or component. .NET MAUI Blazor Hybrid reuses the same components in desktop and mobile apps.

A BLAZOR COMPONENT html

```
@page "/counter"
@rendermode InteractiveServer

<h3>Marks counter</h3>
<p>Total: @total</p>
<input @bind="next" type="number" />
<button class="btn" @onclick="Add">Add</button>

@code {
    private int total;
    private int next = 10;
    private void Add() => total += next;     // runs in C#: on the server or in the browser (WebAssembly)
}
```

Option	Rendering	Best for
MVC	server, per request	large server-rendered sites
Razor Pages	server, per page	forms, admin portals, simpler sites
Blazor Server (interactive)	server; UI diff over SignalR	intranet apps, fast start, full .NET on the server
Blazor WebAssembly	in the browser	offline-capable SPAs, static hosting
ASP.NET Core API + Angular/React/Vue	JavaScript SPA	teams with front-end specialists

22.7 Going further

22.7.1 Caching

Caching stores results of expensive work (database queries, remote calls, rendered pages) to serve later requests faster. ASP.NET Core offers **in-memory caching** (`IMemoryCache`, per server), **distributed caching** (`IDistributedCache` backed by Redis or SQL Server, shared by all servers), **HybridCache** (.NET 9: a fast in-memory layer over a distributed cache, with protection against many requests recomputing the same entry at once), **output caching** of whole HTTP responses (`AddOutputCache`, `.CacheOutput()`) and HTTP **response caching** headers. Every cache needs an expiry or invalidation strategy.

IN-MEMORY AND OUTPUT CACHING

Program.cs

```
using Microsoft.Extensions.Caching.Memory;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddMemoryCache();
builder.Services.AddOutputCache();
var app = builder.Build();
app.UseOutputCache();

int dbCalls = 0;
app.MapGet("/api/toppers", async (IMemoryCache cache) =>
{
    var toppers = await cache.GetOrCreateAsync("toppers", async entry =>
    {
        entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(5);
        dbCalls++;
        await Task.Delay(100);                                  // the expensive query
        return new[] { "Asha", "Meera" };
    });
    return new { toppers, dbCalls };
});
app.MapGet("/api/stats", () => new { generated = DateTime.UtcNow.Ticks % 1000 }).CacheOutput(p =>
p.Expire(TimeSpan.FromSeconds(30)));
app.Run();
```

OUTPUT

```
$ curl http://localhost:5080/api/toppers
{"toppers":["Asha","Meera"],"dbCalls":1}
$ curl http://localhost:5080/api/toppers
{"toppers":["Asha","Meera"],"dbCalls":1}
```

NOTE

The second request is served from the cache, so the "database" was called only once. Calls to /api/stats within 30 seconds return the identical cached response body.

22.7.2 Background services

Long-running work that is not tied to a request (processing a queue, sending scheduled emails, cleaning up expired data) runs in a **hosted service**: a class deriving from `BackgroundService` and registered with `AddHostedService`. Its `ExecuteAsync` starts with the application and receives a CancellationToken that is triggered on shutdown. Hosted services are singletons, so to use scoped services such as a DbContext they create a scope per unit of work. The **Worker Service** template (`dotnet new worker`) builds applications consisting only of background services, for Windows services, Linux daemons and containers.

A PERIODIC BACKGROUND SERVICE

C#

```
builder.Services.AddHostedService<ExpiredSessionCleaner>();

public class ExpiredSessionCleaner(IServiceScopeFactory scopes, ILogger<ExpiredSessionCleaner> log) :
BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        using var timer = new PeriodicTimer(TimeSpan.FromMinutes(10));
        while (await timer.WaitForNextTickAsync(stoppingToken))
        {
            using var scope = scopes.CreateScope();                        // a fresh scope per run
            var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
            int removed = await db.Sessions.Where(s => s.ExpiresAt <
DateTime.UtcNow).ExecuteDeleteAsync(stoppingToken);
            log.LogInformation("Removed {Count} expired sessions", removed);
        }
    }
}
```

22.7.3 SignalR for real-time apps

SignalR adds real-time, two-way communication: the server can push messages to connected clients instantly (chat, live dashboards, notifications, multiplayer games, collaborative editing). It uses WebSockets when available and falls back to Server-Sent Events or long polling automatically. Server code lives in a **hub** class whose public methods clients call; the hub calls methods on clients through `Clients.All`, `Clients.Group(name)`, `Clients.User(id)` or `Clients.Caller`. Clients exist for JavaScript, .NET, Java and Swift.

A SIGNALR CHAT HUB AND CLIENT

C#

```
builder.Services.AddSignalR();
// ...
app.MapHub<ChatHub>("/hubs/chat");

public class ChatHub : Hub
{
    public async Task SendMessage(string room, string user, string text) =>
        await Clients.Group(room).SendAsync("ReceiveMessage", user, text);      // push to everyone in
the room

    public async Task JoinRoom(string room) => await Groups.AddToGroupAsync(Context.ConnectionId, room);
}

// JavaScript client (@microsoft/signalr)
const conn = new signalR.HubConnectionBuilder().withUrl("/hubs/chat").withAutomaticReconnect().build();
conn.on("ReceiveMessage", (user, text) => appendToChat(`${user}: ${text}`));
await conn.start();
await conn.invoke("JoinRoom", "csharp-batch");
await conn.invoke("SendMessage", "csharp-batch", "Asha", "Hello everyone!");
```

22.7.4 gRPC (overview)

gRPC is a high-performance RPC framework using HTTP/2 and **Protocol Buffers**, a compact binary serialization format. The service contract is written in a `.proto` file, from which tooling (package Grpc.AspNetCore / Grpc.Tools) generates strongly typed C# server base classes and clients. gRPC is efficient and supports streaming in both directions, making it ideal for internal service-to-service communication in microservices; for public browser-facing APIs, JSON over HTTP remains more common (gRPC-Web and JSON transcoding bridge the gap).

A GRPC SERVICE CONTRACT AND IMPLEMENTATION C#

```
// Protos/grades.proto
syntax = "proto3";
option csharp_namespace = "School.Grpc";

service Grades {
  rpc GetGrade (GradeRequest) returns (GradeReply);
}
message GradeRequest { string student_id = 1; }
message GradeReply   { string name = 1; int32 marks = 2; string grade = 3; }

// C# server implementation (base class generated from the .proto file)
public class GradesService : Grades.GradesBase
{
    public override Task<GradeReply> GetGrade(GradeRequest request, ServerCallContext context) =>
        Task.FromResult(new GradeReply { Name = "Asha", Marks = 91, Grade = "A" });
}
// Program.cs:  builder.Services.AddGrpc();  app.MapGrpcService<GradesService>();
```

22.7.5 Deploying to Azure, Docker and Linux servers

An ASP.NET Core application is published with `dotnet publish -c Release` and runs on Kestrel anywhere .NET runs. Common targets: **Azure App Service** (managed hosting: deploy from Visual Studio, the CLI or GitHub Actions), **containers** (Docker images run on Azure Container Apps, AKS, AWS ECS or any Kubernetes; the SDK can build images without a Dockerfile), and **Linux servers** (run as a systemd service behind Nginx or Apache as a reverse proxy that handles TLS). In every case: set `ASPNETCORE_ENVIRONMENT=Production`, provide secrets through environment variables or a vault, apply EF Core migrations as part of deployment, and add health checks (`MapHealthChecks("/health")`).

CONTAINERISING AN API C#

```
# Build a container image directly with the .NET SDK (no Dockerfile needed)
dotnet publish -c Release /t:PublishContainer -p ContainerRepository=student-api
docker run -d -p 8080:8080 -e ConnectionStrings__School="..." -e Jwt__Key="..." student-api

# Or a classic multi-stage Dockerfile
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish -c Release -o /app
FROM mcr.microsoft.com/dotnet/aspnet:10.0
WORKDIR /app
COPY --from=build /app .
EXPOSE 8080
ENTRYPOINT ["dotnet", "StudentApi.dll"]
```

SYSTEMD SERVICE AND AZURE APP SERVICE

C#

```
# /etc/systemd/system/student-api.service  (Linux server behind Nginx)
[Unit]
Description=Student API
[Service]
WorkingDirectory=/var/www/student-api
ExecStart=/usr/bin/dotnet /var/www/student-api/StudentApi.dll
Restart=always
Environment=ASPNETCORE_ENVIRONMENT=Production
Environment=ASPNETCORE_URLS=http://localhost:5000
User=www-data
[Install]
WantedBy=multi-user.target

# Azure App Service from the command line
az webapp up --name lbf-student-api --runtime "DOTNETCORE:10.0" --sku B1
```

Web APIs speak HTTP: methods, status codes and headers, organised by REST around resources and JSON. An ASP.NET Core app is built in Program.cs: the builder configures configuration, logging and DI, the middleware pipeline processes every request in order, and endpoints handle routes. DI supplies services with transient, scoped or singleton lifetimes. Build APIs with minimal APIs or controllers, using routing constraints, model binding, validation, DTOs, typed results, OpenAPI documentation and ProblemDetails error handling. Secure them with authentication (JWT, Identity) and role or policy authorization, CORS and HTTPS. Razor Pages, MVC and Blazor build UIs; caching, background services, SignalR, gRPC and deployment to Azure, containers or Linux complete the picture.

PRACTICE CHECKPOINT

Build a **Student Management REST API** with ASP.NET Core and EF Core, including **CRUD endpoints**, **validation**, **DI-based services**, **JWT authentication** and **OpenAPI** docs.

SAMPLE SOLUTION

Program.cs

```
// dotnet new webapi -n StudentApi
// dotnet add package Microsoft.EntityFrameworkCore.Sqlite
// dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
// dotnet user-secrets set "Jwt:Key" "<at least 32 random characters>"
using System.ComponentModel.DataAnnotations;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Http.HttpResults;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;

var builder = WebApplication.CreateBuilder(args);
var signingKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!));

builder.Services.AddDbContext<SchoolDb>(o => o.UseSqlite("Data Source=school.db"));
builder.Services.AddScoped<IStudentService, StudentService>();
builder.Services.AddSingleton(new TokenService(signingKey));
builder.Services.AddValidation();
builder.Services.AddOpenApi();
builder.Services.AddProblemDetails();
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o => o.TokenValidationParameters = new()
    {
        ValidIssuer = "lbf-api", ValidAudience = "lbf-clients", IssuerSigningKey = signingKey
```

```
    });
builder.Services.AddAuthorization();

var app = builder.Build();
using (var scope = app.Services.CreateScope()) scope.ServiceProvider.GetRequiredService<SchoolDb>
().Database.EnsureCreated();
if (app.Environment.IsDevelopment()) app.MapOpenApi();
app.UseExceptionHandler();
app.UseAuthentication();
app.UseAuthorization();

app.MapPost("/auth/login", Results<Ok<TokenResponse>, UnauthorizedHttpResult> (LoginRequest req,
TokenService tokens) =>
    req is { UserName: "admin", Password: "Lbf@2026" } ? TypedResults.Ok(tokens.Create(req.UserName,
"Admin")) : TypedResults.Unauthorized())
   .WithTags("Auth");

var api = app.MapGroup("/api/students").RequireAuthorization().WithTags("Students");

api.MapGet("", (IStudentService s, string? dept, CancellationToken ct) => s.ListAsync(dept, ct));
api.MapGet("/{id:int}", async Task<Results<Ok<StudentDto>, NotFound>> (int id, IStudentService s,
CancellationToken ct) =>
    await s.GetAsync(id, ct) is { } dto ? TypedResults.Ok(dto) :
TypedResults.NotFound()).WithName("GetStudent");
api.MapPost("", async Task<Created<StudentDto>> (CreateStudentRequest req, IStudentService s,
CancellationToken ct) =>
{
    var dto = await s.CreateAsync(req, ct);
    return TypedResults.Created($"/api/students/{dto.Id}", dto);
});
api.MapPut("/{id:int}", async Task<Results<NoContent, NotFound>> (int id, CreateStudentRequest req,
IStudentService s, CancellationToken ct) =>
    await s.UpdateAsync(id, req, ct) ? TypedResults.NoContent() : TypedResults.NotFound());
api.MapDelete("/{id:int}", async Task<Results<NoContent, NotFound>> (int id, IStudentService s,
CancellationToken ct) =>
    await s.DeleteAsync(id, ct) ? TypedResults.NoContent() : TypedResults.NotFound());

app.Run();

// ---------- data ----------
public class Student { public int Id { get; set; } public string Name { get; set; } = ""; public string
Dept { get; set; } = ""; public int Marks { get; set; } }
public class SchoolDb(DbContextOptions<SchoolDb> o) : DbContext(o) { public DbSet<Student> Students =>
Set<Student>(); }

// ---------- contracts ----------
public record StudentDto(int Id, string Name, string Dept, int Marks, string Grade);
public record CreateStudentRequest(
    [property: Required, StringLength(60, MinimumLength = 2)] string Name,
    [property: Required, RegularExpression("^(CSE|ECE|MECH)$")] string Dept,
    [property: Range(0, 100)] int Marks);
public record LoginRequest(string UserName, string Password);
public record TokenResponse(string Token, DateTime ExpiresUtc);

// ---------- services ----------
public interface IStudentService
{
    Task<List<StudentDto>> ListAsync(string? dept, CancellationToken ct);
    Task<StudentDto?> GetAsync(int id, CancellationToken ct);
    Task<StudentDto> CreateAsync(CreateStudentRequest r, CancellationToken ct);
    Task<bool> UpdateAsync(int id, CreateStudentRequest r, CancellationToken ct);
    Task<bool> DeleteAsync(int id, CancellationToken ct);
}

public class StudentService(SchoolDb db) : IStudentService
{
```

```
    static StudentDto ToDto(Student s) => new(s.Id, s.Name, s.Dept, s.Marks, s.Marks >= 85 ? "A" :
s.Marks >= 60 ? "B" : "C");

    public Task<List<StudentDto>> ListAsync(string? dept, CancellationToken ct) =>
        db.Students.AsNoTracking().Where(s => dept == null || s.Dept == dept).OrderBy(s => s.Id)
          .Select(s => ToDto(s)).ToListAsync(ct);

    public async Task<StudentDto?> GetAsync(int id, CancellationToken ct) =>
        await db.Students.FindAsync([id], ct) is { } s ? ToDto(s) : null;

    public async Task<StudentDto> CreateAsync(CreateStudentRequest r, CancellationToken ct)
    {
        var s = new Student { Name = r.Name.Trim(), Dept = r.Dept, Marks = r.Marks };
        db.Students.Add(s);
        await db.SaveChangesAsync(ct);
        return ToDto(s);
    }

    public async Task<bool> UpdateAsync(int id, CreateStudentRequest r, CancellationToken ct) =>
        await db.Students.Where(s => s.Id == id).ExecuteUpdateAsync(u => u
            .SetProperty(s => s.Name, r.Name.Trim()).SetProperty(s => s.Dept, r.Dept).SetProperty(s =>
s.Marks, r.Marks), ct) > 0;

    public async Task<bool> DeleteAsync(int id, CancellationToken ct) =>
        await db.Students.Where(s => s.Id == id).ExecuteDeleteAsync(ct) > 0;
}

public class TokenService(SecurityKey key)
{
    public TokenResponse Create(string user, string role)
    {
        var expires = DateTime.UtcNow.AddHours(1);
        var jwt = new JwtSecurityToken("lbf-api", "lbf-clients",
            [new Claim(ClaimTypes.Name, user), new Claim(ClaimTypes.Role, role)],
            expires: expires, signingCredentials: new SigningCredentials(key,
SecurityAlgorithms.HmacSha256));
        return new TokenResponse(new JwtSecurityTokenHandler().WriteToken(jwt), expires);
    }
}
```

OUTPUT

```
$ curl http://localhost:5080/api/students -w "%{http_code}"
401
$ TOKEN=$(curl -s -X POST http://localhost:5080/auth/login -H "Content-Type: application/json" \
        -d '{"userName":"admin","password":"Lbf@2026"}' | jq -r .token)
$ curl -X POST http://localhost:5080/api/students -H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" \
       -d '{"name":"Asha","dept":"CSE","marks":91}'
{"id":1,"name":"Asha","dept":"CSE","marks":91,"grade":"A"}
$ curl -X POST http://localhost:5080/api/students -H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" \
       -d '{"name":"Ravi","dept":"ECE","marks":72}'
{"id":2,"name":"Ravi","dept":"ECE","marks":72,"grade":"B"}
$ curl -X POST http://localhost:5080/api/students -H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" \
       -d '{"name":"X","dept":"CIVIL","marks":101}' -w " %{http_code}"
{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.1","title":"One or more validation errors
occurred.","status":400,"errors":{...}} 400
$ curl "http://localhost:5080/api/students?dept=CSE" -H "Authorization: Bearer $TOKEN"
[{"id":1,"name":"Asha","dept":"CSE","marks":91,"grade":"A"}]
$ curl -X PUT http://localhost:5080/api/students/2 -H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" \
       -d '{"name":"Ravi","dept":"ECE","marks":88}' -w "%{http_code}"
204
$ curl http://localhost:5080/api/students/2 -H "Authorization: Bearer $TOKEN"
{"id":2,"name":"Ravi","dept":"ECE","marks":88,"grade":"A"}
$ curl -X DELETE http://localhost:5080/api/students/1 -H "Authorization: Bearer $TOKEN" -w "%{http_code}"
204
$ curl http://localhost:5080/openapi/v1.json | jq '.paths | keys'
["/api/students", "/api/students/{id}", "/auth/login"]
```

MODULE 23 · ADVANCED

Testing, debugging, design patterns and best practices

Minimum duration **3 days** | Level **Advanced**

Professional C# code is not only working code: it is tested, readable, well structured and easy to change. This module covers debugging techniques in Visual Studio, VS Code and Rider; unit testing with xUnit (and NUnit and MSTest), data-driven tests, mocking, code coverage, integration testing of ASP.NET Core APIs and test-driven development; code quality with conventions, analyzers and reviews; the SOLID, DRY, KISS and YAGNI principles and Clean Architecture; the most useful design patterns in idiomatic C#; and the tooling of real projects: NuGet, solution structure, Git and continuous integration.

IN THIS MODULE

23.1 Debugging
23.2 Unit testing
23.3 Code quality
23.4 Design principles
23.5 Design patterns in C#
23.6 Tooling

23.1 Debugging

23.1.1 Breakpoints, conditional breakpoints, step into/over/out

A **debugger** runs your program under control: it pauses at chosen points and lets you inspect and change state. Start with F5 (Debug) rather than Ctrl+F5 (run without debugging), and use the Debug build configuration, where optimisations are off and variables are not optimised away. A **breakpoint** (F9, or click the margin) pauses execution when a line is reached. Advanced breakpoints: **conditional** (pause only when `i == 500` or `name == "Ravi"`), **hit count** (pause on the 10th hit), **tracepoints** (log a message without pausing), **function** breakpoints and **data/dependent** breakpoints. Once paused, the stepping commands move through the code.

Command	Visual Studio / VS Code key	Effect
Continue	F5	run until the next breakpoint
Step Over	F10	run the current line; do not enter called methods
Step Into	F11	enter the method called on the current line
Step Out	Shift+F11	run to the end of the current method and return to the caller
Run to Cursor	Ctrl+F10	continue up to the line with the cursor
Set Next Statement	Ctrl+Shift+F10	move the execution point (use with care)
Stop / Restart	Shift+F5 / Ctrl+Shift+F5	end or restart the debugging session

WHERE CONDITIONAL BREAKPOINTS AND TRACEPOINTS HELP

C#

```
public decimal CalculateTotal(List<OrderLine> lines)
{
    decimal total = 0;
    for (int i = 0; i < lines.Count; i++)
    {
        total += lines[i].Price * lines[i].Quantity;   // breakpoint here with condition:
lines[i].Quantity < 0
    }
    return total;                                      // tracepoint: "total = {total}" (logs, does not
pause)
}
```

23.1.2 Watch, Locals, Immediate and Call Stack windows

Window	Shows / does
Locals	all local variables and parameters of the current method, automatically
Autos	variables used on the current and previous lines
Watch	expressions you choose to monitor, such as `order.Lines.Sum(l => l.Price)` or `student?.Name`
Immediate (Debug Console in VS Code)	evaluate expressions and call methods while paused: `? total`, `lines.Count`
Call Stack	the chain of method calls that led to the current point; double-click a frame to see its locals
Threads / Parallel Stacks / Tasks	all threads, their stacks, and async tasks with their state
DataTips	hover over a variable in the editor to inspect and even edit it; pin it to keep it visible

Edit and Continue and **Hot Reload** let you change code while paused or running and continue without restarting. `System.Diagnostics.Debug.WriteLine` writes to the Output window only in Debug builds, and `Debugger.Break()` triggers a breakpoint from code. The `[DebuggerDisplay("{Name} ({Marks})")]` attribute controls how objects appear in these windows.

23.1.3 Debugging async code and exceptions

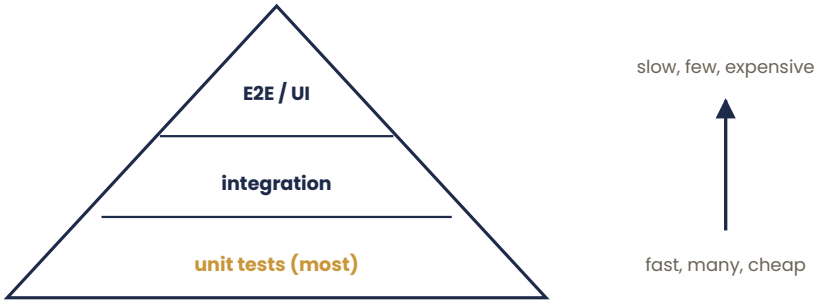
In async code, the Call Stack window shows the logical async call chain (the methods that awaited each other) rather than raw thread frames; the **Tasks** and **Parallel Stacks** windows show which tasks are waiting and on what, which helps find deadlocks. For exceptions, open **Exception Settings** (Ctrl+Alt+E) and tick "Break when thrown" for a type (for example System.NullReferenceException) so the debugger stops at the line that throws, even if the exception is later caught; by default it only stops for unhandled exceptions. The `$exception` pseudo-variable shows the current exception, including InnerException and the stack trace.

Problem	Debugging approach
Exception caught somewhere far away	enable first-chance break for that exception type
Value wrong in a loop at iteration 731	conditional or hit-count breakpoint
Async method never completes	Tasks / Parallel Stacks windows; look for .Result or .Wait()
Only fails in production	structured logs, dotnet-dump / dotnet-trace (Module 20), Application Insights

Problem	Debugging approach
Race condition	logging with thread ids; reproduce under load; tests with many iterations

23.2 Unit testing

A **unit test** is code that calls a small unit of production code (usually one method or class) with known inputs and checks the result automatically. A suite of fast, independent tests lets you change code with confidence: run them after every change and on every push, and a regression is caught in seconds. Tests also document the intended behaviour and push you toward loosely coupled, injectable designs.



THE TEST PYRAMID: MANY UNIT TESTS, FEWER INTEGRATION TESTS, FEW END-TO-END TESTS

23.2.1 Test frameworks: xUnit, NUnit, MSTest

Three mature test frameworks run on the same infrastructure (`dotnet test`, Test Explorer in IDEs, CI). **xUnit** is the most widely used in modern .NET and by the .NET team; a new instance of the test class is created for every test, so shared state is not leaked. **NUnit** has a long history and rich assertions. **MSTest** is Microsoft's framework, built into Visual Studio. Concepts are equivalent; only attribute names differ. A test project references the project under test plus the framework packages.

Concept	xUnit	NUnit	MSTest
Test method	`[Fact]`	`[Test]`	`[TestMethod]`
Parameterised test	`[Theory]` + `[InlineData]`	`[TestCase]`	`[DataTestMethod]` + `[DataRow]`
Per-test setup / teardown	constructor / Dispose	`[SetUp]` / `[TearDown]`	`[TestInitialize]` / `[TestCleanup]`
Shared fixture	`IClassFixture<T>`	`[OneTimeSetUp]`	`[ClassInitialize]`
Assert equal	`Assert.Equal(exp, act)`	`Assert.That(act, Is.EqualTo(exp))`	`Assert.AreEqual(exp, act)`
Assert throws	`Assert.Throws<T>(...)`	`Assert.Throws<T>(...)`	`Assert.ThrowsException<T>(...)`

Creating a solution with a test project

```
$ dotnet new sln -n School
$ dotnet new classlib -n School.Core
$ dotnet new xunit -n School.Core.Tests
$ dotnet sln add School.Core School.Core.Tests
$ dotnet add School.Core.Tests reference School.Core
$ dotnet test
```

23.2.2 Arrange-Act-Assert

Structure every test in three parts. **Arrange**: create the object under test and its inputs. **Act**: call the one method being tested. **Assert**: verify the outcome. Name tests so that a failure explains itself, for example `Method_Scenario_ExpectedResult`. Each test should check one behaviour, run in milliseconds, and not depend on other tests, the clock, the network or a real database.

UNIT TESTS IN THE ARRANGE-ACT-ASSERT STYLE Program.cs

```
// School.Core/GradeCalculator.cs
public class GradeCalculator
{
    public char Grade(int marks)
    {
        ArgumentOutOfRangeException.ThrowIfNegative(marks);
        ArgumentOutOfRangeException.ThrowIfGreaterThan(marks, 100);
        return marks switch { >= 85 => 'A', >= 70 => 'B', >= 40 => 'C', _ => 'F' };
    }
    public double Average(IReadOnlyList<int> marks) =>
        marks.Count == 0 ? throw new InvalidOperationException("No marks") : marks.Average();
}

// School.Core.Tests/GradeCalculatorTests.cs
public class GradeCalculatorTests
{
    private readonly GradeCalculator sut = new();        // "system under test", new for every test in
xUnit

    [Fact]
    public void Grade_MarksAbove85_ReturnsA()
    {
        // Arrange
        int marks = 91;
        // Act
        char grade = sut.Grade(marks);
        // Assert
        Assert.Equal('A', grade);
    }

    [Fact]
    public void Grade_NegativeMarks_Throws() =>
        Assert.Throws<ArgumentOutOfRangeException>(() => sut.Grade(-1));

    [Fact]
    public void Average_EmptyList_ThrowsWithMessage()
    {
        var ex = Assert.Throws<InvalidOperationException>(() => sut.Average([]));
        Assert.Equal("No marks", ex.Message);
    }

    [Fact]
    public void Average_ThreeMarks_ReturnsMean() =>
        Assert.Equal(80.0, sut.Average([70, 80, 90]), precision: 2);
}
```

OUTPUT

```
$ dotnet test
Passed!  - Failed:     0, Passed:     4, Skipped:     0, Total:     4, Duration: 12 ms -
School.Core.Tests.dll (net10.0)
```

NOTE

The duration varies. When a test fails, the runner prints the test name, the expected and actual values (for example "Assert.Equal() Failure: Expected: 'A' Actual: 'B'") and the line, which is why descriptive names and precise assertions matter.

23.2.3 Theory and data-driven tests

Running the same test logic with many inputs is a **data-driven** test. In xUnit, mark the method `[Theory]` and supply rows with `[InlineData]` (constants), `[MemberData]` (a static property or method returning rows) or `[ClassData]`. Each row runs and is reported as a separate test case, which is ideal for boundary values.

THEORIES WITH INLINEDATA AND MEMBERDATA Program.cs

```
public class GradeTheoryTests
{
    private readonly GradeCalculator sut = new();

    [Theory]
    [InlineData(100, 'A')]
    [InlineData(85, 'A')]          // boundary
    [InlineData(84, 'B')]          // just below the boundary
    [InlineData(70, 'B')]
    [InlineData(40, 'C')]
    [InlineData(39, 'F')]
    [InlineData(0, 'F')]
    public void Grade_ReturnsExpectedLetter(int marks, char expected) =>
        Assert.Equal(expected, sut.Grade(marks));

    public static TheoryData<int[], double> AverageCases => new()
    {
        { new[] { 50 }, 50.0 },
        { new[] { 60, 80 }, 70.0 },
        { new[] { 33, 33, 34 }, 33.33 }
    };

    [Theory]
    [MemberData(nameof(AverageCases))]
    public void Average_ReturnsMean(int[] marks, double expected) =>
        Assert.Equal(expected, sut.Average(marks), precision: 2);
}
```

OUTPUT

```
$ dotnet test --filter "FullyQualifiedName~GradeTheoryTests"
Passed!  - Failed:     0, Passed:    10, Skipped:     0, Total:    10, Duration: 9 ms -
School.Core.Tests.dll (net10.0)
```

23.2.4 Mocking with Moq or NSubstitute

A unit test should not send real emails or hit a real database. When the class under test depends on interfaces (thanks to DI), the test passes in **test doubles**: a **stub** returns canned answers, a **mock** also verifies that it was called as expected, and a **fake** is a working lightweight implementation (an in-memory repository). Libraries create them at run time: **NSubstitute** (concise syntax) and **Moq** are the most popular.

MOCKING DEPENDENCIES WITH NSUBSTITUTE Program.cs

```
// production code
public interface IStudentRepository { Student? Find(int id); void Save(Student s); }
public interface IEmailSender { void Send(string to, string subject); }
public record Student(int Id, string Name, string Email, int Marks);

public class ResultService(IStudentRepository repo, IEmailSender email)
```

```
{
    public bool Publish(int id, int marks)
    {
        var s = repo.Find(id);
        if (s is null) return false;
        repo.Save(s with { Marks = marks });
        if (marks >= 85) email.Send(s.Email, "Congratulations on your distinction!");
        return true;
    }
}

// tests (NSubstitute)
using NSubstitute;

public class ResultServiceTests
{
    private readonly IStudentRepository repo = Substitute.For<IStudentRepository>();
    private readonly IEmailSender email = Substitute.For<IEmailSender>();

    [Fact]
    public void Publish_Distinction_SavesAndSendsEmail()
    {
        repo.Find(7).Returns(new Student(7, "Asha", "asha@example.com", 0));      // stub
        var sut = new ResultService(repo, email);

        bool ok = sut.Publish(7, 91);

        Assert.True(ok);
        repo.Received(1).Save(Arg.Is<Student>(s => s.Id == 7 && s.Marks == 91));  // verify interaction
        email.Received(1).Send("asha@example.com", Arg.Any<string>());
    }

    [Fact]
    public void Publish_UnknownStudent_ReturnsFalseAndDoesNothing()
    {
        repo.Find(Arg.Any<int>()).Returns((Student?)null);
        var sut = new ResultService(repo, email);

        Assert.False(sut.Publish(99, 50));
        repo.DidNotReceive().Save(Arg.Any<Student>());
        email.DidNotReceiveWithAnyArgs().Send(default!, default!);
    }
}
```

OUTPUT

```
$ dotnet test
Passed!  - Failed:     0, Passed:     2, Skipped:     0, Total:     2, Duration: 64 ms -
School.Core.Tests.dll (net10.0)
```

THE SAME TEST WITH MOQ C#

```
var repo = new Mock<IStudentRepository>();
repo.Setup(r => r.Find(7)).Returns(new Student(7, "Asha", "asha@example.com", 0));
var email = new Mock<IEmailSender>();
var sut = new ResultService(repo.Object, email.Object);

sut.Publish(7, 91);

repo.Verify(r => r.Save(It.Is<Student>(s => s.Marks == 91)), Times.Once);
email.Verify(e => e.Send("asha@example.com", It.IsAny<string>()), Times.Once);
```

TIP

Mock only what you own and what crosses a boundary (repositories, email, HTTP, clock). Do not mock simple value objects or the class under test, and avoid verifying every call: over-specified tests break on harmless refactoring. For time, depend on `TimeProvider` (.NET 8) and pass a `FakeTimeProvider` in tests.

23.2.5 Code coverage

Code coverage measures which lines and branches of production code were executed by the tests. The xUnit template includes the `coverlet.collector` package; `dotnet test --collect:"XPlat Code Coverage"` writes a Cobertura XML file, and **ReportGenerator** turns it into an HTML report highlighting uncovered lines. Coverage shows what is **not** tested; a high percentage does not prove the tests check the right things. Aim for well-tested business logic (often 70 to 90 percent) rather than chasing 100 percent.

Collecting and viewing coverage (paths vary)

```
$ dotnet test --collect:"XPlat Code Coverage"
Attachments:
  /src/School.Core.Tests/TestResults/5f1c.../coverage.cobertura.xml
$ dotnet tool install -g dotnet-reportgenerator-globaltool
$ reportgenerator -reports:"**/coverage.cobertura.xml" -targetdir:coverage-report -reporttypes:Html
# open coverage-report/index.html: line and branch coverage per class
```

23.2.6 Integration testing ASP.NET Core APIs

Integration tests check that components work together: routing, model binding, validation, DI, middleware, serialization and the database. ASP.NET Core provides `WebApplicationFactory<Program>` (package Microsoft.AspNetCore.Mvc.Testing), which starts the real application in memory with a test server and gives you an `HttpClient` to call it, no network or deployment needed. Override services in the factory to replace the database with SQLite in-memory or a Testcontainers database and to fake external services.

TESTING AN API IN MEMORY WITH WEBAPPLICATIONFACTORY C#

```
// in the API project, make Program visible to tests:   public partial class Program { }
public class StudentApiTests(WebApplicationFactory<Program> factory) :
IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient client = factory.WithWebHostBuilder(b => b.ConfigureServices(services =>
    {
        services.RemoveAll<DbContextOptions<SchoolDb>>();
        services.AddDbContext<SchoolDb>(o => o.UseInMemoryDatabase("tests"));   // or SQLite in-memory /
Testcontainers
    })).CreateClient();

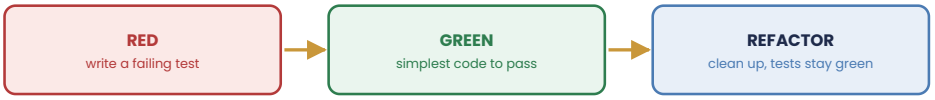
    [Fact]
    public async Task Post_ValidStudent_Returns201AndLocation()
    {
        var response = await client.PostAsJsonAsync("/api/students", new { name = "Asha", dept = "CSE",
marks = 91 });

        Assert.Equal(HttpStatusCode.Created, response.StatusCode);
        Assert.NotNull(response.Headers.Location);
        var body = await response.Content.ReadFromJsonAsync<StudentDto>();
        Assert.Equal("A", body!.Grade);
    }

    [Fact]
    public async Task Post_InvalidMarks_Returns400() =>
        Assert.Equal(HttpStatusCode.BadRequest,
            (await client.PostAsJsonAsync("/api/students", new { name = "Ravi", dept = "ECE", marks = 140
})).StatusCode);
}
```

23.2.7 Test-driven development

Test-driven development (TDD) reverses the usual order: write a failing test for the next small piece of behaviour first, then write just enough code to make it pass, then refactor while keeping all tests green. The **red-green-refactor** cycle keeps designs simple and testable, and produces a complete test suite as a by-product. It is especially effective for business rules and algorithms.



THE TDD CYCLE, REPEATED IN MINUTES

TWO TURNS OF THE TDD CYCLE C#

```
// 1. RED: a test for a rule that does not exist yet
[Fact] public void Fee_LateByThreeDays_Charges15() => Assert.Equal(15m, new LateFee().For(daysLate: 3));
//    does not compile / fails

// 2. GREEN: the simplest code that passes
public class LateFee { public decimal For(int daysLate) => daysLate * 5m; }

// 3. next RED: the fee is capped at 100
[Fact] public void Fee_LateBy30Days_IsCappedAt100() => Assert.Equal(100m, new LateFee().For(daysLate:
30));

// 4. GREEN + REFACTOR
public class LateFee
{
    private const decimal PerDay = 5m, Cap = 100m;
    public decimal For(int daysLate) => Math.Min(Math.Max(daysLate, 0) * PerDay, Cap);
}
```

23.3 Code quality

23.3.1 C# coding conventions and .editorconfig

Consistent style makes code easier to read and review. The widely followed .NET conventions:

Element	Convention	Example
Classes, records, structs, enums, methods, properties, events, namespaces	PascalCase	`StudentService`, `CalculateTotal`
Interfaces	I + PascalCase	`IRepository`
Local variables, parameters	camelCase	`totalMarks`
Private fields	_camelCase	`_balance`
Constants	PascalCase	`MaxSeats`
Async methods	end with Async	`LoadAsync`
Type parameters	T or T + description	`TKey`, `TResult`
Files	one main type per file, named after it	StudentService.cs

Other conventions: use `var` when the type is obvious, file-scoped namespaces, braces on their own lines (Allman style), four-space indentation, explicit access modifiers, and expression bodies only for simple members. An `.editorconfig` file at the repository root encodes these rules so that every IDE formats code the same way and `dotnet format` can fix violations automatically (Module 19 showed the syntax).

Formatting from the command line

```
$ dotnet format                    # fix whitespace, style and analyzer issues in the solution
$ dotnet format --verify-no-changes   # in CI: fail if formatting is not clean
```

23.3.2 Analyzers and warnings as errors

Compiler warnings and Roslyn analyzers (Module 19) catch bugs early: possible null dereferences, unused variables, missing awaits, culture-sensitive string operations, disposable objects not disposed, and security

issues. Treat warnings as errors in CI (`<TreatWarningsAsErrors>true</TreatWarningsAsErrors>`), raise the analysis level, and suppress individual diagnostics only with a justification. A `Directory.Build.props` file applies these settings to every project in the repository.

DIRECTORY.BUILD.PROPS: SHARED SETTINGS FOR ALL PROJECTS xml

```xml
<Project>
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <AnalysisLevel>latest-recommended</AnalysisLevel>
    <EnforceCodeStyleInBuild>true</EnforceCodeStyleInBuild>
  </PropertyGroup>
</Project>
```

23.3.3 Code reviews

In a **code review** (a pull request on GitHub or Azure DevOps), teammates read a change before it is merged. Reviews find bugs, spread knowledge, keep the code base consistent and mentor newcomers. Keep pull requests small and focused, describe what and why, make sure CI is green first, and review the code, not the person.

Reviewer checklist	Questions
Correctness	Does it do what the description says? Edge cases, nulls, empty collections, concurrency?
Tests	Are there tests for the new behaviour and the bug fixed? Do they test behaviour, not implementation?
Design	Single responsibility? Fits the architecture? Any duplication?
Readability	Clear names, small methods, no dead code or commented-out blocks?
Security	Input validated? Parameterized SQL? No secrets in code or logs? Authorization checked?
Performance	N+1 queries, unnecessary allocations in hot paths, missing async?
Operability	Useful logs and error messages? Configuration instead of hard-coded values?

23.4 Design principles

23.4.1 SOLID

SOLID is a set of five object-oriented design principles, collected by Robert C. Martin, that make code easier to understand, extend and test.

Principle	Statement	In C# practice
Single Responsibility	a class should have one reason to change	separate StudentService (rules), StudentRepository (storage), ReportPdfWriter (output)
Open/Closed	open for extension, closed for modification	add a new IDiscountStrategy class instead of editing a growing switch
Liskov Substitution	subtypes must be usable wherever the base type is expected	a derived class must not throw NotSupportedException for base members or weaken guarantees

Principle	Statement	In C# practice
Interface Segregation	many small client-specific interfaces beat one large one	IReadRepository and IWriteRepository instead of a 20-method IRepository
Dependency Inversion	depend on abstractions, not concretions	constructor-inject IEmailSender; register the implementation in DI

SOLID IN A SMALL EXAMPLE

Program.cs

```
// Open/Closed + Dependency Inversion: new fee rules are added as classes, the calculator never changes
var calculator = new FeeCalculator([new BaseFee(), new HostelFee(), new ScholarshipDiscount()]);
Console.WriteLine(calculator.Total(new StudentInfo("Asha", UsesHostel: true, ScholarshipPct: 25)));
Console.WriteLine(calculator.Total(new StudentInfo("Ravi", UsesHostel: false, ScholarshipPct: 0)));

public record StudentInfo(string Name, bool UsesHostel, int ScholarshipPct);

public interface IFeeRule { decimal Apply(StudentInfo s, decimal current); }          // abstraction

public class BaseFee : IFeeRule { public decimal Apply(StudentInfo s, decimal c) => c + 50_000; }
public class HostelFee : IFeeRule { public decimal Apply(StudentInfo s, decimal c) => s.UsesHostel ? c +
30_000 : c; }
public class ScholarshipDiscount : IFeeRule { public decimal Apply(StudentInfo s, decimal c) => c * (100
- s.ScholarshipPct) / 100; }

public class FeeCalculator(IEnumerable<IFeeRule> rules)                                 // depends on the
abstraction
{
    public string Total(StudentInfo s) => $"{s.Name}: {rules.Aggregate(0m, (fee, rule) => rule.Apply(s,
fee)):N0}";
}
```

OUTPUT

```
Asha: 60,000
Ravi: 50,000
```

A CLASSIC LISKOV SUBSTITUTION VIOLATION

Program.cs

```
// Liskov violation: a Square that inherits a mutable Rectangle breaks callers' expectations
Rectangle r = new Square();
r.Width = 4;
r.Height = 5;
Console.WriteLine($"expected area 20, got {r.Area}");      // surprising for code written against
Rectangle

class Rectangle { public virtual int Width { get; set; } public virtual int Height { get; set; } public
int Area => Width * Height; }
class Square : Rectangle
{
    public override int Width { set { base.Width = value; base.Height = value; } }
    public override int Height { set { base.Width = value; base.Height = value; } }
}
```

OUTPUT

```
expected area 20, got 25
```

NOTE

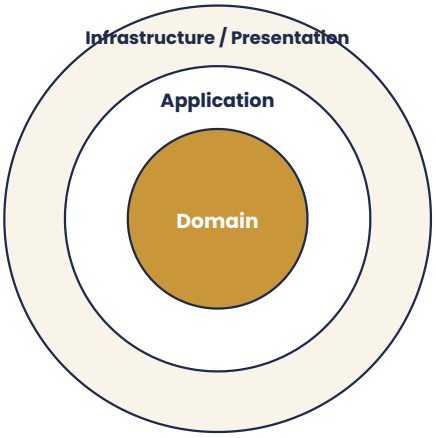
A better design makes Rectangle and Square separate immutable types (records) implementing a common IShape, so neither can surprise code that uses the other.

23.4.2 DRY, KISS, YAGNI

Principle	Meaning	Watch out for
DRY: Don't Repeat Yourself	every piece of knowledge has a single, authoritative place (a validation rule, a tax rate, a query)	copy-pasted blocks; but also premature abstraction of code that is only coincidentally similar
KISS: Keep It Simple, Stupid	prefer the simplest solution that works and that others can read	clever one-liners, needless patterns, deep inheritance
YAGNI: You Aren't Gonna Need It	do not build features or flexibility until they are actually required	generic frameworks "for later", unused configuration options, speculative interfaces

23.4.3 Clean Architecture (overview)

Clean Architecture (also called Onion or Hexagonal/Ports-and-Adapters architecture) arranges code in concentric layers with one rule: **source dependencies point inward**. The **Domain** (entities, value objects, business rules) is at the centre and depends on nothing. The **Application** layer contains use cases and defines interfaces (ports) for what it needs, such as repositories or email. **Infrastructure** implements those interfaces with EF Core, SMTP, files and external APIs, and the **Presentation** layer (ASP.NET Core endpoints, Blazor, console) calls the use cases. Business logic can then be tested without a database or web server and survives technology changes. For small applications, a simpler layered or vertical-slice structure (code organised by feature) is often enough.



dependencies point inward: outer layers know inner ones, never the reverse

CLEAN ARCHITECTURE LAYERS

23.5 Design patterns in C#

Design patterns are proven, named solutions to recurring design problems, catalogued by the "Gang of Four" (GoF) in 1994. Knowing them gives a shared vocabulary ("use a decorator for caching") and ready-made designs. In modern C#, many patterns are lighter thanks to language features (delegates, events, records, DI, LINQ), and several are built into .NET itself.

23.5.1 Creational: Singleton, Factory, Builder

Singleton ensures a class has exactly one instance with global access. In ASP.NET Core, prefer registering a service as a DI singleton over a static Instance property, so it remains testable. **Factory** (Factory Method / Simple Factory) centralises object creation, choosing the concrete type from input, so callers depend only on an abstraction. **Builder** constructs complex objects step by step with a fluent API, as `WebApplication.CreateBuilder`, `StringBuilder` and `HostBuilder` do.

SINGLETON, FACTORY AND BUILDER

Program.cs

```
var cfg1 = AppConfig.Instance;
var cfg2 = AppConfig.Instance;
Console.WriteLine($"singleton: same instance {ReferenceEquals(cfg1, cfg2)}, created {AppConfig.Created}
time(s)");

foreach (string kind in new[] { "email", "sms", "push" })
    Console.WriteLine(NotifierFactory.Create(kind).Send("Fees due"));

var request = new HttpRequestBuilder()
    .WithMethod("POST")
    .WithUrl("https://api.example.com/students")
    .WithHeader("Authorization", "Bearer abc")
    .WithBody("""{"name":"Asha"}""")
    .Build();
Console.WriteLine(request);

public sealed class AppConfig
{
    private static readonly Lazy<AppConfig> instance = new(() => new AppConfig());   // thread-safe lazy
creation
    public static AppConfig Instance => instance.Value;
    public static int Created { get; private set; }
    private AppConfig() => Created++;
}

public interface INotifier { string Send(string msg); }
public class EmailNotifier : INotifier { public string Send(string m) => $"email: {m}"; }
public class SmsNotifier : INotifier { public string Send(string m) => $"sms: {m}"; }
public class PushNotifier : INotifier { public string Send(string m) => $"push: {m}"; }

public static class NotifierFactory
{
    public static INotifier Create(string kind) => kind switch
    {
        "email" => new EmailNotifier(),
        "sms" => new SmsNotifier(),
        "push" => new PushNotifier(),
        _ => throw new ArgumentException($"unknown notifier '{kind}'")
    };
}

public record HttpRequestSpec(string Method, string Url, IReadOnlyDictionary<string, string> Headers,
string? Body)
{
    public override string ToString() => $"{Method} {Url} headers={Headers.Count} body={Body?.Length ??
0} chars";
}

public class HttpRequestBuilder
{
    private string method = "GET", url = "";
    private readonly Dictionary<string, string> headers = new();
    private string? body;
    public HttpRequestBuilder WithMethod(string m) { method = m; return this; }
    public HttpRequestBuilder WithUrl(string u) { url = u; return this; }
    public HttpRequestBuilder WithHeader(string k, string v) { headers[k] = v; return this; }
    public HttpRequestBuilder WithBody(string b) { body = b; return this; }
    public HttpRequestSpec Build() =>
        string.IsNullOrEmpty(url) ? throw new InvalidOperationException("URL is required") : new(method,
url, headers, body);
}
```

```
OUTPUT
singleton: same instance True, created 1 time(s)
email: Fees due
sms: Fees due
push: Fees due
POST https://api.example.com/students headers=1 body=15 chars
```

23.5.2 Structural: Adapter, Decorator, Facade

Adapter converts the interface of an existing class (often a third-party or legacy API) into the interface your code expects. **Decorator** wraps an object implementing an interface with another implementation of the same interface that adds behaviour (caching, logging, retries) before or after delegating; .NET streams (`GZipStream` over `FileStream`) and ASP.NET Core middleware follow this idea. **Facade** provides one simple entry point to a complex subsystem.

ADAPTER, DECORATORS AND FACADE — Program.cs

```
IWeatherService service = new CachingWeatherDecorator(new LoggingWeatherDecorator(new
LegacyWeatherAdapter(new OldWeatherApi())));
Console.WriteLine(service.TemperatureFor("Pune"));
Console.WriteLine(service.TemperatureFor("Pune"));       // served from the cache: no log, no API call
Console.WriteLine(service.TemperatureFor("Delhi"));

var checkout = new CheckoutFacade();
Console.WriteLine(checkout.PlaceOrder("Asha", "book-42", 2));

public interface IWeatherService { string TemperatureFor(string city); }

public class OldWeatherApi                                          // legacy API with an awkward shape
{
    public double GetTempFahrenheit(string cityCode) => cityCode == "PNQ" ? 86.0 : 95.0;
}

public class LegacyWeatherAdapter(OldWeatherApi api) : IWeatherService   // Adapter
{
    public string TemperatureFor(string city)
    {
        string code = city == "Pune" ? "PNQ" : "DEL";
        double c = (api.GetTempFahrenheit(code) - 32) * 5 / 9;
        return $"{city}: {c:F0} C";
    }
}

public class LoggingWeatherDecorator(IWeatherService inner) : IWeatherService   // Decorator: adds
logging
{
    public string TemperatureFor(string city)
    {
        Console.WriteLine($"  [log] fetching {city}");
        return inner.TemperatureFor(city);
    }
}

public class CachingWeatherDecorator(IWeatherService inner) : IWeatherService   // Decorator: adds
caching
{
    private readonly Dictionary<string, string> cache = new();
    public string TemperatureFor(string city) =>
        cache.TryGetValue(city, out var v) ? v : cache[city] = inner.TemperatureFor(city);
}

public class CheckoutFacade                                         // Facade over several subsystems
{
    private readonly Inventory inventory = new();
```

```
    private readonly Payments payments = new();
    private readonly Shipping shipping = new();
    public string PlaceOrder(string customer, string item, int qty)
    {
        inventory.Reserve(item, qty);
        string receipt = payments.Charge(customer, qty * 250m);
        return $"{receipt}; {shipping.Schedule(customer)}";
    }
}
class Inventory { public void Reserve(string item, int qty) { } }
class Payments { public string Charge(string c, decimal amount) => $"charged {c} Rs {amount}"; }
class Shipping { public string Schedule(string c) => "delivery in 3 days"; }
```

OUTPUT

```
  [log] fetching Pune
Pune: 30 C
Pune: 30 C
  [log] fetching Delhi
Delhi: 35 C
charged Asha Rs 500; delivery in 3 days
```

TIP

With DI, decorators are registered by wrapping: `services.AddScoped<IWeatherService>(sp => new CachingWeatherDecorator(new LegacyWeatherAdapter(...)))`, or with the Scrutor library's `Decorate<IWeatherService, CachingWeatherDecorator>()`.

23.5.3 Behavioural: Strategy, Observer (events), Command, Mediator

Strategy defines a family of interchangeable algorithms behind one interface and lets the caller choose at run time (sorting comparers, pricing rules, payment methods); in C# a strategy can also be a `Func`. **Observer** notifies subscribers of changes; C# events are the built-in implementation (Module 15), and `IObservable<T>` with Rx is the reactive version. **Command** wraps a request as an object, enabling queues, logging and undo. **Mediator** routes messages between objects so they do not reference each other directly; the MediatR library popularised it in ASP.NET Core for request handlers.

COMMAND WITH UNDO, AND A MINIMAL MEDIATOR

Program.cs

```
var editor = new TextEditor();
var history = new Stack<ICommand>();
void Run(ICommand c) { c.Execute(); history.Push(c); Console.WriteLine($"[{editor.Text}]"); }

Run(new AppendCommand(editor, "Hello"));
Run(new AppendCommand(editor, ", World"));
Run(new UpperCommand(editor));
history.Pop().Undo(); Console.WriteLine($"undo -> [{editor.Text}]");
history.Pop().Undo(); Console.WriteLine($"undo -> [{editor.Text}]");

var mediator = new Mediator();
mediator.Register<EnrolStudent, string>(cmd => $"enrolled {cmd.Name} in {cmd.Course}");
mediator.Register<GetSeats, int>(q => q.Course == "C#" ? 12 : 0);
Console.WriteLine(mediator.Send<EnrolStudent, string>(new EnrolStudent("Asha", "C#")));
Console.WriteLine($"seats left: {mediator.Send<GetSeats, int>(new GetSeats("C#"))}");

public class TextEditor { public string Text { get; set; } = ""; }

public interface ICommand { void Execute(); void Undo(); }                // Command
public class AppendCommand(TextEditor ed, string text) : ICommand
{
    public void Execute() => ed.Text += text;
    public void Undo() => ed.Text = ed.Text[..^text.Length];
}
public class UpperCommand(TextEditor ed) : ICommand
{
    private string before = "";
    public void Execute() { before = ed.Text; ed.Text = ed.Text.ToUpper(); }
    public void Undo() => ed.Text = before;
}

public record EnrolStudent(string Name, string Course);
public record GetSeats(string Course);

public class Mediator                                                      // Mediator (a tiny
MediatR)
{
    private readonly Dictionary<Type, Delegate> handlers = new();
    public void Register<TRequest, TResponse>(Func<TRequest, TResponse> handler) =>
handlers[typeof(TRequest)] = handler;
    public TResponse Send<TRequest, TResponse>(TRequest request) => ((Func<TRequest,
TResponse>)handlers[typeof(TRequest)])(request);
}
```

OUTPUT

```
[Hello]
[Hello, World]
[HELLO, WORLD]
undo -> [Hello, World]
undo -> [Hello]
enrolled Asha in C#
seats left: 12
```

23.5.4 Repository, Unit of Work and Options patterns

Module 21 introduced **Repository** (collection-like access to aggregates, hiding persistence) and **Unit of Work** (commit a business transaction as a whole; DbContext already plays this role). The **Options pattern** binds configuration sections to strongly typed classes and injects them as `IOptions<T>` (fixed at start-up), `IOptionsSnapshot<T>` (re-read per request) or `IOptionsMonitor<T>` (change notifications), with validation at start-up so misconfiguration fails fast instead of at the first request.

THE OPTIONS PATTERN WITH START-UP VALIDATION

C#

```
public class SmtpOptions
{
    public const string Section = "Smtp";
    [Required] public string Host { get; set; } = "";
    [Range(1, 65535)] public int Port { get; set; } = 587;
    public string From { get; set; } = "noreply@example.com";
}

builder.Services.AddOptions<SmtpOptions>()
    .Bind(builder.Configuration.GetSection(SmtpOptions.Section))
    .ValidateDataAnnotations()
    .ValidateOnStart();                                  // the app refuses to start with a bad config

public class SmtpEmailSender(IOptions<SmtpOptions> options) : IEmailSender
{
    private readonly SmtpOptions o = options.Value;
    public void Send(string to, string subject) { /* connect to o.Host:o.Port, send from o.From */ }
}
```

23.6 Tooling

23.6.1 NuGet packages and versioning

NuGet is the .NET package manager; nuget.org hosts hundreds of thousands of packages. Add packages with `dotnet add package` (recorded as `PackageReference` in the .csproj), restore them automatically on build, and list outdated or vulnerable ones with `dotnet list package --outdated --vulnerable`. Packages use **semantic versioning** MAJOR.MINOR.PATCH: a major version may break the API, a minor adds features compatibly, a patch only fixes bugs. In multi-project solutions, **Central Package Management** (`Directory.Packages.props`) keeps versions consistent. You can publish your own libraries with `dotnet pack` to nuget.org or a private feed.

Working with NuGet

```
$ dotnet add package Serilog.AspNetCore --version 9.0.0
$ dotnet list package --outdated
$ dotnet list package --vulnerable --include-transitive
$ dotnet pack -c Release -o ./artifacts          # creates MyLibrary.1.2.0.nupkg
$ dotnet nuget push ./artifacts/MyLibrary.1.2.0.nupkg --api-key $KEY --source
https://api.nuget.org/v3/index.json
```

23.6.2 Solution structure for multi-project apps

A **solution** (.sln, or the newer XML-based .slnx) groups related projects. Separate projects enforce architectural boundaries: the compiler prevents the Domain project from referencing Infrastructure if no reference exists. A common layout:

A TYPICAL SOLUTION LAYOUT

text

```
School/
├── School.sln                      (or School.slnx)
├── Directory.Build.props           shared build settings
├── Directory.Packages.props        central package versions
├── .editorconfig  .gitignore  README.md
├── src/
│   ├── School.Domain/              entities, value objects, domain rules (no dependencies)
│   ├── School.Application/         use cases, DTOs, interfaces (refs Domain)
│   ├── School.Infrastructure/      EF Core, email, files (refs Application)
│   └── School.Api/                 ASP.NET Core endpoints, Program.cs (refs Application +
Infrastructure)
└── tests/
    ├── School.Domain.Tests/
    ├── School.Application.Tests/
    └── School.Api.IntegrationTests/
```

23.6.3 Git and GitHub

Git is the standard version control system: it records snapshots (commits) of the project, lets several people work in parallel on **branches**, and merges their work. **GitHub** (and Azure DevOps Repos, GitLab) hosts repositories and adds pull requests, code review, issues and CI. Every .NET repository needs a `.gitignore` (`dotnet new gitignore`) so that bin/, obj/, user secrets and IDE files are not committed.

A basic feature-branch workflow

```
$ dotnet new gitignore
$ git init
$ git add .
$ git commit -m "Initial Student API"
$ git switch -c feature/strategy-grading        # a branch for one feature
$ git commit -am "Add grading strategies"
$ git push -u origin feature/strategy-grading    # then open a pull request on GitHub
$ git switch main && git pull                    # after review and merge
```

23.6.4 CI with GitHub Actions or Azure DevOps (overview)

Continuous Integration (CI) builds and tests every push and pull request automatically on a clean machine, so broken builds and failing tests are caught before merging. **Continuous Delivery/Deployment** (CD) then publishes passing builds to test and production environments. GitHub Actions workflows are YAML files in `.github/workflows`; Azure DevOps Pipelines are equivalent.

A GITHUB ACTIONS CI WORKFLOW

yaml

```yaml
# .github/workflows/ci.yml
name: CI
on:
  push: { branches: [ main ] }
  pull_request:
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with: { dotnet-version: '10.0.x' }
      - run: dotnet restore
      - run: dotnet format --verify-no-changes
      - run: dotnet build --no-restore -c Release
      - run: dotnet test --no-build -c Release --collect:"XPlat Code Coverage"
```

Debug with breakpoints (conditional, hit count, tracepoints), stepping, and the Locals, Watch, Immediate, Call Stack and Tasks windows, breaking on thrown exceptions. Write fast, isolated unit tests with xUnit (or NUnit/MSTest) in Arrange-Act-Assert style, use theories for data-driven cases, mock boundaries with NSubstitute or Moq, measure coverage, test APIs end-to-end with WebApplicationFactory, and consider TDD. Keep quality high with conventions, .editorconfig, analyzers, warnings as errors and reviews. Apply SOLID, DRY, KISS and YAGNI and keep dependencies pointing inward. Use patterns where they fit: Singleton, Factory, Builder, Adapter, Decorator, Facade, Strategy, Observer, Command, Mediator, Repository, Unit of Work and Options. Manage packages with NuGet, structure solutions by layer, use Git branches and pull requests, and automate builds and tests with CI.

PRACTICE CHECKPOINT

Add **xUnit tests with mocks** to the Student API's service layer, and refactor one feature to use the **Strategy pattern**.

SAMPLE SOLUTION

Program.cs

```
// ---------- production code: grading refactored to the Strategy pattern ----------
public interface IGradingStrategy { string Name { get; } string Grade(int marks); }

public class AbsoluteGrading : IGradingStrategy
{
    public string Name => "absolute";
    public string Grade(int m) => m >= 85 ? "A" : m >= 70 ? "B" : m >= 40 ? "C" : "F";
}

public class PassFailGrading : IGradingStrategy
{
    public string Name => "pass-fail";
    public string Grade(int m) => m >= 40 ? "PASS" : "FAIL";
}

public interface IStudentRepository { Student? Find(int id); IReadOnlyList<Student> All(); }
public interface IGradingStrategyProvider { IGradingStrategy For(string course); }
public record Student(int Id, string Name, string Course, int Marks);
public record GradedStudent(string Name, string Grade, string Scheme);

public class GradingService(IStudentRepository repo, IGradingStrategyProvider strategies)
{
    public GradedStudent? Grade(int id)
    {
        var s = repo.Find(id);
        if (s is null) return null;
        var strategy = strategies.For(s.Course);                    // chosen at run time
        return new GradedStudent(s.Name, strategy.Grade(s.Marks), strategy.Name);
    }

    public Dictionary<string, int> Distribution(string course) =>
        repo.All().Where(s => s.Course == course)
            .GroupBy(s => strategies.For(course).Grade(s.Marks))
            .ToDictionary(g => g.Key, g => g.Count());
}

// ---------- tests: xUnit + NSubstitute ----------
using NSubstitute;

public class GradingServiceTests
{
    private readonly IStudentRepository repo = Substitute.For<IStudentRepository>();
    private readonly IGradingStrategyProvider provider = Substitute.For<IGradingStrategyProvider>();
    private readonly GradingService sut;
```

```
    public GradingServiceTests() => sut = new GradingService(repo, provider);

    [Fact]
    public void Grade_UnknownStudent_ReturnsNull()
    {
        repo.Find(42).Returns((Student?)null);
        Assert.Null(sut.Grade(42));
        provider.DidNotReceiveWithAnyArgs().For(default!);
    }

    [Theory]
    [InlineData("CS101", 91, "A", "absolute")]
    [InlineData("LAB1", 55, "PASS", "pass-fail")]
    [InlineData("LAB1", 12, "FAIL", "pass-fail")]
    public void Grade_UsesTheCoursesStrategy(string course, int marks, string grade, string scheme)
    {
        repo.Find(1).Returns(new Student(1, "Asha", course, marks));
        provider.For("CS101").Returns(new AbsoluteGrading());
        provider.For("LAB1").Returns(new PassFailGrading());

        var result = sut.Grade(1);

        Assert.Equal(new GradedStudent("Asha", grade, scheme), result);
        provider.Received(1).For(course);
    }

    [Fact]
    public void Distribution_CountsGradesForOneCourse()
    {
        repo.All().Returns([new(1, "Asha", "CS101", 91), new(2, "Ravi", "CS101", 72), new(3, "Meera",
"CS101", 88), new(4, "Kiran", "LAB1", 30)]);
        provider.For("CS101").Returns(new AbsoluteGrading());

        var d = sut.Distribution("CS101");

        Assert.Equal(2, d["A"]);
        Assert.Equal(1, d["B"]);
        Assert.False(d.ContainsKey("F"));
    }

    [Fact]
    public void Strategies_CanBeTestedOnTheirOwn()
    {
        IGradingStrategy s = new PassFailGrading();
        Assert.Equal(new[] { "FAIL", "PASS", "PASS" }, new[] { 39, 40, 100 }.Select(s.Grade).ToArray());
    }
}
```

OUTPUT

```
$ dotnet test
Passed!  - Failed:     0, Passed:     6, Skipped:     0, Total:     6, Duration: 71 ms -
StudentApi.Tests.dll (net10.0)
```

MODULE 24 · ADVANCED

Platforms, data structures, projects and interviews

Minimum duration **5 days** | Level **Advanced**

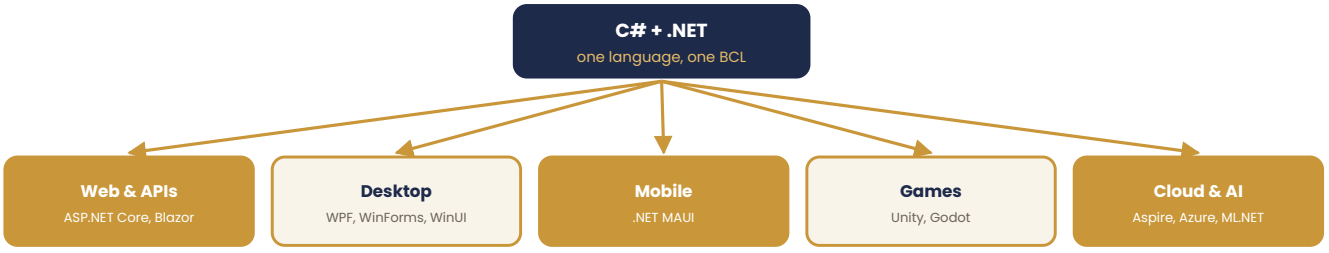
The final module widens the view and prepares for placements. It surveys the other platforms C# reaches (desktop, mobile, games, cloud-native and AI), applies C# to the data structures and algorithms asked in coding rounds (Big-O, lists, stacks, queues, trees, heaps, hash tables, graphs, sorting and the classic problem-solving techniques), proposes capstone projects for a portfolio, collects interview topics and tricky output questions, and outlines the career paths open to C# developers.

IN THIS MODULE

24.1 Other C# platforms (overview)
24.2 Data structures and algorithms in C#
24.3 Capstone projects
24.4 Interview preparation
24.5 Career paths with C#

24.1 Other C# platforms (overview)

The same language, base class library and tools serve very different kinds of applications. Everything learned in Levels 1 and 2 applies to all of them; only the UI or hosting framework changes.



WHERE C# RUNS

24.1.1 Desktop: WPF, Windows Forms, WinUI

Framework	Since	UI definition	Strengths
Windows Forms	2002	drag-and-drop designer, C# code	fastest way to build simple internal tools; huge installed base
WPF	2006	XAML + data binding + MVVM	rich, styleable line-of-business apps; vector graphics
WinUI 3 (Windows App SDK)	2021	XAML, Fluent design	modern native Windows 11 look; successor to UWP
Avalonia, Uno Platform (third party)	-	XAML	cross-platform desktop (Windows, macOS, Linux) and web

All three Microsoft frameworks are Windows-only and run on modern .NET. Desktop apps typically use the **MVVM** pattern (Model-View-ViewModel) with data binding and commands; the CommunityToolkit.Mvvm package generates the boilerplate with source generators (`[ObservableProperty]`, `[RelayCommand]`).

24.1.2 Cross-platform apps with .NET MAUI

.NET MAUI (Multi-platform App UI) builds native apps for **Android, iOS, macOS and Windows** from a single project and code base, with XAML or C# UI, MVVM, and access to device APIs (camera, GPS, sensors, notifications). **Blazor Hybrid** hosts Blazor components inside a MAUI app, sharing UI code with a web app.

A MAUI PAGE WITH MVVM xml

```xml
<!-- MainPage.xaml -->
<VerticalStackLayout Padding="30" Spacing="20">
    <Label Text="{Binding Greeting}" FontSize="24" />
    <Entry Placeholder="Your name" Text="{Binding Name}" />
    <Button Text="Greet" Command="{Binding GreetCommand}" />
</VerticalStackLayout>

// MainViewModel.cs (CommunityToolkit.Mvvm)
public partial class MainViewModel : ObservableObject
{
    [ObservableProperty] private string name = "";
    [ObservableProperty] private string greeting = "Welcome";
    [RelayCommand] private void Greet() => Greeting = $"Hello, {Name}!";
}
```

24.1.3 Game development with Unity

Unity, one of the most widely used game engines (mobile, PC, consoles, AR/VR), uses C# for all game scripting. Scripts are classes deriving from `MonoBehaviour`, attached to game objects, with lifecycle methods such as `Start` (once) and `Update` (every frame). The open-source **Godot** engine also supports C#. Game code relies heavily on vectors, physics, coroutines and avoiding per-frame allocations.

A UNITY MONOBEHAVIOUR SCRIPT C#

```csharp
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    [SerializeField] private float speed = 5f;          // editable in the Unity Inspector
    private Rigidbody2D body;

    void Start() => body = GetComponent<Rigidbody2D>();

    void Update()                                        // called every frame
    {
        float x = Input.GetAxis("Horizontal");
        float y = Input.GetAxis("Vertical");
        body.linearVelocity = new Vector2(x, y) * speed;
    }

    void OnCollisionEnter2D(Collision2D other)
    {
        if (other.gameObject.CompareTag("Coin")) Destroy(other.gameObject);
    }
}
```

24.1.4 Cloud-native apps with .NET Aspire and Azure

.NET Aspire is an opinionated stack for building distributed, cloud-native applications: an **AppHost** project describes in C# which services, databases, caches and queues make up the system; running it starts everything locally with service discovery and a **dashboard** showing logs, traces and metrics (OpenTelemetry); **ServiceDefaults** add health checks, resilience and telemetry to each service; and the model can be deployed to Azure Container Apps or Kubernetes. **Azure** offers first-class .NET support: App Service, Functions (serverless), Container Apps, Azure SQL, Cosmos DB, Service Bus, Key Vault and Azure OpenAI.

DESCRIBING A DISTRIBUTED APP WITH .NET ASPIRE C#

```
// AppHost/Program.cs
var builder = DistributedApplication.CreateBuilder(args);

var cache = builder.AddRedis("cache");
var db = builder.AddPostgres("pg").AddDatabase("school");

var api = builder.AddProject<Projects.School_Api>("api")
                 .WithReference(db)
                 .WithReference(cache);

builder.AddProject<Projects.School_Web>("web")
       .WithExternalHttpEndpoints()
       .WithReference(api);                // the web front end discovers the API by name

builder.Build().Run();
```

24.1.5 Machine learning with ML.NET and AI integration with Microsoft.Extensions.AI

ML.NET trains and runs machine-learning models (classification, regression, recommendation, anomaly detection) in C#, and the Model Builder tool generates training code. For generative AI, **Microsoft.Extensions.AI** provides common abstractions such as `IChatClient` and `IEmbeddingGenerator`, with implementations for OpenAI, Azure OpenAI, Ollama (local models) and others, so applications can switch providers through DI and add middleware such as caching, telemetry and function calling. Higher-level frameworks (Semantic Kernel, the Microsoft Agent Framework) build agents and retrieval-augmented generation on top.

CALLING A LANGUAGE MODEL THROUGH ICHATCLIENT C#

```
// dotnet add package Microsoft.Extensions.AI.OpenAI   (or another provider package)
using Microsoft.Extensions.AI;

IChatClient chat = new OpenAI.Chat.ChatClient("gpt-4o-mini", apiKey).AsIChatClient();

var response = await chat.GetResponseAsync(
[
    new ChatMessage(ChatRole.System, "You are a helpful C# tutor. Answer in two sentences."),
    new ChatMessage(ChatRole.User, "What is the difference between a struct and a class?")
]);
Console.WriteLine(response.Text);

// in ASP.NET Core:  builder.Services.AddChatClient(chat).UseDistributedCache().UseOpenTelemetry();
```

24.2 Data structures and algorithms in C#

Coding rounds test problem-solving with data structures and algorithms (DSA). C# is an excellent interview language: its collections (List, Dictionary, HashSet, Stack, Queue, PriorityQueue, SortedSet) cover most needs,

and tuples, LINQ and spans keep solutions short. Interviewers still expect you to know how the structures work inside and to analyse complexity.

24.2.1 Big-O analysis

Big-O notation describes how the running time (or memory) of an algorithm grows with the input size n, ignoring constant factors and lower-order terms: it answers "what happens when n becomes ten times larger?". Analyse the worst case unless stated otherwise; count nested loops (multiply) and sequential steps (add, keep the largest).

Complexity	Name	Example in C#	n = 1,000,000
O(1)	constant	array index, Dictionary lookup, Stack.Push	1 step
O(log n)	logarithmic	binary search, SortedSet.Add, PriorityQueue.Enqueue	about 20
O(n)	linear	one loop, List.Contains, LINQ Where/Sum	1 million
O(n log n)	linearithmic	Array.Sort, merge sort, OrderBy	about 20 million
O(n²)	quadratic	nested loops over the input, bubble sort	10^{12}: too slow
O(2^n)	exponential	all subsets, naive recursive Fibonacci	impossible
O(n!)	factorial	all permutations	impossible



STEPS NEEDED FOR N = 1,000 (O(N^2) DWARFS EVERYTHING ELSE)

COUNTING OPERATIONS

Program.cs

```
int[] data = Enumerable.Range(1, 1000).ToArray();

int ops = 0;
foreach (int x in data) ops++;                                  // O(n)
Console.WriteLine($"single loop: {ops:N0}");

ops = 0;
foreach (int x in data) foreach (int y in data) ops++;          // O(n^2)
Console.WriteLine($"nested loops: {ops:N0}");

ops = 0;
for (int lo = 0, hi = data.Length - 1; lo <= hi; )              // O(log n): binary search for 777
{
    ops++;
    int mid = (lo + hi) / 2;
    if (data[mid] == 777) break;
    if (data[mid] < 777) lo = mid + 1; else hi = mid - 1;
}
Console.WriteLine($"binary search: {ops} steps");
```

OUTPUT

```
single loop: 1,000
nested loops: 1,000,000
binary search: 8 steps
```

24.2.2 Linked lists, stacks and queues from scratch

Building the basic structures yourself shows how the library versions work and is a common interview exercise. A **singly linked list** chains nodes that each hold a value and a reference to the next node: O(1) insertion at the head, O(n) search. A **stack** (LIFO) can be built on an array that grows, a **queue** (FIFO) on a circular array buffer with head and tail indexes.

LINKED LIST, STACK AND QUEUE IMPLEMENTATIONS

Program.cs

```
var list = new SinglyLinkedList<int>();
foreach (int n in new[] { 3, 1, 4, 1, 5 }) list.AddLast(n);
list.AddFirst(9);
Console.WriteLine($"list: {list}  count {list.Count}");
list.Remove(1);                                     // removes the first 1
list.Reverse();
Console.WriteLine($"after Remove(1) and Reverse: {list}");

var stack = new ArrayStack<string>();
foreach (string s in new[] { "a", "b", "c" }) stack.Push(s);
Console.WriteLine($"stack pop: {stack.Pop()} {stack.Pop()}, peek {stack.Peek()}");

var queue = new CircularQueue<int>(3);
queue.Enqueue(10); queue.Enqueue(20); queue.Enqueue(30);
Console.Write($"queue: {queue.Dequeue()} ");
queue.Enqueue(40);                                  // wraps around in the circular buffer
Console.WriteLine($"{queue.Dequeue()} {queue.Dequeue()} {queue.Dequeue()}, empty {queue.Count == 0}");

public class SinglyLinkedList<T>
{
    private class Node(T value) { public T Value = value; public Node? Next; }
    private Node? head;
    public int Count { get; private set; }

    public void AddFirst(T value) { head = new Node(value) { Next = head }; Count++; }
    public void AddLast(T value)
    {
        var node = new Node(value);
```

```
        if (head is null) head = node;
        else { var cur = head; while (cur.Next is not null) cur = cur.Next; cur.Next = node; }
        Count++;
    }
    public bool Remove(T value)
    {
        Node? prev = null;
        for (var cur = head; cur is not null; prev = cur, cur = cur.Next)
            if (EqualityComparer<T>.Default.Equals(cur.Value, value))
            {
                if (prev is null) head = cur.Next; else prev.Next = cur.Next;
                Count--;
                return true;
            }
        return false;
    }
    public void Reverse()                              // classic interview question: O(n), O(1) extra
space
    {
        Node? prev = null, cur = head;
        while (cur is not null) { var next = cur.Next; cur.Next = prev; prev = cur; cur = next; }
        head = prev;
    }
    public override string ToString()
    {
        var parts = new List<string>();
        for (var c = head; c is not null; c = c.Next) parts.Add($"{c.Value}");
        return string.Join(" -> ", parts);
    }
}

public class ArrayStack<T>
{
    private T[] items = new T[4];
    private int count;
    public void Push(T item) { if (count == items.Length) Array.Resize(ref items, count * 2);
items[count++] = item; }
    public T Pop() => count > 0 ? items[--count] : throw new InvalidOperationException("Stack empty");
    public T Peek() => count > 0 ? items[count - 1] : throw new InvalidOperationException("Stack empty");
}

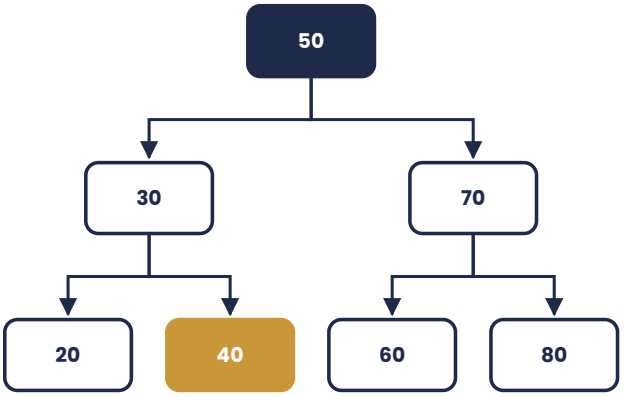
public class CircularQueue<T>(int capacity)
{
    private readonly T[] buffer = new T[capacity];
    private int head, tail;
    public int Count { get; private set; }
    public void Enqueue(T item)
    {
        if (Count == buffer.Length) throw new InvalidOperationException("Queue full");
        buffer[tail] = item; tail = (tail + 1) % buffer.Length; Count++;
    }
    public T Dequeue()
    {
        if (Count == 0) throw new InvalidOperationException("Queue empty");
        T item = buffer[head]; head = (head + 1) % buffer.Length; Count--;
        return item;
    }
}
```

OUTPUT

```
list: 9 -> 3 -> 1 -> 4 -> 1 -> 5  count 6
after Remove(1) and Reverse: 5 -> 1 -> 4 -> 3 -> 9
stack pop: c b, peek a
queue: 10 20 30 40, empty True
```

24.2.3 Binary trees and binary search trees

A **binary tree** is made of nodes with at most two children, left and right. In a **binary search tree** (BST) every value in a node's left subtree is smaller and every value in its right subtree is larger, so search, insertion and deletion take O(height): O(log n) when balanced, O(n) when the tree degenerates into a chain. The library's SortedSet and SortedDictionary are self-balancing red-black trees. Traversals: **in-order** (left, node, right) visits a BST in sorted order; **pre-order** and **post-order** are used for copying and deleting; **level-order** uses a queue.



THE BST BUILT BY INSERTING 50, 30, 70, 20, 40, 60, 80

A BINARY SEARCH TREE

Program.cs

```
var bst = new Bst();
foreach (int v in new[] { 50, 30, 70, 20, 40, 60, 80 }) bst.Insert(v);

Console.WriteLine($"in-order:    {string.Join(" ", bst.InOrder())}");
Console.WriteLine($"pre-order:   {string.Join(" ", bst.PreOrder())}");
Console.WriteLine($"level-order: {string.Join(" ", bst.LevelOrder())}");
Console.WriteLine($"contains 40: {bst.Contains(40)}, contains 45: {bst.Contains(45)}, height
{bst.Height()}, min {bst.Min()}, max {bst.Max()}");
bst.Delete(30);                                          // node with two children
Console.WriteLine($"after deleting 30: {string.Join(" ", bst.InOrder())}, level-order {string.Join(" ",
bst.LevelOrder())}");

public class Bst
{
    private class Node(int v) { public int Value = v; public Node? Left, Right; }
    private Node? root;

    public void Insert(int v) => root = Insert(root, v);
    private static Node Insert(Node? n, int v)
    {
        if (n is null) return new Node(v);
        if (v < n.Value) n.Left = Insert(n.Left, v);
        else if (v > n.Value) n.Right = Insert(n.Right, v);
        return n;                                        // duplicates ignored
    }

    public bool Contains(int v)
    {
        for (var n = root; n is not null; n = v < n.Value ? n.Left : n.Right)
            if (n.Value == v) return true;
        return false;
    }

    public int Height() => Height(root);
    private static int Height(Node? n) => n is null ? 0 : 1 + Math.Max(Height(n.Left), Height(n.Right));
    public int Min() { var n = root!; while (n.Left is not null) n = n.Left; return n.Value; }
    public int Max() { var n = root!; while (n.Right is not null) n = n.Right; return n.Value; }
```

```
    public void Delete(int v) => root = Delete(root, v);
    private static Node? Delete(Node? n, int v)
    {
        if (n is null) return null;
        if (v < n.Value) n.Left = Delete(n.Left, v);
        else if (v > n.Value) n.Right = Delete(n.Right, v);
        else
        {
            if (n.Left is null) return n.Right;                 // zero or one child
            if (n.Right is null) return n.Left;
            var successor = n.Right;                            // two children: smallest in the right
subtree
            while (successor.Left is not null) successor = successor.Left;
            n.Value = successor.Value;
            n.Right = Delete(n.Right, successor.Value);
        }
        return n;
    }

    public IEnumerable<int> InOrder() => InOrder(root);
    private static IEnumerable<int> InOrder(Node? n)
    {
        if (n is null) yield break;
        foreach (int x in InOrder(n.Left)) yield return x;
        yield return n.Value;
        foreach (int x in InOrder(n.Right)) yield return x;
    }

    public IEnumerable<int> PreOrder() => PreOrder(root);
    private static IEnumerable<int> PreOrder(Node? n)
    {
        if (n is null) yield break;
        yield return n.Value;
        foreach (int x in PreOrder(n.Left)) yield return x;
        foreach (int x in PreOrder(n.Right)) yield return x;
    }

    public IEnumerable<int> LevelOrder()
    {
        if (root is null) yield break;
        var q = new Queue<Node>([root]);
        while (q.Count > 0)
        {
            var n = q.Dequeue();
            yield return n.Value;
            if (n.Left is not null) q.Enqueue(n.Left);
            if (n.Right is not null) q.Enqueue(n.Right);
        }
    }
}
```

OUTPUT

```
in-order:    20 30 40 50 60 70 80
pre-order:   50 30 20 40 70 60 80
level-order: 50 30 70 20 40 60 80
contains 40: True, contains 45: False, height 3, min 20, max 80
after deleting 30: 20 40 50 60 70 80, level-order 50 40 70 20 60 80
```

24.2.4 Heaps with PriorityQueue

A **binary heap** is a complete binary tree stored in an array where every parent is smaller than its children (min-heap), so the minimum is always at index 0; insert and remove-min "sift" an element up or down in O(log n). .NET's `PriorityQueue<TElement, TPriority>` is such a min-heap (a 4-ary heap internally). Heaps

solve "top k", "k-th largest", merging sorted lists, scheduling and Dijkstra efficiently. For a max-heap, use a reversed comparer or negate the priority.

HEAP PROBLEMS WITH PRIORITYQUEUE

Program.cs

```
int[] scores = { 72, 95, 61, 88, 79, 99, 54, 91 };

var minHeap = new PriorityQueue<int, int>();              // keep only the k largest: O(n log k)
foreach (int s in scores)
{
    minHeap.Enqueue(s, s);
    if (minHeap.Count > 3) minHeap.Dequeue();             // drop the smallest
}
var top3 = new List<int>();
while (minHeap.Count > 0) top3.Add(minHeap.Dequeue());
Console.WriteLine($"top 3: {string.Join(" ", top3.AsEnumerable().Reverse())}");

var maxHeap = new PriorityQueue<string, int>(Comparer<int>.Create((a, b) => b.CompareTo(a)));   // max-
heap
foreach (var (task, urgency) in new[] { ("email", 2), ("server down", 10), ("lunch", 1), ("deploy", 7) })
    maxHeap.Enqueue(task, urgency);
Console.Write("most urgent first: ");
while (maxHeap.TryDequeue(out var t, out var u)) Console.Write($"{t}({u}) ");
Console.WriteLine();

int[][] sortedLists = { new[] { 1, 4, 9 }, new[] { 2, 3, 10 }, new[] { 5, 6 } };   // merge k sorted
lists
var pq = new PriorityQueue<(int List, int Index), int>();
for (int i = 0; i < sortedLists.Length; i++) pq.Enqueue((i, 0), sortedLists[i][0]);
var merged = new List<int>();
while (pq.TryDequeue(out var pos, out int value))
{
    merged.Add(value);
    if (pos.Index + 1 < sortedLists[pos.List].Length) pq.Enqueue((pos.List, pos.Index + 1),
sortedLists[pos.List][pos.Index + 1]);
}
Console.WriteLine($"merged: {string.Join(" ", merged)}");
```

OUTPUT

```
top 3: 99 95 91
most urgent first: server down(10) deploy(7) email(2) lunch(1)
merged: 1 2 3 4 5 6 9 10
```

24.2.5 Hash tables (how Dictionary works)

A **hash table** stores key-value pairs in an array of **buckets**. To find a key, it computes the key's `GetHashCode()`, maps it to a bucket index (hash modulo the array size), and then compares only the few entries in that bucket with `Equals`. With a good hash function and a low **load factor**, lookup, insertion and removal are O(1) on average; if many keys collide into one bucket, operations degrade toward O(n). When the table fills up, it **resizes** (allocates a larger array, typically a prime size about twice as big, and re-inserts every entry). `Dictionary<K,V>` uses separate chaining implemented with arrays of entries and next-indexes; `HashSet<T>` works the same way without values.

A HASH MAP WITH SEPARATE CHAINING Program.cs

```
var map = new ChainedHashMap<int, string>(capacity: 8);
foreach (var (k, v) in new[] { (10, "ten"), (18, "eighteen"), (3, "three"), (26, "twenty-six"), (7,
"seven") })
    map.Put(k, v);
map.Put(18, "EIGHTEEN");                                    // update an existing key
Console.WriteLine(map.Dump());
Console.WriteLine($"get 26: {map.Get(26)}, contains 11: {map.ContainsKey(11)}, count {map.Count}");

public class ChainedHashMap<TKey, TValue>(int capacity) where TKey : notnull
{
    private readonly List<(TKey Key, TValue Value)>?[] buckets = new List<(TKey, TValue)>?[capacity];
    public int Count { get; private set; }

    private int IndexFor(TKey key) => (key.GetHashCode() & 0x7FFFFFFF) % buckets.Length;   // non-
negative hash -> bucket

    public void Put(TKey key, TValue value)
    {
        var bucket = buckets[IndexFor(key)] ??= new();
        for (int i = 0; i < bucket.Count; i++)
            if (EqualityComparer<TKey>.Default.Equals(bucket[i].Key, key)) { bucket[i] = (key, value);
return; }
        bucket.Add((key, value));                           // collision: chain inside the bucket
        Count++;
    }

    public TValue Get(TKey key)
    {
        foreach (var (k, v) in buckets[IndexFor(key)] ?? []) if (EqualityComparer<TKey>.Default.Equals(k,
key)) return v;
        throw new KeyNotFoundException($"{key}");
    }

    public bool ContainsKey(TKey key) => (buckets[IndexFor(key)] ?? []).Any(e =>
EqualityComparer<TKey>.Default.Equals(e.Key, key));

    public string Dump() => string.Join("\n", buckets.Select((b, i) => $"bucket {i}: {(b is null ? "-" :
string.Join(", ", b.Select(e => $"{e.Key}={e.Value}")))}"));
}
```

OUTPUT

```
bucket 0: -
bucket 1: -
bucket 2: 10=ten, 18=EIGHTEEN, 26=twenty-six
bucket 3: 3=three
bucket 4: -
bucket 5: -
bucket 6: -
bucket 7: 7=seven
get 26: twenty-six, contains 11: False, count 5
```

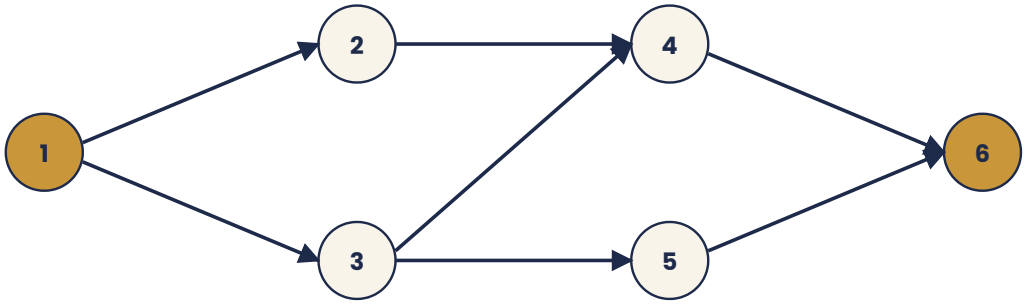
NOTE

For an int, `GetHashCode()` returns the value itself, which makes the bucket indexes easy to follow (10, 18 and 26 all give 2 modulo 8). Real hash codes for strings are randomised per process in .NET to resist hash-flooding attacks, which is one more reason never to persist or depend on them.

24.2.6 Graphs: BFS, DFS, Dijkstra

A **graph** is a set of **vertices** connected by **edges**, directed or undirected, weighted or not: road maps, social networks, course prerequisites, network routes. It is usually stored as an **adjacency list** (`Dictionary<T,`

`List<T>>`). **Breadth-first search** (BFS) explores level by level with a queue and finds shortest paths in unweighted graphs. **Depth-first search** (DFS) goes as deep as possible first (recursion or a stack) and is used for connectivity, cycle detection and topological sorting. **Dijkstra's algorithm** finds shortest paths with non-negative weights using a priority queue, in O((V + E) log V).



THE DIRECTED GRAPH USED FOR BFS AND DFS

BFS, DFS AND AN UNWEIGHTED SHORTEST PATH

Program.cs

```
var adj = new Dictionary<int, List<int>>
{
    [1] = [2, 3], [2] = [4], [3] = [4, 5], [4] = [6], [5] = [6], [6] = []
};
Console.WriteLine($"BFS: {string.Join(" ", Bfs(1))}");
Console.WriteLine($"DFS: {string.Join(" ", Dfs(1, new HashSet<int>()))}");
Console.WriteLine($"shortest path 1 -> 6: {string.Join(" -> ", ShortestPath(1, 6))}");

IEnumerable<int> Bfs(int start)
{
    var seen = new HashSet<int> { start };
    var queue = new Queue<int>([start]);
    while (queue.Count > 0)
    {
        int u = queue.Dequeue();
        yield return u;
        foreach (int v in adj[u]) if (seen.Add(v)) queue.Enqueue(v);
    }
}

List<int> Dfs(int u, HashSet<int> seen)
{
    var order = new List<int>();
    if (!seen.Add(u)) return order;
    order.Add(u);
    foreach (int v in adj[u]) order.AddRange(Dfs(v, seen));
    return order;
}

List<int> ShortestPath(int from, int to)                    // BFS with parent links
{
    var parent = new Dictionary<int, int> { [from] = from };
    var q = new Queue<int>([from]);
    while (q.Count > 0)
    {
        int u = q.Dequeue();
        if (u == to) break;
        foreach (int v in adj[u]) if (parent.TryAdd(v, u)) q.Enqueue(v);
    }
    var path = new List<int> { to };
    while (path[^1] != from) path.Add(parent[path[^1]]);
    path.Reverse();
```

```
    return path;
}
```

OUTPUT

```
BFS: 1 2 3 4 5 6
DFS: 1 2 4 6 3 5
shortest path 1 -> 6: 1 -> 2 -> 4 -> 6
```

DIJKSTRA WITH PRIORITYQUEUE Program.cs

```
var graph = new Dictionary<char, List<(char To, int Km)>>
{
    ['A'] = [('B', 4), ('C', 2)],
    ['B'] = [('C', 1), ('D', 5)],
    ['C'] = [('B', 1), ('D', 8), ('E', 10)],
    ['D'] = [('E', 2), ('F', 6)],
    ['E'] = [('F', 3)],
    ['F'] = []
};

var (dist, prev) = Dijkstra('A');
Console.WriteLine(string.Join("  ", dist.OrderBy(p => p.Key).Select(p => $"{p.Key}={p.Value}")));
var route = new List<char> { 'F' };
while (route[^1] != 'A') route.Add(prev[route[^1]]);
route.Reverse();
Console.WriteLine($"best route A to F: {string.Join(" -> ", route)} ({dist['F']} km)");

(Dictionary<char, int>, Dictionary<char, char>) Dijkstra(char source)
{
    var d = graph.Keys.ToDictionary(k => k, _ => int.MaxValue);
    var p = new Dictionary<char, char>();
    var pq = new PriorityQueue<char, int>();
    d[source] = 0;
    pq.Enqueue(source, 0);
    while (pq.TryDequeue(out char u, out int du))
    {
        if (du > d[u]) continue;                            // stale entry: a shorter path was found
already
        foreach (var (v, w) in graph[u])
            if (du + w < d[v])
            {
                d[v] = du + w;
                p[v] = u;
                pq.Enqueue(v, d[v]);
            }
    }
    return (d, p);
}
```

OUTPUT

```
A=0  B=3  C=2  D=8  E=10  F=13
best route A to F: A -> C -> B -> D -> E -> F (13 km)
```

24.2.7 Sorting: merge sort, quick sort

Merge sort is divide and conquer: split the array in halves, sort each recursively, then merge the two sorted halves. It always takes O(n log n), is **stable**, and needs O(n) extra memory. **Quick sort** picks a **pivot**, partitions the array so that smaller elements come before it and larger after, and recurses on both parts. It sorts in place and is very fast on average (O(n log n)), but degrades to O(n²) with consistently bad pivots, which random or median-of-three pivots avoid. .NET's `Array.Sort` uses **introsort**: quick sort that switches to heap sort if recursion gets too deep and to insertion sort for tiny ranges.

MERGE SORT AND QUICK SORT

Program.cs

```
int[] data = { 38, 27, 43, 3, 9, 82, 10 };
Console.WriteLine($"merge sort: {string.Join(" ", MergeSort(data))}");
int[] q = (int[])data.Clone();
QuickSort(q, 0, q.Length - 1);
Console.WriteLine($"quick sort: {string.Join(" ", q)}");

static int[] MergeSort(int[] a)
{
    if (a.Length <= 1) return a;
    int mid = a.Length / 2;
    int[] left = MergeSort(a[..mid]), right = MergeSort(a[mid..]);
    int[] result = new int[a.Length];
    int i = 0, j = 0, k = 0;
    while (i < left.Length && j < right.Length) result[k++] = left[i] <= right[j] ? left[i++] :
right[j++];   // <= keeps it stable
    while (i < left.Length) result[k++] = left[i++];
    while (j < right.Length) result[k++] = right[j++];
    return result;
}

static void QuickSort(int[] a, int lo, int hi)
{
    if (lo >= hi) return;
    int pivot = a[hi], p = lo;                                  // Lomuto partition, last element as pivot
    for (int i = lo; i < hi; i++)
        if (a[i] < pivot) { (a[i], a[p]) = (a[p], a[i]); p++; }   // move smaller elements to the left
part
    (a[p], a[hi]) = (a[hi], a[p]);
    QuickSort(a, lo, p - 1);
    QuickSort(a, p + 1, hi);
}
```

OUTPUT

```
merge sort: 3 9 10 27 38 43 82
quick sort: 3 9 10 27 38 43 82
```

Algorithm	Best	Average	Worst	Extra memory	Stable
Bubble / insertion	O(n)	O(n²)	O(n²)	O(1)	yes
Selection	O(n²)	O(n²)	O(n²)	O(1)	no
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)	yes
Quick sort	O(n log n)	O(n log n)	O(n²)	O(log n)	no
Heap sort	O(n log n)	O(n log n)	O(n log n)	O(1)	no
Array.Sort (introsort)	O(n log n)	O(n log n)	O(n log n)	O(log n)	no
LINQ OrderBy	O(n log n)	O(n log n)	O(n log n)	O(n)	yes

24.2.8 Two pointers, sliding window, recursion, backtracking, dynamic programming

Most coding-round problems are variations on a handful of **techniques**. Recognising the pattern is half the solution.

Technique	Idea	Typical problems
Two pointers	two indexes moving towards each other or at different speeds over a sorted array or list	pair with target sum, remove duplicates, palindrome, linked-list cycle

Technique	Idea	Typical problems
Sliding window	maintain a window [left, right] and update its summary as it slides instead of recomputing	max sum of k elements, longest substring without repeats, minimum window
Hashing	remember what has been seen in a HashSet/Dictionary for O(1) checks	two sum, anagrams, first unique, subarray sum = k
Recursion / divide and conquer	solve smaller instances and combine	tree problems, merge sort, power, binary search
Backtracking	build candidates step by step and undo choices that cannot lead to a solution	subsets, permutations, N-Queens, Sudoku, word search
Dynamic programming	store answers to overlapping subproblems (memoization or a table)	coin change, LCS, knapsack, climbing stairs, edit distance
Greedy	take the locally best choice when it provably leads to the global best	interval scheduling, activity selection, Huffman coding

TWO POINTERS AND SLIDING WINDOW

Program.cs

```
int[] sorted = { 1, 2, 4, 7, 11, 15 };
var (i, j) = PairWithSum(sorted, 15);
Console.WriteLine($"two pointers: {sorted[i]} + {sorted[j]} = 15 (indexes {i}, {j})");

int[] a = { 2, 1, 5, 1, 3, 2 };
Console.WriteLine($"sliding window: max sum of 3 consecutive = {MaxWindowSum(a, 3)}");
Console.WriteLine($"longest substring without repeats: {LongestUnique("abcabcbb")},
{LongestUnique("pwwkew")}");

static (int, int) PairWithSum(int[] s, int target)
{
    int lo = 0, hi = s.Length - 1;
    while (lo < hi)
    {
        int sum = s[lo] + s[hi];
        if (sum == target) return (lo, hi);
        if (sum < target) lo++; else hi--;                    // move the pointer that can fix the sum
    }
    return (-1, -1);
}

static int MaxWindowSum(int[] a, int k)
{
    int window = a[..k].Sum(), best = window;
    for (int r = k; r < a.Length; r++)
    {
        window += a[r] - a[r - k];                            // slide: add the new element, drop the
oldest
        best = Math.Max(best, window);
    }
    return best;
}

static string LongestUnique(string s)
{
    var lastSeen = new Dictionary<char, int>();
    int start = 0, bestLen = 0, bestStart = 0;
    for (int r = 0; r < s.Length; r++)
    {
        if (lastSeen.TryGetValue(s[r], out int prev) && prev >= start) start = prev + 1;   // shrink past
the repeat
```

```
        lastSeen[s[r]] = r;
        if (r - start + 1 > bestLen) { bestLen = r - start + 1; bestStart = start; }
    }
    return $"\"{s.Substring(bestStart, bestLen)}\" ({bestLen})";
}
```

OUTPUT

```
two pointers: 4 + 11 = 15 (indexes 2, 4)
sliding window: max sum of 3 consecutive = 9
longest substring without repeats: "abc" (3), "wke" (3)
```

BACKTRACKING AND DYNAMIC PROGRAMMING

Program.cs

```
var subsets = new List<string>();
Subsets([1, 2, 3], 0, new List<int>(), subsets);
Console.WriteLine($"backtracking, subsets: {string.Join(" ", subsets)}");
Console.WriteLine($"N-Queens solutions: 4x4 = {NQueens(4)}, 8x8 = {NQueens(8)}");

Console.WriteLine($"DP coin change (1,5,10,25) for 63: {CoinChange([1, 5, 10, 25], 63)} coins");
Console.WriteLine($"DP longest common subsequence of ABCBDAB, BDCABA: {Lcs("ABCBDAB", "BDCABA")}");
Console.WriteLine($"DP 0/1 knapsack (w 1,3,4,5 / v 1,4,5,7, capacity 7): {Knapsack([1, 3, 4, 5], [1, 4,
5, 7], 7)}");
Console.WriteLine($"DP climbing 10 stairs taking 1 or 2 steps: {ClimbStairs(10)} ways");

static void Subsets(int[] items, int start, List<int> current, List<string> result)
{
    result.Add("{" + string.Join(",", current) + "}");
    for (int i = start; i < items.Length; i++)
    {
        current.Add(items[i]);                                  // choose
        Subsets(items, i + 1, current, result);                 // explore
        current.RemoveAt(current.Count - 1);                    // un-choose (backtrack)
    }
}

static int NQueens(int n)
{
    var cols = new bool[n]; var d1 = new bool[2 * n]; var d2 = new bool[2 * n];
    int count = 0;
    void Place(int row)
    {
        if (row == n) { count++; return; }
        for (int c = 0; c < n; c++)
        {
            if (cols[c] || d1[row - c + n] || d2[row + c]) continue;   // attacked: prune
            cols[c] = d1[row - c + n] = d2[row + c] = true;
            Place(row + 1);
            cols[c] = d1[row - c + n] = d2[row + c] = false;
        }
    }
    Place(0);
    return count;
}

static int CoinChange(int[] coins, int amount)
{
    int[] dp = new int[amount + 1];                             // dp[a] = fewest coins for amount a
    Array.Fill(dp, int.MaxValue); dp[0] = 0;
    for (int a = 1; a <= amount; a++)
        foreach (int c in coins)
            if (c <= a && dp[a - c] != int.MaxValue) dp[a] = Math.Min(dp[a], dp[a - c] + 1);
    return dp[amount] == int.MaxValue ? -1 : dp[amount];
}

static int Lcs(string x, string y)
```

```
{
    var dp = new int[x.Length + 1, y.Length + 1];
    for (int i = 1; i <= x.Length; i++)
        for (int j = 1; j <= y.Length; j++)
            dp[i, j] = x[i - 1] == y[j - 1] ? dp[i - 1, j - 1] + 1 : Math.Max(dp[i - 1, j], dp[i, j -
1]);
    return dp[x.Length, y.Length];
}

static int Knapsack(int[] w, int[] v, int capacity)
{
    int[] dp = new int[capacity + 1];
    for (int i = 0; i < w.Length; i++)
        for (int c = capacity; c >= w[i]; c--)                  // backwards: each item used at most once
            dp[c] = Math.Max(dp[c], dp[c - w[i]] + v[i]);
    return dp[capacity];
}

static int ClimbStairs(int n) { int a = 1, b = 1; for (int i = 2; i <= n; i++) (a, b) = (b, a + b);
return b; }
```

OUTPUT

```
backtracking, subsets: {} {1} {1,2} {1,2,3} {1,3} {2} {2,3} {3}
N-Queens solutions: 4x4 = 2, 8x8 = 92
DP coin change (1,5,10,25) for 63: 6 coins
DP longest common subsequence of ABCBDAB, BDCABA: 4
DP 0/1 knapsack (w 1,3,4,5 / v 1,4,5,7, capacity 7): 9
DP climbing 10 stairs taking 1 or 2 steps: 89 ways
```

24.3 Capstone projects

A capstone project combines everything from the course into one realistic application. It becomes the centrepiece of a portfolio and the main topic of placement interviews, so build it properly: version control from day one, a clean structure, tests, a README with screenshots and setup steps, and a live deployment where possible. Choose one project and take it all the way rather than starting several.

24.3.1 Library or Bank Management System (OOP + EF Core)

Aspect	Library Management	Bank Management
Core entities	Book, Author, Member, Loan, Fine	Customer, Account (Savings/Current), Transaction, Beneficiary
OOP focus	inheritance for member types, interfaces for fine policies (Strategy)	abstract Account with overridden interest/withdraw rules, events for low balance
Data	EF Core + SQLite/SQL Server, migrations, one-to-many and many-to-many	EF Core, transactions for transfers, concurrency tokens
Features	search with LINQ, borrow/return, overdue fines, reports to CSV/JSON	deposit, withdraw, transfer, statements, interest calculation
Stretch	Blazor or WPF front end, email reminders via a background service	PDF statements, audit log, role-based admin UI

24.3.2 E-commerce or Student Portal REST API (ASP.NET Core + JWT)

A REST API with ASP.NET Core and EF Core: authentication with JWT (or Identity), role-based authorization (Admin, Teacher/Seller, Student/Customer), CRUD with DTOs and validation, paging, filtering and sorting, global error handling with ProblemDetails, OpenAPI docs, unit tests of the service layer and integration tests

with WebApplicationFactory, structured logging, caching of hot queries, Docker packaging and CI with GitHub Actions. An e-commerce API adds a product catalogue, cart, orders and a payment stub; a student portal adds courses, enrolment, attendance, marks and timetable.

24.3.3 Blazor web app with a database

A full-stack C# application with the Blazor Web App template: Razor components with interactive server or WebAssembly rendering, forms with validation (EditForm, DataAnnotations), EF Core, authentication with Identity, and a component library such as MudBlazor or Fluent UI Blazor. Ideas: an expense tracker with charts, a hostel or event booking system, a quiz and exam platform.

24.3.4 Real-time chat with SignalR

A chat application with an ASP.NET Core SignalR hub: rooms (groups), private messages, typing indicators, online presence, message history in a database, authenticated connections with JWT, and a Blazor or JavaScript client. It demonstrates real-time communication, concurrency and connection management, and can be extended with file sharing or an AI assistant using Microsoft.Extensions.AI.

24.3.5 A 2D game in Unity

A small but complete 2D game in Unity with C# scripts: player movement and physics, collectables and scoring, enemies with simple AI, levels, a menu and pause screen, sound, and saving high scores to a JSON file. Publish a WebGL build on itch.io so that interviewers can play it. It shows object-oriented design, event-driven code and performance awareness (object pooling, avoiding per-frame allocations).

Portfolio checklist	Done?
Public GitHub repository with meaningful commit history and branches	
README: purpose, screenshots/GIF, tech stack, architecture diagram, how to run, test accounts	
Layered or clean solution structure, nullable enabled, no warnings	
Unit tests and at least a few integration tests running in CI	
Configuration and secrets outside source control	
Deployed demo (Azure App Service, a container host, itch.io)	
A list of challenges faced and how you solved them, ready for interviews	

24.4 Interview preparation

24.4.1 Core C# topics checklist: value vs reference types, boxing, OOP, interfaces, generics, delegates, LINQ, async/await, GC, IDisposable

Topic	Be ready to explain	Module
Value vs reference types	stack vs heap storage, copy semantics, struct vs class, string immutability	3, 12, 20
Boxing / unboxing	what allocates, why generics avoid it	3, 14
OOP pillars	encapsulation, inheritance, polymorphism (virtual/override vs new), abstraction	10, 11
Abstract class vs interface	state, multiple implementation, default interface methods	11

Topic	Be ready to explain	Module
Generics	constraints, variance (in/out), default(T)	14
Collections	List vs array, Dictionary internals, HashSet, complexity	14, 24
Delegates, events, lambdas	Func/Action, closures, event vs delegate field, leaks	15
LINQ	deferred vs immediate execution, IEnumerable vs IQueryable, multiple enumeration	16
Exceptions	throw vs throw ex, finally, custom exceptions, Try pattern	13
async/await	state machine, Task vs ValueTask, async void, deadlocks, ConfigureAwait, cancellation	18
Threading	race conditions, lock, Interlocked, deadlock, concurrent collections	18
GC and IDisposable	generations, LOH, finalizers vs Dispose, using, leaks	20
Records and patterns	value equality, with, switch expressions, list patterns	12, 19
Modern C#	nullable reference types, primary constructors, collection expressions, C# 14 features	12, 19

24.4.2 .NET and ASP.NET Core topics: DI lifetimes, middleware, EF Core tracking

Question	Key points of a good answer
Transient vs scoped vs singleton?	per resolution / per request / per app; DbContext is scoped; captive dependencies
What is middleware and why does order matter?	pipeline of request delegates; each can short-circuit; exception handler first, auth before endpoints
Minimal APIs vs controllers?	less ceremony and AOT support vs structure, filters, conventions
How is configuration loaded?	appsettings, environment file, user secrets, env vars, command line; Options pattern
How does JWT authentication work?	signed token with claims, validated per request, 401 vs 403
What does AsNoTracking do?	skips change-tracker snapshots for read-only queries; faster, less memory
What is the N+1 problem?	one query per related row; fix with Include or projections
Migrations vs EnsureCreated?	versioned incremental schema changes vs one-off creation for tests
How do you handle errors globally?	UseExceptionHandler, IExceptionHandler, ProblemDetails, logging with traceId
How do you make an API scale?	async I/O, caching, pagination, statelessness, horizontal scaling, background processing

24.4.3 Output-prediction and tricky questions

Interviewers like short snippets whose output reveals whether you understand the language rules. Predict each line before reading the answer.

FOURTEEN CLASSIC TRICKY SNIPPETS

Program.cs

```
string s = "C#"; string t = s; s += "!";
Console.WriteLine($"Q1: {t}");                                          // strings are immutable
Console.WriteLine("Q2: " + (1 + 2 + "3" + 4 + 5));                      // left to right
Console.WriteLine($"Q3: {0.1 + 0.2 == 0.3} {0.1m + 0.2m == 0.3m}");     // binary vs decimal
object a = 5, b = 5;
Console.WriteLine($"Q4: {a == b} {a.Equals(b)}");                       // two boxes
int x = int.MaxValue; x++;
Console.WriteLine($"Q5: {x}");                                          // unchecked overflow
var actions = new List<Action>();
for (int i = 0; i < 3; i++) actions.Add(() => Console.Write(i));
Console.Write("Q6: "); actions.ForEach(f => f()); Console.WriteLine();  // one shared i
Console.WriteLine($"Q7: {10 / 4 * 4.0}");                               // integer division first
Console.WriteLine($"Q8: {Math.Round(2.5)} {Math.Round(3.5)} {Math.Round(2.5,
MidpointRounding.AwayFromZero)}");
Console.WriteLine($"Q9: {F()}");                                        // return value fixed before
finally
string? n = null;
Console.WriteLine("Q10: [" + n + "]");                                  // null concatenates as empty
char c = 'A'; c++;
Console.WriteLine($"Q11: {c + 1}");                                     // char + int is int
int? m = null;
Console.WriteLine($"Q12: {m > 0 || m <= 0}");                           // lifted comparison with null
var nums = new List<int> { 1, 2, 3 };
var big = nums.Where(v => v > 1);
nums.Add(4);
Console.WriteLine($"Q13: {big.Count()}");                               // deferred execution
Base bd = new Derived();
Console.WriteLine($"Q14: {bd.Hello()} {bd.Virt()}");                    // new hides, override replaces

static int F() { int v = 1; try { return v; } finally { v = 2; } }

class Base { public string Hello() => "Base"; public virtual string Virt() => "Base"; }
class Derived : Base { public new string Hello() => "Derived"; public override string Virt() =>
"Derived"; }
```

OUTPUT

```
Q1: C#
Q2: 3345
Q3: False True
Q4: False True
Q5: -2147483648
Q6: 333
Q7: 8
Q8: 2 4 3
Q9: 1
Q10: []
Q11: 67
Q12: False
Q13: 3
Q14: Base Derived
```

Q	Why
1	s += "!" creates a new string; t still refers to "C#"
2	1 + 2 = 3, then "3" + "3" = "33", then "334", then "3345"
3	double cannot represent 0.1 exactly; decimal can
4	== on object compares references of two different boxes; Equals compares values
5	int arithmetic wraps around silently unless in a checked context

Q	Why
6	all lambdas capture the same loop variable i, which is 3 after the loop
7	10 / 4 is integer division (2), then 2 * 4.0 = 8
8	Math.Round uses banker's rounding (to even) by default
9	the return value is evaluated before finally runs; changing v later does not affect it
10	string concatenation treats null as an empty string
11	c becomes 'B' (66); 'B' + 1 is the int 67
12	any comparison with a null int? is false (except != null)
13	the query runs when Count() is called, after 4 was added: 2, 3, 4
14	non-virtual Hello is chosen by the static type Base; virtual Virt by the object (Derived)

24.4.4 DSA coding rounds in C#

In a coding round: clarify the problem and constraints, talk through a brute-force idea and its complexity, improve it (usually with hashing, sorting, two pointers or a heap), write clean code, then test it with normal, edge and empty inputs. Know the C# tools that keep solutions short:

Need	C# tool
Hash lookups / counting	`Dictionary`, `HashSet`, `CollectionsMarshal.GetValueRefOrAddDefault`, `CountBy`
Sorting with custom order	`Array.Sort(a, (x, y) => ...)`, `OrderBy().ThenBy()`
Min/max element by key	`MinBy`, `MaxBy`
Heap / k-th largest / Dijkstra	`PriorityQueue<TElement, TPriority>`
Ordered set with floor/ceiling	`SortedSet<T>.GetViewBetween`, `Min`, `Max`
Stack / queue / deque	`Stack<T>`, `Queue<T>`, `LinkedList<T>`
Binary search	`Array.BinarySearch`, `List.BinarySearch` (negative = ~insertion point)
Big numbers	`long`, `checked`, `System.Numerics.BigInteger`
Strings	`StringBuilder`, `char.IsLetterOrDigit`, spans, `string.Join`
Multiple return values	tuples `(int, int)`

24.4.5 Explaining your projects

Every interviewer will ask about your projects, and a clear explanation often matters more than the project's size. Prepare a two-minute story using the **STAR** structure (Situation, Task, Action, Result), and be ready to go deep on any part.

- **What** it does and for whom, in one sentence; then the main features.
- **Architecture**: draw the layers or components (UI, API, services, database) and explain why you chose that structure.
- **Technology choices**: why ASP.NET Core minimal APIs, why EF Core, why SQLite or SQL Server, why JWT.
- **Your contribution** in a team project, honestly.

- **The hardest problem** you solved (a concurrency bug, a slow query fixed with AsNoTracking and projections, a deployment issue) and how you found the cause.
- **Testing and quality**: what is tested, how CI works.
- **What you would improve** with more time: caching, more tests, better UI, scaling.
- Have the repository and the live demo link ready to share.

24.5 Career paths with C#

Role	Typical stack	What to learn beyond this course
.NET backend developer	ASP.NET Core Web APIs, EF Core, SQL Server/PostgreSQL, Redis, message queues	SQL tuning, microservices, Docker/Kubernetes, messaging (RabbitMQ, Azure Service Bus), observability
Full-stack developer	ASP.NET Core + Angular, React or Blazor	TypeScript and a front-end framework, HTML/CSS, state management, UX basics
Unity game developer	Unity, C#, shaders, physics	game design patterns, linear algebra, profiling, platform publishing
Desktop and enterprise application developer	WPF, WinUI, WinForms, MAUI, Office/SharePoint integration	MVVM in depth, installers and updates, legacy .NET Framework migration
Cloud developer on Azure	Azure Functions, App Service, Container Apps, Cosmos DB, .NET Aspire	Azure certifications (AZ-204), infrastructure as code (Bicep, Terraform), DevOps, security

Whichever path you choose, the foundations of this course (the language, OOP, collections, LINQ, async, testing and clean design) carry over directly. Keep building projects, contribute to open source, follow the yearly .NET releases, and consider Microsoft certifications to validate your skills.

C# reaches desktop (WPF, WinForms, WinUI), mobile (.NET MAUI), games (Unity), cloud-native systems (.NET Aspire, Azure) and AI (ML.NET, Microsoft.Extensions.AI). For coding rounds, analyse complexity with Big-O, know how linked lists, stacks, queues, BSTs, heaps and hash tables work, use BFS, DFS and Dijkstra on graphs, merge and quick sort, and the techniques of two pointers, sliding windows, backtracking and dynamic programming, using C#'s collections to keep solutions short. Build one complete capstone project with tests, CI and a deployment, prepare core C#, .NET and ASP.NET Core interview topics and tricky snippets, practise explaining your project, and pick a career path to deepen.

PRACTICE CHECKPOINT

Build **one capstone project** with tests and a GitHub repository, **deploy** it, and solve **50 DSA problems** in C#. The program below shows the style for the first five problems of the suggested list.

SAMPLE SOLUTION Program.cs

```
Console.WriteLine($"1. Two Sum [2,7,11,15], 9 -> [{string.Join(",", TwoSum([2, 7, 11, 15], 9))}]");
Console.WriteLine($"2. Valid Parentheses \"({{[]}})\" -> {IsValid("({[]})")}, \"(]\" -> {IsValid("
(]")}");
Console.WriteLine($"3. Best Time to Buy/Sell [7,1,5,3,6,4] -> {MaxProfit([7, 1, 5, 3, 6, 4])}");
Console.WriteLine($"4. Maximum Subarray [-2,1,-3,4,-1,2,1,-5,4] -> {MaxSubArray([-2, 1, -3, 4, -1, 2, 1,
-5, 4])}");
Console.WriteLine($"5. Group Anagrams -> {string.Join(" ", GroupAnagrams(["eat", "tea", "tan", "ate",
"nat", "bat"]).Select(g => "[" + string.Join(",", g) + "]"))}");

static int[] TwoSum(int[] nums, int target)                    // O(n) with a hash map
```

```
{
    var seen = new Dictionary<int, int>();
    for (int i = 0; i < nums.Length; i++)
    {
        if (seen.TryGetValue(target - nums[i], out int j)) return [j, i];
        seen[nums[i]] = i;
    }
    return [];
}

static bool IsValid(string s)                                   // O(n) with a stack
{
    var stack = new Stack<char>();
    foreach (char c in s)
    {
        if (c is '(' or '[' or '{') { stack.Push(c); continue; }
        char expected = c switch { ')' => '(', ']' => '[', '}' => '{', _ => '?' };
        if (stack.Count == 0 || stack.Pop() != expected) return false;
    }
    return stack.Count == 0;
}

static int MaxProfit(int[] prices)                              // O(n) single pass, track the minimum
{
    int min = int.MaxValue, best = 0;
    foreach (int p in prices) { min = Math.Min(min, p); best = Math.Max(best, p - min); }
    return best;
}

static int MaxSubArray(int[] a)                                 // Kadane's algorithm, O(n)
{
    int current = a[0], best = a[0];
    for (int i = 1; i < a.Length; i++) { current = Math.Max(a[i], current + a[i]); best = Math.Max(best,
current); }
    return best;
}

static List<List<string>> GroupAnagrams(string[] words) =>      // sorted letters as the key
    words.GroupBy(w => new string(w.Order().ToArray())).Select(g => g.ToList()).ToList();
```

```
OUTPUT
1. Two Sum [2,7,11,15], 9 -> [0,1]
2. Valid Parentheses "({[]})" -> True, "(]" -> False
3. Best Time to Buy/Sell [7,1,5,3,6,4] -> 5
4. Maximum Subarray [-2,1,-3,4,-1,2,1,-5,4] -> 6
5. Group Anagrams -> [eat,tea,ate] [tan,nat] [bat]
```

Topic	Suggested problems (50 in total)
Arrays and hashing (8)	Two Sum, Contains Duplicate, Valid Anagram, Group Anagrams, Top K Frequent Elements, Product of Array Except Self, Longest Consecutive Sequence, Valid Sudoku
Two pointers (5)	Valid Palindrome, Two Sum II (sorted), 3Sum, Container With Most Water, Trapping Rain Water
Sliding window (4)	Best Time to Buy and Sell Stock, Longest Substring Without Repeating Characters, Longest Repeating Character Replacement, Minimum Window Substring
Stack (4)	Valid Parentheses, Min Stack, Evaluate Reverse Polish Notation, Daily Temperatures
Binary search (4)	Binary Search, Search a 2D Matrix, Find Minimum in Rotated Sorted Array, Search in Rotated Sorted Array
Linked list (5)	Reverse Linked List, Merge Two Sorted Lists, Linked List Cycle, Remove Nth Node From End, Reorder List

Topic	Suggested problems (50 in total)
Trees (7)	Invert Binary Tree, Maximum Depth, Same Tree, Level Order Traversal, Validate BST, Lowest Common Ancestor of a BST, Kth Smallest Element in a BST
Heap (3)	Kth Largest Element in an Array, Last Stone Weight, K Closest Points to Origin
Graphs (4)	Number of Islands, Clone Graph, Course Schedule (topological sort), Network Delay Time (Dijkstra)
Backtracking (3)	Subsets, Permutations, Combination Sum
Dynamic programming (3)	Climbing Stairs, Coin Change, Longest Common Subsequence